The Systems Engineer's Path

A Five-Volume Book — from senior engineer to systems architect: distributed systems, database internals, and platform design from first principles

A training book · with worked examples, build-alongs, exercises, and full solutions

Preface

This is the second volume. The first, *The Data Engineer's Path*, took a computer-science graduate and trained them into a **senior data engineer** — someone who can hold a company's data platform and reason about the systems they operate. This volume takes that engineer the next step: to a **data architect and staff-level systems engineer** who understands those systems *from the inside*, can *build* them, not just use them, and can *design platforms* and defend the trade-offs at scale. Where Volume 1 taught "the shuffle is expensive," A Five-Volume Book shows you the exchange-operator machinery underneath it, has you build a query engine that shuffles, and teaches you to architect the platform it runs on.

It assumes Volume 1 (or an equivalent senior data engineer) and cross-references it generously, so it is usable on its own. It is denser and goes deeper — but it is written for someone senior, so it respects your time: no re-teaching, heavy signposting, and every abstraction motivated by the problem that forced it into existence.

Four threads run through the whole volume.

- **Think from first principles.** Derive a system's behavior from its underlying model — the failure model, the storage model, the consistency model — not from a tool's documentation. You will see a *first-principles* callout that traces each mechanism to the constraint that produced it.
- **Reason about cost, complexity, and scale.** Now with formal tools — amplification and the RUM conjecture, queueing theory and Little's law, the Universal Scalability Law — and back-of-the-envelope estimation at architect scale. A *cost & scale* callout runs throughout.
- **Design under trade-offs.** Every architecture is a deliberate set of trade-offs against constraints and non-functional requirements; naming and defending them *is* the job. A *design trade-off* callout makes each one explicit, and recurring "design review" exercises have you critique and defend real architectures.
- **Communicate the design.** Architecture decision records, diagrams, and the ability to make a skeptical room trust a design — the third thread of Volume 1, elevated. (The leadership and technical-strategy half of the architect role is the subject of Volume 5; this volume stays technical.)

Correctness is non-negotiable, as in Volume 1. Every citation names a real paper, specification, book with free-legal access, or documented system, with the correct author and venue; numbers are dated because hardware and pricing age; fast-moving claims carry their date; and where the field is genuinely unsettled the text says so. The sources are the canon of systems: the distributed-systems and database papers, *Designing Data-Intensive Applications*, Petrov's *Database Internals*, the CMU and MIT graduate courses, and the official specs.

How to use this book. Read it front to back and do the work. Each chapter motivates its topic, develops it with worked examples and traces, and closes with a *key ideas* box, *exercises* with full solutions, and pointers into the primary literature. Several parts end with a **build-along** — a mini Raft module, an LSM store, a query engine, a Kubernetes operator — because at this level, reading is not enough: you understand a system when you have built a small one. The build-alongs are in Python for approachability, with pointers to how production engines differ in Rust and C++.

Contents

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART I

The Machine & the Cost Model

What a program actually runs on: CPU, memory hierarchy, caches, and mechanical sympathy — the real cost model that the RAM abstraction hides.

Chapter 1 — The Real Machine vs. the RAM Model

Before you start: nothing — this is the beginning of Volume 1. · **Pairs with:** Chapter 2 (the CPU up close), Chapter 3 (the memory hierarchy and caches). Together these three install the cost model the rest of the volume optimizes against.

BEFORE YOU READ ON — PREDICT FIRST

No prior chapter to recall, so instead make three *guesses* and write them down; the point is to have a number in your head to be surprised by. (1) A modern CPU core can do roughly how many simple operations — additions, comparisons — per second: about a million, a billion, or a trillion? (2) Reading a value that happens to sit in the CPU's fastest cache versus reading one that must come from main memory (DRAM): same speed, about 10× apart, or about 100× apart? (3) When you write `x = a + b` in C, is "fetch `a` from memory" the same cost as "add"? Answers resolve over the course of this chapter.

Every algorithms course you have ever taken quietly lied to you, and the lie was a good one. It said: assume the machine can read any piece of memory, do one arithmetic operation, and write the result, each in one fixed unit of time, the same unit regardless of *which* piece of memory or *which* operation. On that assumption you learned to count operations, and counting operations told you whether an algorithm scaled. That assumption is the **RAM model**, and it is the foundation of asymptotic analysis. It is also false about the machine on your desk in ways that span six orders of magnitude — and the whole of performance engineering, the whole reason this volume exists, lives in the gap between the model and the metal.

This chapter is about that gap. We will state precisely what the RAM model assumes, see exactly where a real processor and a real memory system diverge from it, and meet the single most important divergence — the **memory wall**, the enormous and growing distance between how fast a CPU can compute and how long it takes to fetch a byte from DRAM. The goal is not to make you distrust big-O; big-O is right about what it claims. The goal is to make you fluent in the second cost model that big-O deliberately hides — the one denominated in *where the data is*, not *how many operations you do* — because at the level this book operates, that second model is where the performance is won and lost.

1.1 What the RAM model assumes

The **Random Access Machine** (RAM) model is the idealized computer that theoretical analysis runs on. It has a processor that executes instructions one at a time, in program order, and a memory of cells that the processor can read or write by address. Its defining assumption is **uniform cost**: every basic operation — an arithmetic op, a comparison, a load from memory, a store to memory — takes the same constant amount of time, usually idealized as "one unit," and crucially that unit does not depend on which address you touch. Reading cell 5 costs the same as reading cell 5,000,000,000. There is no notion of "nearby" or "far away" memory; access is, as the name says, uniformly *random-access*.

This is a spectacularly useful abstraction, and you should not give it up. Because every operation costs one unit, the running time of an algorithm is just the *number of operations* it performs, which you can count by reading the code. That is what lets you say a linear scan is $O(n)$, a comparison sort is $O(n \log n)$, and a nested-loop join is $O(n^2)$, and to know — before running anything — that the last one will eventually outrun any machine you can buy. The RAM model throws away every detail of real hardware precisely so that *growth rate* survives, and growth rate is the thing that no amount of faster hardware can fix. Nothing in this chapter contradicts that. An $O(n^2)$ algorithm is a disaster on any real machine too.

What the model buys with that simplification is a blind spot, and the blind spot is the subject of this volume. By declaring every operation equal, the RAM model asserts three things that are simply untrue of the computer in front of you:

- **All memory accesses cost the same.** They do not. On real hardware the cost of a load depends enormously on *where the data currently sits* — in a register, in a small fast cache, in a larger slower cache, in DRAM, on an SSD, or across a network — and those costs differ by factors of ten to factors of ten million.
- **One instruction takes one unit of time.** It does not. A real CPU overlaps instructions, executes them out of order, guesses the outcome of branches, and can retire several instructions in a single clock tick or stall for hundreds of ticks on a single one — the subject of Chapter 2.
- **Operations are independent and equal.** They are not. A multiply may cost more than an add; a division much more; and an instruction that depends on the result of the previous one cannot start until that result is ready, so *dependencies* between operations, not just their count, govern speed.

Each of these is a place where the abstraction leaks. This chapter takes the first one — the uniform-cost-memory assumption — because it is the largest leak by far, and because understanding it reorganizes how you think about all fast code. Chapter 2 takes the second (the CPU), and Chapter 3 takes the mechanism, caches, that turns the first assumption from "false" into "false but predictable, and therefore exploitable."

> **QUICK CHECK**
>
> State the RAM model's uniform-cost assumption in one sentence, and name the real-hardware fact that most directly contradicts it. (Answer below the next section — try it from memory first.)

1.2 The memory wall

Here is the historical fact that makes the uniform-cost assumption not just wrong but *catastrophically* wrong on a modern machine. For several decades, the speed at which a CPU can execute instructions improved much faster, year over year, than the speed at which main memory (DRAM) can deliver a requested byte. Processor throughput raced ahead; DRAM *latency* — the time from "I want the byte at this address" to "here is the byte" — improved only slowly. The two curves diverged, and the widening gap between them was named the **memory wall** by Wulf and McKee in 1995, whose short paper

"Hitting the Memory Wall" pointed out the then-uncomfortable consequence: as the gap grows, the average time to touch memory comes to dominate program performance, no matter how fast the processor's arithmetic is (Wulf & McKee, *ACM SIGARCH Computer Architecture News*, 1995).

The consequence for us is stark and it is worth stating as baldly as possible. On a modern processor, the time to fetch a single value from DRAM is on the order of a *hundred nanoseconds* — and in that same hundred nanoseconds, a single core can execute on the order of hundreds of instructions. So a load that misses every cache and goes to main memory is not "one unit" of work in any meaningful sense; measured against the arithmetic the core could otherwise be doing, it is hundreds of units. A program that spends its time chasing pointers through DRAM can leave the processor's arithmetic units idle more than 90% of the time, waiting. The CPU is not slow. It is *waiting for memory.* That single sentence explains an enormous fraction of the real-world performance problems this volume teaches you to see.

The hardware's answer to the memory wall is not faster DRAM — DRAM latency is hard physics and improves slowly. The answer is the **memory hierarchy**: small, fast, expensive memories (caches) placed close to the processor, holding copies of the data the program is most likely to touch next, backed by progressively larger, slower, cheaper memories behind them. The hierarchy does not repeal the memory wall; it *hides* it, but only for programs whose access patterns let the caches do their job. Learning to write those programs — to earn the cache's help rather than fight it — is a large part of what "systems performance" means, and it is the whole subject of Chapter 3. For now the load-bearing idea is only this: *the cost of an operation is dominated by where its data lives, and the gap between "close" and "far" is the central fact of modern performance.*

(Answer to the quick check: the RAM model assumes every basic operation, including any memory access, costs one fixed unit of time independent of the address. The fact that most directly contradicts it is the memory hierarchy — accessing data in a nearby cache is one to two orders of magnitude faster than reaching main memory, and further tiers are slower still.)

1.3 Latency, as orders of magnitude only

To reason about the real cost model you need anchors: a rough sense of how far apart the tiers of the hierarchy are. But here we must be careful, because this is exactly the kind of place where a confident, precise-looking number is worse than useless. **The absolute latencies below are approximate, they are machine-dependent, and they drift as hardware changes.** What is stable — what you should actually memorize — is the *shape*: the ordering of the tiers and the rough number of orders of magnitude between them. Treat every specific figure as "as of the mid-2020s, on a typical server-class x86-64 core, give or take a large factor."

With that caveat stated as loudly as it can be, here is the ordering. It is the single most important picture in this volume.

Where the data is	Rough access latency (order of magnitude)	Relative to L1
CPU register	< 1 ns (available essentially at once, within the pipeline)	—
L1 cache	~1 ns (a few clock cycles)	1×
L2 cache	a few ns (~10 cycles)	~10×
L3 cache	~10 ns (tens of cycles)	~10–50×
Main memory (DRAM)	~100 ns (hundreds of cycles)	~100×
Local SSD (NVMe)	tens of µs	~10^4–10^5×
Same-datacenter network round trip	hundreds of µs to ~1 ms	~10^5–10^6×

The register-through-DRAM rows follow the memory-hierarchy access times given for a specific Intel Core i7 in Bryant and O'Hallaron's *Computer Systems: A Programmer's Perspective* (CS:APP, Chapter 6), which are themselves presented there as approximate and processor-specific; the SSD and network rows are conventional community orders of magnitude of the kind popularized by the widely circulated "latency numbers every programmer should know" list (attributed to Jeff Dean, and later made time-adjustable by Colin Scott). **Use them as ratios, never as guarantees.** If your program's performance depends on whether L2 is 8 cycles or 14 on your particular chip, you are measuring, not reading a book — and measuring is exactly what Chapter 3 and the performance-engineering chapters teach.

Read the table for its shape and three facts fall out. First, the jumps are *multiplicative*, not additive: each tier is roughly 10× to 100× slower than the one above it, so the distance from register to network round trip is not "a lot" but a factor of around a million. Second, there is a cliff between on-chip (register, L1/L2/L3 — nanoseconds) and off-chip (DRAM and below — DRAM at ~100 ns, then a huge further step to SSD at microseconds), and the memory wall is the height of that cliff. Third — and this is the one that reorganizes how you write code — the difference between a cache hit and a cache miss to DRAM is the same ~100× that the difference between a good and a bad algorithm often is. You can lose two orders of magnitude of performance without changing your algorithm's big-O at all, purely by where your data lands.

THE SAME IDEA THREE WAYS: RAM MODEL VS. REAL MACHINE

1. In words. The RAM model charges one unit for every memory access. The real machine charges wildly different amounts depending on which tier of the hierarchy holds the data — from under a nanosecond in a register to ~100 ns in DRAM and far more below that.

2. As a picture. A pyramid: registers at the tiny fast apex, then L1, L2, L3, then DRAM, then SSD, then network at the wide slow base. Size grows and speed falls as you descend.

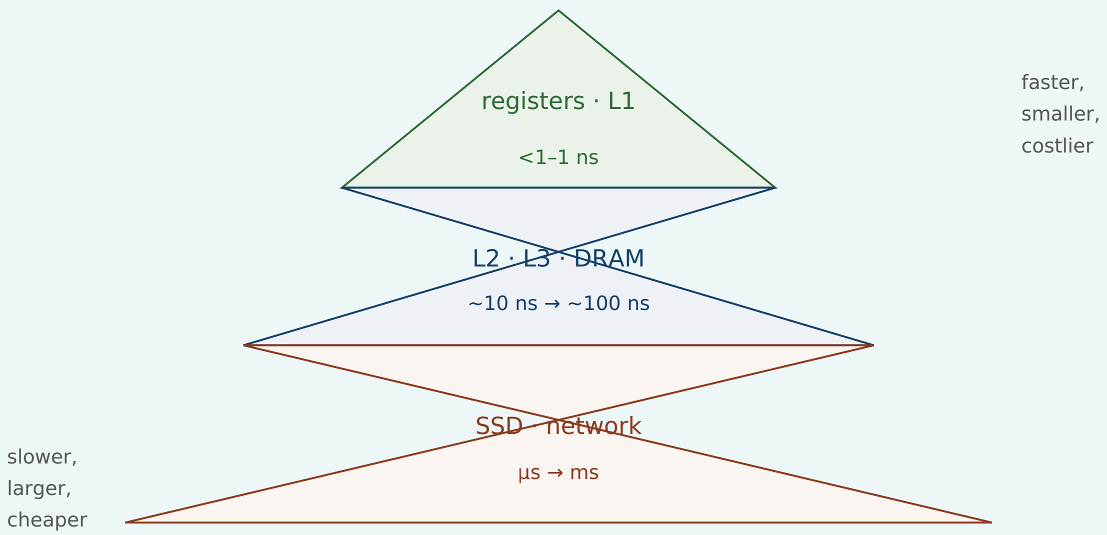


Figure 1.1: The memory hierarchy. Latency rises multiplicatively as you descend; the RAM model pretends the whole pyramid is a single flat tier.

3. As a trace. Consider summing an array of 8 million longs (64 MB), one value at a time. The RAM model says: 8×10^6 loads + 8×10^6 adds = 1.6×10^7 "units," done. The real machine says: the array does not fit in cache, so the cost is dominated by streaming 64 MB up from DRAM; the adds are essentially free, hidden under the wait for memory. Same operation count, completely different account of where the time goes — and only the second account predicts what you will actually measure.

1.4 Why "one operation = one unit of time" is a useful lie

It would be easy to draw the wrong conclusion from all this — to decide the RAM model is broken and should be discarded. That is exactly backwards, and getting the relationship right is the whole point of this chapter. The RAM model is a *useful lie*, in the precise sense that it is false in its literal claims but true and indispensable for the job it is meant to do. Its job is to predict how cost *scales*, and for that job the constant-factor differences between memory tiers are noise: whether a memory access costs 1 ns or 100 ns, an $O(n^2)$ algorithm still loses to an $O(n \log n)$ one once n is large enough. The uniform-cost lie is what makes asymptotic analysis *possible*, and asymptotic analysis is non-negotiable. You must know it and you must trust it, for what it is for.

The mistake is not using the RAM model. The mistake is using it for a question it was never built to answer: *how fast will this actually run?* The RAM model has nothing to say about constant factors, and on modern hardware the constant factors span a hundredfold, all of it hidden inside the word "one" in "one unit of time." Two algorithms with identical O(n) complexity can differ by 50× in wall-clock time because one walks memory in an order the caches love and the other scatters its accesses across DRAM — and the RAM model, by construction, cannot see the difference. So the discipline is to hold two cost models at once and know which question each one answers:

Question	Right model	Denominated in
Will this scale? Which algorithm as $n \to \infty$?	RAM / big-O	operation count
How fast will it actually run at this size on this machine?	the real cost model	where the data lives; bytes moved across tiers

This is the through-line of the entire volume. Every technique that is coming — cache-friendly data layout, avoiding pointer chasing, keeping the CPU's pipeline fed, minimizing system calls, batching I/O — is a way of respecting the second model *after* you have already satisfied the first. You pick a good algorithm with big-O; then you make it fast with mechanical sympathy. Neither replaces the other. An engineer who knows only big-O writes asymptotically optimal code that runs ten times slower than it should; an engineer who knows only micro-optimization polishes an O(n^2) disaster. You need both, in that order.

TRAP: TRUSTING THE RAM MODEL FOR PERFORMANCE

The classic error this chapter exists to prevent: reasoning that because two implementations have the same big-O, they will perform about the same — and therefore choosing between them on style or convenience rather than measuring. "They're both O(n), so it doesn't matter" is true about *scaling* and can be false about *speed* by one to two orders of magnitude, because big-O deliberately discards the very constant factor — where the data lives — that dominates real time. A linked list and an array can both be O(n) to traverse; the array is often many times faster because it streams contiguous memory the cache can prefetch, while the list chases pointers into random DRAM addresses (we will measure a version of this in Chapter 3). The fix is not to distrust big-O — it is right about scaling — but to *stop asking big-O a question it cannot answer*. When the question is "how fast," the answer comes from the memory hierarchy and, ultimately, from measurement, never from the operation count alone.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A colleague says, "Both loops are O(n), so they'll perform the same." Cover the paragraph below and articulate, in two or three sentences, why that reasoning is incomplete — using the words *cache*, *constant factor*, and *memory hierarchy*.

Model answer. "They have the same *asymptotic* cost, which tells us they scale the same way — but big-O throws away the constant factor, and on real hardware that constant is dominated by where the data lives in the memory hierarchy. If one loop streams contiguous memory and the other chases pointers into random DRAM addresses, they can differ by 10× to 100× in wall-clock time despite identical big-O, because a cache hit and a DRAM miss are about two orders of magnitude apart. So same complexity, potentially very different speed — we should measure before assuming they tie." This answer signals seniority precisely because it separates the two cost models and names which one governs the question being asked.

1.5 Mechanical sympathy: the thesis of this volume

There is a phrase for the discipline this chapter is pointing at, borrowed from motor racing and popularized in software by Martin Thompson: **mechanical sympathy**. A driver with mechanical sympathy understands enough about how the car works to get the most out of it without breaking it; a programmer with mechanical sympathy understands enough about how the machine works to write code that runs *with* the grain of the hardware rather than against it. It does not mean writing assembly, and it does not mean premature micro-optimization. It means knowing that the machine has a memory hierarchy, a pipelined out-of-order CPU, a cost for crossing into the operating system, and a real network underneath every abstraction — and letting that knowledge shape the code you write by default.

That is the promise of this volume, and this chapter is its foundation. We have established the one idea everything else builds on: *the RAM model's uniform cost is a useful lie, and real performance comes from the cost model it hides — the one where the price of an operation is set by where its data lives.* From here the volume descends into the machine to make that concrete. Chapter 2 opens up the CPU itself — pipelining, out-of-order execution, branch prediction — to explain the second broken assumption, that one instruction takes one unit of time. Chapter 3 opens up the memory hierarchy — cache lines, locality, the three kinds of miss — to turn "memory access cost varies" from a warning into a set of concrete, exploitable rules. By the end of Part 1 you will look at a loop and see not an operation count but a pattern of movement through the hierarchy, and that shift in vision is the beginning of writing fast code.

CAN YOU… WITHOUT LOOKING?

- State the RAM model's **uniform-cost assumption** and name the three real-hardware facts it gets wrong (memory is not uniform-cost; instructions do not take one unit each; operations are not independent and equal).
- Explain the **memory wall**: CPU throughput outgrew DRAM latency for decades, so average memory-access time comes to dominate performance (Wulf & McKee, 1995).
- Recite the memory hierarchy **in order** — register < L1 < L2 < L3 < DRAM < SSD < network — and say *why the numbers are orders of magnitude only* (machine- dependent, drift with hardware).
- Say why "one operation = one unit of time" is a **useful lie**: right about scaling, silent about the ~100× constant factor that decides real speed.
- Hold both cost models at once and state **which question each answers** (big-O: scaling; the real cost model: actual speed).
- Define **mechanical sympathy** and say why it comes *after* choosing a good algorithm, not instead of it.

RETRIEVAL — CLOSED BOOK

Cover everything above. Answer from memory; then check yourself against the section noted. Rate your confidence (low/med/high) before you look — an overconfident miss is your most valuable signal.

1. **R1.** What single assumption defines the RAM model, and what does adopting it buy you? (§1.1)
2. **R2.** What is the memory wall, who named it, and what is its consequence for program performance? (§1.2)
3. **R3.** Put these in order of increasing latency and give the rough order-of- magnitude gaps: DRAM, L1 cache, register, network round trip, L3 cache, SSD. Why must you present them as orders of magnitude? (§1.3)
4. **R4.** Two implementations have identical O(n) complexity. Can one be 50× slower? Explain in terms of what big-O discards. (§1.4)
5. **R5.** Which cost model answers "will it scale?" and which answers "how fast will it run?" — and in what units is each denominated? (§1.4)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 6 ("The Memory Hierarchy"). The canonical treatment; its Core i7 access- time figures are the source of this chapter's register-through-DRAM orders of magnitude, and it is careful to present them as processor-specific. The free CMU 15-213 course materials cover the same ground.
- **Wulf & McKee, "Hitting the Memory Wall: Implications of the Obvious"** (*ACM SIGARCH Computer Architecture News*, 1995). The two-page paper that named the memory wall; short, readable, and prescient.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007). A long, free, seminal write-up on caches and DRAM. Dated in its specific numbers — read it for the mechanisms, not the figures — but unmatched in depth on why the hierarchy exists.
- **"Latency Numbers Every Programmer Should Know."** The community list attributed to Jeff Dean, and Colin Scott's interactive, year-adjustable version. Use strictly as orders of magnitude; the absolute numbers are dated hardware and are meant to be read as ratios.

Exercises

1.1 WARM-UP State the RAM model's uniform-cost assumption in one sentence. Then list the three things it asserts about real hardware that this chapter says are false.

1.2 WARM-UP Put these seven locations in order from fastest to slowest access, and give the rough order-of- magnitude latency for each: L2 cache, DRAM, register, network round trip (same datacenter), L1 cache, local NVMe SSD, L3 cache.

1.3 WARM-UP In two sentences, explain the memory wall and why it makes the RAM model's uniform-cost assumption worse over time rather than better.

1.4 CORE A DRAM access costs roughly 100 ns and a single core executes on the order of a few billion simple operations per second. (a) Roughly how many simple operations could the core have executed in the time of one DRAM access? (b) In one or two sentences, explain what this ratio implies for a program that spends most of its time following pointers into randomly located memory. State clearly which of your numbers are order-of-magnitude approximations.

1.5 CORE Your colleague replaced an O(n log n) sort with an O(n) bucketing pass and is puzzled that the "faster" version runs slower on their real dataset. Give two distinct explanations grounded in this chapter, and say what single action would settle which one (if either) is right.

1.6 FADED *Worked, with the last step for you.* You must argue in a design review that a proposed change — switching a hot lookup table from a hash map spread across the heap to a sorted array searched by binary search — could be faster *despite* binary search's O(log n) being asymptotically worse than the hash map's O(1). Steps 1–3 are done; complete step 4.
Step 1 (given): Both are cheap in big-O for realistic n, so scaling is not the deciding factor here — this is a constant-factor question.
Step 2 (given): The constant factor is dominated by the memory hierarchy: how many *slow* memory accesses each structure makes per lookup.
Step 3 (given): A hash map lookup typically jumps to one effectively random heap location (likely a cache miss); a binary search over a small contiguous sorted array touches a handful of locations, but they are close together and the array may sit largely in cache.
Step 4 (you): Complete the argument — state the conclusion about which is likely to make fewer costly trips to slow memory, and name the one thing you would do before trusting the argument. Explain *why that final step and not simply* "*pick the lower big-O.*"

1.7 MIXED For each situation, decide whether the RAM/big-O model or the real cost model is the right tool, and say why in one line: (a) choosing between a nested-loop join and a hash join for tables that will grow to billions of rows; (b) explaining why two O(n) array-processing loops on a fixed 100 MB array differ by 8× in wall-clock time; (c) deciding whether an algorithm will still be viable when the input is 1000× bigger; (d) explaining why a linked-list traversal is slower than an array traversal of the same length.

1.8 MIXED A teammate presents two claims: (i) "This algorithm is O(n), so it's optimal." (ii) "This one's O(n) too but ten times faster in practice." Are these claims in contradiction? Decide which cost model each claim belongs to, explain whether both can be true at once, and state what information you would still need to choose between the two implementations for production.

1.9 CORE Define *mechanical sympathy* in your own words, and explain the ordering rule this chapter gives: why you apply it *after* choosing a good algorithm, not instead of choosing one. Give a one-sentence example of each failure mode (knowing only big-O; knowing only micro-optimization).

1.10 STRETCH The latency table in §1.3 is labelled "orders of magnitude only, machine-dependent." Give three distinct reasons the specific numbers cannot be trusted as exact for *your* machine, and explain why the *ratios* between tiers are nonetheless far more stable than the absolute values. What would you do to get numbers you could actually trust for a performance decision?

1.11 LAB On your machine, discover the real numbers instead of trusting the book. Run `getconf LEVEL1_DCACHE_LINESIZE` and `lscpu` (or `sysctl hw` on macOS) and write down your L1/L2/L3 cache sizes and the cache-line size. Then, in any language, write a loop that sums a large array that comfortably exceeds your L3 size and one that fits inside L1, timing each per element. Report the two per-element times and their ratio. State clearly which numbers are yours (the cache sizes, the measured ratio) and which came from the book (nothing — that's the point of the exercise).

Chapter 2 — The CPU Up Close

Before you start: Chapter 1 (the real machine vs. the RAM model — the two cost models). · **Pairs with:** Chapter 3 (the memory hierarchy — the other reason "one instruction = one unit" fails). This chapter attacks the RAM model's *second* false assumption.

BEFORE YOU READ ON

From Chapter 1, recall and answer from memory: (1) The RAM model makes three false assertions about real hardware; §1.1 named them. One was "all memory accesses cost the same" (the memory-hierarchy leak). What was the *second* one — the one this chapter is about? (2) Why is "one operation = one unit of time" called a *useful lie* rather than simply a mistake? (3) *Predict:* a modern CPU executes instructions one after another, finishing each before starting the next — true or false? Answers resolve in §2.1–2.2.

In Chapter 1 we said the RAM model tells three lies, dealt with the first (memory is not uniform-cost), and promised to return for the second: that *one instruction takes one unit of time*. This is the chapter. A modern CPU does not execute your instructions one at a time, in order, each in a tidy tick of the clock. It executes many at once, frequently out of their written order, and it *guesses* — commits to work before it knows whether that work is needed. Understanding this is what separates code that keeps the processor busy from code that leaves it stalling, and the difference, for the same big-O, can again be a large constant factor.

We will stay conceptual and correct. This chapter describes *mechanisms* — the pipeline, superscalar and out-of-order execution, branch prediction, instruction-level parallelism — and gives their costs only as orders of magnitude with explicit caveats, because the exact pipeline depth and misprediction penalty are microarchitecture-specific numbers that differ from chip to chip and that this book will not invent for you. The mechanisms, though, are general and stable, and knowing them changes how you read a hot loop. The through-line: **straight-line, predictable, independent work is fast because it keeps the machine full; branchy, dependent, unpredictable work is slow because it forces the machine to wait or to throw work away.**

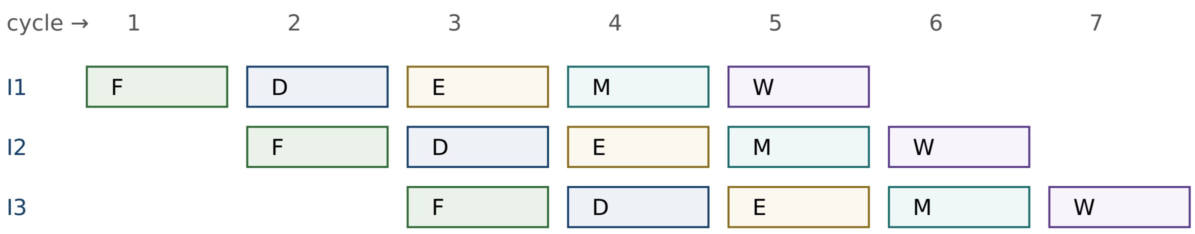
2.1 Fetch–decode–execute, and why it isn't one step

At the coarsest level a CPU repeats a cycle: **fetch** the next instruction from memory, **decode** it into control signals, **execute** it (compute, or read/write memory), and make its results available. This is the von Neumann fetch–decode–execute loop, and if a processor really did it one instruction at a time — finish all stages for instruction 1, then start instruction 2 — then "one instruction = one unit of time" would be roughly true and this chapter would be short. Early processors worked essentially that way.

The trouble is that the stages use *different parts* of the chip. While the execute unit is busy with instruction 1, the fetch unit is idle; while instruction 1 is being decoded, the fetch unit is idle again. Doing one instruction start-to-finish before beginning the next wastes most of the hardware most of the time. The entire architecture of a fast CPU is a series of answers to the question *how do we stop leaving hardware idle?* — and each answer breaks the RAM model's tidy accounting a little more. We take them in order: pipelining (overlap the stages), superscalar (duplicate the units), out-of-order (reorder to avoid waiting), and speculation (guess so you never have to wait for a branch). Bryant and O'Hallaron develop this progression in detail in *Computer Systems: A Programmer's Perspective* (CS:APP), Chapters 4 and 5; we take the conceptual tour.

2.2 Pipelining: overlap the stages

The first idea is an assembly line. Split instruction processing into stages (conceptually: fetch, decode, execute, memory, write-back — real designs have more, and the exact count is a microarchitecture-specific number we will not invent), and let each stage work on a *different* instruction at the same time. While instruction 1 is in "execute," instruction 2 is in "decode," and instruction 3 is in "fetch." Once the pipeline is full, an instruction *completes* every clock cycle even though each individual instruction still takes several cycles to pass all the way through. Throughput goes up by roughly the number of stages, though the latency of any single instruction does not.



Once full, one instruction completes per cycle (F=fetch, D=decode, E=execute, M=memory, W=write-back).

Figure 2.1: A pipeline overlaps stages so different instructions occupy different units in the same cycle. Stage names and count are illustrative; real pipelines have more stages.

Pipelining is why "one instruction = one unit of time" is doubly wrong: an instruction takes several cycles of latency, yet the machine can *complete* one (or more) per cycle. But the assembly line only runs smoothly if the next instruction is always ready and its inputs are available. Two things break that smooth flow — a dependency where an instruction needs a result that is not ready yet (a **data hazard**, which forces a stall), and a branch where the CPU does not yet know *which* instruction comes next (a **control hazard**). The rest of the chapter is about how the hardware fights these two, and how your code can help or hurt that fight.

> **QUICK CHECK**
>
> In a filled pipeline, an instruction's *latency* is several cycles but the pipeline's *throughput* is about one instruction per cycle. In one sentence, why are those two numbers different? (Answer after the next section.)

2.3 Superscalar and out-of-order: more units, smarter order

If one assembly line is good, several are better. A **superscalar** processor has multiple execution units and can fetch, decode, and issue several instructions *per cycle*, so its peak throughput is more than one instruction per cycle — provided it can find independent instructions to feed the parallel units. That proviso is everything. Two instructions can run in the same cycle only if neither needs the other's result; if instruction 2 uses instruction 1's output, they must run in sequence no matter how many units are idle.

To keep those units fed, modern CPUs execute **out of order**. Rather than stalling the whole machine when the next instruction in program order is waiting on a slow input (say, a value still coming from DRAM — recall Chapter 1: that is hundreds of cycles), the processor looks ahead in the instruction stream, finds later instructions whose inputs *are* ready, and executes those now, filling the gap. Results are then reassembled into the original program order before they become visible, so the program's observed behavior is still that of straightforward sequential execution — the reordering is invisible to correctness but decisive for speed. This is why a cache miss does not necessarily stall the CPU: if there is independent work nearby, the machine does that work while the miss is serviced, hiding the latency.

The practical lesson lands directly on how you write hot code. The processor can only overlap work it can prove is independent, and it can only look a bounded distance ahead. Code with lots of **instruction-level parallelism** (ILP) — many operations that do not depend on each other's results — gives the out-of-order engine plenty to chew on and runs near the machine's peak. Code that is a long *dependency chain* — each step needing the previous step's result — starves the parallel units no matter how many there are, because there is never more than one thing that can run next. Same instruction count, very different speed; and, as in Chapter 1, the difference is exactly the constant factor big-O throws away.

(Answer to the quick check: latency counts the cycles for one instruction to traverse all stages; throughput counts how often a new instruction completes. Because stages overlap, a new instruction can finish every cycle even though each one individually took several cycles to get through.)

THE SAME IDEA THREE WAYS: DEPENDENCY CHAINS VS. INDEPENDENT WORK

1. In words. Summing an array into a single accumulator forces each add to wait for the previous add's result — a dependency chain, low ILP. Summing into several independent partial accumulators and combining them at the end lets the superscalar, out-of-order engine run several adds per cycle — high ILP.

2. As a picture. One long chain `a→a→a→a` (each arrow "must finish before") versus four short parallel chains that only merge at the very end.

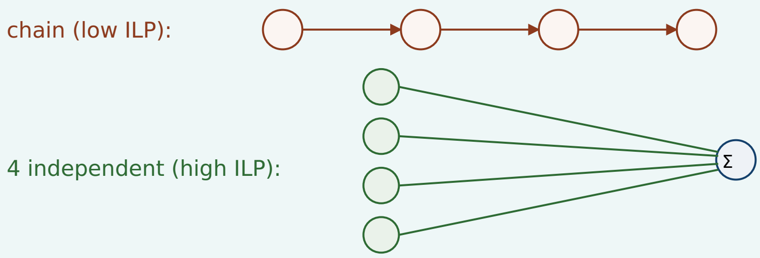


Figure 2.2: A dependency chain serializes execution; independent operations expose parallelism the CPU can exploit. Combining partial sums at the end recovers the same result.

3. As a trace. Both compute the same sum of n numbers — identical operation count, identical $O(n)$. The chained version can retire roughly one add per dependency latency; the four-accumulator version can keep several execution units busy at once, finishing in a fraction of the wall-clock time. (This is a real and measurable effect; CS:APP Chapter 5 develops it as "loop unrolling with multiple accumulators," and the performance-engineering chapters of this volume return to it.)

2.4 Branch prediction and the cost of a miss

Now the control hazard. When the CPU reaches a conditional branch — an `if`, a loop test, a `switch` — it does not yet know which way execution will go, because that depends on a condition that may not be computed yet. But the pipeline needs to fetch *something* next; it cannot sit idle for the several cycles it takes to resolve the branch. So the processor **predicts** which way the branch will go and speculatively executes down the predicted path, doing real work on the assumption that its guess is right.

Modern **branch predictors** are remarkably good — they track the history of each branch and find patterns, and on predictable branches (a loop that runs a million times and exits once; a check that is almost always false) they are right the overwhelming majority of the time, so the speculation is nearly free and the pipeline never stalls. This is why a loop with a highly predictable exit condition runs fast: the CPU correctly guesses "keep looping" every time until the one time it does not.

The cost arrives on a **misprediction**. When the branch resolves and the guess was wrong, all the speculatively executed work on the wrong path must be discarded, and the pipeline must be refilled from the correct target — the instructions already in flight are thrown away and the machine restarts fetching from the right place. The penalty is roughly proportional to how deep the pipeline is: everything that was in flight is lost. On modern deeply pipelined CPUs this penalty is on the order of *ten to a few tens of cycles* per misprediction — an order of magnitude, not a figure to quote precisely, since the exact number is specific to each microarchitecture and this book will not invent one for a particular chip. (CS:APP, Chapter 5, works a concrete example for one processor; treat any single number as machine-specific.) What matters is the shape: a mispredicted branch costs many times what a correctly predicted one does.

The consequence for your code is direct and sometimes surprising. A branch that is *unpredictable* — one that goes each way about half the time, in no pattern the predictor can learn — pays the misprediction penalty roughly half the time it executes. In a tight loop, that can dominate the loop's cost. This is the mechanism behind a famous and initially baffling observation: processing a *sorted* array can be markedly faster than processing the same data *unsorted*, when the loop body contains a data-dependent branch, because sorting makes the branch predictable (long runs of the same outcome) whereas the shuffled data makes it a coin flip the predictor cannot learn. The arithmetic is identical; only the predictability changed. When you see a performance mystery like that, the branch predictor is a prime suspect.

TRAP: THE UNPREDICTABLE BRANCH IN THE HOT LOOP

The error is to assume that because two versions of a loop do the same arithmetic, they cost the same — forgetting that a *data-dependent, unpredictable branch* inside a hot loop can add a misprediction penalty on a large fraction of iterations, dwarfing the arithmetic. It is the CPU-level cousin of the Chapter 1 trap ("same big-O, so same speed"): same operation count, very different wall-clock time, because one version keeps the pipeline full and the other keeps flushing it. The tells: a branch whose outcome depends on the data and follows no pattern, in code that runs an enormous number of times. The fixes are the standard toolkit — make the branch predictable (e.g. by sorting), or remove it entirely by turning control flow into data flow (branchless code, conditional-move or arithmetic tricks, table lookups, or SIMD masks — the performance-engineering chapters cover these). But confirm with measurement first: on *predictable* branches the predictor already makes the branch nearly free, and a branchless rewrite there just adds complexity for no gain.

2.5 What this means for the code you write

Step back and the theme of the chapter is one sentence: **the CPU runs fastest when you feed it a steady stream of predictable, independent work, and stalls when you feed it dependencies and surprises.** That single idea reorganizes a pile of otherwise-disconnected performance folklore into consequences of how the machine actually works:

- **Straight-line code with predictable branches** keeps the pipeline full and speculation cheap — fast.
- **Independent operations (high ILP)** let the superscalar, out-of-order engine run several things per cycle — fast.
- **Long dependency chains** serialize execution and starve the parallel units — slow, even at the same instruction count.
- **Unpredictable, data-dependent branches** cause mispredictions that flush the pipeline — slow, sometimes dramatically.

- **A cache miss (Chapter 1) need not stall the CPU** if there is independent work nearby for the out-of-order engine to do while it waits — which is why ILP and memory behavior are entangled, and why both matter together.

None of this changes an algorithm's big-O, and none of it should tempt you to abandon big-O — a pipeline-friendly O(n^2) is still a disaster. This is, exactly as in Chapter 1, the *second* cost model at work: after you have chosen a good algorithm, the constant factor that decides real speed is set by how well your code fits the machine — its memory hierarchy (Chapter 1 and Chapter 3) and its pipelined, speculative, superscalar core (this chapter). Chapter 3 now returns to the memory side and turns "data location matters" into the concrete, exploitable rules of caches and locality — the other half of mechanical sympathy.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. In a review, someone asks why sorting an array before a filtering loop made the loop faster, even though sorting is extra work and the loop does the same comparisons either way. Cover the paragraph below and explain it in two or three sentences, using *branch predictor*, *misprediction*, and *pipeline*.

Model answer. "The loop has a data-dependent branch. On the unsorted data that branch goes each way unpredictably, so the branch predictor is wrong about half the time, and every misprediction flushes the pipeline — a penalty on the order of tens of cycles per miss, paid on a huge fraction of iterations. Sorting makes the branch outcomes come in long predictable runs, so the predictor is almost always right and the pipeline stays full; the arithmetic is identical, but the misprediction cost nearly vanished. Net of the one-time sort, the loop got cheaper because we made it *predictable*, not because we did less arithmetic." Naming the mechanism — rather than saying "sorting is just faster" — is what marks the answer as senior.

CAN YOU… WITHOUT LOOKING?

- Explain **pipelining** and why it makes an instruction's *latency* (several cycles) differ from the pipeline's *throughput* (about one per cycle).
- Say what **superscalar** and **out-of-order** execution each add, and why both depend on finding *independent* instructions.
- Contrast a **dependency chain** (low ILP, serialized) with **independent work** (high ILP, parallel) and give the multiple-accumulator sum as the example.
- Describe **branch prediction** and the cost of a **misprediction** (flush the pipeline; penalty on the order of tens of cycles — machine-specific, an order of magnitude only).
- Explain why processing a **sorted** array can beat the same data unsorted when the loop has a data-dependent branch.
- State the chapter's one sentence: the CPU is fast on predictable, independent work and stalls on dependencies and surprises — and why this is the RAM model's *second* lie.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate your confidence before checking. An overconfident miss is the signal to reread.

1. **R1.** Why does executing one instruction fully before starting the next waste the hardware, and what is pipelining's fix? (§2.1–2.2)
2. **R2.** What must be true of two instructions for a superscalar CPU to run them in the same cycle, and what does out-of-order execution do when the next in-order instruction is stalled? (§2.3)
3. **R3.** Define instruction-level parallelism. Why is a single-accumulator sum lower ILP than a multi-accumulator sum computing the same total? (§2.3)
4. **R4.** What does a CPU do at a conditional branch before the condition is known, and what exactly is discarded on a misprediction? Give the penalty as an order of magnitude with its caveat. (§2.4)
5. **R5.** Explain the sorted-vs-unsorted array speed difference in terms of the branch predictor. Which of the two cost models from Chapter 1 does this belong to? (§2.4–2.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 4 ("Processor Architecture") and Chapter 5 ("Optimizing Program Performance"). Chapter 4 builds a pipelined processor from scratch; Chapter 5 covers superscalar/out- of-order machines, branch misprediction, dependency chains, and multiple accumulators, all with concrete (processor-specific) numbers. The primary source for this chapter.
- **Agner Fog, optimization manuals** (agner.org, free). Deep, empirical detail on microarchitecture, pipelines, and branch prediction across real x86 CPUs. Dated in places — verify against current parts — but the mechanism-level explanations are excellent.
- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). The lectures on modern processors, ILP, and branch behavior connect these mechanisms to measurable speedups in real code.

Exercises

2.1 WARM-UP Name the four techniques, in the order this chapter introduces them, that a fast CPU uses to stop leaving hardware idle, and give a one-line description of what each one does.

2.2 WARM-UP Distinguish *latency* and *throughput* for a single instruction passing through a pipeline. Why can a pipeline complete about one instruction per cycle while each instruction still takes several cycles from start to finish?

2.3 WARM-UP Define a *data hazard* and a *control hazard* in one sentence each, and say which of the two branch prediction is designed to address.

2.4 CORE Two loops compute the same sum of an array of doubles. Loop A accumulates into a single variable; loop B accumulates into four independent partial sums and adds them at the end. Both do the same number of additions. Explain, using ILP and the out-of-order engine, why B can be substantially faster, and name the one condition on the additions that makes B's parallelism possible.

2.5 CORE A hot loop contains `if (x[i] > threshold) count++;`. On dataset P the branch is taken in long predictable runs; on dataset Q it is taken about half the time in no pattern. The arithmetic is identical. Explain why the loop can run several times faster on P, and estimate qualitatively where the extra time on Q goes.

2.6 COST A branch mispredicts with probability p per execution, and each misprediction costs about k cycles (an order-of-magnitude, machine-specific number — state that caveat). The loop body otherwise takes about b cycles. (a) Write the approximate average cost per iteration. (b) For what range of p does the misprediction term dominate the body? (c) Explain why making the branch predictable attacks p, and a branchless rewrite attacks k (by removing the branch), and why you would measure before choosing.

2.7 FADED *Worked, with the last step for you.* Explain why a cache miss (Chapter 1) does not always stall the CPU. Steps 1–3 given; complete step 4.
Step 1 (given): A load that misses to DRAM takes on the order of hundreds of cycles to return its value (Chapter 1).
Step 2 (given): A modern CPU executes out of order: when the next in-order instruction is waiting, it looks ahead for later instructions whose inputs are already available.
Step 3 (given): If there is independent work near the missing load, the machine can execute that work during the hundreds of cycles the miss is being serviced.
Step 4 (you): State the conclusion about whether the miss stalls the CPU, and name the property of the surrounding code that determines whether the latency gets hidden or not. Explain *why that property, and not the miss latency alone, decides the outcome.*

2.8 MIXED For each observation, decide whether the dominant mechanism is the *memory hierarchy* (Chapter 1), *branch prediction*, or an *instruction-level dependency chain* (this chapter) — and justify in one line: (a) a loop got 3× faster after the input array was sorted, with no change to the arithmetic; (b) a loop got faster after splitting one accumulator into four; (c) a traversal got much slower when the same data was stored as a linked list instead of an array; (d) a tight numeric loop with no branches and no memory pressure is already near peak throughput.

2.9 MIXED A colleague says: "My function is O(n) and yours is O(n), but yours runs 5× faster — that's impossible, big-O says they're the same." Decide which cost model resolves the paradox, list three distinct mechanisms from Chapters 1–2 that could each independently produce a 5× gap at equal big-O, and state the single action that would identify which one is responsible here.

2.10 STRETCH Speculation means the CPU does work it might have to throw away. (a) Explain why this is still a net performance win on typical code. (b) Give a case where heavy speculation on an unpredictable branch is a net *loss*. (c) Speculative execution has also had *security* consequences in real processors. Without inventing specifics, state at a high level why executing instructions before you know they should run could, in principle, leak information — and note that the precise mechanisms and mitigations are a real, separate topic you would need primary sources to describe accurately.

2.11 LAB Measure a dependency chain. In C (or any compiled language), write two loops that sum a large array of `double`: one with a single accumulator, one with four independent accumulators combined at the end. Compile with optimization *off* for the reduction (or use a language/setting that will not auto-vectorize away the difference), time both, and report the ratio. State which numbers are yours and note that the exact ratio depends on your CPU and compiler. If you cannot prevent the compiler from transforming the loop, say so and report what you observed instead — do not report a number you did not actually measure.

Chapter 3 — The Memory Hierarchy & Caches

Before you start: Chapter 1 (the memory wall and the latency hierarchy), Chapter 2 (the CPU; why a miss need not stall). · **Pairs with:** the rest of Part 1 and the performance-engineering chapters, which turn these rules into measured speedups. This chapter makes "data location matters" concrete and exploitable.

BEFORE YOU READ ON

From the previous two chapters, from memory: (1) Chapter 1 said accessing DRAM costs on the order of ~100 ns while a nearby cache hit costs ~1 ns — about how many orders of magnitude apart is that? (2) Chapter 2 said a cache miss does *not* always stall the CPU. Under what condition does the latency get hidden? (3) *Predict:* you sum every element of a large 2-D array. Does the *order* in which you visit the elements — row by row vs. column by column — change how long it takes, given that you touch exactly the same elements either way? By the end of this chapter you will have a measured answer.

Chapter 1 told you the memory wall exists and that hardware answers it with a hierarchy of caches. Chapter 2 told you the CPU can sometimes hide a miss behind other work. This chapter is where "memory access cost varies" stops being a warning and becomes a set of concrete, mechanical rules you can design against — because caches are not magic, they follow simple policies, and once you know the policies you can write code that hits the cache instead of missing it. The payoff is the largest, cheapest performance win available to most programs: not a better algorithm, just the same algorithm arranged so the data is where the CPU expects it.

We will finish with a measurement, not a claim. At the end of the chapter is a short C program that traverses the same array two ways — the cache-friendly way and the cache-hostile way — and the exact speed ratio it produced when run for this book, on a specific machine, so you can see the effect is real and roughly how big it is. The number is machine-dependent, and we will say so; but it is measured, not invented.

3.1 The cache line: memory moves in blocks, not bytes

The first and most important fact about caches overturns a picture you have probably carried since you first learned about memory: that you read one byte, or one `int`, at a time. You do not. When the CPU needs a value that is not already in cache, it does not fetch just that value — it fetches an entire aligned **cache line** (also called a cache block) containing it, and installs the whole line in the cache. On the common x86-64 processors this line is **64 bytes**; Bryant and O'Hallaron state this standard size in *Computer Systems: A Programmer's Perspective* (CS:APP, Chapter 6), and it is what `getconf LEVEL1_DCACHE_LINESIZE` reports on such machines. (It is a hardware parameter, not a law of nature — verify it on your target — but 64 bytes is the near-universal value on current x86-64.)

This single mechanism explains most of cache behavior. When you touch one byte, the seven-plus neighbors sharing its 64-byte line come along for free — they are now in cache, and touching them next costs a cheap hit instead of an expensive miss. So the cost of a memory access is not really "per access"; it is "per line brought in." A program that uses every byte of each line it fetches gets the full value of every expensive DRAM trip. A program that touches one byte per line and then moves far away wastes 63 of every 64 bytes it pays to move — it pays full price for the line and uses a fraction of it. That ratio, *useful bytes per line fetched*, is the hidden efficiency knob behind an enormous amount of performance.

> **QUICK CHECK**
>
> On a machine with 64-byte cache lines, you read one `int` (4 bytes). How many bytes does the hardware actually move into the cache, and how many neighboring `int`s are now cheap to access? (Answer after the next section.)

3.2 Locality: the property caches reward

Caches work — they successfully hide the memory wall — because real programs exhibit **locality of reference**, a strong tendency to access memory that is close in time or space to what they just accessed. There are two kinds, and CS:APP (Chapter 6) frames them as the two properties good code should have:

- **Temporal locality:** if you access a location, you are likely to access it *again soon*. A loop variable, an accumulator, a hot lookup table — accessed repeatedly, so after the first (missing) access they sit in cache and every later access is a hit.
- **Spatial locality:** if you access a location, you are likely to access *nearby* locations soon. Walking an array in order, reading struct fields together — because memory arrives a whole line at a time, spatial locality means the neighbors you fetched are exactly the ones you are about to use.

The hardware bets on both. The cache keeps recently used lines (exploiting temporal locality), and by fetching a full line it pre-loads spatial neighbors (exploiting spatial locality). Some CPUs also *prefetch* — detect a sequential access pattern and fetch lines ahead of the program's demand, so the data is already resident by the time the loop reaches it. All of this is automatic and invisible until you fight it. Your job is to write code whose access pattern *has* locality, so that the hardware's bets pay off. Code with good locality runs near the speed of cache; code that scatters its accesses runs near the speed of DRAM; and from Chapter 1 you know that is about a 100× difference, available on the same algorithm.

(Answer to the quick check: the hardware moves the full 64-byte line, so 64 bytes; that line holds 16 `int`s, so the 15 neighbors sharing the line are now cheap hits.)

3.3 Hits, misses, and the three C's

An access that finds its line already in cache is a **hit**; one that does not is a **miss**, and a miss pays to fetch the line from the next level down (another cache, or DRAM). Since misses are where the time goes, it helps to classify *why* a miss happened. The classic taxonomy, the **three C's** (presented in CS:APP, Chapter 6), sorts every miss into one of three causes:

- **Compulsory (cold) miss:** the first-ever reference to a line. It was never in cache, so the first touch must miss. Unavoidable in principle — you have to load data at least once — but you can make each compulsory miss *count* by using the whole line (spatial locality) once you have paid for it.
- **Capacity miss:** the line was in cache but got evicted because the program's working set — the data it is actively using — is simply larger than the cache. If you stream through 512 MB repeatedly and the cache holds a few MB, lines you will need again get pushed out before you return to them. The fix is to shrink the working set (e.g. process data in cache-sized *blocks*, a technique called blocking or tiling).
- **Conflict miss:** the line was evicted not because the cache was full overall, but because too many of the addresses you are using map to the *same* small set of cache slots and crowd each other out. This one depends on the cache's *associativity*, which we take next.

3.4 Associativity, conceptually

A cache cannot let any line live in any slot — searching the whole cache on every access would be too slow. Instead each memory address maps to a restricted set of slots. In a **direct-mapped** cache, each address has exactly one slot it may occupy; simple and fast, but two hot addresses that happen to map to the same slot will endlessly evict each other even if the rest of the cache is empty — that is a conflict miss. At the other extreme, a **fully associative** cache lets a line go in any slot (no conflict misses, but expensive to build). Real caches are **set-associative**: a compromise where each address maps to a small *set* of slots (say 8, an "8-way" cache) and the line may occupy any slot within its set. This kills most conflict misses while staying cheap enough to build.

You rarely need to compute set indices by hand, and this book will not turn associativity into arithmetic drills. What you need is the intuition: conflict misses are why a data structure can perform badly at one size or memory alignment and fine at another, seemingly at random. A classic symptom is a loop that slows sharply when an array's row length (its "stride") is a power of two that aligns many accesses onto the same cache sets. When performance changes with the exact size or alignment of your data rather than the amount of it, suspect conflict misses — and know the deeper treatment (and the set-index arithmetic) is in CS:APP, Chapter 6.

QUICK CHECK

Classify each miss: (a) the very first read of a freshly allocated array; (b) re-reading a 1 GB array in a loop when the cache holds only a few MB; (c) two hot variables that keep evicting each other though the cache is mostly empty. (Answer below the demonstration.)

3.5 Why arrays are fast and pointer-chasing is slow

Now the payoff that ties the chapter together, and the reason "same big-O, different speed" keeps recurring. Consider two ways to store a sequence of integers you will walk from start to finish. An **array** stores them contiguously: element i+1 sits right after element i in memory. A **linked list** stores each element in its own separately allocated node, connected by pointers; consecutive nodes can be anywhere in the heap, often far apart. Both take O(n) to traverse — the same number of "steps" — and the RAM model says they cost the same. The real machine disagrees, sometimes by more than 10×.

Walking the array has excellent spatial locality: each 64-byte line brought in holds many consecutive elements, so most accesses are hits, and the hardware prefetcher, seeing the sequential pattern, fetches lines ahead of you so the data is waiting. Walking the linked list is **pointer-chasing**: each node's address is only known once you have read the *previous* node's pointer, so the accesses are (a) to scattered, effectively random addresses — poor spatial locality, a likely cache miss per node — and (b) a strict dependency chain, since you cannot fetch node i+1 until node i arrives. That dependency defeats the latency-hiding trick from Chapter 2: there is no independent work to overlap with the miss, so the CPU stalls on the full DRAM latency at every step. Poor locality and a serial dependency, together, are why pointer-chasing through memory is one of the slowest things a program can do, and why cache-friendly, array-based layouts underpin most high-performance data structures. (This is the mechanism behind the array-vs-list remark back in Chapter 1's warning box — now you know exactly why.)

3.6 A measured demonstration: row-major vs. column-major

Let us prove the effect rather than assert it, with the cleanest possible experiment: the *same* arithmetic on the *same* data, differing only in access order. Store an N×N matrix of `long` in a single contiguous block in row-major order (row 0, then row 1, …), and sum every element two ways. The **row-major** traversal walks memory in order — inner loop over columns, so consecutive accesses are adjacent in memory (stride 1), excellent spatial locality. The **column-major** traversal walks down columns — inner loop over rows, so each step jumps N elements forward in memory (stride N), touching a different cache line almost every time. Same elements, same additions, same result; only the order differs.

```
#define N 8192            /* an N x N matrix of long = 512 MiB, far larger than cache */

/* row-major: inner index j varies fastest -> contiguous, stride 1 */
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        sum_row += a[(size_t)i * N + j];

/* column-major: inner index i varies fastest -> stride of N longs per step */
for (int j = 0; j < N; j++)
    for (int i = 0; i < N; i++)
        sum_col += a[(size_t)i * N + j];
```

Both loops produced the identical sum (a correctness check that they really do the same work). Built with `gcc -O2` and run for this book on a 12th-generation Intel Core i7 (i7-12700H) laptop, on a 512 MiB matrix that dwarfs the cache, the row-major traversal took roughly **0.04 s** and the column-major traversal roughly **0.95–0.99 s** — the column-major version was about **24–26× slower** across repeated runs, for doing the very same additions in a different order. **This exact ratio is machine-dependent** — it depends on cache sizes, prefetcher, memory bandwidth, and compiler, and you should expect a different number on your hardware — but the *direction and rough magnitude* (a large multiplier, not a few percent) is the robust, reproducible result, and you can reproduce it yourself with the lab exercise. The mechanism is exactly §3.1–3.2: the row-major loop uses all 8 `long`s of every 64-byte line it fetches and lets the prefetcher run ahead; the column-major loop touches one `long` per line and throws the other 7 away, so it issues many times more line fetches and stalls on the misses it cannot hide.

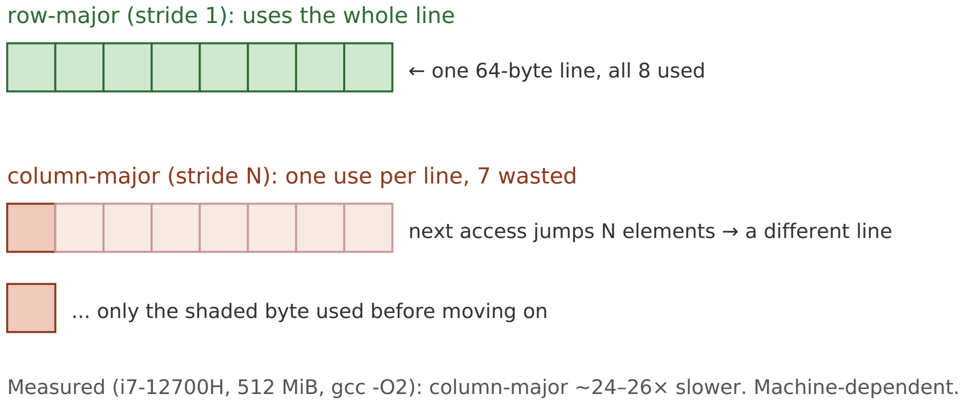


Figure 3.1: The two traversals do identical arithmetic. Row-major consumes every element of each fetched line; column-major uses one element per line and moves on, wasting most of the memory bandwidth it pays for.

(Answer to the §3.4 quick check: (a) compulsory/cold — first-ever reference; (b) capacity — the working set exceeds the cache; (c) conflict — addresses collide on the same cache set despite free space elsewhere.)

THE SAME IDEA THREE WAYS: LOCALITY PAYS

1. In words. Memory arrives one cache line at a time, so code that uses all of each line it fetches (good spatial locality) moves far less memory and hits far more often than code that touches one item per line and jumps away.

2. As a picture. Figure 3.1: a fully-consumed line vs. a line touched once and abandoned.

3. As a trace / measurement. The row-major vs. column-major experiment: identical sum, identical operation count, ~24–26× wall-clock difference on the test machine because one order has spatial locality and the other does not. Prose says "locality matters"; the number says how much.

TRAP: IGNORING CACHE LOCALITY (THE "SAME BIG-O, SO SAME SPEED" FALLACY, MEMORY EDITION)

The recurring error of this Part, now at the cache level: judging two implementations equivalent because they have the same complexity and the same operation count, while ignoring that one has good locality and the other does not. Column-major matrix traversal, linked lists walked as pointer chains, arrays-of-pointers-to-objects scattered across the heap, hash tables with poor probe locality — all can be an order of magnitude slower than a cache-friendly layout doing the identical logical work. The tell is a hot loop over a data structure larger than cache whose access pattern is *not* sequential. The fixes are layout, not algorithm: store data contiguously in the order you traverse it; keep frequently-used fields together; process large data in cache-sized blocks (blocking/tiling); prefer arrays of values to arrays of pointers. And, as always in this Part: confirm by *measuring*, on your data and your machine, rather than trusting any single ratio — including this chapter's ~25×, which is one machine's number, not a constant of nature.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A reviewer asks why you rewrote a nested loop to swap the order of the two indices when "it computes exactly the same thing." Cover the paragraph below and answer in two or three sentences, using *cache line*, *spatial locality*, and *stride*.

Model answer. "It computes the same result, but the original visited the array with a large stride, touching one element per 64-byte cache line and then jumping to a different line — so it moved roughly a line's worth of memory per element and got almost no reuse. Swapping the loop order makes the inner traversal stride-1, so each fetched line is fully consumed before we move on: far fewer misses, the prefetcher can run ahead, and we use the memory bandwidth we're already paying for. Same big-O and same arithmetic; the change is purely spatial locality, and on our data it measured about an order of magnitude faster." Naming the mechanism — line, stride, locality — rather than "I made it faster" is the senior register.

3.7 The rules, and where Part 1 goes next

Three chapters in, the cost model the RAM abstraction hides is now concrete. Chapter 1 established that where data lives dominates cost and that this is a second cost model alongside big-O. Chapter 2 opened the CPU and showed why predictable, independent work runs fast. This chapter turned "data location matters" into rules you can design against: memory moves in 64-byte lines, so use the whole line (spatial locality) and reuse hot data (temporal locality); classify your misses (compulsory, capacity, conflict) to know which fix applies (use each line, shrink the working set, avoid alignment collisions); and remember that contiguous array access is fast while pointer-chasing is slow because it has neither locality nor independent work to hide the misses. These are not micro-optimizations to sprinkle on at the end — they are how you get an order of magnitude on the same algorithm, and you can now *see* them in code.

The rest of Part 1 builds directly on this. The next chapters take the model down to how data is actually laid out in memory — bytes, alignment, the stack and the heap — so you can control locality deliberately rather than by accident, and the performance-engineering chapters return with profiling tools that *show* you the misses and mispredictions these three chapters taught you to predict. The habit to carry forward is the one this Part has drilled from three angles: when two pieces of code have the same big-O but different speed, the difference is the machine — its memory hierarchy and its pipeline — and you now know where to look.

CAN YOU… WITHOUT LOOKING?

- State the **cache line** size on common x86-64 (64 bytes, per CS:APP) and explain why memory moving in lines, not bytes, is the key fact behind cache behavior.
- Define **temporal** and **spatial** locality and say which one a full-line fetch and which one a prefetcher exploits.
- Name the **three C's** — compulsory, capacity, conflict — and give the fix each one points to.
- Explain **associativity** conceptually (direct-mapped vs. set-associative vs. fully associative) and why conflict misses make performance depend on size/alignment, not just data volume.
- Explain why **array traversal is fast and pointer-chasing is slow** in terms of locality *and* the dependency chain from Chapter 2.
- Describe the **row-major vs. column-major** demonstration, its measured order-of-magnitude result, *and* why the exact number is machine-dependent.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a cache line, what is its common size on x86-64, and why does its existence mean cost is "per line fetched," not "per byte"? (§3.1)
2. **R2.** Define temporal and spatial locality and give a code example of each. (§3.2)
3. **R3.** Name the three C's and, for each, the design change that reduces it. (§3.3)
4. **R4.** Why can a loop's speed change sharply with the array's size or alignment even when the amount of data is the same? Which of the three C's is this? (§3.4)
5. **R5.** Give two independent reasons a linked-list traversal is slower than an array traversal of the same length, one about locality and one about dependencies. (§3.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 6 ("The Memory Hierarchy"). The source for this chapter's cache-line size, locality framing, three-C's taxonomy, and set-associativity arithmetic, plus the "memory mountain" experiment that visualizes read throughput across sizes and strides. The free CMU 15-213 materials cover it.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free). The deepest freely available treatment of cache organization, associativity, prefetching, and access patterns. Its absolute numbers are dated — read for mechanism — but its analysis of why layout matters is unmatched.
- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). Lectures on cache-efficient algorithms, blocking/tiling, and measuring cache behavior with real tools — the applied continuation of this chapter.

Exercises

3.1 WARM-UP On a machine with 64-byte cache lines, how many `double`s (8 bytes each) fit in one line? If you read one `double` from a cold line, how many neighbors become cheap hits, and which kind of locality does that reward?

3.2 WARM-UP Give a one-sentence code example of temporal locality and one of spatial locality, and say which cache mechanism (line fill, retention of recent lines, prefetching) exploits each.

3.3 WARM-UP Classify each miss as compulsory, capacity, or conflict: (a) the first read of a just-`malloc`ed buffer; (b) repeatedly scanning a 4 GB dataset with an 8 MB last-level cache; (c) a loop that slows down only when an array's row length is a particular power of two.

3.4 CORE You must sum an $N \times N$ matrix stored in row-major order. One colleague writes the inner loop over columns, another over rows. (a) Which is faster and why, in terms of cache lines and stride? (b) Both are $O(N^2)$ and do the same additions — reconcile that with a large measured speed difference. (c) Name the kind of miss the slow version suffers most.

3.5 CORE A profile shows a data structure of 5 million small objects is slow to traverse. It is currently an array of *pointers*, each pointing to a separately `malloc`ed object. Propose a layout change that would improve locality, and explain both why it reduces misses (spatial locality) and why it helps the CPU hide the remaining misses (recall the dependency argument from Chapter 2).

3.6 COST A traversal touches one 8-byte value per 64-byte cache line (stride 8 values). (a) What fraction of each fetched line does it use? (b) Compared with a stride-1 traversal of the same values, roughly how many times more memory must it move? (c) Explain how this back-of-the-envelope ratio relates to (though it will not exactly equal) the measured row-major/column-major slowdown, and name two hardware factors that make the measured number differ from this simple estimate.

3.7 FADED *Worked, with the last step for you.* Explain why the column-major traversal in §3.6 stalls the CPU more than the row-major one, tying it to Chapter 2. Steps 1–3 given; complete step 4.
Step 1 (given): Column-major access has stride N, so almost every access lands on a new cache line — a likely miss to DRAM (~100 ns, Chapter 1).
Step 2 (given): A cache miss need not stall the CPU if there is independent work nearby to overlap with it (Chapter 2 §2.3).
Step 3 (given): A summation into one accumulator, walking predictably, does have some independent memory-level parallelism — but the row-major version keeps far more useful data resident per fetched line, so it issues far fewer misses to begin with.
Step 4 (you): Complete the explanation: state which effect dominates the ~25× gap here — the sheer *number* of misses (locality) or the inability to hide each one (dependencies) — and justify your choice. Explain *why the number of misses is the primary lever in this particular experiment,* and what you would change about the code to test your claim.

3.8 MIXED For each speedup, decide whether the primary mechanism is *spatial locality / cache lines*, *a capacity miss fixed by blocking*, *branch prediction* (Chapter 2), or an *ILP / dependency chain* (Chapter 2), and justify in a line: (a) tiling a large matrix multiply into cache-sized blocks; (b) sorting the input before a data-dependent filter loop; (c) switching an array-of-structs traversal to consume each cache line fully; (d) summing into four accumulators instead of one.

3.9 MIXED Two implementations of the same O(n) aggregation differ by 15× in wall-clock time. Without running anything yet, list the candidate causes drawn from all of Chapters 1–3 (name at least three distinct mechanisms), then describe the *single* measurement you would take first to narrow it down and why that one is most informative.

3.10 STRETCH The chapter reports a specific ~24–26× row-major/column-major ratio and repeatedly insists it is machine-dependent. (a) List three properties of a machine that would change this ratio, and say for each whether it would tend to raise or lower it. (b) Explain why the *existence* and rough *magnitude* (a large multiplier) of the effect is nonetheless robust across machines, even though the exact number is not. (c) Why is reporting "about 25× on this specific CPU, and reproduce it yourself" more honest and more useful than printing a single universal multiplier?

3.11 LAB Reproduce the demonstration on your own machine. First run `getconf LEVEL1_DCACHE_LINESIZE` and `lscpu` (or the macOS `sysctl hw` equivalents) and record your cache-line size and L1/L2/L3 sizes. Then write the two-loop matrix-sum program (size it so the matrix comfortably exceeds your last-level cache), verify both loops produce the identical sum, time each, and report *your* ratio. Note how it compares to the book's ~25× and to your §3.6 stride estimate. Report only numbers you actually measured; if the compiler optimizes a loop away, change the code (e.g. consume the sum) and say what you did.

Chapter 4 — Virtual Memory & Address Translation

Before you start: Chapter 3 (caches; memory moves in lines; a miss to DRAM costs ~100 ns), Chapter 1 (the latency hierarchy). · **Pairs with:** Chapter 5 (data layout) and the operating-system Part, which builds the page tables this chapter only describes. This chapter adds one more cache to the picture — a cache for *address translations* — and shows why a huge, scattered working set can be slow for a reason the last chapter did not cover.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 3 said the CPU keeps recently used cache lines so a repeated access is a cheap hit — what is the general trick there, in one word? (2) Chapter 1 put a DRAM access at ~100 ns and an L1 hit at ~1 ns — about how many orders of magnitude apart? (3) *Predict:* the pointer `p` in your C program holds an address like `0x7ffd3a08`. When the CPU reads `*p`, do you think that number is the actual electrical address of a cell in your DRAM chips — the physical location — or something the hardware has to *translate* first? By the end of the chapter you will know which, and what the translation costs.

Every address your program has ever used is a lie — a convenient, deliberate, hardware-enforced lie. When your code holds a pointer and dereferences it, the number in that pointer is a **virtual address**: a name in a private, per-process address space that the operating system and CPU maintain, not the **physical address** of a byte in the DRAM chips. Before the load can touch memory, the hardware must *translate* virtual to physical. That translation is invisible, automatic, and — this is the point for a performance book — *not free*. This chapter explains the mechanism (why virtual memory exists, how pages and page tables implement it) and then, in the register of Part 1, isolates its cost: the translation itself is cached, in a structure called the TLB, and when that cache misses, the machine pays a price that scales with how scattered your data is. The through-line from Chapter 3 is exact — **the TLB is a cache for the page table**, and everything you learned about hits, misses, and working sets applies to it too.

4.1 Virtual vs. physical addresses: why the lie exists

On a modern system, each process runs as if it owns a large, contiguous block of memory starting near address zero, all to itself. It does not. Physical DRAM is shared among every process and the kernel, is smaller than the address space the program is allowed to name, and is handed out in scattered fragments. The **virtual address space** is the illusion that papers over this reality: the hardware's memory-management unit (MMU), configured by the OS, maps each process's virtual addresses onto whatever physical frames happen to be free, and keeps different processes' identical-looking virtual addresses pointing at different physical memory. Bryant and O'Hallaron develop this fully in *Computer Systems: A Programmer's Perspective* (CS:APP, Chapter 9, "Virtual Memory"); Arpaci-Dusseau's *Operating Systems: Three Easy Pieces* (OSTEP) devotes its whole virtualization part to it.

The illusion buys three things at once, which is why every general-purpose OS pays for it. First, **isolation**: process A literally cannot name a byte belonging to process B, because A's page tables contain no mapping to B's frames — a security and stability boundary enforced in hardware. Second, **a simple programming model**: your linker can lay out code and data at fixed virtual addresses without knowing where in physical RAM the program will actually land, and every process gets the same clean layout. Third, **flexibility the OS exploits**: a virtual page can be mapped to a physical frame, or to a page of a file on disk, or to nothing yet (allocated lazily on first touch), or shared read-only between processes (as with a common library) — all invisibly to your code. For this chapter, hold onto the first fact and the cost it implies: because the mapping is per-process and arbitrary, *the hardware must consult a translation on essentially every memory access*, and that lookup is what we now follow.

4.2 Pages and page tables

Translating every individual byte address would need a map with one entry per byte — absurd. Instead memory is divided into fixed-size **pages**: the address space is chopped into aligned blocks, and translation happens one page at a time. The near-universal base page size on x86-64 (and many other architectures) is **4 KiB**; CS:APP uses this standard size, and it is what `getconf PAGE_SIZE` reports on such systems. A virtual address then splits cleanly into two parts: the high bits are the **virtual page number** (which page), and the low bits — 12 of them for a 4 KiB page, since $2^{12} = 4096$ — are the **offset** within the page. Only the page number needs translating; the offset is the same in virtual and physical addresses, because a page maps to a physical **frame** of the same size and a byte's position within the page does not move.

The **page table** is the data structure, held in memory and owned by the OS per process, that maps virtual page numbers to physical frame numbers (plus permission and status bits: present or not, readable/writable/executable, accessed, dirty). Conceptually it is one big array indexed by virtual page number. But a flat array is impractical: a 48-bit virtual address space with 4 KiB pages has 2^{36} pages, and a flat table with an entry per page would be enormous *per process*, the overwhelming majority of it empty because most programs use a tiny, sparse fraction of their address space. The standard fix is a **multi-level (hierarchical) page table**: a tree of tables where the virtual page number is split into several fields, each indexing one level. x86-64 uses a four-level page table (a documented architectural fact, described in CS:APP). Only the branches that correspond to actually-used regions of memory need to exist, so a sparse address space costs a sparse tree instead of a giant flat array.

The cost that matters shows up right here. To translate one virtual page number through a four-level table, the hardware must read *each level in turn*: read the top-level table to find the address of the next-level table, read that to find the next, and so on — up to **four dependent memory accesses** to complete one translation on x86-64. This descent is called a **page-table walk**. And notice it is a dependency chain of the exact kind Chapter 3 warned about: each read's address comes from the previous read, so the walk cannot be parallelized, and if the table levels are not in cache, each step is its own trip toward

DRAM. Doing four dependent memory accesses *for every* load and store would erase all the performance we have been chasing — so, as always, the machine caches the result.

> **QUICK CHECK**
>
> A 4 KiB page needs how many offset bits, and why exactly that many? If a virtual address is split into a page number and an offset, which part does the page table translate, and which part is copied unchanged into the physical address? (Answer after the next section.)

4.3 The TLB: a cache for translations

The **translation lookaside buffer (TLB)** is a small, fast, on-CPU cache that stores recent virtual-page-to-physical-frame translations. It is a cache in exactly the sense of Chapter 3 — same idea, different contents. Chapter 3's data caches store recently used *cache lines* so a repeated data access is a cheap hit; the TLB stores recently used *translations* so a repeated access to the same page skips the page-table walk. On each memory access the MMU first checks the TLB with the virtual page number:

- A **TLB hit** — the translation is present — returns the physical frame in roughly a cycle, effectively free, and the access proceeds. This is the common case, because programs have locality: consecutive accesses usually fall on the same few pages (a 4 KiB page holds a lot of data), so one page's translation, cached once, serves thousands of accesses.
- A **TLB miss** — the translation is not cached — triggers the **page-table walk** of §4.2: the several dependent memory accesses to descend the table, after which the freshly found translation is installed in the TLB (evicting an older one) so nearby future accesses hit. On modern x86-64 this walk is done by hardware; on some architectures the OS handles it in software, which is costlier still.

So the cost of a TLB miss is the cost of the walk: on the order of a handful of dependent memory accesses. If those accesses hit in the data caches (the upper page-table levels are hot and small, and are cached), a miss is cheap — a few cycles. If they miss to DRAM, each is a ~100 ns trip (Chapter 1), and a walk can cost on the order of tens to hundreds of nanoseconds. The exact penalty and the exact TLB capacity are hardware parameters that vary by CPU, and this book will not quote a specific chip's numbers as if they were universal — CS:APP (Chapter 9) gives a worked example TLB and walk, and Drepper's memory paper covers TLB behavior in depth. What you must carry is the *shape*: a TLB hit is ~free, a TLB miss costs a page-table walk whose price is one to a few memory accesses and, in the bad case, is a DRAM-latency event stacked *on top of* whatever the data access itself will cost.

(Answer to the quick check: 12 offset bits, because 2^{12} = 4096 = 4 KiB, so 12 bits address every byte within a page. The page table translates the page-number bits; the offset is copied unchanged into the physical address, since a page and its frame are the same size and a byte keeps its position within the page.)

THE SAME IDEA THREE WAYS: THE TLB IS A CACHE FOR THE PAGE TABLE

1. In words. Translating an address needs a multi-step page-table walk. Rather than walk on every access, the CPU caches recent translations in the TLB; a hit skips the walk, a miss pays for it and caches the result — identical logic to a data cache, applied to translations instead of data.

2. As a picture. Figure 4.1: two parallel lookups. The data cache maps *address → line*; the TLB maps *virtual page → physical frame*. Same hit/miss structure, different key and value.

3. As a trace. Access page P: TLB miss → walk the 4-level table (up to 4 memory reads) → install P→frame in the TLB. Next 1000 accesses to page P: TLB hits, ~free. The walk's cost is amortized over every access that stays on the page — which is why sequential code barely notices the TLB and scattered code does.

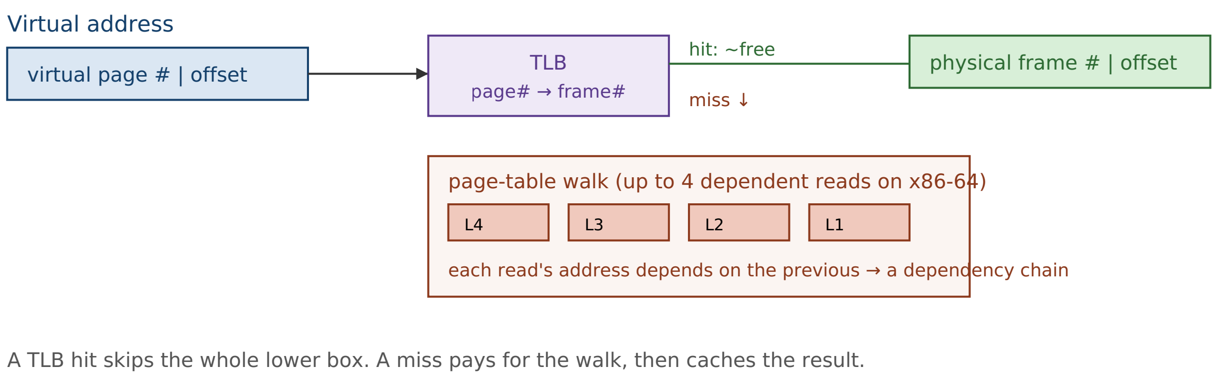


Figure 4.1: Address translation. The TLB caches virtual-page→physical-frame mappings; a hit is near-free, a miss triggers a multi-level page-table walk (a dependency chain of memory reads) whose result is then cached.

4.4 Why huge, scattered working sets thrash the TLB

Here is the performance consequence, and it is a new failure mode not covered by Chapter 3's data caches. The TLB holds only a limited number of translations — a hardware-fixed count, small relative to the number of pages a large program can touch. The total span of memory the TLB can map at once is its **reach**: roughly (number of entries) × (page size). With 4 KiB pages, even a few thousand TLB entries reach only a few megabytes of address space at a time. So a program whose hot data spans *hundreds* of megabytes, accessed in a scattered pattern that jumps from page to page, will keep missing the TLB: each jump lands on a page whose translation was long since evicted, forcing a fresh page-table walk. This is **TLB thrashing**, and it is precisely a *capacity miss* (Chapter 3's taxonomy) at the translation level — the working set of *pages* exceeds what the TLB can hold, just as a data working set can exceed the L2.

Notice the two costs now stack. A scattered access over a huge array can miss the *data* cache (pay ~100 ns to fetch the line) *and* miss the *TLB* (pay a page-table walk to even learn where the line is), and in the worst case the walk itself misses cache and goes to DRAM. This is a large part of why truly random access over a multi-hundred-megabyte structure is so much slower than the raw ~100 ns DRAM figure alone would predict — a fact the Chapter 6 capstone lab measures directly, where a random pointer-chase over 512 MiB costs well above the bare DRAM latency, with the surplus owing partly to exactly this page-walk cost. By contrast, *sequential* access barely touches the TLB: a 4 KiB page holds ~512 `long`s, so one translation serves hundreds of consecutive accesses before you cross into the next page — the same "use the whole thing you paid to bring in" principle as the cache line, one level up.

> **QUICK CHECK**
>
> Two loops touch the same 512 MiB array the same number of times: one walks it sequentially, one jumps to random pages. Which one thrashes the TLB, and why does the other one hardly use it at all? (Answer below the huge-pages section.)

4.5 Huge pages, conceptually

If TLB reach is (entries) × (page size) and you cannot easily add TLB entries (that is fixed hardware), the other lever is the page size. **Huge pages** do exactly that: instead of mapping memory in 4 KiB units, the OS can map it in much larger units — on x86-64, commonly **2 MiB**, and **1 GiB** where supported (both are real, documented x86-64 page sizes; the `pdpe1gb` CPU flag advertises 1 GiB support). One huge-page translation covers 2 MiB instead of 4 KiB, so the *same* number of TLB entries now reaches roughly 512× more memory. A working set that thrashed the TLB with 4 KiB pages may fit entirely within the TLB's reach once mapped with 2 MiB pages, turning a storm of page-table walks into TLB hits. A second, smaller benefit: a huge page's walk descends fewer levels (the translation stops at a higher level of the tree), so even the misses are a bit cheaper.

Huge pages are a real mitigation for real workloads — large in-memory databases, big hash tables, scientific arrays — where TLB misses are a measured bottleneck. They are not free lunch: they can waste memory to internal fragmentation (a 2 MiB page allocated for a few live kilobytes), can be harder for the OS to find contiguously once memory is fragmented, and help *only* when TLB misses were actually a problem. That last clause is the discipline of this whole Part: huge pages are something you reach for *after* a profiler shows page-walk cost is significant, not a switch you flip on faith. This book keeps the treatment conceptual — the mechanics of enabling them (explicit `mmap` flags vs. transparent huge pages) belong to the operating-system Part — but the model is what matters here: *bigger pages, fewer translations, more TLB reach.*

(Answer to the §4.4 quick check: the random-page jumper thrashes the TLB — each jump lands on a different page whose translation has been evicted, forcing a walk. The sequential walker hardly uses the TLB because each 4 KiB page's single translation serves the hundreds of consecutive accesses that fall on that page before it crosses to the next one.)

TRAP: IGNORING TLB COST (THE "IT'S ALL JUST DRAM LATENCY" FALLACY)

Having learned Chapter 3, it is tempting to model every scattered access as a single ~100 ns DRAM miss and stop there. That undercounts. A random access over a large working set can pay *two* independent penalties: the data-cache miss to fetch the line, *and* a TLB miss whose page-table walk is itself one to several memory accesses. When your measured cost per random access sits well above the raw DRAM latency you expected, suspect the TLB — especially if the working set spans far more memory than TLB reach (a few thousand pages × 4 KiB). The tell is a hot, random-access structure of hundreds of megabytes or more. The fixes are the ones this chapter and the last give: prefer sequential or blocked access so one translation serves many accesses (the sequential-vs-random gap is partly a TLB gap); shrink the working set; and, where a profiler confirms page-walk cost, consider huge pages. As always: confirm by measuring page-walk and TLB-miss counters, not by guessing.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our random lookup into the big in-memory index costs about 160 ns per access, but you told me DRAM is only ~100 ns — where does the extra time come from?" Cover the paragraph below and answer in two or three sentences, using *TLB*, *page-table walk*, and *reach*.

Model answer. "The ~100 ns is just the data fetch. Each random lookup also needs an address translation, and because the index spans far more memory than the TLB can map at once (its reach is only a few megabytes with 4 KiB pages), most lookups miss the TLB and trigger a page-table walk — several dependent memory accesses — *before* the data access even starts. That walk cost stacks on top of the DRAM latency, which is where the surplus comes from. If a profiler confirms it, mapping the index with huge pages would raise TLB reach enough to turn most of those walks back into hits." Naming the TLB and page walk — rather than "memory is just slow" — is the senior register, and it points at a concrete, measurable fix.

4.6 What to carry forward

This chapter added the last piece of the machine's cost model that Part 1 needs. The addresses your program uses are virtual, translated to physical one page at a time through a multi-level page table; the translation is cached in the TLB, so it is near-free when your accesses have page-level locality and expensive — a page-table walk of several dependent memory accesses — when they do not. A huge, scattered working set thrashes the TLB the same way a large data working set overflows the L2, and huge pages widen TLB reach as a conceptual mitigation. The unifying idea, straight from Chapter 3, is that *the TLB is one more cache*, subject to the same hit/miss/working-set logic as every other level — which is why the single habit of accessing memory sequentially or in blocks pays off twice: once in the data cache, once in the TLB.

Part 1 now has the whole machine in view: the CPU that runs your instructions (Chapter 2), the cache hierarchy that feeds it data (Chapter 3), and the translation layer that turns your addresses into real memory (this chapter). Chapter 5 spends that knowledge: it is where you deliberately *design* your data's layout — array-of- structs versus struct-of-arrays, hot/cold splitting, alignment — so that both the cache and the TLB work *for* you. Then Chapter 6 puts numbers on all of it in a capstone lab. The posture to keep: when memory is slow, ask not only "did the data miss the cache?" but also "did the *address* miss the TLB?"

CAN YOU… WITHOUT LOOKING?

- Explain the difference between a **virtual** and a **physical** address, and name the three things virtual memory buys (isolation, a simple layout, OS flexibility).
- Split a virtual address into **page number** and **offset**, say why a 4 KiB page has a 12-bit offset, and explain why page tables are **multi-level**.
- State what a **page-table walk** is, why it is a dependency chain, and roughly how many memory accesses it can take on x86-64 (up to four).
- Explain why the **TLB is a cache for the page table**, what a hit and a miss each cost (in shape, not a specific chip's numbers), and how it maps onto Chapter 3's hit/miss model.
- Define **TLB reach** and explain why a huge, scattered working set **thrashes** the TLB while sequential access barely uses it.
- Explain how **huge pages** raise TLB reach, and name a cost of using them.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is the difference between a virtual and a physical address, and why does a general-purpose OS use virtual memory at all? (§4.1)
2. **R2.** Why are page tables multi-level rather than a single flat array, and what is a page-table walk? (§4.2)
3. **R3.** In what precise sense is the TLB "a cache for the page table"? What does a hit cost and what does a miss cost, in shape? (§4.3)
4. **R4.** Define TLB reach and explain TLB thrashing using Chapter 3's miss taxonomy. Which of the three C's is it? (§4.4)
5. **R5.** How do huge pages reduce TLB misses, and when are they *not* worth it? (§4.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 9 ("Virtual Memory"). The source for this chapter's address-translation mechanism, multi-level page tables, TLB, and a worked example of a walk. The free CMU 15-213 materials cover it.
- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)**, the virtualization part (free full text). Paging, TLBs, and page-table structures from the OS side — the complement to CS:APP's hardware view, and where the operating-system Part of this book will pick up.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free). Includes TLB behavior, the cost of misses, and huge pages. Absolute numbers are dated — read it for mechanism.

Exercises

4.1 WARM-UP A system uses 4 KiB pages. (a) How many bits of a virtual address are the page offset? (b) If you access `a[0]` then `a[1]` (adjacent `long`s), do they need one translation or two? (c) Roughly how many consecutive `long`s can you touch before you cross a page boundary and (possibly) need a new translation?

4.2 WARM-UP In one sentence each, name the three benefits virtual memory provides, and state which one is the security/isolation boundary between processes.

4.3 WARM-UP Complete the analogy and justify it in a sentence: a data cache stores recently used ____ so a repeated data access is a cheap hit; the TLB stores recently used ____ so a repeated access to the same ____ skips the ____.

4.4 CORE Explain why a page-table walk is a *dependency chain* and why that matters for its cost. Tie it explicitly to the pointer-chasing argument from Chapter 3: what do a page-table walk and a linked-list traversal have in common?

4.5 CORE A hot in-memory hash table of 400 MiB is probed at random keys. Measurements show ~150 ns per probe, noticeably more than the ~100 ns you expected for a single DRAM miss. (a) Give two independent memory-system costs a random probe pays. (b) Explain, using *TLB reach*, why this structure is prone to TLB misses. (c) Name one change that would reduce the translation cost specifically, and one that would reduce the data-miss cost.

4.6 COST A TLB has E entries and the system uses 4 KiB pages. (a) Write the TLB reach as a formula. (b) For a working set of W bytes accessed randomly by page, under what condition (in terms of W, E, and page size) do you expect heavy TLB thrashing? (c) If you switch to 2 MiB pages without changing E, by what factor does reach change, and how does that move the threshold in (b)?

4.7 FADED *Worked, with the last step for you.* Explain why sequential access over a 512 MiB array barely stresses the TLB while random-by-page access over the same array thrashes it. Steps 1–3 given; complete step 4.
Step 1 (given): Translation happens per page (4 KiB); a page of `long`s holds ~512 elements.
Step 2 (given): The TLB reaches only (entries × 4 KiB) of address space at once — a few megabytes, far less than 512 MiB.
Step 3 (given): Sequential access stays on one page for ~512 consecutive accesses before crossing to the next, so one translation is amortized over many accesses.
Step 4 (you): Complete the argument for the random case: state how often the random walker needs a *new, uncached* translation, why the TLB's small reach means those translations are usually missing, and therefore why nearly every random access pays a page-table walk. Then state the one number you would measure to confirm the TLB — not just the data cache — is the culprit.

4.8 MIXED For each symptom, decide whether the primary cause is most likely a *data-cache capacity miss* (Ch 3), a *TLB miss / page-walk* (Ch 4), a *branch misprediction* (Ch 2), or a *dependency chain / low ILP* (Ch 2), and justify in a line: (a) a random-access loop over 800 MiB is slower than ~100 ns/access predicts, and enabling 2 MiB pages speeds it up; (b) a loop over 64 MiB re-read many times is much slower than one over 1 MiB re-read the same total number of times; (c) a sum over a linked list is far slower than over an array of the same length; (d) a tight loop with a data-dependent `if` speeds up sharply after the input is sorted.

4.9 MIXED A colleague proposes enabling huge pages everywhere "to make memory faster." Without running anything, give (a) the specific condition under which huge pages actually help, (b) two costs or downsides of huge pages, and (c) the one measurement you would take first to decide whether they are worth it for a given workload. Note which chapter's discipline this reflects.

4.10 STRETCH The chapter says the TLB "is a capacity miss at the translation level." (a) Map each of Chapter 3's three C's onto a translation-level analogue if one exists (compulsory, capacity, conflict), or say why it does not. (b) Explain why huge pages address the *capacity* analogue specifically. (c) Argue why increasing the base page size is not a free win at the system level (think of programs with small, sparse memory footprints).

4.11 LAB On your own machine, record the base page size (`getconf PAGE_SIZE`) and confirm your architecture's page-table depth and huge-page sizes from its documentation (do *not* guess — cite what you find). Then write a program that sums the same large array (comfortably larger than your last- level cache, e.g. 512 MiB) two ways: strictly sequentially, and in a random-by-page order (jump to a random page, read a few values, jump again). Time both and report *your* ns-per-access for each. If your platform exposes TLB-miss or page-walk counters (e.g. `perf stat -e dTLB-load-misses` on Linux), record them for both runs and check that the random run has far more. Report only numbers you actually measured, and note anything that surprised you.

Chapter 5 — Mechanical Sympathy & Data-Oriented Design

Before you start: Chapter 3 (cache lines, locality, the three C's), Chapter 4 (the TLB), Chapter 1 (mechanical sympathy as this volume's thesis). · **Pairs with:** Chapter 6 (the capstone lab) and the performance-engineering Part. This is where the last four chapters stop being knowledge *about* the machine and become a way of *designing your data*.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 3 — memory moves in 64-byte lines, so the cost of an access is really "per line fetched." If a loop uses only 8 of the 64 bytes in each line it brings in, roughly what fraction of the memory bandwidth it pays for is wasted? (2) Chapter 1 named this volume's thesis after a driving metaphor — what is the two-word term, and who is it attributed to? (3) *Predict:* you have 16 million "particles," each a struct with 8 fields, and you need to sum just *one* field across all of them. Does it matter for speed whether you store them as one array of whole structs, or as eight separate arrays — one per field? By the end you will have a measured answer.

Chapters 1 through 4 were reconnaissance: the CPU, the cache hierarchy, and the translation layer, each with its own cost model that the RAM abstraction hides. This chapter spends that reconnaissance. The term **mechanical sympathy** — popularized in software by Martin Thompson, and introduced back in Chapter 1 — names the posture: understand enough about how the machine works to get the most out of it. **Data-oriented design** is that posture applied to one specific and enormously consequential decision — *how you lay your data out in memory* — and its central claim, which the rest of this chapter defends, is blunt: *the fastest code respects the memory system.* You do not earn an order of magnitude here by a cleverer algorithm; you earn it by arranging the same data so the cache lines you fetch are full of bytes you actually use and the pages you touch are few. This chapter turns Chapter 3's rules into design choices — array-of-structs versus struct-of-arrays, splitting hot fields from cold, padding and alignment — and, as promised, measures one of them on real hardware.

5.1 The design question: lay data out for the access pattern

Data-oriented design inverts the habit most of us learned first. Object-oriented instinct groups data by *conceptual entity*: a `Particle` "has" a position, a velocity, a mass, so we put them in one struct and make an array of those. That models the domain cleanly — and it is often exactly wrong for performance, because *the hardware does not care about your conceptual entities; it cares about which bytes travel together in a cache line.* The design question data-oriented design asks first is not "what is the natural object?" but "**what is the hot loop's access pattern, and which bytes does it actually touch together?**" You then lay memory out to match that pattern, so every fetched line and every translated page is spent on bytes the loop needs. Everything in this chapter is a special case of that one move.

5.2 Array-of-structs vs. struct-of-arrays

The canonical layout choice. Suppose each record has several fields and you have many records. There are two natural layouts:

- **Array-of-structs (AoS):** one array, each element a full struct — all of record 0's fields, then all of record 1's, and so on. Fields of the *same record* are adjacent in memory.
- **Struct-of-arrays (SoA):** one array *per field* — every record's `x` in one contiguous array, every `y` in another, and so on. The same field across *different records* is adjacent in memory.

Which wins depends entirely on the access pattern, and neither is universally better — this is a genuine trade-off, not a rule. The deciding question is *which bytes your hot loop touches together*:

- If the hot loop **scans one field (or a few) across many records** — sum every `x`, filter by one attribute, do SIMD over one channel — **SoA wins**. In SoA those values are contiguous, so each 64-byte line is full of the values you want and the traversal is stride-1 with perfect spatial locality (§3.1–3.2). In AoS the same scan strides over the whole struct, touching a few bytes per line and wasting the rest — the column-major mistake of §3.6, in a new costume.
- If the hot loop **uses most fields of one record at a time** — process particle *i* completely, then particle *i*+1 — **AoS wins**. All of record *i*'s fields share a line or two, so one fetch brings the whole record; SoA would instead touch one element in each of many separate arrays, scattering the access across many lines and pages.

A measured demonstration: sum one field of 16 million records

Let us measure the SoA-favoring case. Define a record of 8 `double`s — position, velocity, mass, charge — which is exactly **64 bytes**, one cache line, in AoS. Make 16 million of them and sum a single field, `x`, two ways: over the array-of-structs (each step strides 64 bytes to the next record's `x`, using 8 of every 64 bytes fetched) and over a contiguous per-field array (stride-1, every byte of every line used).

```
struct Particle { double x, y, z, vx, vy, vz, mass, charge; };  /* 64 bytes = 1 line */

/* AoS: sum one field -> stride of 64 bytes, 8 of 64 bytes used per line */
for (size_t i = 0; i < N; i++) sum += aos[i].x;

/* SoA: the x field is its own contiguous array -> stride 1, whole line used */
for (size_t i = 0; i < N; i++) sum += soa_x[i];
```

Built with `gcc -O2` and run for this book on a 12th-generation Intel Core i7 (i7-12700H), over 16 million records (best of 5 timed runs), the AoS scan of one field took roughly **0.12 s** and the SoA scan roughly **0.032 s** — the SoA layout was about **3.7–4× faster** for computing the identical sum, differing only in how the data was laid out. **This exact ratio is machine-dependent** and code-dependent, and you should expect a different number on your hardware. Note it is a clear multiple, but *less* than the naive 8× you might predict from "AoS moves 8× the memory": the SoA loop is not purely bandwidth-bound — summing into a single accumulator has a dependency chain (§2.3) that caps its speed, so it cannot fully cash in the bandwidth it saves, which compresses the ratio. Remove that ceiling (multiple accumulators, vectorization) and the gap widens toward the bandwidth-bound 8×. The robust, portable result is the *direction and rough magnitude*: laying the scanned field out contiguously is a large, several-fold win, for the same arithmetic — because SoA turns a one-value-per-line scan into a whole-line scan.

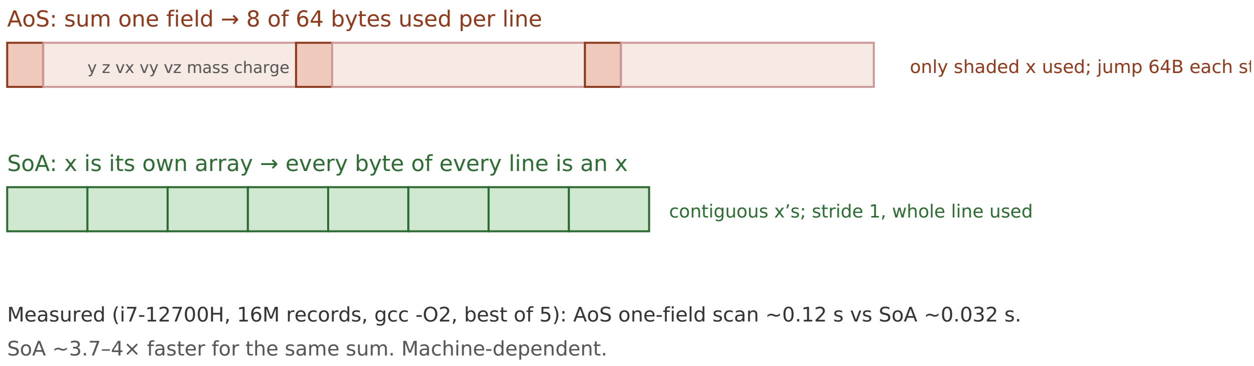


Figure 5.1: Summing one field. AoS strides over whole records, using a fraction of each fetched line; SoA stores that field contiguously, so every fetched line is full of values the loop uses.

QUICK CHECK

You process each record fully — read all 8 fields of particle i, do the physics, then move to particle $i+1$ — for every record. Which layout, AoS or SoA, is better *here*, and why does the answer flip from the sum-one-field case? (Answer after the next section.)

5.3 Hot/cold field splitting

AoS-vs-SoA is the all-or-nothing version; hot/cold splitting is the surgical one. Often a record has a few fields touched by the hot loop (the **hot** fields) and several touched rarely — logging metadata, debug info, a rarely-read description (the **cold** fields). If they all live in one struct, every cache line the hot loop fetches is padded out with cold bytes it will not use, lowering useful-bytes-per-line exactly as the AoS one-field scan did. **Hot/cold splitting** divides the record: put the hot fields in one compact struct (so more hot records fit per line and per page) and move the cold fields to a separate structure, referenced by index or pointer, touched only on the rare path. The hot loop now streams through dense hot records; the cold data is out of its way. This is the same principle as SoA — group by access frequency instead of by conceptual entity — applied at the granularity of "hot vs. cold" rather than "field by field." The tell that you want it: a struct where a hot loop reads two of twelve fields, and the other ten are dead weight in every line.

(Answer to the §5.2 quick check: ***AoS wins*** *when you use most fields of one record at a time. All of particle i's fields are adjacent, so one or two line fetches bring the whole record and every byte is used; SoA would scatter that record's fields across eight separate arrays — eight different lines and possibly pages per record. The answer flips*

because the access pattern flipped: SoA is contiguous across records for one field; AoS is contiguous across fields for one record. Match the layout to whichever direction your hot loop travels.)

5.4 Padding and alignment

Two low-level facts about how the compiler places your struct in memory, both from the C model (CS:APP, Chapter 3 covers data alignment). First, **alignment**: each type has an alignment requirement — an 8-byte `double` is typically placed at an 8-byte-aligned address — because aligned accesses are efficient and, on some architectures, misaligned ones are slow or forbidden. To honor this inside a struct, the compiler inserts **padding**: unused bytes between fields so each field lands at a legal offset, plus tail padding so an array of the struct keeps every element aligned. The consequence for layout: field *order* changes struct size. A struct declared `{ char a; double b; char c; }` can be substantially larger than its 10 bytes of real data because padding is inserted around the fields; reordering to put the large, strictly-aligned fields first and pack the small ones together can shrink it. A smaller struct means more records per cache line and per page — a direct locality win, for free, from field ordering alone.

Second, alignment interacts with the cache line. A value that straddles a 64-byte line boundary can require touching *two* lines to read one value; keeping hot structures aligned to line boundaries (and sized so they tile the line cleanly, as our 64-byte `Particle` did) avoids that. There is a tension worth naming: padding a struct *up* to a nice size can help alignment and, as we will see next, avoid a concurrency hazard — but padding also makes the struct *bigger*, which hurts the locality of a scan. Like everything in this Part, it is a trade-off resolved by knowing the access pattern and, when it matters, measuring.

> **THE SAME IDEA THREE WAYS: GROUP DATA BY HOW IT'S ACCESSED**
>
> **1. In words.** The hardware moves and translates memory in fixed blocks (lines, pages). Lay data out so the bytes a hot loop uses together are stored together — by field (SoA), by temperature (hot/cold), and by alignment — so every block you pay for is full of bytes you need.
>
> **2. As a picture.** Figure 5.1: an AoS line mostly wasted on unused fields vs. an SoA line entirely full of the scanned field.
>
> **3. As a measurement.** Summing one field of 16 M records: SoA ~0.032 s vs. AoS ~0.12 s on the test machine — ~3.7–4× for the identical arithmetic, purely from layout. Prose says "group by access pattern"; the number says the grouping is worth several-fold.

5.5 A preview: false sharing

One layout hazard belongs to the concurrency Part but is worth planting now, because it is the mirror image of everything above. So far, packing related data into the same cache line has been *good* — it raises useful-bytes-per-line. But when *multiple threads* write to *different* variables that happen to share one cache line, the line ping-pongs between the cores' caches: each write forces the line out of the other core's cache to maintain coherence, even though the threads touch logically independent data. This is **false**

sharing — no real data is shared, but the cache line is, and the hardware's coherence protocol pays for it. The fix runs *opposite* to the locality advice of this chapter: pad or align the per-thread variables so each lands on its *own* cache line, deliberately *spreading* them out. That single-threaded packing helps and multi-threaded sharing hurts is not a contradiction — it is the same fact (a whole line moves as a unit) cutting two ways depending on whether one core or several are writing it. The full treatment, with a measurement, waits for the concurrency Part; for now, file it: *when you add threads, revisit which variables share a line.*

> **TRAP: AOS WHEN YOU SCAN ONE FIELD (PACKING THE WRONG AXIS)**
>
> The recurring layout error: choosing a struct layout to model the domain cleanly, then running a hot loop that touches only one or two fields of each record — so every cache line and every page you fetch is mostly bytes the loop ignores. It is the column-major mistake (§3.6) wearing an object-oriented costume, and it silently caps a scan at a fraction of its potential bandwidth. The tells: a hot loop over a large array of multi-field structs that reads only a couple of fields; a SIMD kernel that wants one channel contiguous but finds it strided; a profiler showing high cache-miss rate on a sequential-looking loop. The fixes are the chapter's: switch the scanned field(s) to **SoA**, or **split hot from cold** so the hot loop streams dense records; and reorder fields to shrink padding. And the inverse trap for later: do *not* reflexively pack everything tight once threads are involved — that invites false sharing (§5.5). As always, confirm the layout change with a measurement on your data and machine — the ~4× here is one machine's number, not a law.

> **SAY IT LIKE A SYSTEMS ENGINEER**
>
> *Try it first.* In review, someone asks why you "denormalized a clean `Particle` struct into a bunch of parallel arrays — it's uglier and the objects made more sense." Cover the paragraph and answer in two or three sentences, using *struct-of-arrays*, *cache line*, and *access pattern*.
>
> *Model answer.* "The hot loop scans a single field across all records, so with the struct layout each 64-byte line we fetched held one useful value and seven fields the loop never reads — we were paying for eight times the bandwidth we used. Splitting that field into its own contiguous array (struct-of- arrays) makes the scan stride-1, so every fetched line is full of values we sum, and it measured several times faster for the identical result. If a different loop needed whole records at once I'd keep them as structs — the layout follows the access pattern, not the domain model." Naming the access pattern and useful-bytes-per-line — rather than "arrays are faster" — is the senior register, and it makes the trade-off explicit so the reviewer can check it against the *other* loops.

5.6 The rule, and where Part 1 lands

Five chapters of machine knowledge reduce, at the design level, to one instruction: **lay your data out for the way the hot loop touches it.** Memory moves in cache lines and is translated in pages (Chapters 3–4), so the layout that keeps each fetched line and each touched page full of bytes the loop actually uses wins — array-of-structs when you use whole records, struct-of- arrays when you scan a field, hot/cold splitting when a record is mostly cold, tight field ordering to shrink padding, and (later, under threads) deliberate spreading to avoid false sharing. None of this changes an algorithm's big-O; all of it can change its speed by a large factor, as the AoS/SoA measurement showed. That is why "the fastest code respects the memory system" is not a slogan but a design discipline — the practical cash value of mechanical sympathy.

Part 1 is almost complete. You now have the machine (CPU, caches, TLB) and the design posture (data-oriented layout) to exploit it. Chapter 6 closes the Part with the skill that ties it together: **back-of-the-envelope cost estimation** — using the orders of magnitude from Chapters 1–5 to *predict* whether a workload is compute-, memory-, or I/O-bound, then measuring to confirm. It is where "respect the memory system" becomes a number you can estimate before you write the code — and the bridge into Part 2, where you take manual control of that memory in C.

CAN YOU… WITHOUT LOOKING?

- State the central question of **data-oriented design** (what does the hot loop touch together?) and connect **mechanical sympathy** (Martin Thompson) to it.
- Define **AoS** and **SoA** and say which wins for (a) scanning one field across records and (b) using whole records one at a time — and *why* each.
- Explain **hot/cold splitting** and the tell that you want it.
- Explain **padding and alignment**: why field order changes struct size, and why a smaller struct improves locality.
- Describe **false sharing** in one sentence and why its fix (spread data out) is the *opposite* of single-threaded packing.
- Recall the AoS/SoA **measured result** (~3.7–4× on the test machine), *why* it was less than the naive 8×, and why the exact number is machine-dependent.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What single question does data-oriented design ask before choosing a layout, and why is "what is the natural object?" the wrong one for performance? (§5.1)
2. **R2.** Define AoS and SoA and give the access pattern that favors each. (§5.2)
3. **R3.** What is hot/cold field splitting, and how does it raise useful-bytes-per-line? (§5.3)
4. **R4.** Why does reordering a struct's fields change its size, and why can that be a locality win? (§5.4)
5. **R5.** What is false sharing, and why is its fix the opposite of the packing advice in the rest of the chapter? (§5.5)

FURTHER READING

- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free). The deepest free treatment of why layout and access pattern dominate memory performance; its sections on data structures and access patterns are the direct background for this chapter. Read for mechanism; its absolute numbers are dated.
- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). Lectures on cache-efficient data structures, data layout, and measuring the effect — the applied continuation of this chapter, and where the performance-engineering Part picks up.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 3 (data alignment and struct layout) and Chapter 6 (locality). The reference for how the compiler pads and aligns structs, and why alignment matters.

Exercises

5.1 WARM-UP Define array-of-structs and struct-of-arrays in one sentence each, and state which stores "the same field across different records" contiguously.

5.2 WARM-UP A record is 8 `double`s (64 bytes). You sum one field across many records in AoS. (a) How many bytes of each 64-byte line does the sum use? (b) Roughly how many times more memory does the AoS one-field scan move than an SoA scan of the same field? (c) Which kind of locality (§3.2) does SoA restore?

5.3 WARM-UP Give the tell — in one sentence — that a struct is a candidate for hot/cold field splitting.

5.4 CORE For each workload, choose AoS or SoA and justify in terms of the access pattern and cache lines: (a) a physics step that reads and writes all fields of one particle before moving to the next; (b) a query that sums one numeric column over ten million rows; (c) a SIMD kernel that multiplies every record's `x` by a scalar; (d) rendering that, per object, reads its position, color, and size together.

5.5 CORE A struct is declared `struct S { char flag; double value; int id; };`. (a) Explain why its size is larger than 1+8+4 = 13 bytes. (b) Reorder the fields to minimize padding and explain your ordering. (c) State the locality benefit of the smaller struct when you have an array of ten million of them.

5.6 COST You scan one 8-byte field of records that are S bytes each, laid out AoS, over a large array. (a) In terms of S and the 64-byte line, how many useful bytes per fetched line does the scan use, and what fraction is that? (b) As S grows, what happens to the AoS/SoA bandwidth ratio for this scan? (c) Why might the *measured* speedup be smaller than this bandwidth ratio predicts (name one reason from Chapter 2 and relate it to why this chapter's demo measured ~4×, not 8×)?

5.7 FADED *Worked, with the last step for you.* Argue which layout to choose for a mixed workload that has *two* hot loops: loop A sums one field over all records; loop B updates all fields of each record in turn. Steps 1–3 given; complete step 4.
Step 1 (given): Loop A (scan one field) favors SoA — contiguous field, whole lines used.
Step 2 (given): Loop B (whole record at a time) favors AoS — one record's fields adjacent, one fetch per record.
Step 3 (given): A single layout cannot be optimal for both; the choice depends on which loop dominates the runtime and by how much.
Step 4 (you): State how you would *decide*: what you would measure to learn which loop dominates, how hot/cold splitting or a hybrid layout might serve both partly, and why "pick the layout that helps the hotter loop, then re-measure" is the right posture rather than reasoning it out on paper. Name the discipline this reflects.

5.8 MIXED For each speedup, decide whether the primary mechanism is *SoA / whole-line use* (Ch 5), *hot/cold splitting* (Ch 5), a *TLB / huge-page effect* (Ch 4), a *capacity miss fixed by blocking* (Ch 3), or *ILP / multiple accumulators* (Ch 2), and justify in a line: (a) moving a rarely-read description field out of a hot record; (b) summing a column after switching from row-structs to column arrays; (c) tiling a matrix multiply; (d) enabling 2 MiB pages for a random-access index; (e) summing into four accumulators instead of one.

5.9 MIXED Two versions of the same aggregation over ten million records differ by 5× in wall time; both are O(n). Without running anything, list at least three distinct candidate causes drawn from Chapters 2–5 (include at least one layout cause and one from an earlier chapter), then name the single measurement you would take first and why it best narrows the field.

5.10 STRETCH This chapter both tells you to *pack* related data into shared lines (SoA, hot/cold, tight padding) and warns that packing per-thread data into a shared line causes *false sharing*. (a) Explain why these are not contradictory — reduce both to the one fact about cache lines that drives them. (b) Give a single scenario where you would deliberately pad a struct *up* to a full cache line, and say what you are trading away. (c) Why does the right choice depend on how many cores write the data?

5.11 LAB Reproduce the AoS/SoA demonstration on your own machine. Define a record of several fields (size it to a whole number of your cache line, e.g. 64 bytes), make enough records to comfortably exceed your last-level cache, and sum one field two ways: over the array-of-structs and over a contiguous per-field array. Verify both produce the identical sum, time each (best of several runs), and report *your* ratio. Then try to widen it toward the bandwidth ceiling by removing the accumulator dependency (several accumulators, or let the compiler vectorize) and report what changed. Report only numbers you actually measured; if a loop is optimized away, consume the sum and say what you did.

Chapter 6 — The Cost Model, Applied (Capstone Lab)

Before you start: all of Part 1 — Chapter 1 (the latency hierarchy), Chapter 2 (the CPU), Chapter 3 (caches), Chapter 4 (the TLB), Chapter 5 (data layout). · **Pairs with:** the performance-engineering Part, which turns these estimates into profiled, tool-verified analysis. This chapter is the capstone: you *predict* from the cost model, then *measure*, then *reconcile*.

BEFORE YOU READ ON

From memory, across the whole Part: (1) Chapter 1's orders of magnitude — an L1 hit ~1 ns, an L2 hit a few ns, a DRAM access ~100 ns. About how many times slower is DRAM than L2? (2) Chapter 3 — what does the hardware *prefetcher* do for a sequential access pattern? (3) *Predict, and write it down:* you sum a 1 MiB array (fits in L2) and a 512 MiB array (lives in DRAM), touching the same total number of elements. From the latency numbers alone, how many times slower would you guess the DRAM sum is — 2×? 10×? 100×? Hold your number; this chapter measures it, and the answer is a lesson.

Everything in Part 1 has been building one instrument: a **cost model** — a set of orders of magnitude and mechanisms that lets you estimate, before writing or profiling a line, roughly how fast a piece of code *can* go and what will limit it. This closing chapter uses that instrument for real. First we assemble the estimation toolkit and the single most useful classification in performance work — is this workload **compute-bound**, **memory-bound**, or **I/O-bound**? Then we run the capstone lab: predict the cost of two array sums from the model, measure them on real hardware, and — this is the whole point — *reconcile the surprise* when the measurement defies the naive prediction. The reconciliation is where the model earns its keep, because a cost model you cannot correct against reality is just a table of numbers to misquote.

6.1 Back-of-the-envelope estimation

Back-of-the-envelope estimation means getting the right *order of magnitude* with arithmetic you can do in your head, using the ratios from this Part — not predicting a benchmark to three digits. The toolkit is small, and every number in it is one you already have (treat all as approximate and machine-dependent; the register-through-DRAM figures anchor to CS:APP, the SSD/network ones to the community "latency numbers" list, both introduced in Chapter 1):

- **Latencies** (time to get *one* scattered item): L1 ~1 ns, L2 a few ns, L3 ~10 ns, DRAM ~100 ns, local SSD tens of µs, network round trip hundreds of µs and up — multiplicative jumps, ~10–100× per tier.
- **Bandwidths** (throughput for *streaming* many contiguous items): memory delivers on the order of *gigabytes per second* per core when accessed sequentially, because the prefetcher and full-line fetches keep the pipe full. Latency and bandwidth are

different numbers answering different questions — the distinction is the hinge of this whole chapter.

- **Compute**: a core executes on the order of a few instructions per cycle at a few GHz — so ~10^9–10^{10} simple operations per second per core (Chapters 1–2).
- **Sizes that gate tier**: which tier your data lives in is set by working-set size versus cache sizes (measure your own with `lscpu`/`getconf`). Fits in L2 → L2 speed; spills past L3 → DRAM speed; larger than RAM → SSD/I-O speed.

The method: estimate how many operations and how many bytes the work requires, divide by the relevant rate, and take the *larger* of the compute-time and memory-time estimates as your floor — because the resource in shorter supply is the bottleneck. That single comparison is the next section.

6.2 Compute-bound, memory-bound, or I/O-bound

The most useful question you can ask about a workload is *what resource is it waiting on?* Three broad answers, each pointing at a different toolkit:

- **Compute-bound:** the bottleneck is the CPU's execution units — lots of arithmetic per byte of data, data already in cache. Time scales with operations, not memory traffic. The levers are Chapter 2's: better ILP, fewer/ predictable branches, vectorization, a lower-complexity algorithm.
- **Memory-bound:** the bottleneck is moving data between the CPU and memory — little arithmetic per byte, or a working set that spills to DRAM. Time scales with bytes moved and miss counts, not with the arithmetic. The levers are Chapters 3–5's: locality, layout (AoS/SoA), blocking, and fewer, fuller cache lines and pages.
- **I/O-bound:** the bottleneck is the disk or network — the workload spends its time waiting on tens-of-microseconds-and-up operations. Time scales with I/O operations and their latency; the levers are batching, asynchrony, and reducing round trips (the low-level I/O Part).

The deciding ratio is *work per byte*: an operation that does much arithmetic on each byte it loads (a dense matrix multiply) tends compute-bound; one that does almost nothing per byte (summing an array, copying memory) tends memory-bound; one dominated by waiting on storage or a socket is I/O-bound. Crucially, *the same algorithm can change class with its input size*: a sum whose array fits in cache may be limited by the add pipeline (compute-ish), while the identical sum over an array that spills to DRAM becomes memory-bound. This is why "profile before you optimize" is not a platitude — apply the memory toolkit to a compute-bound loop and you get nothing. Estimate the class first; the estimate tells you which chapter to open.

> **QUICK CHECK**
>
> Classify each as most likely compute-, memory-, or I/O-bound: (a) summing a 500 MiB array once; (b) a dense N×N matrix multiply with N small enough to fit in cache; (c) reading 10,000 small records from an SSD one at a time. (Answer after the lab.)

6.3 The capstone lab, part 1: predict, measure, reconcile a surprise

Now the payoff. We will estimate, then measure, the cost of summing an array that fits in L2 versus one that spills to DRAM — and confront the gap between the naive prediction and the result.

The prediction (naive, from latency numbers). A 1 MiB array of `long` fits in this machine's per-core L2 (1.25 MiB); a 512 MiB array is far past the 24 MiB L3, so it lives in DRAM. If you reach for the *latency* numbers — L2 a few ns, DRAM ~100 ns — the tempting estimate is that the DRAM sum should be roughly **an order of magnitude or more slower**, maybe 10–100×. Write that prediction down; it is the one most readers make, and it is wrong — instructively.

The measurement. Summing into a `long`, touching the same total element count in both cases (the 1 MiB array swept many times, the 512 MiB array once), built with `gcc -O2` and run for this book on an i7-12700H (best of 5):

```
/* both sum the SAME number of elements; only where the data lives differs */
for (pass ...) for (i < n) sum += a[i];   /* small n: L2-resident;  big n: DRAM */
```

Working set	Time (same total elements)	ns / element	Effective GB/s
L2-resident (1 MiB, many passes)	~0.048–0.050 s	~0.71–0.75	~10.7–11.2
DRAM-resident (512 MiB, one pass)	~0.082–0.086 s	~1.22–1.28	~6.3–6.6

The DRAM sum was only about **1.7×** slower than the L2 sum — *not* the 10–100× the latency numbers suggested. (These figures are machine-dependent; the point is the *ratio*'s surprise, not the digits.)

The reconciliation — why the prediction was wrong. The naive estimate used the wrong number. The ~100 ns DRAM figure is a *latency* — the time to get *one* scattered item when you must wait for it. But a sequential sum does not wait for one item at a time: the access pattern is stride-1, so the **hardware prefetcher** (Chapter 3) sees it coming and streams lines ahead of the loop, and the CPU has many independent loads in flight. The workload is therefore **bandwidth-bound, not latency-bound** — its speed is set by how fast memory can stream contiguous data, not by the latency of a single miss. And streaming bandwidth from DRAM, while lower than from L2, is within a small factor of it; here the loop achieved ~11 GB/s out of L2 and ~6 GB/s out of DRAM (both also partly capped by the single-accumulator add dependency of §2.3, which is why neither is dramatic). Prefetching converted a would-be 100× latency gap into a ~1.7× bandwidth gap. *That* is the lesson of the capstone: **use latency for scattered access and bandwidth for streaming access** — reach for the wrong one and your estimate is off by orders of magnitude.

6.4 The capstone lab, part 2: when the memory wall really bites

So does the ~100 ns DRAM latency ever show up? Yes — precisely when the prefetcher *cannot* help, i.e. when access is scattered and dependent, so there is no stream to run ahead of and no independent work to overlap. That is the pointer-chasing pattern of Chapter 3. To expose it, chase a pointer around a random cyclic permutation (each step's address comes from the previous read) over the same two working sets — L2-resident and DRAM-resident — timing an equal number of dependent steps. Measured on the same machine (best of 3):

Working set (random pointer-chase)	ns / dependent step	DRAM/L2 ratio
L2-resident (1 MiB)	~6 ns	~24–25×
DRAM-resident (512 MiB)	~150–158 ns	

Now the latency gap appears in full: ~24–25×, and the DRAM chase costs ~150 ns per step — comfortably in the ~100 ns order of magnitude Chapter 1 gave for a DRAM access, and in fact somewhat *more*. That surplus over the bare ~100 ns is itself a Part-1 lesson: a random chase over 512 MiB also **thrashes the TLB** (Chapter 4) — each jump lands on a new page whose translation is likely evicted — so part of the ~150 ns is the page-table walk stacked on top of the data miss. Same hardware, same data sizes as §6.3; the *only* change is sequential→random access, and the cost went from a ~1.7× bandwidth gap to a ~25× latency gap. This is the whole Part in one contrast: *where your data lives* matters, but *how you walk it* decides whether the memory wall is hidden or paid in full.

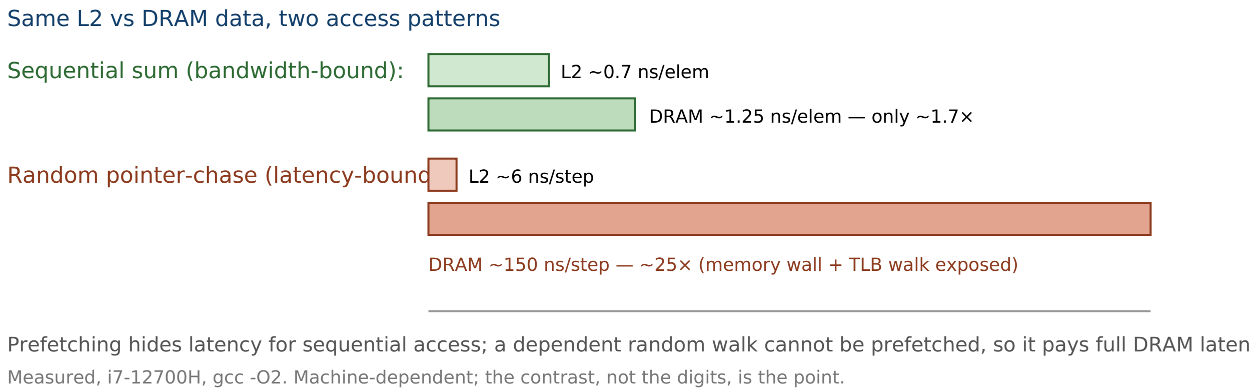


Figure 6.1: The same data sizes, two access patterns. Sequential streaming is bandwidth-bound (DRAM only ~1.7× slower than L2); a dependent random chase is latency-bound (~25×), exposing the full memory wall plus TLB cost.

THE SAME IDEA THREE WAYS: LATENCY VS. BANDWIDTH DECIDES THE ESTIMATE

1. In words. DRAM's ~100 ns is a *latency* — the wait for one scattered item. Sequential access is not a series of waits; the prefetcher streams data, so it runs at memory *bandwidth*, within a small factor of cache. Predict streaming with bandwidth, scattered access with latency.

2. As a picture. Figure 6.1: a small sequential gap (bandwidth) beside a large random gap (latency) over the identical data sizes.

3. As measurements. Sequential sum: DRAM ~1.7× L2. Random chase: DRAM ~25× L2, ~150 ns/step. The model predicted 100× from latency; the streaming case corrected it to ~1.7×; the scattered case confirmed the latency really is there when nothing hides it.

(Answer to the §6.2 quick check: (a) ***memory-bound*** *— one add per 8 bytes, streams from DRAM; (b)* ***compute-bound*** *— much arithmetic per byte, data resident in cache; (c)* ***I/O-bound*** *— dominated by per-record SSD latency, and batching/async would help.)*

TRAP: GUESSING INSTEAD OF PROFILING (AND ESTIMATING WITH THE WRONG NUMBER)

Two failures this Part has hammered, together at last. The first: *optimizing without measuring* — rewriting a loop's layout or algorithm on a hunch, when a five-minute profile would have shown the time is somewhere else entirely. The second, subtler: *estimating with the wrong model* — as in §6.3, where the ~100 ns latency number predicts a 100× gap for a workload that is actually bandwidth-bound and shows ~1.7×. A back-of-the-envelope estimate is a hypothesis about the bottleneck, not a verdict; it tells you where to *look* and which class you expect, and then you measure to confirm the class and find the real limiter. The tell that you have mis-estimated is exactly the §6.3 surprise: a measurement off from your prediction by more than a small factor — treat that gap as information (usually you used latency where bandwidth applies, or vice versa), reconcile it, and update the model. Guessing the cause and skipping the measurement is how good engineers waste days; the discipline of Part 1 is estimate, measure, reconcile, in that order.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A colleague says: "DRAM is 100× slower than cache, so our big sequential scan over the 500 MiB table must be crippled — let's cache it in a fancy structure." Cover the paragraph and answer in three or four sentences, using *latency*, *bandwidth*, *prefetcher*, and *bound*.

Model answer. "That 100× is a *latency* figure — the wait for one scattered access. Our scan is sequential, so the prefetcher streams the data and we run at memory *bandwidth*, which is within a small factor of cache; the scan is bandwidth-bound, not latency-bound, so it's probably only a couple of times slower than if it fit in cache, not 100×. Before we build anything, let's measure the scan's GB/s and confirm it's near memory bandwidth — if it is, a fancy cache buys little and the win, if any, is fewer bytes touched, not lower latency. Where latency *would* bite is if we switched to scattered, pointer-chasing access — then we'd pay the full ~100 ns-plus per step." Naming which resource bounds the workload, and measuring before building, is the senior register.

6.5 Closing Part 1: from the machine to the code that respects it

Six chapters ago the RAM model told a comforting lie: every operation costs one unit, memory is uniform, and speed follows big-O. Part 1 replaced it with the real cost model. You know the machine now — the CPU that overlaps and predicts (Chapter 2), the cache hierarchy that moves memory in lines and rewards locality (Chapter 3), the TLB that translates addresses and punishes scattered pages (Chapter 4) — and you know how to spend that knowledge: lay data out for the access pattern (Chapter 5), and estimate before you optimize, distinguishing latency from bandwidth and compute-bound from memory-bound from I/O-bound (this chapter). The habit that unifies all of it is the one the whole Part drilled: when two pieces of code have the same big-O but different speed, the difference is the machine — and you now know where to look, what to estimate, and how to confirm it by measuring.

This closes Part 1, "The Machine & the Cost Model," and opens the rest of the volume. Part 2 takes you into **C and manual memory**: pointers, the stack and heap, alignment and layout under your direct control, and undefined behavior — programming with no safety net so you understand the net. Everything you estimated here in the abstract, you will there arrange in bytes by hand. The cost model is no longer something the machine hides from you; it is a tool you carry into every line you write.

CAN YOU… WITHOUT LOOKING?

- Do a **back-of-the-envelope** estimate: turn a workload into operations and bytes, divide by the right rate, and take the larger estimate as the floor.
- Distinguish **latency** from **bandwidth** and say which applies to scattered vs. streaming access — and why using the wrong one mispredicts by orders of magnitude.
- Classify a workload as **compute-, memory-, or I/O-bound** from its work-per-byte, and name which chapter's levers each class points to.
- Explain the **§6.3 surprise**: why the sequential DRAM sum was only ~1.7× slower than the L2 sum, not 100× (prefetcher → bandwidth-bound).
- Explain the **§6.4 contrast**: why the random pointer-chase *did* show ~25× and ~150 ns/step, and what part of that surplus is TLB cost.
- State the discipline in order — **estimate, measure, reconcile** — and why a prediction-vs-measurement gap is information, not failure.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Give the back-of-the-envelope method for deciding whether code is compute- or memory-bound, and the "work per byte" intuition behind it. (§6.1–6.2)
2. **R2.** What is the difference between latency and bandwidth, and which one governs a sequential scan? (§6.1, §6.3)
3. **R3.** Why was the sequential DRAM sum only ~1.7× slower than the L2 sum when the latency numbers suggested far more? (§6.3)
4. **R4.** Why did the random pointer-chase show ~25× and ~150 ns/step, and why is that *more* than the bare DRAM latency? (§6.4)
5. **R5.** State the estimate–measure–reconcile discipline and explain why a gap between prediction and measurement is useful rather than embarrassing. (§6.3, warning box)

FURTHER READING

- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). The applied home of this chapter: profiling, distinguishing compute- from memory-bound work, and measuring bandwidth vs. latency with real tools — the direct continuation into the performance-engineering Part.
- **Agner Fog, optimization manuals** (free, *agner.org*). Microarchitectural detail behind the compute-bound levers — throughput vs. latency of instructions, and how many can be in flight. Dated in specifics; verify against your CPU.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free) and **Bryant & O'Hallaron, CS:APP**, Chapters 6 and 9. The mechanisms this chapter estimates against — cache and memory bandwidth, prefetching, and translation cost. Read Drepper for mechanism, not its dated absolute numbers; the "latency numbers" list (Jeff Dean / Colin Scott) supplies the SSD/network orders of magnitude, strictly as ratios.

Exercises

6.1 WARM-UP In one sentence each, define latency and bandwidth, and say which one you would use to estimate the time to (a) fetch one random element from a huge array, and (b) stream through a huge array in order.

6.2 WARM-UP Classify each workload as compute-, memory-, or I/O-bound and give the one-line reason: (a) `memcpy` of 1 GiB; (b) computing SHA-256 over data already in L1; (c) fetching 1,000 rows one-by-one from a remote database.

6.3 WARM-UP State the estimate–measure–reconcile discipline in order, and say in one sentence why a gap between your prediction and the measurement is useful information.

6.4 CORE You must sum a 512 MiB array of `long` once. (a) Estimate the time two ways: from DRAM *latency* (~100 ns per element) and from streaming *bandwidth* (assume ~6 GB/s for this loop). (b) Which estimate is right for a sequential sum, and why? (c) By roughly what factor do the two estimates differ, and what does that factor tell you about the cost of using the wrong model?

6.5 CORE Explain the §6.3 result to a teammate: the DRAM sum was only ~1.7× slower than the L2 sum. (a) Why did the prefetcher make this bandwidth-bound rather than latency-bound? (b) Why was even the L2 sum not much faster than a naive "few-ns-per-access" guess would allow (name the §2.3 factor)? (c) What single change to the code would let it approach memory bandwidth more closely?

6.6 COST A kernel does F floating-point operations and moves B bytes. Your core can do $\sim 10^{10}$ ops/s and stream $\sim 10^{10}$ bytes/s. (a) Write the compute-time and memory-time estimates. (b) Give the condition on F/B (work per byte) under which the kernel is compute-bound. (c) A sum does one add per 8 bytes — is it compute- or memory-bound by this test, and does the measurement in §6.3 agree?

6.7 FADED *Worked, with the last step for you.* Reconcile why the random pointer-chase over 512 MiB cost ~150 ns/step while the sequential sum over the same array cost ~1.25 ns/element. Steps 1–3 given; complete step 4.
Step 1 (given): Both touch DRAM-resident data of the same size; the only difference is access pattern (sequential vs. dependent-random).
Step 2 (given): Sequential access is prefetchable and has many independent loads in flight, so it runs at bandwidth (§6.3).
Step 3 (given): The random chase is a dependency chain — each step's address comes from the previous read — so it cannot be prefetched or overlapped and pays full latency per step (§3.5).
Step 4 (you): Complete the reconciliation: explain why ~150 ns/step is *more* than the ~100 ns bare DRAM latency (name the Chapter 4 mechanism), and state the one measurement that would confirm that extra cost is translation, not data. Then say what this pair of results teaches about when to estimate with latency vs. bandwidth.

6.8 MIXED For each observation, name the most likely bottleneck class (compute / memory / I/O) *and* the specific mechanism from Chapters 1–6, and justify in a line: (a) a sequential scan runs at ~6 GB/s and adding threads barely helps; (b) a loop with a data-dependent branch speeds up 3× after sorting the input; (c) a random lookup into a 2 GiB index costs ~160 ns and huge pages help; (d) a dense matrix multiply that fits in cache is limited by how many multiply-adds per cycle the core issues.

6.9 MIXED A report claims a data pipeline stage is "slow because DRAM is 100× slower than cache." Given only that the stage streams sequentially through a 4 GiB file-backed array, (a) say why the 100× framing is probably the wrong model, (b) predict the rough class and the rough slowdown vs. an in-cache version, and (c) name the two measurements you would take to confirm your reframing before anyone optimizes.

6.10 STRETCH The chapter measured a sequential DRAM/L2 ratio of ~1.7× and a random one of ~25× on one machine. (a) Explain why the *random* ratio is far more portable across machines than the sequential one (think about what each is bounded by). (b) Give one machine change that would raise the sequential ratio and one that would lower the random ns-per-step. (c) Argue why reporting both ratios — with the access pattern that produced each — is more honest and more useful than a single "DRAM is N× slower" headline number.

6.11 LAB **Capstone.** On your own machine: (1) record your L2 and L3 sizes and cache-line size. (2) *Predict*, from the cost model and before running, the time to sum an L2-sized array vs. a DRAM-sized array (same total elements), stating whether you expect it bandwidth- or latency-bound and your predicted ratio — write it down first. (3) Measure both (verify identical sums; best of several runs) and report ns/element and effective GB/s. (4) Then measure the *random pointer-chase* version over the same two sizes and report ns/step. (5) *Reconcile*: compare each measurement to your prediction, explain any surprise using latency-vs-bandwidth and (for the random case) the TLB, and if you can, capture cache-miss and TLB-miss counters (e.g. `perf stat`) to support your explanation. Report only numbers you actually measured; if a loop is elided, consume the result and say what you did.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART II

C & Manual Memory

Pointers, layout, the stack/heap, manual allocation, and undefined behavior — programming with no safety net so you understand the net.

Chapter 7 — Pointers & Memory Layout

Before you start: Chapter 4 (virtual addresses; a pointer holds a *virtual* address the hardware translates), Chapter 5 (alignment and struct padding, met there as a layout lever), Chapter 3 (cache lines and locality). · **Opens Part 2** and pairs with Chapter 8 (stack vs. heap) and Chapter 9 (manual allocation). Part 1 told you what memory *costs*; Part 2 hands you the controls. This chapter is the first: what a pointer actually is, how arithmetic on it works, and how your program's bytes are arranged in the address space.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 4 — when your C pointer holds a number like `0x7ffd3a08` and you dereference it, is that the physical address of a DRAM cell, or a *virtual* address the hardware translates first? (2) Chapter 5 said a struct like `{ char a; double b; char c; }` is bigger than its 10 bytes of real data — what is the compiler inserting, and why? (3) *Predict:* `p` is a pointer to an `int` and `q` is a pointer to a `double`. You write `p + 1` and `q + 1`. Do both advance the address by the same number of bytes? If not, what decides each step? By the end of the chapter you will be able to compute the exact byte a pointer expression lands on.

A pointer is the most demystifiable idea in systems programming, because underneath the syntax it is just a number: **a pointer is a variable that holds a memory address.** That is the whole secret. The type decoration — `int *`, `double *`, `struct Particle *` — does not change what is stored (an address); it tells the compiler what *kind* of thing lives at that address, which is exactly what it needs to know to read the right number of bytes when you dereference, and to scale arithmetic correctly when you do math on the pointer. This chapter builds the model from that one fact. We will see what a pointer holds and what dereferencing does; how pointer arithmetic silently multiplies by the pointee's size; why arrays and pointers are close cousins but *not* the same thing; how your program's memory is carved into regions (text, data, bss, heap, stack); and how `sizeof`, alignment, and struct padding decide the exact byte layout — with every size on this page *verified with the compiler* rather than asserted, because in C those sizes are implementation-defined and you must never guess them.

7.1 A pointer is an address

When you declare `int x = 42;`, the compiler arranges for `x` to live at some address in memory — say byte 0x7ffd...a0. When you then write `int *p = &x;`, the **address-of** operator `&` yields that address, and `p` stores it. So `p` holds a number that names a location; `*p` — the **dereference** — means "go to the location named by `p` and read (or write) the object there." Because `p` was declared `int *`, `*p` reads `sizeof(int)` bytes and interprets them as an `int`. Reconnecting to Chapter 4: the number in `p` is a *virtual* address in this process's address space; dereferencing it triggers the translation-and-fetch machinery we already dissected. The pointer is the programmer-visible handle on the very addresses that chapter was translating.

How big is a pointer? Big enough to hold an address on this machine. On the x86-64 machine used throughout this book, compiled with gcc, a data pointer is **8 bytes** (64 bits) — verified, not assumed:

```
printf("%zu\n", sizeof(void*));   /* prints 8 on this machine (x86-64, LP64) */
```

The C standard does *not* fix the size of a pointer — it is implementation-defined, and different pointer types need not even be the same size in general (a subtlety that rarely bites on flat-address-space machines like x86-64, where all object pointers are the same width). What the standard *does* guarantee is narrower and worth stating precisely: `void*` can hold any object pointer and round-trip it back unchanged, and `sizeof(char) == 1` by definition — a `char` is one *byte*, the unit `sizeof` counts in (cppreference, "sizeof operator"; the C standard, §6.5.3.4). Everything else about sizes we will measure.

QUICK CHECK

`int *p = &x;`. In one sentence each: what does `p` hold, and what does `*p` do? And why does the *type* `int *` matter when you write `*p`, if the stored value is "just an address"? (Answer after the next section.)

7.2 Pointer arithmetic scales by the pointee size

Here is the feature that trips up everyone once. When you add an integer to a pointer, C does *not* add that integer to the raw address. It scales: `p + n` advances the address by `n * sizeof(*p)` bytes, so that `p + n` points at the *n*-th object of that type past `p`, not the *n*-th byte. This is deliberate — it makes `p + 1` mean "the next element," which is what you almost always want — but it means the byte step depends entirely on the pointer's type. Verified on this machine:

```
int    arr[4];   /* &arr[1] - &arr[0] == 4 bytes  (sizeof(int)    == 4) */
double darr[4];  /* &darr[1]- &darr[0]== 8 bytes  (sizeof(double)== 8) */
```

Running exactly that on the book's machine, the `int` pointer stepped **4 bytes** and the `double` pointer stepped **8 bytes** — each equal to the pointee's `sizeof`, which is what "answers your prediction from the opener": no, `p + 1` and `q + 1` do *not* advance by the same number of bytes; each advances by its own element size. The mirror image is pointer *subtraction*: `q - p` for two pointers into the same array yields the number of *elements* between them (of type `ptrdiff_t`), again dividing out the element size. Indexing is just sugar for this: `arr[i]` is defined as `*(arr + i)`, so the subscript is scaled arithmetic wearing brackets.

Two precision points the C standard is strict about, both worth carrying into the UB chapter (Chapter 9). First, pointer arithmetic is only defined *within* a single array object, including the special "one past the last element" address, which you may compute and compare against but must *not* dereference (C standard §6.5.6, additive operators). Forming or dereferencing a pointer further out than one-past-the-end is undefined behavior — not "wraps around," not "reads the next variable," but genuinely undefined.

Second, the scaling is by the *static* type of the pointer, so a mismatched type computes the wrong address silently. The takeaway is mechanical and exact: *to find where `p + n` points, multiply `n` by `sizeof(*p)` and add that many bytes to `p`.*

> **THE SAME IDEA THREE WAYS: POINTER ARITHMETIC SCALES BY THE POINTEE SIZE**
>
> **1. In words.** Adding n to a pointer moves it by n elements, i.e. n × `sizeof(*p)` bytes — so the byte step is the element size, and `arr[i]` is exactly `*(arr + i)`.
>
> **2. As a picture.** Figure 7.1: two rulers over the same bytes. On the `int*` ruler each tick is 4 bytes apart; on the `double*` ruler each tick is 8 bytes apart. "+1" means "next tick," and the tick spacing is `sizeof(*p)`.
>
> **3. As a trace.** `double *q` at address 1000: `q+1` → 1000 + 1×8 = 1008; `q+3` → 1000 + 3×8 = 1024. An `int *p` at 1000: `p+3` → 1000 + 3×4 = 1012. Same "+3", different bytes, because the element size differs.

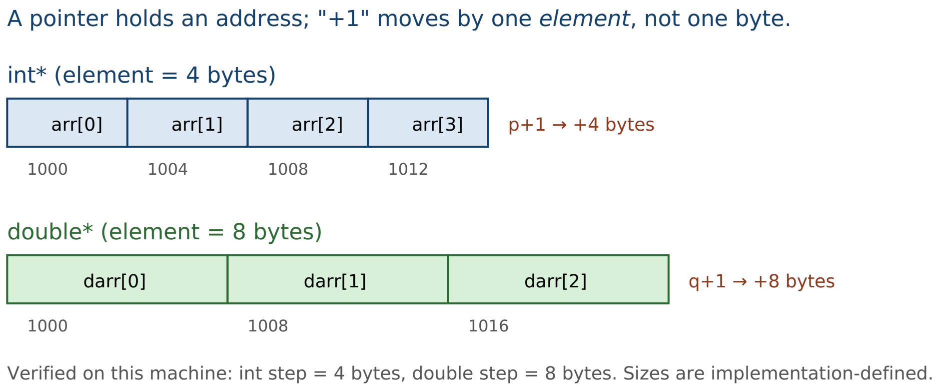


Figure 7.1: Pointer arithmetic scales by the pointee's size. "+1" advances one element — 4 bytes for `int*`, 8 for `double*` on this x86-64 machine.

(Answer to the §7.1 quick check: `p` holds the address of `x`; `*p` goes to that address and reads/writes the object there. The type `int *` matters because it tells the compiler how many bytes to read at that address and how to interpret them — four bytes as an `int` here — and it sets the scale for pointer arithmetic.)

7.3 Arrays vs. pointers, and array-to-pointer decay

Arrays and pointers feel interchangeable in C, and beginners often conclude they are the same. They are not, and the difference is a frequent source of bugs. An **array** is a block of contiguous elements; the array's name refers to that whole block, and the block *is* the storage. A **pointer** is a separate variable that holds an address, which may point at an array's first element but is its own object. The two behave alike in expressions only because of a specific rule: **array-to-pointer decay.** In most expression contexts, an array name is automatically converted ("decays") to a pointer to its first element (C standard §6.3.2.1). That is why you can write `int *p = arr;` and `arr[i]` and pass `arr` to a function expecting `int *` — the array decayed to `&arr[0]`.

The decay does *not* happen in three contexts, and those exceptions expose the difference. The sharpest is `sizeof`. Applied to an array, `sizeof` yields the *whole array*'s size in bytes; applied to a pointer, it yields the *pointer*'s size. Verified on this machine:

```
int  a10[10];
int *p = a10;
sizeof(a10);  /* == 40 : 10 ints × 4 bytes each, the whole array */
sizeof(p);    /* == 8  : just the pointer, regardless of what it points to */
```

On the book's machine those printed **40** and **8**. This is the origin of a notorious trap: passing an array to a function makes it decay to a pointer, so *inside* the function `sizeof(param)` is the pointer size (8), not the array size — the length information is lost at the call boundary, which is why C array-handling functions take an explicit length parameter. (The other two non-decay contexts: the operand of unary `&`, where `&arr` has type "pointer to array"; and a string literal used to initialize a `char` array, where it initializes the array rather than setting a pointer.) The mental model: an array is the land; a pointer is a signpost that can point at it. Usually the signpost is what travels around your program, but the two are not the same object — and `sizeof` is where the difference becomes visible.

TRAP: `SIZEOF` ON A DECAYED ARRAY (MEASURING THE SIGNPOST, NOT THE LAND)

The classic bug: `void f(int a[]) { size_t n = sizeof(a) / sizeof(a[0]); … }` intending to recover the element count. It cannot work: the parameter `a` is a *pointer* (the array decayed at the call), so `sizeof(a)` is the pointer size — 8 on this machine — and the "count" comes out as `8 / sizeof(int) = 2` regardless of the real length. The `sizeof(array)/ sizeof(array[0])` idiom is valid *only* where the true array type is in scope (same function it was declared in, not a decayed parameter). The tell: computing a length from `sizeof` on a function parameter, or on anything you received as a pointer. The fix is the one the standard library uses everywhere — pass the length explicitly alongside the pointer. Confirm your assumption by printing `sizeof` at the point of use if you are ever unsure; on this machine the decayed case prints 8, the in-scope array prints its full byte size (40 for `int[10]`).

7.4 The address-space layout: text, data, bss, heap, stack

Where do all these addresses live? A running process's virtual address space (the illusion of Chapter 4) is conventionally divided into regions, each holding a different kind of data, described in CS:APP (Chapter 9) and OSTEP's address-space chapters. From low addresses upward, the classic picture is:

- **Text** (a.k.a. code): your compiled machine instructions, plus read-only constants like string literals (often a separate `.rodata`). Typically mapped read-only and executable.
- **Data** (`.data`): global and `static` variables that have a non-zero initial value — the initial bytes are stored in the executable and loaded here.
- **BSS** (`.bss`): global and `static` variables that are zero-initialized. They take *no* space in the executable file (only a size is recorded); the OS provides zeroed pages at load. (The name is a historical assembler mnemonic.)

- **Heap**: the region for *dynamic* allocation (`malloc`), which grows upward (toward higher addresses) as you request more. This is Chapter 9's subject.
- **Stack**: automatic storage for function calls — local variables, saved return addresses, arguments — which grows *downward* (toward lower addresses) as calls nest. This is Chapter 8's subject.

You can see the ordering directly. A small program that prints the address of a function (text), an initialized global (data), a zero global (bss), a `malloc`'d block (heap), and a local (stack), built `-no-pie` so the classic fixed layout is visible, printed on this machine:

```
text  (&function)      = 0x401196
.data (&global_init)   = 0x404048
.bss  (&global_bss)    = 0x40405c
heap  (malloc'd block)  = 0x351b2a0
stack (&local)         = 0x7fffa35e9a3c
```

The ordering is exactly the textbook one: text < data < bss < heap, all low, and the stack far up near the top of the space — with the heap growing up and the stack growing down toward each other, which is why the address space leaves a large gap between them. **These specific numbers are machine-, OS-, and build-dependent, not universal.** A default position-independent-executable (PIE) build on the same machine printed entirely different, higher base addresses, because **address-space layout randomization (ASLR)** relocates the segments each run as a security measure — the *relative* structure holds, the absolute numbers do not. What to carry: your program's memory is not one undifferentiated pool; it is regions with different purposes, permissions, and growth directions, and knowing which region a pointer points into tells you a lot about its lifetime — the through-line into the next two chapters.

QUICK CHECK

Which region holds each of these: a `malloc`'d buffer, a function's local `int`, a `static int counter = 5;`, and the machine code of `main`? Which two regions grow *toward* each other, and in which directions? (Answer after the next section.)

7.5 `sizeof`, alignment, and struct padding — computed and verified

Now the exact byte layout. Three rules from the C object model (CS:APP Chapter 3; C standard §6.7.2.1) govern how a struct is laid out, and together they let you compute a struct's size by hand — which we will then check against the compiler.

1. **Each type has a size and an alignment.** `sizeof(T)` is how many bytes an object of type `T` occupies; its alignment is an address requirement — an object of type `T` must be placed at an address that is a multiple of `_Alignof(T)`. On x86-64 the alignment of the scalar types equals their size. Verified on this machine (gcc, x86-64 LP64):

type	char	short	int	long	long long	float	double	void*
sizeof (bytes)	1	2	4	8	8	4	8	8
_Alignof	1	2	4	8	8	4	8	8

These sizes are implementation-/ABI-defined, not guaranteed by the language. The C standard fixes only `sizeof(char) == 1` and minimum ranges (an `int` is *at least* 16 bits, a `long` at least 32); it does not promise `int` is 4 bytes. These are the values for this machine's ABI, measured — on a 16-bit or ILP64 platform they would differ, which is precisely why you print them rather than hard-code them.

1. **Members are laid out in declaration order, at increasing offsets, with the first member at offset 0.** Before each member the compiler inserts as much **padding** as needed to put that member at an offset that is a multiple of *its* alignment.
2. **The struct's own alignment is the largest alignment among its members, and its size is rounded up to a multiple of that alignment** (tail padding), so that in an array of the struct every element stays aligned.

Let us apply this to a concrete struct and predict its size before measuring:

```
struct Mix { char a; int b; char c; double d; };
```

Hand computation, using the verified alignments (char 1, int 4, double 8):

member	align	size	offset	note
`a` (char)	1	1	0	first member at 0; occupies byte 0
— padding —		3	1–3	to reach the next multiple of 4 for `b`
`b` (int)	4	4	4	4 is a multiple of 4; occupies 4–7
`c` (char)	1	1	8	occupies byte 8
— padding —		7	9–15	to reach the next multiple of 8 for `d`
`d` (double)	8	8	16	16 is a multiple of 8; occupies 16–23

Struct alignment = max(1, 4, 1, 8) = **8**; the running size is 24, already a multiple of 8, so no tail padding is needed. Predicted `sizeof(struct Mix)` = **24**, of which only 1 + 4 + 1 + 8 = 14 bytes are real data and **10 bytes are padding.** Now the check — running `offsetof` and `sizeof` on this machine printed *exactly*:

```
offsetof(a)=0  offsetof(b)=4  offsetof(c)=8  offsetof(d)=16
sizeof(struct Mix)=24   _Alignof(struct Mix)=8
```

The hand computation matches the compiler byte for byte. And it exposes the layout lever from Chapter 5: field *order* changes size. Reorder the same fields largest-alignment first —

```
struct Packed { double d; int b; char a; char c; };
```

— and the layout becomes `d` at 0, `b` at 8, `a` at 12, `c` at 13, needing only 2 bytes of tail padding to round 14 up to the next multiple of 8. Verified: `sizeof(struct Packed) = 16` on this machine. Same four fields, same data, **24 vs. 16 bytes** — a third smaller — purely from ordering, which (as Chapter 5 argued) means more records per cache line and per page for free. The point of this section is method as much as result: *you can compute a struct's size from the alignment rules, and you should verify it, because the underlying sizes are implementation-defined.*

(Answer to the §7.4 quick check: `malloc`'d buffer → heap; local `int` → stack; `static int counter = 5;` → data (non-zero initializer); machine code of `main` → text. The heap (growing up) and the stack (growing down) grow toward each other.)

7.6 Endianness: which byte comes first

One more layout fact, invisible until you look at individual bytes. When a multi-byte integer sits in memory, in what order are its bytes stored? **Little-endian** stores the least-significant byte at the lowest address; **big-endian** stores the most-significant byte first. The C standard does *not* mandate either — byte order is part of the implementation-defined representation of integer types (C standard §6.2.6) — so portable code must not assume one. x86-64 is **little-endian**, and rather than assert it, we verify it: store the 32-bit value `0x01020304` and read its bytes one at a time.

```
uint32_t x = 0x01020304u;
unsigned char *b = (unsigned char*)&x;
/* print b[0] b[1] b[2] b[3] */
```

On this machine that printed the bytes **`04 03 02 01`** — the least-significant byte (`0x04`) first, at the lowest address — confirming **little-endian**, as expected for x86-64. On a big-endian machine the same program would print `01 02 03 04`. Endianness matters the moment bytes cross a boundary the CPU does not control: reading a binary file or network packet written by a different machine, or reinterpreting memory through a different-width pointer. Within a single program on one machine you rarely notice it — arithmetic and printing hide it — but at the byte level it is real and, like the sizes above, something you *check* rather than assume. (Network protocols standardize on big-endian, "network byte order," precisely so machines of either kind can interoperate; Beej's Guide covers the `htons`/`ntohl` conversions the sockets Part will use.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate is debugging and says: "I have a `struct { char a; int b; char c; double d; }`, that's 14 bytes of data, but `sizeof` says 24 — is the compiler broken?" Cover the paragraph and answer in two or three sentences, using *alignment*, *padding*, and *field order*.

Model answer. "It's not broken — the compiler inserts padding so each field lands on an address that's a multiple of its alignment: the `int` needs a 4-aligned offset and the `double` an 8-aligned one, so there are 3 bytes of padding after `a` and 7 after `c`, and the whole struct rounds up to a multiple of 8 — 24 bytes, 10 of them padding. If you reorder the fields largest-alignment-first (`double, int, char, char`) it packs to 16, same data, because you stop paying to re-align. You can confirm any layout with `offsetof` and `sizeof` rather than guessing." Naming alignment and offsets — and showing you can recompute the size — is the senior register, and it turns "the compiler is weird" into a controllable layout decision.

7.7 What to carry forward

A pointer is an address in a variable; dereferencing it reads or writes the object there, using the pointer's type to know how many bytes and how to interpret them. Arithmetic on a pointer scales by the pointee's size, so `p + n` lands `n * sizeof(*p)` bytes along and `arr[i]` is just `*(arr + i)` — and stepping outside an array's bounds is undefined, not merely adventurous. Arrays and pointers are close but distinct: an array decays to a pointer to its first element in most contexts, but `sizeof` (and unary `&`) sees the real array, which is why length is lost at a function boundary. Your program's memory is regioned — text, data, bss, heap, stack — with different lifetimes and growth directions, verified here by printing addresses (and remembering ASLR moves the absolute numbers). And a struct's exact size follows from alignment and padding, which you can compute by hand and *must* verify, because the scalar sizes and byte order underneath are implementation-defined — measured here as `sizeof(struct Mix) = 24` and little-endian on this x86-64 machine.

That is the layer of control Part 1 was pointing at. Chapter 8 takes the two regions with the most interesting lifetimes — the **stack** and the **heap** — and asks where a given object should live, why stack allocation is nearly free while the heap is managed, and how a pointer to a stack local becomes a dangling bug the instant its function returns. Chapter 9 then hands you the heap's controls, `malloc` and `free`, and the precise ways manual memory management goes wrong. Everything in those chapters is pointers into these regions — which is why "a pointer is an address, and addresses live in regions with lifetimes" is the model to keep.

CAN YOU… WITHOUT LOOKING?

- State what a **pointer** holds, what **dereferencing** does, and why the pointer's **type** matters even though the stored value is "just an address."
- Compute where `p + n` points for a given pointer type (multiply *n* by `sizeof(*p)`), and say why stepping past one-past-the-end of an array is undefined.
- Explain **array-to-pointer decay**, and why `sizeof` on a function's array *parameter* gives the pointer size, not the array size.
- Name the address-space regions — **text, data, bss, heap, stack** — say what each holds, and which two grow toward each other and in which directions.
- Compute a struct's size by hand from **alignment and padding**, and explain why field *order* changes it — and why you *verify* sizes rather than assume them.
- Define **little- vs. big-endian**, state which x86-64 is, and describe the one-line experiment that checks it.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does a pointer hold, and what two things does its *type* tell the compiler? (§7.1–7.2)
2. **R2.** How many bytes does `p + 3` advance for an `int *p` vs. a `double *p` on this machine, and what general rule gives the answer? (§7.2)
3. **R3.** What is array-to-pointer decay, and what does `sizeof` report for an array vs. for a pointer (or a decayed parameter)? (§7.3)
4. **R4.** Name the five address-space regions low-to-high and say what each holds; which two grow toward each other? (§7.4)
5. **R5.** Give the three struct-layout rules and use them to explain why `{ char; int; char; double; }` is 24 bytes on this machine. (§7.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 3 (machine-level data, pointers, arrays, struct alignment) and Chapter 9 (the process address space and its regions). The reference for this chapter's layout and padding rules; the free CMU 15-213 materials cover them.
- **cppreference.com** and the **C standard** — the `sizeof` operator, pointer arithmetic (§6.5.6), array-to-pointer conversion (§6.3.2.1), and object representation (§6.2.6). The authority for what the language *guarantees* vs. leaves implementation-defined; consult these before assuming any size or byte order.
- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)**, the address-space chapters (free full text). The OS view of how text/data/heap/stack are arranged and why the stack and heap grow toward each other.
- **Beej's Guide to C** (free). Practical, readable coverage of pointers, arrays, and byte order, including the network-byte-order conversions used in the low-level I/O Part.

Exercises

7.1 WARM-UP In one sentence each: (a) what does a pointer variable store? (b) what does `*p` do? (c) what does `&x` produce?

7.2 WARM-UP A `double *q` holds address 2000 (`sizeof(double)==8` here). What address is `q + 4`? What address is `q + 4` if `q` is instead an `int *` (`sizeof(int)==4`)? Show the arithmetic.

7.3 WARM-UP `int a[10]; int *p = a;` on this machine. State the value of `sizeof(a)` and of `sizeof(p)`, and explain the difference in one sentence.

7.4 CORE Explain array-to-pointer decay. Then explain precisely why `size_t len(int a[]) { return sizeof(a) / sizeof(a[0]); }` does *not* return the element count, and what the returned value actually is on this machine.

7.5 CORE For `struct S { char a; short b; char c; int d; };`, compute by hand (using char 1, short 2, int 4, with alignment = size for each) the offset of every member, the total `sizeof`, and how many bytes are padding. Then state how you would verify your answer with the compiler.

7.6 CORE Reorder the fields of `struct S` from 7.5 to minimize its size, give the new `sizeof`, and explain the ordering rule you applied and why it works.

7.7 COST You have an array of N = 10 million records, and a hot loop that scans them. (a) If the record is `struct Mix` (24 bytes here), how many bytes does the array occupy? (b) If reordering shrinks it to 16 bytes, how many bytes now, and roughly how many *more* records fit in a 64-byte cache line (§3.1) and a 4 KiB page (§4.2)? (c) In one sentence, tie the size reduction to the memory-system cost model of Part 1.

7.8 FADED *Worked, with the last step for you.* Compute `sizeof(struct T { double d; char a; int b; };)` on this machine's ABI. Steps 1–3 given; complete step 4.
Step 1 (given): `d` (double, align 8) at offset 0, occupies 0–7.
Step 2 (given): `a` (char, align 1) at offset 8, occupies byte 8.
Step 3 (given): `b` (int, align 4) needs an offset that is a multiple of 4; the next such offset after byte 8 is 12, so 3 bytes of padding go at 9–11 and `b` occupies 12–15.
Step 4 (you): State the struct's alignment, whether any tail padding is needed to round the size up to a multiple of that alignment, the final `sizeof`, and the total padding bytes. Then name the one function you would call to confirm each member's offset.

7.9 MIXED For each symptom, decide which chapter's mechanism is the primary cause and justify in a line: (a) `sizeof(param)` inside a function returns 8 no matter how large the caller's array was; (b) a random-access loop over an 800 MiB table is far slower than ~100 ns/access predicts, and huge pages speed it up; (c) a struct is unexpectedly 24 bytes when its fields sum to 14; (d) reading a binary file written on a different architecture yields byte-swapped integers.

7.10 MIXED Without running code, decide for each what value (or category of value) is produced and why, drawing on Chapters 4–7: (a) the number stored in a pointer — is it a physical or virtual address? (b) `&arr[5] - &arr[2]` for an `int arr[10]`; (c) whether the absolute address printed by a PIE build is stable across runs, and what causes the answer; (d) whether `sizeof(int)` is guaranteed by the C standard to be 4.

7.11 STRETCH The chapter says pointer arithmetic is only defined within a single array object (plus one-past-the-end). (a) Explain why `&arr[10]` for `int arr[10]` is legal to *form* but not to *dereference*. (b) Give a plausible reason the standard makes going further undefined rather than defining it as "wraps" or "reads adjacent memory" (think about the optimizer and about Chapter 4's segmented address space). (c) Connect this to the out-of-bounds bug you will meet in Chapter 9.

7.12 LAB On your own machine, write and run a program that prints: (a) `sizeof` and `_Alignof` for `char, short, int, long, double, void*`; (b) the `offsetof` of every member and the `sizeof` of a struct of your choice with at least three differently-aligned fields, and verify it against your hand computation; (c) the byte order of a known 32-bit value to determine your machine's endianness. Report *your* measured numbers and label the machine/ABI; note any that differ from this chapter's x86-64 values and why they might.

Chapter 8 — The Stack & the Heap

Before you start: Chapter 7 (a pointer is an address; the address-space regions, including stack and heap), Chapter 4 (virtual memory), Chapter 5 (locality). · **Pairs with:** Chapter 9 (the heap's controls, `malloc`/`free`). Chapter 7 named the regions; this chapter takes the two with real lifetimes — the **stack** and the **heap** — and answers where an object should live, what each costs, and how a pointer that outlives its storage becomes one of C's classic bugs.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 7 — of the five address-space regions, which two grow *toward* each other, and in which directions? (2) Chapter 4 — a local variable and a `malloc`'d block both have *virtual* addresses; does that mean they have the same *lifetime*? Guess what determines when each stops being valid. (3) *Predict:* a function creates a local `int n`, and returns `&n` (its address) to the caller. The caller then reads through that pointer. Do you expect that to reliably give back the value of `n` — and if not, what has gone wrong by the time the caller looks? By the end you will be able to state precisely why this is a bug.

Every object in a C program has a **storage duration** — a lifetime — and getting lifetimes wrong is the source of a whole family of bugs that the previous chapter's "a pointer is just an address" model does not, by itself, warn you about. A pointer stays a valid handle only as long as the object it points at is alive; the moment that object's lifetime ends, the pointer becomes **dangling**, and using it is undefined behavior. This chapter is about the two storage durations you manage most directly. **Automatic** storage — the stack — is created and destroyed for you as functions run, is essentially free to allocate, and has a lifetime tied strictly to a block's execution. **Dynamic** storage — the heap — you allocate and free by hand, is managed by a real allocator with real cost, and lives exactly as long as you keep it alive. We will see how the call stack works and why pushing a frame is nearly free; why the heap is different and must be managed; what stack overflow is; and then the payoff — the lifetime rules that make "return a pointer to a local" a bug, shown correct and broken, and compiled to see what actually happens.

8.1 Automatic storage: the call stack and stack frames

When a function is called, the machine needs somewhere to put that call's local variables, its arguments, the address to return to when it finishes, and some bookkeeping. That somewhere is a **stack frame** (or activation record), pushed onto the **call stack** — the stack region from Chapter 7. The stack is a last-in-first-out structure that mirrors the call structure of the program: calling a function *pushes* a new frame on top; returning *pops* it off. Because calls nest (a function calls a function calls a function) and returns unwind in the reverse order, LIFO is exactly the right discipline, and the region grows *downward* (toward lower addresses) as frames stack up, as Chapter 7's address printout showed the stack sitting high and the CPU pushing toward the heap.

The hardware makes this cheap. A CPU register, the **stack pointer**, holds the address of the current top of the stack. Allocating a frame for a new call is, in essence, *subtracting* the frame's size from the stack pointer — moving it down to reserve that many bytes — and deallocating on return is adding it back. That is the sense in which stack allocation is "free": reserving space for all of a function's locals is a single arithmetic adjustment of one register, not a search or a bookkeeping operation. There is no per-variable cost and nothing to track; the locals are simply the bytes between the old and new stack-pointer positions. CS:APP (Chapter 3) develops the exact frame layout and the call/return instruction sequence; the shape to hold is: **a call pushes a frame by bumping a pointer, a return pops it by bumping the pointer back, and the whole frame's storage appears and vanishes together.**

Two consequences follow immediately. First, automatic variables have a lifetime tied to their block: they come into existence when execution enters the block (or function) and **cease to exist when execution leaves it** — the frame is popped, and those bytes are no longer reserved for them (C standard §6.2.4, automatic storage duration). Second, that is why locals are so cheap and why you do not free them: the pop reclaims the entire frame at once. The flip side — the danger — is that the storage is reclaimed *whether or not any pointer still refers to it*. Hold onto that; it is the whole of §8.4.

> **QUICK CHECK**
>
> Why is allocating a function's local variables often described as "free," in terms of what the CPU actually does to reserve the space? And what event reclaims all of a function's locals at once? (Answer after the next section.)

8.2 Dynamic storage: the heap, and why it must be managed

The stack's discipline — allocate on call, free on return, strict LIFO — is exactly what makes it cheap, and also what makes it insufficient. Plenty of data does not follow call structure: a buffer whose size you only learn at runtime, a data structure that must outlive the function that built it, a result you want to hand back to a caller. For these you need storage whose lifetime you control independently of the call stack. That is the **heap**, and you obtain it through the allocation functions `malloc` and friends (the subject of Chapter 9): `malloc(n)` asks for *n* bytes and returns a pointer to them; that block stays alive until *you* explicitly `free` it, across as many function calls and returns as you like.

This freedom is not free. Unlike the stack — where "allocate" is a pointer bump and "free" is a pointer bump back — the heap must track, out of a large pool, which regions are in use and which are available, in whatever arbitrary order you allocate and free them. That tracking is the job of the **allocator**, a nontrivial piece of software (typically inside the C library) that maintains data structures — free lists, size classes — to satisfy requests and reclaim freed blocks. Because allocations and frees can interleave in any order, the allocator must search for a suitable free block, may split or coalesce blocks, and can leave the pool **fragmented** (free space chopped into pieces too small to be useful). So a heap allocation is a genuine function call doing real work, orders of magnitude more expensive than a stack allocation, and the memory it returns must be explicitly returned or it is **leaked**. Chapter 9 opens the allocator up; for now the contrast is the point.

THE SAME IDEA THREE WAYS: STACK VS. HEAP

1. In words. The stack is automatic storage with LIFO lifetime tied to calls, allocated/freed by bumping the stack pointer — nearly free, but a local dies when its function returns. The heap is manually managed storage with a lifetime *you* control, allocated by a real allocator — flexible, but costly and yours to free.

2. As a picture. Figure 8.1: the stack, a tidy pile of frames growing down, each pushed/popped whole; the heap, a pool of scattered blocks (some in use, some free) that an allocator hands out and reclaims in any order.

3. As a table. Stack: allocate = pointer bump, free = automatic on return, lifetime = the block, cost ≈ free, failure = overflow. Heap: allocate = `malloc` (search/split), free = manual `free`, lifetime = until you free it, cost = real, failures = leak / use-after-free / double-free (Ch 9).

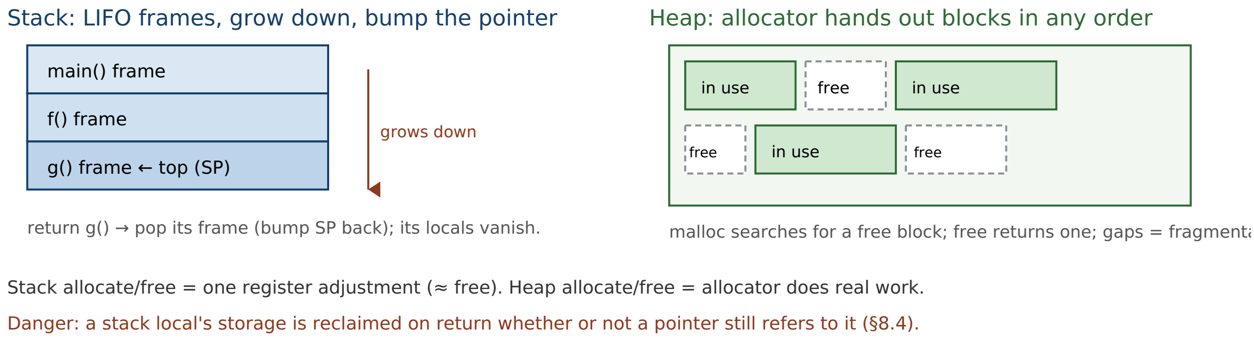


Figure 8.1: The stack pushes/pops whole frames by bumping a pointer (nearly free, LIFO lifetime); the heap is a managed pool the allocator hands out and reclaims in any order (flexible, costly, yours to free).

8.3 Stack overflow

The stack is fast but **bounded**: the OS reserves a finite region for it (commonly on the order of a few megabytes per thread by default on Linux, though the exact size is configurable and platform-dependent — check `ulimit -s` rather than assume a number). If the program keeps pushing frames without returning — most often through unbounded or runaway **recursion**, or by declaring a very large local array — the stack pointer eventually runs past the end of that reserved region. This is **stack overflow**: the access falls on a page with no valid mapping (often a deliberately unmapped *guard page*), the hardware faults, and the OS typically terminates the process (on Linux, a segmentation fault). The two everyday causes are worth naming: recursion with no base case or too deep a recursion for the input, and large automatic arrays (declaring `int buf[10000000];` as a local tries to reserve tens of megabytes on a few-megabyte stack). The fixes match the causes: bound the recursion (or convert it to iteration with an explicit heap-backed structure), and put large or runtime-sized buffers on the *heap*, where the space is far larger and the size can be chosen at runtime. Stack overflow is the stack's characteristic failure, just as leaks and use-after-free (Ch 9) are the heap's.

(Answer to the §8.1 quick check: stack allocation is "free" because reserving space for a call's locals is just subtracting the frame size from the stack-pointer register — one arithmetic adjustment, no per-variable work. Returning from the function — popping the frame by moving the stack pointer back — reclaims all of that function's locals at once.)

8.4 Lifetimes and the dangling-pointer bug: why you can't return a pointer to a local

Now the payoff, and one of the most important bugs in this book. Because a stack frame is reclaimed the instant its function returns (§8.1), **a pointer to a local variable is only valid until that function returns.** After the return, the local's storage is no longer reserved for it — the frame has been popped, the stack pointer moved back, and those bytes are free to be reused by the very next function call. A pointer still holding that address is a **dangling pointer**: it names storage whose lifetime has ended. Reading or writing through it is **undefined behavior** (C standard §6.2.4: the lifetime of an object with automatic storage duration ends when execution leaves its block; using a pointer to an object past its lifetime is UB). This is the answer to the opener's prediction — returning `&n` and dereferencing it later does *not* reliably give back `n`, because `n` no longer exists.

Here is the bug and its correct counterpart, side by side:

```
/* BUG: returns the address of a local; the frame is gone on return. */
int *make_counter_bug(void) {
    int n = 0;
    return &n;          /* dangling: n dies when this function returns */
}

/* CORRECT: the caller owns the storage; the callee just fills it in. */
void fill_counter(int *out) {
    *out = 0;            /* writes into storage the caller keeps alive */
}

int main(void) {
    int *p = make_counter_bug();
    printf("%d\n", *p);  /* undefined behavior: *p reads a dead local */

    int n;
    fill_counter(&n);    /* n lives in main's frame, valid throughout */
    printf("%d\n", n);   /* well-defined */
}
```

Two things about the buggy version deserve care. First, **the compiler warns you.** Built with `gcc -Wall` on this machine, `make_counter_bug` produces `warning: function returns address of local variable [-Wreturn-local-addr]`; clang emits the analogous `-Wreturn-stack-address`. This is a bug the toolchain actively flags — heed it. Second, **what happens at runtime is not guaranteed**, and this is the crux of undefined behavior. On this machine, compiling the buggy program (at both `-O0` and `-O2`) and running it produced a **segmentation fault** (the process crashed). But that outcome is *not* part of any contract: UB means the standard imposes no requirements at all, so a different compiler, optimization level, or run could instead print `0`, print stale garbage, or appear to "work"

— which is worse, because the latent bug hides until conditions change. Do *not* reason "it segfaulted, so at least it's detectable": you cannot rely on the crash any more than on the value. The correct fix, shown on the right, is the standard pattern — **the caller owns the storage and passes a pointer in** (`fill_counter(&n)`), so the object outlives the callee's frame; or, when the callee must create the object, allocate it on the *heap* and return that pointer (the caller then frees it — Ch 9). Either way the rule is the same: *never hand out a pointer whose target will die before the pointer does.*

> **TRAP: RETURNING (OR STORING) A POINTER TO A STACK LOCAL**
>
> The recurring lifetime bug: a function returns `&local`, or stores `&local` into a longer-lived structure (a global, an out-parameter's target, a caller's list), and something reads through that pointer after the function has returned. The local's frame is gone, so the pointer dangles and the access is undefined behavior — it may crash, may return stale or overwritten bytes, or may *appear* to work until the next call reuses the frame, which is the dangerous case because it hides in testing. The tells: returning the address of a local or parameter; taking `&` of a local and stashing it somewhere that outlives the function; a pointer that "worked yesterday" now returning garbage after unrelated code changed the call pattern. Heed `-Wreturn-local-addr` / `-Wreturn-stack-address` — the compiler catches the direct return. The fixes: return by value for small objects; take an out-pointer the caller owns; or heap-allocate and transfer ownership (documenting who frees). And do not trust a crash to reveal it — UB is not obligated to crash, so the discipline is to reason about lifetime, not to test-and-see.

> **QUICK CHECK**
>
> Two functions each need to give the caller an `int` the caller can keep: one returns `&local`; the other takes `int *out` and writes `*out`. Which one is correct and why? Where does the correct one's storage actually live? (Answer below the next section.)

8.5 Choosing where an object should live

The stack/heap choice, made well, is mostly mechanical once you ask two questions: *how long must this object live,* and *how big is it (and is that known at compile time)?* Prefer the **stack** when the object's lifetime fits within the function that creates it and its size is known and modest — this is the common case, and it is free, automatically cleaned up, and cache-friendly (a frame's locals are hot and contiguous). Reach for the **heap** when the object must *outlive* the function that creates it (you need to return it, or store it in a longer-lived structure), when its size is only known at runtime or is too large for the stack (§8.3), or when its lifetime genuinely does not nest with call structure (shared data structures, caches, anything with a dynamic graph of owners). The cost of the heap is not only the allocator's time (§8.2) but the obligation it creates: *someone must free it, exactly once, after the last use* — and getting that wrong is the entire subject of the next chapter. This is why the guidance leans toward the stack: it removes the question. The senior habit is to make lifetime an explicit, stated design decision — "this object lives on the stack in `f`," or "the heap owns this and `g` frees it" — rather than reaching for `malloc` reflexively or letting ownership stay implicit.

(Answer to the §8.4 quick check: the `int *out` version is correct — it writes into storage the caller already owns and keeps alive (a local in the caller's frame, or wherever the caller got the pointer), so the object outlives the callee's return. The `&local` version returns a dangling pointer to a frame that is popped on return. The correct one's storage lives in the caller's frame, not the callee's.)

> **SAY IT LIKE A SYSTEMS ENGINEER**
>
> *Try it first.* In code review, someone defends a helper that does `char *label(int id) { char buf[32]; sprintf(buf, "id-%d", id); return buf; }` with "it compiles and it worked in my test." Cover the paragraph and explain the problem in two or three sentences, using *stack frame*, *lifetime*, and *undefined behavior*.
>
> *Model answer.* "`buf` is an automatic array in `label`'s stack frame, so its lifetime ends the instant `label` returns — the returned pointer dangles, and any read through it is undefined behavior. It 'worked in your test' only by luck: UB isn't required to crash, so the frame's bytes happened to still hold the string until something reused them, which is exactly the bug that hides until the call pattern changes. The compiler will even warn with `-Wreturn-local-addr`. Fix it by having the caller pass in a buffer it owns — `label(id, char *out, size_t n)` — or by returning a heap allocation the caller frees." Naming the frame's lifetime and calling the crash *unreliable* — rather than "it segfaults sometimes" — is the senior register, and it points at the real fix: make ownership explicit.

8.6 What to carry forward

Objects have lifetimes, and a pointer is only valid while its target lives. **Automatic (stack) storage** is created and destroyed with function calls: a frame is pushed by bumping the stack pointer down and popped by bumping it back, which is why stack allocation is nearly free and why a local's lifetime ends exactly when its function returns. **Dynamic (heap) storage** is managed by a real allocator, costs real time, lives until you free it, and exists precisely for objects that must outlive their creating call or whose size or lifetime does not fit the stack's LIFO discipline. The stack's characteristic failure is **overflow** (runaway recursion or huge locals); the heap's failures are the next chapter's. And the through-line bug of this chapter — **returning or storing a pointer to a stack local** — is undefined behavior because the storage is reclaimed on return whether or not a pointer still refers to it; the compiler warns, the crash is *not* guaranteed, and the fix is to make the storage outlive the pointer (caller-owned or heap-owned).

Chapter 9 picks up the heap's controls in full: what `malloc`, `calloc`, `realloc`, and `free` actually promise; what the allocator is doing underneath (free lists, size classes, fragmentation); and the precise catalogue of manual-memory bugs — use-after-free, double-free, leak, buffer overflow, uninitialized read — each stated exactly as the undefined (or indeterminate) behavior it is, with the real tools that catch them. The dangling-pointer idea you just learned on the stack reappears there on the heap as *use-after-free*: the same bug — a pointer outliving its storage — in the region where *you* control the lifetime, and therefore *you* can get it wrong.

CAN YOU… WITHOUT LOOKING?

- Explain what a **stack frame** is, why a call pushing one and a return popping one is nearly **free**, and what a local's **lifetime** is tied to.
- Say why the **heap** must be **managed** (an allocator tracking free space in arbitrary allocate/free order) and how that makes it costlier than the stack.
- Define **stack overflow**, name its two common causes, and give the matching fixes.
- State precisely why **returning a pointer to a local** is a bug — in terms of frame lifetime — and why it is **undefined behavior** rather than "returns garbage."
- Explain why a crash from that bug is *not* guaranteed, and give the two correct patterns (caller-owned storage, or heap-allocate-and-transfer-ownership).
- Decide, for a given object, whether it belongs on the **stack** or the **heap**, from its lifetime and size.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a stack frame, and in what sense is allocating one "free"? What reclaims a function's locals? (§8.1)
2. **R2.** Why must the heap be managed by an allocator, and name two costs that follow. (§8.2)
3. **R3.** What is stack overflow, what are its two usual causes, and how do you fix each? (§8.3)
4. **R4.** Precisely why is returning a pointer to a local undefined behavior, and why is a crash *not* guaranteed? (§8.4)
5. **R5.** Give two questions that decide whether an object belongs on the stack or the heap, and the correct fix for a callee that must hand the caller an object it created. (§8.4–8.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 3 (procedure calls, the stack frame, the call/return sequence and stack-pointer mechanics). The reference for how frames are pushed and popped; the free CMU 15-213 materials cover it.
- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)**, the address-space and memory-API chapters (free full text). The OS view of the stack and heap regions, stack limits and guard pages, and the allocation interface.
- **cppreference.com** and the **C standard** — storage duration (§6.2.4) and object lifetime. The authority for when an automatic object's lifetime ends and why using a pointer past it is undefined.
- **Beej's Guide to C** (free). Practical treatment of the stack, the heap, and the common lifetime mistakes, with runnable examples.

Exercises

8.1 WARM-UP In one sentence each: (a) what is a stack frame? (b) what CPU operation allocates one? (c) when does an automatic (local) variable's lifetime end?

8.2 WARM-UP Name one reason to allocate on the heap rather than the stack, and one cost the heap imposes that the stack does not.

8.3 WARM-UP Give the two most common causes of stack overflow, and the matching fix for each.

8.4 CORE Explain, in terms of stack frames and the stack pointer, precisely why this is a bug: `int *f(void) { int x = 7; return &x; }`. State what happens to `x`'s storage on return, why using the returned pointer is undefined behavior, and what warning gcc/clang emit.

8.5 CORE Rewrite `int *f(void) { int x = 7; return &x; }` two correct ways: (a) so the *caller* owns the storage; (b) so the object lives on the heap. For (b), state who is responsible for freeing it and when.

8.6 CORE A colleague says "my function returns a pointer to a local and it works fine — I tested it." Explain why "it worked" is *not* evidence the code is correct, using the definition of undefined behavior. What is the danger of a bug that happens to appear to work?

8.7 COST Compare the cost of one stack allocation vs. one heap allocation. (a) In terms of operations, what does the machine do to allocate a stack frame's worth of locals? (b) What must an allocator potentially do to satisfy one `malloc`? (c) Given (a) and (b), state a rule of thumb for a hot loop that needs a small scratch buffer each iteration, and justify it.

8.8 FADED *Worked, with the last step for you.* Decide where a function's result should live for: a helper that builds a linked list of unknown length and returns it to its caller. Steps 1–3 given; complete step 4.
Step 1 (given): The list must *outlive* the helper that builds it — the caller uses it after the helper returns.
Step 2 (given): Its size (number of nodes) is only known at runtime.
Step 3 (given): Both facts rule out automatic (stack) storage: the nodes would die on return, and the count is not fixed at compile time.
Step 4 (you): State where the nodes must therefore be allocated, who becomes responsible for freeing them and when, and what bug (from the next chapter's preview) results if that responsibility is forgotten. Name the design principle: who owns the memory?

8.9 MIXED For each symptom, decide which mechanism from Chapters 4–8 is the primary cause and justify in a line: (a) a recursive function crashes with a segfault only on large inputs; (b) a function returns a pointer that gives correct values in unit tests but garbage in production after unrelated code was added; (c) a random-access loop over a huge array is slower than DRAM latency predicts and huge pages help; (d) a struct is larger than the sum of its fields.

8.10 MIXED Without running anything, classify each as *well-defined*, *undefined behavior*, or *a performance issue only*, and say why, drawing on Chapters 7–8: (a) taking `&local` and using it *within* the same function; (b) returning `&local` and dereferencing it in the caller; (c) declaring `int big[8000000];` as a local and using it; (d) storing `&local` into a global and reading it after the function returns.

8.11 STRETCH The chapter draws a parallel: a dangling pointer to a stack local (this chapter) and a use-after-free on the heap (Chapter 9) are "the same bug in different regions." (a) State the one property both violate. (b) Explain why the stack version is in some ways *easier* to catch (name the compiler warning) while the heap version generally is not. (c) Argue why "the storage is reclaimed whether or not a pointer still refers to it" is the root of *both*.

8.12 LAB On your own machine, compile (with `-Wall`) the buggy `int *f(void){ int x=7; return &x; }` and a `main` that dereferences the result. (a) Record the exact compiler warning. (b) Run it and report what happens on *your* machine (crash, a value, garbage) — and note that this outcome is not guaranteed by the standard. (c) Then write the caller-owned-storage fix and confirm it is warning-free and correct. Report only what you actually observed, and state why (b) must not be relied upon.

Chapter 9 — Manual Allocation: `malloc`/`free` and What Goes Wrong

Before you start: Chapter 8 (stack vs. heap; the dangling-pointer bug; lifetimes), Chapter 7 (pointers, arrays, bounds), Chapter 5 (locality). · **Closes the core of Part 2** and sets up **Rust** (Part 5), where the compiler enforces by rule what this chapter makes you do by hand. Here are the heap's controls — `malloc`, `calloc`, `realloc`, `free` — what the allocator does underneath, and the precise catalogue of ways manual memory management goes wrong, each stated as the undefined behavior it is.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 8 — why is a pointer to a stack local invalid after its function returns, and what one word names a pointer whose target's lifetime has ended? (2) Chapter 7 — pointer arithmetic and indexing are only defined *within* an array (plus one-past-the-end). What is stepping outside those bounds, in the language's terms? (3) *Predict:* you `free(p)` a heap block, then a moment later read `p[0]` anyway. What do you think happens — a guaranteed crash, the old value still there, or something the standard refuses to define? By the end you will be able to name this bug and say precisely why its behavior is not guaranteed.

The heap gives you storage whose lifetime you control — and hands you, in exchange, the entire responsibility for managing it correctly. No garbage collector will reclaim what you forget; no bounds checker will stop you writing past the end; no runtime will notice you using a block after you freed it. C gives you the raw controls and trusts you completely, which is exactly why this chapter matters: the bugs here are not "the program crashes," they are **undefined behavior**, and understanding that distinction precisely is the difference between a mysterious, occasional failure and a controllable one. We will cover the four allocation functions and what each actually promises; sketch what an allocator does underneath (free lists, size classes, fragmentation — at the level this book needs); then catalogue the canonical memory bugs — use-after-free, double-free, memory leak, buffer overflow, uninitialized read — each stated exactly, with why it is dangerous going beyond "it crashes." Finally we meet the real tools that catch them, and run one — AddressSanitizer — on this machine to see what it reports. This is the chapter that most directly motivates Rust: every rule here that you must hold in your head, Rust's type system checks for you.

9.1 The allocation functions and what they promise

Four functions from `<stdlib.h>` make up the manual-allocation interface. Their *documented* behavior (cppreference; the C standard §7.22.3) is precise, and precision matters because the failure modes live in the gaps:

- **`void *malloc(size_t size)`** — allocates `size` bytes and returns a pointer to the block, suitably aligned for any object type. **The contents are uninitialized** — indeterminate bytes, *not* zeroed. Returns `NULL` on failure (and if `size` is 0, returns either `NULL` or a unique pointer you may not dereference). You must check the return value before using it.

- **`void *calloc(size_t n, size_t size)`** — allocates space for `n` objects of `size` bytes each, **initialized to all-zero bytes**. Because it takes the count and element size separately, mainstream implementations guard the `n * size` multiplication against overflow (returning `NULL` rather than silently under-allocating) — which is a practical reason to prefer it over `malloc(n * size)` when allocating an array. Use it when you want zeroed memory or are allocating an array.
- **`void *realloc(void *ptr, size_t size)`** — resizes the block `ptr` previously returned to `size` bytes, preserving the contents up to the smaller of old and new sizes. It *may move* the block (returning a new address), so you must assign its result — `p = realloc(p, n)` loses the block on failure; the safe idiom uses a temporary. On failure it returns `NULL` and leaves the *original* block valid and unchanged. `realloc(NULL, size)` behaves like `malloc(size)`.
- **`void free(void *ptr)`** — returns a block previously obtained from `malloc`/`calloc`/`realloc` to the allocator. `free(NULL)` is explicitly a **no-op** (always safe). Passing anything else — a pointer not from the allocator, or one already freed — is **undefined behavior**.

The contract, stated as rules you must uphold: allocate, check for `NULL`, use only within the requested size, free exactly once when done, and never touch the block after freeing. Every bug in §9.3 below is a violation of one of these — and the language does not enforce a single one of them for you.

> **QUICK CHECK**
>
> Does `malloc` zero the memory it returns? Does `calloc`? And why is `p = realloc(p, n)` a dangerous idiom — what do you lose if the `realloc` fails? (Answer after the next section.)

9.2 What an allocator does, conceptually

Between your `malloc` call and the memory you get back sits the **allocator** — usually part of the C library — managing a large pool of memory it obtained from the OS. You do not need its internals to use it, but a conceptual picture explains the costs and one of the bugs (fragmentation). The allocator's job is to satisfy variable-sized requests, in arbitrary allocate/free order, out of that pool. To do so it tracks which regions are free, typically via **free lists**: linked lists of available blocks it searches to find one big enough for a request. To make that search fast, modern allocators group free blocks into **size classes** — separate lists for "16-byte blocks," "32-byte blocks," and so on — so a request goes straight to a list of right-sized blocks instead of scanning one giant list. When a block is freed, the allocator may **coalesce** it with adjacent free blocks into a larger one, and it records bookkeeping (typically a small header near each block) so `free` knows the block's size.

Two consequences matter for this book. First, **cost**: because a request may involve searching a list, splitting a block, or (when the pool is exhausted) asking the OS for more memory, a heap allocation is far more expensive than the stack's pointer-bump (Ch 8) —

which is why hot loops avoid per-iteration `malloc`. Second, **fragmentation**: after many allocations and frees of varying sizes, the free space can end up chopped into many small, non-adjacent pieces. You may have plenty of total free memory yet be unable to satisfy a large request because no single free block is big enough — this is *external* fragmentation. (There is also *internal* fragmentation: rounding a request up to a size class wastes the slack.) Fragmentation is not undefined behavior — it is a performance and capacity problem — but it is a real cost of manual allocation, and it is why allocation patterns (many small short-lived blocks vs. few large long-lived ones) affect a program's memory footprint. CS:APP (Chapter 9) develops allocator design in depth, including the free-list and coalescing mechanics; this book keeps the model conceptual.

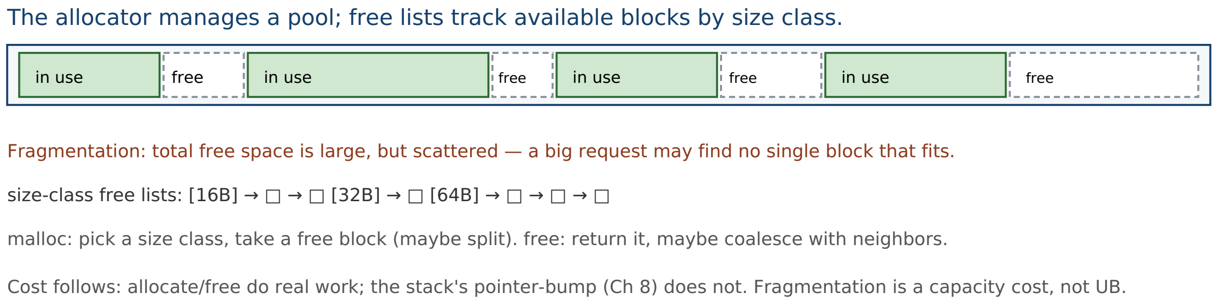


Figure 9.1: An allocator satisfies variable-sized requests from a pool using size-classed free lists, splitting and coalescing blocks. Interleaved allocate/free of varying sizes leaves external fragmentation — free space too scattered to satisfy a large request.

*(Answer to the §9.1 quick check: `malloc` does **not** zero its memory — the contents are indeterminate; `calloc` **does** zero it. `p = realloc(p, n)` is dangerous because if `realloc` returns `NULL` on failure, you have overwritten your only pointer to the still-valid original block — leaking it; assign to a temporary and check it first.)*

9.3 The canonical memory bugs — each stated precisely

Now the catalogue. The critical framing, which this whole book has been building toward: most of these are **undefined behavior (UB)**, a specific term from the C standard (§3.4.3) meaning behavior "for which this International Standard imposes *no requirements*." That is stronger and stranger than "it crashes." Under UB the program may crash, may produce wrong results, may appear to work, may corrupt unrelated data, may do different things on different runs or compilers — and, crucially, the optimizer is permitted to assume UB *never happens*, so code near a latent UB bug can be transformed in surprising ways. It is worth distinguishing three standard categories you will meet:

- **Undefined behavior** (§3.4.3): no requirements at all — anything may happen (e.g. use-after-free, out-of-bounds access, double-free).
- **Unspecified behavior** (§3.4.4): the standard offers two or more possibilities and does not say which you get (e.g. the order function arguments are evaluated) — but each possibility is a valid, non-catastrophic outcome.
- **Implementation-defined behavior** (§3.4.1): unspecified behavior that each implementation must *document* (e.g. `sizeof(int)`, byte order from Ch 7).

The bugs below are firmly in the first category (with uninitialized reads a closely related "indeterminate value" case). Do not read any of the realistic outcomes as guarantees — they are what *tends* to happen, offered only so you recognize the symptom.

Use-after-free. Reading or writing a block after you have `free`d it. The pointer still holds the old address, but the block's lifetime has ended (it may already be handed to another allocation). This is exactly Chapter 8's dangling-pointer bug, now on the heap, and it is **undefined behavior**. Realistically it may read stale data, read data that now belongs to an unrelated allocation, corrupt the allocator's bookkeeping, or crash — and it is a serious security vulnerability class, because an attacker who controls what gets allocated into the freed slot can influence what your dangling pointer reads or writes. This is the answer to the opener: reading `p[0]` after `free(p)` is not a guaranteed crash and not a guaranteed old value — it is UB.

Double-free. Calling `free(p)` twice on the same block (without a reallocation in between). Also **undefined behavior**: the second `free` corrupts the allocator's internal structures (it tries to reclaim a block already on a free list), which can crash immediately, silently corrupt the heap so a *later* unrelated allocation misbehaves, or — again — be exploited. Note `free(NULL)` is *not* a double-free and is always safe; the bug is freeing the same non-null block twice.

Memory leak. Losing the last pointer to a heap block without freeing it, so the memory can never be reclaimed. This one is *not* undefined behavior — the program stays well-defined — but it is a real defect: a long-running program that leaks steadily grows its memory footprint until it exhausts memory and is killed or drags the system into swap. Leaks are the mirror of use-after-free: use-after-free frees too *early* (while a pointer still needs it), a leak frees too *late* (never).

Buffer overflow (out-of-bounds access). Reading or writing outside the bounds of an allocation — `a[4]` on a 4-element array, or writing 20 bytes into a 16-byte block. Recall from Chapter 7 that pointer arithmetic and indexing are only defined within the array (plus one-past-the-end); going past that is **undefined behavior**. An out-of-bounds *write* is especially dangerous: it can overwrite adjacent data, allocator metadata, or (on the stack) a return address — the classic memory-corruption vulnerability behind a large fraction of security exploits. An out-of-bounds *read* can leak adjacent memory (the Heartbleed class of bug).

Uninitialized read. Reading memory before anything has been written to it — a `malloc`'d block (whose contents are indeterminate, §9.1) or an uninitialized local (Ch 8) — and using the value. Using such an **indeterminate value** is at best unspecified and, in general (for objects whose address is never taken, or trap representations), undefined; either way the value is meaningless and the behavior unreliable. The realistic symptom is a bug that changes with optimization level, unrelated code, or run-to-run — because the "value" is whatever bytes happened to be there. The fix is trivial and non-negotiable: initialize before you read (or use `calloc` for zeroed memory).

THE SAME IDEA THREE WAYS: THE MEMORY BUGS ARE LIFETIME/BOUNDS VIOLATIONS

1. In words. Every bug here breaks one of §9.1's rules: touch a block outside its *lifetime* (use-after-free, double-free), outside its *bounds* (buffer overflow), before it is *initialized* (uninitialized read), or never end its lifetime (leak). Most are undefined behavior — no guaranteed outcome.

2. As a picture. A timeline of one block: `malloc` (born, contents garbage) → initialize → use → `free` (dead). Reading before "initialize" = uninitialized read; touching after "dead" = use-after-free; freeing "dead" again = double-free; never reaching "dead" = leak; touching left/right of the block's extent = overflow.

3. As UB, not "a crash." Each undefined case may crash, corrupt, leak data, or seem fine — and may differ by compiler, optimization, or run. The tools in §9.4 turn these unreliable, sometimes-invisible faults into loud, located errors.

TRAP: ASSUMING UNDEFINED BEHAVIOR IS DETERMINISTIC ("IT DIDN'T CRASH, SO IT'S FINE")

The most dangerous misconception in manual memory management is treating a memory bug as if it had a reliable outcome — "the use-after-free returns the old value," "the overflow just writes the next byte," "it ran fine, so it's correct." **Undefined behavior means the standard imposes no requirements** (C §3.4.3): the outcome is not defined, may vary with compiler/optimization/run, and the optimizer may assume the bug cannot occur and transform your code accordingly — so a UB bug can even change behavior elsewhere in the function. A program that "works in testing" with a latent use-after-free or overflow is not correct; it is *unexploded*. The tells: relying on a specific value or crash from a bug; "it only fails in release builds" or "only under load" or "only on the other machine" — all classic UB signatures. The discipline: reason about lifetimes and bounds to *prevent* these (§9.1's rules), and use the sanitizers/Valgrind of §9.4 to *detect* them — never conclude "no crash = no bug." This is precisely the guarantee Rust's type system will provide by construction in Part 5.

9.4 Tools that catch these bugs — and one run on this machine

Because these bugs are unreliable by nature, you do not hunt them by staring; you use tools that instrument memory accesses and turn a silent UB into a loud, located report. Two real, widely used ones:

- **AddressSanitizer (ASan)** — a compiler feature (in gcc and clang) enabled with `-fsanitize=address`. It instruments the program to check every memory access against "shadow memory" and surrounds allocations with *redzones*, catching heap and stack buffer overflows, use-after-free, double-free, and (with its LeakSanitizer component) leaks — reporting the bug's type, address, and source location, with a modest runtime slowdown. It detects *addressability* bugs (touching memory you shouldn't); it does *not* catch reads of uninitialized-but-valid memory — that is MemorySanitizer's job (clang), or Valgrind's.
- **Valgrind (Memcheck)** — a runtime instrumentation tool that runs your unmodified binary on a synthetic CPU, checking each access. It detects use-after-free, invalid frees, leaks, out-of-bounds on heap blocks, and — unlike ASan — use of uninitialized

values, at a larger slowdown. (Valgrind was not installed on this machine, so the runs below use ASan.)

To make this concrete, four tiny buggy programs were compiled with `gcc -fsanitize=address -g` and run on this machine. ASan's actual reports (trimmed to the key lines):

```
/* use-after-free: read p[0] after free(p) */
==ERROR: AddressSanitizer: heap-use-after-free ... READ of size 4
  0x... is located 0 bytes inside of 16-byte region [freed by thread T0 here ...]
SUMMARY: AddressSanitizer: heap-use-after-free  uaf.c:6 in main

/* buffer overflow: a[4] = 7 on a 4-int (16-byte) block */
==ERROR: AddressSanitizer: heap-buffer-overflow ... WRITE of size 4
  0x... is located 0 bytes to the right of 16-byte region [0x..10,0x..20)
SUMMARY: AddressSanitizer: heap-buffer-overflow  oob.c:4 in main

/* double-free: free(p); free(p); */
==ERROR: AddressSanitizer: attempting double-free ...
  [freed by thread T0 here ...]

/* memory leak: malloc(1024), never freed */
==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 1024 byte(s) in 1 object(s) allocated from: ... leak.c:3
```

Every bug was caught, named, and pinned to a source line — the whole point of the tools. For contrast, the **uninitialized read** (printing an uninitialized `int`) ran under ASan *without* an error, printing a garbage value — confirming ASan does not cover that class (you would reach for Valgrind or MemorySanitizer, or catch the simple case at compile time: `gcc -Wall` flags it with `-Wuninitialized`). The lesson is twofold: these tools convert probabilistic, sometimes-invisible UB into deterministic, located failures — run your tests under a sanitizer — and no single tool catches everything, so know which catches which. **Outputs above are from this specific machine and toolchain (gcc 11 on x86-64); exact addresses and wording vary by version.**

> **QUICK CHECK**
>
> Which of these does AddressSanitizer catch: use-after-free, heap buffer overflow, double-free, memory leak, uninitialized read? Which one is the odd one out, and what tool would you use for it? (Answer below the communicate box.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate closes a bug report with "I can't reproduce the crash — it runs fine for me, so I think it's a flaky test." The code has a use-after-free. Cover the paragraph and respond in two or three sentences, using *undefined behavior* and naming a tool.

Model answer. "A use-after-free is undefined behavior, so 'runs fine for me' is exactly what we'd expect some of the time — the standard makes no promise about the outcome, and it can depend on the build, the timing, or what got allocated into the freed slot, which is why it's intermittent rather than flaky. Instead of trying to reproduce by luck, let's build the tests with `-fsanitize=address` (or run under Valgrind); ASan will turn the use-after-free into a deterministic, located error pointing at the exact line. 'Can't reproduce' isn't evidence the bug is gone — with UB, no crash doesn't mean no bug." Naming the behavior as UB and reaching for a sanitizer — rather than accepting "flaky" — is the senior register, and it converts an intermittent mystery into a caught, fixable defect.

*(Answer to the §9.4 quick check: AddressSanitizer catches use-after-free, heap buffer overflow, double-free, and (via LeakSanitizer) memory leaks. The odd one out is the **uninitialized read** — ASan does not detect it; use Valgrind or MemorySanitizer, or catch simple cases with `gcc -Wall` (`-Wuninitialized`).)*

9.5 What to carry forward — and the bridge to Rust

Manual allocation is a contract you uphold by hand. `malloc` gives uninitialized bytes (check for `NULL`), `calloc` gives zeroed bytes (and, on mainstream implementations, checks its $n \times size$ multiply for overflow), `realloc` resizes and may move (assign to a temporary), and `free` returns a block exactly once (`free(NULL)` is a safe no-op). Underneath, the allocator manages a pool with free lists and size classes, splitting and coalescing blocks, at real cost, and can leave the pool fragmented. And the ways it goes wrong form a small, precise catalogue: **use-after-free** and **double-free** (lifetime violations), **buffer overflow** (bounds violation), **uninitialized read** (reading before writing), and **memory leak** (never freeing) — the first four undefined behavior, meaning *no guaranteed outcome*, not "a crash," and the last a defined-but-real defect. AddressSanitizer and Valgrind turn these unreliable faults into located errors, and each covers a different subset — which is why you run your tests under them rather than trusting that "it didn't crash."

Step back and notice the shape of the last three chapters. You learned that a pointer is an address into regioned memory (Ch 7), that objects have lifetimes and a pointer can outlive its target (Ch 8), and that on the heap *you* own every lifetime and every bound — and can violate them, silently, with the language offering no net (Ch 9). Every safe program in C is one where the programmer has, in their head, proven that no pointer is used outside its target's lifetime, no access falls outside its bounds, and every allocation is freed exactly once. That proof obligation is real and it is heavy — the bugs above are what happens when it fails. This is the exact setup for **Rust** (Part 5): its ownership, borrowing, and lifetime system is a way to make the compiler *check that proof* — to enforce, as type rules verified before the program ever runs, the very invariants you just had to maintain by hand. When you reach that Part, the motivation will already be in your hands: you will have felt the weight of the net that Rust removes by building the guarantees into the language.

CAN YOU… WITHOUT LOOKING?

- State what `malloc`, `calloc`, `realloc`, and `free` each do and promise — including which zeroes, which may move, and why `free(NULL)` is safe.
- Sketch what an **allocator** does (free lists, size classes, split/coalesce) and define **fragmentation** — and say why it is a capacity cost, not UB.
- Name and precisely define the five canonical bugs, and say which are **undefined behavior** and which is not.
- Explain why "undefined behavior" is stronger than "it crashes," and why "no crash" is *not* evidence of correctness.
- Say which bugs **AddressSanitizer** catches, which it does not, and what tool covers the gap.
- Explain, in one sentence, how this chapter motivates **Rust** — the compiler enforcing what you had to prove by hand.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does each of `malloc/calloc/realloc/free` promise, and what is the safe `realloc` idiom? (§9.1)
2. **R2.** What does an allocator do underneath, and what is external fragmentation? (§9.2)
3. **R3.** Define use-after-free, double-free, buffer overflow, uninitialized read, and leak — which are UB? (§9.3)
4. **R4.** What precisely does "undefined behavior" mean, and why is "it ran fine" not evidence of correctness? (§9.3)
5. **R5.** Which bugs does AddressSanitizer catch and which does it miss, and how does this chapter set up Rust? (§9.4–9.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 9 (dynamic memory allocation: allocator design, free lists, coalescing, fragmentation, and the common memory bugs). The reference for §9.2's allocator model; the free CMU 15-213 materials cover it.
- **cppreference.com** and the **C standard** — `<stdlib.h>` memory functions (§7.22.3), and the definitions of undefined (§3.4.3), unspecified (§3.4.4), and implementation-defined (§3.4.1) behavior. The authority for what each function promises and what the language leaves undefined.
- **The AddressSanitizer and Valgrind documentation** — the tools' own manuals for what each detects and how to run them. Primary sources; verify flags against your compiler/tool version.
- **The Rust Programming Language ("the book") and the Rustonomicon** (free). Read ahead to ownership and borrowing to see how the invariants this chapter enforced by hand become compiler-checked rules — the payoff Part 5 develops.

Exercises

9.1 WARM-UP In one sentence each: (a) does `malloc` zero its memory? (b) does `calloc`? (c) is `free(NULL)` safe?

9.2 WARM-UP Write the *safe* `realloc` idiom (using a temporary), and explain in one sentence what goes wrong with `p = realloc(p, n);` when the allocation fails.

9.3 WARM-UP Match each bug to its one-line definition: use-after-free, double-free, memory leak, buffer overflow, uninitialized read. Then mark which are undefined behavior.

9.4 CORE Explain precisely why "the use-after-free returns the old value, so it's harmless" is wrong. Use the C standard's definition of undefined behavior, and give two realistic outcomes other than "returns the old value."

9.5 CORE For each snippet, name the bug and state whether it is undefined behavior: (a) `free(p); return p[0];` (b) `int *p = malloc(4*sizeof(int)); p[4] = 0;` (c) `free(p); free(p);` (d) `int *p = malloc(sizeof(int)); printf("%d", *p);` (e) `int *p = malloc(1024); p = NULL;`

9.6 CORE Explain the difference between a *memory leak* and a *use-after-free* in terms of freeing "too late" vs. "too early," and state why exactly one of the two is undefined behavior and the other is not.

9.7 COST (a) Why is a heap allocation much more expensive than a stack allocation (tie to §9.2 and Ch 8)? (b) What is external fragmentation, and why can it make a large `malloc` fail even when total free memory exceeds the request? (c) Is fragmentation undefined behavior? Classify it.

9.8 FADED *Worked, with the last step for you.* Diagnose and fix: `char *dup(const char *s) { char *r = malloc(strlen(s)); strcpy(r, s); return r; }`. Steps 1–3 given; complete step 4.
Step 1 (given): `strcpy` copies the string *including* its terminating `'\0'`, so it writes `strlen(s) + 1` bytes.
Step 2 (given): `malloc(strlen(s))` reserves only `strlen(s)` bytes — one too few.
Step 3 (given): So `strcpy` writes one byte past the end of the block: a heap buffer overflow, which is undefined behavior.
Step 4 (you): Give the corrected allocation size, add the one check the function is missing (what if `malloc` returns `NULL`?), and name the tool from §9.4 that would have caught the original overflow and the line it would point to.

9.9 MIXED For each observation, decide whether the cause is a *use-after-free*, a *buffer overflow*, an *uninitialized read*, a *leak*, or a *non-memory performance issue*, and justify in a line: (a) a long-running server's memory grows without bound until the OOM killer stops it; (b) a test prints a different number each run and only in release builds; (c) an out-of-bounds write corrupts an unrelated variable's value; (d) the program crashes inside `free` with heap-corruption after the same pointer is released twice; (e) a random-access loop over a huge array is slow and huge pages help (careful — one of these is not a memory *bug*).

9.10 MIXED Without running code, decide for each whether it is *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*, drawing on Chapters 7–9: (a) `free(NULL);` (b) allocating in a loop and never freeing; (c) `a[n]` where `a` has exactly `n` elements; (d) reading a `calloc`'d block before writing it; (e) reading a `malloc`'d block before writing it.

9.11 STRETCH The chapter says this material "motivates Rust." (a) State the three proof obligations a C programmer carries for heap memory (lifetime, bounds, free-exactly-once). (b) For each of use-after-free, double-free, and leak, name which obligation it violates. (c) In one or two sentences, describe what it would mean for a *compiler* to check these obligations before the program runs — and why that is more reliable than a sanitizer that checks at runtime.

9.12 LAB On your own machine (if a sanitizer is available), write four tiny programs that each contain exactly one of: a use-after-free, a heap buffer overflow, a double-free, and a leak. Compile each with `-fsanitize=address -g` and run it. (a) Record the exact error type and source line ASan reports for each. (b) Write a fifth program with an uninitialized read and confirm whether ASan flags it; if not, note what tool would. Report only what you actually observed, and label your compiler/tool versions, since the exact output varies.

Chapter 10 — Undefined Behavior & the Optimizing Compiler

Before you start: Chapter 9 (the memory bugs, most of them undefined behavior; the §3.4.3 definition), Chapter 7 (pointers, bounds, and one-past-the-end), Chapter 5 (what the compiler is allowed to reorder). · **Continues Part 2.** Chapter 9 named undefined behavior and told you it was "stronger than a crash." This chapter shows you *why* — how the optimizing compiler actively reasons from the assumption that your program contains no UB, so a latent bug does not merely misbehave where it sits: it lets the compiler transform code elsewhere in surprising, sometimes dangerous ways. We will compile real examples on this machine and read the assembly.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 9 — in the C standard's terms, what does **undefined behavior** mean, and how is it different from *unspecified* and *implementation-defined* behavior? (2) Chapter 7 — indexing a pointer is defined only within an array (plus one-past-the-end for the address, not the dereference); what is stepping outside that? (3) *Predict:* you write `int f(int x) { return x + 1 > x; }`. For *signed* `int`, is `x + 1 > x` always true? What do you think an optimizing compiler emits for this function — a comparison, or the constant `1`? By the end you will be able to read the assembly and say exactly why.

Here is the mental shift this chapter demands. It is tempting to think of C as a "portable assembler": you write operations, the compiler emits the corresponding instructions, and undefined behavior is what happens when those instructions hit a bad state at runtime. That model is wrong, and believing it is the source of a whole category of baffling bugs. The correct model is that **undefined behavior is a promise you make to the compiler** — a precondition it is entitled to assume you never violate — and the optimizer reasons *from* that promise. When you write code that can execute UB, you have not described "what the machine does in the bad case"; you have told the compiler the bad case cannot happen, and it will rewrite your program on that basis. We will make this concrete: the standard's definition and its two milder cousins (§10.1); the single assumption the optimizer runs on (§10.2); a gallery of UB that actually changes the compiled code on this machine, read as assembly (§10.3); and why the language is built this way, plus the tools and flags that defend you (§10.4). This is the intellectual core of why manual memory management (Ch 9) is so heavy, and — again — the exact weight Rust will lift in Part 5.

10.1 Undefined behavior, precisely — and its two milder cousins

Chapter 9 gave the definition; we now lean on it fully. The C standard (C11 §3.4.3) defines **undefined behavior** as behavior "for which this International Standard imposes *no requirements*," and notes that permissible responses range from "ignoring the situation completely with unpredictable results" to "behaving during translation or program execution in a documented manner characteristic of the environment" to "terminating a translation or execution." Read that carefully: "no requirements" and "during translation" together mean the license extends to *compile time* — the compiler, not just the running program, may do anything when UB is possible. Three standard categories, sharpened from §9.3, are worth keeping straight because only the first grants the compiler this license:

- **Undefined behavior** (§3.4.3): no requirements at all. The program has no meaning on the inputs that trigger it. Examples: signed integer overflow, dereferencing a null or dangling pointer, an out-of-bounds access, a data race, reading an object through a pointer of an incompatible type (a strict- aliasing violation).
- **Unspecified behavior** (§3.4.4): the standard gives two or more valid possibilities and does not say which you get — e.g. the order in which a function's arguments are evaluated. Each possibility is a normal, non-catastrophic outcome; the compiler need not document its choice.
- **Implementation-defined behavior** (§3.4.1): unspecified behavior that the implementation must *document* — e.g. the width of `int`, or the byte order (Ch 7). Portable code avoids depending on it; it is not dangerous, merely non-portable.

The distinction that matters for this chapter: unspecified and implementation-defined behavior still produce a *valid program* with *some* defined outcome — you just may not know which. Undefined behavior produces **no valid program at all** on the triggering input. There is no "the value you happen to get"; the standard has stopped describing your program entirely. Everything in §10.2 follows from that single fact.

> **QUICK CHECK**
>
> Which of these is undefined, which unspecified, which implementation-defined: (a) `sizeof(long)`; (b) the evaluation order of `f() + g()`; (c) signed integer overflow? (Answer after the next section.)

10.2 The optimizer's one assumption: UB never happens

An optimizing compiler transforms your code under the **as-if rule**: it may emit any instructions it likes as long as the observable behavior of a *well-defined* program is preserved. The crucial word is *well-defined*. For an input that triggers undefined behavior, the program has no observable behavior the compiler is required to preserve — so the optimizer is free to assume that input never occurs. It does not check; it does not warn (usually); it simply reasons: "this operation would be UB if condition *C* held, and the programmer has promised never to invoke UB, therefore *C* is false — and I may use that fact to simplify everything downstream (and, under the as-if rule, upstream too)."

Two consequences of that reasoning surprise people. First, **the optimizer propagates the assumption**: one operation that is only UB when `p` is null lets the compiler conclude `p` is non-null *everywhere* that value is used, deleting your later null check as provably-dead code. Second, **the effects can appear to "travel" in time**: because a program that executes UB has undefined behavior for the *whole* execution, an optimizer may hoist, sink, or delete work around the UB in ways that make observable effects *before* the buggy line also vanish or change. This is why "it printed the right thing up to the crash" is not something you can rely on. Neither of these is a compiler bug — each is the as-if rule applied to a program whose meaning you forfeited by writing the UB. Lattner's LLVM series and Regehr's guide (Further Reading) walk through the same reasoning at length; the sections below reproduce it on this machine.

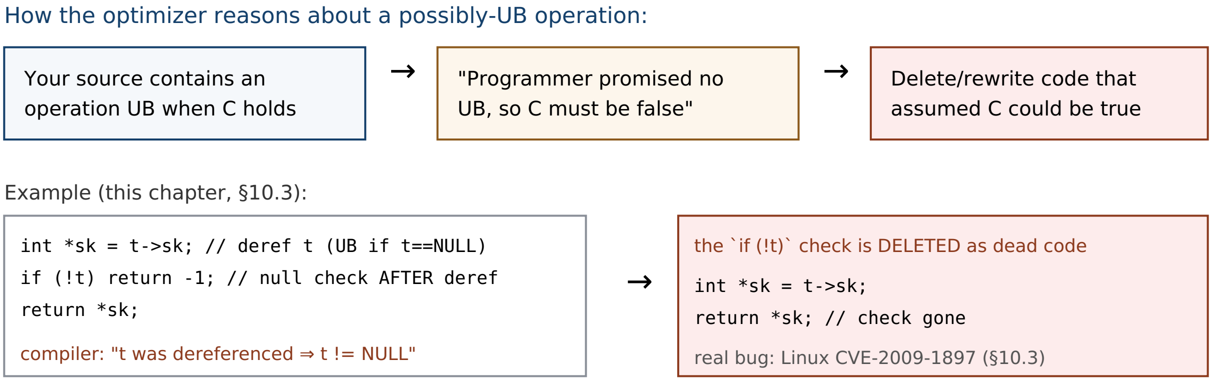


Figure 10.1: The optimizer treats a possibly-UB operation as a promise that the triggering condition never holds, then simplifies on that basis — here deleting a null check because the pointer was already dereferenced. The pattern is the real Linux kernel bug reproduced in §10.3.

(Answer to the §10.1 quick check: (a) `sizeof(long)` is ***implementation-defined*** *— it varies but must be documented; (b) the evaluation order of `f() + g()` is* ***unspecified*** *— one of two valid orders, undocumented; (c) signed integer overflow is* ***undefined behavior*** *— no requirements at all, the case the optimizer exploits.)*

10.3 A gallery of UB that changes the compiled code — read on this machine

Abstract claims about "the optimizer may assume" become believable only when you watch it happen. Each example below was compiled with `gcc 11.4.0` at `-O2` on this x86-64 machine and the relevant assembly read directly. **Exact output varies by compiler and version; these are from this specific toolchain.**

Signed integer overflow (the opener). In C, signed integer overflow is undefined (unsigned overflow is defined and wraps modulo 2^n). So in `int f(int x){ return x + 1 > x; }` the compiler may assume `x + 1` never overflows — and if it never overflows, `x + 1 > x` is *always* true. It compiles the whole function to a constant:

```
/* gcc -O2 -S : int f(int x){ return x + 1 > x; } */
f:
        movl    $1, %eax       ; return 1, unconditionally
        ret
```

No addition, no comparison — just `return 1`. This is correct per the standard and often useful (it lets loops with an `int` counter be optimized as if they cannot wrap). But if you were *relying* on wraparound to detect overflow — the classic `if (x + 1 < x) /* overflowed */` — the check is deleted and your overflow goes undetected. (Compile the same with `-fwrapv`, which *defines* signed overflow as two's-complement wraparound, and the comparison reappears.)

Null check deleted after a dereference (a real CVE). Dereferencing a null pointer is UB, so once a pointer has been dereferenced the compiler may assume it is non-null from then on. Consider the pattern below — a simplified form of the actual Linux kernel function `tun_chr_poll`:

```
struct T { int *sk; };
int poll(struct T *t) {
    int *sk = t->sk;      /* dereference t */
    if (!t) return -1;    /* null check AFTER the dereference */
    return *sk;
}
```

At `-O2`, gcc on this machine emits (the `if (!t)` is *gone* — no test of the pointer against zero):

```
poll:
        endbr64
        movq    (%rdi), %rax   ; sk = t->sk   (dereference t)
        movl    (%rax), %eax   ; return *sk
        ret                    ; no `testq %rdi,%rdi; je ...` -- check deleted
```

Compiled with `-fno-delete-null-pointer-checks`, the guard survives (`testq %rdi,%rdi; je .L3` reappears). This is not a contrived teaching example: it is the shape of **CVE-2009-1897** in the Linux kernel's `drivers/net/tun.c` (kernel 2.6.30). The pointer was dereferenced before the null check; gcc, entitled to assume the dereference could not have been of a null pointer, deleted the check; an attacker able to map memory at address zero could then reach the code that should have been guarded — a privilege-escalation hole. The kernel's fix moved the check before the dereference and, as belt-and-braces, the build adopted `-fno-delete-null-pointer-checks`. The lesson is exact: **a null check that comes after the dereference it is meant to guard is not a weak check — to the optimizer it is a statement that the pointer was already known non-null, and it may be deleted.**

The others you already met (Ch 9), now as UB the optimizer trusts. Out-of-bounds access, use-after-free, and uninitialized reads are all UB; the optimizer is likewise permitted to assume they never happen and to transform code accordingly (for instance, assuming an index stays in range, or that two pointers of incompatible types never alias — the **strict-aliasing** rule, §6.5, which lets the compiler keep a value in a register across a write through an `int*` when it "knows" a `float*` cannot touch the same object; `-fno-strict-aliasing` disables it). The through- line: every UB in Chapter 9's catalogue is not just a runtime hazard but a compile-time license.

THE SAME IDEA THREE WAYS: UB IS A PROMISE, NOT A BEHAVIOR

1. In words. Undefined behavior is a precondition you promise the compiler you will never violate. The compiler does not define "what happens in the bad case"; it assumes the bad case is impossible and optimizes on that assumption. So UB has no outcome to reason about — only consequences the optimizer derives from "this can't happen."

2. As a decision. For an operation that is UB exactly when condition *C* holds, the compiler branches: "either *C* is false (fine, proceed) or *C* is true (UB, undefined — I owe nothing)." Both branches let it act as if *C* is false. So it deletes anything that only mattered when *C* was true — like a null check after a dereference.

3. As compiled code. `x + 1 > x` becomes `return 1;` `if (!t)` after `t->sk` vanishes. The transform is visible in the assembly (§10.3) and correct per the standard — the "surprise" is entirely in the reader's wrong mental model of C as a portable assembler.

TRAP: "IT WORKED AT -O0, SO THE OPTIMIZER BROKE MY CODE"

When a program behaves at `-O0` and misbehaves at `-O2`, the instinct is "the optimizer introduced a bug." Almost always the truth is the reverse: **the undefined behavior was already in your program, and the optimizer merely exercised the license that UB granted it.** At `-O0` the compiler emits naive code that happens to do what you expected; at `-O2` it reasons from "no UB" and the latent bug surfaces. The tells: "only fails in release builds," "only with optimizations on," "adding a `printf` makes it go away" (the print perturbs what the optimizer can prove). The wrong fix is to compile at `-O0` or sprinkle `volatile` until it "works" — that hides UB, it does not remove it, and the next compiler version may re-expose it. The right response is to find and eliminate the undefined behavior: build with `-fsanitize=undefined` (UBSan) and `-Wall -Wextra`, and reason about which operation had no defined meaning. "Works at `-O0`" is not evidence of correctness, exactly as "didn't crash" was not in Chapter 9.

10.4 Why the language is built this way — and how to defend

It is fair to ask why the standard grants this license at all. The answer is a deliberate bargain. Leaving these situations undefined — rather than mandating a checked outcome — is what lets a C compiler generate fast code across wildly different hardware without inserting a bounds check on every array access, an overflow check on every addition, or a null check on every dereference. Signed-overflow UB, for example, lets the compiler promote loop counters, strength-reduce, and assume loops terminate; defined wraparound would forbid some of those. The cost of the bargain is precisely this chapter: the responsibility for never triggering UB shifts entirely to you, and the compiler will not (in general) tell you when you have. Different languages strike the bargain differently — Java defines integer overflow and mandates bounds checks (paying runtime cost for safety); Rust checks bounds and, in debug builds, overflow, and confines the UB-bearing operations to explicitly `unsafe` blocks (Part 5). C chose maximum latitude for the compiler, and hands you the bill.

Given that, your defenses are concrete and layered:

- **Sanitizers.** `-fsanitize=undefined` (UndefinedBehaviorSanitizer, in gcc and clang) instruments the program to detect many UB categories at runtime — signed overflow, out-of-bounds on known-size arrays, null dereference, misaligned access, and more — reporting the file and line. Combined with AddressSanitizer (Ch 9) for memory errors, it turns much silent UB into loud, located failures. Run your tests under both.

- **Warnings.** `-Wall -Wextra` catch a meaningful fraction statically (uninitialized reads, some overflows, format-string mismatches). Free; turn them on and treat them as errors (`-Werror`) where you can.
- **Defining-the-undefined flags.** When you deliberately depend on a behavior the standard leaves undefined, tell the compiler to define it: `-fwrapv` (signed overflow wraps), `-fno-strict-aliasing` (allow type-punning through pointers), `-fno-delete-null-pointer-checks` (keep null checks even after a dereference). These trade some optimization for predictability and are how the Linux kernel, among others, tames the sharpest edges.
- **Write UB-free code.** The durable fix. Check pointers before dereferencing, indices before indexing, and sizes before copying (Ch 11); use unsigned types or explicit overflow-checked arithmetic when you need wraparound or overflow detection; never read before initializing. Every rule from §9.1 is really a rule for staying inside the defined language.

QUICK CHECK

Your code does `if (x + 1 < x)` to detect signed overflow and the check keeps getting optimized away. Name two ways to make the detection actually work — one that changes the flag you compile with, one that changes the type or arithmetic you use. (Answer below the communicate box.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate files a compiler bug: "gcc deleted my null check! Look — I clearly write `if (!t) return -1;` and it's not in the assembly. This is a miscompilation." The code dereferences `t` one line above the check. Cover the paragraph and respond in three or four sentences, using *undefined behavior* and the as-if rule.

Model answer. "It's not a miscompilation — it's the as-if rule doing exactly what the standard permits. You dereference `t` with `t->sk` *before* the check, and dereferencing a null pointer is undefined behavior, so the compiler is entitled to assume `t` was non-null at that point — which makes the later `if (!t)` provably dead code it can delete. This is literally the shape of the Linux kernel bug CVE-2009-1897. The fix isn't to report gcc; it's to move the null check *before* the dereference (and, if we want a belt-and-braces guard, build with `-fno-delete-null-pointer-checks`). A check that comes after the dereference it guards isn't a weak check — to the optimizer it's a promise the pointer was already non-null." Naming the as-if rule and the CVE, rather than blaming the compiler, is the senior register — and it turns a "compiler bug" into a one-line code fix.

*(Answer to the §10.4 quick check: (1) compile with `-fwrapv`, which defines signed overflow as two's-complement wraparound so the comparison is no longer optimized away; or (2) do the arithmetic in an **unsigned** type (defined to wrap) or use a checked-arithmetic*

*builtin such as `__builtin_add_overflow`, which reports overflow without relying on UB. Detecting signed overflow by **letting it happen** and testing afterward is itself UB — that is why the naive check disappears.)*

10.5 What to carry forward

Undefined behavior is not "what the machine does in the bad case." It is a precondition you promise the compiler you will never violate, and the optimizer reasons from that promise: under the as-if rule it may assume every UB-triggering condition is false and rewrite your program accordingly — deleting a null check after a dereference, folding `x + 1 > x` to the constant `1`, keeping a value in a register across a write it "knows" cannot alias. We watched each of these happen in the assembly on this machine. The categories to keep separate are undefined (no requirements — the compiler's license), unspecified (a valid but unstated choice), and implementation-defined (unspecified but documented); only the first grants the license. The language makes this bargain on purpose — it is what lets C compile to fast code across every architecture without safety checks on every operation — and it hands you the whole cost. Your defenses are UBSan and ASan for detection, `-Wall -Wextra` for the static fraction, the define-the-undefined flags (`-fwrapv`, `-fno-strict-aliasing`, `-fno-delete-null-pointer-checks`) when you must depend on an edge, and — above all — writing code that never triggers UB.

This chapter closes the loop opened in Chapter 9. There we cataloged the manual-memory bugs and called most of them UB; here you saw why that word carries so much weight — UB is what makes those bugs unpredictable, build-dependent, and, as CVE-2009-1897 shows, exploitable. It also sets up the next chapter: the buffer overflow of Chapter 9 is UB, and Chapter 11 follows exactly what an out-of-bounds write can do to a program's control flow — the single most consequential UB in the history of security. And it is the sharpest possible motivation for Rust (Part 5): a language whose safe subset has *no* undefined behavior, confining every UB-bearing operation to code you must mark `unsafe`, so the promises this chapter made you keep by hand are enforced by the type system before the program ever runs.

CAN YOU… WITHOUT LOOKING?

- State the C standard's definition of **undefined behavior** and distinguish it from **unspecified** and **implementation-defined** behavior.
- Explain the **as-if rule** and why it lets the optimizer assume UB-triggering conditions are false — including how an assumption "propagates" and can affect code before the buggy line.
- Walk through why `int f(int x){ return x + 1 > x; }` can compile to `return 1`, and why a null check *after* a dereference can be deleted (CVE-2009-1897).
- Explain why "it worked at `-O0`" and "adding a `printf` fixed it" are UB signatures, not evidence of correctness.
- Name four defenses (UBSan, warnings, define-the-undefined flags, UB-free code) and say what each buys.
- Say, in one sentence, why the language leaves these behaviors undefined at all — the optimization/safety bargain — and how Rust strikes it differently.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Define undefined, unspecified, and implementation-defined behavior, and say which grants the compiler its license. (§10.1)
2. **R2.** State the as-if rule and explain how "the optimizer assumes UB never happens" leads to deleting code. (§10.2)
3. **R3.** Why can `x + 1 > x` become `1`, and how does `-fwrapv` change that? (§10.3–10.4)
4. **R4.** Describe CVE-2009-1897: what pattern caused it, why the check was deleted, and the two-part fix. (§10.3)
5. **R5.** Name the bargain the language makes and four defenses against UB. (§10.4)

FURTHER READING

- **Chris Lattner, "What Every C Programmer Should Know About Undefined Behavior" (LLVM Project Blog, 2011, three parts).** The optimizer-writer's own account of how and why the compiler exploits UB, with worked examples (null checks, signed overflow, strict aliasing). Free.
- **John Regehr, "A Guide to Undefined Behavior in C and C++" (Embedded in Academia, 2010, three parts).** A researcher's catalogue of UB and its interaction with aggressive compilers, including the "UB can time-travel" framing. Free (`blog.regehr.org`).
- **The C standard** — §3.4.1/3.4.3/3.4.4 (the three behavior categories), §6.5 (aliasing), and the integer-overflow rules. The authority for what is undefined; **cppreference.com** summarizes it accessibly.
- **UndefinedBehaviorSanitizer (UBSan) documentation** (Clang/GCC). The runtime detector for many UB categories; verify the exact checks and flags against your compiler version.

Exercises

10.1 WARM-UP In one sentence each, define *undefined*, *unspecified*, and *implementation-defined* behavior, and state which one grants the optimizer license to assume the situation never happens.

10.2 WARM-UP Is *unsigned* integer overflow undefined behavior in C? Is *signed* integer overflow? Answer each and, for the defined one, say what value results.

10.3 WARM-UP A program prints correct output at `-O0` but wrong output at `-O2`. State the single most likely explanation, and name the two build flags you would add to hunt for it.

10.4 CORE Explain, step by step, why `int f(int x){ return x + 1 > x; }` can be compiled to `return 1`. Then explain what breaks if a programmer used `if (x + 1 < x)` to detect overflow, and give a correct way to detect it.

10.5 CORE Given `int *sk = t->sk; if (!t) return -1; return *sk;`: (a) name the UB, (b) explain why an optimizer may delete the `if (!t)` check, (c) give the one-line code fix, and (d) name the flag that would preserve the check without changing the code. This is a real CVE — name the pattern.

10.6 CORE "The compiler has a bug — my code is fine, it just miscompiled." A teammate says this about the §10.5 null- check example. Explain, using the as-if rule, why the compiler is behaving correctly and the code is not.

10.7 COST The standard *could* have defined all of these behaviors (mandating bounds checks, overflow checks, null checks). (a) What runtime cost would that impose? (b) Name one specific optimization that signed-overflow UB enables. (c) State the bargain in one sentence: what the programmer gives up and what the compiler gains.

10.8 FADED *Worked, with the last step for you.* A teammate reports: "My bounds check `if (i < 0 || i >= n)` works, but after I access `a[i]` *first* for logging and check `i` afterward, the check stopped firing." Diagnose and fix.
Step 1 (given): Accessing `a[i]` with `i` out of range is an out- of-bounds access — undefined behavior.
Step 2 (given): Once `a[i]` has executed, the compiler may assume `i` was in range (else UB), i.e. `0 <= i < n`.
Step 3 (given): Therefore the later `if (i < 0 || i >= n)` is provably false and can be deleted — same shape as the null-check CVE.
Step 4 (you): State the fix (what must come before what), name one sanitizer that would have flagged the original out-of-bounds access at runtime, and say why moving the log line does not merely "reorder output" but changes what the compiler is allowed to prove.

10.9 MIXED For each, decide whether it is *undefined*, *unspecified*, or *implementation-defined*, and justify in a line: (a) the value of `sizeof(int)`; (b) the order `a()` and `b()` are called in `a() + b()`; (c) `INT_MAX + 1`; (d) right-shifting a negative signed integer; (e) reading a `malloc`'d block before writing it. (One of these is a §9.3 memory bug; recognizing that is the point.)

10.10 MIXED A colleague proposes three "fixes" for an intermittent bug that only appears in release builds: (a) always compile at `-O0`; (b) mark the variables `volatile` until it stops; (c) build the tests with `-fsanitize=undefined,address` and fix whatever it reports. Rank them from least to most effective and say, for each, whether it removes the bug or merely hides it.

10.11 STRETCH The chapter says a UB assumption can "propagate" and even affect code *before* the buggy line. (a) Explain why, using the definition of UB as applying to the whole program execution. (b) Give a concrete scenario where deleting a downstream null check (because of an upstream dereference) turns a would-be crash into a silent security hole. (c) Argue why a *static* guarantee (Rust's safe subset having no UB) is qualitatively stronger than a runtime sanitizer here.

10.12 LAB On your own machine, compile `int f(int x){ return x + 1 > x; }` with `gcc -O2 -S` and read the assembly; then recompile with `-fwrapv` and compare. Do the same for the §10.5 `poll` function with and without `-fno-delete-null-pointer-checks`. (a) Record what changed in each case. (b) Write a tiny program with a signed-overflow bug and confirm whether `-fsanitize=undefined` reports it. Report only what you observed and label your compiler/version, since output varies.

Chapter 11 — Arrays, Strings & Buffer Safety

> **Before you start:** Chapter 10 (a buffer overflow is undefined behavior — and the optimizer trusts it never happens), Chapter 9 (the buffer-overflow bug, stated precisely), Chapter 8 (the stack, frames, and the return address that sits on it), Chapter 7 (arrays, pointers, and array-to-pointer decay). · **Continues Part 2.** This chapter takes the buffer overflow — Chapter 9's most consequential bug — and follows it all the way to its worst outcome: overwriting a return address and seizing a program's control flow. Then it covers the C string model that makes overflows so easy, the string-function minefield, and the layered defenses real systems deploy — measuring each on this machine.

> **BEFORE YOU READ ON**
>
> From memory, reaching back: (1) Chapter 8 — a function's local array and its return address both live in the same stack frame; which direction does the stack grow, and which of those two is at a *higher* address on a typical downward-growing stack? (2) Chapter 9 — an out-of- bounds *write* was called "especially dangerous." Why more dangerous than an out-of-bounds read? (3) *Predict:* you replace an unsafe `strcpy(dst, src)` with `strncpy(dst, src, n)` where `n` is exactly the size of `dst`, and `src` is `n` characters long. Do you think the result is a valid, null-terminated C string? We will compile exactly this and read the bytes.

Almost every catastrophic memory-corruption vulnerability in the history of software traces to the same root: a program wrote past the end of a buffer. Chapter 9 named it and called it undefined behavior; Chapter 10 showed why UB is dangerous in the abstract. This chapter makes it visceral, because the buffer overflow is not merely "corrupt some adjacent bytes" — on the stack, the bytes adjacent to a local array include the saved return address, and overwriting *that* hands an attacker control of what the CPU executes next. We will build up to it deliberately: why the C string is a fragile convention rather than a real type (§11.1); the anatomy of a stack buffer overflow and why it is exploitable (§11.2); the string-function minefield and the sized functions that replace it — including the trap that `strncpy` is *not* the safe `strcpy` (§11.3); the layered defenses (stack canaries, `_FORTIFY_SOURCE`, ASLR, NX/DEP) and precisely what each does and does not guarantee, run on this machine (§11.4); and the disciplines that actually prevent the bug (§11.5). Throughout, the refrain from Chapters 9 and 10 holds: the bug is UB, so a program that "runs fine in testing" may be one crafted input away from disaster.

11.1 The C string is a convention, not a type

C has no string type. What C calls a string is a **convention**: a contiguous array of `char` terminated by a null byte `'\0'` (a zero byte, value 0). The length is not stored anywhere — it is *implied* by the position of the terminator, which is why `strlen` is an O(n) scan that walks bytes until it finds the zero. This one design decision is the source of a startling share of C's danger. Because the length is not carried with the data, every function that touches a string must either be told the buffer's capacity or trust that a terminator exists within bounds — and when that trust is misplaced, the read or write runs off the end. Recall from Chapter 7 that an array decays to a pointer to its first element the moment you pass it to a function: the callee receives an address with *no length information at all*.

A `char *` tells you where a string starts; it tells you nothing about how much room there is. Bounds safety in C is therefore never automatic — it is something the programmer must reconstruct by hand, by tracking capacities alongside pointers.

Two consequences follow immediately. First, **a missing terminator is a bug waiting to happen**: a "string" with no `'\0'` in bounds makes `strlen`, `printf("%s")`, and every other string function read past the end — an out-of-bounds read (UB), which can leak adjacent memory (the Heartbleed class, Ch 9). Second, **a copy into an undersized buffer overflows it**: writing an *n*-plus-one-byte string (n characters plus the terminator) into an *n*-byte buffer writes one byte past the end — an out-of-bounds write (UB), the subject of the rest of this chapter. The C string's elegance (a string is just bytes) and its danger (a string is *just* bytes, with no length and no bounds) are the same fact.

> **QUICK CHECK**
>
> Why is `strlen` an O(n) operation rather than O(1)? And when you pass a `char buf[64]` to a function, how much does the function know about `buf`'s capacity? (Answer after the next section.)

11.2 The stack buffer overflow — anatomy and why it is exploitable

Recall the stack frame from Chapter 8. When a function runs, its local variables — a `char buf[8]`, say — live in its stack frame, and (on the common downward-growing stack of x86-64 and most architectures) the frame also holds the **saved return address**: the address the CPU will jump to when this function returns. Critically, the return address sits at a *higher* address than the local buffer, while a copy into the buffer writes toward *higher* addresses. So a write that overruns `buf` marches straight toward the saved return address. Overflow it by enough bytes and you overwrite the return address itself. Now the picture is stark: when the function returns, the CPU loads that overwritten value into the instruction pointer and jumps *wherever the overflowing bytes told it to*. An attacker who controls the input controls where the program goes next.

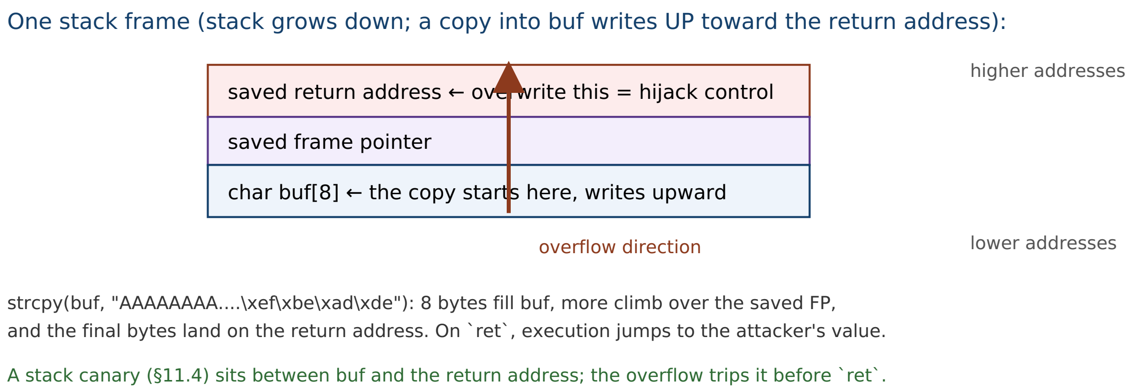


Figure 11.1: A stack buffer overflow. Because the saved return address sits above the local buffer and a copy writes upward, overrunning the buffer can overwrite the return address, redirecting control flow when the function returns. A stack canary is placed in the gap to detect exactly this.

This is not a hypothetical; it is the mechanism behind decades of exploits, laid out for the first time in a widely read form by **Aleph One's "Smashing the Stack for Fun and Profit"** (Phrack 49, 1996) and developed rigorously in CS:APP (§3.10). The classic exploit places the attacker's own machine code ("shellcode") in the buffer and overwrites the return address to point at it; modern mitigations (§11.4) have made that specific technique much harder, but the fundamental hazard — an out-of-bounds write reaching control data — remains, and newer techniques (return-oriented programming) route around the defenses. To make it concrete, this program overflows an 8-byte buffer:

```
#include <string.h>
int main(void){
    char buf[8];
    strcpy(buf, "this string is much too long for the buffer");  /* 44 bytes into 8 */
    return buf[0];
}
```

Compiled with stack protection on this machine (`gcc -O0 -fstack-protector-all`) and run, it does *not* silently corrupt and continue — the canary catches it (§11.4). gcc even warns at compile time before you run it:

```
warning: '__builtin_memcpy' writing 44 bytes into a region of size 8 overflows the
destination
         [-Wstringop-overflow=]
$ ./a.out
*** stack smashing detected ***: terminated
Aborted (core dumped)          # exit status 134 (SIGABRT)
```

Without those defenses — an older toolchain, or a copy whose length the compiler cannot see — the same overflow would have corrupted the return address silently. **Outputs are from this machine (gcc 11.4.0, x86-64); wording and status vary by toolchain.**

TRAP: "STRNCPY IS THE SAFE STRCPY"

The most common half-fix in C is to reach for `strncpy(dst, src, n)` believing the `n` makes it safe. It bounds the write, which prevents the overflow — but **`strncpy` does not guarantee a null terminator**. Its actual contract (cppreference; the C standard): it copies at most `n` bytes; if `src` is shorter than `n` it pads the rest with zeros, but if `src` is *n* or more bytes long it writes exactly `n` bytes and stops — leaving `dst` **unterminated**. The next `strlen` or `printf("%s")` then runs off the end: you traded a bounded overflow for an unbounded over-read. On this machine, `char dst[4]; strncpy(dst, "abcd", 4);` leaves the four bytes `97 98 99 100` (`'a' 'b' 'c' 'd'`) with *no* terminating zero — a "string" that is not a string. `strncpy` was designed in the 1970s for fixed-width records (like early UNIX directory entries), *not* as a safe string copy, and using it as one is a classic trap. If you use it, you must terminate by hand (`dst[n-1] = '\0';`) — or, better, use `snprintf` (§11.3), which always terminates.

*(Answer to the §11.1 quick check: `strlen` is O(n) because a C string stores no length — it must scan byte by byte until it finds the terminating `'\0'`. And a function receiving `char buf[64]` knows **nothing** about the capacity: the array decayed to a bare `char *`, an address with no length attached — you must pass the size separately.)*

11.3 The string-function minefield — and the sized alternatives

The classic C string functions are unsafe by construction because they take no destination size: they write until they hit a terminator in the *source*, with no regard for the *destination*'s capacity. The worst offenders and their safer replacements:

- **`gets(char *s)`** — reads a line into `s` with *no* bound whatsoever. It was so irredeemably unsafe that it was deprecated in C99 and **removed from the language in C11**. Never use it; use `fgets(s, size, stdin)`, which takes the buffer size.
- **`strcpy(dst, src)` / `strcat(dst, src)`** — copy/append with no destination bound; overflow `dst` whenever `src` is too long. Prefer `snprintf` (below), or bounded copies where you control termination.
- **`sprintf(dst, fmt, ...)`** — formats into `dst` with no bound; a long formatted result overflows. Replace with **`snprintf(dst, size, fmt, ...)`**, which writes at most `size` bytes *including* the terminator, always null-terminates (when `size > 0`), and returns the length it *would* have written — so you can even detect truncation.
- **`strncpy` / `strncat`** — bounded but treacherous: `strncpy` may not terminate (the §11.2 trap), and `strncat`'s size argument is the number of bytes to append, not the buffer size, which readers routinely get wrong. Usable, but only with care and a manual terminator.

The reliable pattern for building a string safely is `snprintf`: it is bounded, always terminates, and reports truncation. When you truly need a raw copy, carry the capacity and check it — `if (srclen + 1 <= dstcap) memcpy(dst, src, srclen + 1);` — the same "reconstruct the length by hand" discipline §11.1 demanded. (Some platforms offer `strlcpy`/`strlcat`, which always terminate and return the source length; they are widely available but are *not* in the C standard, so their portability must be checked. The standard's own `_s` "bounds-checking" functions, Annex K, exist but are optional and unevenly implemented — do not assume them.)

> **QUICK CHECK**
>
> Name the one standard string function that was *removed* from C11 for being unsafe, and its safe replacement. And what does `snprintf` guarantee that `strncpy` does not? (Answer below the communicate box.)

11.4 Defense in depth — canaries, FORTIFY, ASLR, NX — and what each buys

Because buffer overflows are so consequential, modern toolchains and operating systems layer several *mitigations*. It is essential to understand what each actually guarantees, because none of them *removes* the bug — they raise the cost of exploiting it, and each defeats a specific technique while leaving others open. Four you will meet, three demonstrated on this machine:

- **Compiler bounds warnings.** With optimization and `-Wall`, gcc's `-Wstringop-overflow` can catch overflows it can see at compile time. On this machine the §11.2 program drew `warning: '__builtin_memcpy' writing 44 bytes into a region of size 8 overflows the destination` — a static catch, but only when the sizes are visible to the compiler.

- **`_FORTIFY_SOURCE`.** Building with `-D_FORTIFY_SOURCE=2 -O2` makes glibc substitute checked variants (`__strcpy_chk`, `__memcpy_chk`, …) that know the destination size and abort on overflow — catching many cases at compile time *or* runtime. On this machine, fortifying `char buf[4]; strcpy(buf, "hello");` produced `warning: '__builtin___strcpy_chk' writing 6 bytes into a region of size 4`. It only helps where the size is known and requires optimization; it is not universal.
- **Stack canaries** (`-fstack-protector` / `-fstack-protector-all`). The compiler places a random "canary" value between the local buffers and the saved return address (Figure 11.1) and checks it just before returning; if an overflow overwrote it, the program aborts instead of returning to corrupted control data. On this machine, the §11.2 overflow tripped it: `*** stack smashing detected ***: terminated`, exit status 134. Canaries defeat the classic "overwrite the return address by linear overflow" attack — but not overwrites that skip the canary, or non-control-data corruption, and they act *after* the corruption (they convert exploitation into a crash, i.e. a denial of service, not into "no bug").
- **ASLR and NX/DEP** (OS-level). **Address Space Layout Randomization** randomizes the base addresses of the stack, heap, and libraries each run, so an attacker cannot easily predict where to jump. **NX/DEP** (the no-execute page permission, tied to virtual memory from Chapter 4) marks the stack and heap non-executable, so injected shellcode cannot run as code. Together they defeat the original "inject shellcode on the stack and jump to it" recipe — which is why real attacks moved to *return-oriented programming*, chaining existing code fragments rather than injecting new ones. These are runtime/OS defenses, not compiler ones, and again they raise cost rather than eliminate the bug.

The load-bearing point: **every one of these is a mitigation, not a fix.** They assume the overflow exists and try to contain its blast radius — turning a silent hijack into a crash, or a reliable exploit into an unreliable one. The bug is still there, still undefined behavior (Ch 10), and a determined attacker or an unlucky input may route around any single layer. The only true fix is to not write out of bounds in the first place (§11.5) — or to use a language that enforces bounds by construction (Part 5).

THE SAME IDEA THREE WAYS: A BUFFER OVERFLOW IS A BOUNDS VIOLATION WITH CONSEQUENCES

1. In words. Writing past a buffer's end (§9.3's bounds violation) is undefined behavior; on the stack the bytes just past a local array include control data — the saved return address — so the overflow can redirect execution, not merely corrupt data.

2. As a picture. Figure 11.1: `buf` low, saved frame pointer above it, return address above that; a copy writes upward, so an overrun climbs over the frame pointer onto the return address. A canary sits in the gap to catch the climb.

3. As UB the compiler and OS fight in layers. The write is UB (Ch 10) — no defined outcome. Compiler warnings and `_FORTIFY_SOURCE` catch some cases; canaries turn a hijack into a crash; ASLR and NX defeat specific exploit recipes. None removes the UB; only bounds-correct code does.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. In a review, a colleague writes: "I turned on `-fstack-protector`, so our buffer overflows are handled — we don't need to fix the `strcpy` calls." Cover the paragraph and respond in three or four sentences, distinguishing *mitigation* from *fix*.

Model answer. "A stack canary is a mitigation, not a fix: it detects a linear stack overflow just before the function returns and aborts, so it converts a control-flow hijack into a crash — a denial of service, still a real incident, and it does nothing for heap overflows, non-control-data corruption, or overwrites that skip the canary. The overflow is still there, still undefined behavior, so the compiler is free to do surprising things around it, and a canary that fires in production is an outage. We should keep the canary *and* ASLR *and* `_FORTIFY_SOURCE` as defense in depth, but the actual fix is to remove the unbounded copies — replace `strcpy` with `snprintf` or a capacity-checked `memcpy` — so the out-of-bounds write cannot happen." Naming the distinction — mitigation contains blast radius, only bounds-correct code removes the bug — is the senior register, and it keeps the defenses without treating them as a license to leave the bug in.

(Answer to the §11.3 quick check: `gets` was removed from C11; its safe replacement is `fgets`, which takes the buffer size. And `snprintf` always null-terminates its output (when the size is nonzero) and reports truncation, whereas `strncpy` may leave the destination with no terminator at all.)

11.5 What to carry forward

The C string is a convention — a null-terminated byte array with no stored length — and that single fact is why bounds safety in C is manual and why buffer overflows are so easy. An out-of-bounds write is undefined behavior (Ch 9, Ch 10), and on the stack it is the most consequential UB in security: the saved return address lives just past a local buffer, so an overrun can overwrite it and redirect control flow — the mechanism Aleph One documented and CS:APP develops. The classic string functions (`gets`, `strcpy`, `strcat`, `sprintf`) take no destination size and are unsafe by construction; `gets` was removed from C11 outright. The sized replacements — `fgets`, `snprintf`, capacity-checked `memcpy` — are safe *when you carry the capacity and check it*, and `strncpy` is a trap because it can leave a string unterminated. Layered defenses (compiler warnings, `_FORTIFY_SOURCE`, stack canaries, ASLR, NX/DEP) each raise the cost of exploitation and we watched three of them fire on this machine — but every one is a mitigation that contains the blast radius, not a fix that removes the bug.

The discipline that actually prevents the bug is the same one Chapters 7–10 kept circling: a bare pointer carries no bounds, so *you* must carry them — keep the capacity beside every buffer, check every copy against it, prefer sized APIs, and never trust that a terminator is present. That is a proof obligation, on every string operation, that the programmer discharges by hand and that no amount of mitigation discharges for you. It is also the cleanest possible statement of what a safer language buys: in Rust (Part 5) a string and a slice *carry their length*, indexing is bounds-checked, and an out-of-bounds access is a defined panic rather than undefined memory corruption — the "carry the length by hand" discipline of this chapter, promoted into the type system. Part 2 has now taken you from "a pointer is an address" (Ch 7) to "and you own every lifetime, every bound, and every promise to the compiler" (Ch 8–11). The final chapter turns the last of those responsibilities — allocation — from a hazard you survive into a tool you wield.

CAN YOU… WITHOUT LOOKING?

- Explain why a C string is "a convention, not a type," and why that makes `strlen` O(n) and bounds safety manual.
- Draw a stack frame and explain why a buffer overflow can overwrite the **saved return address** and hijack control flow.
- State why `strncpy` is *not* the safe `strcpy` — the exact contract that bites — and name a genuinely safe alternative.
- Name the four classic unsafe string functions and their sized replacements, and which one C11 removed.
- Describe stack canaries, `_FORTIFY_SOURCE`, ASLR, and NX/DEP, and say for each what it buys and why it is a mitigation, not a fix.
- State the one discipline that actually prevents overflows, and how Rust promotes it into the type system.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a C string, physically, and why does it store no length? (§11.1)
2. **R2.** Why can a stack buffer overflow hijack control flow — what sits next to the buffer, and in which direction does a copy write? (§11.2)
3. **R3.** State `strncpy`'s exact contract and the trap it creates; give a safer function. (§11.2–11.3)
4. **R4.** For canaries, `_FORTIFY_SOURCE`, ASLR, and NX/DEP, what does each defeat, and why is each a mitigation not a fix? (§11.4)
5. **R5.** What single discipline prevents overflows, and how does Rust encode it? (§11.5)

FURTHER READING

- **Aleph One, "Smashing the Stack for Fun and Profit," Phrack 49 (1996).** The first widely read, step-by-step account of stack buffer overflow exploitation; historically pivotal. Free (`phrack.org`). Read it for the mechanism, not as current exploit practice (the mitigations of §11.4 postdate it).
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), §3.10.** Buffer overflows, stack smashing, and the mitigations (canaries, ASLR, NX) developed rigorously with the stack-frame model of Chapter 8. The reference for this chapter.
- **cppreference.com** and the **C standard** — the exact contracts of `strcpy`, `strncpy`, `strcat`, `snprintf`, `fgets`, and the removal of `gets` in C11. The authority for what each function promises.
- **Beej's Guide to C.** Free (`beej.us`). Practical, readable coverage of C strings and the safe idioms.

Exercises

11.1 WARM-UP In one sentence each: (a) what physically marks the end of a C string? (b) why is `strlen` O(n)? (c) when you pass an array to a function, what does the function learn about its capacity?

11.2 WARM-UP Which standard string function was *removed* from the language in C11 for being unsafe, and what should you use instead?

11.3 WARM-UP True or false, with a one-line reason each: (a) `strncpy(dst, src, n)` always produces a null- terminated string. (b) A stack canary removes the buffer-overflow bug. (c) `snprintf` always null- terminates when its size argument is nonzero.

11.4 CORE Explain, using a stack-frame diagram in words, why an out-of-bounds *write* into a local array can hijack a program's control flow but an out-of-bounds *read* (usually) cannot. What lies just past the buffer, and in which direction does a copy write?

11.5 CORE For each, state whether it can overflow the destination and why, and give a safe rewrite: (a) `strcpy(buf, user_input);` (b) `sprintf(buf, "%s-%s", a, b);` (c) `char d[4]; strncpy(d, "abcd", 4); printf("%s", d);`

11.6 CORE A colleague argues: "We enabled stack canaries and ASLR, so buffer overflows are handled." Give three distinct reasons this is wrong — each naming something a canary or ASLR does *not* prevent — and state what the actual fix is.

11.7 COST (a) Why is `strlen` O(n) while a length-carrying string type (as in Rust or C++ `std::string`) makes "get the length" O(1)? (b) What runtime cost do stack canaries and `_FORTIFY_SOURCE` add, and what do you get for it? (c) Is a canary check on the hot path free? Classify the trade-off (safety vs. speed).

11.8 FADED *Worked, with the last step for you.* Diagnose and fix: `void greet(const char *name){ char msg[16]; strcpy(msg, "Hello, "); strcat(msg, name); puts(msg); }`.
Step 1 (given): `"Hello, "` is 7 characters plus a terminator, using 8 of the 16 bytes.
Step 2 (given): `strcat` appends `name` with no bound; if `name` is longer than 7 characters the total exceeds 16 bytes.
Step 3 (given): So a long `name` overflows `msg` — an out-of- bounds write, undefined behavior, and (on the stack) a potential control-flow hijack.
Step 4 (you): Rewrite the function using `snprintf` so it can never overflow regardless of `name`'s length; state what `snprintf` guarantees that `strcat` does not; and name one §11.4 defense that would turn the original overflow into a crash rather than a silent hijack.

11.9 MIXED For each symptom, name the most likely cause — *unterminated string (over-read)*, *buffer overflow (over-write)*, *use-after-free* (Ch 9), or *a mitigation firing* — and justify in a line: (a) a program prints a corrupted string with garbage bytes appended after the expected text; (b) a service aborts with `*** stack smashing detected ***` on a long request; (c) reading a freed heap block returns another request's data; (d) `printf("%s", s)` reads far past where the data should end and leaks adjacent memory.

11.10 MIXED Classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*, drawing on Chapters 9–11: (a) `strncpy(d, s, sizeof d)` then reading `d` as a string, where `strlen(s) >= sizeof d`; (b) `snprintf(d, sizeof d, "%s", s)` then reading `d`; (c) writing one byte past a heap block; (d) a canary aborting the process on a detected overflow.

11.11 STRETCH The chapter calls the mitigations "defense in depth." (a) Explain what each of stack canaries, NX/DEP, and ASLR defeats, and name the exploit technique (return-oriented programming) that was developed to route around NX. (b) Argue why layering mitigations is still worthwhile even though none is a fix. (c) Explain what it means for Rust to make the bug "not happen" rather than "not exploit" — and why a bounds-checked panic is categorically different from undefined memory corruption.

11.12 LAB On your own machine, write the §11.2 8-byte-buffer overflow. (a) Compile with `-fstack-protector-all`, run it, and record the exact message and exit status. (b) Compile the same with `-D_FORTIFY_SOURCE=2 -O2` and record any compile-time diagnostic. (c) Write `char d[4]; strncpy(d, "abcd", 4);` and print the four bytes as integers to confirm whether a terminator is present. Report only what you observed and label your compiler/version, since output varies.

Chapter 12 — Custom Allocators: Arenas, Pools & Free Lists

Before you start: Chapter 9 (what the allocator does — free lists, size classes, fragmentation, cost), Chapter 10 (a use-after-free is undefined behavior, whoever owns the memory), Chapter 5 (mechanical sympathy: cost follows layout and access pattern). · **Closes Part 2.** Chapter 9 treated the general-purpose allocator as a fact of the machine and cataloged the ways manual memory goes wrong. This chapter turns allocation from a hazard you survive into a tool you wield: when your allocation *pattern* is known, a custom allocator — an arena, a pool, a free list — can be dramatically faster than `malloc`, and we will build and measure one on this machine. It also completes the arc of the part: even your own allocator does not relax a single rule from Chapters 9–11.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 9 — why is a heap `malloc` far more expensive than the stack's pointer-bump, and what two things (a search structure and a per-block record) does the allocator maintain? (2) Chapter 9 — what is *external fragmentation*, and why can it make a large allocation fail even when total free memory is ample? (3) *Predict:* suppose you allocate a million small objects that all die at the same moment. If you could replace a million individual `free` calls with a *single* operation that reclaims them all at once, roughly how much faster do you think freeing would be? We will measure exactly this.

Chapter 9's allocator is a marvel of generality: it satisfies any size, in any order, at any time, and it is the right default for the overwhelming majority of code. But generality has a price — every allocation may search a free list, split a block, and update bookkeeping; every free may coalesce; and interleaved requests fragment the pool. When you know something specific about *how* your program allocates — that many objects share a lifetime, or that they are all the same size — you can trade the general allocator's flexibility for speed and simplicity by writing an allocator specialized to that pattern. This chapter builds the two most useful specializations: the **arena** (also called a region or bump allocator), which makes allocation a pointer increment and frees everything at once (§12.2), and the **pool** or free-list allocator for fixed-size objects (§12.3). We start by asking precisely when the general allocator loses (§12.1), build and measure an arena on this machine, then close with an honest accounting of when a custom allocator is worth it — and the crucial fact that it does not lift the burden of Chapters 9–11 (§12.4–12.5). This is the capstone of "manual memory": having felt its hazards, you now use it deliberately.

12.1 Why not just `malloc`? — allocation patterns

The general-purpose allocator is optimized for the case where it knows *nothing* about your requests. That ignorance is what costs you. Three patterns where a specialized allocator can do far better, precisely because it exploits knowledge the general allocator lacks:

- **Many objects, one lifetime.** A request handler, a compiler pass, a frame of a game — all allocate a burst of objects that become garbage together at a known moment.

The general allocator makes you `free` each one individually (and track every pointer to do so); an allocator that knows they share a lifetime can free the whole batch in one operation.

- **Many objects, one size.** A pool of network connections, tree nodes, fixed-size buffers. The general allocator's size-class machinery and per-block headers are overhead you do not need if every object is identical; a fixed-size allocator can hand out and reclaim blocks in O(1) with almost no bookkeeping and no fragmentation *within* that size.
- **Hot allocation loops.** Per-iteration `malloc`/`free` in a tight loop pays the general allocator's cost every iteration. Amortizing that — allocating a big block once and sub-dividing it cheaply — can dominate.

The mechanism behind all three is the same idea as Chapter 5's mechanical sympathy: when you know the access or lifetime pattern, you can lay memory out and manage it to match, and the machine rewards you. But there is a discipline here that this chapter insists on and §12.4 returns to: custom allocators are a *measured* optimization, not a default. Berger, Zorn, and McKinley's study of eight real programs using custom allocators (OOPSLA 2002) found that for most of them a good general-purpose allocator matched or beat the custom one — the big wins came specifically from the region/arena pattern (the "one lifetime" case), where they measured improvements up to 44%. The lesson is not "always write your own"; it is "when the pattern is right, the win is real and large — and when it is not, you have added complexity for nothing."

> **QUICK CHECK**
>
> Name the allocation pattern an *arena* exploits and the one a *fixed-size pool* exploits. Which single piece of knowledge does each have that the general-purpose allocator does not? (Answer after the next section.)

12.2 The arena (region / bump) allocator — build and measure it

An **arena** (or region, or bump allocator) exploits the "many objects, one lifetime" pattern. The idea is minimal: reserve one large block up front, keep a single *offset* into it, and satisfy each allocation by returning the current offset and bumping it forward by the requested size (rounded up for alignment). There is no free list, no size class, no per-block header, and — most importantly — no per-object free: individual objects are *never* freed. Instead the entire arena is reclaimed at once, either by resetting the offset to zero (reuse the memory) or by freeing the backing block. Allocation is a pointer increment; bulk-free is a single assignment or one `free`. Here is the whole allocator:

```
typedef struct { unsigned char *base; size_t cap, off; } Arena;

Arena arena_new(size_t cap){ Arena a; a.base = malloc(cap); a.cap = cap; a.off = 0; return
a; }

void *arena_alloc(Arena *a, size_t n){
    size_t p = (a->off + 15) & ~(size_t)15;   /* round up to 16-byte alignment */
    if (p + n > a->cap) return NULL;           /* arena exhausted */
    a->off = p + n;                             /* bump the offset */
    return a->base + p;
}

void arena_reset(Arena *a){ a->off = 0; }       /* frees EVERYTHING in O(1) */
```

That is a complete, working allocator. Compiled and run on this machine, it hands out correctly aligned, independent blocks (`x=42`, `s="hello"`, `y=7`, each 16-byte aligned), and `arena_reset` returns the offset to 0, reclaiming all of them in one operation. The payoff is speed, and it is worth measuring honestly. The arena wins when many objects are live *simultaneously* and die together — so the fair comparison is: allocate N objects that all stay live, then free them. With the arena that is N pointer-bumps and one bulk free; with `malloc` it is N allocations and N individual frees. For N = 5,000,000 small objects on this machine (gcc 11.4.0, `-O2`, x86-64):

```
arena  (N live, 1 bulk free) :  ~50-90 ms
malloc (N live, N frees)     : ~240-370 ms
speedup                      :  ~4-5x
```

Roughly a **four-to-five-fold** speedup, which lines up with the region wins Berger et al. reported. **These figures are illustrative and from this specific machine/run — they vary run-to-run (the ranges above are across repeated runs) and with size, allocator, and hardware; measure your own.** The win has a clear source: the arena replaced a million bookkeeping-heavy `free` calls with a single offset reset, and made each allocation a pointer add instead of a free-list search.

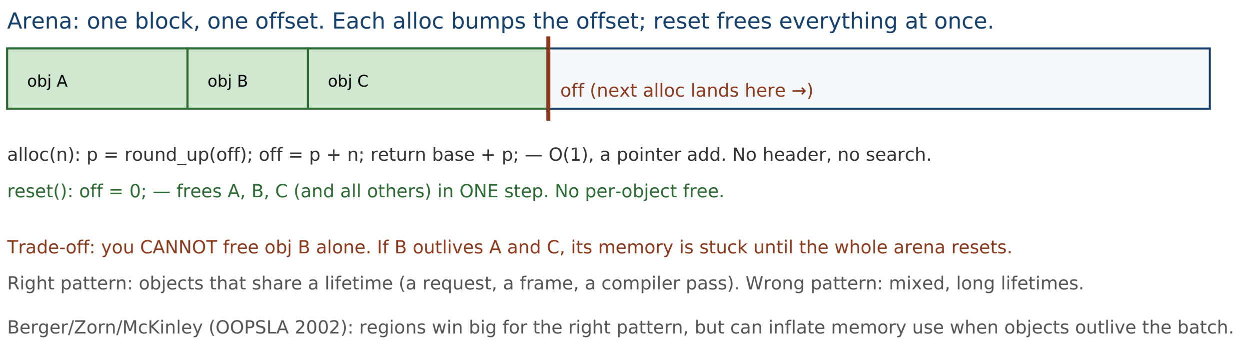


Figure 12.1: An arena allocates by bumping a single offset and frees everything at once by resetting it — O(1) allocation, O(1) bulk free, no fragmentation, no per-object free. The cost is inflexibility: you cannot free one object early, so memory is held until the whole region is reset.

The trade-off is exactly that inflexibility, and it is not free. Because you cannot free an individual object, any object that outlives the batch pins the *entire* arena's memory until reset — so an arena used with the wrong lifetime pattern can inflate memory consumption badly (Berger et al. note precisely this downside of regions). And — the theme of §12.5 — resetting an arena while a pointer into it is still in use is a use-after-free by another name: undefined behavior, exactly as in Chapter 10, now with *you* as the allocator.

*(Answer to the §12.1 quick check: an **arena** exploits "many objects share one lifetime" — it knows the batch dies together, so it can skip per-object frees and reclaim all at once. A **fixed-size pool** exploits "many objects, one size" — it knows every block is identical, so it needs no size classes, no split/coalesce, and no per-block size header.)*

12.3 The pool / free-list allocator — fixed-size, O(1) both ways

When objects share a *size* rather than a lifetime — a pool of tree nodes, connection structs, fixed buffers — a **pool allocator** (a fixed-size free-list allocator) is the specialization that fits. Carve one large block into an array of equal-size slots and thread the free slots onto a singly linked **free list**: because every slot is the same size and currently unused, you can store the "next free slot" pointer *inside the slot itself*, costing no extra memory. Allocation pops the head of the free list; freeing pushes the slot back onto the head. Both are O(1), a couple of pointer operations, with no search, no splitting, no coalescing, and — because all slots are identical — no external fragmentation within the pool at all. The sketch:

```
/* free_list points at the first free slot; each free slot's first bytes hold the next
pointer */
void *pool_alloc(Pool *p){
    if (!p->free_list) return NULL;            /* pool exhausted */
    void *slot = p->free_list;
    p->free_list = *(void **)slot;             /* pop: next free = this slot's stored next */
    return slot;
}
void pool_free(Pool *p, void *slot){
    *(void **)slot = p->free_list;             /* push: this slot's next = old head */
    p->free_list = slot;                        /* new head = this slot */
}
```

Unlike the arena, a pool *does* support freeing individual objects — that is its point — so it fits the "allocate and free repeatedly, but always the same size" pattern the arena cannot. What it gives up is generality: it serves exactly one size. This is not a toy idea. The Linux kernel's **slab allocator** — introduced in Jeff Bonwick's 1994 paper (USENIX) for SunOS and adopted widely since — is a production-grade elaboration: it maintains per-size caches of pre-initialized objects ("slabs") so that allocating a kernel object is a free-list pop and freeing it a push, retaining the object's initialized state between uses and adding a cache-coloring scheme to spread objects across cache lines (Ch 3). Production general allocators (glibc's `malloc`, jemalloc, tcmalloc) likewise use size-classed free lists internally — the pool is the general allocator's own core trick, exposed for you to specialize.

> **THE SAME IDEA THREE WAYS: SPECIALIZE THE ALLOCATOR TO THE PATTERN**
>
> **1. In words.** The general allocator is fast *because it knows nothing* is a contradiction — its generality costs search, headers, and fragmentation. If you know the pattern (shared lifetime → arena; shared size → pool), you drop the machinery you do not need and get O(1) with almost no bookkeeping.
>
> **2. As two pictures.** Arena (Fig 12.1): one offset, bump to allocate, reset to free all. Pool: an array of equal slots with a free list threaded through the unused ones; pop to allocate, push to free — both O(1), no fragmentation within the size.
>
> **3. As numbers.** Arena on this machine: ~4–5× faster than `malloc` for the one-lifetime batch (§12.2). Pool: allocation and free are each a couple of pointer writes versus the general allocator's free-list search and coalesce. The win is real *only* when the pattern holds (§12.4).

12.4 Choosing an allocator — an honest accounting

With three allocators in hand — general-purpose, arena, pool — the engineering question is when to reach for each. The honest answer starts with a warning that the research backs up: **the general-purpose allocator is the right default, and custom allocation is a measured optimization, not a reflex.** Berger, Zorn, and McKinley found that for six of eight real programs a state-of-the-art general allocator matched or beat the hand-written custom one — the programmers' custom allocators were often *slower* as well as buggier and harder to maintain. Custom allocation earned its keep in exactly the region/arena case. So the decision procedure is: profile first (Chapter 9's cost model, and the performance-engineering discipline Part 6 develops — measure before you optimize); confirm allocation is actually a bottleneck; then match the allocator to the pattern:

- **General-purpose `malloc`** — the default. Any size, any order, any lifetime, individual frees. Use it unless you have measured a reason not to.
- **Arena** — when a burst of objects shares a lifetime and dies together (a request, a frame, a parse). O(1) alloc, O(1) bulk free, no per-object free. Watch the memory downside: anything that outlives the batch pins the whole region.
- **Pool** — when objects share a fixed size and are allocated/freed repeatedly. O(1) both ways, no fragmentation within the size, individual frees supported. Serves exactly one size.

Each choice is a trade, not a free win. The arena trades the ability to free individually for speed and simplicity — and can waste memory if the lifetime assumption is wrong. The pool trades generality (one size only) for O(1) operations and zero fragmentation. And every custom allocator trades the maturity, debugging support, and hardening of the system allocator for code you now own and must get right. That last cost is the bridge to §12.5.

TRAP: "A CUSTOM ALLOCATOR MAKES THE MEMORY BUGS GO AWAY"

It is tempting to think that once *you* control allocation, the hazards of Chapters 9–11 are behind you. The opposite is true: **writing your own allocator makes you responsible for the very invariants the language and the system allocator's tooling used to help enforce** — and it introduces new ways to violate them. Reset an arena while a pointer into it is still live, and the next allocation reuses that memory under the old pointer: a *use-after-reset*, which is a use-after-free (Ch 9), undefined behavior (Ch 10), with you as the allocator. `pool_free` the same slot twice and you corrupt the free list — a double-free in your own code. Hand out a slot without checking the pool is non-empty and you return a null you may dereference. Worse, sanitizers (Ch 9) understand `malloc/free` but do *not*, by default, understand *your* arena's reset or *your* pool's recycling, so they may not catch a use-after-reset at all — you have moved the bug below the tool's visibility. (ASan offers manual "poisoning" hooks precisely so custom allocators can be made visible to it, but you must add them.) A custom allocator does not repeal lifetimes, bounds, and free-exactly-once; it makes you their implementer. Every rule from Chapters 9–11 still holds — you have just taken over enforcing them.

QUICK CHECK

You reset an arena to reclaim a request's memory, but one object from that request was stored in a cache that outlives the request. What bug have you just created, and what is its Chapter-9 name? (Answer below the communicate box.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: "Our service is slow — let's replace `malloc` with an arena allocator everywhere for a quick speed win." Cover the paragraph and respond in three or four sentences, drawing on §12.1 and §12.4.

Model answer. "An arena is a big win for a specific pattern — a burst of objects that share a lifetime and die together, like everything allocated while handling one request — but it's the wrong tool 'everywhere,' because it can't free individual objects, so any object that outlives its batch pins the whole region and can inflate our memory badly. The evidence backs the caution: Berger, Zorn, and McKinley found a good general allocator matched or beat hand-written custom allocators for most programs, with the arena/region case being the real exception. Let's profile first to confirm allocation is actually the bottleneck, then apply an arena precisely where the lifetime pattern fits — per-request scratch memory — and keep `malloc` for the mixed-lifetime long-lived state. And whatever we pick, we still own every lifetime and bound: resetting an arena with a live pointer into it is a use-after-free that our sanitizers won't see by default." Naming the pattern, citing the measured result, and insisting on profiling — rather than a blanket swap — is the senior register.

(Answer to the §12.4 quick check: you have created a ***use-after-free*** *— specifically a use-after-reset. Resetting the arena ended the lifetime of that object's storage; the cached pointer now dangles, and the next allocation from the arena will reuse those bytes under it. It is undefined behavior, and by default the sanitizers will not flag it, because they do not know your arena reset ended a lifetime.)*

12.5 What to carry forward — and out of Part 2

The general-purpose allocator is optimized for knowing nothing about your requests, and that ignorance is its cost — search, headers, coalescing, fragmentation (Ch 9). When you *do* know the pattern, a custom allocator trades generality for speed. An **arena** exploits shared lifetime: allocate by bumping an offset, free everything at once by resetting it — O(1) both ways, no per-object free, measured at roughly four-to-five times faster than `malloc` for the batch pattern on this machine, at the cost of being unable to free one object early. A **pool** exploits shared size: an array of equal slots with a free list threaded through the unused ones, O(1) allocation and free with no fragmentation within the size — the same trick the kernel's slab allocator and production `malloc`s use internally. But custom allocation is a *measured* optimization, not a default: the research (Berger et al.) shows a good general allocator usually matches hand-rolled ones, with the arena/region pattern the clear exception — so profile, confirm the bottleneck, and match the allocator to the pattern. And, decisively, writing your own allocator does not lift one obligation from Chapters 9–11: you become the implementer of lifetimes, bounds, and free-exactly-once, with new ways to violate them (use-after-reset, pool double-free) that your tools may not see.

That last point closes Part 2. Across six chapters you went from "a pointer is an address into regioned memory" (Ch 7), through lifetimes and the stack/heap (Ch 8), manual allocation and its bug catalogue (Ch 9), undefined behavior and the optimizer that trusts it (Ch 10), the buffer overflow that hijacks control flow (Ch 11), to writing the allocator itself (Ch 12) — and the through-line never changed: **in C you hold, by hand, a proof that no pointer is used outside its lifetime, no access outside its bounds, every block freed exactly once, and no undefined behavior ever triggered.** That proof is real, it is heavy, and every bug in this part is what happens when it fails. You now understand the machine to the metal and the net that is absent. Two paths lead out. The operating system (Part 3) is what sits beneath all of this — the processes, threads, and virtual memory that gave you the address space and the heap in the first place; you will see the system-call boundary from the other side. And Rust (Part 5) is the language that makes the compiler *check* the proof you have been carrying — ownership, borrowing, and lifetimes turning the invariants of this entire part into type rules verified before the program runs. You have felt the weight; the rest of the book is about the machine underneath it and the tools that lift it.

CAN YOU… WITHOUT LOOKING?

- State the three allocation patterns where a custom allocator beats general-purpose `malloc`, and the knowledge each exploits.
- Write an **arena** allocator from memory (base, capacity, offset; bump to allocate, reset to free all) and state its one big trade-off.
- Write a **pool**/free-list allocator's alloc and free (pop/push a free list threaded through the slots) and say why both are O(1) with no fragmentation within the size.
- Give the honest decision rule (general by default; profile; arena for shared lifetime; pool for shared size) and cite what the Berger et al. study found.
- Explain why a custom allocator does *not* remove the Chapter 9–11 obligations — and name a bug (use-after-reset) your sanitizers may miss.
- Summarize Part 2's through-line and how it sets up the OS (Part 3) and Rust (Part 5).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Name three allocation patterns where a custom allocator wins, and what each exploits. (§12.1)
2. **R2.** Describe an arena's allocate and free, its complexity, and its trade-off. (§12.2)
3. **R3.** Describe a pool/free-list allocator and why it has no fragmentation within its size. (§12.3)
4. **R4.** What did Berger et al. find about custom vs. general allocators, and what is the decision rule? (§12.1, §12.4)
5. **R5.** Why does a custom allocator not remove the Chapter 9–11 obligations? Name a bug it introduces. (§12.4–12.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 9.** Dynamic memory allocation: allocator design, free lists, coalescing, and the mechanics an arena and pool specialize. The reference for §12.1's cost model.
- **Berger, Zorn & McKinley, "Reconsidering Custom Memory Allocation," OOPSLA 2002.** The empirical study behind §12.4: a good general allocator matches most hand-written custom allocators, with regions (arenas) the clear exception (improvements up to 44%, but a memory-consumption downside). Free from the authors.
- **Bonwick, "The Slab Allocator: An Object-Caching Kernel Memory Allocator," USENIX 1994.** The production pool/slab allocator behind §12.3, still used in the Linux kernel. Free.
- **The jemalloc / tcmalloc documentation.** Real, widely deployed general allocators whose internals (size-classed free lists, thread caches) are the pool idea at scale. Documented behavior only; verify against the version you use.

Exercises

12.1 WARM-UP In one sentence each: (a) what does an arena exploit? (b) what does a pool exploit? (c) which of the two can free an individual object?

12.2 WARM-UP What is the time complexity of an arena's `alloc` and its bulk `reset`? What is the one thing an arena cannot do that `malloc` can?

12.3 WARM-UP In a fixed-size pool, where is the "next free slot" pointer stored, and why does that cost no extra memory?

12.4 CORE Write, from memory, an arena allocator (a struct with base/capacity/offset, plus `arena_alloc` with 16-byte alignment and `arena_reset`). Explain why `arena_alloc` is O(1) and why `arena_reset` frees everything without touching individual objects.

12.5 CORE Write a pool allocator's `pool_alloc` and `pool_free` using a free list threaded through the slots. Explain why both are O(1) and why the pool has no external fragmentation within its size class.

12.6 CORE A teammate wants to replace `malloc` with an arena "everywhere for speed." Give two concrete reasons this is a bad blanket policy — one about the arena's inability to free individually, one citing the Berger et al. finding — and state the correct decision procedure.

12.7 COST (a) Why is an arena's per-allocation cost lower than `malloc`'s — name two things the general allocator does that the arena skips (tie to Ch 9). (b) The chapter measured ~4–5× on this machine for the "N live, then free" pattern; why does that comparison favor the arena, and what different pattern would narrow or erase the gap? (c) What memory-consumption risk does an arena carry, and when does it bite?

12.8 FADED *Worked, with the last step for you.* A per-request arena is reset at the end of each request. A developer adds a cache that stores, across requests, a pointer to a string that was allocated *in* the arena. Diagnose the bug.
Step 1 (given): `arena_reset` ends the lifetime of everything in the arena by returning the offset to 0.
Step 2 (given): The cached pointer still holds the string's old address, but that storage is now considered free.
Step 3 (given): The next request's allocations will reuse those bytes, so a later read through the cached pointer sees another request's data or garbage.
Step 4 (you): Name the bug using Chapter 9's vocabulary, state whether it is undefined behavior, explain why the ASan/Valgrind of §9.4 will likely *not* catch it by default, and give a fix (what must be copied where, so the cache does not point into the arena).

12.9 MIXED For each workload, choose *general* `malloc`, *arena*, or *pool*, and justify in a line: (a) a web handler that allocates many small objects while serving one request, all discardable when the response is sent; (b) a graph engine that constantly creates and destroys same-size nodes throughout its run; (c) a general-purpose library that cannot predict its callers' allocation patterns; (d) a compiler pass that allocates a burst of AST nodes freed together when the pass ends.

12.10 MIXED Classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*, drawing on Chapters 9–12: (a) reading through a pointer into an arena after `arena_reset`; (b) an arena holding a long-lived object that pins the whole region, growing memory use; (c) `pool_free`-ing the same slot twice; (d) resetting an arena when no live pointers into it remain.

12.11 STRETCH Part 2 has argued that in C the programmer carries, by hand, a proof about lifetimes, bounds, and freeing. (a) State that proof obligation in one sentence. (b) Explain how a custom allocator *relocates* rather than *removes* that obligation — who now enforces "free exactly once" for arena and pool memory? (c) In one or two sentences, describe what it would mean for a language (Rust, Part 5) to tie an allocation's lifetime to a scope so that "use after reset" becomes a compile-time error — and why that is stronger than a runtime sanitizer that does not understand your allocator.

12.12 LAB On your own machine, implement the arena of §12.2. (a) Verify correctness: allocate three objects of different sizes, confirm they are independent and 16-byte aligned, and that `arena_reset` returns the offset to 0. (b) Time allocating N objects that stay live then freeing them, arena (one bulk free) vs. `malloc` (N frees), for a large N, and report the ratio. (c) Repeat with a pattern where each object is freed immediately after allocation and explain why the arena's advantage shrinks. Report only what you measured and label your compiler/hardware, since numbers vary.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART III

The Operating System

Processes, threads, scheduling, virtual memory, and the system-call boundary — how the OS multiplexes the machine.

Chapter 13 — The System-Call Boundary

Before you start: Chapter 4 (virtual addresses; the hardware translates every pointer and can refuse the access), Chapter 11 (a buffer overflow corrupts *this* process — but only this one), Chapter 12 (you became the allocator; the backing memory came from somewhere). · **Opens Part 3.** Part 2 left you holding, by hand, a proof about the memory *inside* one program. This part is about the operating system underneath all of it — the code that gave you an address space, a heap, and the illusion of owning the machine. We start at the seam where your program meets that code: the **system call**, the single controlled doorway from your program into the kernel.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 11 — a stack buffer overflow can overwrite the saved return address and hijack *this* program's control flow. Can it, by the same mechanism, overwrite memory belonging to a *different* running program? What stops it? (2) Chapter 4 — when you dereference a pointer, the address is virtual and the hardware translates it; who set up the translation tables, and could your code just rewrite them to point anywhere in physical RAM? (3) *Predict:* when your C program calls `write()` to print to the screen, does *your* code push the bytes out to the terminal device — or does something else do it on your behalf? Guess what stops an ordinary program from writing straight to the disk controller. By the end you will be able to trace exactly what happens on that call.

Every useful program eventually needs to do something it is not allowed to do. Reading a file, sending a network packet, getting more memory from the OS, creating a process, drawing on the screen — all of these touch hardware or shared state that a single process must not control directly, because the whole point of an operating system is that one buggy or malicious program cannot seize the machine or read another program's secrets. So the machine runs your code in a restricted mode, and the only way to get privileged work done is to *ask*: to make a **system call**, a controlled transfer into the kernel that switches into privileged mode, runs a specific pre-approved handler, and returns. This chapter is about that boundary. We establish the two privilege modes and why they exist (§13.1), trace exactly how a system call crosses the boundary on this machine, register by register (§13.2), separate the C library's friendly wrappers from the kernel underneath them (§13.3), measure what a crossing actually costs and how to avoid paying it (§13.4), and carry the boundary forward into processes and threads (§13.5). This is the seam Chapter 12 promised you would see "from the other side."

13.1 Two modes: user and kernel

The CPU itself enforces the boundary. A modern processor runs in one of (at least) two **privilege modes**: **user mode**, in which most of your code runs, and **kernel mode** (also called supervisor or privileged mode), in which the operating system runs. On x86 these are implemented as *rings* — ring 3 for user code, ring 0 for the kernel. The difference is not speed; it is *authority*. In user mode a set of instructions simply will not execute: you cannot halt the CPU, disable interrupts, load the page tables that define address translation (Chapter 4), or read and write device registers. Attempt one and the hardware traps into the kernel instead of obeying you.

That single mechanism is what makes the isolation of Chapter 11 real. Your buffer overflow can wreck *this* process because it runs with this process's authority — but it cannot reach another process's memory, because the memory-management unit (Chapter 4) translates every address through *this* process's page tables and faults on anything outside them, and it cannot rewrite those page tables to escape, because installing page tables is a privileged instruction it is not allowed to run. Two hardware mechanisms, working together: the MMU draws the boundary around each process's *memory*, and the user/kernel mode bit draws the boundary around the machine's *instructions and devices*. Everything you have written in this book so far has run in ring 3, with no direct access to any of it. The kernel is the only software that runs privileged, and it is the referee that hands out access under its own rules.

This is the answer to the opener's first two questions. A process cannot corrupt another process's memory because it is confined to its own translated address space and cannot touch the translation machinery; it cannot rewrite the page tables because that instruction traps. The confinement is not politeness — it is enforced by silicon on every instruction.

> **QUICK CHECK**
>
> Name one privileged operation that a program in user mode is *not* allowed to perform directly, and say what the hardware does if it tries anyway. (Answer after the next section.)

13.2 Crossing the boundary: the system-call mechanism

If user code cannot touch devices or memory tables, how does it ever read a file? It asks the kernel to do it, through a deliberately narrow door. The mechanism is a **trap**: a special instruction that simultaneously switches the CPU into kernel mode *and* jumps to a single, fixed entry point that the kernel installed at boot. This is the crucial safety property — user code does not get to switch into kernel mode and then run wherever it likes; the trap lands only at the kernel's chosen handler, which decides what to do. On 64-bit x86 Linux that instruction is `syscall` (the older 32-bit path used the software interrupt `int 0x80`).

Because the boundary is narrow, there is a strict convention — an **application binary interface (ABI)** — for how you name the call and pass its arguments. On x86-64 Linux (verified against the `syscall(2)` manual page): the **system-call number** goes in register `rax`; the first six arguments go in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` (note `r10`, not `rcx` — the `syscall` instruction itself clobbers `rcx` and `r11`); you execute `syscall`; and the kernel returns the result in `rax`. A negative return in the range of a small error code means failure — the raw kernel interface returns `-errno` (§13.3 turns that into the friendly `errno` you know). The kernel also *validates* what you pass: hand it a pointer to write into, and it checks that the pointer really belongs to your address space, so you cannot trick it into scribbling on kernel memory on your behalf.

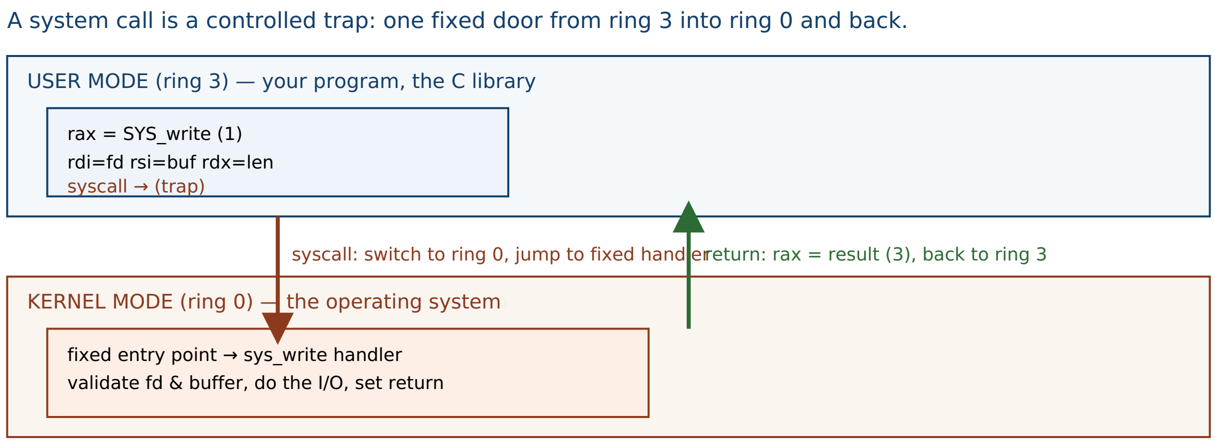


Figure 13.1: A system call. User code loads the call number and arguments into the ABI's registers and executes `syscall`; the CPU switches to kernel mode and jumps to one fixed entry point (never an address of the caller's choosing); the kernel validates, does the privileged work, and returns the result in `rax`. The boundary is narrow by design — that narrowness is the security.

Here is the whole path made concrete. When a program runs `write(1, "hi\n", 3)` — write three bytes to file descriptor 1, standard output — the sequence is: put the number for `write` in `rax`, put `1` in `rdi`, the buffer address in `rsi`, and `3` in `rdx`; execute `syscall`; the CPU traps to the kernel, which writes the bytes and returns `3` (the count written) in `rax`. We can watch this happen with `strace`, which intercepts and prints every system call a program makes. Compiling a three-line C program that does exactly that and running it under `strace` on this machine (gcc 11.4.0, x86-64 Linux) shows the crossing directly:

```
$ strace -e trace=write ./hello
write(1, "hi\n", 3)                     = 3
+++ exited with 0 +++
```

That one line *is* the boundary: the arguments the program marshaled, and the `= 3` the kernel returned. Everything else the program did — computing, moving data in its own memory — never appears, because none of it crossed into the kernel.

(Answer to the §13.1 quick check: a user-mode program may not execute privileged instructions such as halting the CPU, disabling interrupts, loading the page tables, or accessing device registers directly. If it tries, the hardware does not obey — it traps into the kernel, which typically terminates the offending process.)

13.3 The library wrapper vs. the raw system call

You have almost certainly never written `syscall` by hand, and you rarely will. The C library (glibc on most Linux systems) provides an ordinary-looking function for each system call — `write`, `read`, `open`, `fork` — and that **wrapper** does the register marshaling for you: it moves your arguments into `rdi/rsi/rdx`, sets `rax`, executes `syscall`, and inspects the result. On the negative-return failure convention from §13.2, the wrapper does the translation you already know: it stores the error code in the global `errno` and returns `-1`. So the raw kernel interface returns `-EBADF`; the wrapper you call returns `-1` and sets `errno == EBADF`. The wrapper is a thin shell around one `syscall`.

The distinction that trips people up is that *not every library function is a wrapper*, and the ones that matter most are not. `printf` is **not** a system call. It is a substantial C library routine that parses your format string, converts numbers to text, and — critically — *buffers* the result in user-space memory, only calling the `write` system call later, when the buffer fills or is flushed. Similarly `malloc` (Chapters 9 and 12) is a library function that manages a heap in user space; it crosses into the kernel only occasionally, calling the `brk` or `mmap` system calls when it needs to grow the heap — which is exactly where your arena's backing block came from in Chapter 12. This is the "other side" that chapter promised: your allocator handed out bytes that the C library had obtained, in bulk, from the kernel across this very boundary. The layering is real and worth holding precisely: *your code* calls a *library function*, which may buffer and compute in user space, and which crosses into the *kernel* only when it must, through a *system-call wrapper*.

> **QUICK CHECK**
>
> You run a program that prints with `printf`, then crashes before returning from `main` — and the printed line never appears in your terminal. Nothing was wrong with the format string. What most likely happened to those bytes? (Answer below the communicate box.)

13.4 The cost of crossing — and how to avoid paying it

Crossing the boundary is not free, and the cost is the reason a great deal of systems performance work exists. A system call must switch privilege mode, save and later restore the caller's state, enter the kernel, validate arguments, and eventually switch back — and along the way it can disturb the CPU's caches and TLB (Chapters 3–4). Compared with an ordinary function call, which on a modern core is a couple of instructions, that is enormously more expensive. Measured on this machine (gcc 11.4.0, `-O2`, x86-64), calling a trivial non-inlined function ten million times versus making the `getpid` system call the same number of times:

```
plain function call :  ~1-2 ns each
getpid system call  :  ~150-190 ns each
ratio               :  roughly two orders of magnitude
```

These figures are illustrative, from this specific machine and run — they vary with hardware, kernel, and mitigations (some CPU-vulnerability mitigations make the boundary crossing markedly more expensive), and you should measure your own. But the shape is robust and the lesson is durable: a system call costs on the order of a hundred function calls or more. Three consequences follow, and each is a technique you will use:

- **Batch — do not cross per item.** Reading a megabyte one byte at a time is a million crossings; reading it in large blocks is a handful. This is exactly why the C library buffers: `printf` and `fread` accumulate in user space and cross the boundary rarely. The single most common I/O performance bug is an unbuffered per-item system call in a hot loop.

- **Let the kernel share read-only data without a crossing — the vDSO.** A few "system calls" that only *read* kernel-maintained data, such as `clock_gettime` and `gettimeofday`, are served from the **vDSO** (virtual dynamic shared object): a page the kernel maps read-only into every process, containing the data and the code to read it, so the call runs entirely in user mode with no trap. On this machine, a program that calls `clock_gettime` one million times produces, under `strace -c`, *zero* `clock_gettime` system calls — the crossings simply are not there, because the vDSO answered in user space. (You can see the vDSO mapped into any process: `ldd` on a dynamically linked program lists `linux-vdso.so.1`.)
- **Observe the boundary with `strace`.** Because every crossing is visible, `strace` (and `strace -c` for a counted summary) is the first tool to reach for when a program is mysteriously slow or stuck: it shows you exactly which system calls it is making, how many, and where it is blocked.

THE SAME IDEA THREE WAYS: A SYSTEM CALL IS A CONTROLLED TRAP

1. In words. User code cannot touch devices or memory tables, so it *asks* the kernel by executing a trap instruction that switches to kernel mode and jumps to one fixed handler — never an address of the caller's choosing. The kernel validates, does the privileged work, and returns. Narrowness is the security.

2. As a picture. Figure 13.1: two stacked bands (ring 3, ring 0) with a single arrow down (`syscall` → fixed entry) and a single arrow up (return in `rax`).

3. As numbers. The ABI: number in `rax`; args in `rdi`,`rsi`,`rdx`,`r10`,`r8`,`r9`; result in `rax`; `-errno` on failure. The cost: ~150-190 ns for `getpid` versus ~1-2 ns for a function call on this machine — roughly 100×, which is why you batch and why the vDSO exists.

TRAP: "`PRINTF` IS A SYSTEM CALL" — CONFUSING THE LIBRARY WITH THE KERNEL

It is tempting to treat every function that produces output as a direct line to the operating system. **It is not: `printf` is a C library function that formats and *buffers* in user-space memory, and calls the `write` system call only when that buffer flushes.** Conflating the two produces real, confusing bugs. A program that `printfs` a line and then crashes may show *nothing* in the terminal, because the bytes were sitting in the library's buffer and the crash skipped the flush — the boundary was never crossed (this is the §13.3 quick check, and it returns in Chapter 14 when `fork` *duplicates* that unflushed buffer and a line prints twice). The fix when you need output to be real *now* is to force the crossing: `fflush(stdout)`, or configure line buffering, or call `write` directly — which is unbuffered and goes straight across. The mental model that prevents a dozen bugs: *your code → a library function (may buffer) → a system-call wrapper → the kernel*. Know which layer you are on. The library's job is often to *avoid* the kernel; the kernel is only reached deliberately.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate reports: "Our service writes one log line per request with a bare `write()` call, and profiling says it is spending huge time in the kernel. Let's just delete the logging." Cover the paragraph and respond in three or four sentences, drawing on §13.3–13.4.

Model answer. "The problem is almost certainly not *that* we log, but *how* — one unbuffered `write` per line is one system-call crossing per line, and a crossing costs on the order of a hundred function calls or more (on our box `getpid` was ~150 ns versus ~1-2 ns for a call), so at high request rates those crossings dominate. The fix is to batch: buffer log lines in user space and flush in large blocks, which is exactly why the C library buffers `printf` in the first place; we cross the boundary a few times instead of thousands. Let's confirm with `strace -c` that `write` is the hot call, switch to a buffered logger, and re-measure — deleting logging throws away observability to fix a batching bug." Naming the crossing cost, reaching for batching rather than removal, and confirming with `strace` is the senior register.

(Answer to the §13.3 quick check: the bytes were almost certainly still in `printf`'s user-space buffer. Because `printf` buffers and only crosses into the kernel via `write` on a flush, a crash before the flush loses them — they never reached the terminal. Force the crossing with `fflush`, line buffering, or a direct `write`.)

13.5 What to carry forward

The CPU runs your code in **user mode** (ring 3), where privileged instructions — touching devices, loading page tables, disabling interrupts — simply will not execute, and the operating system in **kernel mode** (ring 0) is the only software with that authority. Together the MMU (Chapter 4) and this mode bit are what make process isolation real: a bug confined to your address space cannot reach another process or the machine. To get privileged work done, user code makes a **system call** — a controlled trap (`syscall` on x86-64) that switches mode and jumps to *one fixed kernel entry point*, passing a call number in `rax` and arguments in `rdi,rsi,rdx,r10,r8,r9`, receiving a result in `rax` (with `-errno` on failure). The C library wraps each call in an ordinary function and translates the error convention into `errno` and `-1` — but many library functions (`printf`, `malloc`) are *not* system calls; they compute and buffer in user space and cross only when they must. And the crossing is expensive — on this machine roughly two orders of magnitude more than a function call — so you batch, you lean on the vDSO for cheap read-only data like the time, and you observe the boundary with `strace`.

This is the seam the rest of Part 3 lives on. A **process** (Chapter 14) is the isolated container this boundary protects — its own address space, created and destroyed through system calls (`fork`, `execve`, `exit`, `wait`) that cross exactly this line. A **thread** (Chapter 15) is a second flow of control that shares one side of the boundary — the same address space — trading isolation for cheap sharing. Every abstraction the OS gives you, from the heap that fed your allocator to the files and sockets you will read later, is reached through the doorway you have just learned to trace.

CAN YOU… WITHOUT LOOKING?

- Name the two CPU privilege modes and give two operations user mode is forbidden to do — and say what enforces the ban.
- Explain why a system call jumps to a *fixed* kernel entry point rather than an address the caller chooses, and why that matters.
- State the x86-64 Linux system-call ABI: where the call number, the arguments, and the return value go.
- Explain how `printf` differs from the `write` system call, and why a crash can lose a `printf`ed line.
- Give the rough cost of a system call versus a function call on a machine you have measured, and name two ways to cross the boundary less (batching, the vDSO).
- Say how process isolation follows from the MMU plus the user/kernel mode bit.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is the difference between user mode and kernel mode, and what enforces it? (§13.1)
2. **R2.** Describe the system-call mechanism on x86-64 Linux: instruction, where the number and arguments go, where the result comes back. (§13.2)
3. **R3.** Is `printf` a system call? What actually crosses the boundary, and when? (§13.3)
4. **R4.** Roughly how much more expensive is a system call than a function call, and why? (§13.4)
5. **R5.** What is the vDSO, and why does calling `clock_gettime` often *not* show up as a system call? (§13.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Mechanism: Limited Direct Execution."** The free chapter that develops exactly this boundary — user vs. kernel mode, traps, and the system-call mechanism. The reference for §13.1–13.2.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8.** Exceptional control flow: how the hardware and OS handle traps and system calls from the programmer's view.
- **The Linux `syscall(2)` and `vdso(7)` manual pages.** The primary sources for the per-architecture calling convention of §13.2 and the vDSO of §13.4. Verify against your kernel and architecture.
- **`strace` documentation and man page.** The tool used throughout this chapter to make the boundary visible; `strace -c` for a counted summary. Free.

Exercises

13.1 WARM-UP Name the two CPU privilege modes, and give one operation a program in user mode is not allowed to perform directly.

13.2 WARM-UP On x86-64 Linux, which register holds the system-call number, and which register holds the return value?

13.3 WARM-UP Is `printf` a system call? If not, what does it do first, and which system call does it eventually use to produce output?

13.4 CORE Trace, step by step, what happens when a program executes `write(1, buf, 5)`: what does the C library put in which registers, what instruction crosses the boundary, and what comes back on success? (Use the ABI from §13.2.)

13.5 CORE Explain the relationship between the kernel's raw error convention and the `errno`/`-1` you see from a C library call. If a raw system call returns `-EBADF`, what does the wrapper return and what does it set?

13.6 CORE You run `strace` on a program and see, among other lines: `openat(...) = 3`, `read(3, ..., 4096) = 512`, and then thousands of `read(3, ..., 1) = 1` lines. In one or two sentences, say what the last pattern tells you is wrong and what to change.

13.7 COST (a) Name two reasons a system call costs far more than an ordinary function call. (b) A program reads a 1 MB file by calling `read` one byte at a time; explain why this is slow in terms of boundary crossings, and give the fix. (c) Why does calling `clock_gettime` in a tight loop often cost *no* system calls at all?

13.8 FADED *Worked, with the last step for you.* A program prints a diagnostic line with `printf` just before dereferencing a bad pointer and crashing. The line never appears in the terminal, even though the program definitely reached the `printf`.
Step 1 (given): `printf` writes into the C library's user-space output buffer, not directly to the terminal.
Step 2 (given): The buffer is flushed (crossed to the kernel via `write`) only when it fills, at a newline for a line-buffered terminal, or at normal program exit.
Step 3 (given): A crash from an invalid dereference terminates the process abnormally, skipping the normal flush.
Step 4 (you): State where the bytes actually were when the program died, explain why the line was lost (referencing the boundary), and give two ways to make sure the line appears before a possible crash.

13.9 MIXED For each, decide whether it requires crossing the system-call boundary or is pure user-space computation, and say why in a few words: (a) adding two integers in registers; (b) opening a file; (c) sorting an in-memory array; (d) sending a network packet; (e) formatting a number into a string with `sprintf`.

13.10 MIXED Drawing on Chapters 10–13, classify each as *undefined behavior*, *a boundary crossing (system call)*, or *neither*: (a) reading one past the end of a stack array; (b) calling `read` on a valid file descriptor; (c) signed integer overflow in a loop counter; (d) a buffered `printf` that has not yet flushed.

13.11 STRETCH The kernel validates every pointer you pass to a system call such as `write`, checking it belongs to your address space. (a) Relate this to Chapter 4's address translation: why can a process not simply pass a pointer to another process's memory or to kernel memory and have the kernel read it? (b) Sketch what could go wrong for security if the kernel skipped this validation. (c) Explain how the two mechanisms — per-process address translation and the user/kernel mode bit — together make one process unable to spy on another.

13.12 LAB On your own machine: (a) write a program that prints with `write` and run it under `strace -e trace=write`; confirm you see the `write` call and its return value. (b) With `strace -c`, count the total system calls of a trivial "hello world" and note how many are startup overhead (dynamic linking, etc.) versus your own output. (c) Measure the cost of the `getpid` system call versus a trivial function call in a loop, and (d) call `clock_gettime` a million times and confirm with `strace -c` that it does *not* appear as a million system calls. Report your numbers and label your compiler, kernel, and hardware, since figures vary.

Chapter 14 — Processes: fork, exec, and wait

Before you start: Chapter 13 (the system-call boundary — `fork`, `exec`, and `wait` are all system calls), Chapter 4 (each process has its own virtual address space, translated through its own page tables), Chapter 11 (a memory bug is confined to *one* process's address space). · Pairs with Chapter 15 (threads share an address space instead of copying it). Chapter 13 showed the boundary that protects a process; this chapter is about the process itself — what it is, how the OS makes a new one, how one program becomes another, and how a parent collects a finished child.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 13 — to do something privileged, user code must do what, and through which instruction on x86-64? (2) Chapter 11 — a buffer overflow can corrupt the process it runs in; can it corrupt a *different* running process's memory, and what two mechanisms decide the answer? (3) *Predict:* `fork()` makes a new process that is a copy of the calling one. If your program is holding a 1 GB array in memory and calls `fork()`, do you think the operating system immediately copies all 1 GB? Guess, and note why the answer matters for how often you can afford to `fork`. We measure the idea behind it shortly.

A running program is not the same thing as the file on disk. The file is inert bytes; the running instance — with its own memory, its own place in the CPU, its own open files — is a **process**, and the process is the operating system's fundamental unit of isolation and accounting. Everything Chapter 13 protected was protecting a process: its private address space, its authority level, its resources. This chapter opens the process up. We define precisely what a process bundles together (§14.1); we build a new one with the Unix system call `fork`, and see why it returns *twice* and why copying a process can be cheap (§14.2); we turn a copied process into a *different* program with `exec`, and meet the fork-then-exec pattern that every shell uses (§14.3); we collect a finished child with `wait`, and meet the zombie and the orphan (§14.4); and we carry the model into threads (§14.5). By the end you will be able to write, and reason about, the create-run-reap lifecycle that underlies every command you have ever typed.

14.1 What a process is — the unit of isolation

A process is three things bundled and protected together. First, an **address space**: the private virtual memory of Chapters 4, 7, and 8 — text, initialized data, bss, heap, and stack — that the MMU translates through this process's own page tables so that its pointers name only its own memory. Second, one or more **threads of execution**: the actual running context — the program counter, the stack pointer, the register values — that says *where* in the code the process currently is (this chapter's processes have exactly one; Chapter 15 adds more). Third, a bundle of **kernel-managed resources**: the open file descriptors, the process ID (PID) and parent PID, the current working directory, the user and group credentials, and more, all tracked by the kernel on the process's behalf.

The reason these are bundled is **isolation**. The kernel gives each process the illusion of owning the machine: its own address space (so one process's pointers are meaningless in another's — the answer to Chapter 11's confinement), its own turn on the CPU, its own resource handles. A fault in one process — a wild pointer, a crash, even a security compromise — is contained to that process's address space and cannot, by the mechanisms of Chapter 13, corrupt another process or the kernel. This is why the process is the natural boundary for a sandbox, a multi-tenant service, or any place you need one failure not to become everyone's failure. The process is precisely the thing the system-call boundary was built to protect.

> **QUICK CHECK**
>
> Name the three things a process bundles together. Which one makes two processes' identical-looking pointer values refer to completely different memory? (Answer after the next section.)

14.2 Creating a process: `fork` and copy-on-write

Unix creates a new process in a way that surprises everyone the first time: the `fork()` system call is called *once* but returns *twice*. It makes the calling process (the **parent**) into two nearly identical processes — the parent and a new **child** — and then both continue from the point of the return. The child is a copy: same code, same current values of every variable, copies of the open file descriptors. The one difference is the return value, which is how each copy knows who it is: `fork` returns **0 in the child** and **the child's PID in the parent** (or `-1` on failure). You branch on it. Here is the whole pattern, compiled and run on this machine (gcc 11.4.0, x86-64 Linux):

```
int x = 100;
pid_t pid = fork();          /* one call, returns TWICE */
if (pid == 0) {              /* child sees 0 */
    x += 1;
    printf("child : pid=%d ppid=%d x=%d  (fork returned %d)\n", getpid(), getppid(), x, pid);
} else {                     /* parent sees the child's pid */
    int st; waitpid(pid, &st, 0);
    x += 10;
    printf("parent: pid=%d x=%d  (fork returned %d; child exit=%d)\n", getpid(), x, pid,
WEXITSTATUS(st));
}
```

```
child : pid=1341934 ppid=1341933 x=101  (fork returned 0)
parent: pid=1341933 x=110  (fork returned 1341934; child exit=0)
```

Read that output closely, because it shows every essential fact. The two processes have different PIDs (1341934 and 1341933), and the child's parent PID equals the parent's PID — the family tree is real. Both started with `x == 100`, but the child's `x += 1` and the parent's `x += 10` did not interfere: the child ends at 101 and the parent at 110. Their memory is *independent* — the same variable name, at what is even the same virtual address, but two separate physical copies, because each process has its own address space (§14.1). And `fork` returned 0 down one path and the child's PID down the other, which is the only thing distinguishing the twins.

Now the predict from the opener: if the parent held a 1 GB array, did `fork` copy a gigabyte? No — and it could not afford to, or forking would be unusable. The kernel uses **copy-on-write (COW)**: rather than duplicating the parent's pages, it maps the *same* physical pages into both processes and marks them read-only. Both share the physical memory as long as they only read it. The instant *either* one writes to a page, the hardware faults, the kernel makes a private copy of just that one page for the writer, and both continue. So `fork`'s cost is proportional not to the size of the address space but to the number of pages actually modified afterward — which is why forking a huge process is cheap, and why the child's `x += 1` above triggered a copy of exactly the one page holding `x` and nothing else. Copy-on-write is Chapter 4's page tables doing double duty: the same translation-and-fault machinery that isolates processes also lets them share until the moment sharing becomes incorrect.

THE SAME IDEA THREE WAYS: `FORK` RETURNS TWICE INTO TWO ADDRESS SPACES

1. In words. One call, two returns: the parent gets the child's PID, the child gets 0, and from then on they are separate processes with separate (copy-on-write) memory. You branch on the return value to run different code in each.

2. As a picture. Figure 14.1: a single arrow entering `fork` and two arrows leaving, into two boxes that start as copies and diverge as each writes its own pages.

3. As a trace. Before: one process, `x = 100`. After `fork`: two processes, both `x = 100`, distinguished by return value (child 0, parent child-PID). After each runs: child `x = 101`, parent `x = 110` — independent, exactly as the measured output shows.

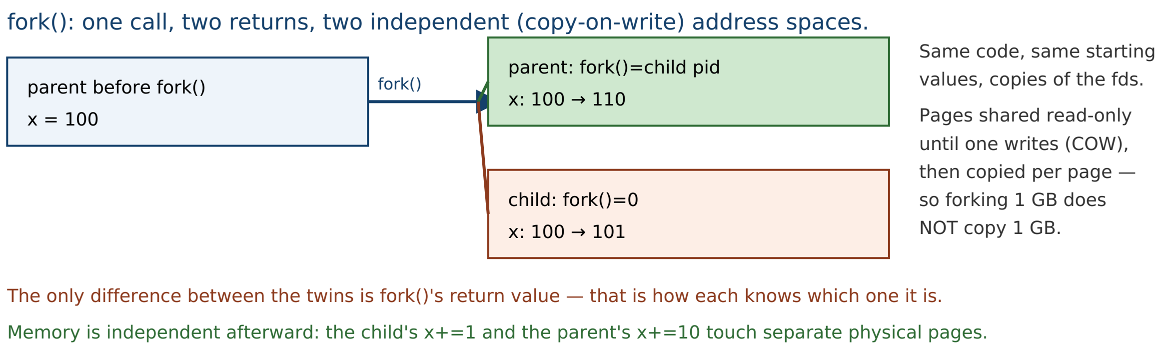


Figure 14.1: `fork` returns twice — 0 in the child, the child's PID in the parent — producing two processes that start as copies and, thanks to copy-on-write, share physical pages until one writes. The branch on the return value is how each copy runs different code.

*(Answer to the §14.1 quick check: a process bundles an **address space**, one or more **threads of execution**, and a set of **kernel resources** such as open file descriptors and its PID. The **address space** — separate per process and translated through its own page tables — is what makes two processes' identical-looking pointer values refer to different memory.)*

14.3 Running a new program: `exec` and the fork-exec pattern

`fork` gives you a copy of the *same* program. To run a *different* program you need `exec` — the `execve` system call and its convenience wrappers. `exec` does something drastic: it **replaces the current process image** — text, data, heap, stack — with a brand-new program loaded from disk, and jumps to its entry point. Everything the process was running is gone; the new program starts fresh. Two things survive the replacement, and they are the important ones: the **PID stays the same** (it is the same process, now running different code), and the **open file descriptors stay open** (unless marked close-on-exec). On success, `exec` *does not return* — there is nothing to return to, because the code that called it no longer exists.

Put the two together and you get the pattern behind every command you run: **fork, then exec, then wait**. To launch a program, a process `fork`s; the child `exec`s the new program (replacing itself with it); and the parent `wait`s for the child to finish. This is exactly what a shell does for every command you type — `fork` a child, have it `exec` `/bin/ls`, and wait for it to complete before printing the next prompt — and it is how your terminal launched the demo program in §14.2. The pattern is also why the parent in that demo called `waitpid` before continuing.

Why split creation (`fork`) from loading (`exec`) into two calls, when almost every use runs them back to back? Because the *gap between them* is where the elegance lives. After the child is forked but before it execs, it is still running the parent's code and can adjust its inherited resources: it can redirect standard output to a file, wire its input or output to a pipe connected to a sibling, drop privileges, or change directory — and then `exec` the target program, which starts up already plumbed into that environment without knowing anything about it. Unix I/O redirection and pipelines (`ls | grep foo > out.txt`) are built entirely in that fork-to-exec gap. The two-call design is not clumsiness; it is the seam that makes composition possible.

> **QUICK CHECK**
>
> A shell runs the command `ls`. In order, which of `fork`, `exec`, and `wait` does the shell process perform, which does the new child process perform, and which call does *not* return on success? (Answer below the communicate box.)

14.4 Reaping: `wait`, exit status, zombies, and orphans

A process ends by calling `exit` (or returning from `main`, or being killed). But it does not vanish the instant it ends. The kernel must keep a small amount of information about it — most importantly its **exit status**, the number it passed to `exit` — so that the parent can find out how the child fared. A process that has terminated but whose exit status has not yet been collected is a **zombie** (shown as `<defunct>` in `ps`): it holds no memory or CPU, just a slot in the process table and its exit status, waiting to be read.

The parent collects it by calling `wait` or `waitpid`, which blocks until a child finishes (or returns immediately if one already has), hands back the child's PID and its exit status (decode it with macros like `WEXITSTATUS`), and lets the kernel finally free the zombie's last slot. This is **reaping**. It is the parent's responsibility, and forgetting it is a real defect: a long-running parent that `fork`s children and never `wait`s accumulates zombies, slowly filling the process table. Note the category carefully — a leaked zombie is a *defined* resource leak, not undefined behavior; the program is not doing anything the language forbids, it is just failing to clean up, exactly as a memory leak is defined-but-bad (Chapter 9).

The mirror-image case is the **orphan**: a child whose parent exits *first*. An orphan is not left dangling — the kernel **reparents** it to `init` (PID 1, or a subreaper), which `wait`s for it when it finishes, so orphans are reaped automatically. The lifecycle, then, is: a process is `fork`ed, runs, `exit`s (becoming a zombie holding its status), and is reaped by a `wait`ing parent (or by `init` if orphaned). Create, run, exit, reap.

TRAP: `FORK` DUPLICATES YOUR UNFLUSHED OUTPUT BUFFER (THE DOUBLE-PRINT)

Chapter 13 warned that `printf` buffers in user-space memory and crosses to the kernel only on a flush. `fork` copies the *entire* address space — **including that buffer with its unflushed contents.** So if you `printf` a line and *then* `fork`, the un-crossed bytes sit in both the parent's and the child's copy of the buffer, and when each process later flushes (at exit), the line is written *twice* — once per process — even though your code printed it once. This is not hypothetical: constructing exactly this (a `printf` before the fork, with output going to a pipe so buffering is block-mode) reproduces the doubled line every time. The trap is subtle because it disappears when output goes to a terminal (line-buffered, so the newline already flushed before the fork) and reappears when output is redirected to a file or pipe. The fix is the same as Chapter 13's: force the crossing before you branch — `fflush(stdout)` before `fork` (or use unbuffered `write`). The deeper lesson: `fork` copies *everything*, including the parts of your program's state you had forgotten were state.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: "Let's handle each incoming request by `fork`ing a fresh process — that way one bad request can't take down the others." Cover the paragraph and respond in three or four sentences, drawing on §14.1–14.2.

Model answer. "That is a genuinely good instinct for *isolation*: a separate process has its own address space, so a crash or memory corruption in one request is contained and can't touch a sibling — it's the classic robustness pattern, and `fork` is cheaper than it looks because copy-on-write means we don't duplicate the whole address space, only the pages a child actually writes. The costs to weigh are that processes don't share memory, so passing data between them needs explicit IPC (pipes, shared memory) rather than a pointer, and each request pays process-creation and context-switch overhead, which can hurt at very high rates. So process-per-request is the right call when fault-containment or a security boundary is the priority; if instead we need cheap shared-memory concurrency and can enforce discipline, threads (Chapter 15) share the address space and skip the copy — at the cost of exactly that isolation. Let's decide based on whether containment or shared-state throughput is what this service needs, and measure." Naming copy-on-write, the isolation-vs-sharing trade, and deferring to a measurement is the senior register.

(Answer to the §14.3 quick check: the shell process performs `fork` and then `wait`; the new child process performs `exec` (replacing itself with `ls`). `exec` is the one that does not return on success — on success there is no original program left to return to.)

14.5 What to carry forward

A **process** is the operating system's unit of isolation: an **address space** (private virtual memory, Chapter 4), one or more **threads of execution** (the running context), and a bundle of **kernel resources** (open files, PID, credentials), all protected by the boundary of Chapter 13. You create one with `fork`, which is called once and **returns twice** — 0 in the child, the child's PID in the parent — producing two processes that start as copies and diverge; copying is cheap because of **copy-on-write**, which shares physical pages until one side writes, so forking a huge process costs only the pages later modified. You turn a copied process into a different program with `exec`, which **replaces the image** (keeping the PID and open descriptors) and does not return on success; the universal **fork-exec-wait** pattern — with the fork-to-exec gap used to set up redirection and pipes — is how every shell runs every command. And you clean up with `wait`, which reaps a finished child's exit status; an unreaped terminated child is a **zombie** (a defined leak), while an orphaned child is reparented to `init` and reaped for you.

Every process in this chapter had a single thread of execution — one program counter walking through the code. Chapter 15 relaxes that. A **thread** is a second (third, hundredth) flow of control *within one process*, sharing its address space rather than copying it as `fork` does. That one change — share instead of copy — buys cheap concurrency and drops the isolation this chapter spent its length building: the very memory that `fork` kept separate, threads deliberately share, and the proof burden of Part 2 returns, now with multiple threads reaching for the same bytes at once.

CAN YOU… WITHOUT LOOKING?

- State the three things a process bundles together, and which one enforces memory isolation between processes.
- Explain why `fork` returns twice, what it returns to each side, and how a program uses that to run different code in parent and child.
- Explain copy-on-write and why forking a 1 GB process does not copy 1 GB.
- Describe what `exec` replaces and what it keeps, and why it does not return on success.
- Lay out the fork-exec-wait pattern and say what the gap between fork and exec is used for.
- Define a zombie and an orphan, say who reaps each, and classify a leaked zombie (UB or defined defect?).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What three things make up a process, and what does the address space give it? (§14.1)
2. **R2.** What does `fork` return in the parent and in the child, and why is that the only difference between them? (§14.2)
3. **R3.** What is copy-on-write, and why does it make `fork` cheap? (§14.2)
4. **R4.** What does `exec` do, what survives it, and does it return on success? (§14.3)
5. **R5.** What is a zombie, what is an orphan, and who reaps each? (§14.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Abstraction: The Process" and "Interlude: Process API."** The free chapters that build exactly this material — the process abstraction and the `fork/exec/wait` API. The reference for §14.1–14.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8 (Exceptional Control Flow).** Processes, process creation, and reaping children, from the programmer's view.
- **The Linux `fork(2)`, `execve(2)`, and `wait(2)` manual pages.** Primary sources for the exact semantics, including copy-on-write, the fate of file descriptors across `exec`, and zombie/orphan behavior. Verify per system.
- **Beej's Guide to Unix IPC / C.** Free, practical walkthroughs of `fork/exec` and inter-process communication (pipes) — the plumbing that lives in the fork-to-exec gap.

Exercises

14.1 WARM-UP Name the three things a process bundles together.

14.2 WARM-UP What does `fork` return in the parent process, and what does it return in the child?

14.3 WARM-UP Does `exec` return on success? What does it replace, and name one thing it keeps.

14.4 CORE Write, from memory, the `fork/if-else` skeleton that distinguishes child from parent, with the parent calling `waitpid`. Explain in a sentence what each branch is and how each process knows which it is.

14.5 CORE Explain copy-on-write: what does the kernel actually do to the parent's pages at `fork` time, what triggers a real copy, and why does this make `fork`ing a 1 GB process fast?

14.6 CORE Describe the fork-exec-wait pattern a shell uses to run a command. Then explain what useful work can happen in the gap between `fork` and `exec`, and give one concrete example (e.g. I/O redirection).

14.7 COST (a) Explain why `fork` is cheaper than the size of the address space would suggest — name the mechanism. (b) Describe a workload for which copy-on-write buys you almost nothing (the child writes to nearly all inherited pages) and say why. (c) Compare process-per-request and thread-per-request on one cost each pays that the other does not.

14.8 FADED *Worked, with the last step for you.* A long-running server `fork`s a worker for each connection and never calls `wait`. After hours, `ps` shows hundreds of `<defunct>` entries and the server eventually cannot create new processes.
Step 1 (given): When a child exits, the kernel keeps its exit status until the parent collects it, leaving the child as a zombie.
Step 2 (given): This server never calls `wait`/`waitpid`, so no child's status is ever collected.
Step 3 (given): Each zombie holds a process-table slot, and the table is finite.
Step 4 (you): Name the accumulating state, explain why it eventually stops new `fork`s from succeeding, say whether this is undefined behavior or a defined resource leak, and give a fix (how the parent should reap, mentioning `wait`/`waitpid` or handling `SIGCHLD`).

14.9 MIXED For each goal, decide whether you need `fork`, `exec`, or `fork` then `exec`, and say why in a line: (a) run `/bin/ls` from inside your program and return to your program afterward; (b) create a second process that runs the *same* code on the second half of an array; (c) a shell running a command the user just typed.

14.10 MIXED Drawing on Chapters 9–14, classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*: (a) after `fork`, the parent expects to see changes the child made to `x` in the child's memory; (b) a server that never reaps its children; (c) both parent and child printing a line that was `printf`ed (unflushed) before the `fork`; (d) a child that `exec`s a new program while the parent keeps running.

14.11 STRETCH Tie process isolation back to Chapters 11 and 13. (a) Explain why a buffer overflow in process A cannot corrupt process B's memory, naming the two hardware mechanisms (Chapters 4 and 13) that enforce it. (b) Explain why this makes a separate process, not a thread, the right unit for sandboxing untrusted code. (c) In one or two sentences, describe what a multi-tenant service gains by putting each tenant's work in its own process rather than sharing one.

14.12 LAB On your own machine: (a) write the `fork` demo of §14.2 and confirm from its output that `fork` returned twice, that the child's `ppid` equals the parent's `pid`, and that the two processes' copies of a variable are independent. (b) Add a `printf` of some line *before* the `fork`, without an `fflush`, and run the program with its output redirected to a file; observe whether the line appears once or twice and explain why. (c) Add `fflush(stdout)` before the `fork` and confirm the fix. Report your output and your explanation; note your compiler and system.

Chapter 15 — Threads: Shared Memory and Its Hazards

Before you start: Chapter 14 (a process copies its address space; threads will share it), Chapter 13 (the system-call boundary — `clone` is the syscall underneath thread creation), Chapter 12 (in C you carry, by hand, a proof about memory — that proof is about to get a new clause). · Chapter 14's processes each had one thread of execution. This chapter adds more threads to a single process — a cheaper concurrency than `fork`, bought by giving up the isolation that made processes safe.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 13 — a system call is a controlled trap; name the instruction on x86-64 and say where the system-call number goes. (2) Chapter 14 — `fork` gives the child its own address space (via copy-on-write), so when the parent and child both hold a variable `x`, are they the same storage or two copies? (3) *Predict:* two threads share a single counter and each executes `counter++` one million times, with no locking. When both finish, is the final value exactly two million, less than two million, or possibly more? Write down your guess and your reasoning — we will run this exact program and measure the answer.

Chapter 14 gave you a way to do two things at once — `fork` a second process — but at the price of a second, isolated address space: the processes cannot see each other's memory, so sharing data means explicit inter-process communication. Often that isolation is exactly what you want. But often you want the opposite: several flows of control working on the *same* data structures in the *same* memory, cheaply. That is a **thread**. This chapter defines a thread as a second flow of control inside one address space (§15.1), creates threads with the POSIX `pthreads` API (§15.2), explains how the kernel actually schedules them and why Linux uses a one-to-one model (§15.3), and then confronts the hazard that sharing buys you — the **data race**, which we will reproduce and measure losing more than half of a program's work (§15.4). We close by carrying the new proof burden forward toward the concurrency part that makes shared memory correct (§15.5). Threads make sharing trivial; this chapter is about why that is both the point and the danger.

15.1 A thread is a second flow of control in one address space

A process can contain more than one **thread** of execution, and the defining fact of threads is *what they share and what they don't*. All threads in a process **share** one address space: the same code (text), the same heap, the same global and static variables, and the same open file descriptors. What each thread keeps **private** is only what it needs to be an independent flow of control: its own **stack**, its own **registers** (including the program counter and stack pointer), and its own thread-local storage. That is the whole model. Two threads in one process see the same `malloc`'d object at the same address and can both read and write it — directly, through an ordinary pointer, with no copy and no IPC.

Set this against Chapter 14 precisely, because the contrast *is* the concept. When you `fork`, the child gets a *copy* of the address space (copy-on-write), so the parent's and child's variables are separate storage — writes in one are invisible to the other, and that isolation is what contains a crash. When you create a thread, the new thread gets the *same* address space, so a write by one thread is immediately visible to the others — and a wild pointer or a crash in one thread corrupts or kills the *whole* process, all threads with it. Process is copy-and-isolate; thread is share-and-expose. The power of threads (cheap communication through shared memory) and their danger (no isolation, and the races of §15.4) are the same fact viewed twice.

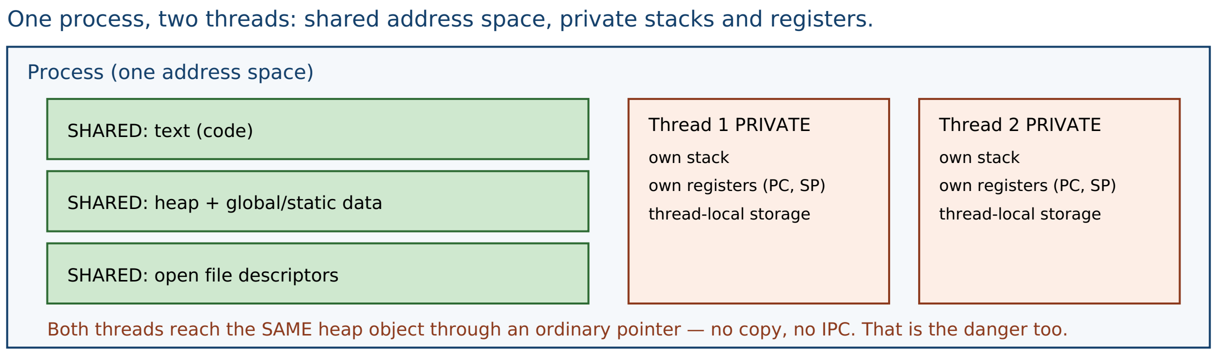


Figure 15.1: Threads in one process share the address space — code, heap, globals, file descriptors — while each keeps a private stack, registers, and thread-local storage. Compare Chapter 14's `fork`, which *copies* the address space. Sharing is why threads communicate cheaply and why they race.

QUICK CHECK

Name two things all threads in a process share, and two things each thread keeps private. If thread A stores 7 into a global variable, can thread B see it — and how does that differ from a parent and child after `fork`? (Answer after the next section.)

15.2 Creating threads: the `pthreads` model

On Unix systems the portable interface is **POSIX threads** (`pthreads`). Two calls carry most of the weight. `pthread_create(&tid, attr, fn, arg)` starts a new thread that begins by running `fn(arg)` concurrently with the caller, and writes the new thread's identifier into `tid`. `pthread_join(tid, &ret)` blocks until that thread finishes and collects its return value — it is the thread analog of Chapter 14's `wait`: create is to fork as join is to wait. The thread function has the fixed signature `void *fn(void *)`; a thread ends by returning from it (or calling `pthread_exit`). You compile such a program with `-pthread`. A minimal two-thread program:

```
void *worker(void *arg){ /* runs concurrently */ return NULL; }

int main(void){
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);   /* start thread 1 */
    pthread_create(&t2, NULL, worker, NULL);   /* start thread 2 */
    pthread_join(t1, NULL);                     /* wait for it, like wait() for a child */
    pthread_join(t2, NULL);
    return 0;
}
```

After the two `pthread_create` calls, three flows of control exist in one address space — the main thread and the two workers — all sharing the heap and globals, each on its own stack. The two `pthread_join` calls make the main thread wait for both to finish before returning, so the process does not exit (and tear down the shared memory the workers are using) out from under them. That is the whole lifecycle: create, run concurrently, join.

15.3 Kernel threads, user threads, and the one-to-one model

Who actually schedules a thread onto a CPU? There are two classic models. In a **user-level** (or **M:N**) model, a runtime library multiplexes many user threads onto a smaller number of kernel-scheduled entities, switching between them in user space; this makes creating and switching threads very cheap, but it has a sharp edge — if one user thread makes a blocking system call, it can stall the underlying kernel thread and thus every user thread mapped onto it. In a **kernel-level** (or **one-to-one**, **1:1**) model, each user thread corresponds to its own kernel-scheduled thread, so the kernel schedules them independently and a blocking call in one blocks only that one, at the cost of a kernel-managed object per thread.

Linux uses the **one-to-one** model, implemented by the **NPTL** (Native POSIX Thread Library, which replaced the older LinuxThreads around the 2.6 kernel). Under the hood, `pthread_create` calls the `clone` system call with a set of flags that ask for sharing — `CLONE_VM` (share the address space), `CLONE_FILES` (share the file-descriptor table), `CLONE_THREAD`, `CLONE_SIGHAND`, and others — so the new thread is a full kernel-scheduled entity that happens to share memory and resources with its creator. This reveals a clean underlying truth: `clone` is the general primitive, and `fork` and thread-creation are two points on its spectrum. `fork` (Chapter 14) is `clone` asking to *copy* the address space; thread creation is `clone` asking to *share* it. A thread and a process are the same kind of schedulable thing; they differ only in how much they share. (User-space schedulers do still exist above the kernel — language runtimes such as Go schedule many lightweight tasks onto a pool of OS threads — but on Linux the OS threads underneath are 1:1 kernel threads.)

> **QUICK CHECK**
>
> Does Linux map each `pthread` to its own kernel-scheduled thread (1:1) or multiplex many onto few (M:N)? Which system call does `pthread_create` use underneath, and which flag makes the new thread share the caller's memory? (Answer below the communicate box.)

15.4 The shared-memory hazard: the data race

Now the predict. Threads share memory, so two threads can touch the same variable at the same time — and that is where correctness goes to die. Take the simplest possible shared operation, `counter++` on a shared integer. It looks atomic in C, but it is not: at the machine level it is a **read-modify-write** — load the current value into a register, add one, store it back — three steps that can interleave with another thread's three steps. If thread A loads 5, thread B loads 5, both add one, both store 6, then two increments produced a net change of *one*: an update was lost. Run this at scale and the losses pile up. Here is the exact program from the opener — four threads, each incrementing a shared counter one million times, no locking — compiled and run on this machine (gcc 11.4.0, `-O2`, x86-64), five times:

```
void *bump(void *a){ for (int i = 0; i < 1000000; i++) counter++; return NULL; }
/* 4 threads all run bump() on the same shared `counter`, then we join and print it */

expected = 4000000, got = 1092939
expected = 4000000, got = 1170537
expected = 4000000, got = 1372703
expected = 4000000, got = 1810815
expected = 4000000, got = 1324249
```

The program should have counted to four million. It counted to between roughly 1.1 and 1.8 million — **more than half of the four million increments simply vanished**, and the answer was different every run. This is a **data race**: concurrent accesses to the same memory location, at least one of them a write, with no synchronization ordering them. In C and C++ a data race is **undefined behavior** — Chapter 10's UB returns, now arising from concurrency rather than from a bad pointer or an overflow — and the nondeterministic, mostly-wrong output above is exactly what UB looks like when it manifests. The lost updates come straight from the interleaved read-modify-writes: whenever two threads read the same value before either stores, one increment is overwritten and lost.

THE SAME IDEA THREE WAYS: A DATA RACE LOSES UPDATES

1. In words. `counter++` is not atomic — it is load, add, store. Two threads running it on the same variable can both load the same value, both add one, and both store, so their two increments net only one. Nothing orders them, so results are nondeterministic. That unordered, conflicting access is a data race, and in C/C++ it is undefined behavior.

2. As a picture. An interleaving timeline: A loads 5 · B loads 5 · A adds → 6 · B adds → 6 · A stores 6 · B stores 6. Two increments, one net change.

3. As numbers. Four threads, one million increments each, no lock: expected 4,000,000; measured on this machine ~1.1–1.8 million across five runs — over half the work lost, and different every time. Adding synchronization (a mutex or an atomic increment) restores exactly 4,000,000.

TRAP: "IT PRINTED THE RIGHT ANSWER ON MY MACHINE, SO THE CODE IS CORRECT"

A data race is undefined behavior *whether or not it visibly misbehaves on a given run*, and it can easily produce the right answer while being thoroughly broken — which is exactly what makes it dangerous. **Correct-looking output is not evidence of correctness for racing code.** Writing this very chapter, an earlier version of the counter program — compiled at `-O2` without the variable marked `volatile` — printed `4000000` on every single run, because the optimizer had collapsed each thread's million-iteration loop into a single add, shrinking the race window to almost nothing so the threads happened to serialize. The bug was fully present (four threads doing unsynchronized read-modify-writes on shared memory); it simply did not *manifest*, just as Chapter 10's undefined behavior need not misbehave to be undefined. A race that surfaces only under a particular compiler, optimization level, CPU load, or core count will pass every test on your laptop and corrupt data in production. The lesson: reason about shared-memory correctness from the *rules* (is there an unsynchronized conflicting access? then it is a race, full stop), never from whether a test run looked fine. The fix is real synchronization — a mutex around the update, or an atomic increment — not a comforting green test.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Threads are just cheaper processes. Let's convert our process-per-request server to threads across the board for the speed win." Cover the paragraph and respond in three or four sentences, drawing on §15.1 and §15.4.

Model answer. "Threads *are* cheaper — they share the address space instead of copying it, so creation skips the copy-on-write setup and they can communicate through shared memory with no IPC — but 'cheaper processes' misses the crucial difference: threads drop the isolation that processes gave us. A memory bug or crash in one thread takes down the whole process — every in-flight request — where a bad process only took itself down, and every piece of mutable state we now share becomes a potential data race, which is undefined behavior. That is not theoretical: an unsynchronized shared counter under four threads lost over half its increments in our measurement and gave a different wrong answer each run, and a race can even pass tests by printing the right number. So threads are right where we want cheap shared-memory concurrency and can enforce synchronization discipline; keep processes where fault containment or a security boundary matters. Let's not convert blindly — decide per component and protect every shared write." Naming the isolation trade, the data-race hazard, and refusing the blanket swap is the senior register.

(Answer to the §15.3 quick check: Linux uses the one-to-one (1:1) model via NPTL — each `pthread` is its own kernel-scheduled thread. `pthread_create` uses the `clone` system call underneath, and `CLONE_VM` is the flag that makes the new thread share the caller's address space.)

15.5 What to carry forward — and toward concurrency

A **thread** is a second flow of control inside one process: all threads **share** the address space — code, heap, globals, file descriptors — while each keeps a **private** stack, registers, and thread-local storage. You create and reap them with `pthreads` — `pthread_create` starts a thread running a `void *fn(void *)`, `pthread_join` waits for it and collects its result (create/join is the thread's fork/wait). Linux schedules threads **one-to-**

one with kernel threads via NPTL, and `pthread_create` is really the `clone` system call asking to share memory (`CLONE_VM`) — the same primitive `fork` uses to *copy* it, so a thread and a process differ only in how much they share. The price of sharing is the **data race**: unsynchronized conflicting access to the same memory, which is undefined behavior in C/C++ and which we measured losing more than half of a four-million-increment workload, nondeterministically — and which can hide by printing the right answer, so correctness must be reasoned from the rules, not from a passing run.

That last point reopens the theme of Part 2 at a new level. Chapter 12 said that in C you carry, by hand, a proof about lifetimes, bounds, and freeing. Threads add a clause: *no unsynchronized concurrent access to shared mutable memory* — and violating it is undefined behavior just like the rest. Making shared-memory concurrency correct is a large subject of its own: the synchronization primitives that order accesses (locks, condition variables, atomics), the lock-free techniques that avoid them, and the memory model that defines precisely what a multithreaded program means. That is the concurrency-and-parallelism part of this volume, which the data race you just measured is the doorway into. And it is one more place where a language can help: just as Rust's ownership makes use-after-free a compile error, its type system also encodes which data may cross between threads and which must be synchronized — turning "don't race" from a discipline you enforce by hand into an invariant the compiler checks, the payoff this whole part has been building toward.

CAN YOU… WITHOUT LOOKING?

- State what threads in a process share and what each keeps private, and contrast that with parent/child after `fork`.
- Write a minimal two-thread `pthreads` program and explain `pthread_create`/ `pthread_join` as the thread analog of `fork`/`wait`.
- Explain the difference between 1:1 and M:N threading, say which Linux uses, and name the system call and flag underneath `pthread_create`.
- Explain why `counter++` from two threads can lose updates, decomposing it into load/add/store with an interleaving.
- State what a data race is, why it is undefined behavior, and why a passing test does not prove racing code correct.
- Give the new clause threads add to Part 2's hand-carried proof, and name the fix category (synchronization).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does a thread share with its process's other threads, and what does it keep private? (§15.1)
2. **R2.** What do `pthread_create` and `pthread_join` do, and what earlier pair are they analogous to? (§15.2)
3. **R3.** What is the 1:1 threading model, which library implements it on Linux, and what system call/flag underlie a thread? (§15.3)
4. **R4.** Why can two threads doing `counter++` lose updates, and what did the measurement show? (§15.4)
5. **R5.** What is a data race, why is it undefined behavior, and why can it pass a test yet be wrong? (§15.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Concurrency: An Introduction" and "Interlude: Thread API."** The free chapters that introduce threads, the shared/private split, the `pthreads` API, and the race that motivates all of concurrency. The reference for §15.1–15.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 12 (Concurrent Programming).** Threads, shared variables, and races from the programmer's view.
- **The Linux `pthreads(7)` and `clone(2)` manual pages.** Primary sources for the NPTL one-to-one model of §15.3 and the exact `clone` flags (`CLONE_VM`, `CLONE_FILES`, `CLONE_THREAD`, …) that distinguish a thread from a `fork`. Verify per system.
- **The C11/C++11 standard's memory model (see cppreference.com).** The authority for why a data race is undefined behavior and what "synchronizes-with" means — the formal ground the concurrency part builds on.

Exercises

15.1 WARM-UP Name two things all threads in a process share, and two things each thread keeps private.

15.2 WARM-UP What does `pthread_create` take as arguments, and what does `pthread_join` do?

15.3 WARM-UP Does Linux use a 1:1 or an M:N threading model, and which system call does `pthread_create` use underneath?

15.4 CORE Write, from memory, a minimal program that starts two threads running the same function and waits for both. Explain how `pthread_create`/`pthread_join` mirror Chapter 14's `fork`/`wait`, and why the joins are necessary before `main` returns.

15.5 CORE Explain why two threads each doing `counter++` on a shared variable can lose updates. Decompose `counter++` into its machine-level steps and give a concrete interleaving in which two increments produce a net change of one.

15.6 CORE Contrast a process and a thread on four axes: address space, isolation, cost to create, and what data sharing costs you. For each, say which is cheaper/safer and why.

15.7 COST (a) Why is creating a thread cheaper than `fork`ing a process — what work does thread creation skip? (b) What new cost or hazard do you take on in exchange for sharing memory? (c) The measured demo lost more than half of its increments and gave a different total each run; is that loss deterministic, and what does its nondeterminism imply for whether tests can be trusted to catch it?

15.8 FADED *Worked, with the last step for you.* A web service keeps a shared `long hits` counter that each request thread increments with `hits++`. Under load, the reported hit count is noticeably lower than the true number of requests — but every unit test passes.
Step 1 (given): `hits++` is a non-atomic read-modify-write on memory shared by all request threads.
Step 2 (given): The unit tests exercise one request at a time, so no two increments ever overlap and nothing races.
Step 3 (given): Under real load, many threads increment concurrently and their read-modify-writes interleave, so some increments overwrite others.
Step 4 (you): Name the bug precisely, say whether it is undefined behavior, explain why the passing tests failed to catch it, and name the category of fix (and one concrete mechanism) that makes the count correct.

15.9 MIXED For each need, choose a separate *process* (via `fork`) or a *thread*, and justify in a line: (a) run an untrusted plugin so that if it crashes or corrupts memory it cannot take down the host; (b) compute a parallel sum over a large in-memory array that all workers read; (c) run a command the user typed at a shell.

15.10 MIXED Drawing on Chapters 9–15, classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*: (a) two threads writing the same non-atomic `long` with no synchronization; (b) two threads each writing a *different* variable with no synchronization; (c) a thread reading through a pointer whose target was already `free`d; (d) two threads reading (never writing) the same shared value.

15.11 STRETCH Chapter 12 said that in C the programmer carries, by hand, a proof about lifetimes, bounds, and freeing. (a) State the additional clause that threads add to that proof. (b) Explain why violating it is undefined behavior and why, unlike a use-after-free, it may never reproduce in testing. (c) In one or two sentences, describe how a language could move this from a runtime hope to a compile-time guarantee — for example, a type system that tracks which data may be shared across threads and which accesses must be synchronized (Rust's `Send`/`Sync` and borrow checking; forward reference, prose only).

15.12 LAB On your own machine: (a) run the shared-counter race — four threads, one million increments each of one shared counter, no lock — five times, and report the wrong totals and how much they vary run to run. (b) Add a mutex around the increment (or use an atomic increment) and confirm the total is now exactly four million every time. (c) Try building the unsynchronized version at `-O0` versus `-O2`, and with the counter marked `volatile` or not; report which combinations make the bug visible and which hide it — and explain why a "correct" run does not mean the code is correct. Label your compiler, core count, and hardware.

Chapter 16 — CPU Scheduling

Before you start: Chapter 15 (Linux threads are 1:1 kernel-scheduled entities — *these* are what the scheduler picks among), Chapter 14 (a process blocks in `wait` or on I/O — a blocked task is off the CPU and off the scheduler's radar), Chapter 13 (a system call is a trap into the kernel — where the scheduler lives). · Chapters 14 and 15 gave you more runnable things than you have CPUs. This chapter is about the OS's answer to the question that creates: *who runs next, and for how long?*

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 14 — when a process calls `read` on a socket with no data, it *blocks*; is a blocked process using the CPU while it waits? (2) Chapter 15 — Linux schedules each `pthread` one-to-one with a kernel thread; so is the unit the scheduler places on a CPU the process or the thread? (3) *Predict:* three jobs arrive at almost the same instant — one needs 100 ms of CPU, the other two need 10 ms each. If the scheduler runs them strictly in arrival order (the 100 ms job first), what is the *average* time from arrival to completion across the three? What if it ran the two short ones first instead? Write both numbers down — we will compute them exactly in §16.2.

A modern machine has a handful of CPUs and, at any moment, dozens or hundreds of runnable threads. Something must decide which runnable thread occupies each CPU and when to take it away and give the CPU to another — that something is the **scheduler**, and the decision is **scheduling**. Chapter 13 gave the mechanism the scheduler rides on (a timer interrupt traps into the kernel, which can switch the running thread); this chapter is about the **policy** — the algorithm that chooses. We start by naming what a scheduler optimizes and the metrics that make "good" precise (§16.1); build up the classic policies from the simplest, FIFO, through shortest-job-first and its convoy trap (§16.2); add preemption and the time slice with round-robin, exposing the response-vs-turnaround trade-off (§16.3); and then meet the schedulers real systems ship — the multi-level feedback queue and Linux's proportional-share scheduler — which approximate the ideal without an oracle (§16.4). We close by carrying forward what "the OS multiplexes the CPU" really costs and buys (§16.5).

16.1 The scheduling problem, and how to measure a scheduler

At any instant each thread is in one of a few states. A **running** thread is on a CPU right now. A **ready** (runnable) thread could run but no CPU is free for it. A **blocked** thread is waiting for something — I/O to complete, a child to exit, a lock to free — and *cannot* run until that event arrives, so it is not the scheduler's concern until it wakes. The scheduler's job is to pick, from the ready threads, which one each CPU runs, and a **context switch** (Chapter 13's mechanism) is how it hands a CPU from one thread to another: save the outgoing thread's registers, load the incoming thread's, resume. The switch is not free — it costs the direct register save/restore plus the indirect cost of a cold cache and TLB (Chapter 3) for the new thread — which is why the scheduler cannot simply switch on every instruction.

"Good" needs a definition, and there are several, often in tension. The two that matter most here are **turnaround time** — completion time minus arrival time, how long the whole job took to finish, the metric a batch workload cares about — and **response time** — first-run time minus arrival time, how long until the job *started* getting served, the metric an interactive user feels as lag. A scheduler also cares about **throughput** (jobs completed per unit time), **fairness** (each job gets a defensible share), and avoiding **starvation** (no ready job waits forever). The reason scheduling is hard is that these goals conflict: as we will see, the policy that minimizes average turnaround is not the one that minimizes response time, and the one that is "fair" may not be either.

> **QUICK CHECK**
>
> Define turnaround time and response time in one line each. A job arrives at t=0, first gets the CPU at t=5, and finishes at t=30. What is its response time and its turnaround time? (Answer after the next section.)

16.2 FIFO, shortest-job-first, and the convoy effect

The simplest possible policy is **FIFO** (first-in, first-out, also called FCFS): run jobs to completion in arrival order, no preemption. It is obviously fair in one sense and trivial to implement — and it has a sharp failure mode. Take the opener's workload: three jobs, A needs 100 ms and B and C need 10 ms each, all arriving at essentially t=0 with A first. FIFO runs A to completion, then B, then C. A finishes at 100, B at 110, C at 120, so average turnaround is (100 + 110 + 120) / 3 = **110 ms**. The two short jobs sat behind the long one for almost their entire wait. This is the **convoy effect**: short jobs pile up behind a long one like cars behind a truck, and average turnaround balloons because everyone inherits the long job's runtime.

The fix, if you knew the run times, is to run the short jobs first. **Shortest-Job-First (SJF)** schedules the ready jobs in increasing order of length. On the same workload it runs B, then C, then A: B finishes at 10, C at 20, A at 120, and average turnaround drops to (10 + 20 + 120) / 3 = **50 ms** — less than half of FIFO's, for the exact same jobs. That is not a coincidence: when all jobs arrive together, SJF is *provably optimal* for average turnaround (any swap that puts a longer job earlier only adds its length to more jobs' waits). When jobs arrive over time, the preemptive version — **Shortest-Time-to-Completion-First (STCF)**, which preempts the running job if a newly arrived one has less work left — extends the same idea. These are the numbers you predicted; here they are, computed by a scheduling simulator:

```
workload: A=100ms, B=10ms, C=10ms, all arrive ~t=0
FIFO (order A,B,C): finish A=100 B=110 C=120  -> avg turnaround 110.0 ms
SJF  (order B,C,A): finish B=10  C=20  A=120  -> avg turnaround  50.0 ms
```

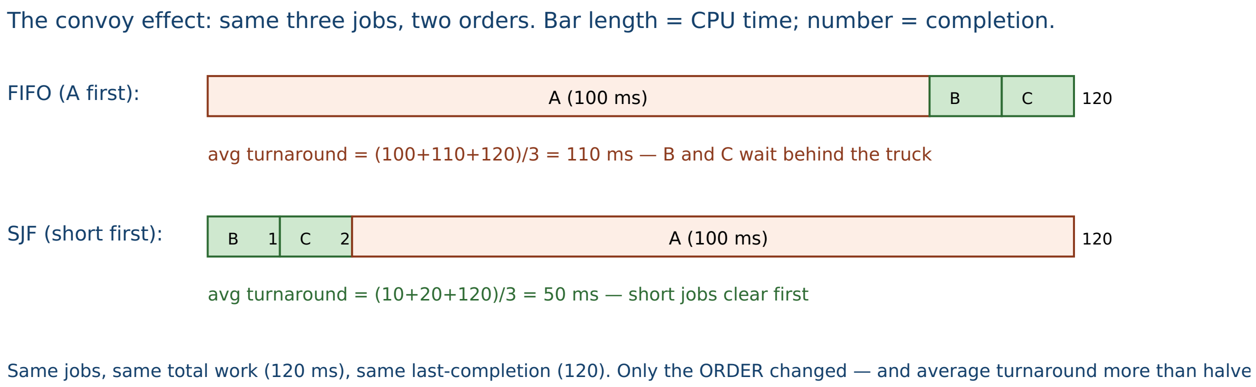


Figure 16.1: FIFO vs SJF on A=100, B=10, C=10 (arriving together). Running the long job first makes every short job inherit its runtime (the convoy effect); running short jobs first minimizes average turnaround — which SJF provably does when all jobs arrive at once.

THE SAME IDEA THREE WAYS: ORDER DOMINATES AVERAGE TURNAROUND

1. In words. Turnaround is completion minus arrival. When jobs share a CPU, a job's turnaround includes the runtime of everything scheduled before it. So putting a long job first adds its length to every later job's turnaround; putting short jobs first minimizes the total. That is why SJF beats FIFO on average turnaround.

2. As a picture. Figure 16.1: the same three bars, packed truck-first versus short-first. The sum of the completion marks — not the total width — is what average turnaround measures, and reordering shrinks it.

3. As numbers. A=100, B=10, C=10: FIFO averages (100+110+120)/3 = 110 ms; SJF averages (10+20+120)/3 = 50 ms. Identical work, identical finish of the last job (120 ms), less than half the average turnaround — purely from order.

(Answer to the §16.1 quick check: response time = first-run − arrival = 5 − 0 = 5; turnaround = completion − arrival = 30 − 0 = 30.)

16.3 Preemption, the time slice, and round-robin

SJF minimizes turnaround, but it has two problems. First, it needs an **oracle** — you must know each job's length in advance, which for interactive and server workloads you never do. Second, even with the oracle its *response* time is bad: a job that arrives just after a long one starts must wait for it to finish (or, with STCF, only if it is shorter). Interactive workloads care about response time above all — you want the cursor to move the instant you type — so we need a policy that shares the CPU in *small slices* rather than running each job to completion. That policy is **round-robin (RR)**: run each ready job for a fixed **time slice** (quantum), then preempt it (via the timer interrupt) and move to the next, cycling forever until jobs finish.

Round-robin's whole point is response time. On the A/B/C workload with a 1 ms slice, all three jobs get their first turn within the first few milliseconds, so every job's response time is essentially immediate (about 1 ms on average) rather than SJF's — where C waited 10 ms and A waited 20. But that responsiveness has a price: RR's average *turnaround* is worse than SJF's, because it stretches every job out by interleaving. Here are all three policies on the same workload from the simulator:

```
workload: A=100ms, B=10ms, C=10ms
FIFO:        avg turnaround 110.0 ms,  avg response  70.0 ms  (B waits 100, C waits 110)
SJF:         avg turnaround  50.0 ms,  avg response  10.0 ms  (C waits 10, A waits 20)
RR (q=1ms):  avg turnaround  59.7 ms,  avg response  ~1.0 ms  (every job starts almost at
once)
```

That is the fundamental trade-off in one table: **SJF wins turnaround, round-robin wins response, and you cannot have both**. The **size** of the time slice is the knob. A tiny slice gives excellent response (jobs cycle fast) but pays a context-switch cost on every single switch — and if the slice approaches the switch cost itself, the CPU spends a large fraction of its time switching rather than working. A large slice amortizes the switch cost (better throughput) but degrades response time toward FIFO's. Real schedulers pick a slice large enough that the switch overhead is a small percentage, and small enough that interactivity stays crisp — typically a few milliseconds.

> **QUICK CHECK**
>
> Round-robin makes the time slice small to improve one metric and worsen another — which is which? And what goes wrong if you make the slice as small as the cost of a single context switch? (Answer below the communicate box.)

16.4 What real schedulers do: MLFQ and proportional share

Real systems cannot use pure SJF (no oracle) or pure round-robin (ignores that jobs differ in importance and behavior). Two ideas dominate real schedulers, and both aim to get SJF-like behavior — favor short, interactive work — *without* knowing job lengths in advance, by **learning from the past**.

The first is the **multi-level feedback queue (MLFQ)**. Keep several ready queues, each a priority level; always run a job from the highest non-empty level, round-robin within a level. The trick is how a job's priority *changes*: a new job enters at the top; if it uses its whole time slice (CPU-bound behavior) it is demoted a level; if it gives up the CPU before the slice ends (blocking on I/O — interactive behavior) it stays high. Over a few slices this *learns* each job's character: interactive jobs, which block often, float near the top and get scheduled quickly (great response time); CPU-bound jobs sink to the bottom and share the leftovers. That approximates "short jobs first" using observed behavior instead of an oracle. Two refinements make it work in practice: to prevent a long job from **starving** at the bottom forever, MLFQ periodically **boosts** every job back to the top; and to stop a job from **gaming** the rule (yielding just before each slice ends to keep its priority), it accounts for total CPU used at a level rather than per-slice behavior.

The second idea is **proportional share** (fair-share): rather than juggle priorities, give each job a **weight** and hand out CPU time in proportion to weight, so a job with twice the weight gets twice the CPU over time. This is the family Linux's normal scheduler belongs to. For over a decade Linux used the **Completely Fair Scheduler (CFS)**, which tracks each task's accumulated virtual runtime and always runs the task that has had the least, weighted by priority; kernel 6.6 (2023) replaced CFS with **EEVDF** (Earliest Eligible Virtual Deadline First), a proportional-share scheduler with better latency guarantees. In both, a process's **nice** value (−20 to +19, set with `nice`/`renice`; lower is "less nice," i.e. higher priority) sets its weight: a lower nice value claims a larger share. You can see this directly. Two identical CPU-bound loops pinned to the *same* core so they must compete, one at nice 0 and one at nice 19, run for five wall-clock seconds on this machine (Linux 6.18, which uses EEVDF; numbers illustrative):

```
taskset -c 0 nice -n 0  ./hog    ->  nice0:  cpu-seconds used = 4.66
taskset -c 0 nice -n 19 ./hog    ->  nice19: cpu-seconds used = 0.09
```

The nice-0 loop got roughly fifty times the CPU of the nice-19 loop over the same wall-clock window — not because nice 19 was "paused," but because the proportional-share scheduler weighted its slice far smaller. That is the whole idea made visible: `nice` does not change what a program computes, only what *fraction of the CPU* it is entitled to when there is contention. (Linux also offers strict real-time classes — `SCHED_FIFO` and `SCHED_RR` — that sit above the normal class and always preempt it; those are for bounded-latency work and are a foot-gun for everything else, as the warning below explains.)

TRAP: REACHING FOR REAL-TIME PRIORITY (`SCHED_FIFO`) TO "MAKE IT FASTER"

When a latency-sensitive service is being starved by a background job, the tempting fix is to promote the service to a real-time scheduling class — `SCHED_FIFO` or `SCHED_RR` — so it always wins. This is *strict fixed-priority* scheduling, and it does not mean "a bit faster": a runnable `SCHED_FIFO` thread preempts **every** normal task and runs until it blocks or a higher real-time thread appears. If that thread is CPU-bound and never blocks — a busy loop, a spin-wait, a bug — it will monopolize its core and **starve everything else on it, including kernel threads**, and can wedge the machine hard enough to require a reboot. (Linux mitigates this with real-time throttling — `sched_rt_runtime_us` caps RT time per period — precisely because the default is so dangerous.) Real-time priority is the right tool only for work that is both genuinely latency-critical *and* bounded in CPU (it blocks regularly). To bias the normal scheduler toward an important service, reach first for `nice` or cgroup CPU weights, which change the *share* under contention without the power to starve the system. The lesson: "higher priority" is not a speed dial; strict priority is a loaded gun pointed at every other task on the CPU.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our request service and the nightly compaction job share a box, and compaction is starving the service's tail latency. Let's just give the service `SCHED_FIFO` real-time priority so it always wins." Cover the paragraph and respond in three or four sentences, drawing on §16.4.

Model answer. "The instinct is right — the service should win against a background batch job — but `SCHED_FIFO` is a much bigger hammer than we want: it is strict fixed priority, so if any of the service's threads ever spins or busy-loops without blocking, it will preempt everything on that core, including kernel threads, and can hang the box, which is why the kernel even throttles real-time time by default. What we actually want is a bigger *share* under contention, not absolute priority — so lower the service's `nice` value or, better, put the two workloads in separate cgroups and give the service a larger CPU weight (and cap or de-prioritize compaction). That fixes the starvation through the fair scheduler, keeps the system stable, and leaves headroom if compaction ever needs to burst. Save real-time scheduling for work that is both latency-critical and provably bounded in CPU." Distinguishing "bigger share" from "strict priority," and naming the starvation and hang risk, is the senior register.

*(Answer to the §16.3 quick check: a small time slice improves **response** time (jobs cycle onto the CPU quickly) and worsens **turnaround** and throughput (more interleaving and more context switches). If the slice approaches the cost of one context switch, the CPU spends a large fraction of its time switching rather than doing useful work — overhead dominates.)*

16.5 What to carry forward

The **scheduler** decides which ready thread each CPU runs and for how long; a **context switch** is the (non-free) mechanism that hands a CPU from one thread to another. We measure schedulers by **turnaround** (completion − arrival) and **response** (first-run − arrival) time, among others, and the central fact is that these goals conflict. FIFO is simple but suffers the **convoy effect**; **SJF/STCF** minimize average turnaround but need an oracle and give poor response; **round-robin** with a small time slice wins response at the cost of turnaround, with the slice size trading responsiveness against switch overhead. Real schedulers approximate "favor short, interactive work" without an oracle by learning from behavior — the **multi-level feedback queue** (priorities that adapt, with boosting to prevent starvation) — or by handing out CPU in proportion to a **weight** — Linux's proportional-share scheduler (CFS, then EEVDF from 6.6), where the **nice** value sets the share, which we watched split one core roughly fifty to one between a nice-0 and a nice-19 loop.

Step back and see what this part is assembling. Chapter 14 gave you many processes and Chapter 15 many threads; this chapter is how one machine *pretends* to run them all at once — by slicing the CPU in time so fast that each thread perceives a CPU (nearly) to itself. That is the CPU half of the operating system's core illusion. The other half is the same trick played on *memory*: giving each process the illusion of a large private address space on a machine whose physical memory is smaller and shared. Chapter 4 showed the hardware that makes an address virtual; the next chapter is the OS policy that makes the illusion hold when memory runs short — demand paging, page faults as a mechanism, and the replacement decisions that are this chapter's turnaround-versus-response trade-off played again on pages instead of milliseconds.

CAN YOU… WITHOUT LOOKING?

- Name the three scheduling-relevant states of a thread (running, ready, blocked) and say which ones the scheduler chooses among.
- Define turnaround time and response time, and compute both for a job given its arrival, first-run, and completion times.
- Explain the convoy effect with the A=100/B=10/C=10 example, and give FIFO's and SJF's average turnaround (110 vs 50 ms).
- State why SJF is optimal for average turnaround yet unusable in practice, and what round-robin trades to win response time.
- Explain how a multi-level feedback queue approximates SJF without an oracle, and why it needs priority boosting.
- Explain what a `nice` value does under Linux's proportional-share scheduler, and why `SCHED_FIFO` is dangerous.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What are the running/ready/blocked states, and which does the scheduler pick among? (§16.1)
2. **R2.** Define turnaround and response time. (§16.1)
3. **R3.** What is the convoy effect, and what were FIFO's and SJF's average turnarounds on A/B/C? (§16.2)
4. **R4.** Why does round-robin improve response time but hurt turnaround, and what does the time-slice size trade? (§16.3)
5. **R5.** How does an MLFQ learn to favor interactive jobs without knowing their lengths, and what does a `nice` value control? (§16.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Scheduling: Introduction" and "Scheduling: The Multi-Level Feedback Queue."** The free chapters that develop turnaround/response metrics, FIFO/SJF/STCF/round-robin, and MLFQ (including boosting and anti-gaming). The reference for §16.1–16.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8 (Exceptional Control Flow).** Context switches and the timer interrupt that preemption rides on, from the programmer's view.
- **The Linux `sched(7)` and `nice(2)` / `sched_setscheduler(2)` manual pages.** Primary sources for the scheduling classes (`SCHED_OTHER`/normal, `SCHED_FIFO`, `SCHED_RR`), the nice range, and real-time throttling. Verify per kernel version.
- **The Linux kernel scheduler documentation (`docs.kernel.org/scheduler/`), including the EEVDF scheduler page.** The authority for the CFS-to-EEVDF change in 6.6 and how proportional share is implemented. Fast-moving; treat details as version-specific.

Exercises

16.1 WARM-UP Name the three scheduling-relevant states of a thread, and say which state(s) the scheduler chooses the next runner from.

16.2 WARM-UP Define turnaround time and response time. For a job that arrives at t=0, first runs at t=8, and completes at t=40, give both.

16.3 WARM-UP In one sentence, what is the convoy effect, and which simple scheduling policy suffers from it?

16.4 CORE Three jobs arrive at t=0: X needs 8 ms, Y needs 4 ms, Z needs 4 ms. Compute the average turnaround time under FIFO (order X, Y, Z) and under SJF, showing each job's completion time. Which is lower, and why is that no accident?

16.5 CORE Explain why SJF minimizes average turnaround when all jobs arrive together, and give the two reasons it is nonetheless unusable as a general-purpose scheduler.

16.6 CORE Describe how a multi-level feedback queue decides a job's priority over time, and explain how this makes interactive (I/O-bound) jobs get scheduled quickly and CPU-bound jobs sink — without the scheduler ever being told which is which.

16.7 COST (a) What does shrinking the round-robin time slice buy you, and what does it cost? (b) A context switch on a machine costs roughly a few microseconds; if you set the time slice equal to that, what fraction of the CPU is wasted on switching, and why? (c) Give the qualitative reason real schedulers use a slice of a few milliseconds rather than a few microseconds or a few seconds.

16.8 FADED *Worked, with the last step for you.* A batch analytics job and an interactive dashboard API share one server. Users complain the dashboard is laggy whenever the batch job runs, even though the box has spare capacity at night.
Step 1 (given): The dashboard is I/O-bound and latency-sensitive (response time matters); the batch job is CPU-bound and only turnaround matters.
Step 2 (given): Under a pure round-robin or a naive equal-share policy, the CPU-heavy batch job gets an equal turn each cycle, so the dashboard waits behind it and its response time suffers.
Step 3 (given): The two workloads should not get equal CPU: the dashboard should be favored under contention, but the batch job must still make progress (not starve).
Step 4 (you): Name a concrete Linux mechanism (not real-time priority) that gives the dashboard a larger share under contention while letting the batch job run when the dashboard is idle, and say in one line why this is safer than `SCHED_FIFO`.

16.9 MIXED For each situation, name the scheduling metric that matters most (turnaround, response time, or throughput), and justify in a line: (a) a text editor reacting to keystrokes; (b) a nightly batch job that must finish before the business day; (c) a build farm maximizing jobs completed per hour across many machines.

16.10 MIXED Drawing on Chapters 13–16, classify each claim as *true* or *false* and fix the false ones: (a) "A process blocked on `read` is consuming CPU while it waits." (b) "On Linux the scheduler picks among processes, not threads." (c) "A context switch is free, so the time-slice size does not matter." (d) "Lowering a process's `nice` value increases the CPU share it gets under contention."

16.11 STRETCH Starvation is a scheduler's cardinal sin. (a) Show how strict shortest-job-first can starve a long job indefinitely if short jobs keep arriving. (b) Explain how a multi-level feedback queue's periodic priority *boost* prevents this, and what would go wrong without it. (c) Explain why a proportional-share scheduler (weight-based) does not starve a low-weight job even under load — contrast "smaller share" with "no share," and connect this to why `nice` cannot starve a task but `SCHED_FIFO` can.

16.12 LAB On your own machine: (a) write a CPU-bound loop that runs for a fixed number of wall-clock seconds and reports the CPU-seconds it actually accumulated (via `CLOCK_THREAD_CPUTIME_ID`). (b) Run two copies pinned to the *same* core with `taskset -c 0` so they must compete, one at `nice 0` and one at `nice 19`, and report the CPU-seconds each got — you should see the nice-0 copy dominate by a large ratio. (c) Now run them on *different* cores (or without pinning on a multi-core box) and explain why the nice difference nearly disappears. Label your kernel version, core count, and scheduler (CFS vs EEVDF). Explain what `nice` did and did not change.

Chapter 17 — Virtual Memory: Demand Paging and Page Replacement

Before you start: Chapter 4 (the hardware that makes an address virtual — the MMU, page tables, and the TLB translate a virtual page to a physical frame; *this* chapter is the OS policy that fills those tables), Chapter 16 (the OS multiplexes the CPU in time; it multiplexes memory in space the same way), Chapter 13 (a page fault is exceptional control flow — a trap into the kernel). · Chapter 4 showed *how* a virtual address becomes a physical one. This chapter is what the OS does when the answer is "that page isn't in memory."

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 4 — an address is *virtual*; what hardware structure translates a virtual page number to a physical frame, and what small cache speeds that translation up? (2) Chapter 16 — the scheduler faced conflicting goals (turnaround vs response); name the sin a scheduler must avoid (a job that never runs). (3) *Predict:* a program calls `mmap` to reserve 512 MiB of memory and immediately checks its resident memory (RSS) *before touching any of it*. Is its RSS about 512 MiB, about 0, or somewhere in between? Then it writes to every byte and checks again. Write down both guesses — we will measure them in §17.2.

Chapter 4 gave a process the illusion of a large, private, contiguous address space, and showed the hardware — the MMU walking a page table, the TLB caching recent translations — that turns each virtual address into a physical one on every memory access. But that chapter assumed the page was *there*: that every virtual page a process uses has a physical frame behind it. It usually does not, and cannot: the sum of every process's address space is far larger than physical RAM. The operating system sustains the illusion anyway, and this chapter is how. We reframe memory as the OS sees it — physical frames it hands out and page tables it owns (§17.1); introduce **demand paging**, where a page gets a frame only when first touched, and the **page fault** that makes it happen — measured (§17.2); go beyond physical memory to **swapping** and the replacement question it forces (§17.3); and work through the replacement policies — OPT, FIFO, LRU — and the anomaly that makes FIFO treacherous (§17.4). We close by carrying forward the mechanisms this unlocks — `mmap`, copy-on-write, the page cache — that the rest of the book runs on (§17.5).

17.1 Memory as the OS sees it: frames, page tables, and the illusion

Recall the two vocabularies. A process sees **virtual pages**: its address space, chopped into fixed-size pages (4 KiB on x86-64 by default). Physical memory is chopped into equal-size **frames**. A **page table**, one per process and owned by the OS, maps each virtual page to a frame — or records that it has none. Each page-table entry (PTE) holds the frame number plus control bits, and the one that matters here is the **present bit**: set if the page currently lives in a frame, clear if it does not. On every access the MMU consults the page table (via the TLB); if the translation says present, the access proceeds at hardware speed. This is the machinery of Chapter 4, now seen from the OS's chair: the OS is the party that *fills in* those tables, hands out frames, and decides what to do when a PTE's present bit is clear.

That last case is the whole subject of this chapter. Because every process has its own page table, each gets its own private virtual address space — the same virtual address in two processes maps to different frames, which is the isolation Chapter 14 relied on. And because the OS controls the present bit, it can lie productively: it can promise a process a page it has not actually backed with a frame yet, and make good on the promise only when the process reaches for it. When the process touches a page whose PTE is not present, the hardware cannot proceed — so it traps.

> **QUICK CHECK**
>
> What does the *present bit* in a page-table entry mean, and who sets it — the hardware or the OS? If a process accesses a virtual page whose PTE has the present bit clear, what happens next? (Answer after the next section.)

17.2 Demand paging and the page fault, measured

When a running instruction accesses a virtual page whose PTE is not present, the MMU raises a **page fault** — a hardware exception, Chapter 13's exceptional control flow, that traps into the kernel's page-fault handler. The handler inspects why the page is absent and chooses: if the access is *legal* but the page simply was never materialized (a freshly `mmap`'d region, the first touch of newly allocated heap), it finds a free frame, fills it appropriately — zeroed for anonymous memory, read from a file for a file mapping — sets the PTE's present bit, and restarts the faulting instruction, which now succeeds. If the access is *illegal* (a wild pointer into an unmapped region), the handler delivers `SIGSEGV` instead — the segmentation fault of Chapters 7–11, now revealed as "the page-fault handler found no valid mapping." This lazy strategy — give a page a frame only when it is first used — is **demand paging**, and it is why allocating memory is cheap: `malloc` or `mmap` mostly just records a promise in the page tables; the physical frames are spent only on the pages you actually touch.

You can watch it directly. A program that `malloc`s 100 MiB and writes one byte to every 4 KiB page should take exactly one page fault per page — 100 MiB / 4 KiB = 25,600 of them — and, because anonymous pages are served from zeroed frames with no disk I/O, they should all be **minor** faults (frame available without I/O), never **major** (required disk I/O). Running it under `/usr/bin/time -v` (gcc 11.4.0, Linux 6.18, x86-64):

```
Maximum resident set size (kbytes): 103852        # ~100 MiB actually resident after touching
Minor (reclaiming a frame) page faults: 25672     # ~one per 4 KiB page touched (25,600) + a
little
Major (requiring I/O) page faults: 0              # no disk I/O — anonymous demand-zero pages
```

And the reservation-versus-use split from the opener: a program that `mmap`s 512 MiB and checks its resident memory before and after touching it:

```
after mmap 512 MiB: VmRSS = 1364 kB       # the mapping exists, but almost nothing is resident
after touching all: VmRSS = 525976 kB     # now every page has been faulted in (~512 MiB)
```

The 512 MiB mapping cost essentially *no* physical memory until it was used: RSS was about 1.3 MiB (just the program itself), not 512 MiB, right after the `mmap` succeeded. Touching every page faulted each one in, and only then did RSS climb to ~512 MiB. That is demand paging in two numbers: **allocation is a promise; residency is paid page by page, on first touch**.

THE SAME IDEA THREE WAYS: DEMAND PAGING PAYS PER TOUCHED PAGE

1. In words. Reserving address space (via `mmap`/`malloc`) only records mappings in the page table with the present bit clear; no frame is spent. The first access to each page faults, the kernel gives it a frame and sets present, and the instruction restarts. So physical memory is consumed in proportion to pages *touched*, not pages *reserved*.

2. As a picture. Figure 17.1: a virtual page with present=0 → access → page fault trap → handler grabs a free frame, zeroes/loads it, sets present=1 → instruction restarts and succeeds. Later faults on other pages repeat the path.

3. As numbers. `mmap` 512 MiB: RSS 1,364 kB before touching, 525,976 kB after touching all. Writing one byte per page across 100 MiB took 25,672 minor faults (≈ one per 4 KiB page) and *zero* major faults — anonymous pages are served from zeroed frames, no disk I/O.

*(Answer to the §17.1 quick check: the present bit means the page currently occupies a physical frame; the **OS** sets it (the hardware only reads it during translation). Accessing a not-present page raises a page fault — a trap into the kernel's fault handler, which either materializes the page or delivers `SIGSEGV`.)*

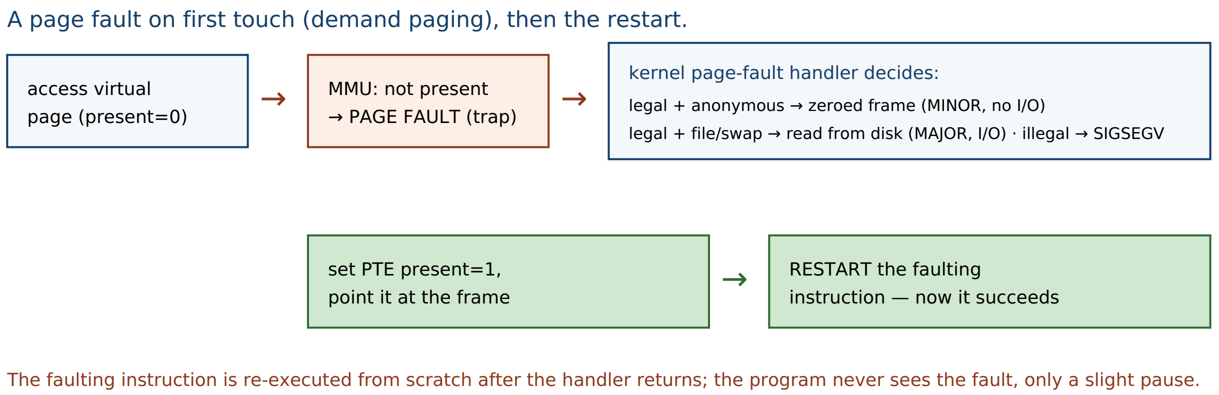


Figure 17.1: The demand-paging fault path. A first touch of a not-present page traps into the kernel, which materializes the page (a minor fault if no disk I/O is needed, a major fault if it must read from a file or swap), sets the present bit, and restarts the instruction. Illegal accesses become `SIGSEGV` instead.

17.3 Beyond physical memory: swapping and the replacement question

Demand paging is easy while free frames last. The hard case is when physical memory fills and a new page must be brought in with no frame to spare. The OS's answer is

swapping (paging to a backing store): it *evicts* some resident page — writing it out to a swap area on disk if it has been modified since it was loaded (a "dirty" page; a clean page backed by a file can just be dropped and re-read later) — freeing that frame for the incoming page. Later, if the evicted page is touched again, it faults back in from disk. This is where the minor/major distinction bites: a fault that can be satisfied from a free or droppable frame is a **minor** fault (fast, no I/O); a fault that must *read the page back from disk or swap* is a **major** fault, and it costs a disk access — Chapter 3's numbers make the gap vivid, since a disk (even an SSD) is orders of magnitude slower than DRAM, and a spinning disk millions of times slower than a cache hit.

Swapping keeps programs running past the size of RAM, but it turns the choice of *which page to evict* into a performance decision of the first order: every eviction of a page that is about to be needed again forces an expensive major fault to bring it back. This is the **page-replacement problem**, and it is Chapter 16's dilemma in a new guise — a policy question about a scarce, multiplexed resource, where a wrong choice is not incorrect, merely slow. The metric here is the **page-fault rate** (equivalently, the miss rate against the frames you have), and the goal is a replacement policy that minimizes it.

QUICK CHECK

What is the difference between a *minor* and a *major* page fault, and which one involves disk I/O? When the OS evicts a page to make room, why does it matter whether that page is "dirty" (modified since it was loaded)? (Answer below the communicate box.)

17.4 Replacement policies: OPT, FIFO, LRU, and Belady's anomaly

What should the OS evict? The unbeatable baseline is **OPT** (Belady's MIN, 1966): evict the page whose *next* use is farthest in the future. It provably minimizes faults — but it needs to see the future, so it is only a yardstick, not an implementable policy. The simplest real policy is **FIFO**: evict the page that has been resident longest, regardless of use. It is trivial but ignores *how* pages are used. The workhorse approximation is **LRU** (least-recently-used): evict the page that has gone longest without being accessed, betting that the recent past predicts the near future — which, thanks to **locality of reference** (Chapter 3), it usually does. On a short reference string, simulated exactly:

```
reference string: 0 1 2 0 1 3 0 3 1 2 1   (3 frames)
FIFO: 7 page faults
LRU : 5 page faults
OPT : 5 page faults        # the lower bound; no policy can do better on this string
```

FIFO takes 7 faults here where LRU and OPT take 5, because FIFO evicts pages by age even when they are about to be reused. But FIFO's real hazard is subtler and worth naming, because it violates an intuition everyone has: that giving a program *more* memory can only help. For FIFO, it can hurt. On the classic reference string of Belady, Nelson & Shedler (1969), FIFO takes *more* faults with four frames than with three:

```
reference string: 1 2 3 4 1 2 5 1 2 3 4 5
FIFO, 3 frames: 9 page faults
FIFO, 4 frames: 10 page faults      # MORE memory, MORE faults
```

This is **Belady's anomaly**: for FIFO, adding a frame can increase the fault count, because a larger FIFO queue changes the eviction order in a way that can evict a soon-to-be-reused page it would have kept with fewer frames. LRU and OPT are immune — they belong to a class (stack algorithms) whose resident set with N frames is always a superset of its resident set with N−1, so more memory never adds faults. The anomaly is the sharpest argument against FIFO and for use-based policies like LRU: a policy that ignores usage can be not just suboptimal but *non-monotonic*, punishing you for adding memory. (Real kernels do not implement true LRU — tracking exact recency on every access is too expensive — but approximate it cheaply with a per-page reference bit and a clock/second- chance sweep, keeping LRU's monotonic, locality-exploiting behavior without its bookkeeping cost.)

TRAP: THRASHING — WHEN THE WORKING SET DOESN'T FIT

Replacement policy only helps while the pages a program actively uses — its **working set** — fit in the frames it has. When the sum of all running processes' working sets exceeds physical memory, *no* replacement policy can win: every page the OS evicts is one that is about to be needed, so the system generates major fault after major fault, and the CPU sits idle waiting on disk while throughput collapses. This is **thrashing**, and its signature is a machine that is nearly idle on CPU yet grinding — high major-fault and swap-I/O rates, everything slow. The trap is diagnosing it as a CPU or code problem and "optimizing the hot loop," when the real fix is to reduce the memory footprint (smaller working set, fewer concurrent jobs) or add RAM so the working set fits. Modern Linux often does not let you thrash gently to a halt: when memory (plus swap) is truly exhausted, the **OOM killer** picks a process and kills it outright. Either way the lesson is the same — paging is a graceful extension of memory only up to the working-set boundary; past it, performance does not degrade linearly, it falls off a cliff. Watch major-fault and swap rates, not just CPU.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our service `malloc`ed a 40 GB buffer on a 16 GB box and the allocation *succeeded* — so we clearly have enough memory, and we can fill it up. Great." Cover the paragraph and respond in three or four sentences, drawing on §17.2 and the thrashing trap.

Model answer. "A successful `malloc` doesn't mean the memory exists — it means the kernel recorded the mapping; under demand paging (and overcommit) no physical frames are spent until you *touch* the pages, so a 40 GB allocation on a 16 GB box can succeed while backing almost nothing. The moment we actually write across all 40 GB, each page faults in, resident memory climbs past 16 GB, and the OS has to swap — if our working set genuinely exceeds RAM we'll thrash (near-idle CPU, endless major faults, everything crawling) or get OOM-killed outright. So allocation success proves nothing about capacity; what matters is the *resident working set*. Let's size the buffer to what fits with headroom, stream instead of buffering everything, and watch RSS and major-fault rates under load — not just whether `malloc` returned non-NULL." Separating "reserved" from "resident," and naming thrashing and the OOM killer, is the senior register.

*(Answer to the §17.3 quick check: a **minor** fault is satisfied without disk I/O — the page is served from a zeroed or already-cached frame; a **major** fault must read the page from a file or swap, so it costs a disk access. A "dirty" (modified) page must be **written out** to swap before its frame can be reused, whereas a clean page — an unmodified copy of file data — can be dropped and re-read later, so evicting clean pages is cheaper.)*

17.5 What to carry forward

The OS owns physical **frames** and each process's **page table**, and the **present bit** lets it back a virtual page with a frame lazily. **Demand paging** materializes a page only on first touch: the access to a not-present page raises a **page fault** (a trap), and the handler either supplies the page — a **minor** fault if no I/O is needed, a **major** fault if it must read from disk or swap — or delivers `SIGSEGV`. Allocation is therefore a promise (we watched a 512 MiB mapping cost ~1 MiB of RSS until touched), and residency is paid page by page (25,672 minor faults to touch 100 MiB). When RAM fills, **swapping** evicts pages to disk, making the choice of victim — the **page-replacement problem** — a performance decision: OPT is the unbeatable but unimplementable yardstick, FIFO is simple but suffers **Belady's anomaly** (more memory, more faults), and LRU (approximated in real kernels) exploits locality and is monotonic. Past the working-set boundary, no policy saves you: **thrashing**, then the OOM killer.

With this the operating system's core illusion is complete: Chapter 16 sliced the CPU in time, this chapter backs each address space with a smaller physical memory, and together they let many processes share one machine as if each owned it. The mechanisms this chapter unlocked are the ones the rest of the book leans on. The same page table that holds the present bit also holds protection and sharing bits, which give us **memory-mapped files** (`mmap` a file and read/write it as memory, the page cache faulting data in on demand) and **copy-on-write** — the trick from Chapter 14's `fork`, now seen fully: parent and child share every frame read-only, and only a write faults and copies the one page touched. Copy-on-write and shared mappings are also how separate processes can deliberately share a region of memory — the fast path for the inter-process communication that a later chapter takes up, and the reason the isolation of processes need not mean the cost of copying. Virtual memory is not only protection; it is the substrate for sharing, laziness, and speed.

CAN YOU… WITHOUT LOOKING?

- Explain, using the present bit, how the OS can promise a process a page it has not yet backed with a frame.
- Trace the page-fault path for a first touch of anonymous memory, and say why it is a minor fault, not a major one.
- State what demand paging makes cheap, and give the two measured numbers (RSS before/after touch; minor faults per page).
- Distinguish minor and major faults, and explain why evicting a dirty page costs more than evicting a clean one.
- Rank OPT, FIFO, and LRU, and explain Belady's anomaly and why LRU/OPT are immune to it.
- Define thrashing and the working set, and explain why a successful `malloc` does not prove you have the memory.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does the present bit mean, and what happens on an access to a not-present page? (§17.1, §17.2)
2. **R2.** What is demand paging, and what did the `mmap`-512-MiB measurement show about reserved vs resident memory? (§17.2)
3. **R3.** Distinguish minor and major page faults; which involves disk I/O? (§17.2, §17.3)
4. **R4.** What is the page-replacement problem, and how do OPT, FIFO, and LRU differ? (§17.4)
5. **R5.** What is Belady's anomaly, and what is thrashing? (§17.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Paging: Introduction," "Beyond Physical Memory: Mechanisms," and "Beyond Physical Memory: Policies."** The free chapters on the page as the unit, the page fault and swapping mechanism, and the replacement policies (OPT/FIFO/LRU, the anomaly, clock). The reference for §17.1–17.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 9 (Virtual Memory).** Address spaces, page tables, demand paging, and memory mapping from the programmer's view; the companion to Chapter 4's hardware treatment.
- **Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal* 5(2):78–101 (1966);** and **Belady, Nelson & Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," *Communications of the ACM* 12(6):349–353 (1969).** The primary sources for the optimal (MIN) policy and for Belady's anomaly, respectively.
- **The Linux `mmap(2)`, `madvise(2)`, and `proc(5)` manual pages.** Primary sources for memory mapping, demand-paging hints, and the `/proc/<pid>/status` fields (`VmRSS`) used to measure residency. Verify per kernel version.

Exercises

17.1 WARM-UP Define a virtual page and a physical frame, and say what a page-table entry's present bit records.

17.2 WARM-UP What is a page fault, and what two broad outcomes can the kernel's fault handler produce for it?

17.3 WARM-UP In one line each, distinguish a minor page fault from a major page fault.

17.4 CORE A program `mmap`s 1 GiB and its RSS is a few megabytes immediately afterward, then climbs toward 1 GiB as it works. Explain what happened at each stage in terms of demand paging, and why the allocation did not consume 1 GiB of physical memory up front.

17.5 CORE Simulate FIFO and LRU on the reference string 1 2 3 1 4 2 5 with 3 frames. Give the number of page faults for each and show the resident set after each reference. Which policy does better here, and why?

17.6 CORE Explain Belady's anomaly: state what it is, which policy exhibits it, and why LRU and OPT cannot. Use the phrase "resident set is a superset with more frames" in your explanation of the immune policies.

17.7 COST (a) Why is a major page fault dramatically more expensive than a minor one — put rough numbers on it using Chapter 3's memory hierarchy. (b) Why does evicting a *clean* page cost less than evicting a *dirty* one? (c) A service's p99 latency has sudden 50 ms spikes that correlate with high major-fault counts; what is the likely cause, and is the fix in the CPU-bound code?

17.8 FADED *Worked, with the last step for you.* A batch job processes a 30 GB dataset on a 32 GB machine by `mmap`ing the whole file and iterating over it once, sequentially. It runs fine. A colleague "optimizes" it to make ten concurrent passes over the same mapping from ten threads to "use all the cores," and now the machine crawls with near-zero CPU utilization.
Step 1 (given): The single sequential pass has a small working set at any instant (a sliding window of recently touched pages), so it fits comfortably in RAM and pages stream in and out cleanly.
Step 2 (given): Ten concurrent passes at different file offsets touch ten far-apart regions at once, so the combined working set is large and scattered.
Step 3 (given): That combined working set exceeds physical memory, so every page the OS evicts is soon needed by another thread, generating major fault after major fault.
Step 4 (you): Name the phenomenon precisely, explain why CPU utilization is near *zero* during it (what are the cores waiting on?), and give the direction of the fix (not "optimize the loop").

17.9 MIXED For each observation, name the most likely cause and one thing to measure to confirm it: (a) allocating a huge array succeeds instantly but the program slows down later as it fills the array; (b) a program that fit in RAM last week now runs 100× slower after the dataset grew slightly; (c) reading a memory-mapped file is fast the second time through but slow the first.

17.10 MIXED Drawing on Chapters 4, 14, and 17, classify each statement as *true* or *false* and fix the false ones: (a) "After a successful 8 GB `malloc`, 8 GB of physical RAM is now in use." (b) "A segmentation fault is just a page fault the handler could not satisfy legally." (c) "`fork` copies the parent's entire address space immediately." (d) "Two processes with the same virtual address are reading the same physical frame."

17.11 STRETCH Copy-on-write ties this chapter to Chapter 14. (a) Explain how the page table lets `fork` give the child a "copy" of the address space while physically copying no data at first. (b) Describe exactly what happens, page by page, when the child later writes to one page — which fault fires, what the handler does, and how many pages get copied. (c) Explain why this makes the common `fork`-then-`exec` pattern (Chapter 14) nearly free even for a large parent, and what a single write to a shared page costs.

17.12 LAB On your own machine: (a) write a program that `malloc`s (or `mmap`s) a few hundred MiB and touches one byte per 4 KiB page; run it under `/usr/bin/time -v` and report the minor-fault count — confirm it is about one per page. (b) Read `VmRSS` from `/proc/self/status` before and after touching a large mapping, and report both values to show reservation vs residency. (c) Optional and careful: on a machine you can afford to stress (or a VM), allocate and touch more than physical RAM and observe swap I/O / major faults climbing — describe the thrashing behavior and stop before you wedge the box. Label your kernel version, page size, and RAM.

Chapter 18 — Signals: Asynchronous Control Flow

> **Before you start:** Chapter 14 (when a child exits, the kernel notifies the parent with `SIGCHLD` — that notification *is* a signal; and `wait` reaps the child), Chapter 13 (a blocking system call can be interrupted — this chapter is what interrupts it), Chapter 11 (a bad memory access became a segmentation fault — `SIGSEGV` is a signal too). · Every earlier chapter's control flow was *synchronous*: the program ran its own instructions in order. This chapter adds a second, *asynchronous* flow the kernel can force on a process at almost any instant.

> **BEFORE YOU READ ON**
>
> From memory, reaching back: (1) Chapter 14 — after a child process exits but before the parent `wait`s for it, what is the child called, and how did the parent learn the child had finished? (2) Chapter 13 — a process blocked in a `read` that never completes is off the CPU; name one thing that could wake it besides the data arriving. (3) *Predict:* a program installs a handler for `SIGUSR1`, then blocks that signal, sends it to itself *five* times in a tight loop, and finally unblocks it. How many times does the handler run — five, one, or zero? Write down your guess and why — we will measure it in §18.4.

So far a program's control flow has been its own: it executes its instructions, calls functions, makes system calls, and returns — a single thread of cause and effect (threads in Chapter 15 added more such flows, but each was still running *its own* code). A **signal** breaks that assumption. It is an **asynchronous notification** the kernel delivers to a process to tell it that an event happened — a software analog of a hardware interrupt — and when one arrives, the process's normal flow is suspended, a designated **handler** runs, and then the normal flow resumes where it left off. You have already met signals without naming them: the segmentation fault (`SIGSEGV`) of Chapter 11, the `Ctrl-C` that kills a program (`SIGINT`), the `SIGCHLD` that told a parent its child had exited (Chapter 14). This chapter makes the mechanism precise: what a signal is and its default dispositions (§18.1), how to catch one with a handler (§18.2), how signals interrupt blocking system calls — the `EINTR` you must handle (§18.3) — and the sharp hazards of running code asynchronously, namely async-signal-safety and the fact that standard signals do not queue (§18.4). We close by placing signals among the OS's communication mechanisms (§18.5). Signals are the kernel's way of interrupting your program to tell it something; the danger is that "at almost any instant" means the handler can run in the middle of anything.

18.1 What a signal is, and its default disposition

A **signal** is a small integer, identified by name, delivered to a process to indicate an event. Some are sent by the kernel in response to hardware or program conditions — `SIGSEGV` (invalid memory access), `SIGFPE` (arithmetic error like divide-by-zero), `SIGCHLD` (a child changed state). Some are sent by users or programs to control a process — `SIGINT` (from `Ctrl-C`), `SIGTERM` (the polite "please exit," what `kill` sends by default), `SIGKILL` (the un-catchable "die now"), `SIGUSR1`/`SIGUSR2` (application-defined). A process can send a signal to another (subject to permissions) with the `kill` system call — poorly named, since it sends *any* signal, not only lethal ones.

Every signal has a **default disposition** — what happens if the process does nothing about it. The common defaults are: **Terminate** the process (`SIGTERM`, `SIGINT`); **Terminate and dump core** (`SIGSEGV`, `SIGABRT` — the core file for a debugger); **Ignore** the signal (`SIGCHLD`'s default is to be ignored — which is why an un-reaped child still becomes a zombie, per Chapter 14; the notification is delivered but discarded); and **Stop / Continue** the process (`SIGSTOP`/`SIGCONT`, job control). A process can *change* the disposition of most signals — catch them with a handler, or ask to ignore them — with two crucial exceptions: **`SIGKILL` and `SIGSTOP` can neither be caught nor ignored.** That is by design: the OS must retain an unblockable way to kill or halt any process, so that a program cannot make itself immortal by trapping every signal.

> **QUICK CHECK**
>
> Name the four broad default dispositions a signal can have. Which two signals can a process *not* catch or ignore, and why does the OS insist on that? (Answer after the next section.)

18.2 Catching a signal: handlers and `sigaction`

To catch a signal, a process installs a **signal handler**: a function the kernel will call when that signal is delivered. The portable, well-defined way to install one is the `sigaction` system call (the older `signal()` function has historically ambiguous semantics across systems; prefer `sigaction`). You fill in a `struct sigaction` with your handler function and flags, and register it for a given signal number:

```
void handler(int signum){ /* runs asynchronously when the signal arrives */ }

struct sigaction sa;
memset(&sa, 0, sizeof sa);
sa.sa_handler = handler;
sigaction(SIGINT, &sa, NULL);   /* from now on, Ctrl-C runs handler() instead of terminating
*/
```

Once installed, the handler runs *asynchronously*: when `SIGINT` arrives, the kernel suspends whatever the process was doing, calls `handler`, and — when the handler returns — resumes the interrupted code exactly where it left off. The program did not *call* the handler; the kernel injected it into the control flow. This is the defining strangeness of signals and the source of every hazard in §18.4: the handler can begin executing between any two instructions of your main code, including in the middle of a library call. The handler's signature is fixed (`void handler(int)`, receiving the signal number so one handler can serve several signals), and while it runs, the same signal is (by default) blocked so the handler does not interrupt itself.

QUICK CHECK

Which system call installs a signal handler the well-defined way, and why is the older `signal()` discouraged? In what sense does the program *not* call its own handler? (Answer below the communicate box.)

18.3 Signals interrupt system calls: `EINTR`

Here is the consequence that bites every real program. Suppose a process is blocked in a slow system call — `read` waiting for input that has not arrived, `wait` for a child, `sleep` — and a signal arrives that the process catches. The kernel cannot both run the handler and leave the process cleanly blocked, so it interrupts the system call: it runs the handler, and then the blocked call returns early with the error **`EINTR`** ("interrupted system call"). The call did not fail for any reason of its own — the data may still be coming — it was simply cut short to deliver the signal. Here is a program that blocks in `read` on an empty pipe, arranges `SIGALRM` to fire after one second (handler installed *without* the restart flag), and reports what `read` returns (gcc 11.4.0, Linux 6.18):

```
alarm(1);                                   /* deliver SIGALRM in 1 second */
ssize_t n = read(fd[0], buf, sizeof buf);  /* blocks; nobody writes to the pipe */
/* ... one second later the signal fires, the handler runs, and read returns: */

read returned -1, errno=4 (Interrupted system call)
```

The `read` returned `-1` with `errno == EINTR`, not because the pipe failed but because the signal interrupted it. A correct program must expect this: any slow system call that can be interrupted by a caught signal may return `EINTR`, and the usual fix is to **retry the call** (the classic `while ((n = read(...)) < 0 && errno == EINTR);` loop). Alternatively you can install the handler with the **`SA_RESTART`** flag, which asks the kernel to *automatically restart* many interrupted system calls instead of failing them with `EINTR` — but the coverage is incomplete (some calls still return `EINTR` even with `SA_RESTART`), so robust code handles `EINTR` regardless. The naive assumption that a system call either blocks until it succeeds or fails for a real reason is simply wrong once signals are in play.

THE SAME IDEA THREE WAYS: A CAUGHT SIGNAL CUTS A BLOCKING CALL SHORT

1. In words. A process blocked in a slow system call cannot stay cleanly blocked while a handler runs, so the kernel interrupts the call: it runs the handler, then the call returns `-1` with `errno == EINTR`. The operation did not truly fail; it was cut short to deliver the signal, so the program must retry it (or use `SA_RESTART`, with caveats).

2. As a picture. Figure 18.1: main thread blocked in `read` → signal arrives → handler runs → control returns to `read`, which unwinds and returns `EINTR` rather than resuming the block.

3. As numbers. A `read` on an empty pipe with a `SIGALRM` after one second (no `SA_RESTART`): `read returned -1, errno=4 (Interrupted system call)`. With a correct retry loop it would resume waiting; with `SA_RESTART` the kernel would restart it for you.

(Answer to the §18.1 quick check: the four default dispositions are Terminate, Terminate-and-dump-core, Ignore, and Stop/Continue. `SIGKILL` and `SIGSTOP` cannot be caught or ignored, so the OS always retains a way to kill or halt any process — a program cannot trap its way to immortality.)

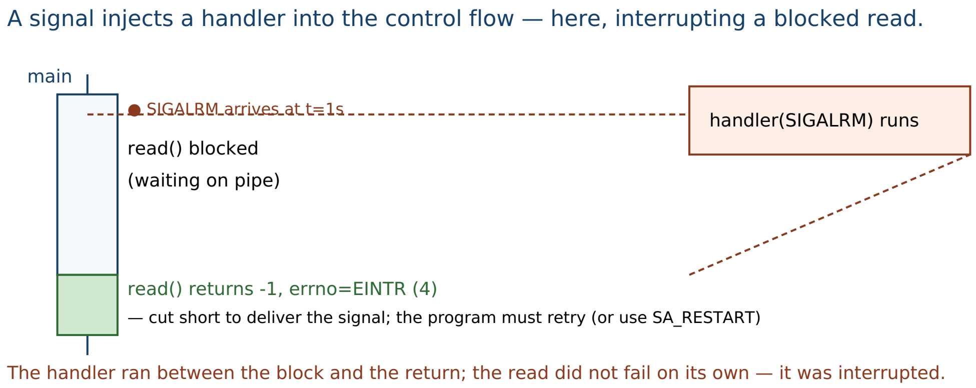


Figure 18.1: A caught signal interrupts a blocking system call. The kernel suspends the block, runs the handler, then the interrupted call returns `-1` with `errno == EINTR` instead of continuing to wait — which the program must handle (retry) or opt out of with `SA_RESTART`.

18.4 The hazards: async-signal-safety and non-queuing

Because a handler can run between any two instructions of your program — including inside a library function that is halfway through updating its own state — the code a handler may safely execute is sharply restricted. A function is **async-signal-safe** if it is safe to call from a signal handler, which essentially means it is reentrant: it does not rely on global state that a signal could catch mid-update. Most of the standard library is *not* async-signal-safe. The classic trap is calling `printf` (or `malloc`) in a handler: if the signal arrives while the main program is already inside `printf` or `malloc` — holding an internal lock or partway through a data structure — the handler's second call to the same function re-enters it and can deadlock (waiting on a lock the interrupted call holds) or corrupt the heap. The kernel documents the exact set of functions guaranteed safe (the `signal-safety(7)` man page); it is a short list — `write`, `_exit`, and a handful of others — and `printf`/`malloc`/most of libc are not on it. The safe idiom is to do almost nothing in a handler: set a flag of type `volatile sig_atomic_t` and return, then let the main loop notice the flag and do the real work synchronously.

The second hazard is delivery semantics, and it settles the opener's prediction. Standard signals (numbers 1–31) are **not queued**: a signal is either pending or not, a single bit, so if the same signal is generated several times while it is blocked, the process still sees it *once* when unblocked — the extra instances coalesce and are lost. Here is the opener's exact program — install a `SIGUSR1` handler that counts calls, block `SIGUSR1`, send it to ourselves five times, then unblock:

```
sigprocmask(SIG_BLOCK, ...);       /* block SIGUSR1 */
for (int i=0;i<5;i++) raise(SIGUSR1);   /* send it 5 times while blocked */
/* handler count so far = 0  (blocked, so not delivered yet) */
sigprocmask(SIG_UNBLOCK, ...);     /* unblock: the pending signal is delivered */

sent 5 while blocked; handler count so far = 0
after unblock, handler ran 1 time(s)
```

The handler ran **once**, not five times: the five sends collapsed into a single pending bit. This is why you must never use a standard signal as a counter or a reliable event stream — if two children exit at nearly the same moment, the parent may get a single `SIGCHLD`, so a correct `SIGCHLD` handler reaps *all* available children in a loop (`while (waitpid(-1, ..., WNOHANG) > 0)`), not one per signal. (POSIX real-time signals, `SIGRTMIN` through `SIGRTMAX`, *do* queue and can carry a small payload, at the cost of more machinery — the exception that proves the rule.) The robust modern pattern for turning asynchronous signals into something a normal event loop can consume is the **self-pipe trick** (the handler does one async-signal-safe `write` to a pipe the main loop reads) or Linux's `signalfd`, both of which move the real work out of the handler and back into the synchronous main flow.

TRAP: DOING REAL WORK IN A SIGNAL HANDLER

The natural instinct on catching a signal — log a message, update a data structure, free some resources, flush state — is exactly what you must not do, because a handler runs *asynchronously, in the middle of arbitrary code*. Calling anything not on the async-signal-safe list (which is nearly all of libc, including `printf`, `malloc`/`free`, and most of the standard library) risks a deadlock or heap corruption if the signal interrupted that very function. The bug is vicious because it is *rare*: it only manifests when the signal happens to land inside the unsafe call, so the program runs fine in testing and hangs or crashes in production under load — the same "passes the test, broken anyway" pathology as the data race of Chapter 15, and for the same reason (a schedule-dependent window). The discipline: a handler should do the minimum async-signal-safe thing — set `volatile sig_atomic_t got_signal = 1;` (or `write` one byte down a self-pipe) and return — and the main loop does the real work when it is safe. If you find yourself wanting to `printf` from a handler to debug it, that itch is the bug.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "For graceful shutdown I added a `SIGTERM` handler that logs 'shutting down,' flushes the in-memory cache to disk, closes the DB connections, and calls `exit` — all right there in the handler. Clean, right?" Cover the paragraph and respond in three or four sentences, drawing on §18.4.

Model answer. "The intent is right but the handler is doing far too much: it runs asynchronously, possibly in the middle of a `malloc` or an `fprintf`, and logging, flushing, and closing all call functions that are not async-signal-safe — so if `SIGTERM` lands at the wrong instant, the handler deadlocks on a lock the interrupted code holds or corrupts the heap, and your 'graceful' shutdown hangs the process precisely when you're trying to stop it cleanly. The safe pattern is to make the handler trivial — set a `volatile sig_atomic_t shutdown_requested = 1;` (or write a byte to a self-pipe) and return — and let the main loop, on seeing the flag, do the flush, close, and exit synchronously where all of libc is safe to call. That also lets you bound and order the shutdown steps instead of racing them inside an interrupt. Keep handlers to a flag; do the work in the main flow." Naming async-signal-safety and moving the work out of the handler is the senior register.

*(Answer to the §18.2 quick check: `sigaction` is the well-defined way to install a handler; the older `signal()` is discouraged because its semantics — handler persistence, whether the signal is blocked during its own handler, and system-call restart behavior — historically vary across systems. The program does not call its handler: the kernel **injects** it into the control flow at delivery time, suspending whatever was running and resuming it when the handler returns.)*

18.5 What to carry forward

A **signal** is an asynchronous notification the kernel delivers to a process — a software interrupt — and each has a **default disposition** (terminate, terminate-and-core, ignore, stop/continue), with `SIGKILL` and `SIGSTOP` the two that can never be caught or ignored. A process catches a signal by installing a handler with `sigaction`; the handler runs asynchronously, injected into the control flow, and returns to where the program was interrupted. Two consequences dominate correct use. First, a caught signal **interrupts blocking system calls**, which return `-1` with `errno == EINTR` (we measured a `read` cut short to `errno 4`) — you must retry, or use `SA_RESTART` with care. Second, a handler runs in the middle of arbitrary code, so it may call only **async-signal-safe** functions (not `printf`/`malloc`/most of libc), and standard signals **do not queue** — five rapid `SIGUSR1`s coalesced into a single delivery in our measurement — so the safe pattern is a handler that sets a `volatile sig_atomic_t` flag (or writes a self-pipe) and defers the real work to the synchronous main loop.

Signals complete the picture of how the operating system talks *to* a process: the system-call boundary (Chapter 13) is how a process asks the kernel for things synchronously; signals are how the kernel (or another process) interrupts a process asynchronously to tell it something happened. But a signal carries almost no information — just its number (real-time signals add a small payload) — so it is a doorbell, not a conversation. When processes need to exchange real *data* rather than notifications, they need genuine inter-process communication: pipes and FIFOs for byte streams, shared memory (built, as Chapter 17 showed, on shared mappings in the page tables) for the fastest sharing, and sockets for talking across machines. That IPC toolkit — the channels that carry data between the isolated address spaces this part has spent five chapters building and protecting — is where the operating system part of this volume finishes, and it is the bridge to the low-level I/O and networking that the data systems in later volumes are made of.

CAN YOU… WITHOUT LOOKING?

- Define a signal as asynchronous control flow, and name its four default dispositions.
- Say which two signals can never be caught or ignored, and why the OS insists on that.
- Install a handler with `sigaction` from memory, and explain in what sense the kernel "injects" the handler.
- Explain `EINTR`: why a caught signal makes a blocking call return it, and the two ways to cope.
- Explain async-signal-safety and why `printf` in a handler is a bug, and give the safe handler pattern.
- Explain why standard signals do not queue, and what that means for a `SIGCHLD` handler reaping children.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a signal, and what are the four default dispositions? (§18.1)
2. **R2.** Which two signals cannot be caught or ignored, and why? (§18.1)
3. **R3.** What does `sigaction` do, and in what sense does a handler run asynchronously? (§18.2)
4. **R4.** What is `EINTR`, when does it occur, and how do you handle it? (§18.3)
5. **R5.** What does async-signal-safe mean, and why do standard signals not queue? (§18.4)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8 (Exceptional Control Flow), §8.5 (Signals).** Signals as exceptional control flow, handlers, blocking, and the concurrency hazards of handlers — the reference for §18.1–18.4.
- **Stevens & Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. (2013), Chapter 10 (Signals).** The canonical, thorough treatment of signal semantics, `sigaction`, reliable signals, and interrupted system calls. `[⚠ reference — not free]`
- **The Linux `signal(7)`, `sigaction(2)`, and `signal-safety(7)` manual pages.** Primary sources for the signal list and default dispositions, the `SA_RESTART`/EINTR behavior, and the exact set of async-signal-safe functions. Verify per system.
- **The Linux `signalfd(2)` manual page and the "self-pipe trick" (D. J. Bernstein).** The documented ways to turn asynchronous signal delivery into a synchronous, event-loop-friendly readable stream — the robust alternative to doing work in a handler.

Exercises

18.1 WARM-UP In one sentence, what is a signal, and how does it differ from a normal function call in how it enters the program?

18.2 WARM-UP Name the four broad default dispositions a signal can have, with one example signal for each if you can.

18.3 WARM-UP Which two signals can a process neither catch nor ignore, and what would go wrong if it could?

18.4 CORE A program blocks in `read` waiting for input and catches `SIGINT` with a handler. Explain, step by step, what happens when the user presses `Ctrl-C`: what the `read` returns, what `errno` is, and why the call did not simply resume waiting. Give the two standard ways to make the program robust to this.

18.5 CORE Explain why calling `printf` (or `malloc`) inside a signal handler is unsafe. What property must a function have to be callable from a handler, and what is the safe pattern for a handler that needs to trigger real work?

18.6 CORE Explain why standard signals do not queue, and why this means a `SIGCHLD` handler must reap children in a loop (with `WNOHANG`) rather than reaping exactly one child per delivered signal. Tie this back to Chapter 14's zombies.

18.7 COST (a) Why is a handler that only sets a `volatile sig_atomic_t` flag both cheap and safe, compared with one that does the work directly? (b) What is the latency cost of the self-pipe / `signalfd` approach versus a direct handler, and what correctness benefit do you buy with it? (c) Why can a signal-based design never be a reliable *counter* of events under load?

18.8 FADED *Worked, with the last step for you.* A long-running server installs a `SIGHUP` handler to reload its config. The handler re-reads the config file, rebuilds several in-memory structures with `malloc`, and logs progress with `fprintf`. In testing it works; in production, under heavy request load, the server occasionally hangs completely when config is reloaded.
Step 1 (given): The handler runs asynchronously and can interrupt the main code at any instant, including while the main code is inside `malloc` or `fprintf`.
Step 2 (given): `malloc` and stdio hold internal locks / mutable state; they are not async-signal-safe.
Step 3 (given): Under heavy load the odds rise that `SIGHUP` lands while a request thread is inside one of those functions, so the handler's own call to it re-enters and deadlocks.
Step 4 (you): Name the bug precisely, explain why it appears only under load (not in tests), and describe the redesign that fixes it — what the handler should do and where the reload work should actually run.

18.9 MIXED For each need, say whether a *signal* is the right mechanism or whether you want real IPC (a pipe, shared memory, or a socket), and justify in a line: (a) tell a running daemon to re-read its configuration; (b) stream a megabyte of data from one process to another on the same host; (c) notify a parent that a child has exited; (d) send a structured request and get a structured reply between two processes.

18.10 MIXED Drawing on Chapters 13–18, classify each statement as *true* or *false* and fix the false ones: (a) "A signal handler is called by the program the way any other function is." (b) "If you install a handler with `SA_RESTART`, you never have to worry about `EINTR`." (c) "Sending a process `SIGKILL` lets it run cleanup code first." (d) "A segmentation fault is the delivery of the `SIGSEGV` signal."

18.11 STRETCH A signal handler and the main program both touch a shared `int running` flag. (a) Why must that flag be declared `volatile sig_atomic_t` rather than a plain `int` — what could the compiler or the hardware otherwise do? (b) Compare this hazard with the data race of Chapter 15: how is a handler-vs-main-flow conflict similar to a thread-vs-thread race, and how is it different (single thread, asynchronous interruption)? (c) Explain why this makes signals a form of concurrency even in a single-threaded program.

18.12 LAB On your own machine: (a) write a program that blocks in `read` on an empty pipe, sets an `alarm`, and prints what `read` returns and `errno` when `SIGALRM` fires — confirm you see `EINTR` (errno 4); then reinstall the handler with `SA_RESTART` and observe the difference. (b) Write a program that installs a counting handler for `SIGUSR1`, blocks it, `raise`s it five times, unblocks, and prints how many times the handler ran — confirm it is once, not five. (c) Explain both results in terms of interrupted system calls and non-queuing standard signals. Label your compiler and kernel version.

Chapter 19 — Inter-Process Communication: Pipes, Shared Memory, and Sockets

Before you start: Chapter 14 (`fork` gives each process its own *isolated* address space — this chapter is how those isolated processes talk), Chapter 17 (a shared mapping lets two page tables point at the same physical frame — that is the machinery under shared memory), Chapter 18 (a signal is a doorbell, not a conversation — it notifies but carries no data). · Chapter 14 spent real effort keeping processes apart; this chapter is the controlled set of channels that let them exchange data *across* that isolation, and it is where Part III finishes.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 17 — copy-on-write and shared mappings both rest on one fact about page tables; what lets two processes' page tables refer to the *same* physical frame? (2) Chapter 18 — why did we call a signal a "doorbell, not a conversation," and what does that tell you it is bad at? (3) *Predict:* two related processes `mmap` the same page with `MAP_SHARED`; one writes the value 42 into it and exits, the other reads. Does the reader see 42? Now suppose the mapping had instead been `MAP_PRIVATE` — same question. Write down both guesses; we measure them in §19.3.

A process is an island. Chapter 14 built that isolation deliberately: each process gets its own virtual address space, so a wild pointer in one cannot corrupt another, and Chapter 17 showed the page tables that enforce it. Isolation is a safety property you want — and an obstacle the moment two processes must cooperate. A shell pipeline (`sort | uniq`), a web server handing a request to a worker, a database client talking to a server: all need to move data between address spaces that, by construction, cannot see each other's memory. **Inter-process communication** (IPC) is the family of kernel-provided channels that bridge the gap. This chapter covers the three that matter most, along the axis of *what* they move and *how far*: **pipes and FIFOs**, the unidirectional byte stream that powers the shell (§19.2); **shared memory**, the fastest channel because it moves nothing — the processes map the same frames (§19.3); and **sockets**, the bidirectional channel that, unlike the others, also reaches *across machines* (§19.4). We start by placing them on a map (§19.1) and close by carrying the toolkit forward into concurrency and networking (§19.5). The through-line: every channel trades cost against reach against safety, and shared memory in particular gives you speed by handing back a hazard you have met before.

19.1 Why IPC exists, and the shape of the toolkit

Because address spaces are private, one process cannot simply pass a pointer to another: the same virtual address means different physical memory (or nothing) in the other process, so a raw pointer is meaningless across the boundary. Every IPC mechanism is therefore either a **channel the kernel copies bytes through** or an **arrangement to share the same physical memory**. That single distinction organizes the whole toolkit. A *copying* channel — a pipe, a FIFO, a socket — is safe and simple: the sender hands bytes to the kernel, the kernel buffers them, the receiver reads them out, and neither ever touches the other's memory. A *sharing* arrangement — shared memory — is fast because it skips the copy entirely: both processes map one region and read and write it directly, at memory speed, but they inherit the full burden of coordinating that concurrent access themselves.

Cutting the other way is **reach**. Pipes, FIFOs, shared memory, and Unix-domain sockets are *same-machine* mechanisms: they rely on a shared kernel, so they cannot connect two processes on different computers. Network sockets (TCP/UDP) are the exception — the same socket API that talks to a process next door talks to one across the world, at the price of copies, protocol overhead, and the possibility of failure in between. And cutting a third way is **message shape**: some channels preserve the boundaries between writes (a datagram socket delivers each message whole), while others are pure byte streams that erase them (a pipe or a TCP connection). Keep those three axes — copy vs. share, local vs. remote, stream vs. message — in mind; every mechanism below is a point in that space, and choosing well is mostly a matter of locating your need on the map.

> **QUICK CHECK**
>
> Why can't one process simply pass a pointer to another and have it work? Name the single distinction that splits the IPC toolkit in two, and say which side "moves no data." (Answer after the next section.)

19.2 Pipes and FIFOs: the byte stream

A **pipe** is the oldest and simplest IPC channel: a unidirectional, in-kernel byte buffer with two file descriptors, a read end and a write end. The `pipe(fd)` system call fills `fd[0]` (read) and `fd[1]` (write); bytes written to `fd[1]` come out of `fd[0]` in order. On its own a pipe connects nothing — the trick is `fork`: a parent creates the pipe, forks, and now parent and child share both descriptors (file descriptors survive `fork`, Chapter 14), so one can close the read end and one the write end, leaving a one-way channel between relatives. That is exactly how the shell builds `a | b`: it pipes `a`'s standard output to `b`'s standard input. A **FIFO** (named pipe) is the same buffer given a *filesystem name* with `mkfifo`, so that *unrelated* processes — with no common ancestor to inherit descriptors from — can rendezvous by opening the same path.

The defining property of a pipe is that it is a **byte stream with no message boundaries**. The kernel does not remember how many `write`s produced the bytes; it just concatenates them. Two writes of three bytes each are indistinguishable from one write of six. Here is that fact measured (gcc 11.4.0, Linux 6.18): write `"ABC"` then `"DEF"` to a pipe, then do a single `read`:

```
write(fd[1], "ABC", 3);
write(fd[1], "DEF", 3);
ssize_t n = read(fd[0], buf, sizeof buf);

two writes of 3 bytes; one read got 6 bytes: "ABCDEF"
```

One read returned all six bytes at once: the boundary between the writes is gone. This is not a bug — it is what a stream *is* — but it means that if you need messages (records, requests), you must impose them yourself with **framing**: length prefixes, delimiters, or fixed-size records. (Small writes are still atomic up to a limit: POSIX guarantees a write of at most `PIPE_BUF` bytes — 4096 on Linux — will not be interleaved with another writer's, which is why the boundary-erasing above is about *your own* back-to-back writes, not corruption.) A pipe is also **finite**: it holds a bounded amount before a writer must wait. Filling a non-blocking pipe one byte at a time until it refuses shows the capacity:

```
while (write(fd[1], &c, 1) == 1) total++;

pipe full after 65536 bytes (errno=11 EAGAIN)
```

The kernel's pipe buffer is **65536 bytes** (16 pages of 4096) by default on Linux — matching the documented capacity in `pipe(7)`, adjustable with `fcntl`'s `F_SETPIPE_SZ`. This bound gives pipes their flow control: a fast writer *blocks* when the buffer is full until the reader drains it, and a reader *blocks* on an empty pipe until data arrives — so a pipeline naturally paces its stages to the slowest one. Two more behaviors complete the model. When *all* write ends are closed, a `read` returns **0** — end-of-file — which is how the reader learns the writer is done. And writing to a pipe whose *read* end is all closed raises **`SIGPIPE`** (default: terminate), or, if you ignore that signal, fails the `write` with `errno == EPIPE` — the kernel's way of telling a writer that nobody is listening, so it should stop rather than pour bytes into a void.

> **QUICK CHECK**
>
> A pipe delivered six bytes from two three-byte writes as one read. What property of a pipe does that show, and what must you add on top if you need discrete messages? What does a `read` return when every write end is closed? (Answer below the communicate box.)

19.3 Shared memory: the fastest channel moves nothing

Pipes and sockets copy: bytes go from the sender's memory into a kernel buffer and out again into the receiver's memory — two copies and a system call per exchange. **Shared memory** is the mechanism that removes the copy by removing the boundary. Two processes arrange for a region of their address spaces to be backed by the *same physical frames* — precisely the shared-mapping machinery of Chapter 17, where two page tables point at one frame — and thereafter one process's store to that region *is* the other's load. Nothing traverses the kernel per access; communication runs at the speed of memory. The arrangement comes in a few forms: an *anonymous* `mmap` with `MAP_SHARED` inherited across `fork` (for related processes); POSIX shared memory (`shm_open` to get a named object, then `mmap` it) for unrelated processes; and the older System V `shmget`/`shmat` interface. All reach the same end: one region, many mappers.

The opener's prediction pins down what "shared" means and what it does not. Map one page `MAP_SHARED|MAP_ANONYMOUS` and another `MAP_PRIVATE|MAP_ANONYMOUS`, `fork`, let the child write 42 into both, and have the parent read them back:

```
int *shared  = mmap(NULL, 4096, ..., MAP_SHARED |MAP_ANONYMOUS, -1, 0);
int *private = mmap(NULL, 4096, ..., MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);
/* child: *shared = 42; *private = 42; exit */

after child wrote 42: parent sees shared=42, private=0
```

The parent sees `shared == 42` but `private == 0`. The `MAP_SHARED` page is one frame both processes map, so the child's write is visible to the parent; the `MAP_PRIVATE` page is copy-on-write, so the child's write faulted a private copy the parent never sees (the same copy-on-write from `fork` in Chapter 14). Shared memory is `MAP_SHARED`: genuine sharing, zero copies. But here is the catch, and it is the whole reason concurrency is the next part of this book. Shared memory gives you the sharing and *nothing else* — no ordering, no mutual exclusion. Two processes writing the same shared counter with no coordination is **exactly the data race of Chapter 15**, now across process boundaries rather than threads. Measured with two processes each incrementing a shared counter a million times, no synchronization (gcc 11.4.0 at `-O0`, so each `++` is a real load–add–store):

```
two processes each +1000000 on a shared counter, no lock: final=1305815 (expected 2000000)
```

Roughly 700,000 updates vanished — the same lost-update interleaving as Chapter 15's threads, for the same reason: `counter++` is not atomic, and two flows of control raced through its load–add–store. Shared memory hands you raw, unsynchronized concurrency; making it correct needs the locks, condition variables, and atomics of the next part. The map's lesson is sharp: shared memory is the fastest channel and the most dangerous, because it is the only one that does not serialize access through the kernel for you.

THE SAME IDEA THREE WAYS: SHARED MEMORY SHARES FRAMES, NOT SYNCHRONIZATION

1. In words. A `MAP_SHARED` region is backed by physical frames that appear in both processes' page tables, so a write by one is directly a read by the other — no copy, no kernel round-trip. What the kernel does *not* supply is any ordering or exclusion between those accesses; that is the caller's job.

2. As a picture. Figure 19.1: two page tables, both pointing at frame F (shared) — one write, both see it — versus two page tables pointing at private copies (copy-on-write), where a write splits them apart.

3. As numbers. `MAP_SHARED`: parent sees the child's 42. `MAP_PRIVATE`: parent still sees 0. And two processes racing a shared counter a million times each land at `1305815`, not `2000000` — sharing without synchronization is the Chapter 15 data race again.

(Answer to the §19.1 quick check: a pointer is meaningless across the boundary because the same virtual address maps to different physical memory in each process's private address space. The toolkit splits into channels the kernel ***copies*** *bytes through and arrangements that* ***share*** *the same physical memory; the sharing side — shared memory — moves no data.)*

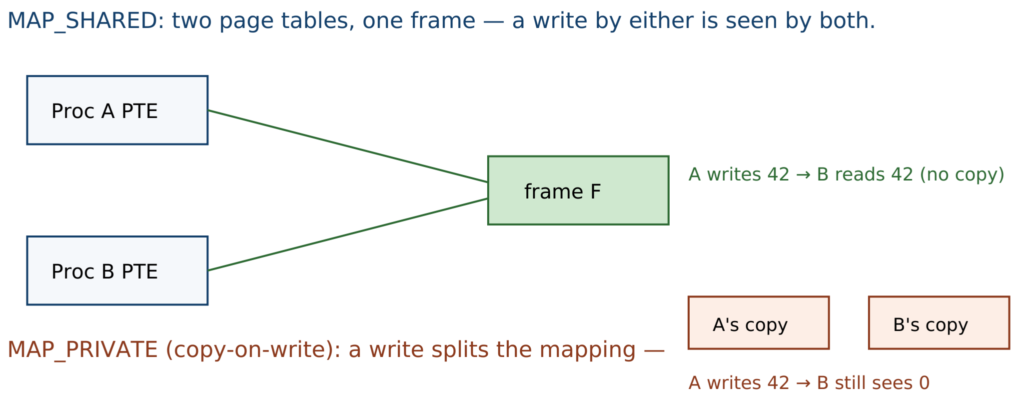


Figure 19.1: Shared memory (`MAP_SHARED`) maps one physical frame into both processes, so a store by one is a load by the other with no copy. A private (copy-on-write) mapping instead gives each a separate copy on the first write, so changes do not cross. Sharing frames is not sharing synchronization — coordinating the concurrent access is still the program's job.

19.4 Sockets: bidirectional channels that cross machines

Pipes are unidirectional and local; the moment you need two-way traffic, or need to reach a process on another host, you reach for a **socket**. A socket is a bidirectional endpoint, created with `socket()`, that comes in two dominant flavors distinguished by *message shape*. A **stream socket** (`SOCK_STREAM`, TCP over a network) is a reliable, ordered *byte stream* — like a pipe, it has **no message boundaries**, so the same framing burden applies. A **datagram socket** (`SOCK_DGRAM`, UDP) preserves *message boundaries* — each `send` is one `recv` — but drops the reliability and ordering guarantees, so datagrams can be lost, duplicated, or reordered. The same API spans reach: with the `AF_UNIX` address family a socket is a fast **local** channel (a Unix-domain socket, which can even pass open file descriptors between processes); with `AF_INET`/`AF_INET6` the identical call sequence talks **across machines**. That uniformity is the socket's great virtue: one programming model from "next door" to "across the world."

What you buy with that reach is paid in copies and uncertainty. Every socket exchange, local or remote, passes bytes through kernel buffers — so, like a pipe, a socket *copies* rather than shares, and cannot match shared memory's zero-copy speed for two processes on one machine. And once the channel spans the network, the far end can vanish mid-conversation: a socket write can fail, a read can hang, a connection can reset. Those failure modes — and the epoll/io_uring machinery for handling thousands of them efficiently — are the subject of the low-level I/O and networking material later in this volume; here the point is placement. Use a pipe or FIFO for a local, one-way byte stream; shared memory when two local processes must exchange data at memory speed and you are prepared to synchronize; a Unix-domain socket for local bidirectional or descriptor-passing needs; and a network socket when the other process is on another machine and you accept the cost and the failure modes that come with distance.

TRAP: ASSUMING A STREAM PRESERVES YOUR MESSAGE BOUNDARIES

The single most common IPC bug is treating a **byte stream** — a pipe, a FIFO, or a TCP connection — as if it preserved the boundaries of your `write`s. You `send` a 200-byte request in one call and assume the peer's one `recv` returns exactly those 200 bytes; it does not have to. The stream may coalesce two sends into one read (as we measured for pipes: two 3-byte writes came back as one 6-byte read) or split one send across two reads. Over TCP this is guaranteed to bite eventually, because the network fragments and recombines the stream at will, so the code that "worked on localhost" corrupts messages under real traffic — the same "passes the test, broken anyway" pathology as a data race, and just as schedule- and load-dependent. The fix is **framing**: agree on a length prefix or a delimiter and loop `read`ing until you have a full message. If you need discrete messages for free, use a *datagram* mechanism (a `SOCK_DGRAM` socket, which preserves boundaries) — but then handle loss and reordering. Never let "one write, one read" be an unstated assumption in stream code.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our two services run on the same box and swap a lot of data, so I'll skip the socket and just put everything in a big `MAP_SHARED` region — it's basically free, right? Both sides read and write the struct directly." Cover the paragraph and respond in three or four sentences, drawing on §19.3.

Model answer. "Shared memory is the right instinct for speed — it's the only local channel with no per-message copy, because both processes map the same frames, so it can be far faster than a socket for high-volume same-host exchange. But it is *not* free: it gives you the sharing and nothing else — no ordering, no mutual exclusion — so the instant both sides write the same fields concurrently you have the Chapter 15 data race, across processes now, and we measured that losing hundreds of thousands of updates. To use it you must add your own synchronization in the shared region — a mutex or semaphore both processes can see, or lock-free structures with atomics — and think about what happens if one side crashes mid-update and leaves the data half-written. So: shared memory for the throughput, plus explicit coordination, plus a recovery story — not 'basically free.'" Naming that shared memory buys speed by handing back the synchronization problem is the senior register.

(Answer to the §19.2 quick check: the six-bytes-from-two-writes result shows a pipe is a ***byte stream with no message boundaries*** *— the kernel concatenates writes — so to send discrete messages you must add framing (length prefixes, delimiters, or fixed records). A* `read` *returns* ***0*** *(end-of-file) once every write end is closed.)*

19.5 What to carry forward

Processes are isolated by design, so **inter-process communication** is the kernel's set of channels across that isolation, and each channel is a point on three axes: *copy vs. share*, *local vs. remote*, and *stream vs. message*. A **pipe** is a unidirectional, in-kernel byte stream shared across `fork` (a **FIFO** is a pipe with a filesystem name for unrelated processes); it has no message boundaries — we watched two 3-byte writes return as one 6-byte read — is bounded (65536 bytes on Linux, giving flow control by blocking a full writer and an empty reader), signals end-of-file with a `read` of 0, and raises `SIGPIPE`/`EPIPE` when no reader remains. **Shared memory** (`MAP_SHARED`) is the fastest channel because it copies nothing — both processes map the same frames, the Chapter 17 machinery — but it supplies sharing without synchronization, so two processes racing a shared counter lost ~700,000 of two million updates, the Chapter 15 data race again. **Sockets** are bidirectional and, uniquely, cross machines: stream sockets (TCP) are boundary-less byte streams needing framing, datagram sockets (UDP) preserve message boundaries but drop reliability, and the same API spans a local Unix-domain socket to a worldwide connection, paying copies and failure modes for the reach.

With this, Part III is complete. The operating system took one machine and multiplexed it: the system-call boundary (Chapter 13) is how a process asks the kernel for service; `fork`/`exec`/`wait` (Chapter 14) create and isolate processes; threads (Chapter 15) add flows of control inside one; scheduling (Chapter 16) shares the CPU in time; virtual memory (Chapter 17) shares physical memory across address spaces; signals (Chapter 18) let the kernel interrupt a process; and IPC (this chapter) opens controlled channels between the isolated processes the OS worked to keep apart. Two threads run out of this part. One points *forward in this volume*: shared memory just handed us unsynchronized concurrency across processes, and threads did the same within one — the data race is now a debt owed twice over. Making shared-memory concurrency *correct* — mutual exclusion with locks, waiting with condition variables and semaphores, ordering with atomics and the memory model, and the lock-free techniques that avoid locks entirely — is the concurrency-and-parallelism part that begins next. The other points toward *later volumes*: sockets, framing, and the cost of the kernel boundary are the substrate under every networked data system, and the efficient handling of many connections at once (epoll, io_uring, zero-copy) is the low-level I/O material this volume closes with. The channels are open; now we have to make the traffic on them correct.

CAN YOU… WITHOUT LOOKING?

- Explain why a raw pointer cannot be passed between processes, and state the copy-vs-share distinction that organizes the IPC toolkit.
- Describe a pipe as a unidirectional byte stream, explain why it has no message boundaries, and say what framing is for.
- State a pipe's flow-control behavior (block on full/empty), what a `read` of 0 means, and when `SIGPIPE`/`EPIPE` fires.
- Explain why shared memory is the fastest channel, and why it is also the most dangerous — tie it to the Chapter 15 data race.
- Contrast `MAP_SHARED` and `MAP_PRIVATE` across `fork` using the measured 42-vs-0 result.
- Distinguish stream and datagram sockets by message shape, and say what sockets uniquely offer that the other channels do not.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is IPC for, and what are the three axes a channel sits on? (§19.1)
2. **R2.** What is a pipe, why does it have no message boundaries, and what is a FIFO? (§19.2)
3. **R3.** What is a pipe's capacity and flow-control behavior, and what does a `read` of 0 mean? (§19.2)
4. **R4.** Why is shared memory the fastest channel, and what does it fail to provide? (§19.3)
5. **R5.** How do stream and datagram sockets differ, and what do sockets offer that pipes and shared memory cannot? (§19.4)

FURTHER READING

- **Stevens & Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. (2013), Chapters 15 (Interprocess Communication), 16 (Network IPC: Sockets), and 17 (Advanced IPC).** The canonical, thorough treatment of pipes, FIFOs, shared memory, and sockets, including Unix-domain sockets and descriptor passing. `[⚠ reference — not free]`
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 10 (System-Level I/O) and Chapter 11 (Network Programming).** Unix I/O and the sockets interface from the programmer's view — the reference for §19.2 and §19.4. `[⚠ reference — not free]`; the CMU 15-213 materials are free.
- **Beej's Guide to Network Programming** (`beej.us`). Free. A practical, readable introduction to the sockets API, stream vs. datagram, and the framing you must add on top of TCP.
- **The Linux `pipe(2)`, `pipe(7)`, `mmap(2)`, `shm_overview(7)`, `unix(7)`, and `socket(2)` manual pages.** Primary sources for the pipe capacity and `PIPE_BUF` atomicity, `MAP_SHARED` semantics, POSIX shared memory, Unix-domain sockets, and the socket types. Verify per kernel version.

Exercises

19.1 WARM-UP In one sentence each, name the copy-vs-share distinction and give one IPC mechanism on each side of it.

19.2 WARM-UP A pipe is unidirectional and only usefully connects processes created how? What does a FIFO add that lets unrelated processes use it?

19.3 WARM-UP What does a `read` on a pipe return when all write ends are closed, and what happens when you `write` to a pipe whose read ends are all closed?

19.4 CORE Explain why a pipe has no message boundaries, using the measured "two 3-byte writes, one 6-byte read" result. If your protocol needs discrete messages over a pipe or a TCP stream, what must you add, and give two concrete framing schemes.

19.5 CORE Explain why shared memory is the fastest local IPC channel *and* why it is the most hazardous. Contrast `MAP_SHARED` and `MAP_PRIVATE` across `fork` using the 42-vs-0 measurement, and tie the hazard back to Chapter 15.

19.6 CORE Distinguish a stream socket from a datagram socket by message shape and by guarantees. For each of these, say which you would pick and why: (a) a reliable in-order request/response protocol; (b) low-latency telemetry where an occasional lost sample is fine. What does `AF_UNIX` change versus `AF_INET`?

19.7 COST (a) Roughly how many copies does a byte make going from one process to another through a pipe versus through shared memory, and why does that make shared memory faster? (b) The pipe holds 65536 bytes before a writer blocks — explain how that bound provides flow control for a pipeline. (c) What is the cost you pay to get shared memory's zero copies, stated as an engineering burden rather than a number?

19.8 FADED *Worked, with the last step for you.* A logging service on one host receives log lines from many local producers. The team uses a single shared-memory ring buffer that all producers and the one consumer read and write directly, with no locks, "because shared memory is fast." Lines occasionally come out garbled or duplicated under load.
Step 1 (given): Shared memory shares the frames but provides no mutual exclusion or ordering.
Step 2 (given): Multiple producers advancing the ring's write index concurrently is a read–modify–write on shared state — a data race.
Step 3 (given): The race only manifests when two producers hit the index at overlapping moments, so light load looks fine and heavy load corrupts.
Step 4 (you): Name the bug precisely, explain why it appears only under load, and describe two fixes — one using a synchronization primitive placed in the shared region, and one that changes the channel choice to avoid the shared write index entirely.

19.9 MIXED For each need, choose the best IPC mechanism (pipe/FIFO, shared memory, Unix-domain socket, or network socket) and justify in a line: (a) a shell pipeline `grep | sort`; (b) a video frame passed 60 times a second between two processes on the same GPU host; (c) a client on a laptop calling a service in a datacenter; (d) two local daemons that must exchange requests *and* pass an open file descriptor between them.

19.10 MIXED Drawing on Chapters 14–19, classify each statement as *true* or *false* and fix the false ones: (a) "A TCP connection delivers each `send` as exactly one `recv`." (b) "Two processes can communicate by one passing the other a pointer into its heap." (c) "Shared memory needs no synchronization because it is just memory." (d) "A signal can carry a kilobyte of structured data from one process to another." (e) "Closing all write ends of a pipe is how the reader learns the stream is finished."

19.11 STRETCH A pipeline `producer | consumer` runs with the producer far faster than the consumer. (a) Explain, using the 65536-byte buffer and blocking semantics, why the producer does not run away and exhaust memory — where does it end up waiting? (b) How is this *backpressure* related to the flow control you would otherwise have to build by hand over a raw shared-memory channel? (c) Now the consumer dies. Trace what the producer observes on its next write, naming the signal and the errno, and explain why the OS designed it to notify the writer rather than silently discard the bytes.

19.12 LAB On your own machine: (a) write a program that creates a pipe, writes `"ABC"` then `"DEF"`, and does one `read` — confirm you get six bytes with no boundary. (b) Fill a non-blocking pipe one byte at a time and print the capacity; confirm it near 65536 and try changing it with `fcntl(F_SETPIPE_SZ)`. (c) `mmap` one page `MAP_SHARED` and one `MAP_PRIVATE`, `fork`, have the child write both, and print what the parent sees — confirm shared changes cross and private ones do not. Label your compiler and kernel version.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART IV

Concurrency & Parallelism

Threads, locks and their hazards, lock-free techniques, atomics, and the memory model that makes concurrent code correct (or not).

Chapter 20 — Mutual Exclusion: Locks and the Critical Section

Before you start: Chapter 15 (two threads incrementing a shared counter lost more than half their updates — the data race this chapter fixes), Chapter 19 (shared memory gave two processes raw sharing with no synchronization — the same debt), Chapter 16 (the scheduler can preempt a thread between *any* two instructions — which is why the race window is always open). · Part III showed how the OS creates concurrency; Part IV is how to make it *correct*, and it starts with the most basic tool: excluding all but one thread from a critical section.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 15 — decompose `counter++` into the three machine steps it really is, and show one interleaving of two threads that loses an update. (2) Chapter 19 — shared memory gave two processes the sharing; what one thing did it *not* give them, that this chapter supplies? (3) *Predict:* wrapping a one-nanosecond increment in a lock and unlock, on an uncontended lock (no other thread competing), how much time do you think the lock and unlock add — about the same, ~10×, ~1000×? Write your guess; we measure it in §20.3.

Chapter 15 left us with a debt and Chapter 19 doubled it: whenever two flows of control — threads in a process, or processes over shared memory — read and write the same memory without coordination, the result is a **data race**, which loses updates nondeterministically and is undefined behavior in C and C++. The fix has a name and a shape. The updates were lost because `counter++` is really *load, add, store*, and two threads interleaved those steps; the cure is to ensure that once a thread begins that sequence, no other thread may begin it until the first finishes — to make the sequence a **critical section** that runs with **mutual exclusion**. This chapter builds that guarantee: what a critical section is and why it needs mutual exclusion (§20.1), the **mutex** that provides it and what has to exist underneath (§20.2), the cost of locking — uncontended and contended (§20.3) — and the sharpest failure mode locks introduce, **deadlock**, with the four conditions that produce it (§20.4). We close by carrying locks forward to the coordination primitives of the next chapter (§20.5). A lock is the simplest correct answer to the data race, and every complication in this part is a reaction to a cost a lock imposes.

20.1 The critical section and mutual exclusion

A **race condition** is a situation where the correctness of a computation depends on the relative timing or interleaving of multiple flows of control. The specific race we keep meeting is on a shared read–modify–write: `counter++` loads the value, adds one, and stores it back, and if a second thread loads the same old value before the first stores, one increment is overwritten. The region of code that must not be interleaved — here, the load–add–store — is a **critical section**: code that touches shared state and produces a wrong answer if two threads execute it concurrently. The property we need over a critical section is **mutual exclusion**: at most one thread is inside it at any instant. Mutual exclusion turns "load–add–store" from a sequence three other threads can climb into the middle of into an *indivisible* step, restoring the intuition that `counter++` just adds one.

Here is the debt and its repayment measured on the same workload — four threads, each incrementing a shared counter one million times, so the correct total is 4,000,000 (gcc 11.4.0, Linux 6.18):

```
no lock:    counter=1187737 (expected 4000000)
no lock:    counter=1144730 (expected 4000000)
no lock:    counter=2766329 (expected 4000000)
with mutex: counter=4000000 (expected 4000000)
with mutex: counter=4000000 (expected 4000000)
with mutex: counter=4000000 (expected 4000000)
```

Unsynchronized, the count is wrong and *different every run* — over half the updates lost in some runs, nondeterministically, because the interleaving depends on the scheduler. With a mutex guarding the increment, the count is exactly 4,000,000 every time. That determinism is the point of mutual exclusion: it does not make the program faster (it is slower, §20.3) — it makes it *correct*, and correct *reproducibly*, which no amount of re-running an unlocked version can achieve. As Chapter 15 warned, a racing program can even print the right answer by luck; mutual exclusion is how you stop depending on luck.

QUICK CHECK

Define a critical section and the property (mutual exclusion) you need over it. Why does the unsynchronized counter give a *different* wrong answer each run, while the locked one is always exactly right? (Answer after the next section.)

20.2 The mutex: how to lock, and what is underneath

The primitive that provides mutual exclusion is the **mutex** (mutual-exclusion lock). It has two operations: **lock** (acquire) and **unlock** (release). A mutex is either free or held by one thread; `lock` on a free mutex takes it and returns, `lock` on a held mutex *blocks* the caller until the holder unlocks. So the discipline is: lock before the critical section, unlock after, and only the one thread holding the lock runs the protected code. In POSIX threads:

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&m);      /* enter critical section — blocks if another thread holds m */
counter++;                    /* protected: at most one thread here at a time */
pthread_mutex_unlock(&m);    /* leave; a waiting thread may now proceed */
```

Two disciplines matter beyond the mechanics. First, all threads that touch the shared state must lock the *same* mutex — a lock excludes only the threads that agree to take it; a critical section guarded by a lock one racer ignores is not guarded at all. Second, hold the lock for *as short a critical section as correctness allows*: while you hold it, every other thread that needs it is stalled, so the lock serializes exactly the code it covers (§20.3). There is a bonus the mutex delivers beyond exclusion: the POSIX lock and unlock operations also **synchronize memory**, establishing an ordering (a happens-before edge) between the unlock in one thread and the next lock in another. That is why the non-atomic `counter++` inside a critical section is not merely excluded but *well-defined* — the lock supplies the ordering the memory model otherwise denies a plain shared access (the memory model itself is a later chapter in this part).

What has to exist under `lock` for it to work? The lock operation must itself be race-free — but "check if the mutex is free and take it" is a read–modify–write, the very thing we cannot do safely without help. The help comes from the **hardware**: CPUs provide atomic read-modify-write instructions — test-and-set, compare-and-swap, atomic exchange — that perform "read the old value and write a new one" as one indivisible step no other core can split. A mutex is built on one of these. (Pure-software mutual exclusion is possible — Dekker's and Peterson's algorithms achieve it with only ordinary loads and stores — but it is subtle and slow, and real locks use the hardware atomic; those atomics are a chapter of their own later in this part.) On Linux, a typical mutex takes the fast path entirely in user space with an atomic when uncontended, and only traps into the kernel — via the `futex` system call — to sleep when it must actually wait, which is why an uncontended lock is cheap but not free, the subject of §20.3.

> **QUICK CHECK**
>
> What are the two operations on a mutex, and what does `lock` do when the mutex is already held? Why must a lock be built on a hardware atomic instruction rather than an ordinary "if free, take it"? (Answer below the communicate box.)

20.3 The cost of a lock: uncontended and contended

A lock is not free even when no one is competing for it. Measured single-threaded — a plain increment versus the same increment wrapped in an uncontended lock and unlock (gcc 11.4.0 `-O2`, Linux 6.18; illustrative, machine- and run-dependent):

```
plain increment:                    ~1.0 ns/op
lock + increment + unlock:          ~12 ns/op
ratio:                              ~12x
```

The uncontended lock adds roughly an order of magnitude to a trivial operation — a handful of nanoseconds for the atomic and the bookkeeping, answering the opener: not "about the same," and not 1000×, but ~10× on a bare increment. That overhead is the *floor*. The real cost arrives under **contention**, when threads actually compete, and it grows in three ways. (1) **Serialization**: the lock forces the critical section to run one thread at a time, so if a large fraction of the work is inside the lock, adding cores stops helping — this is Amdahl's law applied to a critical section, and it is why "hold the lock briefly" is a performance rule, not just style. (2) **Cache-line bouncing**: the lock word (and the data it guards) ping-pongs between cores' caches as ownership changes hands, and each transfer is a cross-core cache miss — the mechanical-sympathy cost from Part I, now paid per contended acquisition. (3) **Context switches**: when a blocking mutex makes a waiting thread sleep, acquiring the lock later means waking it, a scheduler round-trip (Chapter 16) far more expensive than the atomic itself.

These costs drive the central design tension of locking: **coarse-grained** versus **fine-grained**. One big lock around a whole data structure is simple and easy to reason about, but it serializes all access, throttling concurrency. Many small locks — one per bucket, per node, per shard — let independent operations proceed in parallel, but they multiply the overhead, complicate the code, and, crucially, open the door to **deadlock** when a thread must hold more than one at a time (§20.4). There is no free lunch: you trade simplicity and safety against concurrency, and the right point depends on how contended the structure actually is — which, per Part I's discipline, you should *measure* before assuming.

THE SAME IDEA THREE WAYS: A LOCK SERIALIZES A CRITICAL SECTION

1. In words. A mutex admits one thread at a time to the critical section; others block at `lock` until the holder `unlock`s. That guarantees correctness (mutual exclusion) and imposes a cost (serialization plus per-acquisition overhead), so the lock should cover the smallest region that is still correct.

2. As a picture. Figure 20.1: two threads' timelines; thread B's `lock` blocks while A is inside the critical section, then proceeds after A's `unlock` — the protected region never overlaps.

3. As numbers. Uncontended, a lock+unlock turns a ~1 ns increment into ~12 ns (~10×). Contended, the four-thread counter is *correct* (4,000,000 every run) but slower than the racing version — you buy determinism with throughput.

(Answer to the §20.1 quick check: a critical section is code touching shared state that misbehaves if two threads run it concurrently; mutual exclusion means at most one thread is inside it at a time. The unlocked counter varies because the lost updates depend on the scheduler's interleaving, which differs each run; the locked counter is always exactly right because mutual exclusion makes the load–add–store indivisible, removing the interleaving entirely.)

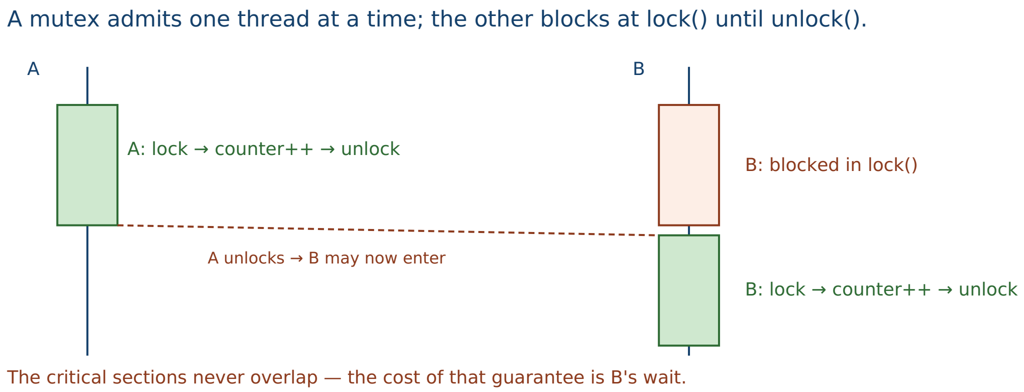


Figure 20.1: Mutual exclusion by a mutex. While thread A holds the lock, thread B blocks inside `lock()`; only after A's `unlock()` does B enter its critical section. The protected regions are serialized — that is the correctness guarantee and, simultaneously, the throughput cost.

20.4 Deadlock and the four conditions

Fine-grained locking buys concurrency but introduces a new way to fail completely. When a thread must hold more than one lock at a time, two threads acquiring the same locks in *different orders* can each grab one and then wait forever for the other's — a **deadlock** (Dijkstra's "deadly embrace"). Here are two threads and two mutexes A and B: thread 1 locks A then B, thread 2 locks B then A. Run with opposite orders versus a consistent order (gcc 11.4.0, Linux 6.18; the deadlocking run is killed after a 3-second timeout):

```
opposite lock order (t1: A,B ; t2: B,A):   hangs; killed after 3s (exit 124 = timed out)
consistent lock order (both: A,B):         t1 got both; t2 got both; both threads finished
```

With opposite orders the program hangs: thread 1 holds A and waits for B while thread 2 holds B and waits for A — neither can proceed, forever. With a consistent order it always completes, because two threads wanting both locks can never form the cycle. That fix generalizes into the theory. Deadlock requires **all four** of the classic conditions (Coffman, Elphick & Shoshani, 1971) to hold simultaneously: **mutual exclusion** (a resource is held exclusively), **hold-and-wait** (a thread holds one resource while requesting another), **no preemption** (a resource cannot be forcibly taken from its holder), and **circular wait** (a cycle of threads each waiting on the next). Break *any one* and deadlock is impossible. The most practical break is the last: impose a **global lock order** — always acquire locks in the same agreed sequence — and no circular wait can form, which is exactly what the consistent-order run did.

Two relatives round out the failure family. **Livelock**: threads are not blocked but keep reacting to each other and make no progress (two people stepping aside in the same direction repeatedly). And **priority inversion**: a low-priority thread holds a lock a high-priority thread needs, and a medium-priority thread — runnable, so it preempts the low one under a strict-priority scheduler (Chapter 16's `SCHED_FIFO`) — keeps the low thread off the CPU, so it never releases the lock, and the high-priority thread is stuck behind both. (This is the mechanism famously diagnosed on the 1997 Mars Pathfinder mission; the standard cure is *priority inheritance*, where the lock holder temporarily inherits the priority of the highest waiter.) All three share a moral: locks compose badly, and the more locks you hold at once, the more carefully you must reason about the order and the priorities.

TRAP: INCONSISTENT LOCK ORDERING (THE DEADLY EMBRACE)

The classic deadlock bug is two code paths that acquire the same two locks in *opposite* orders — one `lock(A); lock(B)`, another `lock(B); lock(A)`. Each thread grabs its first lock, then blocks forever on the second, which the other holds: total, permanent stall. The bug is vicious for the usual reason — it needs the two paths to interleave at just the wrong moment, so it is *rare*: the code passes tests and every demo, then one day under load two requests hit the two paths simultaneously and the service freezes with no crash, no error, no log — just threads stuck in `lock()`. It is especially insidious when the two locks are acquired far apart in the call graph, or hidden inside library calls, so nobody sees both orders in one place. The discipline: define a **global lock ordering** and always acquire multiple locks in that order (break circular wait), hold as few locks as possible for as short a time as possible (attack hold-and-wait), and use `trylock` with a back-off if you truly cannot order them. If you ever write `lock(X)` while already holding `lock(Y)`, you owe yourself a proof that no other path ever takes them the other way.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our service froze in production — no crash, no error, CPU near zero, it just stopped serving. I restarted it and it was fine for a week, now it's frozen again. Must be a hardware glitch, right?" You have a thread dump showing several threads stuck in `pthread_mutex_lock`. Cover the paragraph and respond in four or five sentences, drawing on §20.4.

Model answer. "That is the signature of a *deadlock*, not hardware: CPU near zero (nobody is spinning, everyone is blocked), no crash or error (deadlock is a permanent wait, not a fault), and threads parked in `pthread_mutex_lock` — which the dump confirms. It reproduces rarely because it needs two code paths to acquire the same locks in opposite orders and to interleave at just the wrong instant, so it hides for a week and then, under the right concurrent load, forms a cycle: each thread holds one lock and waits on the other forever. The fix is to establish a *global lock ordering* so every path takes those locks in the same sequence — that breaks the circular-wait condition and makes the cycle impossible — plus shrink the critical sections so we hold fewer locks at once. Let's diff the two paths in the dump to find the inconsistent order, and add a lint or a runtime lock-order check so the next one fails loudly in testing instead of silently in production." Naming the deadlock from its fingerprint — blocked threads, zero CPU, no error — and prescribing a lock order is the senior register.

(Answer to the §20.2 quick check: the two operations are `lock` (acquire) and `unlock` (release); `lock` on a held mutex ***blocks*** *the caller until the holder releases it. A lock must be built on a hardware atomic read-modify-write because "check if free, then take it" is itself a read–modify–write — without an indivisible instruction, two threads could both see the mutex free and both "take" it, which is the very race the lock is supposed to prevent.)*

20.5 What to carry forward

A **race condition** is correctness that depends on interleaving; the region that must not interleave is a **critical section**; and the property that fixes it is **mutual exclusion** — at most one thread inside at a time. A **mutex** supplies it with `lock`/`unlock`: we watched a four-thread counter go from a nondeterministic ~1.1–2.8 million to exactly 4,000,000 every run. A lock also *synchronizes memory*, giving the ordering that makes the guarded non-atomic accesses well-defined, and it rests on a hardware atomic instruction because "if free, take it" is itself a race. Locks are not free: an uncontended lock is ~10× a bare increment, and under contention the costs are serialization (Amdahl on the critical section), cache-line bouncing between cores, and context switches when waiters sleep — which is why you hold the lock for the smallest correct region and choose coarse vs. fine granularity by measurement. And locks introduce **deadlock**: all four Coffman conditions (mutual exclusion, hold-and-wait, no preemption, circular wait) must hold, so breaking one — most practically by imposing a global lock order — prevents it, as the consistent-order run showed against the hanging opposite-order one.

Locks give us *safety*: they keep threads out of each other's critical sections. What they do not give us is a way to *wait for a condition*. A consumer thread that needs data another thread has not yet produced cannot make progress by locking harder — and it must not sit spinning while holding the lock, since that both burns the CPU and keeps the producer out. What it needs is to *release the lock and sleep until the condition it is waiting for becomes true*, then wake and re-check — coordination, not just exclusion. That is the **condition variable**, and its counting cousin the **semaphore**: the next chapter builds the producer–consumer pattern on them, and shows why the wait must sit in a loop rather than an `if`. Mutual exclusion was the first half of shared-memory correctness; waiting and signalling is the second, and together they are the classical toolkit that everything lock-free and every memory-model rule later in this part is measured against.

CAN YOU… WITHOUT LOOKING?

- Define race condition, critical section, and mutual exclusion, and say how they relate.
- Write a mutex-guarded critical section from memory, and explain why every racing thread must take the *same* lock.
- Explain why a mutex must rest on a hardware atomic, and what "if free, take it" gets wrong without one.
- State the uncontended lock cost (~10× a bare op) and the three ways contention makes it worse.
- Explain the coarse- vs. fine-grained locking trade-off in terms of concurrency, overhead, and deadlock risk.
- List the four deadlock conditions and give the most practical one to break, with the fix that breaks it.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a critical section, and what property must hold over it? (§20.1)
2. **R2.** What are a mutex's two operations, and why must all racers take the same one? (§20.2)
3. **R3.** Why must a lock be built on a hardware atomic instruction? (§20.2)
4. **R4.** Name the three ways contention makes a lock expensive. (§20.3)
5. **R5.** List the four deadlock conditions and name the most practical one to break. (§20.4)

FURTHER READING

- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Locks" and "Lock-based Concurrent Data Structures."** Free (`pages.cs.wisc.edu/~remzi/OSTEP/`). Building a lock from hardware atomics (test-and-set, compare-and-swap), spin vs. sleeping locks, and locking real data structures — the reference for §20.1–§20.3.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 12 (Concurrent Programming).** Threads, shared variables, mutual exclusion, and synchronization from the programmer's view. `[⚠ reference — not free]`; CMU 15-213 materials free.
- **Coffman, Elphick & Shoshani, "System Deadlocks," *ACM Computing Surveys* 3(2): 67–78 (1971), DOI 10.1145/356586.356588.** The primary source for the four necessary conditions for deadlock. Author/venue/year verified via the ACM Digital Library.
- **Dijkstra, "Cooperating Sequential Processes" (EWD123, 1965; published 1968).** The origin of the critical-section and mutual-exclusion problem, the "deadly embrace" (deadlock), and the semaphore. Free via the E. W. Dijkstra Archive (`cs.utexas.edu/~EWD/`).
- **The Linux `pthread_mutex_lock(3)` and `futex(2)` manual pages.** Primary sources for POSIX mutex semantics and the fast-user-space/kernel-sleep mechanism under a mutex. Verify per system.

Exercises

20.1 WARM-UP In one sentence each, define a critical section and mutual exclusion, and say why `counter++` needs the latter.

20.2 WARM-UP What are the two operations on a mutex, and what does the acquire operation do when the mutex is already held by another thread?

20.3 WARM-UP Name the four conditions that must all hold for deadlock, and give the one most practical to break in application code.

20.4 CORE Explain why a mutex must be built on a hardware atomic read-modify-write instruction. Walk through what could go wrong if `lock` were implemented as "if the mutex is free, set it to held" with ordinary loads and stores, and connect that failure to the data race of Chapter 15.

20.5 CORE Two threads each lock mutexes A and B, but thread 1 does `A then B` and thread 2 does `B then A`. (a) Show an interleaving that deadlocks. (b) Explain which of the four conditions your fix removes when you make both threads acquire in the order `A then B`. (c) Why does this bug typically pass tests and only appear in production?

20.6 CORE A mutex both *excludes* other threads and *synchronizes memory*. Explain each role, and why the second is what makes a non-atomic `counter++` inside a critical section not just serialized but well-defined under the C/C++ memory model. Tie this to why a plain shared access without a lock is undefined behavior (Chapter 15).

20.7 COST (a) An uncontended lock+unlock measured ~12 ns against a ~1 ns bare increment — explain where that ~10× goes. (b) Under heavy contention the cost is far higher; name the three sources (serialization, cache-line bouncing, context switches) and say which dominates when the critical section is tiny versus when threads must sleep. (c) Using Amdahl's reasoning, explain why shrinking the critical section can matter more than making the lock itself faster.

20.8 FADED *Worked, with the last step for you.* A bank-transfer function locks the source account, then the destination account, then moves money. Under load the service occasionally freezes; a thread dump shows threads blocked in `lock`.
Step 1 (given): Transfer `X→Y` locks X then Y; a concurrent transfer `Y→X` locks Y then X.
Step 2 (given): If both start together, the first holds X and waits for Y; the second holds Y and waits for X.
Step 3 (given): That is a circular wait — all four Coffman conditions now hold — so both threads block forever.
Step 4 (you): Name the bug, explain why it is load- and timing-dependent, and give a concrete fix that imposes a global lock order on accounts (say, by account id) so no cycle can form — and state which condition your fix removes.

20.9 MIXED For each scenario, decide whether the right tool is (i) a single mutex, (ii) fine-grained per-item mutexes, or (iii) no lock at all because there is no sharing — and justify: (a) a global counter incremented by all threads; (b) a hash map where operations on different buckets are independent; (c) a value each thread keeps in its own thread-local storage; (d) a rarely-updated configuration read by everyone on every request.

20.10 MIXED Drawing on Chapters 15–20, classify each as *true* or *false* and fix the false ones: (a) "If two threads take different locks, they are safely mutually exclusive." (b) "A deadlocked program spins the CPU at 100%." (c) "A mutex makes the guarded code faster." (d) "Holding a lock longer is always safer." (e) "You can implement a correct lock with only ordinary loads and stores and no hardware atomic."

20.11 STRETCH A strict priority scheduler (Chapter 16's `SCHED_FIFO`) runs three threads: low L holds a lock, high H needs it, medium M is CPU-bound and needs no lock. (a) Explain how H can end up blocked indefinitely even though it is the highest priority — name the phenomenon. (b) Why does M, not H, keep L off the CPU? (c) Describe priority inheritance and exactly which link in the chain it breaks. (d) Relate this to why holding a lock while doing unbounded work is dangerous under real-time scheduling.

20.12 LAB On your own machine: (a) run four threads incrementing a shared counter one million times each, first with no lock, then with a mutex — confirm the unlocked total is wrong and varies while the locked total is exactly 4,000,000. (b) Time a plain increment versus a lock+increment+unlock in a tight loop and report the ratio (expect roughly an order of magnitude). (c) Write two threads that lock two mutexes in opposite orders and observe the hang; then fix the order and confirm it completes. Label your compiler and kernel version, and note that the timing numbers are illustrative.

Chapter 21 — Coordinating Threads: Condition Variables and Semaphores

Before you start: Chapter 20 (a mutex gives mutual exclusion over a critical section, but no way to *wait* for a condition), Chapter 16 (a blocked thread is off the CPU; a runnable one competes for it — the difference between sleeping and spinning), Chapter 15 (threads share memory, so one can produce what another consumes). · Locks kept threads *out* of each other's way; this chapter is how one thread *waits* for another to make something true — coordination on top of exclusion.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 20 — what does a mutex guarantee, and name one of the four conditions required for deadlock. (2) Chapter 16 — a thread *spinning* in a loop checking a flag versus a thread *blocked* waiting: which one competes for the CPU, and what does that cost? (3) *Predict:* a consumer waits ~0.5 seconds for a producer, first by spinning in a loop that re-checks a flag under a lock, then by blocking on a proper wait primitive. How much CPU time do you think each consumes during that half-second? Write both guesses; we measure them in §21.1.

A mutex answers "may I touch the shared state?" It does not answer "is the shared state *ready* yet?" Consider a bounded buffer between a producer and a consumer: the consumer must not dequeue from an empty buffer, and the producer must not enqueue into a full one. A lock protects the buffer's fields, but it cannot make a consumer *wait* for an item to appear — and the naive fix, spinning in a loop re-checking under the lock, is a disaster: it burns a whole CPU doing nothing and, worse, holds the lock the producer needs, so the very condition it waits for can never come true. What a waiting thread needs is to **release the lock and sleep until signalled**, then wake and re-check. This chapter builds that: why busy-waiting is wrong (§21.1); the **condition variable** — wait, signal, broadcast — and the non-negotiable rule that the wait sits in a `while` loop, not an `if` (§21.2); the **semaphore**, Dijkstra's counting primitive that *remembers* signals (§21.3); and the classic ways to get coordination wrong, plus the monitor discipline that packages it (§21.4). We close by carrying the blocking toolkit toward atomics and the memory model (§21.5). Exclusion was half of shared-memory correctness; waiting is the other half.

21.1 Waiting without wasting: why busy-waiting is wrong

The obvious way to wait for a condition is to loop until it holds — a **busy-wait**, or spin. It is almost always wrong for a thread that may wait more than a few instructions, for two reasons. First, a spinning thread is *runnable*, so the scheduler keeps giving it the CPU (Chapter 16) while it does no useful work — it burns a core. Second, and more insidious in the bounded-buffer case, if the spin holds the lock while re-checking, it locks out the very thread that would make the condition true, so the wait can never end: a self-inflicted deadlock. Here is the CPU cost measured — a thread waits ~0.5 s for a signal, first by busy-polling a flag under a lock, then by blocking on a condition variable (gcc 11.4.0, Linux 6.18; CPU time via `getrusage`):

```
busy-poll (spins):  waited ~0.5s wall, consumed 0.499 s CPU
condvar (blocks):   waited ~0.5s wall, consumed 0.001 s CPU
```

The spinning waiter burned essentially a full core-second of CPU for half a second of wall-clock waiting — 100% of a core doing nothing — while the blocking waiter consumed a thousandth of that, because it was *off the CPU*, asleep, until woken. That five-hundred-fold difference answers the opener and states the whole motivation: a thread that must wait should **sleep**, not spin, so the CPU goes to threads with work to do. (Spinning is not always wrong — for a wait known to be shorter than a context switch, a brief spin, or a spinlock, can beat sleeping; the point is that unbounded or long waits must block.) The primitive that lets a thread sleep until a condition changes, without holding the lock while it sleeps, is the condition variable.

> **QUICK CHECK**
>
> Give the two reasons busy-waiting is wrong for a thread that may wait a while, and say which one turns a spin-under-lock into a deadlock. Roughly how much CPU did the spinning waiter burn versus the blocking one? (Answer after the next section.)

21.2 Condition variables: wait, signal, and the `while` rule

A **condition variable** is a queue of threads waiting for some predicate over shared state to become true. It is always paired with a mutex (the one guarding that state) and has three operations. `wait(cv, m)` must be called with the mutex held; it *atomically* releases the mutex and puts the thread to sleep on the condition variable, and when the thread is later woken it *re-acquires the mutex* before returning — so the thread sleeps without holding the lock but wakes holding it again. `signal(cv)` wakes one waiting thread; `broadcast(cv)` wakes all of them. The atomic "release-and-sleep" is the crucial part: it closes the window in which a thread could check the predicate, decide to wait, and miss a signal that arrives before it actually sleeps.

The rule that trips up everyone learning this is: **always wait in a `while` loop that re-checks the predicate, never a bare `if`**. Here is the producer–consumer bounded buffer done correctly (gcc 11.4.0, Linux 6.18) — note the `while`:

```
/* consumer */
pthread_mutex_lock(&m);
while (count == 0)                 /* while, NOT if */
    pthread_cond_wait(&not_empty, &m);
item = buf[out]; out = (out+1) % CAP; count--;
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&m);

produced sum=210 consumed sum=210 (1..20 sum=210); buffer never exceeded CAP=4
```

Producer and consumer agreed exactly (both sums 210, the sum of 1..20) and the buffer never exceeded its capacity of four — the coordination is correct. The `while` is mandatory for two independent reasons. (1) **Spurious wakeups**: POSIX explicitly permits `cond_wait` to return without any signal, so on waking you must re-verify the predicate actually holds. (2) **Stolen conditions**: even with a real signal, between the signal and this thread re-acquiring the mutex, *another* thread may have run and consumed the item, so the predicate you were signalled about may already be false again. In both cases the `while` simply re-checks and, if the predicate is still false, waits again — turning "I was woken" into "I woke *and* confirmed the condition," which is what correctness requires. An `if` assumes the wakeup means the predicate holds; under either hazard, that assumption is a bug.

> **THE SAME IDEA THREE WAYS: WAIT RELEASES THE LOCK, SLEEPS, RE-CHECKS IN A LOOP**
>
> **1. In words.** `cond_wait` atomically releases the mutex and sleeps; a `signal` wakes a waiter, which re-acquires the mutex and — because the wakeup may be spurious or the condition already re-consumed — re-tests the predicate in a `while` loop, waiting again if it is still false.
>
> **2. As a picture.** Figure 21.1: the consumer holds the mutex, finds `count == 0`, calls `wait` (drops the lock, sleeps); the producer locks, enqueues, `signal`s; the consumer wakes, re-acquires the lock, re-checks `count`, and proceeds.
>
> **3. As numbers.** The bounded buffer with `while`-loops: produced and consumed sums both 210 over items 1..20, buffer never above CAP = 4. Blocking on the condition variable cost 0.001 s CPU versus the spinner's 0.499 s.

(Answer to the §21.1 quick check: busy-waiting is wrong because a spinning thread stays runnable and burns a CPU doing no work, and — the deadlock case — if it spins while holding the lock, it locks out the thread that would make the condition true, so the wait never ends. The spinning waiter burned ~0.499 s of CPU for a ~0.5 s wait, roughly 500× the ~0.001 s the blocking waiter used.)

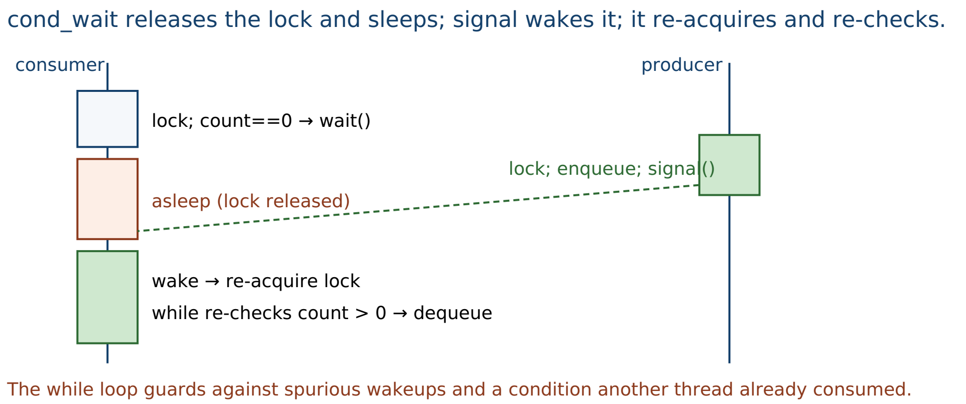


Figure 21.1: A condition variable. The consumer, holding the mutex, finds the buffer empty and `wait`s — atomically releasing the lock and sleeping. The producer enqueues and `signal`s; the consumer wakes, re-acquires the mutex, and re-checks the predicate in its `while` loop before proceeding.

21.3 Semaphores: counting permits that remember

A **semaphore** (Dijkstra, 1965) is an integer with two atomic operations and a waiting queue: **wait** (historically *P*; POSIX `sem_wait`) decrements the count, blocking if it is already zero, and **post** (historically *V*; POSIX `sem_post`) increments the count, waking a waiter if any. A *binary* semaphore (count 0 or 1) behaves much like a mutex; a *counting* semaphore initialized to N models a pool of N interchangeable permits. Ten threads competing for a semaphore initialized to three never exceed three inside the guarded section (gcc 11.4.0, Linux 6.18):

```
sem_init(&slots, 0, 3);   /* three permits */
/* each worker: sem_wait(&slots) ... work ... sem_post(&slots) */

10 threads, semaphore initialized to 3: max concurrently in section = 3
```

The semaphore capped concurrency at exactly its initial count — the natural tool for "at most N at once" (a connection pool, a bounded worker set, N copies of a resource). The producer–consumer buffer has an elegant semaphore form too: a semaphore `empty` initialized to the buffer size counts free slots (the producer `wait`s it before enqueuing), and a semaphore `full` initialized to zero counts filled slots (the consumer `wait`s it before dequeuing), with a mutex protecting the buffer indices. The crucial property that distinguishes a semaphore from a condition variable is **memory**: a semaphore's count *remembers* posts. A `sem_post` with no waiter increments the count, so a later `sem_wait` succeeds immediately — the signal is not lost. A `cond_signal` with no waiter, by contrast, *vanishes*: it wakes no one and leaves no trace. That single difference is why a condition variable must always be paired with a shared-state predicate checked under the mutex (the state is the memory; the condition variable is only the wakeup), whereas a semaphore *is* its own state.

> **QUICK CHECK**
>
> What do a semaphore's two operations do, and what does a counting semaphore initialized to N give you? State the one key difference between a semaphore and a condition variable regarding a signal sent with no one waiting. (Answer below the communicate box.)

21.4 Getting it wrong: lost wakeups, `if`-not-`while`, and the monitor

The coordination primitives are small, and the bugs cluster around a few classic mistakes — all versions of confusing "I was woken" with "the condition holds," or of signalling without the shared state to back it. The most common is using `if` where the rule demands `while`: a single check on waking, which a spurious wakeup or a condition consumed by another thread turns into acting on a false predicate — dequeuing from a now-empty buffer, proceeding on data that is not there. Its cousin is the **lost wakeup**: signalling a condition variable *without* updating (and holding the lock over) the shared state, so a waiter that has not yet slept misses the signal and a waiter that has slept is never woken. Because a condition variable does not remember signals (§21.3), the discipline is ironclad: change the shared state under the mutex, *then* signal; and always wait in a loop on a predicate over that state. A semaphore, which remembers, forgives the timing — but a semaphore misused as a mutex you forget to post, or posted the wrong number of times, deadlocks or over-admits just as badly.

These patterns recur so reliably that they have a structuring discipline: the **monitor** (Hoare, 1974; Brinch Hansen). A monitor bundles shared state with the mutex that protects it and the condition variables threads wait on, so that all access goes through procedures that hold the lock and coordinate through the conditions — the programmer stops hand-wiring locks and signals and instead works with an object whose methods are automatically mutually exclusive. This is exactly what a Java object's `synchronized` methods with `wait/notify` package, and what higher-level constructs like a thread-safe blocking queue or a Go channel give you: the mutex, the condition variables, and the predicate discipline, wrapped so you cannot forget the `while` loop or the "signal under the lock" rule. The lesson to carry: prefer the highest-level correct abstraction available — a concurrent queue over hand-rolled condition variables, a monitor over raw locks and signals — and drop to the primitives only when you must, because the primitives are where the subtle bugs live.

TRAP: IF INSTEAD OF `WHILE` AROUND `COND_WAIT` (AND SIGNALLING WITHOUT THE STATE)

Two coupled mistakes cause most condition-variable bugs. The first is guarding `cond_wait` with an `if` instead of a `while`: you check the predicate once, wait, and on waking assume it holds — but POSIX permits **spurious wakeups** (waking with no signal), and even a real signal can be "stolen" by another thread that re-acquires the lock first and consumes the condition, so on waking the predicate may be false. Acting on it — dequeuing from an empty buffer, using data that is not there — corrupts state or crashes, and it is rare and load-dependent, so it survives testing. The second is the **lost wakeup**: because a condition variable does *not* remember a signal sent with no waiter (unlike a semaphore's count), if you signal without holding the lock and updating the shared state, a thread that is about-to-wait can miss the signal and sleep forever. The discipline that kills both: (1) always `while (!predicate) cond_wait(...)`, never `if`; (2) always update the shared state under the mutex and then signal. If you find yourself reasoning "the wakeup means the item is there," you have the `if`-not-`while` bug.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "My worker waits for jobs like this: `if (queue_empty) pthread_cond_wait(&cv, &m);` then it pops a job. And to avoid missing wakeups I call `pthread_cond_signal` right before I take the lock to enqueue. Simple and fast — what could go wrong?" Cover the paragraph and respond in four or five sentences, drawing on §21.2 and §21.4.

Model answer. "Two bugs, both timing-dependent, so they'll pass your tests and fail in production. First, the `if` must be a `while`: `cond_wait` can return spuriously, and even on a real signal another worker may have popped the job before you re-acquire the lock, so on waking you must re-check `queue_empty` — with `if`, you'll pop from an empty queue. Second, signalling *before* taking the lock and enqueuing is a lost-wakeup race: a condition variable doesn't remember a signal sent with no waiter, so if the worker is between its check and its `cond_wait` when you signal, it misses it and may sleep forever with a job sitting in the queue. The fix is the standard discipline — enqueue under the mutex, *then* signal while still holding it (or right after), and have the worker loop `while (queue_empty) cond_wait(...)`. Honestly, I'd wrap this in a thread-safe blocking queue so nobody has to remember these rules per call site." Naming both the `if`-not-`while` and the lost-wakeup, and reaching for a higher-level abstraction, is the senior register.

(Answer to the §21.3 quick check: `wait`/P decrements the count, blocking if it is zero; `post`/V increments it, waking a waiter if any. A counting semaphore initialized to N admits at most N threads at once — a pool of N permits. The key difference from a condition variable: a semaphore ***remembers*** *a post made with no waiter (the count rises, so a later wait succeeds), whereas a condition-variable signal with no waiter is lost.)*

21.5 What to carry forward

Locks (Chapter 20) exclude; this chapter's primitives **coordinate**. A thread that must wait for a condition should **sleep, not spin** — we measured a busy-poll burning 0.499 s of CPU against a blocking waiter's 0.001 s for the same half-second wait. A **condition variable** lets a thread atomically release its mutex and sleep until `signal`/`broadcast` wakes it, whereupon it re-acquires the mutex; the wait *must* sit in a `while` loop re-checking the predicate, because wakeups can be spurious and the condition can be consumed by another thread first — our bounded buffer with `while`-loops kept producer and consumer in exact agreement (sums 210) and the buffer within capacity. A **semaphore** is a counting primitive whose `wait`/`post` (P/V) admit at most its initial count of threads — three permits capped ten threads at three — and, unlike a condition variable, it *remembers* posts sent with no waiter, which is why a condition variable must always pair with a predicate over shared state while a semaphore is its own state. The bugs cluster on `if`-not-`while` and lost wakeups; the **monitor** (and its descendants — Java `synchronized`/`wait`/`notify`, blocking queues, channels) packages the discipline so you need not rebuild it per call site.

With locks, condition variables, and semaphores, you have the classical *blocking* synchronization toolkit — enough to write correct shared-memory concurrency. But it rests on assumptions this part must now examine directly. Every primitive here blocks a thread (a scheduler cost), and every one relies on the guarantee that memory operations become visible in a sensible order across cores — a guarantee that real multicore hardware does *not* provide for plain loads and stores, and that these primitives quietly supply through the same atomics that a lock is built on. The rest of this part opens that box: the hardware **atomics** (compare-and-swap and friends) that let you build synchronization — and lock-free data structures that avoid blocking entirely — and the **memory model** that states precisely what a concurrent program means, why a data race is undefined behavior, and what ordering you are and are not entitled to assume. The primitives you now know are the correct, comprehensible baseline; the chapters ahead show what they cost, how they work underneath, and when you can do without them.

CAN YOU… WITHOUT LOOKING?

- Give the two reasons busy-waiting is wrong, and the CPU numbers that show a blocking wait beating a spin.
- Explain what `cond_wait` does atomically, and why that atomicity closes a missed-signal window.
- State the `while`-not-`if` rule and give both reasons it is mandatory (spurious wakeups; stolen condition).
- Describe a semaphore's `wait`/`post`, what a counting semaphore of N gives you, and that it remembers posts.
- State the one difference between a semaphore and a condition variable regarding a signal with no waiter.
- Explain the lost-wakeup bug and the two-part discipline (update-under-lock-then-signal; wait in a loop) that prevents it.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why is busy-waiting wrong, and which failure makes a spin-under-lock a deadlock? (§21.1)
2. **R2.** What does `cond_wait` do, and why must the wait be in a `while` loop? (§21.2)
3. **R3.** What do a semaphore's two operations do, and what does an initial count of N mean? (§21.3)
4. **R4.** How does a semaphore differ from a condition variable when a signal is sent with no waiter? (§21.3)
5. **R5.** What is a lost wakeup, and what discipline prevents it? (§21.4)

FURTHER READING

- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Condition Variables," "Semaphores," and "Common Concurrency Problems."** Free (`pages.cs.wisc.edu/~remzi/OSTEP/`). The `while`-loop rule, the producer–consumer and reader–writer problems, semaphores, and the taxonomy of concurrency bugs — the reference for §21.1–§21.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 12 (Concurrent Programming), §12.5 (synchronizing threads with semaphores).** Semaphores, mutual exclusion, and producer–consumer from the programmer's view. `[⚠ reference — not free]`; CMU 15-213 materials free.
- **Dijkstra, "Cooperating Sequential Processes" (EWD123, 1965; published 1968).** The origin of the semaphore and its P/V operations. Free via the E. W. Dijkstra Archive (`cs.utexas.edu/~EWD/`).
- **Hoare, "Monitors: An Operating System Structuring Concept," *Communications of the ACM* 17(10):549–557 (1974), DOI 10.1145/355620.361161.** The primary source for monitors, with the bounded buffer and reader–writer examples. Author/venue/year verified via the ACM Digital Library.
- **The Linux `pthread_cond_wait(3)` and `sem_overview(7)` manual pages.** Primary sources for POSIX condition-variable semantics (including permitted spurious wakeups) and POSIX semaphores. Verify per system.

Exercises

21.1 WARM-UP In one sentence, why should a thread that may wait a while *sleep* rather than *spin*?

21.2 WARM-UP What three operations does a condition variable have, and what does `wait` do to the mutex when it is called?

21.3 WARM-UP What does a counting semaphore initialized to N guarantee about how many threads can be inside the section it guards?

21.4 CORE Explain why `cond_wait` must be called inside a `while` loop that re-checks the predicate, not an `if`. Give the two independent reasons (spurious wakeups and a condition consumed by another thread) and show, with a two-consumer example, how an `if` leads to acting on a false predicate.

21.5 CORE A condition variable does not remember a signal sent with no waiter, but a semaphore's count does. (a) Explain the lost-wakeup bug this creates for condition variables. (b) State the discipline (update shared state under the lock, then signal; wait in a loop) that prevents it. (c) Why does a semaphore not need this discipline for the "signal arrives before the waiter sleeps" case?

21.6 CORE Implement (in pseudocode) a bounded buffer with a mutex and two condition variables (`not_full`, `not_empty`). Show the producer and consumer, mark where each waits and signals, and explain why the buffer can never overflow or underflow and why the loops are necessary.

21.7 COST (a) A busy-poll waiter burned ~0.499 s of CPU for a ~0.5 s wait while a blocking waiter used ~0.001 s — explain the ~500× gap in terms of runnable-vs-blocked (Chapter 16). (b) When is a short spin actually *cheaper* than blocking, and what is the crossover quantity? (c) What does `cond_wait` cost that a spin does not (name the two scheduler events on the sleep/wake path), and why is it still the right default for a long wait?

21.8 FADED *Worked, with the last step for you.* A thread pool's workers wait for tasks. Each worker does `if (queue.empty()) cond_wait(&cv, &m);` then pops. Occasionally, under load, a worker pops from an empty queue and crashes; occasionally a submitted task is never picked up though workers are idle.
Step 1 (given): The `if` checks the predicate only once; on waking (spuriously, or after another worker took the task) the queue may be empty.
Step 2 (given): If the producer signals without holding the lock and updating the queue, a not-yet-sleeping worker can miss the signal.
Step 3 (given): A condition variable does not remember a signal with no waiter, so the missed signal is gone and the task waits with no one woken.
Step 4 (you): Name both bugs precisely, explain why each is intermittent and load-dependent, and give the corrected worker and producer (the `while` loop and the enqueue-under-lock-then-signal order), stating which bug each change fixes.

21.9 MIXED For each need, choose the best primitive (a plain mutex, a condition variable + mutex, or a counting semaphore) and justify in a line: (a) protect a single shared counter; (b) let at most 5 requests use a resource pool at once; (c) have a consumer block until a producer makes an item available; (d) signal "shutdown requested" to many waiting threads at once.

21.10 MIXED Drawing on Chapters 18–21, classify each as *true* or *false* and fix the false ones: (a) "A `cond_signal` with no thread waiting is remembered and delivered to the next waiter." (b) "`cond_wait` keeps holding the mutex while it sleeps." (c) "A binary semaphore behaves much like a mutex." (d) "You can safely replace the `while` around `cond_wait` with an `if` if your platform has no spurious wakeups." (e) "A blocking mutex, like a signal handler flag, is a way to coordinate two flows of control over shared memory."

21.11 STRETCH A monitor bundles shared state, its mutex, and its condition variables. (a) Explain what discipline a monitor enforces that raw locks-and-signals leave to the programmer, and how that eliminates the `if`-not-`while` and lost-wakeup traps at the call site. (b) A Java object's `synchronized` methods with `wait`/ `notify`, and a thread-safe blocking queue, are both monitors — explain what each hides. (c) Argue for "prefer the highest-level correct abstraction" using the fact that the primitives are where the subtle bugs live, and give one case where you would still drop to raw condition variables.

21.12 LAB On your own machine: (a) write a waiter that blocks ~0.5 s two ways — busy-polling a flag under a lock, and blocking on a condition variable — and measure CPU time with `getrusage`; confirm the spinner burns ~100% of a core and the blocker ~0. (b) Build a producer–consumer bounded buffer with a mutex and two condition variables (using `while` loops); confirm produced and consumed totals match and the buffer never exceeds its capacity. (c) Run ten threads against a semaphore initialized to three and confirm at most three are ever in the section at once. Label your compiler and kernel version, and note the CPU numbers are illustrative.

Chapter 22 — Atomics and Compare-and-Swap: Building Blocks Below the Lock

Before you start: Chapter 20 (a lock gives mutual exclusion — but what is a lock *built from*?), Chapter 21 (blocking primitives coordinate threads; all of them rest on an indivisible hardware operation), Chapter 15 (threads share memory, so an unsynchronized read-modify-write races), Chapter 3 (a core sees memory through its cache; "atomic" is enforced at the cache line). · The last two chapters used locks and condition variables as given; this one opens the box and shows the hardware primitive — the atomic read-modify-write — that every one of them is built on.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 15 — why is `counter++` across two threads a race, and what are the three machine steps hiding inside that one line of C? (2) Chapter 20 — a mutex `lock()` has to test-and-set some word indivisibly; what would go wrong if the test and the set were two separate instructions? (3) *Predict:* four threads each add 1 to a shared counter one million times. With a plain `count++`, what final value do you expect (the ideal is 4,000,000)? With an atomic increment? Write both guesses; we measure them in §22.1.

A lock is not a primitive of the machine; it is a *program*, and like every program it needs some smaller indivisible operation to stand on. That operation is the **atomic read-modify-write** (RMW): a single hardware instruction that reads a memory location, computes a new value, and writes it back with no other core able to observe or interpose a write in the middle. This chapter builds up from the problem that motivates it. First, why an ordinary `x++` on shared memory silently loses updates — it is three steps, not one, and threads interleave inside it (§22.1). Then the atomic RMW that fixes it, and how the hardware actually provides indivisibility — the x86 `LOCK` prefix and the cache-line lock beneath it (§22.2). Then the one RMW that is more than a fixed arithmetic operation, the **compare-and-swap** (CAS), and the retry loop that turns it into any update you like (§22.3). Finally, why CAS in particular is the *universal* primitive — Herlihy's consensus hierarchy — and what atomics cost, since they are far from free (§22.4). We close by carrying CAS toward the lock-free data structures it makes possible (§22.5). Locks were the abstraction; atomics are the mechanism underneath.

22.1 The lost update: `x++` is three steps, not one

A single line, `count++`, compiles to a **read-modify-write**: load `count` into a register, add one, store it back. Those are three separate instructions, and a thread can be preempted — or simply run in parallel on another core — between any two of them. When two threads read the same old value before either has stored, both compute the same "new" value and both store it: two increments, one net effect. The update is *lost*. It is not a rare corner case; under contention it is the common case. Here are four threads each incrementing a shared `long` one million times, compiled at `-O0` so each iteration is a genuine load-add-store (gcc 11.4.0, Linux 6.18, x86-64):

```
expected         = 4000000
plain  x++       = 1355281   (lost 2644719 updates)
atomic fetch_add = 4000000
```

The plain counter lost roughly two-thirds of its four million increments — and the exact figure changes every run, because it is a function of how the threads happened to interleave, which is precisely the hallmark of a data race. The atomic version, using a single indivisible increment, landed on 4,000,000 exactly, every run. (At `-O2` the plain loop can appear to "work": the optimizer collapses the million iterations into one `count += 1000000`, shrinking the race window to a single load-add-store per thread that rarely collides — a reminder that a data race is undefined behavior, so a passing test proves nothing.) The lesson is blunt: **a read-modify-write on shared memory must be performed as one indivisible operation**, or updates vanish. The tool that makes it one operation is the atomic.

> **QUICK CHECK**
>
> Name the three machine steps hiding inside `count++`, and describe the interleaving of two threads that loses one increment. Why does the plain counter give a *different* wrong answer each run? (Answer after the next section.)

22.2 Atomic read-modify-write and how the hardware makes it indivisible

C11 (and C++11) added `<stdatomic.h>`: types like `atomic_long` and operations like `atomic_fetch_add`, `atomic_fetch_or`, `atomic_exchange` that the compiler lowers to a single atomic machine instruction. Replacing `count++` with `atomic_fetch_add(&count, 1)` is the whole fix in §22.1. What does "single atomic instruction" mean at the hardware level? On x86, an ordinary increment of a memory operand and an atomic one differ by one prefix byte. Compare the compiler's output for a plain post-increment versus `atomic_fetch_add` (gcc 11.4.0, `-O2`, `objdump -d`):

```
plain_add:                        atomic_add:
  mov    (%rdi),%rax                mov    $0x1,%eax
  lea    0x1(%rax),%rdx             lock xadd %rax,(%rdi)
  mov    %rdx,(%rdi)                ret
  ret
```

The plain version is exactly the three steps of §22.1: `mov` (load), `lea` (add one), `mov` (store) — three instructions, interruptible between any pair. The atomic version is *one* instruction, `lock xadd` ("exchange-and-add"), which loads, adds, and stores as a single indivisible unit. The magic is the `LOCK` prefix. It tells the CPU to hold exclusive ownership of the affected cache line for the duration of the read-modify-write, so no other core can read or write that line in between — historically by asserting a bus lock, and on modern chips by the cache-coherence protocol (Chapter 3) refusing to yield the line while the locked operation completes. The result is that the whole load-add-store is atomic with respect to every other core. Note the cost this implies, which §22.4 measures: the core must own the line exclusively, so a `lock`-prefixed op serializes at the cache line and cannot be reordered around freely — it is much dearer than a plain store.

THE SAME IDEA THREE WAYS: AN ATOMIC RMW IS ONE INDIVISIBLE READ-MODIFY-WRITE

1. In words. A plain `x++` is load, add, store — three instructions another core can interleave into, losing updates. An atomic RMW (`lock xadd`) does the same read-modify-write as a single instruction that owns the cache line throughout, so no other core can observe or interpose a write in the middle.

2. As a picture. Figure 22.1: two threads each load the same old value 41, each add one, each store 42 — one increment lost — versus the atomic path where the second thread cannot touch the line until the first's read-modify-write retires, so it reads 42 and stores 43.

3. As numbers. Four threads × one million: plain ++ lost 2,644,719 of 4,000,000 (and a different count each run); atomic `fetch_add` lost none — exactly 4,000,000, every run. In the disassembly, three instructions collapse to one `lock xadd`.

(Answer to the §22.1 quick check: the three steps are load `count` into a register, add one, store it back. If thread A loads 41 and, before it stores, thread B also loads 41, both compute 42 and both store 42 — two increments, one net change, so one is lost. The wrong answer differs each run because it depends on the exact interleaving of loads and stores across cores, which the scheduler and hardware vary run to run — the signature of a data race.)

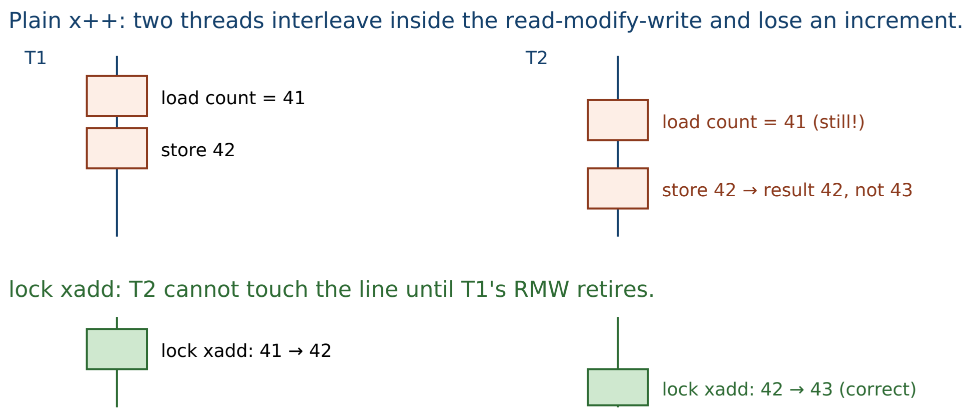


Figure 22.1: The lost update. With a plain `x++`, T2 reads the counter before T1 has stored, so both write 42 and one increment is lost. With an atomic `lock xadd`, T2's read-modify-write cannot begin until T1's has completed, because the `LOCK` prefix holds the cache line exclusively for the whole operation.

22.3 Compare-and-swap: the conditional RMW and the retry loop

`fetch_add` and its siblings each hard-wire one operation (add, or, exchange). But most interesting updates are not a fixed arithmetic function of the old value — "install this new head only if the head is still what I read," "raise the maximum only if my value is larger." The primitive that expresses *any* such conditional update is **compare-and-swap**. In C11 it is `atomic_compare_exchange_weak(obj, &expected, desired)`: it atomically compares the value in `obj` with `expected`, and *only if they are equal* writes `desired` and returns true; otherwise it writes the current value back into `expected` and returns false. In one indivisible step it asks "is it still what I think?" and, if so, updates it. That conditional is what lets you build a lock-free update out of a plain read, an arbitrary computation, and a CAS in a **retry loop**: read the current value, compute the new one, attempt to CAS it in; if the CAS fails (someone else changed it first), the failure has already reloaded the current value, so loop and try again. Here is an atomic *maximum* — an operation with no dedicated instruction — built from a CAS loop, run by eight threads (gcc 11.4.0, Linux 6.18):

```
void atomic_max(atomic_long *p, long v) {
    long cur = atomic_load(p);
    while (v > cur)                                    /* only if we'd raise it */
        if (atomic_compare_exchange_weak(p, &cur, v))   /* cur is reloaded on failure */
            return;                                      /* installed */
}
/* 8 threads hammering atomic_max: */
max seen = 7499  (true max = 7499)
```

Eight threads racing to raise a shared maximum agreed exactly on the true maximum, 7499, with no lock. The structure is the template for essentially every lock-free algorithm in the next chapter: **read a snapshot, compute off it, and commit with a CAS that succeeds only if nothing changed underneath you** — retrying if it did. Two details matter. The `expected` argument is updated by a failed CAS to the value it actually found, so the loop always retries against fresh state without a separate reload. And the `_weak` form is allowed to fail *spuriously* (report mismatch even when the values were equal) in exchange for cheaper code on architectures that implement CAS with load-linked/store-conditional; because it already lives in a retry loop, a spurious failure just means one more iteration, so `_weak` is the right choice inside a loop and `_strong` (no spurious failures) for a one-shot check.

QUICK CHECK

State exactly what `compare_exchange` does on success and on failure, including what happens to the `expected` argument when it fails. Why does the CAS-loop idiom not need a separate reload of the current value on retry? When would you pick `_weak` over `_strong`? (Answer below the communicate box.)

22.4 Why CAS is universal — and what atomics cost

Why single out compare-and-swap rather than, say, atomic add? Because CAS is *universal* in a precise, proven sense. Herlihy's "Wait-Free Synchronization" (1991) ranks synchronization primitives by their **consensus number**: the maximum number of threads that can use the primitive (plus plain reads and writes) to solve *consensus* — all agreeing irrevocably on one value despite arbitrary delays. Atomic reads and writes alone have consensus number **1**: they cannot solve consensus for even two threads. Atomic `fetch_add`, `test-and-set`, and their kin sit higher but still at a finite number. Compare-and-swap has consensus number **infinity**: with CAS, any number of threads can reach consensus, and from it you can build a wait-free implementation of *any* object with a sequential specification. That is why hardware designers provide CAS (or load-linked/

store-conditional, its equivalent) and why every lock-free structure in Chapter 23 is a CAS loop: CAS is the primitive from which all the others, and all lock-free objects, can be constructed. Plain loads and stores cannot substitute — no arrangement of them solves two-thread consensus.

Universality does not mean free. An atomic RMW must own its cache line exclusively and cannot be reordered around like a plain access, so it costs far more than an ordinary operation — and under contention, more still, because the cache line ping-pongs between cores (Chapter 3's coherence traffic). Measured uncontended, one operation each, 100 million times (gcc 11.4.0, `-O2`):

```
plain  ++          :  0.9 ns/op
atomic fetch_add   : 15.3 ns/op        (~15× a plain op)
mutex lock/unlock  : 11.9 ns/op        (itself two atomic ops)
```

A plain increment is under a nanosecond; the atomic is an order of magnitude dearer, and a full mutex lock/unlock — which is itself built from atomics — is in the same league (a lock is not magically slower than an atomic; it *is* atomics plus a slow path for when it must block). These are illustrative single-machine numbers, but the shape is universal: atomics are cheap enough to use freely for a shared counter and expensive enough that a hot, highly contended atomic can dominate a profile. The other cost is *correctness*: atomic loads and stores are individually indivisible, but they do **not compose** into an atomic compound. Two separate atomic operations have a gap between them, and the whole point of §22.1 was that gaps lose updates — which is exactly the trap that sends you to CAS.

TRAP: COMPOSING ATOMIC OPERATIONS (AN ATOMIC LOAD THEN AN ATOMIC STORE IS NOT ATOMIC)

Because `atomic_load` and `atomic_store` are each atomic, it is tempting to write a "conditional update" as two of them: `if (atomic_load(&p) == expected) atomic_store(&p, desired);`. Each call is indivisible, but *together* they are not: between the load and the store, another thread can change `p`, and your store clobbers its update — the lost-update bug of §22.1 wearing atomic clothing. This is a check-then-act race, and it is the reason compare-and-swap exists: CAS folds the compare **and** the swap into one indivisible operation, so nothing can slip between them. Any time you find yourself reading an atomic, deciding based on the value, and writing back a value that depends on that decision, you need a single CAS (usually in a retry loop), not a load followed by a store. The tell is a sentence of the form "if it's still X, make it Y" spanning two atomic calls — that gap is the bug.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I don't need a mutex for this flag. I'll just do `if (atomic_load(&busy) == 0) atomic_store(&busy, 1);` to claim it, and `atomic_store(&busy, 0)` to release. Both are atomic, so it's a lock-free lock — right?" Cover the paragraph and respond in four or five sentences, drawing on §22.3 and §22.4.

Model answer. "The two operations are each atomic, but the sequence isn't, and that gap is the whole problem: two threads can both load `busy == 0`, both decide it's free, and both store 1 — you've handed the 'lock' to two threads at once, which is the lost-update race from the counter example. To claim the flag indivisibly you need a single *compare-and-swap*: `expected = 0; atomic_compare_exchange_strong(&busy, &expected, 1)` succeeds for exactly one thread and fails for the other, because the compare and the set happen as one operation with no window between them. That is, in fact, roughly how a spinlock's fast path is built. But before you build your own, use a real mutex or a tested lock-free type: hand-rolled synchronization is where the subtle, load-dependent bugs live, and the atomic `compare_exchange` is the sharp tool you reach for only when you must." Naming the check-then-act gap and reaching for CAS — not two atomic accesses — is the senior register.

(Answer to the §22.3 quick check: on success, `compare_exchange` finds `*obj == expected`, writes `desired`, and returns true; on failure, it leaves `*obj` unchanged, writes the current value of `*obj` into `expected`, and returns false. The loop needs no separate reload because a failed CAS has already refreshed `expected` with what it found. Prefer `_weak` inside a retry loop, where a rare spurious failure just costs one more iteration and the code is cheaper on load-linked/store-conditional machines; prefer `_strong` for a single, non-looping check.)

22.5 What to carry forward

Beneath every lock and condition variable is an **atomic read-modify-write**: a single hardware instruction that reads, computes, and writes with no other core able to interpose. An ordinary `x++` is not atomic — it is load, add, store, and threads interleave inside it, losing updates (our four-threaded counter lost 2.6 million of 4 million; the atomic lost none). The hardware provides indivisibility through a locked operation — on x86 the `LOCK` prefix, which holds the cache line exclusively for the whole read-modify-write (`lock xadd` is one instruction where the plain version is three). The one RMW that expresses *arbitrary* conditional updates is **compare-and-swap**: "write this only if the value is still what I read," folded into one indivisible step, and wrapped in a **retry loop** — read, compute, CAS, retry on failure — it implements any update lock-free (our eight-threaded atomic maximum agreed on 7499). CAS is not merely convenient but *universal*: Herlihy's consensus hierarchy places plain reads/writes at consensus number 1 and CAS at infinity, so any lock-free object can be built from it. Atomics are far from free (roughly fifteen times a plain op, more under contention), and — the trap — they do not compose: an atomic load then an atomic store leaves a gap another thread slips through, which is precisely why CAS exists.

With compare-and-swap in hand, a new possibility opens: synchronization *without locks at all*. If a single CAS can commit an update only when nothing changed underneath it, then a data structure can let many threads race, each preparing an update and committing it with a CAS, retrying the few that lose — no thread ever blocked, no lock ever held. The next chapter builds exactly these **lock-free data structures**: the Treiber stack and the Michael–Scott queue, both nothing but a CAS loop over a shared pointer. It also confronts what CAS alone does not solve — the notorious *ABA problem*, where a value returns to what you read while its meaning has changed underneath, defeating the compare — and how real systems reclaim memory safely without locks. And underneath all of it sits an assumption we have quietly leaned on and must finally make explicit: that these atomic operations become visible to other cores in a sensible order. The chapter after next, on the memory model, shows that plain loads and stores carry no such guarantee, and that the ordering a lock or a lock-free structure needs is something the atomic must be asked for by name.

CAN YOU… WITHOUT LOOKING?

- Explain why `x++` on shared memory loses updates, naming the three steps and the racing interleaving.
- Say what the x86 `LOCK` prefix does, and why `lock xadd` is atomic where three plain instructions are not.
- State precisely what `compare_exchange` does on success and failure, and write the CAS retry-loop idiom.
- Explain what "CAS is universal" means via consensus numbers (plain reads/writes = 1, CAS = infinity).
- Give the rough cost of an atomic RMW versus a plain op, and why contention makes it worse.
- State why two atomic operations do not compose, and what to use instead for a conditional update.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why is `count++` across threads a race, and what does an atomic RMW change? (§22.1)
2. **R2.** What does the `LOCK` prefix do at the cache-line level, and how many instructions is `lock xadd` versus a plain increment? (§22.2)
3. **R3.** What does `compare_exchange` return and write on success and on failure, and what is the CAS-loop idiom? (§22.3)
4. **R4.** What is a consensus number, and what are the consensus numbers of plain reads/writes and of CAS? (§22.4)
5. **R5.** Why do two atomic operations fail to compose into an atomic compound, and what fixes it? (§22.4)

FURTHER READING

- **Herlihy, "Wait-Free Synchronization," *ACM Transactions on Programming Languages and Systems* 13(1):124–149 (1991), DOI 10.1145/114005.102808.** The consensus hierarchy: plain reads/writes have consensus number 1, compare-and-swap has infinity, and CAS is universal for wait-free constructions. The source for §22.4; author, venue, and year verified via dblp and the ACM Digital Library.
- **Herlihy & Shavit, *The Art of Multiprocessor Programming* (Morgan Kaufmann; 2nd ed. 2020).** The standard text on atomics, the consensus hierarchy, and lock-free programming, with the primitives of this chapter developed rigorously. `[⚠ reference — not free]`.
- **cppreference, `<stdatomic.h>` / `std::atomic` and `atomic_compare_exchange` (`en.cppreference.com`).** The precise contract for `fetch_add`, the `_weak` vs `_strong` compare-exchange, and spurious failure. Primary reference for §22.2–§22.3; verify per standard version.
- **Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 3A, "Locked Atomic Operations" (the `LOCK` prefix and `CMPXCHG`/`XADD`).** Primary source for how x86 makes a read-modify-write atomic at the cache line. Free from Intel.
- **Drepper, "What Every Programmer Should Know About Memory" (Red Hat, 2007).** Free (`akkadia.org/drepper/cpumemory.pdf`). Cache coherence and why a `lock`-prefixed operation costs what it does — background for the numbers in §22.4.

Exercises

22.1 WARM-UP Name the three machine steps that `count++` compiles to, and in one sentence say why that makes it non-atomic across threads.

22.2 WARM-UP What does the x86 `LOCK` prefix guarantee about the cache line during a read-modify-write, and how many instructions is `lock xadd`?

22.3 WARM-UP In one sentence each: what does `atomic_compare_exchange` return on success, and what does it write into its `expected` argument on failure?

22.4 CORE Two threads run `count++` on a shared counter starting at 0, each once. (a) Enumerate an interleaving of the six machine steps (three per thread) that ends with `count == 1` instead of 2. (b) Explain why replacing the body with `atomic_fetch_add(&count, 1)` makes that interleaving impossible. (c) Why can the same program appear correct at `-O2` yet be lost-update-prone at `-O0`?

22.5 CORE Write the compare-and-swap retry loop for an atomic operation with no dedicated instruction — say, atomically setting a counter to the larger of its current value and `v` (an atomic max). Explain what the `expected` argument holds after a failed CAS and why the loop therefore needs no explicit reload, and state one reason to use `_weak` rather than `_strong` inside the loop.

22.6 CORE A colleague implements "claim a resource" as `if (atomic_load(&owner) == 0) atomic_store(&owner, my_id);`. (a) Show the interleaving that lets two threads both believe they own the resource. (b) Rewrite it with a single compare-and-swap so exactly one thread succeeds. (c) Explain the general rule: which shape of update always requires CAS rather than a separate atomic load and store?

22.7 COST Using the measured figures (plain `++` ~0.9 ns, atomic `fetch_add` ~15 ns, uncontended): (a) explain where the ~15× comes from in terms of the cache line and instruction reordering. (b) Predict qualitatively what happens to the atomic's cost when eight cores hammer the *same* counter, and why (invoke Chapter 3's coherence traffic). (c) Given that, when is a per-thread counter summed at the end preferable to one shared atomic, and what does that cost instead?

22.8 FADED *Worked, with the last step for you.* You must maintain a running maximum latency observed across many worker threads, lock-free.
Step 1 (given): A plain `if (v > hi) hi = v;` is a check-then-act race — two threads can both pass the test and the smaller write can land last.
Step 2 (given): There is no `atomic_fetch_max` in C11, so you build it from `compare_exchange`.
Step 3 (given): The loop reads `cur = atomic_load(&hi)`, and while `v > cur` tries `compare_exchange_weak(&hi, &cur, v)`; a failed CAS reloads `cur`.
Step 4 (you): Write the complete function, explain why it terminates (each failed retry means someone else raised `hi`, so `cur` only grows), and state what invariant it guarantees about `hi` versus every `v` ever submitted.

22.9 MIXED For each need, choose the cheapest correct tool — a plain access, an atomic `fetch_*`, a single CAS, or a CAS retry loop — and justify in a line: (a) increment a shared hit counter; (b) set a flag to 1 exactly once and detect who did it first; (c) push onto a shared linked-list stack; (d) read a value no other thread ever writes.

22.10 MIXED Drawing on Chapters 20–22, classify each as *true* or *false* and fix the false ones: (a) "A mutex is a fundamental hardware primitive, unlike an atomic add." (b) "`lock xadd` and a plain `add` to memory differ by a one-byte prefix." (c) "Because each is atomic, an `atomic_load` followed by an `atomic_store` forms an atomic compound operation." (d) "Compare-and-swap can solve consensus for any number of threads, but plain reads and writes cannot solve it for two." (e) "An uncontended atomic increment costs about the same as a plain increment."

22.11 STRETCH Herlihy's consensus hierarchy ranks primitives by consensus number. (a) Explain what it means that atomic read/write has consensus number 1, and give the intuition for why two threads cannot reach consensus with reads and writes alone. (b) Explain what "CAS has consensus number infinity, and is universal" buys you — what class of objects can you build from CAS? (c) Connect this to the previous two chapters: in what sense is every lock and every lock-free structure "just" CAS (or an equivalent primitive), and why does the hardware bother to provide CAS specifically?

22.12 LAB On your own machine: (a) write four threads each doing one million `count++` on a shared `long`, compile at `-O0`, and confirm the total falls short of 4,000,000 and varies run to run; switch to `atomic_fetch_add` and confirm it is exactly 4,000,000 every run; then recompile the plain version at `-O2` and observe it may now "pass" (explain why). (b) Disassemble a plain memory increment and an `atomic_fetch_add` with `objdump -d` and find the `lock` prefix. (c) Build an atomic max from a `compare_exchange_weak` loop, run it under many threads, and confirm it agrees with the true maximum. Label your compiler and CPU; note the timing figures are illustrative.

Chapter 23 — Lock-Free Data Structures

Before you start: Chapter 22 (compare-and-swap and the retry loop — the one primitive this whole chapter is built from), Chapter 20 (a lock serializes access; what if a delayed lock-holder must not stall everyone else?), Chapter 7 (pointers and nodes — a stack and a queue are pointer surgery), Chapter 9 (free and use-after-free — reclaiming a node another thread still holds is a disaster). · Chapter 22 gave you CAS; this chapter uses nothing but CAS in a loop to build shared stacks and queues that no thread ever locks.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 22 — write the compare-and-swap retry-loop idiom in three steps (read, compute, commit-or-retry), and say what a failed CAS does to your local copy of the value. (2) Chapter 20 — name one thing that goes wrong when the thread holding a lock is preempted, delayed, or crashes. (3) *Predict:* eight threads push and pop on a *shared* stack with no lock — just a CAS on the head pointer. Do you expect any pushed item to be lost or duplicated? And if a node can be freed and its address immediately reused, what could break? Write your guesses; §23.2 and §23.4 settle them.

A lock makes concurrency correct by making it *sequential*: one thread at a time inside the critical section. That is simple and often right, but it has a structural weakness — the fate of every thread is tied to the one holding the lock. If that thread is preempted (Chapter 16), page-faults (Chapter 17), or is simply slow, everyone waiting is stuck, and if it dies holding the lock, they are stuck forever. **Lock-free** programming removes that coupling: many threads operate on the shared structure at once, each committing its change with a single compare-and-swap that succeeds only if nothing moved underneath it, and retrying if it did. No thread ever holds a lock, so no thread's delay can stall the others. This chapter builds the idea concretely. First, what "lock-free" actually guarantees — the ladder of progress conditions from blocking up to wait-free, and why "lock-free" is a promise about the *system*, not the absence of locks (§23.1). Then the two canonical structures, each a CAS loop over a pointer: the **Treiber stack** (§23.2) and the **Michael-Scott queue** (§23.3). Then the hazard that makes lock-free memory management genuinely hard — the **ABA problem** and safe reclamation (§23.4). We close by carrying the open question of *ordering* — which lock-free code depends on and which we have not yet justified — into the memory model (§23.5).

23.1 What "lock-free" means: the ladder of progress

"Lock-free" is a precise progress guarantee, not a synonym for "uses no mutex." The guarantees form a ladder. A **blocking** (lock-based) algorithm offers no non-blocking guarantee at all: a thread delayed inside the critical section blocks every other thread indefinitely. **Obstruction-free**, the weakest non-blocking level, promises that a thread will finish if it runs *in isolation* for long enough (no guarantee under contention). **Lock-free** promises that the *system as a whole* always makes progress: out of all contending threads, *some* thread completes its operation in a bounded number of steps, so the structure can never be globally stuck — though an unlucky individual thread might retry forever (starve) while others succeed. **Wait-free**, the strongest, promises that *every* thread completes in a bounded number of *its own* steps, with no starvation. These are Herlihy's conditions, and they trade strength for cost: wait-free is ideal but often intricate and slower, so most practical "lock-free" structures — including the two in this chapter — give the lock-free guarantee, which is enough to eliminate the lock-holder problem.

Why does the guarantee matter in practice? Because a lock-free structure is immune to the pathologies that come from a thread being stopped at the wrong moment. There is no lock to hold, so there is no **deadlock** and no **priority inversion** (a low-priority thread cannot block a high-priority one by holding a lock, Chapter 16); a thread preempted mid-operation delays only itself, never the structure; and a thread that dies leaves the structure usable by everyone else. The price is subtlety: every operation must be designed so that its single committing CAS is a **linearization point** — the instant the operation appears to take effect atomically, in Herlihy and Wing's sense (1990), so that concurrent operations still look like *some* valid sequential order. Get the linearization point right and the structure is correct under any interleaving; get it wrong and you have a race that no test reliably catches.

> **QUICK CHECK**
>
> Put the four progress conditions in order from weakest to strongest, and state the difference between the lock-free and wait-free guarantees (whose progress is promised?). Name one pathology that lock-free code cannot suffer, and why. (Answer after the next section.)

23.2 The Treiber stack: a CAS loop on the head pointer

The simplest lock-free structure is a stack, published by Treiber (IBM, 1986). It is a singly linked list with an atomic `top` pointer, and both operations are the CAS retry loop of Chapter 22 applied to that pointer. To **push**: set the new node's `next` to the current `top`, then CAS `top` from that old value to the new node; if the CAS fails (someone else pushed or popped first), the failure reloaded `top`, so re-point `next` and retry. To **pop**: read `top`, and CAS it to `top->next`; retry if it moved. That is the whole algorithm — no lock, no blocking. Eight threads pushing 100,000 items each and then popping every node concurrently agree exactly (gcc 11.4.0, Linux 6.18, x86-64):

```
void push(long v) {
    node *n = malloc(sizeof *n); n->val = v;
    node *old = atomic_load(&top);
    do { n->next = old; }
    while (!atomic_compare_exchange_weak(&top, &old, n));   /* old reloaded on failure */
}
node *pop(void) {
    node *old = atomic_load(&top);
    while (old) { if (atomic_compare_exchange_weak(&top, &old, old->next)) return old; }
    return NULL;
}
/* 8 threads push 100000 each, then 8 threads pop concurrently: */
pushed total = 800000
popped count = 800000, popped sum = 800000   (no node lost or duplicated)
```

Every one of the 800,000 nodes was popped exactly once — none lost, none returned twice — under eight-way contention with no lock. The linearization point is the successful CAS: a push "happens" the instant its CAS installs the new top, a pop the instant its CAS unlinks the old top, and any two concurrent operations therefore order by which CAS won. Under contention the losers simply retry, which is why the guarantee is lock-*free* (the system always advances — some CAS always wins) and not wait-free (a persistently unlucky thread could keep losing). One honest caveat, deferred to §23.4: this code *leaks* — it never frees popped nodes. Freeing them safely while other threads may still be reading them is the hard part of lock-free programming, and it is where the ABA problem lives.

THE SAME IDEA THREE WAYS: PUSH IS A CAS LOOP THAT INSTALLS A NEW HEAD

1. In words. Point the new node at the current top, then compare-and-swap the top from that old value to the new node. If another thread changed the top first, the CAS fails and hands you the new top; re-point and try again.

2. As a picture. Figure 23.1: thread A reads `top = X`, sets `new->next = X`; thread B pushes first, so `top` is now Y; A's CAS(`top`, X, new) fails (top is Y, not X), reloads `old = Y`, re-points `new->next = Y`, and its second CAS succeeds.

3. As numbers. Eight threads × 100,000 pushes then concurrent pops: 800,000 pushed, 800,000 popped, sum 800,000 — no node lost or duplicated. Under an 8-thread push/pop benchmark the lock-free stack ran at ~176 ns/op against a mutex-guarded stack's ~359 ns/op (illustrative, contention-dependent).

(Answer to the §23.1 quick check: weakest to strongest — blocking, obstruction-free, lock-free, wait-free. Lock-free guarantees that the ***system*** *makes progress (some thread always completes), allowing an individual thread to starve; wait-free guarantees that* ***every*** *thread completes in a bounded number of its own steps. Lock-free code cannot deadlock or suffer priority inversion, because there is no lock to hold — a delayed thread stalls only itself, never the structure.)*

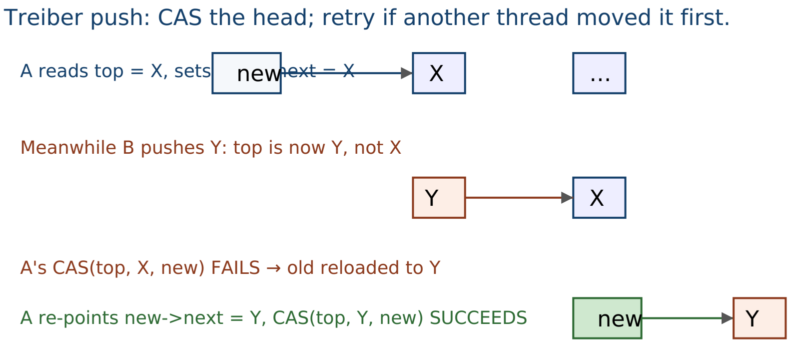


Figure 23.1: The Treiber stack's push. Thread A prepares a node pointing at the top it read (X), but thread B pushes Y first, so A's compare-and-swap fails — and the failure reloads the current top (Y) into A's local variable. A re-points its node at Y and retries; the second CAS succeeds. The successful CAS is the push's linearization point.

23.3 The Michael–Scott queue: two pointers, and helping

A FIFO queue is harder than a stack because two ends change independently: enqueue appends at the `Tail`, dequeue removes at the `Head`. The classic lock-free solution is the **Michael–Scott queue** (PODC 1996), the algorithm behind Java's `ConcurrentLinkedQueue` and countless runtimes. It keeps a permanent **dummy node** so the list is never empty (Head and Tail start pointing at it), which removes the awkward special cases at the boundaries. Enqueue is a two-step CAS: first CAS the current tail node's `next` from `NULL` to the new node (this is the linearization point — the item is now in the queue); then CAS `Tail` forward to the new node. Because those two CASes are not one atomic step, another thread can arrive between them and find `Tail` "lagging" — pointing at a node whose `next` is no longer `NULL`. The elegant fix is **helping**: any thread that observes a lagging tail advances it (CAS `Tail` to `tail->next`) before proceeding, so no operation ever waits for the thread that fell behind. Four producers and four consumers hammering the queue agree exactly (gcc 11.4.0, Linux 6.18):

```
enqueue: link new node after tail (CAS tail->next: NULL -> n)  // linearization point
         then swing Tail forward   (CAS Tail: tail -> n)       // may be done by a helper
dequeue: read value from Head->next, then CAS Head forward to it

4 producers x 200000, 4 consumers:
expected  count=800000 sum=1279999600000
dequeued  count=800000 sum=1279999600000   OK
```

All 800,000 items came out exactly once and the checksums matched, with no lock anywhere. The two lessons generalize beyond the queue. First, when a logical update spans more than one atomic step, you cannot hold a lock across them — so you design the structure to be *correct in every intermediate state* and let any thread **help** complete a half-finished operation it stumbles upon; this "help the laggard" pattern recurs throughout lock-free design. Second, the linearization point is a single, identifiable CAS (here, linking the new node), which is what lets you reason that the concurrent execution is equivalent to some sequential one. As with the stack, this code as written leaks nodes; making dequeue free the node it removed — safely, when no other thread is still looking at it — is the reclamation problem of the next section.

> **QUICK CHECK**
>
> Why does the Michael–Scott queue keep a permanent dummy node? Enqueue takes two CAS steps — what can another thread observe between them, and what does "helping" do about it? Which single step is the enqueue's linearization point? (Answer below the communicate box.)

23.4 The ABA problem and safe memory reclamation

Everything so far leaked memory on purpose, because reclaiming it exposes the subtlest bug in lock-free programming. A CAS checks whether a pointer *still equals* the value you read — but "equal" means the same bits, not the same object. If between your read and your CAS a value changes from **A** to **B** and back to **A**, your CAS sees `A`, concludes "nothing changed," and succeeds — even though the world moved on underneath it. This is the **ABA problem**. It bites the Treiber stack precisely when nodes are freed and reused: a thread reads `top = A` and plans to CAS `top` to `A->next`; meanwhile other threads pop `A` and `B`, free `A`, and a fresh `malloc` hands back `A`'s address for a new node that is pushed, so `top` is `A` again — but `A->next` now points somewhere entirely different (or into freed memory). The stalled thread's CAS succeeds on the recycled address and installs a stale, dangling `next`, corrupting the stack. ABA is intermittent by nature — it needs a specific interleaving plus address reuse — which is exactly what makes it dangerous: it survives testing and strikes in production.

There are three standard defenses, and every real lock-free system uses one. A **tagged (versioned) pointer** pairs the pointer with a counter that is incremented on every update, and CASes the pair; A-to-B-to-A now differs in the counter, so the CAS correctly fails. This needs a double-width CAS — on x86-64 the `lock cmpxchg16b` instruction (compile with `-mcx16`) — and the versioned Treiber stack passes the same 800,000-node test the plain one did. **Hazard pointers** (Michael, 2004) take a different tack: before dereferencing a node, a thread publishes it in a per-thread "hazard" slot; a thread that wants to free a node first checks that no hazard pointer references it, deferring reclamation until it is unreferenced — so a node in use is never freed, and ABA-via-reuse cannot happen. **Epoch-based reclamation** and **RCU** (read-copy-update) generalize this: readers announce they are active, and a retired node is freed only after every reader that could have seen it has departed. All three exist for one reason — a lock-free structure cannot `free` a node the moment it is unlinked, because another thread may still be about to read it, and freeing it is a use-after-free (Chapter 9) waiting to happen.

TRAP: THE ABA PROBLEM (A CAS SEES THE SAME VALUE, NOT THE SAME WORLD)

A compare-and-swap only checks that a location still holds the *bits* you read, so a value that goes A → B → A slips through: your CAS succeeds as if nothing happened, though the object those bits refer to may have been popped, freed, and replaced. In the Treiber stack this corrupts the list — the CAS installs a `next` pointer read from a node that has since been recycled, silently losing or duplicating items, or dangling into freed memory. It is intermittent (it needs a preemption at the wrong instant plus address reuse), so it passes tests and fails under production load. The tell is any lock-free code that frees nodes and reuses their memory while a CAS elsewhere depends on a pointer's identity. The fixes are the three of §23.4: a tagged pointer (version the value so A-to-A differs), hazard pointers, or epoch/RCU reclamation (never free a node another thread might still hold). If you are writing a lock-free structure that frees nodes and you have not chosen one of these, you have an ABA bug you simply have not hit yet.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I wrote a lock-free stack — push and pop are just a CAS on the head, straight out of the textbook. In the unit test it's perfect. But under load it occasionally segfaults or loses an item, and I can't reproduce it in the debugger. The CAS is obviously correct — it only swaps the head if it hasn't changed. What's going on?" Cover the paragraph and respond in four or five sentences, drawing on §23.2 and §23.4.

Model answer. "The CAS logic is right; the bug is memory reclamation, and the symptom is the ABA problem. If pop frees the node it removed, that address can be handed back by `malloc` and pushed again while another thread is mid-pop holding the old head value — its CAS sees the same pointer bits, concludes nothing changed, and succeeds on a node whose `next` now points into recycled or freed memory, which is your segfault-or-lost-item. It's intermittent because it needs a preemption at exactly the wrong moment plus address reuse, so it never shows up in a single-stepping debugger. The fix isn't the CAS — it's safe reclamation: a tagged/versioned pointer with a double-width CAS so A-to-A differs by version, or hazard pointers, or epoch-based reclamation so you never free a node another thread might still be reading. Honestly, unless we need this specific structure, I'd use a vetted concurrent queue from the standard library — lock-free reclamation is exactly where hand-rolled code goes wrong." Naming ABA, tying it to reclamation rather than the CAS, and reaching for a vetted implementation is the senior register.

(Answer to the §23.3 quick check: the dummy node keeps the list non-empty so Head and Tail always point at a real node, eliminating null-boundary special cases. Between enqueue's two CASes another thread can see `Tail` lagging — pointing at a node whose `next` is no longer `NULL`; "helping" means that thread advances `Tail` itself (CAS it to `tail->next`) rather than waiting. The linearization point is the first CAS, linking the new node after the tail — that is the instant the item is in the queue.)

23.5 What to carry forward

Lock-free programming trades the simplicity of a lock for immunity to the lock-holder's fate: many threads operate at once, each committing with a single compare-and-swap that wins only if nothing moved underneath, retrying if it lost, so no thread ever blocks another. "Lock-free" is a precise guarantee — the *system* always makes progress (some thread completes), which is weaker than **wait-free** (every thread completes) but strong enough to rule out deadlock and priority inversion. The two canonical structures are each a CAS loop over a pointer: the **Treiber stack** (CAS the head; our eight-threaded run pushed and popped 800,000 nodes with none lost) and the **Michael–Scott queue** (a dummy node, a two-step enqueue, and *helping* to advance a lagging tail; four producers and four consumers moved 800,000 items exactly). Each operation has a single CAS as its **linearization point**, which is what makes the concurrent execution equivalent to a sequential one. The genuinely hard part is memory: a CAS checks bits, not identity, so a value that cycles A–B–A defeats it — the **ABA problem** — and you cannot free an unlinked node while another thread may still read it. The three defenses are tagged pointers (double-width CAS), hazard pointers, and epoch/RCU reclamation.

Every algorithm in this chapter quietly assumed something we have not earned: that when one thread's CAS publishes a node, another thread that reads the updated pointer also sees the node's *contents* — its `val` and `next` — fully written. On the abstract machine that is obvious; on real multicore hardware it is not. Cores have store buffers and caches, and both the compiler and the processor reorder memory operations, so a thread can observe a new pointer while the writes that initialized what it points to are still in flight — a reader following a freshly published pointer into *garbage*. What saves the code is that the atomic operations carry **ordering** guarantees, and those guarantees are not automatic: they must be requested, by name, through the atomic's memory-ordering argument. The final chapter of this part makes that machinery explicit — the **memory model**: why a data race is undefined behavior, why plain loads and stores may be reordered across cores, and what `memory_order_relaxed`, `acquire/release`, and `seq_cst` actually promise. It is the specification that says precisely when the node you just published is safe for another thread to read.

CAN YOU… WITHOUT LOOKING?

- Order the four progress conditions and state what lock-free versus wait-free each guarantee.
- Write the Treiber stack's push and pop as CAS loops, and name each operation's linearization point.
- Explain the Michael–Scott queue's dummy node, two-step enqueue, and the "helping" that advances a lagging tail.
- Explain the ABA problem: why a CAS on the same bits can be wrong, and where it bites the Treiber stack.
- Name the three safe-reclamation techniques (tagged pointers, hazard pointers, epoch/RCU) and what each prevents.
- Say why lock-free code cannot deadlock or suffer priority inversion.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does the lock-free guarantee promise, and how does it differ from wait-free? (§23.1)
2. **R2.** Write the Treiber stack's push and pop, and name the linearization point of each. (§23.2)
3. **R3.** Why does the Michael–Scott queue use a dummy node and "helping"? (§23.3)
4. **R4.** What is the ABA problem, and how does it corrupt the Treiber stack when nodes are freed? (§23.4)
5. **R5.** Name the three safe-reclamation techniques and what each does. (§23.4)

FURTHER READING

- **Michael & Scott, "Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms," *Proc. 15th ACM PODC* (1996), pp. 267–275, DOI 10.1145/248052.248106.** The lock-free queue of §23.3, with the dummy node and the tail-helping mechanism. Author, venue, and year verified via dblp and the ACM Digital Library.
- **Treiber, "Systems Programming: Coping with Parallelism," IBM Almaden Research Report RJ 5118 (April 1986).** The compare-and-swap stack of §23.2 (the "Treiber stack"). Report number and date verified via the IBM Research report archive.
- **Herlihy & Wing, "Linearizability: A Correctness Condition for Concurrent Objects," *ACM TOPLAS* 12(3):463–492 (1990), DOI 10.1145/78969.78972.** The definition of the linearization point used throughout this chapter. Verified via dblp and the ACM Digital Library.
- **Michael, "Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects," *IEEE Transactions on Parallel and Distributed Systems* 15(6):491–504 (2004), DOI 10.1109/TPDS.2004.8.** The hazard-pointer technique of §23.4 and its explicit treatment of the ABA problem. Verified via the IEEE/ACM Digital Library.
- **Herlihy & Shavit, *The Art of Multiprocessor Programming* (Morgan Kaufmann; 2nd ed. 2020), chapters on concurrent stacks, queues, and memory reclamation.** The textbook development of everything here, with proofs of linearizability. `[⚠ reference — not free]`.

Exercises

23.1 WARM-UP In one sentence, what does the *lock-free* progress guarantee promise about the system as a whole?

23.2 WARM-UP In the Treiber stack, what single machine operation is the linearization point of a push? Of a pop?

23.3 WARM-UP State the ABA problem in one sentence: what does a CAS check, and why can "the same value" be misleading?

23.4 CORE Implement the Treiber stack's `push` and `pop` as CAS loops (pseudocode is fine). For each, explain (a) what a failed CAS tells you and how the loop recovers, (b) why the successful CAS is the linearization point, and (c) why the structure gives the lock-free guarantee but not the wait-free one.

23.5 CORE The Michael–Scott queue enqueues in two CAS steps. (a) Why can it not be done in one? (b) Describe what a second thread can observe about `Tail` after the first CAS but before the second, and how "helping" keeps that from stalling anyone. (c) Explain why the dummy node removes the empty-queue special cases at both ends.

23.6 CORE Walk through a concrete ABA interleaving on the Treiber stack with nodes freed and reused: thread A reads `top = A` and is about to CAS to `A->next`; describe the sequence of pops, frees, and a push that returns address `A`, and show exactly how A's CAS then corrupts the stack. Then say which of the three defenses would prevent it and how.

23.7 COST (a) Under low contention, why can a lock-free stack beat a mutex-guarded one (recall the ~176 ns vs ~359 ns/op figure), and what is the lock-free structure spending its time on instead of blocking? (b) Under *very high* contention on a single hot pointer, why might the lock-free version's throughput *collapse* — what does a storm of failing CASes cost (Chapter 3's coherence traffic)? (c) Name one structural change (e.g. elimination, or per-thread sub-stacks) that reduces contention, and what it trades away.

23.8 FADED *Worked, with the last step for you.* You must make the leaking Treiber stack free popped nodes safely.
Step 1 (given): Freeing a node the instant `pop` unlinks it is unsafe: another thread mid-pop may still hold that node as its `old` and be about to read `old->next`.
Step 2 (given): Immediate free also enables ABA: the freed address can be reused and pushed, so a stale CAS succeeds on recycled memory.
Step 3 (given): Hazard pointers have each thread publish the node it is about to dereference; a would-be reclaimer frees a retired node only if no hazard pointer references it.
Step 4 (you): Describe how a hazard-pointer-protected `pop` works step by step (publish `top` as hazardous, re-verify it is still `top`, then read `next` and CAS), and explain why this makes both the use-after-free and the ABA impossible.

23.9 MIXED For each requirement, choose between a mutex-guarded structure and a lock-free one, and justify in a line: (a) a shared work queue in a soft-real-time audio callback that must never block; (b) a rarely-contended configuration map read at startup; (c) a structure accessed from a signal handler (Chapter 18); (d) a structure whose operations are large and complex, with modest contention.

23.10 MIXED Drawing on Chapters 20–23, classify each as *true* or *false* and fix the false ones: (a) "Lock-free means the code uses no atomic instructions." (b) "A lock-free structure can deadlock if two threads retry forever." (c) "The Michael–Scott queue's dummy node is a performance optimization that can be omitted." (d) "A CAS that finds the pointer equal to what you read proves nothing was popped and freed in between." (e) "Wait-free is a strictly stronger guarantee than lock-free."

23.11 STRETCH "Helping" is a recurring lock-free pattern. (a) Explain what problem helping solves in the Michael–Scott queue and why a thread advances *another* thread's half-finished operation rather than waiting. (b) Argue how helping relates to the lock-free guarantee: why does a design where every thread can complete any pending operation avoid a stall when one thread is preempted mid-update? (c) Contrast this with a lock, where a preempted holder stalls everyone — and explain the cost helping pays for that immunity (extra CASes, more complex invariants).

23.12 LAB On your own machine: (a) implement the Treiber stack with CAS, run eight threads pushing a known number of items and then popping concurrently, and confirm no item is lost or duplicated (leak the nodes for now). (b) Benchmark it against a mutex-guarded stack under contention and report ns/op for both (note the result is workload-dependent). (c) Implement the tagged-pointer version using a double-width CAS (`-mcx16` on x86-64, linking `-latomic`), confirm it passes the same correctness test, and find the `lock cmpxchg16b` in the disassembly. Label your compiler and CPU; note the throughput figures are illustrative.

Chapter 24 — The Memory Model: memory_order and Happens-Before

Before you start: Chapter 22 (atomics — every one takes a memory-ordering argument we have so far ignored), Chapter 23 (a lock-free structure publishes a pointer and trusts the reader to see what it points to — why is that safe?), Chapter 10 (undefined behavior and the optimizing compiler — a data race is UB, and the compiler exploits it), Chapter 3 (each core sees memory through its own cache and store buffer). · The last two chapters assumed memory operations become visible in a sensible order across cores; this chapter shows they do not, and specifies exactly what ordering you may demand and how to ask for it.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 10 — what does it mean that something is *undefined behavior*, and what license does that give the optimizer? (2) Chapter 3 — a store does not go straight to shared memory; where does it sit first, and why does that let one core lag another's view? (3) *Predict:* two threads run `x = 1; r0 = y;` and `y = 1; r1 = x;` from `x = y = 0`. On paper, can both `r0` and `r1` end up 0? On a real x86 machine, out of two million trials, how many times do you think both read 0? Write your guesses; §24.1 measures it.

Every chapter of this part has leaned on an assumption so natural it was invisible: that there is *one* memory, and that all cores see writes to it in a single consistent order — the model Lamport named **sequential consistency** (1979). It is the mental picture behind "thread A sets the flag, thread B sees the flag, so thread B sees the data A wrote first." That picture is false. Real hardware has per-core store buffers and caches, and both the CPU and the optimizing compiler reorder memory operations for speed, so a program can observe outcomes no sequentially consistent execution allows. This chapter makes the true model precise. First, the demonstration that reordering is real, on x86 no less — a litmus test whose "impossible" outcome happens millions of times (§24.1). Then the framework the C11/C++11 standard built to tame it: a **data race is undefined behavior**, and correctness is defined through the **happens-before** relation (§24.2). Then the menu of orderings an atomic can request — `relaxed`, `acquire`/`release`, and `seq_cst` — and the *synchronizes-with* edge that makes one thread's writes visible to another (§24.3). Then the workhorse pattern, message passing with acquire/release, what it guarantees, and what it costs on real hardware (§24.4). We close the part by carrying this specification forward as the ground truth beneath every concurrent program (§24.5).

24.1 The single-memory illusion: reordering is real

Consider the **store-buffering** litmus test (it underlies Dekker's mutual-exclusion algorithm). Two threads, starting from `x = y = 0`: thread 0 does `x = 1` then reads `r0 = y`; thread 1 does `y = 1` then reads `r1 = x`. Reason it through under a single shared memory: whichever store happens first, the later thread's load must see a 1, so `r0 == 0 && r1 == 0` is *impossible*. Yet it happens — because each core's store sits in a private **store buffer** (Chapter 3) before reaching cache, so each thread reads the other's not-yet-published value as 0. Here it is with relaxed atomics (to avoid the undefined behavior of a plain data race), two million trials on x86-64 (gcc 11.4.0, Linux 6.18):

```
relaxed store-buffering, 2000000 trials:
r0==0 && r1==0 observed 349165 times   (about 1 in 5; sequential consistency forbids it)
```

The "impossible" outcome occurred in roughly one trial in five. This is not a bug in the program or the hardware: x86's memory model (formalized as **x86-TSO**, total store order, by Sewell et al., 2010) explicitly allows a store to be reordered after a later load to a *different* address, precisely because of the store buffer. And x86 is one of the *stronger* models — ARM and POWER reorder far more aggressively. Two forces reorder your memory operations: the **hardware**, as here, and the **compiler**, which freely reorders and elides plain loads and stores it cannot prove another thread observes. The consequence is stark: the intuitive single-memory, everyone-agrees-on-one-order model you have been using is not what the machine provides. To write correct concurrent code you need a precise model of what *is* guaranteed — and a way to buy back the ordering you need where you need it.

> **QUICK CHECK**
>
> In the store-buffering test, explain why `r0 == 0 && r1 == 0` is impossible under sequential consistency but common on real x86. Name the hardware structure responsible, and name the *other* agent (besides the CPU) that reorders memory operations. (Answer after the next section.)

24.2 Data races are undefined behavior; happens-before

Faced with hardware and compilers that reorder, the C11/C++11 standard did not promise sequential consistency for ordinary code — that would forbid the optimizations that make programs fast. Instead it drew a bright line. A **data race** — two accesses to the same location from different threads, at least one a write, not ordered by synchronization, with at least one non-atomic — is **undefined behavior** (Chapter 10). Not "unspecified value," not "torn read": undefined, the compiler may assume it never happens and optimize accordingly, so a racy program has no meaning at all. The flip side is the contract: if your program is **data-race-free** — every conflicting pair of accesses ordered by synchronization through atomics — the standard guarantees it behaves as some sequentially consistent execution. This is the DRF-SC guarantee that Boehm and Adve's "Foundations of the C++ Concurrency Memory Model" (2008) established for C and C++: race-free code gets the simple model; racy code gets nothing.

"Ordered by synchronization" is made precise by **happens-before**. Within a single thread, operations are **sequenced-before** one another in program order. Across threads, an atomic store and the atomic load that reads its value can form a **synchronizes-with** edge (the details in §24.3). Happens-before is, roughly, the transitive closure of sequenced-before and synchronizes-with: if A happens-before B, then A's effects are guaranteed visible to B. Two conflicting accesses *not* ordered by happens-before are a data race. And the default atomic ordering, `memory_order_seq_cst`, additionally forces all such atomic operations into a single global total order — restoring the sequentially consistent illusion for code that uses it. Rerun the store-buffering test with seq_cst instead of relaxed (gcc 11.4.0):

```
seq_cst store-buffering, 2000000 trials:
r0==0 && r1==0 observed 0 times   (sequential consistency restored)
```

THE SAME IDEA THREE WAYS: MEMORY OPERATIONS REORDER UNLESS AN ORDERING FORBIDS IT

1. In words. Plain and relaxed memory operations may be reordered by the CPU and compiler, so one thread can see another's operations out of program order. Sequential consistency (the default `seq_cst` atomics) forces a single global order that forbids such reorderings; a data race — conflicting accesses not ordered by happens-before — is undefined behavior.

2. As a picture. Figure 24.1: each thread's store sits in a private store buffer before reaching shared memory, so thread 0 reads `y` (still 0) before thread 1's `y = 1` drains — and symmetrically — yielding `r0 == r1 == 0`.

3. As numbers. Store-buffering, two million trials: with relaxed atomics the SC-forbidden outcome happened 349,165 times (~1 in 5); with seq_cst it happened 0 times.

*(Answer to the §24.1 quick check: under sequential consistency the two stores occur in some global order, and the thread that stores second must, when it later loads, see the first thread's already-committed store — so both loads reading 0 cannot happen. On x86 each store first enters a private **store buffer** and is not yet visible to the other core, so each thread's load reads the other's stale 0. Besides the CPU, the **compiler** also reorders memory operations.)*

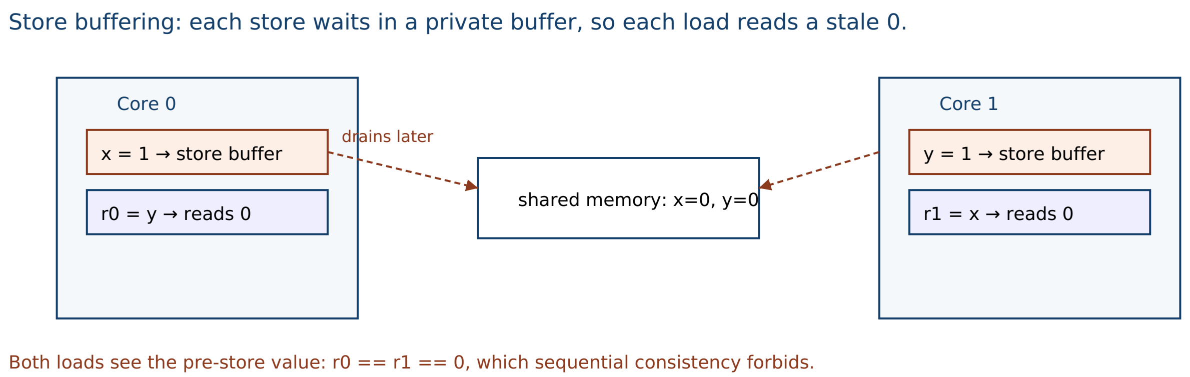


Figure 24.1: Store buffering. Each core's store to `x` (resp. `y`) sits in a private store buffer before it reaches shared memory, so each core's subsequent load of the other variable reads the old value 0. The outcome `r0 == r1 == 0` is impossible under sequential consistency yet common on real x86 — measured here about one trial in five.

24.3 The ordering menu: relaxed, acquire/release, seq_cst

Every atomic operation takes a `memory_order` argument choosing how much ordering it enforces — a dial from "atomic but unordered" to "globally ordered," trading cost for guarantees. There are three levels worth knowing. **`memory_order_relaxed`** guarantees only atomicity (no torn or lost updates) and coherence on the *same* object; it imposes *no* ordering on other memory operations and creates no synchronizes-with edge. It is right for a statistics counter whose exact interleaving with other data does not matter — the store-buffering test used it, and indeed it let the reordering show. **`memory_order_release`** (on a store) paired with **`memory_order_acquire`** (on a load) is the key construct: if an acquire-load reads the value written by a release-store to the same atomic, the store **synchronizes-with** the load, and everything sequenced-before the release in the storing thread happens-before everything sequenced-after the acquire in the loading thread. That is the machinery that makes a published pointer safe: all the writes that set up the object are visible to any thread that acquires the pointer. **`memory_order_seq_cst`**, the default, adds to acquire/release a single global total order over all seq_cst operations — the strongest and most expensive, and the only one that forbids the store-buffering reordering of §24.1.

The relationships to keep straight: *sequenced-before* is ordering within one thread (program order); *synchronizes-with* is the cross-thread edge a release/acquire pair (or a lock release/acquire, or a thread create/join) creates; and *happens-before* is their transitive closure, the relation that actually determines visibility and defines a data race. Relaxed atomics participate in none of the cross-thread ordering — they are atomic dots with no edges — which is exactly why relaxed is cheap and why it cannot, by itself, publish anything but the atomic's own value. When you need one thread's ordinary writes to become visible to another, you need a release/acquire (or stronger) pair, not relaxed.

QUICK CHECK

What does `memory_order_relaxed` guarantee, and what does it *not*? State precisely what a release-store/acquire-load pair establishes when the acquire reads the release's value. What does `seq_cst` add on top of acquire/release? (Answer below the communicate box.)

24.4 Acquire/release in practice: message passing, and what it costs

The canonical use of acquire/release is **message passing**: one thread fills a payload with ordinary writes, then publishes a flag with a release-store; another thread waits on the flag with an acquire-load and, once it sees the flag, reads the payload. The release/acquire pair guarantees the payload writes happen-before the reads — so the reader never sees a published flag but stale payload. This is precisely the guarantee the lock-free structures of Chapter 23 needed when they published a node pointer. A producer and consumer handing off a four-word payload a million times, verifying every one (gcc 11.4.0, Linux 6.18):

```
/* producer */                          /* consumer */
payload[0..3] = ...;      /* plain */   while (load_acquire(&flag) != r) {}
store_release(&flag, r);  /* publish */  read payload[0..3];   /* guaranteed visible */

message passing via release/acquire, 1000000 rounds: payload mismatches = 0
```

Not one of a million handoffs saw a stale payload: the acquire that observed the flag was guaranteed, by synchronizes-with, to see every write sequenced before the release. Now the cost, which is where the memory model gets concrete and surprising. On x86 the ordering you pay for depends entirely on which order you ask for. Compare the compiler's code for stores and loads at each level (gcc 11.4.0, `-O2`, `objdump`):

```
store_relaxed :  mov   %edi, f(%rip)     ← plain store
store_release :  mov   %edi, f(%rip)     ← plain store (identical!)
store_seqcst  :  xchg  %edi, f(%rip)     ← locked exchange — drains the store buffer
load_acquire  :  mov   f(%rip), %eax     ← plain load
```

On x86-TSO, release-stores and acquire-loads compile to *plain* `mov` instructions — identical to relaxed — because the hardware already guarantees the ordering they need (it never reorders store-store or load-load, only store-load). The ordering is enforced only by forbidding the *compiler* from reordering across them, at no runtime cost. Only `seq_cst`'s store costs a real instruction — the `xchg` (a locked operation) that drains the store buffer and thereby forbids the store-load reordering that produced §24.1's result. This is the honest picture of the memory model: acquire/release is often free on strong hardware and lets the compiler still forbid its own reorderings, while seq_cst buys a global order at the price of a fence. On weaker architectures (ARM, POWER) acquire and release require real barrier instructions too — which is why using the weakest ordering that is correct matters for portable performance. But the danger runs the other way: using an ordering that is too *weak* is not slow, it is *wrong*.

TRAP: USING `RELAXED` WHERE YOU NEED RELEASE/ACQUIRE (RELAXED ORDERS NOTHING BUT ITSELF)

Because `memory_order_relaxed` is the cheapest ordering and "the flag is atomic anyway," it is tempting to publish data with a relaxed store and read it with a relaxed load. But relaxed creates *no* synchronizes-with edge and therefore *no* happens-before: it guarantees the atomic's own value is not torn, and nothing whatsoever about the ordinary writes around it. So a consumer can see the relaxed flag set to "ready" while the payload writes that preceded it are still invisible — it reads the flag as published and the data as garbage. Worse, it may work on x86 (where the hardware happens to order store-store) and fail on ARM, so it passes your tests and corrupts on a phone. The rule: to publish data, the store that publishes must be `release` and the load that observes it must be `acquire` (or use seq_cst); relaxed is only for atomics whose *ordering relative to other memory does not matter*, such as a statistics counter or a stop flag whose late arrival is harmless. If you are using an atomic to make *other* writes visible, relaxed is a correctness bug, not an optimization.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I made the handoff faster. The producer fills the buffer and sets `ready` with a `relaxed` atomic store; the consumer spins on a `relaxed` load of `ready` and then reads the buffer. It's all atomic and it passes every test on our CI machines — why do you keep saying it's broken?" Cover the paragraph and respond in four or five sentences, drawing on §24.3 and §24.4.

Model answer. "It passes because your CI machines are x86, whose hardware happens to keep stores in order, so the buffer writes reach memory before the flag — but the C memory model gives you no such guarantee, and the bug is real. A `relaxed` store creates no synchronizes-with edge and thus no happens-before, so nothing orders the buffer writes relative to the flag; the consumer is entitled to see `ready` set and the buffer still empty, which is exactly what will happen on ARM. The fix costs nothing on x86: make the publishing store `release` and the observing load `acquire`, and now every write before the release is guaranteed visible after the acquire — that pairing is the whole point of the ordering argument. Relaxed is correct only when the atomic's ordering relative to other data doesn't matter, like a counter; here you're using it to *publish* data, so it's a correctness bug, not a speedup." Explaining that x86 hides the bug, naming synchronizes-with/happens-before, and prescribing release/acquire is the senior register.

(Answer to the §24.3 quick check: `relaxed` guarantees atomicity and same-object coherence but imposes no ordering on other memory and creates no synchronizes-with edge. A release-store paired with an acquire-load that reads its value establishes that the store synchronizes-with the load, so everything sequenced-before the release happens-before everything sequenced-after the acquire — making the storing thread's prior writes visible to the loading thread. `seq_cst` adds a single global total order over all seq_cst operations, restoring sequential consistency and forbidding the store-buffering reordering.)

24.5 What to carry forward

The single-memory, everyone-agrees model — **sequential consistency** — is an illusion real hardware and compilers break for speed. Our store-buffering litmus test produced the SC-forbidden outcome `r0 == r1 == 0` about one time in five on x86 (349,165 of two million), because each core's store waits in a private buffer before it is visible; the CPU and the compiler both reorder plain memory operations. The C11/C++11 memory model tames this with a bright line: a **data race is undefined behavior**, but a **data-race-free** program behaves as some sequentially consistent execution (Boehm and Adve's DRF-SC guarantee). Correctness is defined through **happens-before** — the transitive closure of *sequenced-before* (program order in a thread) and *synchronizes-with* (the cross-thread edge from a release-store to the acquire-load that reads it). The `memory_order` menu picks how much ordering an atomic buys: `relaxed` (atomic but unordered — no synchronizes-with), `acquire`/`release` (the publish/observe pair that makes prior writes visible — our message-passing handoff verified a million payloads with zero stale reads), and `seq_cst` (a global total order, the only one that forbade the store-buffering reordering, and on x86 the only one that costs a real fence). The trap: relaxed publishes nothing but its own value, so using it to hand off data is a correctness bug that x86 hides and ARM exposes.

This closes the concurrency part, and with it the last piece of shared-memory correctness. You now hold the full toolkit and, more importantly, the model beneath it: threads share memory and race (Chapter 15); locks and condition variables give blocking mutual exclusion and coordination (Chapters 20–21); atomics and compare-and-swap are the hardware primitives they rest on (Chapter 22); lock-free structures use those primitives to avoid blocking entirely (Chapter 23); and the memory model is the specification that says, precisely, when a write by one thread is guaranteed visible to another (this chapter). The recurring lesson has been the same at every level — the abstract machine's simple, sequential memory is not what runs, and the gap between the two is where concurrency bugs live. What comes next is a language built to close that gap by construction. Everything in this part you had to prove by hand and hold in your head — which data is shared, which access needs a lock, which ordering an atomic requires, whether a pointer outlives the thread reading it — a type system can instead enforce at compile time, turning "I reasoned about it carefully" into "it does not compile otherwise." The next part turns to Rust: ownership, borrowing, and lifetimes as the machinery that encodes these very invariants — data-race freedom not as a discipline you maintain, but as a property the compiler checks.

CAN YOU… WITHOUT LOOKING?

- Describe the store-buffering test and why `r0 == r1 == 0` is SC-forbidden yet common on x86, and name the store buffer.
- State the DRF-SC contract: what a data race is (and that it is UB), and what a race-free program is guaranteed.
- Define sequenced-before, synchronizes-with, and happens-before, and how they relate.
- Say what `relaxed`, `acquire/release`, and `seq_cst` each guarantee.
- Explain the message-passing pattern and why a release/acquire pair makes the payload visible.
- Explain why publishing data with a relaxed store is a bug, and why it can pass on x86 and fail on ARM.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why does the store-buffering outcome `r0 == r1 == 0` occur on x86, and what forbids it? (§24.1)
2. **R2.** What is a data race, what does the standard say it is, and what does DRF-SC guarantee? (§24.2)
3. **R3.** Define sequenced-before, synchronizes-with, and happens-before. (§24.2–§24.3)
4. **R4.** What do `relaxed`, `acquire/release`, and `seq_cst` guarantee, and which is free on x86? (§24.3–§24.4)
5. **R5.** Why is publishing data with a relaxed store a correctness bug, and why might it pass on x86? (§24.4)

FURTHER READING

- **Boehm & Adve, "Foundations of the C++ Concurrency Memory Model," *Proc. ACM PLDI* (2008), pp. 68–78, DOI 10.1145/1375581.1375591.** The DRF-SC design of §24.2: race-free programs get sequential consistency, races are undefined. Author, venue, and year verified via dblp and the ACM Digital Library.
- **Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," *IEEE Transactions on Computers* C-28(9):690–691 (1979), DOI 10.1109/TC.1979.1675439.** The definition of sequential consistency — the model §24.1 shows hardware violating. Verified via IEEE Xplore and dblp.
- **Sewell, Sarkar, Owens, Zappa Nardelli & Myreen, "x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors," *Communications of the ACM* 53(7): 89–97 (2010), DOI 10.1145/1785414.1785443.** The precise model of x86 memory ordering behind the store-buffering result of §24.1. Verified via the ACM Digital Library.
- **cppreference, `std::memory_order` (`en.cppreference.com`).** The exact semantics of `relaxed`, `acquire/release`, and `seq_cst`, and of synchronizes-with and happens-before. Primary reference for §24.3–§24.4; verify per standard version.
- **Preshing, "Memory Reordering Caught in the Act" (blog, `preshing.com`, 2012).** A hands-on walkthrough of the store-buffering experiment reproduced in §24.1, with the reordering measured directly. Accessible background; a blog, not a peer-reviewed source.

Exercises

24.1 WARM-UP In one sentence, what is sequential consistency, and name one hardware structure that causes real machines to violate it.

24.2 WARM-UP What is a data race, and what does the C11/C++11 standard say a program containing one means?

24.3 WARM-UP What does `memory_order_relaxed` guarantee, and what does it not guarantee about other memory operations?

24.4 CORE For the store-buffering test (`x = 1; r0 = y;` and `y = 1; r1 = x;` from `x = y = 0`): (a) prove `r0 == r1 == 0` is impossible under sequential consistency. (b) Explain, using the store buffer, why it happens on x86 anyway. (c) Which single change — and to which operations — forbids the outcome, and what does it cost on x86 (name the instruction)?

24.5 CORE Define *sequenced-before*, *synchronizes-with*, and *happens-before*, and state how happens-before is built from the other two. Then explain, in those terms, why two conflicting accesses not ordered by happens-before are a data race.

24.6 CORE Write the message-passing pattern with a payload and a flag. (a) Mark which store must be `release` and which load must be `acquire`. (b) Explain, via synchronizes-with and happens-before, why the consumer is guaranteed to see the payload once it sees the flag. (c) Show what breaks if both are `relaxed`, and why it may still pass on x86.

24.7 COST Using the disassembly (relaxed store = `mov`, release store = `mov`, seq_cst store = `xchg`, acquire load = `mov`): (a) explain why acquire/release are free on x86 but seq_cst's store is not. (b) What does the `xchg` do that the plain `mov` does not, and how does that forbid store-buffering? (c) Why might the same acquire/release require real barrier instructions on ARM, and what does that imply for "use the weakest correct ordering"?

24.8 FADED *Worked, with the last step for you.* A lock-free structure publishes a freshly built node by CAS-ing a pointer, and readers follow the pointer and read the node's fields. Occasionally on an ARM device a reader sees the new pointer but garbage fields.
Step 1 (given): The node's fields are written with plain stores, then the pointer is published.
Step 2 (given): The publish and the read use `relaxed` atomics, which create no synchronizes-with edge.
Step 3 (given): With no happens-before between the field writes and the reader's field reads, the reader may observe the pointer before the field writes are visible — undefined behavior in general, and visible reordering on ARM.
Step 4 (you): State the fix precisely (which operation becomes `release`, which becomes `acquire`), explain why that pairing makes the field writes visible, and say why the bug hid on x86.

24.9 MIXED For each atomic use, choose the weakest correct ordering — `relaxed`, `acquire`/`release`, or `seq_cst` — and justify in a line: (a) a hit counter incremented by many threads, read only at shutdown; (b) a flag that publishes a fully initialized buffer to a consumer; (c) a "please stop" flag a worker polls, where a slightly late stop is harmless; (d) two flags in a Dekker-style algorithm needing store-load ordering.

24.10 MIXED Drawing on Chapters 22–24, classify each as *true* or *false* and fix the false ones: (a) "A data race produces an unspecified value but defined behavior." (b) "On x86, a release-store compiles to the same instruction as a relaxed store." (c) "`memory_order_relaxed` makes the writes around it visible to other threads in order." (d) "A race-free program using seq_cst atomics behaves as some sequentially consistent execution." (e) "seq_cst is free on x86, just like acquire/release."

24.11 STRETCH The DRF-SC guarantee says race-free programs get sequential consistency and racy ones get undefined behavior. (a) Explain why this bargain lets the compiler and hardware keep their reordering optimizations while still giving well-behaved programs a simple model. (b) Argue why "undefined," not merely "unspecified," is the right (if harsh) definition for a data race — what would the compiler be forbidden from doing under a weaker definition? (c) Connect this to Chapter 10: how is a data race like other undefined behavior the optimizer is allowed to assume never happens, and what does that mean for "but it worked when I tested it"?

24.12 LAB On your own machine: (a) implement the store-buffering test with two threads and relaxed atomics, run a couple million trials, and count how often `r0 == r1 == 0` — confirm it is nonzero on x86; switch the atomics to `seq_cst` and confirm the count drops to zero. (b) Disassemble a relaxed store, a release store, a seq_cst store, and an acquire load with `objdump`, and identify which one differs (the seq_cst store's locked instruction). (c) Build the message-passing handoff with release/acquire and verify a large number of payloads with zero stale reads. Label your compiler and CPU architecture; note that the reordering counts are hardware-dependent.

Chapter 25 — Thread Pools and Task Parallelism

Before you start: Chapter 15 (threads share an address space — the substrate every worker here runs on), Chapter 21 (condition variables and the producer–consumer pattern — a work queue *is* a producer–consumer), Chapter 16 (the scheduler multiplexes runnable threads onto cores — more threads is not more parallelism), Chapter 6 (the cost model — a thread is not free). · The last five chapters were about making concurrent code *correct*; this one is about making parallel code *fast to structure* — decoupling the work you have from the threads that run it.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 21 — sketch the producer–consumer pattern: how does a consumer wait for an item without busy-spinning, and what wakes it? (2) Chapter 6 — a thread has a stack (often ~8 MiB of address space) and must be set up and torn down by the kernel; is creating one closer to a function call or to a system call in cost? (3) *Predict:* on a laptop, roughly how many microseconds does it take to `pthread_create` a thread and `pthread_join` it? If you have 100,000 tiny tasks, how much faster might a pool of 8 reused workers be than spawning one thread per task? Write your guesses; §25.1 and §25.2 measure both.

You have a machine with many cores and a pile of work. The naive move is to spawn a thread for each unit of work and let the scheduler sort it out. It is a trap on both ends: a thread is expensive to create and destroy, and thousands of runnable threads oversubscribe the cores and drown the scheduler in context switches. The discipline that fixes both is to *separate the work from the workers*. A fixed pool of long-lived threads pulls units of work — **tasks** — off a shared queue; you submit tasks, the pool runs them. This chapter builds that idea up from the cost of a bare thread. First, why a thread-per-task does not scale — the measured cost of creation and the problem of oversubscription (§25.1). Then the **thread pool**: workers draining a queue, built from the very condition-variable machinery of Chapter 21 (§25.2). Then **task parallelism** and the **fork–join** model — decomposing a computation into tasks whose results you wait on, with **futures** as handles to not-yet-computed values (§25.3). Then the scalability problem a single shared queue creates, and its classic answer, **work-stealing** (§25.4). We close by carrying the task abstraction forward as the unit in which parallel work is expressed (§25.5).

25.1 The cost of a thread: why not one per task

A thread is not a function call. Creating one asks the kernel to allocate a stack, set up a task structure, and make the thread schedulable; joining it tears all that down. That is real work on the system-call boundary of Chapter 13, not the handful of cycles a call costs. Measure it directly — create and immediately join an empty thread, 200,000 times (gcc 11.4.0, `-O2 -pthread`, Linux 6.18 under WSL2; this environment has higher thread-setup overhead than bare metal, so treat the absolute number as illustrative):

```
pthread_create + pthread_join, 200000 iters: 44.07 s  (220.4 us each)
```

Roughly *220 microseconds* per thread on this machine. In that time a modern core executes on the order of a million instructions — so if your task is smaller than that, you spend more time managing the thread than doing the work. And there is a second, subtler cost: **oversubscription**. Spawning 100,000 threads for 100,000 tasks does not give you 100,000-way parallelism; you have a fixed number of cores (Chapter 16), so the scheduler must time-slice all those threads onto them, paying a context switch and cache disruption at every slice, plus the memory for every thread's stack. More runnable threads than cores is not more parallelism — it is more overhead. The fix follows from stating the problem precisely: you want as much *parallelism* as you have cores, and as many *tasks* as the problem naturally has, and those two numbers should be decoupled. That decoupling is exactly what a thread pool provides.

> **QUICK CHECK**
>
> Name the two distinct costs that make thread-per-task a bad structure for many small tasks — one paid at creation/teardown, one paid while they run. Why does spawning 100,000 threads on an 8-core machine not give 100,000-way parallelism? (Answer after the next section.)

25.2 The thread pool: workers draining a work queue

A **thread pool** creates a fixed set of worker threads once, up front, and keeps them alive. Work is submitted as **tasks** onto a shared queue; each idle worker loops: take a task, run it, repeat. This is the producer–consumer pattern of Chapter 21 wearing a new name — the queue is guarded by a mutex, and workers block on a condition variable when it is empty rather than busy-waiting, so an idle pool costs no CPU. The core of a worker is exactly the Chapter 21 consumer:

```
void *worker(void *_) {
    for (;;) {
        pthread_mutex_lock(&mtx);
        while (queue_empty() && !shutdown)   /* predicate loop, never an if */
            pthread_cond_wait(&not_empty, &mtx);
        if (queue_empty() && shutdown) { pthread_mutex_unlock(&mtx); return NULL; }
        task_t t = dequeue();
        pthread_mutex_unlock(&mtx);         /* run the task OUTSIDE the lock */
        t.fn(t.arg);                        /* do the work */
    }
}
```

The pool pays the ~220 us thread-creation cost exactly `W` times — once per worker at startup — and then *amortizes* it across every task ever submitted. Run 100,000 tiny tasks (each a ~200-iteration arithmetic loop) two ways on this machine: one thread per task versus a pool of 8 workers (gcc 11.4.0, `-O2 -pthread`):

```
thread-per-task : 100000 tasks, 21.73 s   (217.2 us/task)
thread pool (8) : 100000 tasks,  0.10 s   (  1.04 us/task)
```

The pool ran the same work about **200× faster** per task, because it stopped paying the thread tax on every task and paid it only eight times. Two design points matter. First, run the task *outside* the lock — the mutex guards the queue, not the work, so holding it during `t.fn()` would serialize the whole pool back down to one worker (that is the contention lesson of the next chapter, previewed). Second, the pool size is a knob: too few workers and cores sit idle; too many and you are back to oversubscription. A common default is one worker per hardware thread for CPU-bound work, more if tasks block on I/O. The pool has bought us the decoupling we wanted: submit as many tasks as the problem has, run them on as many workers as the machine has.

> **THE SAME IDEA THREE WAYS: A POOL AMORTIZES THREAD COST ACROSS MANY TASKS**
>
> **1. In words.** Creating a thread is expensive and having more runnable threads than cores just adds overhead. A thread pool creates a fixed set of workers once and feeds them tasks from a shared queue, so the creation cost is paid a handful of times and reused for every task, and parallelism is capped at the number of workers rather than the number of tasks.
>
> **2. As a picture.** Figure 25.1: producers enqueue tasks; a fixed row of workers each loop "lock → wait if empty → dequeue → unlock → run", blocking on a condition variable when the queue drains.
>
> **3. As numbers.** 100,000 tiny tasks: thread-per-task took 21.73 s (217 us/task, paying ~220 us creation every time); a pool of 8 took 0.10 s (1.04 us/task) — about 200× faster.

(Answer to the §25.1 quick check: the two costs are (1) ***creation/teardown*** *— the kernel allocates a stack and task structure and makes the thread schedulable, ~220 us measured here, and (2)* ***oversubscription*** *— once runnable threads exceed cores, the scheduler time-slices them, paying context switches and cache disruption. 100,000 threads on 8 cores gives at most 8-way parallelism because only 8 threads run at once; the other ~99,992 just wait, so you get the overhead of many threads with the parallelism of eight.)*

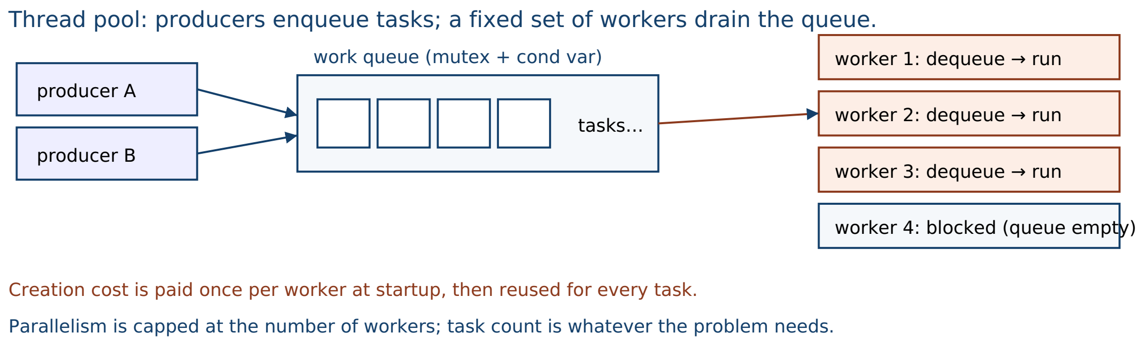


Figure 25.1: A thread pool. A fixed set of workers repeatedly lock the queue, wait on a condition variable while it is empty, dequeue a task, unlock, and run the task outside the lock. Thread-creation cost is paid once per worker; every submitted task reuses those workers.

25.3 Task parallelism and fork–join

A pool runs whatever tasks you give it; **task parallelism** is the discipline of *decomposing a computation* into independent tasks that can run concurrently, as opposed to **data parallelism**, which applies the same operation across many data elements. The canonical structure is **fork–join**: split the work into parts (*fork* subtasks), run them in parallel, then *join* — wait for all of them and combine results. Summing a large array is the textbook case: partition it into `P` ranges, hand each to a thread, and add the partial sums. Measured on this machine (64M doubles, each element passed through `sqrt`; gcc 11.4.0 `-O2 -pthread -lm`; warmed, averaged over 3 runs):

```
P= 1 : 0.369 s  (baseline)
P= 2 : 0.186 s   speedup 1.98x
P= 4 : 0.134 s   speedup 2.75x
P= 8 : 0.084 s   speedup 4.38x
P=16 : 0.076 s   speedup 4.86x
```

Two threads nearly halve the time (1.98×) — near-perfect for a problem that splits cleanly — but the speedup curve bends below linear as `P` grows: 4.38× at eight threads, not 8×. Why it bends (memory bandwidth, a serial remainder, this CPU's mix of performance and efficiency cores) is the whole subject of the next chapter; here the point is the *structure*. Fork–join needs a way to hand back a subtask's result, and the abstraction for that is a **future** (sometimes a *promise* on the producing side): a handle to a value that is not computed yet. Submitting a task returns a future immediately; calling `.get()` on it blocks until the task completes and yields the result — the join. Futures let you express a dependency graph directly: fork several tasks, get their futures, and a task that needs their outputs simply gets them, blocking only if they are not ready. The pool of §25.2 plus futures is enough to express most fork–join parallelism: decompose, submit, collect.

> **QUICK CHECK**
>
> Distinguish *task* parallelism from *data* parallelism in one line each. In a fork–join computation, what does “join” do, and what does a *future* represent? Why does calling `future.get()` sometimes block and sometimes return immediately? (Answer below the communicate box.)

25.4 Load balancing and work-stealing

The single shared queue of §25.2 has a scaling flaw: every worker locks the *same* mutex to get its next task, so as you add workers they contend on that one lock, and the queue itself becomes a serial bottleneck — precisely the contention the next chapter measures. It also balances load only coarsely. The standard answer, introduced by the Cilk system (Frigo, Leiserson & Randall, 1998) and analyzed by Blumofe and Leiserson, is **work-stealing**. Give *each* worker its own double-ended queue (a *deque*) of tasks. A worker pushes newly forked subtasks onto, and pops them from, the *bottom* of its own deque — a purely local, uncontended operation in the common case. When a worker's own deque runs empty, it becomes a **thief**: it picks another worker at random and steals a task from the *top* of that victim's deque. Pushing/popping one end locally while thieves take from the other end keeps the fast path contention-free and distributes load automatically — idle workers go find work rather than waiting to be fed.

This is not just an engineering nicety; it comes with a proof. Blumofe and Leiserson ("Scheduling Multithreaded Computations by Work Stealing," 1999) showed that for a fully-strict (well-structured fork–join) computation with work T_1 (total operations) and span T_∞ (the critical-path length, the time on infinitely many processors), a randomized work-stealing scheduler runs in expected time $O(T_1/P + T_\infty)$ on P processors — asymptotically optimal, with provably bounded space and communication. In words: you get near-linear speedup as long as there is enough parallelism ($T_1/T_\infty > P$) to keep the processors fed. Work-stealing is why fork–join frameworks scale: Cilk, Intel's TBB, and Doug Lea's Java Fork/Join framework (2000, the engine under Java's parallel streams) are all work-stealing pools. When you write "fork these subtasks and join," a work-stealing runtime is what turns that into balanced parallelism without your managing which core does what.

TRAP: SUBMITTING TASKS THAT BLOCK ON OTHER TASKS IN THE SAME BOUNDED POOL (POOL DEADLOCK)

A thread pool has a fixed number of workers, and that number is a resource that can be exhausted. The classic deadlock: a task running on the pool submits a *subtask* to the same pool and then *blocks waiting for that subtask's result*. If every worker is simultaneously blocked waiting on a subtask that is still sitting in the queue — because there is no free worker to run it — the pool is wedged forever: the waiters hold the only threads that could make progress. With a pool of `W` workers, `W` tasks that each block on an un-started subtask is all it takes. This bites people who treat a pool as infinitely deep, and it is especially nasty because it depends on timing and pool size, so it passes tests and hangs in production under load. The disciplines that avoid it: never block a pool worker waiting on work queued to the *same* pool; use a work-stealing runtime whose join can execute the pending subtask on the current thread instead of blocking (this is exactly what fork–join frameworks do); or separate blocking and CPU-bound work into different pools. If your tasks have dependencies, express them as futures the runtime understands — do not have a worker sit on a lock waiting for a task that has nowhere to run.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: “Our request handler spawns a fresh thread for every incoming job so they run in parallel. Under load the box is at 100% CPU but throughput actually *drops*, and latency is all over the place. Should we just spawn the threads at higher priority?” Cover the paragraph and respond in four or five sentences, drawing on §25.1–§25.2.

Model answer. “Higher priority won't help — the problem is that you have far more runnable threads than cores, so the CPU is busy context-switching and thrashing caches instead of doing work, and you're also paying ~hundreds of microseconds to create and destroy a thread per job. Bound the concurrency: use a fixed thread pool sized to about the number of hardware threads for CPU-bound jobs, and submit jobs as tasks onto its queue. Now creation cost is paid once per worker and amortized over every job, and you never oversubscribe the cores, so throughput goes up and latency stabilizes. If jobs block on I/O rather than CPU, size that pool larger or give I/O its own pool — but the fix is bounding and reusing threads, not spawning more of them faster.” Naming oversubscription and creation cost, and prescribing a bounded reused pool, is the senior register.

*(Answer to the §25.3 quick check: **task** parallelism decomposes a computation into different tasks that run concurrently; **data** parallelism applies the same operation across many data elements. “Join” waits for all forked subtasks to finish and combines their results. A **future** is a handle to a value that may not be computed yet; `future.get()` blocks if the task is still running and returns immediately if it has already completed — it is how a dependent task waits exactly as long as necessary and no longer.)*

25.5 What to carry forward

The organizing idea of this chapter is to **separate the work from the workers**. A thread is expensive — about 220 microseconds to create and join on the machine here — and more runnable threads than cores is not more parallelism, just more context-switching overhead, so spawning a thread per task fails on both cost and oversubscription. A **thread pool** fixes both: a fixed set of long-lived workers drains a shared work queue (the producer–consumer pattern of Chapter 21, built from a mutex and a condition variable so an idle pool burns no CPU), paying the creation cost a handful of times and amortizing it across every task — 100,000 tiny tasks ran ~200× faster on a pool of 8 than one-thread-per-task. **Task parallelism** decomposes a computation into tasks; the **fork–join** model forks subtasks, runs them in parallel, and joins their results, with **futures** as handles to not-yet-computed values (our parallel sum hit 1.98× on two threads, then bent below linear). A single shared queue contends, so real fork–join runtimes use **work-stealing** — per-worker deques with idle workers stealing from busy ones — which Blumofe and Leiserson proved runs a well-structured computation in near-optimal $O(T_1/P + T_\infty)$ time. The trap: a bounded pool is a finite resource, so a worker that blocks waiting on a subtask queued to the same pool can deadlock it.

You now have the vocabulary for the rest of the part and much of the book: *task* as the unit of work, *pool* as the bounded set of workers, *fork/join* and *future* as the way to express and collect parallel work, and *work-stealing* as the runtime that balances it. But every speedup number in this chapter came with a caveat — two threads gave 1.98×, eight gave only 4.38×. That gap between the parallelism you asked for and the speedup you got is not sloppiness; it is fundamental, and it has names. The next chapter makes it precise: **Amdahl's law** and its serial fraction, **Gustafson's** rejoinder about growing the problem, and the two silent killers of scalability — **lock contention** and **false sharing** — that turn adding cores into a slowdown. Having learned to express parallel work, we turn to the question of how much of it actually pays off.

CAN YOU… WITHOUT LOOKING?

- State the two costs (creation/teardown and oversubscription) that make thread-per-task a poor structure.
- Describe a thread pool and how it is the Chapter 21 producer–consumer pattern, and why the task runs outside the lock.
- Explain why a pool amortizes thread-creation cost, and roughly what that bought in the 100,000-task measurement.
- Define task vs data parallelism, the fork–join model, and what a future represents.
- Explain work-stealing: per-worker deques, local push/pop vs stealing, and what it buys over one shared queue.
- Explain how submitting a blocking, dependent task to a bounded pool can deadlock it, and how to avoid that.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why is thread-per-task a bad structure for many small tasks — name both costs. (§25.1)
2. **R2.** What is a thread pool, which Chapter 21 pattern is it, and why is the creation cost amortized? (§25.2)
3. **R3.** Define task parallelism, the fork–join model, and a future. (§25.3)
4. **R4.** What is work-stealing, and what did Blumofe & Leiserson prove about its running time? (§25.4)
5. **R5.** How can a bounded pool deadlock, and how do you prevent it? (§25.4)

FURTHER READING

- **Blumofe & Leiserson, "Scheduling Multithreaded Computations by Work Stealing," *Journal of the ACM* 46(5):720–748 (1999), DOI 10.1145/324133.324234.** The provably-good work-stealing scheduler of §25.4, with the $O(T_1/P + T_\infty)$ time and space bounds. First author Robert D. Blumofe; verified via dblp and the ACM Digital Library.
- **Frigo, Leiserson & Randall, "The Implementation of the Cilk-5 Multithreaded Language," *Proc. ACM PLDI* (1998), pp. 212–223, DOI 10.1145/277650.277725.** The work-first principle and the runtime that popularized work-stealing fork–join. First author Matteo Frigo; verified via dblp and the ACM Digital Library.
- **Lea, "A Java Fork/Join Framework," *Proc. ACM 2000 Java Grande Conference*, pp. 36–43, DOI 10.1145/337449.337465.** A practical work-stealing pool — the design under Java's parallel streams. First author Doug Lea; verified via dblp and the ACM Digital Library.
- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, "Concurrency" chapters (free online, `ostep.org`).** Threads, locks, and the producer–consumer / condition-variable foundation the pool of §25.2 is built on. Accessible background; verify chapter numbers per edition.

Exercises

25.1 WARM-UP In one sentence each, name the two costs that make spawning one thread per task a poor structure for many small tasks.

25.2 WARM-UP A thread pool is an instance of which pattern from Chapter 21? Name the two synchronization primitives its work queue uses and what each is for.

25.3 WARM-UP What is a *future*, and what happens when you call `.get()` on one whose task has not finished versus one whose task has?

25.4 CORE Explain why a pool of `W` workers running `N` tasks pays the thread-creation cost only about `W` times, not `N` times. Using the measured ~220 us creation cost and the 1.04 us/task pool result, estimate what fraction of the thread-per-task time (217 us/task) was creation overhead rather than useful work, and why the pool avoids it.

25.5 CORE In the worker loop of §25.2, the task is run *after* unlocking the queue mutex. (a) What would go wrong for throughput if you held the lock while running `t.fn()`? (b) Why must the empty-queue wait be a `while` loop around `cond_wait`, not an `if` (recall Chapter 21)? (c) How does the pool ensure an idle worker uses no CPU?

25.6 CORE Describe work-stealing precisely: what data structure does each worker own, which end does the owner use versus a thief, and why does that split keep the common case contention-free? Then state, in words, the Blumofe–Leiserson running-time bound and what condition on the computation is needed for near-linear speedup.

25.7 COST Consider a single-shared-queue pool with `P` workers, each task taking time `t`, where dequeuing requires the one global lock held for time `c`. (a) Argue that when `c` is not negligible relative to `t`, throughput stops improving past some number of workers, and identify what caps it. (b) Explain qualitatively how per-worker deques with work-stealing change this. (c) Why does making tasks *coarser* (more work per task) improve a shared-queue pool's scaling, and what is the downside of tasks that are too coarse?

25.8 FADED *Worked, with the last step for you.* A service uses a pool of 4 workers. Each request task, midway through, submits a second task to the *same* pool and blocks on its future before finishing. Under light load it is fine; under load it hangs.
Step 1 (given): The pool has exactly 4 worker threads; parallelism is capped at 4 running tasks.
Step 2 (given): Under load, 4 request tasks each reach the point of submitting a subtask and blocking on its future — all 4 workers are now blocked.
Step 3 (given): The 4 subtasks sit in the queue with no free worker to run them, so none of the futures will ever complete.
Step 4 (you): Name this failure, explain precisely why it is a deadlock (what holds what, what waits on what), and give two concrete fixes — one about pool structure and one about how dependent work should be expressed.

25.9 MIXED For each workload, say whether a fixed pool sized to the hardware-thread count is right, and if not what to change: (a) CPU-bound numeric tasks that never block; (b) tasks that spend most of their time blocked on network I/O; (c) a recursive divide-and-conquer computation with millions of tiny subtasks and deep nesting; (d) a mix of short CPU tasks and occasional long-blocking database calls. Justify each in a line.

25.10 MIXED Drawing on Chapters 21 and 25, classify each as *true* or *false* and fix the false ones: (a) “Spawning one thread per task gives you as much parallelism as you have tasks.” (b) “A thread pool's work queue is a producer–consumer buffer guarded by a mutex and a condition variable.” (c) “An idle thread pool spins, burning CPU while it waits for work.” (d) “Work-stealing eliminates the single shared queue by giving each worker its own deque.” (e) “Calling `future.get()` always blocks until the whole pool is idle.”

25.11 STRETCH Work-stealing is described as “idle workers steal from busy ones.” (a) Explain why stealing from the *top* (opposite end from the owner's push/pop) both reduces contention with the owner and tends to steal the *oldest*, largest-grained task — and why stealing a large task is good for load balance. (b) The Blumofe–Leiserson bound is $O(T_1/P + T_\infty)$. Interpret each term, and explain why a computation with too little parallelism (span T_∞ close to work T_1) cannot be sped up much no matter how many workers you add — and connect this to the next chapter's Amdahl's law.

25.12 LAB On your own machine: (a) measure the cost of `pthread_create` + `pthread_join` by creating and joining an empty thread in a loop; report microseconds per thread and your CPU/OS. (b) Build a small thread pool (fixed workers, a mutex+condvar work queue) and run a large number of tiny tasks through it, then run the same tasks one-thread-per-task; report the per-task times and the speedup. (c) Implement a fork–join parallel sum of a large array for `P = 1, 2, 4, 8` threads and plot speedup; note where it bends below linear. Warm up and average your runs, and state that absolute numbers are hardware- and OS-dependent.

Chapter 26 — Scalability: Amdahl's Law, Contention, and False Sharing

Before you start: Chapter 25 (fork–join gave 1.98× on two threads but only 4.38× on eight — this chapter explains the gap), Chapter 6 (the cost model — speedup is a ratio you measure, not assume), Chapter 3 (caches and the 64-byte line — false sharing lives here), Chapter 20 (a critical section is serialized — that is a serial fraction). · The last chapter taught you to *express* parallel work; this one asks the harder question — how much of it actually pays off, and what silently steals the rest.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 20 — a critical section runs one thread at a time; if every thread must pass through the same lock, what happens to parallel speedup? (2) Chapter 3 — a cache coherence protocol keeps one value per line consistent across cores; what must happen when one core writes a line another core holds? (3) *Predict:* eight threads each increment their *own* separate counter, a billion times. If the eight counters sit in one array (adjacent in memory) versus each padded onto its own 64-byte cache line, how much slower do you think the adjacent version runs — 1.1×, 2×, or 10×? Write your guess; §26.4 measures it.

Adding cores is supposed to make a program faster. Chapter 25 showed it does — up to a point, and then the curve bends. This chapter is about that ceiling and the two forces that push you well below it. The first ceiling is arithmetic and unavoidable: any part of the program that must run serially caps the whole thing, no matter how many cores you throw at it — that is **Amdahl's law** (§26.1). Its optimistic counterpart, **Gustafson's law**, observes that in practice we grow the problem to fit the machine, which changes the accounting (§26.2). Then the two silent killers, both of which turn "add a core" into "go slower." **Lock contention**: when threads pile up on the same lock, the critical section becomes a serial bottleneck and scaling goes *negative* (§26.3). And **false sharing**: two threads touching *different* variables that happen to share a cache line ping-pong that line between cores, paying coherence traffic for data they never actually share (§26.4). We close the concurrency part by carrying forward a discipline: measure speedup honestly, and hunt the serial fraction (§26.5).

26.1 Amdahl's law: the serial fraction is the ceiling

Suppose a fraction s of a program's runtime is inherently serial — it cannot be parallelized — and the remaining $1 - s$ parallelizes perfectly across P processors. The parallel part shrinks to $(1 - s)/P$, but the serial part s does not move. So the speedup is

```
speedup(P) = 1 / ( s + (1 - s)/P )
```

This is **Amdahl's law** (Gene Amdahl, 1967). Its brutal consequence is the limit as $P \to \infty$: the $(1 - s)/P$ term vanishes, and speedup tops out at $\mathbf{1/s}$. If 10% of your program is serial, the most you can ever get — with infinite cores — is 10×. Not 100×, not 1000×: ten. The serial fraction, not the core count, is the ceiling. Measure it: a compute-bound workload (no shared memory, so we isolate the serial fraction from the contention effects below) with a serial section of about 9% and the rest split across P threads (gcc 11.4.0, `-O2 -pthread -lm`, this machine; illustrative):

```
measured serial fraction s = 0.089   (ceiling 1/s = 11.2x)
P= 2 : speedup 2.01x   Amdahl predicts 1.84x
P= 4 : speedup 3.20x   Amdahl predicts 3.16x
P= 8 : speedup 4.58x   Amdahl predicts 4.93x
P=16 : speedup 6.37x   Amdahl predicts 6.85x
```

The measured speedups track Amdahl's prediction closely and are visibly bending toward the 11.2× ceiling set by the 9% serial part — sixteen threads bought only 6.4×, and no number of threads would break 11.2×. This reframes optimization: to make a parallel program scale, attack the *serial fraction* — the sequential setup, the single lock every thread passes through, the one-threaded merge at the end. Doubling cores helps less and less; shrinking s raises the ceiling for everyone. Amdahl's law is why “we'll just add more cores” is rarely the answer, and why profiling to find what is *not* parallel is the highest-leverage move.

QUICK CHECK

Write Amdahl's law for speedup on P processors with serial fraction s. What is the limit as $P \to \infty$? If a program is 5% serial, what is the best possible speedup, and does buying a 128-core machine change that ceiling? (Answer after the next section.)

26.2 Gustafson's law: grow the problem with the machine

Amdahl's law fixes the *problem size* and asks how much faster a bigger machine runs it — a pessimistic framing, and for a fixed problem, an honest one. But John Gustafson (1988), working on a 1024-processor machine at Sandia, observed that people do not buy bigger machines to run the *same* problem faster; they buy them to run *bigger* problems in the same time. When you scale the problem up with the processor count, the serial part (setup, I/O) tends to stay roughly constant while the parallel part grows with the data. Under that assumption, **Gustafson's law** gives the *scaled* speedup:

```
scaled_speedup(P) = P - s*(P - 1)     (s = serial fraction of the scaled work)
```

This grows nearly linearly with P — no fixed ceiling — because as the problem grows, the parallel fraction grows too, and the constant serial part becomes a smaller share of a larger job. Gustafson reported over 1000-fold speedups on 1024 processors precisely this way. The two laws are not contradictory; they answer different questions. **Amdahl** is *strong scaling*: fixed problem, more cores — the serial fraction caps you at $1/s$. **Gustafson** is *weak scaling*: problem grows with cores — scaling stays near-linear. Which one governs depends on what you actually do when you get a bigger machine. If your problem size is fixed (render *this* frame, sort *this* file), Amdahl rules and you must attack the serial fraction. If it grows (train on more data, simulate a finer grid), Gustafson says throwing cores at a proportionally bigger problem keeps paying off. The practical lesson is to know which regime you are in before you reason about whether more cores will help.

> **THE SAME IDEA THREE WAYS: THE SERIAL FRACTION GOVERNS SCALING**
>
> **1. In words.** For a fixed problem, speedup is capped by the serial fraction at $1/s$ (Amdahl) — the part that can't parallelize dominates as cores grow. If instead you grow the problem with the machine, the parallel part grows too and the constant serial part shrinks in relative terms, so speedup stays near-linear (Gustafson).
>
> **2. As a picture.** Figure 26.1: the serial slice (dark) stays fixed while the parallel slice (light) shrinks as you add cores — Amdahl; or the parallel slice grows with the problem while the serial slice stays put — Gustafson.
>
> **3. As numbers.** With s = 0.089, Amdahl caps speedup at 11.2× (16 threads got 6.4×, tracking the prediction). Gustafson: $P - s(P-1)$ at P = 16 with the same s is 14.7× — near-linear, because the work grew with the cores.

*(Answer to the §26.1 quick check: speedup(**P**) = 1/(**s** + (1−**s**)/**P**); as **P** → ∞ it approaches 1/**s**. At 5% serial the ceiling is 1/0.05 = 20×, and a 128-core machine cannot beat it — the serial fraction, not the core count, sets the limit.)*

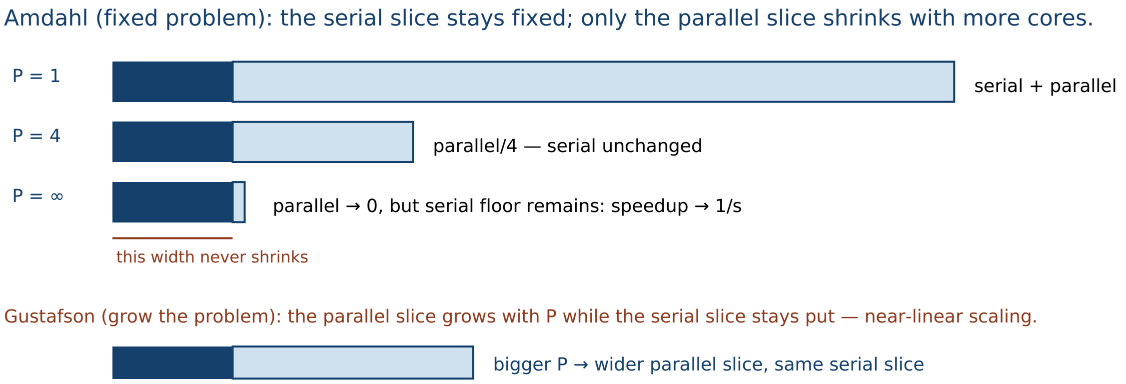


Figure 26.1: Amdahl vs Gustafson. Under Amdahl (fixed problem) the serial slice is a fixed floor, so speedup approaches $1/s$ no matter how many cores divide the parallel slice. Under Gustafson (problem grows with the machine) the parallel slice grows while the serial slice stays constant, so scaled speedup stays near-linear.

26.3 Contention: the critical section is a serial fraction

Amdahl's serial fraction is often not an explicit sequential block — it is *hidden inside a lock*. A critical section runs one thread at a time (Chapter 20), so if every thread must pass through the same lock on every operation, that lock *is* the serial fraction, and Amdahl's ceiling applies to it. Worse, contention does not merely fail to speed up — it can make the program *slower* as you add threads, because the lock itself becomes a contended cache line that bounces between cores, and threads spend their time waiting and handing off rather than working. Measure the extreme case: many threads incrementing one shared counter, each increment under one global mutex (gcc 11.4.0, `-O2 -pthread`, this machine):

```
one global mutex, 40M increments total:
P= 1 : 1.80 s   speedup 1.00x
P= 2 : 5.42 s   speedup 0.33x
P= 4 : 5.52 s   speedup 0.33x
P= 8 : 7.06 s   speedup 0.26x
```

Adding threads made it *nearly three times slower*, not faster. This is **negative scaling**: the entire work is inside the critical section, so it is 100% serial (Amdahl ceiling 1×), and on top of that the lock's cache line ping-pongs among the contending cores, adding coherence overhead that grows with P. The lesson is that a lock on a hot path is a scalability tax, and the fixes are all about *shrinking or removing the serial fraction*: make the critical section smaller (hold the lock for less), shard the state so threads touch different locks (per-thread or per-bucket counters combined at the end), or use a lock-free structure (Chapter 23) or per-thread accumulation to remove the shared write entirely. “Just add threads” applied to a contended lock is how you make a program slower and call it optimization.

QUICK CHECK

Why is a lock on a hot path a “serial fraction” in Amdahl's sense? Explain how adding threads to a single-mutex counter can make it *slower*, not just fail to speed up — name what physically bounces between cores. Give one fix that shrinks the serial fraction. (Answer below the communicate box.)

26.4 False sharing: contention you did not write

The most insidious scalability bug involves no shared variable at all. Recall from Chapter 3 that cache coherence operates at the granularity of a **cache line** — 64 bytes on this machine, not one variable. When one core writes any byte of a line, the coherence protocol invalidates that line in every other core's cache; a core that then reads or writes its own, *different* variable in the same line must re-fetch the whole line. So if two threads update two *independent* variables that happen to land in the same 64-byte line, every update by one invalidates the other's cache, and the line ping-pongs between cores on every write — the threads pay the full cost of sharing a variable while sharing nothing. This is **false sharing**. Measure it: eight threads, each incrementing its own counter 200 million times, with the eight counters packed adjacently in an array (they fall in the same one or two lines) versus each padded to its own 64-byte line (gcc 11.4.0, `-O2 -pthread`, this machine, 64-byte lines):

```
packed (false sharing) : 2.56 s
padded (own cache line): 0.25 s
false sharing slowdown : ~10x
```

The *same* arithmetic ran about **ten times slower** purely because the counters shared cache lines. No variable is shared between threads; the bug is entirely in the memory layout. The fix is to **pad and align** hot per-thread data so each thread's data owns its own cache line — the padded version's per-counter struct is exactly 64 bytes. This is the same mechanical-sympathy discipline as Chapter 3 and 5: layout is performance. False sharing is especially dangerous because it is invisible in the source — the code looks embarrassingly parallel and correct, and it *is* correct; it is just slow, and profilers that don't attribute cache-coherence stalls can hide it. Whenever independent threads write to nearby memory — adjacent array slots, fields of a shared struct, a counters table — suspect false sharing and check the line layout.

TRAP: FALSE SHARING — INDEPENDENT DATA COLLIDING ON ONE CACHE LINE (CORRECT BUT 10× SLOW)

You split a counter into one slot per thread precisely to avoid contention: `long counts[NUM_THREADS];`, each thread touches only `counts[my_id]`. There is no shared variable, no data race, the code is correct — and it is several times slower than you expect and gets *worse* as you add threads. The reason is that `counts` is a dense array, so eight `long`s fit in one or two 64-byte cache lines, and every thread's write to its own slot invalidates the whole line in every other thread's cache — the line ping-pongs between cores on every increment. The threads share a cache line even though they share no data, so they pay the full coherence cost of true sharing. The tell is that padding each slot to its own line — `struct { long v; char pad[56]; } counts[NUM_THREADS];`, or C11 `alignas(64)` / `__attribute__((aligned(64)))` — makes the slowdown vanish (the §26.4 measurement went from 2.56 s to 0.25 s). The discipline: hot, frequently-written per-thread data must not share a cache line with another thread's hot data; align it to the line size, and be suspicious of any "per-thread" array of small elements. It is a correctness-preserving performance bug, which is why it survives testing and hides in production.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I parallelized the histogram: each of the 8 threads owns one entry of a `long hist[8]` array, so there's no lock and no shared variable — but with 8 threads it's barely faster than 1, and with 16 it's actually slower. The threads are totally independent, so I don't get it." Cover the paragraph and respond in four or five sentences, drawing on §26.3–§26.4.

Model answer. "The threads are independent in the source but not in the cache: those eight `long`s sit in one 64-byte cache line, so every increment by any thread invalidates that line in every other core, and the line ping-pongs between cores on every write — that's false sharing, and it costs as much as truly sharing the variable. Nothing is wrong with correctness; the bug is the memory layout. Pad each counter onto its own cache line — an `alignas(64)` struct per thread, or 64 bytes of stride — and the slowdown disappears; in a similar test that took a per-counter workload from ~2.5 s to ~0.25 s. Adding a sixteenth thread made it worse because you added another core fighting over the same line, more coherence traffic, not more work done. The rule is that hot per-thread data must own its cache line." Naming false sharing, the 64-byte line, and the padding fix — and distinguishing it from a correctness bug — is the senior register.

*(Answer to the §26.3 quick check: a lock serializes its critical section — one thread at a time — so a lock every thread passes through on the hot path is a serial fraction, and Amdahl caps speedup at 1 over that fraction. Adding threads makes it slower because the lock's own **cache line** bounces between the contending cores (coherence traffic) and threads spend time waiting and handing off rather than working, so overhead grows with **P**. A fix that shrinks the serial fraction: shard the state into per-thread or per-bucket counters combined at the end, or shrink/remove the critical section with a lock-free or per-thread approach.)*

26.5 What to carry forward

Parallel speedup has a ceiling, and it is set by the part that does *not* parallelize. **Amdahl's law** — speedup(P) = $1/(s + (1-s)/P)$ — says a fixed problem with serial fraction s tops out at $1/s$ no matter how many cores you add (our 9%-serial workload bent toward 11.2× and 16 threads got only 6.4×). **Gustafson's law** — $P - s(P-1)$ — answers the other question: grow the problem with the machine and scaling stays near-linear, because the parallel work grows while the serial part stays constant. Strong scaling (fixed problem) is Amdahl; weak scaling (growing problem) is Gustafson; know which you are in. Two forces push you below even Amdahl's ceiling. **Lock contention** turns a hot lock into a serial fraction *and* a bouncing cache line, so adding threads can go *negative* — our single-mutex counter went from 1.8 s at one thread to 7.1 s at eight. **False sharing** makes independent threads collide on a shared 64-byte cache line, paying the full cost of sharing while sharing nothing — our packed counters ran ~10× slower than the padded ones, fixed by aligning hot per-thread data to its own line.

This closes the concurrency and parallelism part, and with it Volume 1's tour of how the machine really behaves. The through-line of the whole part has been a single, unforgiving fact: the abstract machine's simple, sequential, single-memory model is not what runs, and every gap between the model and the metal — reordering, the store buffer, the coherence protocol, the cost of a thread, the serial fraction hiding in a lock — is where concurrency's bugs and slowdowns live. You now hold the full shared-memory toolkit: threads and their hazards (Chapter 15), locks and condition variables (Chapters 20–21), atomics and lock-free structures (Chapters 22–23), the memory model that says when a write is visible (Chapter 24), the pool-and-task machinery that structures parallel work (Chapter 25), and now the scalability laws that tell you how much of it pays off. Every one of these correctness properties you have had to prove by hand and hold in your head — which data is shared, which access needs a lock, which ordering an atomic requires, whether padding avoids false sharing. The next part turns to a language built to encode those very invariants in its type system, so that data-race freedom and memory safety become properties the compiler checks rather than disciplines you maintain: Rust, and it begins with the idea beneath all of it — ownership.

CAN YOU… WITHOUT LOOKING?

- State Amdahl's law and its $1/s$ limit, and explain why the serial fraction — not the core count — is the ceiling.
- State Gustafson's law and explain how it answers a different question (weak vs strong scaling).
- Explain why a hot lock is a serial fraction and how contention can make a program slower as threads are added.
- Define false sharing, name the granularity (the 64-byte cache line) that causes it, and give the padding/alignment fix.
- Given a scaling problem, say whether to suspect the serial fraction, lock contention, or false sharing, and why.
- Explain why false sharing is a correctness-preserving performance bug that hides in testing.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** State Amdahl's law and its limit as $P \to \infty$; what does it imply about a 5%-serial program? (§26.1)
2. **R2.** State Gustafson's law and explain how it differs from Amdahl (weak vs strong scaling). (§26.2)
3. **R3.** Why is a hot lock a serial fraction, and how can contention cause negative scaling? (§26.3)
4. **R4.** What is false sharing, what granularity causes it, and what is the fix? (§26.4)
5. **R5.** Contrast lock contention and false sharing — what is actually shared in each? (§26.3–§26.4)

FURTHER READING

- **Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," *AFIPS Conference Proceedings*, vol. 30 (1967), pp. 483–485, DOI 10.1145/1465482.1465560.** The original statement of the serial-fraction ceiling of §26.1. First author Gene M. Amdahl; verified via dblp and the ACM Digital Library.
- **Gustafson, "Reevaluating Amdahl's Law," *Communications of the ACM* 31(5):532–533 (1988), DOI 10.1145/42411.42415.** The scaled-speedup (weak-scaling) rejoinder of §26.2, from the 1024-processor experiments at Sandia. First author John L. Gustafson; verified via the ACM Digital Library.
- **Hill & Marty, "Amdahl's Law in the Multicore Era," *IEEE Computer* 41(7):33–38 (2008), DOI 10.1109/MC.2008.209.** Amdahl's law applied to multicore chip design — how the serial fraction shapes core-count trade-offs. First author Mark D. Hill; verified via IEEE Xplore.
- **Drepper, "What Every Programmer Should Know About Memory" (2007, Red Hat; freely available online).** Section 6 covers false sharing and cache-line effects behind §26.4 in depth. A technical whitepaper (no DOI); widely cited reference, verify the current PDF.

Exercises

26.1 WARM-UP Write Amdahl's law for speedup on P processors with serial fraction s, and state its limit as $P \to \infty$.

26.2 WARM-UP Define *false sharing* in one sentence, and name the memory granularity (and its size on a typical x86 machine) that causes it.

26.3 WARM-UP In one sentence, why is a lock that every thread passes through on the hot path a "serial fraction" in Amdahl's sense?

26.4 CORE A program is 20% serial. (a) Compute the best possible speedup with infinite cores. (b) Compute the actual speedup with $P = 4$ and $P = 16$ from Amdahl's law. (c) Going from 4 to 16 cores quadruples the hardware — by what factor does the speedup actually improve, and what does that say about "just add cores"?

26.5 CORE Explain, in Amdahl's terms, why the single-mutex counter of §26.3 exhibited *negative* scaling (1.80 s at one thread, 7.06 s at eight). (a) What is the serial fraction of that workload? (b) Beyond the serial fraction, what extra cost grows with the number of threads, and what physically causes it? (c) Give two distinct fixes and say what each does to the serial fraction.

26.6 CORE Two threads each increment their own `long` counter a billion times. Version A stores them as `long c[2];`; version B pads each onto its own 64-byte cache line. (a) Which is faster and by roughly how much (cite the §26.4 result), and why — what happens to the cache line on each write? (b) Confirm there is no data race or shared variable in either version. (c) Show the padding fix in code (a struct with padding, or an alignment attribute).

26.7 COST For a workload with serial fraction s and perfectly parallel remainder: (a) Derive the efficiency *speedup(P)/P* as a function of s and P, and explain why efficiency falls as P grows for any $s > 0$. (b) At what point does buying more cores stop being cost-effective, in terms of efficiency? (c) Contrast this with Gustafson's regime: why can efficiency stay high under weak scaling even though Amdahl's efficiency falls?

26.8 FADED *Worked, with the last step for you.* A parallel word-count gives each of 8 threads a slot in `long counts[8]` to tally its chunk. It is correct but barely faster than single-threaded, and adding threads makes it worse.
Step 1 (given): There is no lock and no shared variable — each thread writes only `counts[my_id]`, so it is not lock contention.
Step 2 (given): The eight `long`s occupy 64 bytes — one cache line — so all eight slots live on the same line.
Step 3 (given): Each thread's write invalidates that line in every other core's cache, so the line ping-pongs between cores on every increment.
Step 4 (you): Name the phenomenon, explain why more threads make it worse, and give the precise fix (including how you would lay out or align the counters), citing what the §26.4 measurement showed the fix is worth.

26.9 MIXED For each scaling problem, name the most likely cause — *serial fraction (Amdahl)*, *lock contention*, or *false sharing* — and one diagnostic that would confirm it: (a) speedup plateaus at ~10× regardless of cores, with no lock on the hot path; (b) throughput *drops* as threads are added, and a profiler shows most time in `pthread_mutex_lock`; (c) an embarrassingly parallel loop over a per-thread results array barely scales, with no locks anywhere; (d) a single-threaded merge at the end takes a fixed 2 seconds no matter how fast the parallel phase runs.

26.10 MIXED Drawing on Chapters 20, 23, and 26, classify each as *true* or *false* and fix the false ones: (a) “With enough cores, any parallel program can reach arbitrarily high speedup.” (b) “Gustafson's law contradicts Amdahl's law.” (c) “False sharing is a data race.” (d) “A lock on the hot path can make a program slower as you add threads.” (e) “Padding per-thread counters to separate cache lines changes the program's results.”

26.11 STRETCH Amdahl and Gustafson give different answers because they hold different things fixed. (a) Show algebraically that if you fix the problem size and let P grow, Amdahl's $1/s$ ceiling follows; if instead you fix the *time* and grow the problem with P, Gustafson's near-linear scaled speedup follows. (b) A team reports “linear speedup to 1000 cores” on their simulation. Explain how that can be true under Gustafson yet not contradict Amdahl, and what question you would ask to tell which regime they measured. (c) Connect this to Chapter 25's span bound $O(T_1/P + T_\infty)$: how is the span like Amdahl's serial fraction?

26.12 LAB On your own machine: (a) build a workload with a tunable serial fraction (a fixed serial section plus a parallel section split across P threads), measure speedup for P = 1, 2, 4, 8, 16, and compare to Amdahl's prediction for your measured s. (b) Increment one shared counter under a single mutex with 1, 2, 4, 8 threads and observe whether scaling goes negative. (c) Have N threads each increment their own counter a billion times, packed in an array versus padded to 64-byte lines, and report the false-sharing slowdown. Label your CPU (and its cache-line size) and note that the numbers are hardware-dependent.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART V

Rust for Systems

Ownership, borrowing, and lifetimes — memory safety without a GC, and how the type system encodes the invariants the previous parts made you prove by hand.

Chapter 27 — Ownership and Moves

Before you start: Chapter 8 (the stack and the heap — ownership is about who is responsible for heap memory), Chapter 9 (`malloc`/`free` and what goes wrong — use-after-free, double-free, leaks are exactly what ownership prevents), Chapter 24 (a data race is undefined behavior — the memory model you had to obey by hand, a type system can enforce), Chapter 10 (undefined behavior and the optimizing compiler — the failure mode ownership rules out). · This chapter opens the Rust part; every hazard the C parts made you track by hand, Rust encodes as a rule the compiler checks — and it all rests on one idea: ownership.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 9 — what are the three classic bugs of manual `malloc/free` memory management, and which one does calling `free` twice on the same pointer cause? (2) Chapter 8 — when does a stack variable's storage get reclaimed, and does the compiler need you to say so? (3) *Predict:* in C, you write `char *t = s;` then `free(s)` then use `t` and `free(t)`. Does that *compile*? Does it *run* correctly? In Rust, the closest equivalent — `let t = s;` then using `s` — will the compiler accept it? Write your guesses; §27.1 and §27.3 answer all three.

A note on this part before we begin. The C parts of this book gave you no safety net on purpose: you allocated and freed by hand, reasoned about lifetimes yourself, and proved data-race freedom by careful thought (Chapter 24). Rust keeps the same low-level control — no garbage collector, the same machine model, predictable performance — but moves those proofs into the type system, so the compiler rejects the programs you would otherwise have to keep correct by discipline. The engine of that is **ownership**, and this chapter is only about ownership and its most surprising consequence, **moves**. First, the problem ownership solves — the manual-memory bugs of Chapter 9, shown in real C that compiles and then crashes (§27.1). Then the three **ownership rules** and how "dropped when the owner goes out of scope" gives automatic, deterministic cleanup with no GC (§27.2). Then **move semantics**: assigning or passing a value transfers ownership and *invalidates the source*, so use-after-move is a compile error, not a runtime crash (§27.3). Then ownership across function boundaries, and a first glimpse of why this same rule makes data races impossible (§27.4). We close by carrying ownership forward toward borrowing — the way to use a value without taking ownership (§27.5).

On the code in this part. The book's environment has a C toolchain (gcc 11.4.0) but *no Rust compiler installed*, so the C programs below were compiled and run and their output is real, whereas the Rust snippets were *not* compiled here; their behavior and the quoted compiler diagnostics are drawn from *The Rust Programming Language* (Klabnik & Nichols, 2nd ed., 2023) and the official Rust documentation, and are labeled as such. Nothing about Rust's behavior below is invented — it is cited.

27.1 The problem ownership solves

Recall the three failure modes of manual memory management from Chapter 9: **use-after-free** (touching memory you already returned), **double-free** (returning the same block twice), and **leaks** (never returning it). All three come from the same root: in C, freeing memory is decoupled from any notion of *who owns it*, so nothing stops two pointers from aliasing one block and both trying to manage it. Here is the bug in real C — it compiles cleanly with no warning, and then aborts at run time (gcc 11.4.0, `-O2`, Linux 6.18):

```
char *s = malloc(16);
strcpy(s, "hello");
char *t = s;           /* alias: t and s point at the SAME buffer */
free(s);               /* one "owner" frees it */
printf("%s\n", t);     /* use-after-free through t  — undefined behavior */
free(t);               /* double free */

$ gcc -O2 usefree.c -o usefree      # compiles, exit 0, no warning
$ ./usefree
free(): double free detected in tcache 2
Aborted (core dumped)               # exit 134 (SIGABRT)
```

The compiler accepted it — the aliasing and the second `free` are invisible to the C type system — and the program died at run time (a lucky outcome; use-after-free often *silently* corrupts instead). The root cause is that `s` and `t` both claim the right to free the same buffer, and C has no way to express that only one of them should. Rust's answer is to make *ownership* a first-class, compiler-enforced concept: every value has exactly one owner, responsibility for freeing follows the owner, and the type system tracks it. The promise is **memory safety without a garbage collector** — the same manual, predictable, no-runtime layout as C, but with use-after-free, double-free, and leaks of owned memory ruled out at compile time. That the guarantee is real, not just a convention, was established formally by the RustBelt project (Jung et al., 2018), which gave a machine-checked safety proof for a realistic subset of the language. The rest of this chapter is how the rule works.

QUICK CHECK

In the C snippet, name the two distinct bugs that occur once `t` aliases `s` and `s` is freed. Why can the C compiler not catch either? State Rust's one-sentence promise about memory safety and the garbage collector. (Answer after the next section.)

27.2 The ownership rules and drop

Rust's model is three rules, stated in *The Rust Programming Language* (Klabnik & Nichols, 2023, ch. 4):

```
1. Each value in Rust has an owner.
2. There can only be one owner at a time.
3. When the owner goes out of scope, the value is dropped.
```

“Dropped” means Rust automatically runs cleanup — for a heap-owning type like `String`, its `drop` frees the heap buffer — at the exact point the owning variable's scope ends. This is **RAII** (resource acquisition is initialization, the C++ idea from Chapter 12's arena discussion): cleanup is tied to scope, not to a manual call. Because there is exactly *one* owner (rule 2), the buffer is freed exactly *once* (rule 3) — no double-free — and because drop is automatic at end of scope, it is never forgotten — no leak of owned memory. Consider (Rust; not compiled here — behavior per the cited book):

```
{
    let s = String::from("hello");   // s owns a heap buffer
    // ... use s ...
}                                    // s goes out of scope here: Rust calls drop(s),
                                     // which frees the buffer — automatically, exactly once
```

No `free` call appears, yet the memory is released deterministically at the closing brace — not by a garbage collector scanning the heap later, but by a `drop` the compiler inserts at compile time, at a place you can point to in the source. This is the crucial difference from a GC language: the cleanup is *deterministic* and has the same performance characteristics as the manual `free` you would have written in C, without your having to write it or risk forgetting it. Ownership turns "remember to free exactly once, at the right time" — a discipline you maintained by hand and sometimes got wrong — into a property of the program's structure that the compiler guarantees. The single-owner rule is what makes it sound; the next section shows the price it extracts, which is that ordinary assignment does not mean what it means in C.

THE SAME IDEA THREE WAYS: ONE OWNER FREES EXACTLY ONCE, AUTOMATICALLY

1. In words. Every value has exactly one owner; when the owner's scope ends, the value is dropped (its resources freed). One owner means one free — no double-free — and automatic drop means no forgotten free — no leak — all decided at compile time, with no garbage collector.

2. As a picture. Figure 27.1: a `String` value is a small stack record (pointer, length, capacity) owning a heap buffer; when the owning variable leaves scope, `drop` frees the buffer — one arrow, one free.

3. As numbers/contrast. The C snippet of §27.1 has two pointers to one buffer and calls `free` twice → abort. The Rust equivalent has one owner and one automatic drop → freed once; the aliasing that caused the double-free cannot even be written (see §27.3).

(Answer to the §27.1 quick check: once `t` aliases `s` and `s` is freed, using `t` is a ***use-after-free*** *and the second `free(t)` is a* ***double-free****. The C compiler cannot catch either because its type system does not track ownership or lifetimes — a pointer carries no information about whether the memory it points to is still valid or who is responsible for freeing it. Rust's promise: memory safety — no use-after-free, double-free, or leak of owned memory — with no garbage collector, enforced at compile time.)*

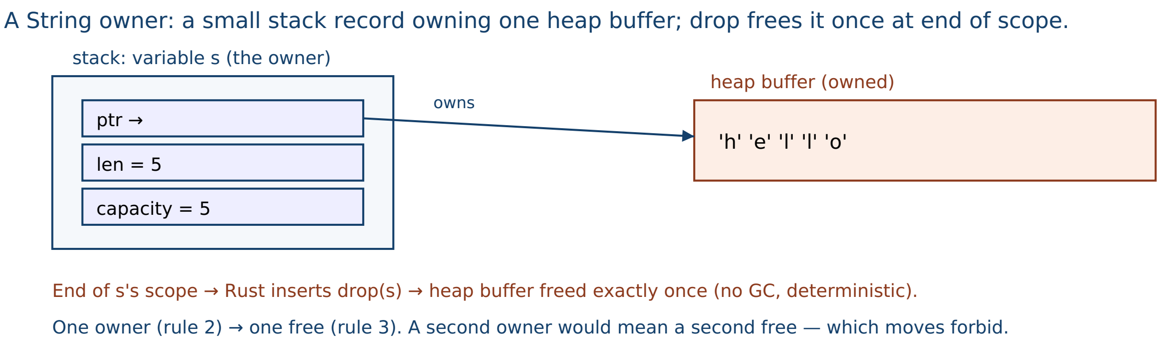


Figure 27.1: A `String` owner. The stack record (pointer, length, capacity) owns a heap buffer; when the owning variable leaves scope, the compiler-inserted `drop` frees the buffer exactly once. One owner means one free — the structural reason double-frees and leaks of owned memory cannot occur.

27.3 Moves: assignment transfers ownership

Here is where Rust departs sharply from C and even C++. In C, `char *t = s;` copies the pointer, giving two pointers to one buffer — the aliasing that caused §27.1's double-free. Rust cannot allow that, because two owners would mean two drops. So for a heap-owning type, assignment is a **move**: it transfers ownership to the destination and *invalidates the source*. After the move the source is no longer a valid owner, and any use of it is a compile error. Consider (Rust; not compiled here — diagnostic quoted from the cited documentation):

```
let s1 = String::from("hello");
let s2 = s1;            // MOVE: ownership transfers from s1 to s2; s1 is now invalid
println!("{s1}");       // error: use of moved value

error[E0382]: borrow of moved value: `s1`
  = note: move occurs because `s1` has type `String`, which does not implement the `Copy`
trait
```

The program does not compile: after `let s2 = s1;`, the value has moved, so `s1` is dead and touching it is rejected at compile time with error `E0382`. This is the whole game. The C double-free of §27.1 is *unwriteable* in Rust: the moment you make a second binding, the first stops being an owner, so there is never a moment when two owners could both free. The bug that compiled-and-crashed in C is caught before the program ever runs — and note that this is a *compile-time* rejection, costing nothing at run time. Two refinements complete the picture. If you genuinely want two independent copies, you ask for one explicitly with `.clone()`, which allocates a new buffer and deep-copies — now there are two owners of two *separate* buffers, each dropped once. And small values that live entirely on the stack — integers, `bool`, `char`, and tuples of them — implement the `Copy` trait, so `let y = x;` *copies* them instead of moving (there is no heap buffer to double-free, so copying is safe and cheap); `x` stays usable. The rule of thumb: values that own a resource move; plain stack data copies.

QUICK CHECK

In Rust, after `let s2 = s1;` where `s1` is a `String`, what is the status of `s1`, and what happens if you use it? Why does the same pattern work fine for `let y = x;` when `x` is an `i32`? How do you get two independent `String`s from one? (Answer below the communicate box.)

27.4 Ownership across functions, and a first look at safety

Ownership follows the same move rule across function boundaries: passing a value to a function *moves* it into the function (unless the type is `Copy`), so the caller can no longer use it; returning a value moves ownership back out. This makes the flow of responsibility explicit in the signatures (Rust; not compiled here — behavior per the cited book):

```
fn consume(s: String) {          // s is moved IN; consume now owns it
    println!("{s}");
}                                // s dropped here — buffer freed

let a = String::from("hi");
consume(a);                      // a is MOVED into consume
// println!("{a}");              // error[E0382]: a was moved, no longer valid
```

If a function needs a value but should not take ownership, moving it in and out by value is clumsy — which is exactly the motivation for *borrowing*, the subject of the next chapters: a way to lend access without transferring ownership. But ownership already buys something profound beyond memory safety. Because a moved-from value is dead, and because there is only ever one owner, ownership controls *aliasing* — and aliasing plus mutation is the root of both memory-corruption bugs and data races (Chapter 24). A taste: when you hand a value to another thread, Rust *moves* it there, so the original thread can no longer touch it — two threads cannot both own and mutate the same value by accident, which is the beginning of how Rust makes data races a compile error rather than the undefined behavior you had to prevent by hand in Chapter 24. The full story needs borrowing and the `Send`/`Sync` traits, developed in the chapters ahead. The point to hold now is that ownership is not merely a memory-management scheme; it is a discipline on aliasing, and aliasing is where the hazards of the whole first volume lived.

TRAP: EXPECTING ASSIGNMENT OR A FUNCTION CALL TO COPY (IT MOVES, AND THE SOURCE GOES DEAD)

Coming from C or a garbage-collected language, the deepest habit to unlearn is that `let b = a;` or `f(a)` leaves `a` usable. For an owning type in Rust it does not: the value *moves*, and `a` is invalidated — using it afterward is a compile error (`E0382`, “borrow of moved value”). Beginners read this as the compiler being obstinate and reach for the wrong fixes: sprinkling `.clone()` everywhere to silence it (correct but often needlessly copies heap data, throwing away the performance you came to Rust for), or fighting the borrow checker without understanding that the error is *telling them something true* — that they tried to use a value whose ownership they gave away. The right response is to decide, at each move, what you actually mean: if the callee should take over the value, move is correct and you simply must not use the source again; if the caller still needs it, you want to *borrow* it (lend a reference — the next chapters) rather than move it; and only if you truly need a second independent value should you `clone`. The move error is not noise; it is the ownership rule catching an aliasing decision you have to make consciously. Reaching for `clone` to make the compiler quiet is the analog of casting away a warning in C — it works and hides the point.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate porting C to Rust says: “This is ridiculous — I wrote `let b = a; use(a);` and it won't compile, saying `a` was moved. In C this just copies the pointer and works. I've been adding `.clone()` everywhere to get it to build. Isn't the borrow checker just getting in the way?” Cover the paragraph and respond in four or five sentences, drawing on §27.1–§27.3.

Model answer. “In C, `let b = a;` copying the pointer is exactly the bug that bites you later — now two pointers own one buffer, and whoever frees second gets a double-free or use-after-free, which is what that C program did at run time. Rust makes assignment of an owning type a *move*: ownership transfers to `b` and `a` goes dead, so there is never a moment when two owners could both free — the crash is turned into a compile error. So the checker isn't in the way; it's reporting that you used a value after giving its ownership away. `clone` silences it by making a real second buffer, which is sometimes right but often wasteful — if you only need to *look at* the value in `use()`, you want to borrow a reference, not clone. Decide per case: move if you're handing it off, borrow if the caller still needs it, clone only if you truly need two independent values.” Framing the move as preventing the C double-free, and clone-vs-borrow as a real decision, is the senior register.

*(Answer to the §27.3 quick check: after `let s2 = s1;` with a `String`, `s1` has been **moved from** and is invalid; using it is a compile error (`E0382`). It works for `let y = x;` with an `i32` because integers implement the `Copy` trait — they live entirely on the stack with no heap resource to double-free, so assignment copies the bits and leaves `x` valid. To get two independent `String`s, call `s1.clone()`, which allocates and deep-copies a separate buffer.)*

27.5 What to carry forward

Ownership is the idea beneath everything in this part. The manual-memory bugs of Chapter 9 — use-after-free, double-free, leaks — all stem from C's having no notion of who is responsible for a piece of memory: our C snippet aliased one buffer with two pointers, compiled without a warning, and aborted at run time on the double-free. Rust replaces the discipline with three compiler-enforced rules: every value has exactly **one**

owner, and when the owner goes out of scope the value is **dropped** — cleanup tied to scope (RAII), deterministic, with no garbage collector, so owned memory is freed exactly once and never forgotten. The consequence is **move semantics**: assigning or passing an owning value *transfers* ownership and *invalidates the source*, so the two-owners situation that caused the C double-free literally cannot be written — using a moved-from value is a compile error (`E0382`), caught before the program runs and at zero runtime cost. Explicit `.clone()` makes a genuine second copy when you want one; small stack types (`Copy`) copy instead of move. And ownership is more than memory management: because a moved-from value is dead and there is only ever one owner, ownership is a discipline on *aliasing* — the root of both corruption and the data races of Chapter 24.

You have now seen the core move, but two things are still missing, and they are what the rest of the part builds. First, constantly moving values in and out of functions just to read them is untenable — you need a way to *use* a value without *owning* it. That is **borrowing**: taking a reference, shared or exclusive, under rules that keep aliasing safe — the next chapter. Second, borrowing raises the question of *how long* a reference may live relative to the value it points at, which is what **lifetimes** make precise — and together, ownership, borrowing, and lifetimes are how Rust's type system encodes, and checks at compile time, the very invariants you spent the C parts proving by hand: that memory is freed once and used only while valid, and that shared, mutable state is not raced upon. Everything you learned about the machine in Volume 1 still holds underneath — Rust compiles to the same instructions, the same cache and memory model — but the proofs have moved from your head into the compiler. We begin, in the next chapter, with borrowing.

CAN YOU… WITHOUT LOOKING?

- Name the three manual-memory bugs ownership prevents, and why C's type system cannot catch them.
- State the three ownership rules and what “dropped when the owner goes out of scope” means (RAII, no GC).
- Explain why single ownership means owned memory is freed exactly once and never leaked.
- Define a *move*: what happens to the source, and why using it afterward is a compile error.
- Distinguish move, `clone`, and `Copy`, and say which applies to a `String` vs an `i32`.
- Explain how ownership controls aliasing, and connect that to preventing data races (Chapter 24).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What three manual-memory bugs does ownership prevent, and why can't the C compiler catch them? (§27.1)
2. **R2.** State the three ownership rules and what "drop" does at end of scope. (§27.2)
3. **R3.** What is a move — what happens to the source — and why is using a moved-from value a compile error? (§27.3)
4. **R4.** Distinguish move, `clone`, and `Copy`; which applies to `String` vs `i32`? (§27.3)
5. **R5.** How does ownership control aliasing, and why does that matter beyond memory management? (§27.4)

FURTHER READING

- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 4 "Understanding Ownership" (also free at `doc.rust-lang.org/book`).** The primary source for the ownership rules, moves, `clone`, and `Copy` of §27.2–§27.3; the quoted rules and `E0382` behavior are from here. Authors and edition verified via the publisher; the book was not compiled in this environment.
- **Jung, Jourdan, Krebbers & Dreyer, "RustBelt: Securing the Foundations of the Rust Programming Language," *Proc. ACM on Programming Languages* 2(POPL), Article 66 (2018), DOI 10.1145/3158154.** The first machine-checked safety proof for a realistic subset of Rust — the formal basis for §27.1's "safety without a GC" claim. First author Ralf Jung; verified via dblp and the ACM Digital Library.
- ***The Rustonomicon* (official, `doc.rust-lang.org/nomicon`).** The advanced guide to ownership, moves, and the memory model underneath `unsafe` Rust — the "why" behind the rules of this chapter. Reference for later chapters; verify against the current edition.
- **Stroustrup, *The C++ Programming Language*, 4th ed. (Addison-Wesley, 2013), on RAII and move semantics.** The C++ lineage of the drop/RAII idea of §27.2 and of moves — useful context for where Rust's model came from.

Exercises

27.1 WARM-UP State the three ownership rules of Rust in your own words.

27.2 WARM-UP In one sentence, what does it mean that a value is "dropped" when its owner goes out of scope, and what C operation does drop correspond to for a heap-owning value?

27.3 WARM-UP After `let s2 = s1;` where `s1` is a `String`, is `s1` still usable? What does the compiler do if you try to use it?

27.4 CORE Take the C snippet of §27.1 (alias, free, use, free). (a) Identify each of the two bugs and the line it occurs on. (b) Explain why it compiles in C despite being wrong. (c) Explain precisely why the analogous Rust code cannot be written — what happens at the second binding that removes the possibility of a double-free?

27.5 CORE Explain how the three ownership rules together guarantee that owned heap memory is freed *exactly once* and is *never leaked*, and why this needs no garbage collector. Which rule prevents the double-free, and which prevents the leak?

27.6 CORE Distinguish *move*, `clone`, and `Copy`. (a) For each, say what happens to the source value and whether a heap buffer is duplicated. (b) Why does `let y = x;` move when `x: String` but copy when `x: i32`? (c) When is reaching for `.clone()` the wrong instinct, and what should you often use instead?

27.7 COST Compare the runtime cost of Rust's ownership checks with a garbage collector. (a) At what time — compile or run — is the move/drop machinery resolved, and what runtime cost does the ownership checking itself add? (b) Contrast the memory-reclamation behavior of `drop`-at-end-of-scope with a tracing GC: which is deterministic, and what does that imply for pause times and peak memory? (c) Why is `.clone()` a real, measurable cost while a move is essentially free — what does each do at the machine level?

27.8 FADED *Worked, with the last step for you.* A Rust newcomer writes `let cfg = load_config();` then `process(cfg);` then `log(cfg);` and gets `error[E0382]: use of moved value: cfg` on the `log` line.
Step 1 (given): `cfg` has an owning type (say `Config`, not `Copy`).
Step 2 (given): `process(cfg)` takes its argument by value, so it *moves* `cfg` into `process`; after that call `cfg` is invalid.
Step 3 (given): `log(cfg)` then tries to use a moved-from value, which the compiler rejects.
Step 4 (you): Give two different correct fixes — one that lets both functions use the value without cloning, and one where cloning is genuinely appropriate — and say how you would decide between them and why blindly cloning is the wrong default.

27.9 MIXED For each, say whether assignment/passing *moves* or *copies*, and whether the source stays usable: (a) `let b = a;` with `a: String`; (b) `let b = a;` with `a: i32`; (c) passing a `Vec<u8>` by value to a function; (d) `let b = a;` with `a: (i32, bool)`. Justify each in a line, naming `Copy` where relevant.

27.10 MIXED Drawing on Chapters 9, 24, and 27, classify each as *true* or *false* and fix the false ones: (a) “Rust achieves memory safety with a garbage collector.” (b) “In Rust, `let s2 = s1;` for a `String` leaves both `s1` and `s2` valid.” (c) “A move error is caught at compile time and costs nothing at run time.” (d) “`.clone()` and a move do the same thing.” (e) “Ownership only matters for memory management, not for concurrency.”

27.11 STRETCH Ownership controls aliasing, and aliasing is the root of both memory-corruption bugs and data races. (a) Explain how the single-owner rule, applied across threads by *moving* a value into a thread, prevents two threads from both owning and mutating the same value. (b) Argue why “aliasing + mutation” is the common cause of both a use-after-free (Chapter 9) and a data race (Chapter 24), and how ownership attacks the shared root rather than the two symptoms separately. (c) What remaining capability does ownership-by-move alone *not* give you — that is, why do you still need borrowing to write useful programs, and what will borrowing have to guarantee to stay safe?

27.12 LAB On your own machine (a C toolchain suffices; a Rust toolchain is ideal if you can install one): (a) In C, compile and run the alias/free/use/free snippet of §27.1; confirm it compiles without warning and observe what happens at run time (a double-free abort or, with a sanitizer such as `-fsanitize=address`, a diagnosed use-after-free). (b) If you have `rustc`, write the corresponding `let s2 = s1; println!("{s1}");` and confirm it fails to compile with `E0382`; if you do not, explain from this chapter exactly which line would be rejected and why. (c) Write down, for three variables in a small program, whether each assignment is a move or a copy, and justify from the type. Note your toolchain versions.

Chapter 28 — Borrowing and References

Before you start: Chapter 27 (ownership and moves — borrowing is the way to use a value *without* taking ownership, so the source stays valid), Chapter 9 (`malloc`/`free` and use-after-free — a reference that outlives or aliases badly is the same bug), Chapter 11 (iterator/pointer invalidation when a buffer is reallocated), Chapter 24 (a data race is two accesses to one location, one a write, unsynchronized — the borrow rule is exactly the aliasing discipline that rules it out). · Chapter 27 left us moving values in and out of functions just to read them; this chapter is how to lend access instead, and the one rule the compiler enforces to keep lending safe.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 27 — after `let s2 = s1;` for a `String`, why is `s1` dead, and what problem would arise if it were *not*? (2) Chapter 11 — in C, you hold a pointer into a growable array and then the array is reallocated larger; what can happen to that pointer? (3) *Predict:* in Rust you take `let r = &v;` (a shared reference to a vector) and then, while `r` is still in use, call `v.push(x)` (which needs to mutate `v`). Does the compiler accept it? Write your guesses; §28.2 and §28.3 answer all three.

Ownership alone is not enough to write real programs. If the only way to give a function access to a value were to *move* it in (Chapter 27), every function that merely reads a value would have to move it back out, and two functions could never look at the same value at once. What you want is to **lend** access without transferring ownership — a **reference** — and the discipline that keeps lending safe is **borrowing**. This chapter is only about references and the borrow rule. First, why references are needed and what a shared reference (`&T`) is: read-only access, any number at once (§28.1–§28.2). Then the mutable reference (`&mut T`): exclusive write access, and why there may be only one, with no other reference alive alongside it (§28.3). Then the borrow checker's single rule — *one writer XOR many readers* — and how it turns the iterator-invalidation and data-race hazards of the earlier volumes into compile errors (§28.4). We close by carrying borrowing forward to the question it forces open: how long may a reference live relative to the value it points at — lifetimes (§28.5).

On the code in this part. The book's environment has a C toolchain (gcc 11.4.0) but *no Rust compiler installed*, so the C programs below were compiled and run and their output is real, whereas the Rust snippets were *not* compiled here; their behavior and the quoted compiler diagnostics are drawn from *The Rust Programming Language* (Klabnik & Nichols, 2nd ed., 2023) and the official Rust error index, and are labeled as such. Nothing about Rust's behavior below is invented — it is cited.

28.1 Why references: using without owning

Recall the awkwardness at the end of Chapter 27: a function that only wants to *read* a `String` — say, to compute its length — must take it by value, which *moves* it in and leaves the caller unable to use it afterward. To keep the value, the function would have to return it back out, threading ownership through every call. That is untenable. A **reference** solves it: `&s` creates a value that *points at* `s` without owning it, so the caller keeps ownership and the function borrows access for the duration of the call (Rust; not compiled here — behavior per the cited book):

```
fn length(s: &String) -> usize {   // s borrows: it does NOT own the String
    s.len()
}                                    // s goes out of scope, but nothing is dropped —
                                     // the reference did not own the buffer

let owner = String::from("hello");
let n = length(&owner);             // lend a reference; ownership stays with `owner`
println!("{owner} has len {n}");     // `owner` is STILL valid — it was never moved
```

The `&` in `&owner` is "take a reference to"; the `&String` in the signature is "a reference to a `String`." The reference is a borrow: the callee may use the value through it, but it does not own the value and so does not drop it when the reference goes out of scope — only the owner drops. Because no move happened, `owner` is still usable after the call. This is the everyday shape of Rust code: pass owning values by reference to read them, and reserve moves for when you genuinely mean to hand off ownership. But a reference is a pointer into someone else's value, and Chapters 9 and 11 are a catalogue of how pointers into other people's memory go wrong — dangling after a free, aliasing a buffer that is then mutated out from under you. The rest of the chapter is the two kinds of reference and the one rule that makes them safe where the C pointer was not.

QUICK CHECK

Why does passing `&owner` to `length` leave `owner` usable afterward, when passing `owner` by value would not? When the reference parameter `s` goes out of scope at the end of `length`, is the `String`'s buffer freed? Why or why not? (Answer after the next section.)

28.2 Shared references: many readers, no writers

The reference of §28.1 is a **shared reference**, written `&T`. Its defining property: you may have *any number* of shared references to a value at the same time, and through a shared reference you may *read* the value but not mutate it. That is safe for the same reason many threads reading (but not writing) shared memory is safe (Chapter 24): if nobody writes, no reader can observe a torn or stale value, and no two accesses conflict. Shared references are `Copy` — cheap to duplicate, since they are just pointers — so handing the same value to several readers is free (Rust; not compiled here — behavior per the cited book):

```
let v = vec![1, 2, 3];
let a = &v;          // shared reference #1
let b = &v;          // shared reference #2 — fine, many readers allowed
println!("{} {}", a.len(), b.len());   // both read v; no conflict
```

Two immutable views of one vector coexist without trouble because neither can change it. The read-only-ness is not a convention you must remember — it is in the type: a `&T` simply has no method that mutates, so an attempt to write through one does not compile.

This is the “many readers” half of Rust's borrowing rule, and it is exactly as permissive as it can safely be: sharing is unlimited precisely while nothing mutates. The moment you need to *change* the value, you need a different, stricter kind of reference — because mutation plus aliasing is where every hazard lived.

> **THE SAME IDEA THREE WAYS: ONE WRITER XOR MANY READERS**
>
> **1. In words.** At any point a value may be borrowed either by any number of shared references (`&T`, read-only) *or* by exactly one mutable reference (`&mut T`, read-write) — never both, never two mutable ones. Sharing is safe while nothing writes; writing is safe only when nothing else can observe the value meanwhile.
>
> **2. As a picture.** Figure 28.1: a value with several thin arrows labelled `&` pointing in (all readers, allowed together), versus the same value with one bold arrow labelled `&mut` and no other arrow permitted (the exclusive writer).
>
> **3. As numbers/contrast.** The C program of §28.4 holds one pointer into a buffer and then reallocates the buffer → the pointer dangles → heap-use-after-free (ASan-confirmed). The Rust equivalent cannot compile: the reallocation needs `&mut`, which the outstanding shared reference forbids (`E0502`) → the bug is rejected before it runs.

(Answer to the §28.1 quick check: passing `&owner` lends a reference and does not move the value, so `owner` keeps ownership and stays valid; passing by value would move the `String` into the function and invalidate `owner`. When the reference parameter `s` goes out of scope, nothing is freed, because a reference does not own the buffer — only the owner drops it, and the owner is still `owner` back in the caller.)

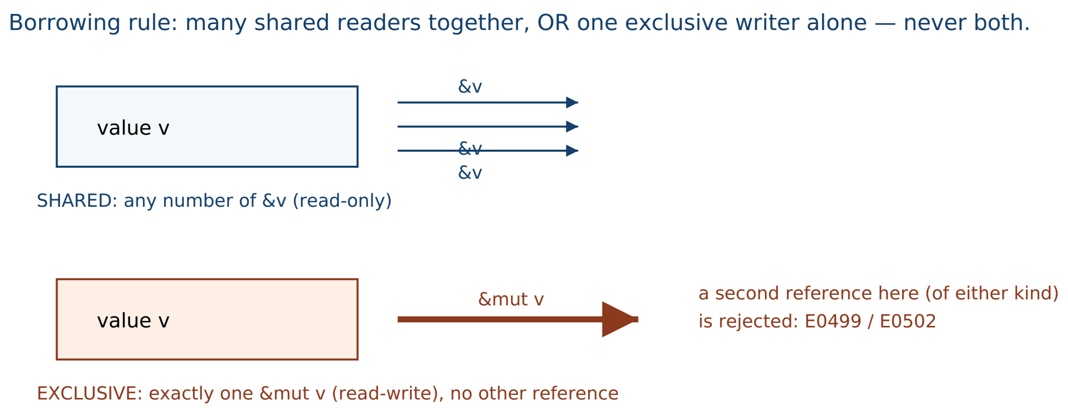


Figure 28.1: The borrowing rule. Top: a value may have any number of shared references (`&T`, read-only) at once. Bottom: or exactly one mutable reference (`&mut T`, read-write), with no other reference of either kind alive alongside it. Never both. This is aliasing XOR mutation, enforced by the type system.

28.3 Mutable references: exactly one writer, no aliases

To mutate a value through a reference you take a **mutable reference**, `&mut T`. Its rule is the opposite of the shared reference's permissiveness: while a `&mut T` to a value exists, it must be the *only* live reference to that value — no second mutable reference, and no shared reference either. A mutable borrow is *exclusive*. The reason is precisely the aliasing-plus-mutation hazard: if two references could reach one value and at least one could write, you would have, in a single thread, the same conflict a data race is across threads (Chapter 24) — a write that another accessor can observe half-done or unexpectedly, or that invalidates a pointer the other holds. Rust rules it out by construction (Rust; not compiled here — diagnostics quoted from the official Rust error index):

```
let mut v = vec![1, 2, 3];
let m = &mut v;          // exclusive mutable borrow of v
let n = &mut v;          // ERROR: a second mutable borrow while m is live
m.push(4);

error[E0499]: cannot borrow `v` as mutable more than once at a time
```

```
let mut v = vec![1, 2, 3];
let r = &v;              // shared (read) borrow
let m = &mut v;          // ERROR: mutable borrow while a shared borrow is live
println!("{}", r[0]);

error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
```

The first is rejected with `E0499` — *"cannot borrow as mutable more than once at a time"* — because two mutable references would be two writers to one value. The second is rejected with `E0502` — *"cannot borrow as mutable because it is also borrowed as immutable"* — because a writer alongside a reader is exactly aliasing-plus-mutation. Both diagnostics are from the official Rust error index. Note what the exclusivity buys: through a `&mut T` the compiler *knows* nothing else can see the value, so mutation through it is as safe and as optimizable as if the value were not shared at all — the reference is effectively a compiler-checked exclusive lock, but one with zero runtime cost, resolved entirely at compile time. The borrow of a value ends when the reference is last used (this is "non-lexical lifetimes": the borrow's span is how long the reference is actually used, not the enclosing braces), so you can take a shared borrow, finish with it, and then take a mutable one — just never overlapping.

> **QUICK CHECK**
>
> State the two-part borrowing rule in one sentence. Which error code fires when you take two `&mut` to one value, and which when you take a `&mut` while a `&` is still live? Why is a `&mut T` being exclusive the single-threaded analog of preventing a data race? (Answer below the communicate box.)

28.4 The borrow checker, and how it kills iterator invalidation and data races

The two rules — many shared XOR one exclusive, and (next chapter) references must not outlive their value — are enforced by the **borrow checker**, a compile-time pass. To see it earning its keep, take a bug from Chapter 11 that C compiles without complaint. Here is real C: hold a pointer into a heap buffer, then grow the buffer with `realloc`, which is free to move it — after which the old pointer dangles. Compiled and run in this environment (gcc 11.4.0):

```
int *v = malloc(4 * sizeof(int));
for (int i = 0; i < 4; i++) v[i] = i * 10;
int *p = &v[0];                    /* alias into the buffer */
v = realloc(v, 4096 * sizeof(int)); /* may move the block, invalidating p */
printf("via p: %d\n", *p);          /* use-after-free if the block moved */
free(v);

$ gcc -O2 -Wall borrow.c -o borrow          # compiles, NO warning
$ gcc -O1 -fsanitize=address borrow.c -o borrow && ./borrow
==ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010
    ... in main
SUMMARY: AddressSanitizer: heap-use-after-free
```

The plain build gives no warning at all; only AddressSanitizer, at run time, catches the **heap-use-after-free**. This is the iterator-invalidation hazard of Chapter 11 in its rawest form — a live pointer into a container that is then reallocated. Now the Rust equivalent. A `Vec`'s `push` may reallocate, so its signature takes `&mut self`; holding a reference into the vector means holding a `&` borrow of it. The two cannot overlap (Rust; not compiled here — diagnostic per the official Rust error index):

```
let mut v = vec![1, 2, 3];
let p = &v[0];          // shared borrow: a reference INTO the vector
v.push(4);               // push needs &mut v — conflicts with the live borrow p
println!("{}", p);       // use of p after the (attempted) reallocation

error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
```

The exact bug the C program hid until run time — a reference into a buffer that is then reallocated — is a compile error in Rust, `E0502`, because `push`'s mutable borrow cannot coexist with the shared borrow `p`. The borrow checker does not need to know that `push` reallocates; the rule “no mutable borrow while a shared borrow is live” is enough. And this is the same rule that forbids data races: a data race (Chapter 24) is two accesses to one location, at least one a write, without synchronization — which is aliasing (two accessors) plus mutation (a write). Rust's borrow rule bans exactly that combination for a single value, and, extended across threads by the `Send`/`Sync` traits (a later chapter), it is why safe Rust cannot have a data race. The undefined behavior you proved absent by hand in Chapters 11 and 24 is, here, a property the compiler checks.

TRAP: MUTATING A COLLECTION WHILE A REFERENCE INTO IT IS ALIVE (THE “FIGHTING THE BORROW CHECKER” REFLEX)

The most common early-Rust frustration is writing a loop that reads from a collection and mutates it in the same breath — `for x in &v { if cond(x) { v.push(...) } }` — and being stopped by `E0502`: *cannot borrow* `v` *as mutable because it is also borrowed as immutable*. Beginners read this as the compiler being pedantic about a program that “obviously works,” and reach for the wrong escapes: cloning the whole collection to get an owned copy to iterate, or wrapping everything in `RefCell` to push the check to run time, or dropping into `unsafe`. But the error is reporting a *real* hazard — the exact iterator-invalidation bug the C program above hit at run time, where mutating the container can reallocate its buffer and dangle the reference you are iterating through. The right responses match what you actually mean: if you only need to *read* while deciding what to change, collect the decisions first (e.g. gather the indices or new items into a separate vector) and then apply the mutations after the read-borrow ends; if you need to mutate in place without growing, iterate with `&mut` (`iter_mut`), which is a single exclusive borrow and is allowed; if you genuinely need shared mutable access with runtime-checked rules, `RefCell` is a deliberate, documented choice, not a way to silence the compiler. Fighting the borrow checker by cloning or reaching for `unsafe` to make the message go away is the analog of casting away a warning in C — it works and discards the thing the warning was protecting you from.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: “The borrow checker is being ridiculous. I'm iterating over a vector and pushing to it inside the loop, and it won't compile — `E0502`, can't borrow as mutable because it's borrowed as immutable. This is a five-line loop that would just work in C++ or Python. Why is Rust making me clone the whole thing?” Cover the paragraph and respond in four or five sentences, drawing on §28.3–§28.4.

Model answer. “That loop in C++ is the classic iterator-invalidation bug: pushing can reallocate the vector's buffer, and the reference you're iterating through then dangles — in C the same pattern is a silent heap-use-after-free that only a sanitizer catches at run time. Rust's rule is that a mutable borrow (`push` needs one) can't coexist with the shared borrow your loop is holding, which is exactly why it flags it. So the checker isn't being pedantic; it caught the reallocation hazard before the program ran. Don't clone the whole vector to dodge it — that pays for a full copy to hide a real bug. Collect the items you want to add into a separate vector during the read pass, then push them after the loop, or use `iter_mut` if you're mutating in place; the fix is to separate the read from the write, which is what the rule is telling you to do.” Framing `E0502` as the compile-time form of the C use-after-free, and offering the collect-then-mutate fix instead of a clone, is the senior register.

(Answer to the §28.3 quick check: the rule is “at any time a value may have either any number of shared references `&T` or exactly one mutable reference `&mut T`, never both.” Two `&mut` to one value is `E0499` (“cannot borrow as mutable more than once at a time”); a `&mut` while a `&` is live is `E0502` (“cannot borrow as mutable because it is also borrowed as immutable”). A `&mut T` being exclusive is the single-threaded analog of preventing a data race because a data race is aliasing plus a write; exclusivity guarantees no other accessor can observe the value while it is being written.)

28.5 What to carry forward

Borrowing is how you use a value without owning it. A **reference** points at a value it does not own, so lending one leaves the owner intact and drops nothing when the reference's scope ends. There are two kinds, and the whole of borrowing is one rule dividing them: a value may be borrowed by *any number of shared references* (`&T`, read-only) or by *exactly one mutable reference* (`&mut T`, read-write), never both at once and never two mutable ones. That is **aliasing XOR mutation**: unlimited sharing is safe precisely while nothing writes, and writing is safe precisely when nothing else can see the value meanwhile. The **borrow checker** enforces this at compile time, at zero runtime cost, and in doing so it turns concrete hazards from the earlier volumes into compile errors: the iterator-invalidation use-after-free that the C program of §28.4 hid until a sanitizer caught it is, in Rust, `E0502` at compile time (`push`'s mutable borrow cannot coexist with a live shared borrow into the vector), and two writers to one value are `E0499`. The same “no aliasing with mutation” rule, lifted across threads later, is why safe Rust has no data races — the Chapter 24 hazard made structural.

One question is now unavoidable, and it is the whole of the next chapter. A reference is a pointer into another value; the second borrow rule is that a reference must never be used after the value it points at is gone — a dangling reference (the C return-a-pointer-to-a-local bug) must be impossible. But how does the compiler *know*, at the point you use a reference, that the value behind it is still alive — especially when references are passed into and out of functions, stored in structs, and returned? The answer is **lifetimes**: a way to name and relate the spans over which references are valid, so the borrow checker can prove that no reference outlives its referent. Everything the machine does underneath is unchanged — a Rust reference compiles to exactly the pointer C would use, with no extra runtime data — but the compiler now carries a proof, expressed in lifetimes, that the pointer is always valid when dereferenced. That proof, and the small annotations it sometimes needs from you, is where we go next.

CAN YOU… WITHOUT LOOKING?

- Say what a reference is, and why lending one leaves the owner valid where moving would not.
- State the two-part borrowing rule (many shared XOR one exclusive) and why each half is safe.
- Explain why a `&mut T` must be the only live reference, in terms of aliasing plus mutation.
- Name the error codes for two mutable borrows (`E0499`) and a mutable-while-shared borrow (`E0502`).
- Show how the borrow rule turns the C iterator-invalidation use-after-free into a compile error.
- Connect the borrow rule to data-race prevention (aliasing XOR mutation, Chapter 24).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a reference, and why does passing `&owner` leave `owner` usable while passing it by value does not? (§28.1)
2. **R2.** How many shared references (`&T`) to one value may exist at once, and what may you do through them? (§28.2)
3. **R3.** State the exclusivity rule for `&mut T`, and give the two error codes it is enforced by. (§28.3)
4. **R4.** Take the C realloc/dangling-pointer bug; explain why the analogous Rust code is `E0502` at compile time. (§28.4)
5. **R5.** Why is "aliasing XOR mutation" the same rule that prevents data races? (§28.4)

FURTHER READING

- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 4.2 "References and Borrowing" (also free at `doc.rust-lang.org/book`).** The primary source for shared vs. mutable references and the borrowing rule of §28.2–§28.3. Authors and edition verified via the publisher; the book was not compiled in this environment.
- **The Rust Error Index, entries `E0499`, `E0502`, and `E0505` (`doc.rust-lang.org/error_codes`).** The official one-line meanings quoted in §28.3–§28.4: `E0499` "borrowed as mutable more than once," `E0502` "borrowed as mutable because also borrowed as immutable," `E0505` "a value was moved out while it was still borrowed." Each verified against the current error index.
- **Jung, Jourdan, Krebbers & Dreyer, "RustBelt: Securing the Foundations of the Rust Programming Language," *Proc. ACM on Programming Languages* 2(POPL), Article 66 (2018), DOI 10.1145/3158154.** The machine-checked proof that the borrowing discipline of this chapter is sound. First author Ralf Jung; verified via the ACM Digital Library.
- **The Rust reference and the "non-lexical lifetimes" RFC 2094 (`rust-lang.github.io/rfcs`).** Why a borrow ends at last use rather than at the closing brace — the refinement noted in §28.3. Reference for the precise model; verify against the current edition.

Exercises

28.1 WARM-UP In one sentence each, define a shared reference (`&T`) and a mutable reference (`&mut T`), saying what you may do through each and how many of each may exist at once.

28.2 WARM-UP Why does `fn length(s: &String)` leave the caller's `String` usable after the call, whereas `fn length(s: String)` would not?

28.3 WARM-UP State the two-part borrowing rule in one sentence, and name the two situations it forbids together with their error codes.

28.4 CORE Take the C snippet of §28.4 (pointer into a buffer, `realloc`, use the old pointer). (a) Identify the bug and the line it occurs on. (b) Explain why the plain `gcc -O2 -Wall` build gives no warning, and what AddressSanitizer reports at run time. (c) Explain precisely why the analogous Rust code is `E0502` at compile time — which borrow conflicts with which, and why the checker does not need to know that `push` reallocates.

28.5 CORE Explain why a mutable reference must be exclusive. (a) What single-threaded hazard would two references to one value, at least one mutable, reproduce? (b) Connect this to the definition of a data race from Chapter 24. (c) What does the exclusivity of `&mut T` let the compiler assume, and why is that good for optimization?

28.6 CORE For each, say whether it compiles, and if not, which error code fires and why: (a) `let a = &v; let b = &v;` (both shared); (b) `let m = &mut v; let n = &mut v;`; (c) `let r = &v; let m = &mut v;`; (d) a shared borrow used and finished, then a mutable borrow taken afterward (non-overlapping).

28.7 COST References are "zero-cost." (a) At the machine level, what is a `&T` — how much runtime data does it carry beyond the pointer C would use? (b) At what time is the borrow checking done, and what runtime cost does it add? (c) Contrast this with `RefCell`, which enforces the same borrow rule at run time: what does `RefCell` cost that a plain `&mut` does not, and when is paying it justified?

28.8 FADED *Worked, with the last step for you.* A newcomer writes a loop that scans a `Vec<i32>` and pushes each value's double back onto the same vector, and gets `error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable`.
Step 1 (given): `for x in &v { ... }` holds a shared borrow of `v` for the whole loop body.
Step 2 (given): `v.push(2 * x)` inside the body needs a mutable borrow of `v`, which cannot coexist with the shared borrow.
Step 3 (given): this is the iterator-invalidation hazard: a push could reallocate `v` and dangle the iterator.
Step 4 (you): Give two correct fixes — one that separates the read pass from the write pass without cloning the whole vector, and one using `iter_mut` if the mutation were in place — and say why cloning the entire vector to dodge the error is the wrong default.

28.9 MIXED For each, state whether it is allowed by the borrow rule and justify in a line: (a) three `&v` at once; (b) one `&mut v` and one `&v` at once; (c) two `&mut v` at once; (d) a `&mut v` after all shared borrows of `v` have ended.

28.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "You may have many mutable references to a value as long as they don't write at the same time." (b) "A shared reference lets you mutate the value it points at." (c) "The borrow checker adds runtime overhead to every dereference." (d) "Holding a reference into a vector and then pushing to it is a compile error." (e) "A reference drops (frees) the value when it goes out of scope."

28.11 STRETCH The borrow rule is "aliasing XOR mutation." (a) Show that both a use-after-free through an aliased pointer (Chapter 9/11) and a data race (Chapter 24) are instances of "two paths to one value, at least one writing," and how the single rule attacks their shared root. (b) Explain why unlimited *shared* references are nonetheless safe, and unlimited access is only a problem once mutation enters. (c) The rule as stated is for a single value in one thread; name what still has to be added to extend "no data races" across threads, and why the borrow rule alone is not yet enough there.

28.12 LAB On your own machine (a C toolchain suffices; a Rust toolchain is ideal): (a) Compile and run the §28.4 C snippet (pointer into a buffer, `realloc` to a much larger size, use the old pointer). Confirm it builds without warning under `-O2 -Wall`, then rebuild with `-fsanitize=address` and observe the heap-use-after-free. (b) If you have `rustc`, write the `let p = &v[0]; v.push(4); println!("{p}");` version and confirm it fails with `E0502`; if not, name the exact conflicting borrows from this chapter. (c) Write a version that compiles by ending the shared borrow before the `push`, and explain why it is now allowed. Note your toolchain versions.

Chapter 29 — Lifetimes

Before you start: Chapter 28 (borrowing — a lifetime is how the compiler proves the second borrow rule, that a reference never outlives its value), Chapter 27 (ownership and drop — a value is dropped at end of scope, so a reference to it must not be used past that point), Chapter 8 (the stack: a local's storage is reclaimed when its scope ends), Chapter 9 (dangling pointers and use-after-free — exactly what lifetimes rule out). · Chapter 28 left one borrow rule unproven: a reference must never be used after the value it points at is gone. This chapter is how the compiler proves it — by tracking, and sometimes making you name, the spans over which references are valid.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 8 — when is a stack local's storage reclaimed, and what becomes of a pointer to it afterward? (2) Chapter 28 — a reference does not own its value; who is responsible for dropping the value, and when? (3) *Predict:* in C you write a function that returns `&local` (the address of one of its own stack variables) and the caller dereferences it. Does `gcc -Wall` warn? In Rust, a function that returns a reference to one of its own locals — will the compiler accept it? Write your guesses; §29.1 and §29.3 answer all three.

Chapter 28 gave one borrow rule in full — aliasing XOR mutation — and named a second only in passing: a reference must never be used after the value it refers to is gone. That second rule is what this chapter is about, and the machinery that enforces it is **lifetimes**. A lifetime is not a runtime thing and it changes nothing about the code the machine runs; it is a *compile-time* name for a region of the program over which a reference is valid, and the borrow checker uses lifetimes to prove that every reference is dereferenced only while its referent is still alive. First, the hazard: the dangling reference — the C return-a-pointer-to-a-local bug — and why the compiler must forbid it (§29.1). Then how references usually need no annotation, and when they do: naming a lifetime `'a` in a function signature to relate an output reference to its inputs (§29.2). Then what the borrow checker actually checks, and the `E0597` error it produces when a value does not live long enough (§29.3). Then lifetime **elision** — why most signatures omit lifetimes — and structs that hold references, where a lifetime is mandatory (`E0106`) (§29.4). We close by carrying lifetimes forward: ownership, borrowing, and lifetimes together are the whole memory-safety story, and the next chapters build reusable abstractions on top of it (§29.5).

On the code in this part. The book's environment has a C toolchain (gcc 11.4.0) but *no Rust compiler installed*, so the C programs below were compiled and run and their output is real, whereas the Rust snippets were *not* compiled here; their behavior and the quoted compiler diagnostics are drawn from *The Rust Programming Language* (Klabnik & Nichols, 2nd ed., 2023) and the official Rust error index, and are labeled as such. Nothing about Rust's behavior below is invented — it is cited.

29.1 The dangling reference, and why it must be forbidden

A reference is a pointer into a value it does not own. The value is dropped by its owner at end of scope (Chapters 27, 28); if a reference is used after that point, it points at reclaimed storage — a **dangling reference**, the use-after-free of Chapter 9 in reference form. The archetype is returning a pointer to a stack local. Here is real C; the trivial form is caught by a best-effort warning, but that warning is a heuristic, not a guarantee. Compiled in this environment (gcc 11.4.0):

```
int *make(void) {
    int local = 42;
    return &local;      /* address of a stack local; reclaimed when make returns */
}
/* caller: */ int *r = make(); printf("%d\n", *r);   /* dangling: use-after-scope */

$ gcc -O2 -Wall dangle.c -o dangle
dangle.c: In function 'make':
dangle.c:3:12: warning: function returns address of local variable [-Wreturn-local-addr]
```

gcc catches this direct form. But launder the same pointer through a struct and the heuristic misses it — the program compiles clean and is only caught at run time by a sanitizer (gcc 11.4.0, this environment):

```
struct Ref { int *p; };
struct Ref borrow_local(void) {
    int local = 42;
    struct Ref r; r.p = &local;   /* store address of a local in a struct */
    return r;                       /* r.p now dangles */
}
/* caller: */ struct Ref r = borrow_local(); printf("%d\n", *r.p);

$ gcc -O2 -Wall dangle2.c -o dangle2      # compiles, NO warning
$ gcc -O1 -fsanitize=address dangle2.c -o d && ./d
==ERROR: AddressSanitizer: stack-use-after-scope ... in main
```

The point is not that C never warns — it is that C's warnings are a best-effort heuristic that follows only simple data flow and gives up as soon as the pointer passes through a struct, a second function, or a container. The general problem — does this reference outlive the value it points at? — is not something C's type system tracks at all. Rust makes it a rule the type system enforces in *every* case: a reference may not be used after its referent is dropped, no exceptions, checked at compile time. To do that in the general case — references flowing into and out of functions, stored in structs, returned — the compiler needs a way to *name* and *compare* the spans over which references and values are valid. That name is a lifetime.

> **QUICK CHECK**
>
> Why is returning `&local` from a function a bug — what has happened to `local`'s storage by the time the caller dereferences the result? Why does `gcc -Wall` catch the direct form but not the version laundered through a struct? State the general rule Rust enforces that the C heuristic only approximates. (Answer after the next section.)

29.2 Lifetimes named: relating an output reference to its inputs

Most of the time you never write a lifetime — the compiler infers the valid regions on its own (§29.4). You must name one when a function returns a reference and the compiler cannot tell, from the signature alone, *which* input the returned reference borrows from — because that determines how long the result stays valid. The canonical case is a function returning whichever of two string slices is longer (Rust; not compiled here — behavior per the cited book):

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

The `<'a>` declares a **lifetime parameter** named `'a` (lifetimes are written with a leading apostrophe and are usually short and lowercase). The signature now says: for some region `'a`, both inputs are references valid for at least `'a`, and the returned reference is valid for `'a` too. This is a *constraint*, not a runtime value — it does not change what the function does or make anything live longer; it tells the borrow checker how the output's validity relates to the inputs'. The payoff is at the call sites: the compiler now knows the returned reference cannot outlive the shorter-lived of the two arguments, and it will reject any caller that tries to use the result after either input has gone. Without the annotation the compiler could not know whether the result borrows from `x` or `y`, so it could not check the callers — which is exactly why the annotation is required here. Lifetime parameters, then, are how a function's signature carries a promise about the validity of the references it returns, so that borrow-checking composes across function boundaries.

THE SAME IDEA THREE WAYS: A REFERENCE MAY NOT OUTLIVE ITS REFERENT

1. In words. Every reference is valid only for a region — its lifetime — bounded by the scope of the value it points at. The borrow checker requires that a reference is used only within that region; using it after the referent is dropped is rejected. A lifetime parameter like `'a` lets a signature state how an output reference's region relates to its inputs'.

2. As a picture. Figure 29.1: a value's lifetime drawn as a bar from its creation to its drop; a reference's use drawn as a shorter bar that must fit *inside* the value's bar. A reference bar that extends past the value's drop is the error.

3. As numbers/contrast. The C function of §29.1 returns a pointer whose referent's storage is reclaimed the instant the function returns — the reference bar extends past the value bar → use-after-scope (ASan-confirmed). The Rust equivalent is `E0597` at compile time: the borrowed value does not live long enough, so the reference cannot be returned.

(Answer to the §29.1 quick check: `local` lives on the function's stack frame, which is reclaimed the moment the function returns, so by the time the caller dereferences the returned pointer the storage is gone — a use-after-scope. gcc's `-Wreturn-local-addr` follows simple direct data flow and catches `return &local;`, but it gives up once the address is stored in a struct and returned indirectly, so that version compiles clean. Rust's rule,

enforced in all cases at compile time, is that a reference may never be used after the value it refers to has been dropped.)

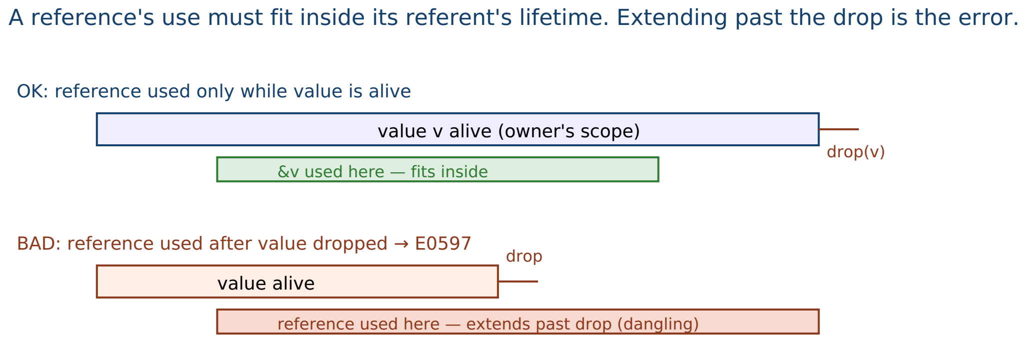


Figure 29.1: Lifetimes as fitting. A reference is valid only within its referent's lifetime (top, allowed). A reference used after the value is dropped (bottom) is a dangling reference — the C use-after-scope of §29.1, and in Rust a compile error, `E0597`, “borrowed value does not live long enough.”

29.3 What the borrow checker proves, and E0597

With lifetimes as its vocabulary, the borrow checker's second rule is a containment check: for every use of a reference, the region over which the reference is used must be contained in the region over which its referent is alive. When it is not, the compiler reports that the borrowed value does not live long enough. Here is the dangling-reference bug of §29.1 in Rust — a function trying to return a reference to its own local (Rust; not compiled here — diagnostic quoted from the official Rust error index):

```
fn dangle<'a>() -> &'a str {
    let s = String::from("hello");   // s is a local; it will be dropped at the `}`
    &s                               // return a reference to s
}                                    // s is dropped here — the reference would dangle

error[E0597]: `s` does not live long enough
  = note: values in a scope are dropped in the opposite order they are defined
```

The compiler rejects it with `E0597` — *the borrowed value does not live long enough*, per the official Rust error index — because `s` is dropped at the closing brace, so the returned reference would point at freed storage; its required lifetime (as long as the caller keeps it) exceeds `s`'s actual lifetime (the function body). This is the C use-after-scope of §29.1, but caught at compile time in every case, not by a heuristic that a struct can defeat. The fix is never to make the reference live longer — you cannot extend a dropped value — but to return an *owned* value (return `String`, moving ownership out, Chapter 27) or to borrow from something that outlives the function (an input reference, §29.2). Note again that lifetimes are inert at run time: they are erased before code generation, so a correct program pays nothing for them, and the reference compiles to the same bare pointer C would use. The entire cost of the guarantee is borne by the compiler, once.

QUICK CHECK

What does `E0597` report, and what condition on regions is the borrow checker verifying when it fires? Why can't you fix a "does not live long enough" error by making the reference live longer, and what are the two real fixes? At run time, what does a lifetime cost? (Answer below the communicate box.)

29.4 Elision, and structs that hold references

If every reference needed an explicit lifetime, Rust would be unbearable. It does not, because of **lifetime elision**: a set of deterministic rules by which the compiler fills in the common cases so you write no annotation at all. The three elision rules (Klabnik & Nichols, 2023, ch. 10) are: (1) each elided input reference gets its own distinct lifetime; (2) if there is exactly one input lifetime, it is assigned to all elided output lifetimes; and (3) if there are multiple input lifetimes but one of them is `&self` or `&mut self` (a method), the lifetime of `self` is assigned to all elided outputs. When these rules determine every lifetime, you write none; the `longest` function of §29.2 needed an explicit `'a` only because it has two input references and neither is `self`, so rule 2 and rule 3 do not apply and the compiler cannot guess which input the output borrows. Elision is not a special case in the type system — it is pure syntactic sugar for lifetimes you could always write out.

One place a lifetime is never optional: a struct that *holds* a reference. Such a struct is only valid as long as the value it borrows, and that relationship must be named. Omitting it is `E0106` (Rust; not compiled here — diagnostic per the official Rust error index):

```
struct Excerpt { part: &str }        // ERROR: reference field with no lifetime
error[E0106]: missing lifetime specifier

struct Excerpt<'a> { part: &'a str } // OK: the struct borrows for 'a and may not
                                     //     outlive the &str it holds
```

The first is rejected with `E0106` — *missing lifetime specifier* — because a struct with a reference field carries an implicit promise ("I do not outlive what I borrow") that must be written into the type as `<'a>`. Once annotated, the borrow checker enforces that no `Excerpt` value is used after the `str` it points into has been dropped — the same containment rule as §29.3, now for a reference living inside a struct. This is why data structures that own their data (holding `String`, `Vec`) are simpler than ones that borrow it (holding `&str`, `&[T]`): owning avoids lifetime parameters entirely, at the cost of an allocation; borrowing avoids the allocation, at the cost of threading a lifetime through the type. That trade — own vs. borrow — is a recurring design decision, and lifetimes are the language in which the borrowing side of it is expressed.

TRAP: READING A LIFETIME AS "HOW LONG THE VALUE LIVES" AND TRYING TO MAKE REFERENCES LIVE LONGER

The word "lifetime" misleads newcomers into thinking `'a` is a duration you set — that annotating a reference with a longer lifetime makes the underlying value live longer, the way a smart pointer or a garbage collector would keep it alive. It does not. A lifetime is purely *descriptive*: it names a region that the reference is valid for, bounded by facts already fixed by ownership — when the value is created and when its owner drops it. Writing `'a` does not extend anything; it only lets the compiler *check* a relationship. So when you hit `E0597`, "`s` does not live long enough," the wrong instinct is to search for a lifetime annotation that will make the error disappear — adding `'static`, or inventing longer-named lifetimes — because no annotation can make a value outlive its owner's scope. The error is telling you a structural truth: you are trying to use a reference past the life of what it points at. The real fixes are to change ownership, not the annotation: return an owned value (move it out, Chapter 27), borrow from something that genuinely outlives the use (an input reference, or a value in an outer scope), or restructure so the reference is never needed past the value's death. Reaching for `'static` to silence `E0597` is the analog of casting a pointer to make a type error vanish — it either does not compile anyway or forces the value to be truly static, which is rarely what you meant.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I have a function that builds a string and returns a `&str` to it, and I keep getting `E0597`, does not live long enough. I tried annotating it `'a`, then `'static`, and it still won't compile. Lifetimes are just confusing syntax — why can't I just tell it the reference lives as long as I need?" Cover the paragraph and respond in four or five sentences, drawing on §29.3–§29.4.

Model answer. "The annotation can't fix it because a lifetime doesn't *make* anything live longer — it only names how long a value already lives, which ownership fixed: your local string is dropped when the function returns, so any reference to it would dangle, which is exactly the C 'return a pointer to a local' bug. That's what `E0597` is reporting, and no lifetime name, not even `'static`, can extend a value past its owner's scope. The fix is about ownership, not syntax: return the `String` by value so ownership moves out to the caller, or borrow from an input that outlives the call. Adding `'static` only makes it compile if the data really is static, which a freshly built string isn't. So don't hunt for the magic annotation — decide who should own the result." Framing the error as the C return-a-local bug and the fix as an ownership decision, not an annotation search, is the senior register.

(Answer to the §29.3 quick check: `E0597` reports that a borrowed value does not live long enough; the borrow checker is verifying that the region over which a reference is used is contained within the region its referent is alive. You cannot fix it by making the reference "live longer" because a lifetime only describes existing facts — it cannot extend a value past its owner's drop; the two real fixes are to return an owned value (move ownership out) or to borrow from something that genuinely outlives the use. At run time a lifetime costs nothing — lifetimes are erased before code generation.)

29.5 What to carry forward

Lifetimes complete the memory-safety story. Ownership (Chapter 27) says every value has one owner and is dropped at end of scope; borrowing (Chapter 28) lets you use a value without owning it, under aliasing-XOR-mutation; and lifetimes enforce the second borrow rule — that a reference is never used after its referent is dropped. A **lifetime** is a compile-time name for the region a reference is valid over; the borrow checker proves, for every use, that the reference's region is contained in its referent's, and reports `E0597` — “borrowed value does not live long enough” — when it is not, catching the C dangling-pointer/use-after-scope bug in every case, where C's warning is only a best-effort heuristic a struct can defeat. Most lifetimes are inferred by **elision**, so you rarely write one; you must name a lifetime (`'a`) when a function returns a reference whose origin among several inputs is ambiguous, and you must give a struct a lifetime parameter when it holds a reference (`E0106`, “missing lifetime specifier”). Crucially, lifetimes are inert at run time — erased before code generation — so the whole guarantee is paid for once, at compile time, and a Rust reference is the same bare pointer C would emit.

You now have the three pillars — ownership, borrowing, lifetimes — that let Rust guarantee memory safety without a garbage collector, encoding in the type system the very invariants you proved by hand across the C volumes: memory freed exactly once, used only while valid, and never shared-and-mutated in a way that races. What is missing is not safety but *reuse*. The `longest` function worked on `&str`; what about a function that finds the largest element of any slice, of any type that can be compared? Writing one version per type is the C-with-copy-paste world these volumes have been leaving behind. The next chapter is how Rust expresses code that is generic over types while staying fully checked and monomorphized to concrete, zero-overhead machine code: **traits and generics**. And traits, it will turn out, are also how the concurrency guarantee is finally stated — the `Send` and `Sync` markers that lift borrowing's aliasing discipline across threads. The machine underneath is unchanged; what grows is the vocabulary in which you state, and the compiler checks, what your code requires of the types it works with.

CAN YOU… WITHOUT LOOKING?

- Define a lifetime as a compile-time region, and say what the second borrow rule is.
- Explain why returning `&local` is a dangling reference, and why C's warning is only a heuristic.
- Read the `longest<'a>` signature and say what the `'a` asserts and why it is needed there.
- State what `E0597` reports and the two real fixes (own vs. borrow-from-something-longer-lived).
- Give the elision intuition and say why a struct holding a reference needs a lifetime (`E0106`).
- Say what a lifetime costs at run time, and why.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a dangling reference, and why does returning `&local` create one? Why is C's `-Wreturn-local-addr` only best-effort? (§29.1)
2. **R2.** In `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`, what does `'a` assert, and why is an annotation required here? (§29.2)
3. **R3.** What does `E0597` report, and what containment condition is the borrow checker verifying? (§29.3)
4. **R4.** State the intuition for lifetime elision, and why a struct with a reference field needs a lifetime (`E0106`). (§29.4)
5. **R5.** What do lifetimes cost at run time, and how do the three pillars (ownership, borrowing, lifetimes) fit together? (§29.3, §29.5)

FURTHER READING

- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 10.3 "Validating References with Lifetimes" (also free at `doc.rust-lang.org/book`).** The primary source for lifetime syntax, the `longest` example, and the three elision rules of §29.2 and §29.4. Authors and edition verified via the publisher; the book was not compiled in this environment.
- **The Rust Error Index, entries `E0597` and `E0106` (`doc.rust-lang.org/error_codes`).** The official meanings quoted in §29.3–§29.4: `E0597` "a value was dropped while it was still borrowed" (borrowed value does not live long enough) and `E0106` "a lifetime is missing from a type" (missing lifetime specifier). Each verified against the current error index.
- **Jung, Jourdan, Krebbers & Dreyer, "RustBelt: Securing the Foundations of the Rust Programming Language," *Proc. ACM on Programming Languages* 2(POPL), Article 66 (2018), DOI 10.1145/3158154.** The formal model in which lifetimes-as-regions and the "references never outlive their referent" guarantee are proved sound. First author Ralf Jung; verified via the ACM Digital Library.
- **The "non-lexical lifetimes" RFC 2094 and the Polonius project notes (`rust-lang.github.io`).** How the borrow checker computes lifetime regions precisely (last-use rather than lexical scope) — the modern engine behind §29.3. Reference for the checker's internals; verify against the current edition.

Exercises

29.1 WARM-UP In one sentence, what is a lifetime, and what is the second borrow rule that lifetimes enforce?

29.2 WARM-UP Why is `fn make() -> &i32 { let x = 5; &x }` a bug? What has happened to `x` by the time the caller would dereference the result?

29.3 WARM-UP In `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`, state in words what the three occurrences of `'a` jointly assert.

29.4 CORE Take the two C snippets of §29.1 (direct `return &local;` and the version laundered through a struct). (a) Why is each a use-after-scope? (b) Why does `gcc -Wall` warn on the first but not the second? (c) Explain how Rust's rule differs from a best-effort warning, and what error code the analogous Rust function produces.

29.5 CORE Explain what the borrow checker verifies when it emits `E0597`. (a) State the containment condition on regions. (b) Why can't the error be fixed by annotating a longer lifetime? (c) Give the two structural fixes and say which each is appropriate for.

29.6 CORE State the three lifetime-elision rules and apply them: (a) `fn first(s: &str) -> &str` — why is no annotation needed? (b) `fn get(&self, i: usize) -> &T` — which rule supplies the output lifetime? (c) `fn longest(x: &str, y: &str) -> &str` — why do the rules fail to determine the output, forcing you to write `'a`?

29.7 COST Lifetimes are “zero-cost.” (a) At what phase are lifetimes checked, and what happens to them before code generation? (b) What is the runtime representation of a `&'a T` compared with the C pointer it replaces? (c) Contrast this with a garbage-collected or reference-counted language's approach to the same “is this reference still valid?” question: what does each pay, and when?

29.8 FADED *Worked, with the last step for you.* A newcomer writes `fn greeting() -> &str { let s = String::from("hi"); &s }` and gets ``error[E0597]: `s` does not live long enough``.
Step 1 (given): `s` is a local `String`, dropped at the function's closing brace.
Step 2 (given): `&s` would be returned to the caller, who could use it after `s` is dropped — a dangling reference.
Step 3 (given): no lifetime annotation (including `'static`) can make `s` outlive its scope, so the error is structural, not syntactic.
Step 4 (you): Give the correct fix (change the return type so ownership moves out) and one alternative (borrow from an input that outlives the call), and explain why reaching for `'static` is the wrong instinct here.

29.9 MIXED For each, say whether a lifetime annotation is *required*, *elided*, or the code is a *dangling-reference error*, and justify: (a) `fn id(x: &i32) -> &i32`; (b) `struct Wrapper { r: &u8 }`; (c) a function returning a reference to its own local; (d) `fn pick(a: &str, b: &str) -> &str` returning one of the two.

29.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “Annotating a reference `'static` makes the value it points at live forever.” (b) “Lifetimes exist at run time and cost a pointer of extra data.” (c) “A struct with a `&str` field can omit its lifetime.” (d) “`E0597` means the reference outlives its referent.” (e) “Most function signatures need explicit lifetimes.”

29.11 STRETCH Owning vs. borrowing in data structures. (a) Contrast a struct that stores a `String` with one that stores a `&'a str`: what does each cost (allocation vs. lifetime parameter), and what constraint does the borrowing version place on how the struct may be used? (b) Explain why a self-referential struct (a field that borrows from another field of the same struct) is hard to express with lifetimes, and why. (c) Relate the own-vs-borrow choice back to the C question of “who owns this buffer” from Chapters 9 and 27, and argue that lifetimes are just that question made explicit and checked.

29.12 LAB On your own machine (a C toolchain suffices; a Rust toolchain is ideal): (a) Compile the two C snippets of §29.1 with `-O2 -Wall`; confirm the direct `return &local;` warns (`-Wreturn-local-addr`) and the struct-laundered version does not, then catch the latter with `-fsanitize=address` (stack-use-after-scope). (b) If you have `rustc`, write the `fn dangle() -> &str` version and confirm `E0597`; if not, explain from this chapter which line is rejected and why. (c) Write the `longest` function with its lifetime annotation, then a struct with a `&str` field, and note what happens if you omit the struct's lifetime (`E0106`). Note your toolchain versions.

Chapter 30 — Traits and Generics

> **Before you start:** Chapter 29 (lifetimes — with ownership and borrowing, the memory-safety story is complete; this chapter is about *reuse* on top of it), Chapter 27 (ownership; `Copy`, `Clone`, and `Drop` are *traits*, so this chapter names machinery you have already met), Chapter 10 (the optimizing compiler — monomorphization is a compile-time specialization that makes generics free), Chapter 5 (mechanical sympathy — static vs. dynamic dispatch is a real cost you can now name). · The `longest` function of Chapter 29 worked only on string slices; this chapter is how to write code once that works over many types, fully checked and with no runtime cost.

> **BEFORE YOU READ ON**
>
> From memory, reaching back: (1) Chapter 27 — `Copy` decided whether `let y = x;` copies or moves; what kind of thing is `Copy` (a keyword? a type? something a type can implement)? (2) Chapter 10 — what does it mean to specialize code at compile time, and why can a compile-time transformation be free at run time? (3) *Predict:* in C you write a generic sort with `qsort`, passing a comparison *function pointer*; how is the comparison call dispatched — is the target known at compile time or chosen at run time? In Rust, a function generic over a type `T` that you compare with `<` — does the compiler need to know something about `T` to allow the comparison? Write your guesses; §30.2 and §30.3 answer all three.

Ownership, borrowing, and lifetimes made Rust memory-safe; they did nothing for *reuse*. The `longest` function worked on `&str`; a function to find the largest element of a slice would, in C, be written once per element type or smeared over `void*` and function pointers that throw away compile-time type checking. Rust's answer is two intertwined features. **Generics** let you write code parameterized over a type `T`; **traits** let you say what a `T` must be able to *do*, and are how shared behavior is named and required. First, the reuse problem in C, concretely, with a real `qsort` run (§30.1). Then generics and **monomorphization**: the compiler stamps out a specialized copy per concrete type, so generic code is as fast as hand-written type-specific code — zero-cost static dispatch (§30.2). Then traits and **trait bounds**: a generic can only use capabilities its bounds guarantee, and a missing bound is a compile error, `E0277` (§30.3). Then the other dispatch strategy — **trait objects** (`dyn Trait`), one shared version that dispatches through a vtable at run time, exactly the C function-pointer mechanism — and the cost trade between the two (§30.4). We close by carrying traits forward: they are also how error handling and, later, thread-safety are expressed (§30.5).

On the code in this part. The book's environment has a C toolchain (gcc 11.4.0) but *no Rust compiler installed*, so the C programs below were compiled and run and their output is real, whereas the Rust snippets were *not* compiled here; their behavior and the quoted compiler diagnostics are drawn from *The Rust Programming Language* (Klabnik & Nichols, 2nd ed., 2023) and the official Rust error index, and are labeled as such. Nothing about Rust's behavior below is invented — it is cited.

30.1 The reuse problem in C

C gives you two ways to write code that works over many types, and both are unsatisfying. The first is duplication: write `max_int`, `max_float`, `max_long`, one per type, differing only in a type name. The second is type erasure through `void*` plus a function pointer — how the standard library's `qsort` sorts anything. Here is a real `qsort`, compiled and run in this environment (gcc 11.4.0):

```
int cmp_int(const void *a, const void *b) {
    return (*(const int*)a) - (*(const int*)b);   /* caller-supplied comparison */
}
int a[] = {5, 2, 9, 1, 7};
qsort(a, 5, sizeof(int), cmp_int);                /* generic sort via void* + fn ptr */

$ gcc -O2 -Wall gen.c -o gen && ./gen
1 2 5 7 9
```

This works, and it is genuinely generic — `qsort` sorts arrays of any element type. But look at the two costs. First, *type safety is gone*: `qsort` sees `void*`, so if you pass a comparator written for `double` while sorting `int`s, it compiles cleanly and corrupts silently — the compiler cannot check that the comparator matches the array, because the types were erased. Second, every comparison is an **indirect call through a function pointer**: `cmp_int` is chosen at run time, so the CPU cannot inline it and pays an indirect branch per comparison (Chapter 5). Rust's generics fix the first cost entirely and let you *choose* whether to pay the second. The `qsort` pattern — a function pointer resolved at run time — is exactly what Rust calls a trait object (§30.4); the duplication-free, fully-checked, fully-inlined alternative is generics with monomorphization (§30.2). Both start from a single idea: name the behavior a type must provide, and let the type system check it.

> **QUICK CHECK**
>
> Name the two ways C lets you write code over many types, and the cost of each. In the `qsort` call, why can the compiler not check that `cmp_int` matches the array being sorted? How is each comparison dispatched — is the target known at compile time? (Answer after the next section.)

30.2 Generics and monomorphization

A **generic** function is parameterized over a type. You write it once, with a type parameter `T`, and the compiler generates a specialized copy for each concrete type you actually use it with — a process called **monomorphization** (Rust; not compiled here — behavior per the cited book):

```
fn largest<T: PartialOrd + Copy>(list: &[T]) -> T {
    let mut biggest = list[0];
    for &item in list {
        if item > biggest { biggest = item; }   // needs T: PartialOrd to use `>`
    }
    biggest
}
// called with &[i32] and &[f64]; the compiler emits a separate specialized
// copy of `largest` for i32 and for f64 — as if you'd written each by hand.
```

At compile time the compiler sees that you called `largest` with an `&[i32]` and an `&[f64]`, and it stamps out two concrete functions — one operating on `i32`, one on `f64` — each with the type filled in. There is no type parameter left at run time: each call goes to an ordinary, type-specific function whose comparison is a direct, inlinable operation, not an indirect call. This is **static dispatch**, and it is *zero-cost*: generic code runs exactly as fast as the hand-duplicated `max_int`/`max_float` of §30.1, because after monomorphization that is literally what it is — but you wrote it once and the compiler checked it once. The price, paid only in binary size and compile time, is that each concrete instantiation is a separate copy of the code (code bloat if a large generic is used at many types). The abstraction is free where it matters — at run time — which is the recurring promise of this whole part: expressive source, no runtime tax.

THE SAME IDEA THREE WAYS: WRITE ONCE, SPECIALIZE PER TYPE, PAY NOTHING AT RUN TIME

1. In words. A generic function is written once over a type parameter `T`. The compiler monomorphizes it — emits a separate specialized copy for each concrete type used — so at run time there is no type parameter and no dispatch: each call is an ordinary direct call, as fast as hand-written type-specific code.

2. As a picture. Figure 30.1: one generic `largest<T>` box fanning out at compile time into `largest_i32` and `largest_f64` (static dispatch, direct calls), contrasted with one `largest(&dyn ..)` box that stays single and dispatches through a vtable at run time (dynamic dispatch).

3. As numbers/contrast. The C `qsort` of §30.1 pays one indirect function-pointer call per comparison (chosen at run time, not inlinable). Monomorphized Rust generics pay zero: the comparison is a direct, inlinable operation in each specialized copy — the same machine code you'd get from writing the type-specific function by hand.

(Answer to the §30.1 quick check: C lets you write over many types by (1) duplicating code per type — a maintenance cost — or (2) erasing types through `void*` plus a function pointer, as `qsort` does — which costs type safety and an indirect call. The compiler cannot check that `cmp_int` matches the array because `qsort`'s parameters are `void*`: the element type is erased, so a mismatched comparator compiles anyway. Each comparison is dispatched through the function pointer at run time, so the target is not known at compile time and cannot be inlined.)

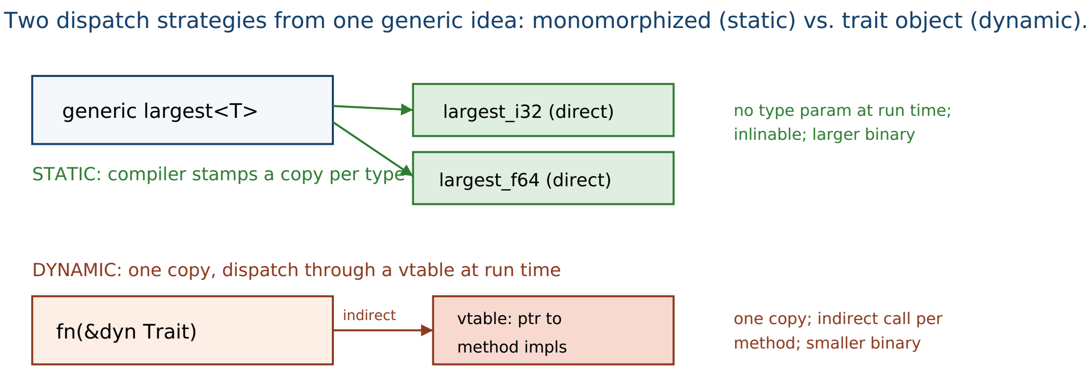


Figure 30.1: Static vs. dynamic dispatch. A generic monomorphizes into a direct, inlinable copy per concrete type (top, zero runtime cost, larger binary). A trait object keeps one copy and dispatches each method through a vtable at run time (bottom, an indirect call, smaller binary) — the same mechanism as C's `qsort` function pointer.

30.3 Traits and trait bounds

The `largest` function used `item > biggest`. For that to typecheck, the compiler must know that `T` supports comparison — otherwise a caller could instantiate `largest` with a type that has no ordering, and `>` would be meaningless. A **trait** names a set of behaviors a type can implement; a **trait bound** on a generic (`T: PartialOrd`) requires that `T` implement the trait, and in return lets the generic body use that trait's operations. You have already met traits: `Copy`, `Clone`, and `Drop` from Chapter 27 are traits, which is why "`i32` is `Copy`" was phrased as implementing something. A trait definition and an implementation look like this (Rust; not compiled here — behavior per the cited book):

```
trait Summary {                        // a named set of behavior
    fn summarize(&self) -> String;
}
impl Summary for Article {             // Article promises to provide it
    fn summarize(&self) -> String { format!("{}: ...", self.title) }
}

fn notify<T: Summary>(item: &T) {     // bound: T must implement Summary
    println!("Breaking! {}", item.summarize());   // allowed BECAUSE of the bound
}
```

The bound `T: Summary` is a contract in both directions: the caller must supply a type implementing `Summary`, and in exchange the body of `notify` may call `summarize`. If you drop the bound and try to call `summarize` anyway, or call `notify` with a type that does not implement `Summary`, the compiler rejects it with `E0277` — *the trait bound is not satisfied* (per the official Rust error index):

```
notify(&5);   // i32 does not implement Summary
error[E0277]: the trait bound `i32: Summary` is not satisfied
```

This is the decisive difference from C's `void*` genericity of §30.1. There, a type mismatch was invisible and corrupted silently at run time; here, "this type does not provide the required behavior" is a compile error naming the exact unmet bound. It is also, notably, different from C++ templates, where an unsatisfied requirement surfaces as an error deep inside the instantiated body; Rust checks the bound at the *definition*, so the generic is verified once, against its stated requirements, independent of any caller. Trait bounds are how a generic states precisely what it needs of its type parameters — no more (so it stays maximally reusable) and no less (so the body is fully checked) — and `E0277` is the compiler holding both sides to the contract.

> **QUICK CHECK**
>
> What is a trait, and what does a trait bound like `T: PartialOrd` give the caller and the body of a generic respectively? Which error code fires when a required bound is unmet, and what does it say? Why is that better than C's silent `void*` mismatch and than a C++ template error deep in the body? (Answer below the communicate box.)

30.4 Static vs. dynamic dispatch: trait objects

Monomorphization needs to know every concrete type at compile time. Sometimes you cannot — you want a single `Vec` holding a mix of types that all implement one trait, or a plugin resolved at run time. For that Rust offers a **trait object**, written `dyn Trait` behind a pointer (`&dyn Summary` or `Box<dyn Summary>`). A trait object is a pair of pointers: one to the value, one to a **vtable** — a table of the trait's method implementations for that value's concrete type — and each method call is dispatched indirectly through the vtable at run time (Klabnik & Nichols, 2023, ch. 17.2). This is **dynamic dispatch**, and it is precisely the C `qsort` mechanism of §30.1: an indirect call through a pointer chosen at run time (Rust; not compiled here — behavior per the cited book):

```
// static dispatch: one specialized copy per type, direct/inlinable calls, larger binary
fn notify<T: Summary>(item: &T) { println!("{}", item.summarize()); }

// dynamic dispatch: one copy, vtable call per invocation, smaller binary, any Summary type
fn notify_dyn(item: &dyn Summary) { println!("{}", item.summarize()); }

let items: Vec<Box<dyn Summary>> = vec![Box::new(article), Box::new(tweet)];
// a heterogeneous collection — impossible with a single monomorphized type
```

The trade is explicit and it is a cost trade you can now name from Chapter 5. Static dispatch (generics) resolves the call at compile time: direct, inlinable, zero dispatch cost, but a separate code copy per type (larger binary) and every type known at compile time. Dynamic dispatch (trait objects) resolves at run time through the vtable: one copy of the code and the ability to mix types or choose at run time, but an indirect call per method (not inlinable, an indirect branch — the same cost as `qsort`'s function pointer) and the values must sit behind a pointer. Neither is “better”; the default in Rust is generics (pay nothing, specialize) and you reach for `dyn` deliberately when you need heterogeneity or run-time choice, or to bound code size. The important thing is that unlike C — where `void*` dynamic dispatch also threw away type safety — a Rust trait object is still fully type-checked: `&dyn Summary` can only hold a type that actually implements `Summary`, so you get the flexibility of the function-pointer approach without the silent-corruption risk of §30.1.

TRAP: REACHING FOR `BOX<DYN TRAIT>` BY DEFAULT, AND MISREADING `E0277` AS "MY TYPE IS BROKEN"

Two related early mistakes. The first: coming from an object-oriented language, newcomers make everything a `Box<dyn Trait>` because it feels like the familiar interface/subclass pattern — and thereby pay an indirect call on every method and give up inlining, when a generic with a trait bound would have cost nothing at run time and inlined fully. Dynamic dispatch is a real tool (heterogeneous collections, run-time-chosen implementations, keeping binary size down), but it is not the default; generics with bounds are, and reaching for `dyn` reflexively is the analog of making every C call go through a function pointer for uniformity's sake. The second: reading `` error[E0277]: the trait bound `T: Foo` is not satisfied `` as "my type is wrong" and casting or wrapping to make it quiet. The error is not saying your type is broken; it is saying the type does not (yet) provide the behavior the generic requires — and the two correct responses are to *implement the trait* for your type (give it the behavior) or, if the generic asked for more than it needs, to *relax the bound* to what the body actually uses. Neither is a cast. As with a C warning, `E0277` is naming a real gap between what the code requires and what the type provides; the fix is to close the gap, not to silence the message.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "My generic function won't compile until I add `T: PartialOrd` — Rust makes me spell out every requirement, when in C++ a template just works and in Python I'd never write any of this. Isn't this bound stuff just ceremony?" Cover the paragraph and respond in four or five sentences, drawing on §30.2–§30.4.

Model answer. "The bound isn't ceremony — it's what lets the compiler check the generic *once*, at its definition, instead of C++'s way where a missing requirement blows up deep inside the instantiated template with a wall of errors, or Python's way where it blows up at run time on some input you didn't test. `T: PartialOrd` is you telling the compiler exactly what the body needs (an ordering to use `>`), and in exchange it guarantees every caller supplies a type that has it — that's the `E0277` you'd get otherwise, caught at compile time and naming the missing trait. It also costs nothing at run time: the generic monomorphizes into a direct, inlinable copy per type, so it's as fast as a hand-written type-specific function. So the bound buys you C's performance with checking C++ and Python can't give you at that stage. Write the minimal bound the body actually uses and you get maximal reuse with full checking." Framing the bound as check-once-at-the-definition and zero-cost after monomorphization is the senior register.

(Answer to the §30.3 quick check: a trait names a set of behaviors a type can implement; a bound `T: PartialOrd` requires the caller to supply a `T` that implements `PartialOrd`, and in return lets the generic's body use that trait's operations (here `>`). An unmet bound is `E0277`, "the trait bound is not satisfied." It beats C's silent `void*` mismatch — which corrupts at run time — and a C++ template error deep in the instantiated body, because Rust checks the bound at the generic's definition and names the exact missing trait.)

30.5 What to carry forward

Generics and traits are how Rust gets reuse without giving up the checking or the performance of the earlier volumes. A **generic** function is written once over a type parameter and **monomorphized** — the compiler emits a specialized copy per concrete type — so it runs as fast as hand-written type-specific code with no runtime dispatch:

zero-cost **static dispatch**. A **trait** names behavior a type can implement (you already knew `Copy`, `Clone`, `Drop`), and a **trait bound** (`T: PartialOrd`) states exactly what a generic requires of its type — checked once, at the definition, with an unmet bound reported as `E0277`, "the trait bound is not satisfied." When you need heterogeneity or a run-time-chosen implementation, a **trait object** (`dyn Trait`) gives **dynamic dispatch** through a vtable — the same indirect-call mechanism as C's `qsort` function pointer, but still fully type-checked, unlike C's `void*`. The choice between them is the Chapter 5 cost trade made explicit: static is the default (free, specialized, larger binary), dynamic is deliberate (one copy, flexible, an indirect call per method).

With ownership, borrowing, lifetimes, generics, and traits, you now have the whole conceptual core of Rust — and the last two chapters of this part put it to work. Traits are not only for behavior like `summarize`; they are how Rust expresses its most important guarantees as ordinary, checkable requirements. The next chapter is **error handling**: `Option<T>` and `Result<T, E>` are generic types (built on exactly the generics of this chapter), and they replace C's error-code-and-`errno` conventions and null pointers with types the compiler forces you to handle — turning "forgot to check the return value" into a thing that does not compile. And the chapter after it, on fearless concurrency, rests on two traits — `Send` and `Sync` — that lift the aliasing discipline of borrowing across threads, making the data races of Chapter 24 a compile error rather than undefined behavior. The pattern to carry forward is that in Rust, a guarantee is usually a trait: name the behavior or the capability, require it in the type, and let the compiler check it — the same move, now applied to errors and to threads.

CAN YOU… WITHOUT LOOKING?

- State the two ways C writes code over many types and the cost of each (duplication; `void*`+function pointer).
- Define monomorphization and explain why generic code is zero-cost at run time.
- Say what a trait and a trait bound are, and what each side of a bound gets.
- Name the error code for an unmet bound (`E0277`) and why compile-time checking beats C's silent mismatch.
- Contrast static and dynamic dispatch: mechanism, cost, and when to choose each.
- Explain how a trait object is like C's `qsort` function pointer but still type-safe.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What are C's two options for generic code, and what does each cost? Why can't `qsort`'s comparator be type-checked? (§30.1)
2. **R2.** What is monomorphization, and why is a Rust generic zero-cost at run time? (§30.2)
3. **R3.** What is a trait bound, what does each side gain, and which error reports an unmet one? (§30.3)
4. **R4.** What is a trait object, how is a method call on it dispatched, and how does that compare to `qsort`? (§30.4)
5. **R5.** When do you choose static dispatch vs. dynamic dispatch, and what is the cost of each? (§30.4)

FURTHER READING

- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 10.1 "Generic Data Types" and ch. 10.2 "Traits" (also free at `doc.rust-lang.org/book`).** The primary source for generics, monomorphization, traits, and trait bounds of §30.2–§30.3, including the `largest` and `Summary/notify` examples. Authors and edition verified via the publisher; not compiled here.
- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 17.2 "Using Trait Objects That Allow for Values of Different Types."** The source for trait objects, the value-plus-vtable representation, and dynamic dispatch of §30.4. Chapter number and title verified via the publisher's listing and `doc.rust-lang.org/book`.
- **The Rust Error Index, entry `E0277` (`doc.rust-lang.org/error_codes`).** The official meaning quoted in §30.3: "you tried to use a type which doesn't implement some trait in a place which expected that trait" — the unmet-trait-bound error. Verified against the current error index.
- **Stroustrup, *The C++ Programming Language*, 4th ed. (Addison-Wesley, 2013), on templates.** The C++ template model — instantiation-time errors deep in the body — that Rust's define-time bound checking of §30.3 improves on; useful contrast for where trait bounds come from and what they fix.

Exercises

30.1 WARM-UP In one sentence each, define a generic function, a trait, and a trait bound.

30.2 WARM-UP What is monomorphization, and why does it mean a generic function has no runtime dispatch cost?

30.3 WARM-UP What error code does the compiler produce when a generic is used with a type that does not satisfy its trait bound, and what does the message say?

30.4 CORE Take the C `qsort` of §30.1. (a) In what sense is it generic, and what are the two costs it pays? (b) Show concretely how passing a comparator written for the wrong element type would compile yet misbehave, and why the compiler cannot catch it. (c) Explain which Rust feature (generics or trait objects) corresponds to the `qsort` mechanism, and which gives the duplication-free, fully-checked, fully-inlined alternative.

30.5 CORE Explain the two-directional contract of a trait bound `T: Summary` on `fn notify<T: Summary>(item: &T)`. (a) What must the caller supply? (b) What may the body do that it could not without the bound? (c) Contrast Rust's checking the bound at the definition with C++'s reporting an unmet requirement inside the instantiated template body.

30.6 CORE Contrast static dispatch (generics) with dynamic dispatch (trait objects). (a) How is a call resolved in each — at compile time or run time — and what is the machine-level cost of each? (b) What is a trait object's runtime representation? (c) Give one situation that requires dynamic dispatch and one where static dispatch is clearly preferable.

30.7 COST Weigh the costs of monomorphization vs. trait objects. (a) What does monomorphization cost that dynamic dispatch does not, and in what resource? (b) What does dynamic dispatch cost per call that a monomorphized generic does not? (c) For a small trait method called in a tight loop over a homogeneous collection, which dispatch would you choose and why; for a large plugin interface with many implementations, which — and what is the deciding factor each time?

30.8 FADED *Worked, with the last step for you.* A newcomer writes `fn largest<T>(list: &[T]) -> T { let mut b = list[0]; for &x in list { if x > b { b = x; } } b }` and gets `error[E0277]: the trait bound `T: PartialOrd` is not satisfied` on the `x > b` line (and a second on the `Copy` requirement).
Step 1 (given): the body uses `>`, which requires `T` to have an ordering — the `PartialOrd` trait.
Step 2 (given): the body also copies elements out of the slice (`b = x`, `list[0]`), which requires `T: Copy`.
Step 3 (given): without those bounds, the compiler cannot know `T` supports the operations, so it rejects the definition — before any caller exists.
Step 4 (you): Give the corrected signature with the minimal bounds, explain why bounding at the definition (not the call site) is what lets the generic be checked once, and say why casting or wrapping to silence `E0277` is the wrong instinct.

30.9 MIXED For each, say whether static dispatch (generic + bound) or dynamic dispatch (trait object) is the better fit, and why: (a) summing a slice of any numeric type in a hot loop; (b) a `Vec` holding several different shape types that all implement `Draw`; (c) a function called at exactly one concrete type; (d) a rendering pipeline that loads drawable plugins chosen at run time.

30.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “A generic function carries a type parameter at run time and dispatches on it.” (b) “A trait object is dispatched at run time through a vtable.” (c) “`E0277` means your type has a bug.” (d) “Monomorphization makes binaries smaller.” (e) “Like C's `void*` generics, Rust trait objects give up type safety.”

30.11 STRETCH “A guarantee in Rust is usually a trait.” (a) Explain how `Copy` (Chapter 27) is a trait that governs whether assignment moves or copies, and how `T: PartialOrd` governs whether `>` is allowed — both the same mechanism. (b) Preview how `Option<T>` and `Result<T, E>` (next chapter) use generics to make error handling checkable. (c) Preview how thread-safety could be expressed as a trait requirement (`Send`/`Sync`): what would a bound like “`T: Send`” have to mean, and why would requiring it at a thread-spawn boundary turn a data race into a compile error?

30.12 LAB On your own machine (a C toolchain suffices; a Rust toolchain is ideal): (a) Compile and run the §30.1 `qsort` demo; then deliberately pass a comparator written for a different element type and confirm it still compiles (type erasure). (b) If you have `rustc`, write `largest<T: PartialOrd + Copy>`, call it at two types, and confirm it compiles; then remove the bounds and observe `E0277`; if not, explain from this chapter which line is rejected and which bound is missing. (c) Write a trait `Draw` with two implementors and put them in a `Vec<Box<dyn Draw>>`; explain why a monomorphized generic could not hold both in one vector. Note your toolchain versions.

Chapter 31 — Error Handling

> **Before you start:** Chapter 30 (traits and generics — `Option` and `Result` are generic enums, and `?` leans on the `From` trait, so this chapter is those tools put to work), Chapter 9 (manual allocation and what goes wrong — C signals failure with return codes, `errno`, and NULL, none of which the compiler makes you check), Chapter 27 (ownership — an error value is owned and moved like any other, which is why `?` can hand it back to the caller), Chapter 10 (undefined behavior — dereferencing a NULL you forgot to check is exactly the UB a type can rule out). · Chapter 30 promised that in Rust a guarantee is usually a type; this chapter is that promise applied to failure — "did you handle the error?" becomes a thing that does not compile.

> **BEFORE YOU READ ON**
>
> From memory, reaching back: (1) Chapter 30 — what is a generic type, and what does a trait bound like `T: From<E>` let a body assume? (2) Chapter 9 — name the ways C reports that an operation failed (a sentinel return value, `errno`, a NULL pointer); does the C compiler force the caller to inspect any of them? (3) *Predict:* in C, `FILE *fp = fopen(path, "r");` returns NULL when the file is missing; if you forget to check and call `fgets(buf, n, fp)`, does that *compile*? does it *run* correctly? In Rust, `File::open` returns a `Result`; can you use the file handle without first confronting the failure case? Write your guesses; §31.1 and §31.3 answer all three.

Every real program meets conditions it cannot control: a file is missing, a socket closes, a parse fails, an allocation returns nothing. The question is not whether errors happen but whether the language *makes you deal with them*. C's answer, from Chapter 9, is a set of conventions — a sentinel return value (`-1`, `NULL`), the global `errno`, sometimes a status code — all of which the compiler is happy to let you ignore. Rust's answer is to make failure a value with a type the compiler forces you to handle: **`Option<T>`** for "a value that might be absent" (there is no null), and **`Result<T, E>`** for "an operation that might fail." Both are the ordinary generic enums of Chapter 30, and handling them is ordinary pattern matching. First, how C signals failure and why every signal is silently ignorable, in real C that compiles and then segfaults (§31.1). Then `Option<T>`: the type that abolishes the null pointer and forces the absent case into the open (§31.2). Then `Result<T, E>` for recoverable failure, marked `#[must_use]` so ignoring it warns (§31.3). Then the **`?` operator**, which propagates an error to the caller in one character, and **`panic!`** for the unrecoverable — with the compile error `E0277` that fires when `?` is used in a function that returns neither (§31.4). We close by carrying error values forward into concurrency, where a panic on one thread and a `Result` crossing a channel are the next chapter's concern (§31.5).

On the code in this part. The book's environment has a C toolchain (gcc 11.4.0) but *no Rust compiler installed*, so the C programs below were compiled and run and their output is real, whereas the Rust snippets were *not* compiled here; their behavior and the quoted compiler diagnostics are drawn from *The Rust Programming Language* (Klabnik & Nichols, 2nd ed., 2023) and the official Rust error index, and are labeled as such. Nothing about Rust's behavior below is invented — it is cited.

31.1 How C signals failure — and why it is ignorable

C has no single error mechanism; it has conventions. A function returns a *sentinel* — `NULL` from `fopen` and `malloc`, `-1` from `read` and `open` — and often also sets the global `errno` to say why. The caller is supposed to check. Nothing enforces it. Here is real C that forgets one check, compiled and run in this environment (gcc 11.4.0):

```
FILE *fp = fopen("does-not-exist.txt", "r");   /* returns NULL on failure */
char buf[64];
fgets(buf, sizeof buf, fp);                    /* fp never checked for NULL */
printf("read: %s\n", buf);

$ gcc -O2 -Wall cerr.c -o cerr
cerr.c:5: warning: ignoring return value of 'fgets' ... [-Wunused-result]
$ ./cerr
Segmentation fault (core dumped)               # exit status 139
```

Look closely at what the compiler did and did not say. It emitted a warning — but about the wrong thing: `fgets` happens to be marked `warn_unused_result`, so ignoring *its* return drew a note. The actual bug — that `fp` is NULL and is dereferenced inside `fgets` — drew nothing, because checking `fopen`'s return is a convention, not a rule. The program compiles cleanly enough to ship and then segfaults on the missing file: the NULL dereference is the undefined behavior of Chapter 10, reached because the failure was expressible but not enforced. This is the recurring shape of C error handling: the information is there (the sentinel, `errno`), and the discipline to check it is entirely on you. Rust's move is to make the failure part of the return *type*, so there is no unchecked value to forget — the success value is only reachable by first accounting for the failure. That type is an enum, and we meet the simpler one first.

QUICK CHECK

Name three ways C reports failure. In the snippet above, which check was skipped, and why did the compiler not flag the real bug even with `-Wall`? What runtime event does the skipped check ultimately cause, and which chapter's concept is that? (Answer after the next section.)

31.2 `Option<T>`: there is no null

Rust has no null pointer. A value that may be absent has type **`Option<T>`**, a generic enum with two variants — `Some(T)` carrying a value, or `None`. Because absence is a distinct variant rather than a special pointer bit, the type system knows a `T` might not be there and refuses to let you use it as if it always is; you must pattern-match both cases (Rust; not compiled here — behavior per the cited book):

```
enum Option<T> { Some(T), None }        // in the standard library, generic over T

fn first_char(s: &str) -> Option<char> {
    s.chars().next()                     // Some(c) if non-empty, None if empty
}

match first_char(name) {
    Some(c) => println!("starts with {c}"),
    None    => println!("empty name"),   // the compiler forces this arm to exist
}
```

The point is what you *cannot* do: you cannot treat an `Option<char>` as a `char` and call methods on it, because it is a different type. To get at the inner value you must go through the `None` case — by `match`, or by a combinator like `unwrap_or`, or by `?` (§31.4). This is precisely the check C omitted in §31.1, now made unskippable: the "might be missing" is in the type, so the missing case cannot be silently dereferenced. It is the language-level fix for what C. A. R. Hoare, who introduced the null reference in 1965, later called his "billion-dollar mistake" — a value that inhabits every reference type yet supports no operation, so every use is a latent crash (Hoare, QCon London, 2009). `Option` puts the absent case back into the type where the compiler can see it, turning a whole family of C's NULL-dereference segfaults into a match arm you are made to write.

31.3 `Result<T, E>`: recoverable failure as a value

Absence is one thing; a failed operation that carries a *reason* is another. For that Rust has **`Result<T, E>`**, a generic enum over the success type `T` and the error type `E`, with variants `Ok(T)` and `Err(E)`. Fallible standard-library operations return it — `File::open` returns `Result<File, io::Error>` — and it is annotated `#[must_use]`, so ignoring one is a warning, not a shrug (Rust; not compiled here — behavior per the cited book):

```
enum Result<T, E> { Ok(T), Err(E) }      // generic over success T and error E

let f = File::open("data.txt");          // Result<File, io::Error>
let file = match f {
    Ok(handle) => handle,                // the File, only reachable through Ok
    Err(e)     => return Err(e),         // must confront the failure to proceed
};
// File::open("x");   // warning: unused `Result` that must be used  (#[must_use])
```

Compare this line by line with the C of §31.1. There, `fopen` returned a `FILE*` that was `NULL` on failure, and reaching the "file" without checking was legal and segfaulted. Here, `File::open` returns a `Result` whose `File` is *inside* the `Ok` variant: the only way to a usable `File` is through code that also names what to do on `Err`. If you throw the whole `Result` away, the `#[must_use]` attribute makes the compiler warn that a result that must be used was not — the diagnostic C's `warn_unused_result` gestured at, here applied to every fallible operation by construction. You still have escape hatches — `unwrap()` returns the `Ok` value or **panics** on `Err`, and `expect("msg")` does the same with your message — but they are visible, greppable choices to convert a recoverable error into a crash, not the silent default. Handling the error is the path of least resistance; ignoring it takes a deliberate keystroke.

QUICK CHECK

What are the two variants of `Result<T, E>`, and where does the success value live? What does `#[must_use]` buy you over C's conventions, and what do `unwrap()` / `expect()` do on an `Err`? (Answer below the communicate box.)

31.4 Propagation with `?`, and `panic!` for the unrecoverable

Matching every `Result` by hand would bury real logic under error plumbing. The **`?` operator** is the shorthand: on an expression of type `Result`, `x?` evaluates to the inner value if it is `Ok`, and otherwise *returns the `Err` from the enclosing function* — an early return that hands the error to the caller. When the caller's error type differs, `?` converts through the `From` trait of Chapter 30 (Rust; not compiled here — behavior per the cited book):

```
fn read_config(path: &str) -> Result<String, io::Error> {
    let mut f = File::open(path)?;       // if Err, return it now; else bind the File
    let mut s = String::new();
    f.read_to_string(&mut s)?;           // same: propagate on Err, continue on Ok
    Ok(s)                                // reached only if every step succeeded
}
```

Each `?` is the whole match-and-early-return of §31.3 compressed to one character, and the function reads as the happy path with the failures factored out. The catch — and it is a helpful one — is that `?` only makes sense in a function that itself returns a `Result` (or an `Option`): it needs somewhere to return the `Err` *to*. Use `?` in a function returning `()` and the compiler stops you with **`E0277`** — "the `?` operator can only be used in a function that returns `Result` or `Option`" (per the official Rust error index) — the same error code as an unmet trait bound in Chapter 30, because "this function can absorb a propagated error" is itself a trait requirement. The fix is to give the function a `Result` return type, not to reach for `unwrap`.

Not every failure is recoverable. When a program hits a bug or a violated invariant it cannot sensibly continue past — an index out of bounds, an assertion, a "this should be impossible" branch — it calls **`panic!`**, which unwinds the stack (running destructors) and aborts the thread. The distinction is the whole design: `Result` is for *expected, recoverable* conditions the caller should handle (a missing file, a bad request); `panic!` is for *unexpected, unrecoverable* ones that signal a bug (Klabnik & Nichols, 2023, ch. 9). Choosing `unwrap()` is choosing "if this is `Err`, that is a bug, so panic" — sometimes right (a hard-coded string you know parses), often a shortcut that turns a routine error into a crash. The rule of thumb: return a `Result` when the caller can do something about it; `panic!` only when continuing would mean running on broken assumptions.

THE SAME IDEA THREE WAYS: AN ERROR IS A VALUE THE TYPE SYSTEM FORCES YOU TO HANDLE

1. In words. Fallibility is encoded in the return type — `Option<T>` for absence, `Result<T, E>` for failure-with-reason — so the success value is reachable only through code that also accounts for the failure. `?` propagates an `Err` to the caller; `panic!` is reserved for unrecoverable bugs. Ignoring a `Result` warns (`#[must_use]`); using `?` without a place to return the error is `E0277`.

2. As a picture. Figure 31.1: one call fanning into two arms — `Ok(T)/Some(T)` to the happy path, `Err(E)/None` to a handler — with `?` drawn as an arrow from the `Err` arm straight back out to the caller, versus C's single return value with a dashed "did you remember to check?" branch that falls through to a NULL dereference.

3. As numbers/contrast. The C of §31.1 compiled and reached exit status 139 (SIGSEGV) on a missing file because the NULL check was optional. The Rust equivalent cannot reach the `File` without a matching `Err` handler; the same "forgot to check" is a compile-time warning (`#[must_use]`) or, for `?` in the wrong context, a compile error (`E0277`) — a runtime segfault turned into a message before the program runs.

(Answer to the §31.1 quick check: C reports failure by sentinel return values (`NULL`, `-1`), by setting `errno`, and by explicit status codes. The skipped check was `fopen`'s return for `NULL`; `-Wall` did not flag it because checking a return sentinel is a convention the compiler does not enforce — it warned only about ignoring `fgets`'s own `warn_unused_result`. The unchecked NULL is dereferenced inside `fgets`, causing a segmentation fault — the undefined behavior of Chapter 10.)

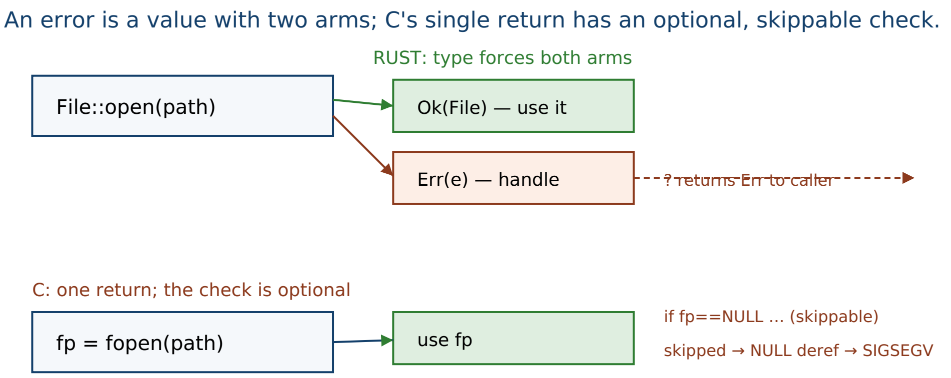


Figure 31.1: A `Result/Option` forces both arms — the success value lives inside `Ok/Some`, and `?` routes an `Err` back to the caller. C's single return value makes the failure check optional, and skipping it falls through to the NULL dereference of §31.1.

TRAP: REFLEXIVE `.UNWRAP()`, AND SILENCING `#[MUST_USE]` WITH `LET _ =`

Two related shortcuts undo everything this chapter buys. The first is reaching for `.unwrap()` or `.expect()` on every `Result` to make the types line up, because matching feels like ceremony. Each `unwrap` is a decision that "an `Err` here is an unrecoverable bug" — fine for a literal you know parses, but on a file open, a network read, or user input it converts a routine, recoverable error into a `panic!` that aborts the thread. The mechanical fix when the compiler complains that `?` needs a `Result`-returning function (`E0277`) is *not* to slap on `unwrap`; it is to give the function a `Result` return type and propagate. The second shortcut is silencing the `#[must_use]` warning with `let _ = fallible();` — the exact analog of ignoring C's return sentinel, now with a paper trail. Both are the §31.1 mistake wearing Rust syntax: the error is still there, you have just re-hidden it. Handle it, propagate it with `?`, or — if a failure truly is a bug — `panic!` *deliberately* with a message that says why; do not launder a recoverable error into a crash to quiet the compiler.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "`Result` and `?` everywhere are just checked exceptions with extra typing — in C I return `-1` and check it, which is simpler and I can see the control flow. Isn't all this enum stuff ceremony?" Cover the paragraph and respond in four or five sentences, drawing on §31.1–§31.4.

Model answer. "The difference isn't typing, it's what the compiler enforces. In C the `-1`-and-check is a convention — nothing stops you from skipping it, and when you do you don't get an exception, you get the §31.1 segfault at run time on a missing file. `Result<T, E>` puts the success value inside `Ok`, so you literally can't reach it without naming the `Err` case, and `#[must_use]` warns if you drop the whole thing — that's the check you keep meaning to write in C, made non-optional. And `?` is not hidden control flow like an exception: it's a visible, one-character early return that only compiles in a function that returns `Result`, so the error path is explicit and type-checked all the way up. It's also not exceptions — there's no invisible unwinding through ten stack frames; a panic is the separate, deliberate tool for *unrecoverable* bugs. So you get C's explicit control flow plus the compiler making sure you didn't forget the check." Framing `Result` as "the check you meant to write, enforced" and separating it cleanly from panics is the senior register.

(Answer to the §31.3 quick check: `Result<T, E>` has variants `Ok(T)` and `Err(E)`, and the success value lives inside `Ok` — reachable only through code that also names the `Err` case (a match, a combinator, or `?`). `#[must_use]` makes the compiler warn if you discard the whole `Result`, an enforcement C's return-value convention lacks. `unwrap()` returns the `Ok` value or `panic!`s on `Err`, and `expect("msg")` does the same with your message — visible, greppable ways to convert a recoverable error into a crash.)

31.5 What to carry forward

Error handling in Rust is the Chapter 30 pattern once more: name the possibility in a type, and let the compiler force the handling. **`Option<T>`** (`Some`/`None`) replaces the null pointer, so a value that might be absent cannot be used as if it were present — abolishing a whole class of C's NULL-dereference crashes. **`Result<T, E>`** (`Ok`/`Err`) makes a fallible operation return its success value *inside* the success variant and is `#[must_use]`, so the failure cannot be reached without being named and cannot be silently discarded. The **`?` operator** propagates an `Err` to the caller in one character, converting error types through `From`, and is only valid where there is a `Result` or `Option` to return to (`E0277` otherwise). **`panic!`** is the separate tool for the unrecoverable — a bug or violated invariant — and `unwrap()`/`expect()` are the visible, greppable way to opt into it, never the silent default. The whole design turns §31.1's optional-and-forgotten check, whose penalty was a runtime segfault, into a compile-time obligation.

The next chapter, **fearless concurrency**, is where these threads (literally) come together. Errors do not stop at a thread boundary: a worker thread can `panic!`, and its `Result` — of joining it, of the work it did — has to travel back to whoever spawned it, often across a channel that itself yields `Result` when the other end has hung up. And the deeper link is the one Chapter 30 previewed: just as `Result` encodes "this might fail" in a type, the traits **`Send`** and **`Sync`** encode "this is safe to move to, or share across, threads," lifting the ownership and borrowing discipline of Chapters 27–28 over the thread boundary — so the data races you had to reason about by hand in Chapter 24 become, like the unchecked errors here, a compile error rather than undefined behavior. Same move, applied to time and threads instead of to failure.

CAN YOU… WITHOUT LOOKING?

- Name three ways C signals failure and explain why each is silently ignorable (§31.1's segfault).
- State the variants of `Option<T>` and why it abolishes the null pointer.
- State the variants of `Result<T, E>`, where the success value lives, and what `#[must_use]` does.
- Explain what `?` does, how it converts error types, and why `E0277` fires when it is misused.
- Distinguish `Result` (recoverable) from `panic!` (unrecoverable), and say what `unwrap()` chooses.
- Explain why reflexive `unwrap` and `let _ =` re-create the C mistake.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** How does C signal failure, and why did the §31.1 program compile with `-Wall` yet segfault? (§31.1)
2. **R2.** What is `Option<T>`, and how does it eliminate a class of NULL-dereference bugs? (§31.2)
3. **R3.** What is `Result<T, E>`, where is the success value, and what does `#[must_use]` add over C? (§31.3)
4. **R4.** What does `?` do, what trait does it use to convert errors, and which error code reports its misuse? (§31.4)
5. **R5.** When do you return a `Result` vs. call `panic!`, and what does `unwrap()` really choose? (§31.4)

FURTHER READING

- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 9 "Error Handling" (also free at `doc.rust-lang.org/book`).** The primary source for `Result`, `panic!`, the `?` operator, and the recoverable-vs-unrecoverable guidance of §31.3–§31.4; ch. 6 covers `Option` and enums. Authors, edition, and chapter numbers verified via the publisher and the online book; not compiled here.
- **The Rust Error Index, entries `E0277` and `E0308` (`doc.rust-lang.org/error_codes`).** The official meanings used in §31.4: `E0277` covers "the `?` operator can only be used in a function that returns `Result` or `Option`" (an unsatisfied `FromResidual`/trait bound), and `E0308` is "mismatched types." Both verified against the current error index.
- **C. A. R. Hoare, "Null References: The Billion Dollar Mistake," QCon London, 2009 (recorded talk, InfoQ).** Hoare's account of introducing the null reference in ALGOL W (1965) and why it was a mistake — the motivation for `Option<T>` in §31.2. A talk, not a paper; speaker, venue, and year verified via the InfoQ recording.
- **The Rust standard-library documentation for `std::option::Option` and `std::result::Result` (`doc.rust-lang.org/std`).** The definitions, the `#[must_use]` annotation on `Result`, and the combinators (`unwrap`, `expect`, `unwrap_or`, `map`, `and_then`) referenced throughout. Reference for the exact API; verify against the current stable docs.

Exercises

31.1 WARM-UP In one sentence each, say what `Option<T>` and `Result<T, E>` represent, and name the two variants of each.

31.2 WARM-UP What does the `?` operator do to an expression of type `Result<T, E>` — what happens on `Ok`, and what on `Err`?

31.3 WARM-UP What is the difference between returning a `Result` and calling `panic!`, and which is `unwrap()` a shortcut for?

31.4 CORE Take the C of §31.1. (a) Which failure did the program not check, and how does C signal that failure? (b) Why did `gcc -Wall` not warn about the real bug, and what was the warning it *did* emit about? (c) Explain, in terms of Chapter 10, what runtime event the missing check causes and why it is undefined behavior.

31.5 CORE Explain how `Result<T, E>` makes the §31.1 mistake hard to repeat. (a) Where does the success value live, and what must you write to reach it? (b) What does `#[must_use]` add if you discard the whole `Result`? (c) In what sense is `unwrap()` the closest Rust analog to skipping C's check, and why is it at least visible?

31.6 CORE A function `fn load(path: &str)` with return type `()` contains `let f = File::open(path)?;` and fails to compile with `error[E0277]: the ? operator can only be used in a function that returns Result or Option`. (a) Why is `?` illegal here specifically? (b) Give the corrected signature. (c) Why is adding `.unwrap()` instead the wrong fix in general?

31.7 COST Consider the cost of the two error styles, not in CPU cycles but in *failure modes*. (a) What is the worst-case runtime outcome of a forgotten check in C (as in §31.1)? (b) What is the worst-case outcome of the corresponding omission in Rust — distinguishing "dropped the `Result`" from "reflexively `unwrap()`ed it"? (c) Argue which class of failure (compile-time diagnostic, warning, or runtime UB) each lands in, and why that ordering matters for a system you cannot easily debug in production.

31.8 FADED *Worked, with the last step for you.* A newcomer writes `fn first(path: &str) -> String { let s = std::fs::read_to_string(path)?; s }` in a function returning `String` and gets `error[E0277]` on the `?`.
Step 1 (given): `read_to_string` returns `Result<String, io::Error>`, so `?` may early-return an `io::Error`.
Step 2 (given): the function's declared return type is `String`, which has no place for that propagated `Err` to go — hence `E0277`.
Step 3 (given): to keep `?`, the function must return a `Result` whose error type the `io::Error` can become.
Step 4 (you): Give the corrected signature and body (returning `Result<String, io::Error>`), and explain why this is preferable to changing `?` to `.unwrap()` for a file that might legitimately be missing.

31.9 MIXED For each situation, say whether you would return `Result`, return `Option`, or `panic!`, and why: (a) looking up a key that may or may not be in a map; (b) parsing a network request whose bytes may be malformed; (c) indexing a slice with an integer your own code just computed and proved in-range; (d) opening a user-specified config file.

31.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "Rust's null pointer is written `None`." (b) "`?` works in any function." (c) "Dropping a `Result` is a compile error." (d) "`unwrap()` on an `Err` returns a default value." (e) "`?` can convert one error type into another on the way out."

31.11 STRETCH "An error is a value the type system forces you to handle." (a) Explain how this is the same mechanism as Chapter 30's trait bounds — what "requirement" is `?`-in-a-function stating, and why is its violation the *same* error code (`E0277`) as an unmet bound? (b) Show how `Option` and `Result` being *generic* (Chapter 30) is what lets one `?` work uniformly across every fallible operation. (c) Preview: if a worker thread's result must return to its spawner, what type would carry "the work, or why it failed," and how might a thread `panic!` be observed by the parent?

31.12 LAB On your own machine (a C toolchain suffices; a Rust toolchain is ideal): (a) Compile and run the §31.1 `cerr.c`; confirm it builds and observe the exit status on a missing file (expect a SIGSEGV, status 139). Add the missing `if (fp == NULL)` check and confirm the crash becomes a clean error message. (b) If you have `rustc`, write `read_config` from §31.4 returning `Result<String, io::Error>`, call it on a missing path, and observe that the error is returned rather than crashing; then change the return type to `()` and observe `E0277`. Without `rustc`, explain from this chapter which line each change affects. (c) Replace one `?` with `.unwrap()` and describe how the failure mode changes. Note your toolchain versions.

Chapter 32 — Fearless Concurrency

> **Before you start:** Chapter 24 (the memory model — a data race is undefined behavior you had to rule out by hand; this chapter makes it a compile error), Chapter 28 (borrowing — aliasing XOR mutation, the rule that `Send` and `Sync` lift across threads), Chapter 20 (mutual-exclusion locks — Rust's `Mutex` bundles the lock *with* the data it guards), Chapter 31 (error handling — joining a thread and receiving on a channel both yield `Result`). · This chapter finishes the Rust part: the aliasing discipline you proved by hand in the concurrency volume becomes two traits the compiler checks, so a data race stops compiling.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 24 — what exactly is a *data race*, and what is its status in C and C++ (defined behavior? undefined?)? (2) Chapter 28 — state the borrow rule in one line (how many mutable references may coexist with how many shared ones?). (3) *Predict:* in C, two threads each increment a shared `long` a million times with no lock; does the program *compile*? does it print the expected total? In Rust, can you hand the same `&mut` to two threads at once? Write your guesses; §32.1 and §32.3 answer all three.

The concurrency volume (Chapters 20–26) taught the hazards — data races, torn reads, lost updates, deadlock — and the tools to avoid them: locks, atomics, the memory model. What it could not give you was a *guarantee*. In C, whether your locking is correct is a property you argue for by inspection, and a single unlocked access to shared mutable state is undefined behavior (Chapter 24) that may work in testing and corrupt in production. Rust calls its alternative **fearless concurrency**: the same aliasing rule that made single-threaded code safe — one mutable reference or many shared ones, never both (Chapter 28) — is extended across threads by two marker traits, **`Send`** and **`Sync`**, so that sharing mutable state without synchronization does not type-check. First, the C data race, compiled and run so you can see it compile cleanly and print the wrong answer (§32.1). Then `Send` and `Sync`: what each means and how `thread::spawn` uses them to police the thread boundary (§32.2). Then the compiler stopping the §32.1 race with two named diagnostics — **`E0373`** (a closure would outlive its borrow) and **`E0277`** (a non-thread-safe type crossing threads) (§32.3). Then the safe way to share: **`Arc<Mutex<T>>`** and message-passing **channels**, where the `Mutex` holds the data it guards and joining or receiving yields the `Result` of Chapter 31 (§32.4). We close by completing the Rust part and turning toward performance (§32.5).

On the code in this part. The book's environment has a C toolchain (gcc 11.4.0) but *no Rust compiler installed*, so the C programs below were compiled and run and their output is real, whereas the Rust snippets were *not* compiled here; their behavior and the quoted compiler diagnostics are drawn from *The Rust Programming Language* (Klabnik & Nichols, 2nd ed., 2023), the Rustonomicon, and the official Rust error index, and are labeled as such. Nothing about Rust's behavior below is invented — it is cited.

32.1 The C data race, compiled and run

Here is the smallest interesting concurrency bug: two threads increment one shared counter, with no synchronization. Each `counter++` is a read-modify-write — load, add one, store — and if the two threads interleave between a load and its store, one update is lost (Chapter 24). Compiled and run in this environment (gcc 11.4.0), without optimization so each `++` is a real load/add/store:

```
static long counter = 0;                /* shared, no lock */
static void *bump(void *arg) {
    for (int i = 0; i < 2000000; i++) counter++;   /* a data race */
    return NULL;
}
// main: create two threads running bump, join both, print counter

$ gcc -O0 -Wall -pthread race.c -o race
$ ./race
expected 4000000, got 2453233                      # and a different number each run:
$ ./race                                            #   2284135, 2799577, 2159965, ...
expected 4000000, got 2284135
```

Two things to notice. First, it *compiled cleanly* — the data race is invisible to the C compiler; there is no rule in the language against two threads touching `counter` at once, only your discipline to add a lock. Second, the answer is *wrong and nondeterministic*: roughly two million of the four million increments vanished, and a different number vanishes each run, because the loss depends on the exact interleaving. This is the undefined behavior of Chapter 24 made concrete — the program has no defined meaning, so "works in testing" tells you nothing. The C fix is a mutex around `counter++`, which you must remember to place on *every* access; forget one and the race returns, silently. Rust's move is to make the sharing itself illegal unless it is synchronized — and to do it with the ordinary trait mechanism of Chapter 30.

> **QUICK CHECK**
>
> Why did the §32.1 program compile despite the data race? Why is the printed total both wrong and different each run, and which chapter's concept is that? What is the C fix, and what is its fragility? (Answer after the next section.)

32.2 `Send` and `Sync`: thread-safety as a trait

Rust encodes thread-safety in two **marker traits** — traits with no methods, which simply classify types. **`Send`** means a value of the type is safe to *transfer* (move) to another thread. **`Sync`** means it is safe to *share* a reference to the type across threads — precisely, `T` is `Sync` when `&T` is `Send` (Rustonomicon, "Send and Sync"). They are *auto traits*: the compiler derives them structurally, so a struct is `Send` if all its fields are, and you almost never write them by hand. Most types are both; the important exceptions are the ones that would make a race possible. Crucially, `thread::spawn` requires its closure to be `Send` and `'static`, which is where the borrow rule of Chapter 28 crosses the thread boundary (Rust; not compiled here — behavior per the cited sources):

```
// thread::spawn's bound (simplified): the closure F must be Send and 'static
pub fn spawn<F>(f: F) -> JoinHandle<...> where F: FnOnce() + Send + 'static { ... }

let data = vec![1, 2, 3];
let handle = thread::spawn(move || {     // `move` gives the closure ownership of data
    println!("{:?}", data);              // data is Send (Vec<i32> is Send), so this is
allowed
});
handle.join().unwrap();                  // wait for the thread; join yields a Result
```

The bound `F: Send + 'static` is the whole guarantee in one line. `Send` ensures every value captured by the closure is safe to move to the new thread; `'static` ensures the closure holds no borrow that could dangle when the parent frame ends (Chapter 29). What the type system will not let you do is give two threads simultaneous mutable access to the same data without synchronization — because that would require sharing something that is not `Sync`, or aliasing a `&mut`, both of which the borrow rules forbid. The `§32.1` C race required exactly that unsynchronized shared mutation; in Rust it cannot be expressed. The reference-counting pointer `Rc<T>` from Chapter 30's world of smart pointers is the canonical *not-`Send`* type: its reference count is a plain non-atomic integer, so sending an `Rc` to another thread could race on that count — and the compiler knows, because `Rc` is marked not `Send`.

> **QUICK CHECK**
>
> Define `Send` and `Sync` in one line each, and state the relationship between them. What two bounds does `thread::spawn` put on its closure, and what does each prevent? Why is `Rc` not `Send`? (Answer below the next section.)

32.3 The compiler stops the race: two named errors

Turn the §32.1 race into Rust and it does not compile — in one of two ways, each a diagnostic you should learn to read. The first arises when a closure *borrows* local data and is handed to a thread that may outlive the borrow. The compiler reports **`E0373`**, “closure may outlive the current function, but it borrows a value owned by the current function,” and suggests the fix: `move`, to give the closure ownership instead of a borrow (Rust; not compiled here — behavior per the cited sources):

```
let v = vec![1, 2, 3];
let h = thread::spawn(|| println!("{:?}", v));   // borrows v
error[E0373]: closure may outlive the current function, but it borrows `v`,
              which is owned by the current function
help: to force the closure to take ownership, use the `move` keyword
// fix: thread::spawn(move || println!("{:?}", v));
```

The second arises when you try to *share* a value that is not thread-safe. Suppose you reach for `Rc` to share a counter between threads: because `Rc` is not `Send`, the `Send` bound on `spawn`'s closure is unsatisfied, and the compiler reports **`E0277`** — the same trait-bound error as Chapters 30 and 31 — with a message naming the exact missing capability (per the official Rust error index):

```
let count = Rc::new(Mutex::new(0));
let h = thread::spawn(move || { *count.lock().unwrap() += 1; });   // Rc is !Send
error[E0277]: `Rc<Mutex<i32>>` cannot be sent between threads safely
note: the trait `Send` is not implemented for `Rc<Mutex<i32>>`
// fix: use Arc (atomic reference count), which IS Send + Sync
```

This is what "fearless" means. The §32.1 program that compiled and silently lost updates is, in Rust, a compile error that names the problem before the program runs — either "this borrow could dangle, take ownership" (`E0373`) or "this type is not safe to send/share across threads" (`E0277`). And both are ordinary consequences of machinery you already have: `E0373` is the lifetime rule of Chapter 29 applied to a closure that escapes to a thread; `E0277` is the trait-bound check of Chapter 30 applied to `Send`. The data race did not become easier to avoid by convention — it became impossible to write, because the type of the shared value refuses to cross the boundary unsynchronized.

THE SAME IDEA THREE WAYS: THREAD-SAFETY IS A TRAIT THE COMPILER CHECKS, NOT A DISCIPLINE YOU KEEP

1. In words. `Send` (safe to move to a thread) and `Sync` (safe to share across threads, i.e. `&T` is `Send`) are auto traits the compiler derives structurally. `thread::spawn` requires its closure to be `Send + 'static`, so unsynchronized shared mutation — the §32.1 race — cannot be expressed; attempting it is `E0373` (dangling borrow) or `E0277` (a non-`Send` type crossing threads).

2. As a picture. Figure 32.1: two threads reaching for one box of data; a "`Send/Sync` gate" on the boundary passes an `Arc<Mutex<T>>` (thread-safe) and rejects a bare `&mut` or an `Rc` with a red `E0277/E0373` stamp — versus C's boundary, which is wide open and lets the unlocked `counter++` through to a lost update.

3. As numbers/contrast. The C race of §32.1 compiled and printed `2453233` of an expected `4000000` — a different wrong number each run. The Rust equivalent produces *zero* runs, wrong or right: it stops at compile time with `E0373` or `E0277`. A nondeterministic runtime corruption becomes a deterministic message before execution.

(Answer to the §32.1 quick check: it compiled because C has no rule against two threads touching shared mutable state — the data race is invisible to the compiler, only to your discipline. The total is wrong because increments interleave and lose updates, and different each run because the loss depends on the nondeterministic interleaving — the undefined behavior of Chapter 24. The C fix is a mutex around every access to `counter`; its fragility is that you must remember it on every access, and one omission silently reintroduces the race.)

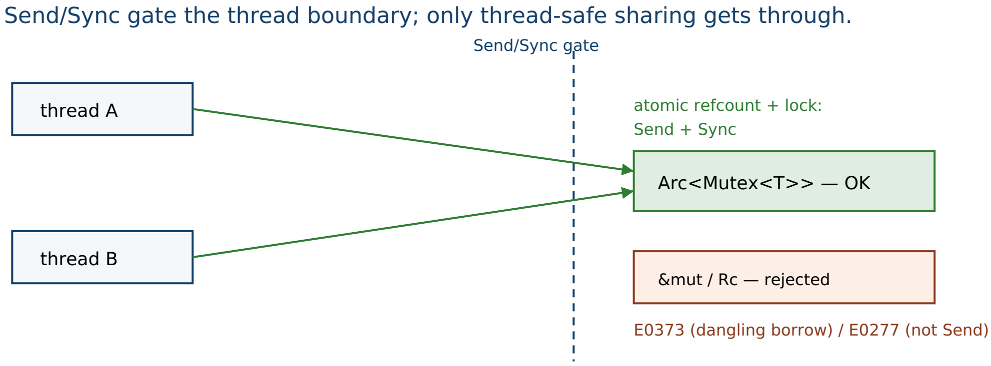


Figure 32.1: The `Send`/`Sync` gate on the thread boundary. A thread-safe `Arc<Mutex<T>>` passes; a bare `&mut` or a non-`Send` `Rc` is rejected at compile time with `E0373` or `E0277`. C's boundary has no gate, so the unsynchronized `counter++` of §32.1 goes through to a lost update.

32.4 Sharing safely: `Arc<Mutex<T>>` and channels

Two patterns cover most shared state. The first is **shared mutable state behind a lock**. To share ownership of a value across threads you wrap it in **`Arc<T>`** — an *atomic* reference-counted pointer, the `Send + Sync` sibling of `Rc` whose count is updated with the atomics of Chapter 22 — and to mutate it you wrap the inner value in **`Mutex<T>`**. The decisive difference from Chapter 20's C mutex is that Rust's `Mutex` *owns the data it guards*: there is no way to reach the value without calling `lock()`, which returns a guard granting access and releases the lock automatically when the guard is dropped (Rust; not compiled here — behavior per the cited book):

```
let counter = Arc::new(Mutex::new(0));           // shared, lock bundled with the data
let mut handles = vec![];
for _ in 0..10 {
    let c = Arc::clone(&counter);               // bump the atomic refcount; move the clone
    handles.push(thread::spawn(move || {
        let mut n = c.lock().unwrap();           // acquire lock; `n` is a guard, not an int
        *n += 1;                                 // the ONLY way to the data is through the
lock
    }));                                         // guard dropped here -> lock released
}
for h in handles { h.join().unwrap(); }
// *counter.lock().unwrap() == 10, deterministically — no lost updates
```

Because the integer lives *inside* the `Mutex`, the Chapter 20 mistake of accessing shared data without holding its lock is not a discipline you maintain — it is unrepresentable, since the only handle to the data is the guard `lock()` hands back. Two threads of Chapter 31 also reappear here: `lock()` returns a `Result` because a `Mutex` becomes *poisoned* if a thread panics while holding it (so the `unwrap()` is you deciding a poisoned lock is unrecoverable), and `join()` returns a `Result` whose `Err` carries a panicked thread's payload. The second pattern is **message passing**: rather than share memory, threads communicate over a **channel** (`std::sync::mpsc`), where sending a value *moves* its ownership down the channel (Chapter 27), so the sender can no longer touch it and there is nothing to race on. Receiving returns a `Result` — `Err` when the sending end has hung up. Both patterns turn Chapter 24's shared-memory hazards into ownership and type problems the compiler checks.

TRAP: `UNSAFE IMPL SEND` TO SILENCE `E0277`, AND TREATING `MUTEX` AS DEADLOCK-PROOF

Two failure modes, one at each end of caution. The first is over-riding the compiler: seeing `error[E0277]: Rc<...> cannot be sent between threads safely` and writing `unsafe impl Send for MyType {}` to make it compile. That does not make the type thread-safe; it *asserts* to the compiler that it is, switching off the exact check that was protecting you and reintroducing the §32.1 race with a signed confession (`unsafe`) attached. The right fix is almost always to use the thread-safe type — `Arc` for `Rc`, an atomic for a plain counter — not to claim `Send` you have not earned. The second failure is the opposite complacency: assuming `Arc<Mutex<T>>` makes concurrency "safe" in every sense. It abolishes *data races* — unsynchronized access is impossible — but it does *not* abolish the higher-level hazards of Chapter 20: two locks acquired in opposite orders by two threads still **deadlock**, and holding a `Mutex` across a long or blocking operation still serializes everything (Chapter 26's contention). Rust guarantees you locked the data; it does not guarantee your *lock ordering* or *granularity* is sane. Fearless means no data races, not no concurrency bugs.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "'Fearless concurrency' is marketing. A mutex is a mutex; Rust just makes me fight the borrow checker across threads, and I can deadlock in Rust same as in C. What does it actually buy me?" Cover the paragraph and respond in four or five sentences, drawing on §32.1–§32.4.

Model answer. "It buys you exactly one thing, and it's the thing that ruins your week: it makes *data races* — unsynchronized access to shared mutable state — a compile error instead of undefined behavior. The C version in §32.1 compiled fine and printed a different wrong number every run; the Rust version won't compile until the shared thing is `Send`/`Sync`, which in practice means you wrapped it in `Arc<Mutex>` or an atomic — so 'I forgot to lock this one access' is unrepresentable, because the data lives inside the lock. You're right that it does *not* save you from deadlock or from holding a lock too long — those are lock-ordering and granularity bugs, not data races, and Rust doesn't check them. So the honest claim is narrow but huge: the entire category of nondeterministic memory-corruption concurrency bugs, the ones that pass tests and page you at 3 a.m., is gone at compile time; the logical concurrency bugs are still yours to design around." Naming precisely what Rust does and does not guarantee — races yes, deadlock no — is the senior register.

(Answer to the §32.2 quick check: `Send` means a value is safe to move to another thread; `Sync` means it is safe to share a reference across threads, i.e. `T: Sync` exactly when `&T: Send`. `thread::spawn` bounds its closure `Send + 'static`: `Send` so every captured value can move to the new thread, `'static` so the closure holds no borrow that could dangle after the parent returns. `Rc` is not `Send` because its reference count is a non-atomic integer, so sending it across threads could race on that count.)

32.5 What to carry forward

Fearless concurrency is the last application of the pattern this whole part has repeated: name a property in the type system and let the compiler enforce it. A **data race** — the undefined behavior of Chapter 24 — is prevented not by discipline but by two marker traits: **`Send`** (safe to move to a thread) and **`Sync`** (safe to share across threads, `&T: Send`), auto-derived structurally, required by `thread::spawn` as `Send + 'static`. Attempting the unsynchronized sharing of §32.1 is a compile error — **`E0373`** (a borrow that would outlive its owner, fixed by `move`) or **`E0277`** (a non-`Send` type crossing threads, fixed by the thread-safe type). Safe sharing uses **`Arc<Mutex<T>>`**, where the `Mutex` owns the data so it cannot be touched without the lock, or **channels**, where sending moves ownership so there is nothing to race on — with poisoning and hang-ups surfacing as the `Result`s of Chapter 31. And the guarantee is honest about its edge: it kills data races, not deadlock or contention.

That completes the Rust part, and with it the argument the last six chapters have been making. Across the C volumes you proved invariants by hand: memory freed exactly once and used only while valid, references that never dangle, shared state accessed only under a lock, no data races. Rust encodes each as something the compiler checks — **ownership and moves** (Chapter 27), **borrowing** (Chapter 28), **lifetimes** (Chapter 29), **traits and generics** (Chapter 30), **error handling** (Chapter 31), and now **data-race freedom** — all with the same machine underneath and, after monomorphization and lifetime erasure, the same generated code. What Rust does *not* give you is speed: a memory-safe, race-free program can still be slow, cache-hostile, and blocked on I/O. The next part turns to that. It opens with the discipline the whole book has implied and never stated outright — **measure before you optimize** — because the cost model of Chapter 5 is only useful if you can see where the time actually goes. Safety was the compiler's job; performance is yours, and it starts with a profiler, not a guess.

CAN YOU… WITHOUT LOOKING?

- Explain why the §32.1 C program compiled yet printed a wrong, different-each-run total.
- Define `Send` and `Sync` and state the relationship between them.
- Say what bounds `thread::spawn` puts on its closure and what each prevents.
- Name the two compile errors that stop the race (`E0373`, `E0277`) and the fix for each.
- Explain how `Mutex<T>` owning its data makes "access without the lock" unrepresentable.
- State precisely what fearless concurrency does and does not guarantee (races vs. deadlock).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why did the §32.1 program compile, and why is its output wrong and nondeterministic? (§32.1)
2. **R2.** Define `Send` and `Sync`; what does `thread::spawn` require, and why is `Rc` not `Send`? (§32.2)
3. **R3.** What do `E0373` and `E0277` each report in the concurrency setting, and what fixes each? (§32.3)
4. **R4.** How does `Arc<Mutex<T>>` make unsynchronized access impossible, and where does `Result` appear? (§32.4)
5. **R5.** What does fearless concurrency guarantee, and what class of concurrency bug does it *not* prevent? (§32.4)

FURTHER READING

- **Klabnik & Nichols, *The Rust Programming Language*, 2nd ed. (No Starch Press, 2023), ch. 16 "Fearless Concurrency" (also free at `doc.rust-lang.org/book`).** The primary source for `thread::spawn`, the `move` closure, `Arc<Mutex<T>>`, channels, and `Send`/`Sync` of §32.2–§32.4; ch. 15 covers `Rc`/smart pointers. Authors, edition, and chapter numbers verified via the publisher and the online book; not compiled here.
- **The Rustonomicon, "Send and Sync" (`doc.rust-lang.org/nomicon/send-and-sync.html`).** The authoritative account of the two marker traits used in §32.2: what each means, why they are auto traits derived structurally, and why `Rc` opts out. Reference for the exact semantics; verify against the current edition.
- **The Rust Error Index, entries `E0373` and `E0277` (`doc.rust-lang.org/error_codes`).** The official meanings used in §32.3: `E0373` "closure may outlive the current function… use the `move` keyword," and `E0277` for an unsatisfied `Send` bound ("cannot be sent between threads safely"). Both verified against the current error index.
- **Jung, Jourdan, Krebbers & Dreyer, "RustBelt: Securing the Foundations of the Rust Programming Language," *Proc. ACM on Programming Languages* 2(POPL), Article 66 (2018), DOI 10.1145/3158154.** The formal proof that Rust's type system — including `Send`/`Sync` and the aliasing discipline — actually guarantees data-race freedom. First author Ralf Jung; verified via the ACM Digital Library.

Exercises

32.1 WARM-UP In one sentence each, define `Send` and `Sync`, and state the relationship between them.

32.2 WARM-UP What is a data race, and what is its status in C (per Chapter 24)? What is its status in safe Rust?

32.3 WARM-UP What does the `move` keyword do to a closure passed to `thread::spawn`, and which error does its absence often cause?

32.4 CORE Take the C of §32.1. (a) Why did it compile despite the data race? (b) Explain, in read-modify-write terms, why the total is both too low and different each run. (c) State the C fix and one way it is fragile that Rust's approach is not.

32.5 CORE Explain how `thread::spawn`'s bound `F: Send + 'static` prevents the §32.1 race. (a) What does the `Send` part rule out? (b) What does the `'static` part rule out, and which chapter's concept is that? (c) Why does trying to share an `Rc` across threads violate the `Send` part specifically?

32.6 CORE Contrast `E0373` and `E0277` as they appear in concurrent code. (a) What does each report, and what borrowing/ trait concept from earlier chapters is each an instance of? (b) Give the standard fix for each. (c) Why is `unsafe impl Send` the wrong fix for `E0277` here?

32.7 COST Weigh `Arc<Mutex<T>>` against message passing for a shared counter updated by many threads. (a) What does the `Mutex` approach cost per update, and what Chapter 26 hazard grows as thread count rises? (b) What does the channel approach cost, and what does moving ownership down the channel buy you? (c) For a high-contention counter specifically, name a third option from Chapter 22 that avoids a lock entirely, and say when you would still prefer the `Mutex`.

32.8 FADED *Worked, with the last step for you.* A newcomer shares a counter with `let c = Rc::new(Mutex::new(0));` and spawns threads that each do `let cc = c.clone(); thread::spawn(move || *cc.lock().unwrap() += 1);`, getting `` error[E0277]: `Rc<Mutex<i32>>` cannot be sent between threads safely ``.
Step 1 (given): `thread::spawn` requires the closure — and thus everything it captures — to be `Send`.
Step 2 (given): `Rc` is not `Send` because its reference count is non-atomic, so the bound fails: `E0277`.
Step 3 (given): the fix is a shared pointer whose count is atomic and which is therefore `Send + Sync`.
Step 4 (you): Name the replacement type, rewrite the two changed lines, and explain why this is a real fix whereas `unsafe impl Send for ...` would only silence the check.

32.9 MIXED For each, say whether it is prevented by safe Rust's type system, and if so how (or if not, why not): (a) two threads incrementing a shared `i32` with no synchronization; (b) two threads acquiring locks `A` then `B` vs. `B` then `A` (deadlock); (c) sending an `Rc` to another thread; (d) holding a `Mutex` guard across a slow network call (serializing everyone).

32.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "`Send` and `Sync` have methods you implement." (b) "A Rust `Mutex` can be unlocked-and-accessed like C's, with the data separate from the lock." (c) "`Arc` is just `Rc` with an atomic reference count." (d) "Fearless concurrency prevents deadlock." (e) "Sending a value down a channel copies it and leaves the sender a usable copy."

32.11 STRETCH "A guarantee in Rust is usually a trait." Tie the whole part together. (a) Show how `Send`/`Sync` lift the Chapter 28 borrow rule (aliasing XOR mutation) across the thread boundary — what would sharing a `&mut` between two threads violate? (b) Explain why the concurrency errors reuse the *same* error code (`E0277`) as an unmet bound in Chapter 30 and a misused `?` in Chapter 31. (c) In two sentences, state the through-line of Chapters 27–32: what did the C volumes make you prove by hand, and what does Rust do with each such proof?

32.12 LAB On your own machine (a C toolchain suffices; a Rust toolchain is ideal): (a) Compile the §32.1 `race.c` at `-O0` and run it several times; record the totals and confirm they are below `4000000` and vary. Then compile at `-O2` and note whether the race still manifests, and hypothesize why the optimizer changed the outcome. (b) Add a `pthread_mutex_t` around `counter++` and confirm the total becomes exactly `4000000`. (c) If you have `rustc`, write the `Arc<Mutex<i32>>` version from §32.4 and confirm it is deterministic; then try the `Rc` version and observe `E0277`. Without `rustc`, explain from this chapter which line each error points at. Note your toolchain versions.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART VI

Performance Engineering

Measure before you optimize: profiling, benchmarking, cache-aware and data-oriented design, SIMD, and reasoning about where time actually goes.

Chapter 33 — Measure First: Profiling and Benchmarking

Before you start: Chapter 5 (mechanical sympathy and the cost model — this part makes that model operational, by measuring where the costs land), Chapter 26 (Amdahl's law — the ceiling on speeding up a part, and why the dominant cost is the only one worth attacking), Chapter 10 (the optimizing compiler — dead-code elimination will silently wreck a naive benchmark), Chapter 3 (memory hierarchy and caches — one of the main places the time actually goes). · This chapter opens the performance part with the discipline the whole book has implied: *measure before you optimize*, because a guess about where time goes is almost always wrong.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 26 — state Amdahl's law in one line; if a function is 5% of total runtime and you make it infinitely fast, what is the most you can speed up the whole program? (2) Chapter 10 — what is the optimizing compiler permitted to do to a computation whose result is never used? (3) *Predict:* you write a loop that sums a billion products into a variable, never read the variable, and time it at `-O2`; what elapsed time do you expect? And if you run that timing exactly once, is the number you get representative? Write your guesses; §33.2 and §33.3 answer all three.

A note on this part before we begin. The previous five parts built up, from the hardware to the type system, a *model* of what a program costs to run. This part is about the other half of engineering with that model: finding out where the cost actually falls, which is almost never where intuition says. The cardinal rule is **measure before you optimize**. Programmers are notoriously bad at guessing bottlenecks — you spend an afternoon hand-tuning a function that turns out to be 2% of runtime while the real cost sits in an allocation or a cache-missing loop you never suspected. Donald Knuth's much-abused line — “premature optimization is the root of all evil” — is precisely this point: he argued that we should forgo optimization in the non-critical 97% of code and instead find, by measurement, the critical few percent where effort pays (Knuth, 1974). So this chapter is the toolset for that finding. First, why measuring is not optional and how Amdahl's law decides *what* to measure (§33.1). Then the single most common way a benchmark lies to you — the optimizer deleting the work you meant to time — shown in real C where the same loop reports 0.0000 s or 1.56 s depending only on whether its result is used (§33.2). Then how to time correctly: which clock, and why you run many trials and report the minimum rather than one number (§33.3). Then **profiling** — sampling vs. instrumentation, and reading a profile as a map of where the time went, so Amdahl points your effort (§33.4). We close by making the measure–optimize loop explicit and turning toward the specific costs the rest of the part attacks (§33.5).

On the code in this part. Unlike the Rust part, the programs here are C, and they were compiled with gcc 11.4.0 and run in this environment, so their *output is real*. Absolute times, however, are hardware- and load-specific: the numbers below are illustrative of the *effect* (a deleted loop, run-to-run variance, a lopsided phase split), not benchmarks of any particular machine, and your own numbers will differ. Where a tool is named that is *not* installed here (`perf`, `gprof`, `valgrind`), it is discussed from its documentation and cited, not run.

33.1 Measure before you optimize

The discipline has two halves, and Amdahl's law (Chapter 26) supplies the second. The first half is: do not optimize on a hunch, because hunches about performance are wrong at a rate that would be comical if it were not so expensive. The compiler, the cache, the allocator, and the branch predictor all move the real cost somewhere other than the arithmetic you can see in the source. The second half is: even once you know a function is slow, speeding it up is only worth it in proportion to its *share of total runtime*. This is Amdahl's law read as a budgeting rule. If a function is 5% of runtime, then making it infinitely fast — not twice as fast, *infinitely* — speeds the whole program by at most 5% (from 100 units to 95). If another is 90% of runtime, a mere 2× speedup on it saves 45 units, nine times the entire budget of the first. So the order is fixed: *find the dominant cost first, then optimize it*, and measurement is the only way to know which is which. Optimizing before measuring is not just wasted effort; it is effort spent, by the arithmetic of Amdahl, on the part that mathematically cannot matter. Everything else in this chapter serves this: benchmarking to compare two implementations honestly (§33.2–§33.3), and profiling to locate the dominant cost in the first place (§33.4).

> **QUICK CHECK**
>
> State the two halves of “measure before you optimize.” Using Amdahl's law, how much can you speed up a program by perfectly optimizing a part that is 5% of its runtime? Why does that make locating the *dominant* cost the first job? (Answer after the next section.)

33.2 The benchmark that lies: dead-code elimination

The most common benchmarking bug is that the compiler deletes the very work you are trying to time. From Chapter 10: if a computation's result is never observed, the optimizer is free to remove the computation entirely — it is not required to run code whose output no program can see. A microbenchmark that computes into a variable and never uses it hands the optimizer exactly that license. Here is the same loop compiled two ways, differing only in whether the result is consumed — compiled and run in this environment (gcc 11.4.0, `-O2`):

```
double s = 0.0;
for (long i = 0; i < 1000000000L; i++) s += (double)(i & 1023) * 0.5;
// version A: never use s.   version B: consume s (print it / store to volatile).

$ gcc -O2 bench.c -o nosink && ./nosink
no sink   : 0.0000 s                         # the loop was deleted entirely
$ gcc -O2 -DSINK bench.c -o sink && ./sink
with sink : 1.5592 s  (checksum 255749934464.0)   # the loop actually ran
```

Version A reports `0.0000 s` not because a billion iterations are free but because they never happened: with `s` unused, the compiler proved the loop had no observable effect and removed it, exactly as Chapter 10 said it may. A newcomer reads "0.0000 s" as "my code is blazing fast" and ships a benchmark that measures nothing. Version B prints the checksum, so `s` is observed, so the loop must run, and now you see its real cost — 1.56 s here. The fix is a rule: **make the result observable**, by printing a checksum, writing to a `volatile`, or feeding it to a compiler-opaque sink, so the optimizer cannot elide the work. Two companion rules follow. Benchmark the build you will *ship* — measure at `-O2`, not `-O0`, since the debug build's costs are not the release build's. And be sure the input is not a compile-time constant the optimizer can fold away. The through-line is that a benchmark is only honest if the work it claims to time is work the compiler was actually forced to do.

> **QUICK CHECK**
>
> Why did version A report `0.0000 s` — what did the compiler do, and by which Chapter 10 rule? What single change turned the loop back on? Give two further rules for a benchmark that is not lying to you. (Answer below the communicate box.)

33.3 Timing correctly: which clock, and how many runs

Suppose the loop is safe from the optimizer. Two questions remain: *which clock* you read, and *how many times*. For elapsed time you want a monotonic wall clock that never jumps — on Linux, `clock_gettime(CLOCK_MONOTONIC, &t)`, which is immune to wall-time adjustments (NTP, daylight saving) that can make a naive timer run backwards. (Wall-clock time and CPU time differ: a program blocked on I/O consumes little CPU while wall time passes, so choose the one that matches the question you are asking.) The second question matters more than beginners expect: *a single measurement is nearly meaningless*, because runtime varies from run to run due to the OS scheduler, cache and TLB state, frequency scaling, and other tenants on the machine. You must run the workload several times. Compiled and run here (gcc 11.4.0, the same kernel run five times):

```
for (int r = 0; r < 5; r++) { t0 = now(); sink += work(300000000L); t1 = now();
                              printf("trial %d: %.4f s\n", r+1, t1 - t0); }

$ gcc -O2 trials.c -o trials && ./trials
trial 1: 0.6824 s
trial 2: 0.7081 s
trial 3: 0.6694 s
trial 4: 0.8198 s     <- an outlier: the OS scheduled something else here
trial 5: 0.6743 s
min over 5 trials: 0.6694 s
```

The five runs of identical work span 0.6694 s to 0.8198 s — a 22% spread, from nothing the program did. Reporting any single one of them, especially the first (cold caches) or the fourth (an interruption), would misrepresent the code. The usual practice is to run many trials and report the **minimum** (the run least disturbed by outside interference, closest to the code's intrinsic cost) or the median, and to state the spread rather than hide it behind one number; a first, warm-up run is often discarded because it pays cold-cache and page-fault costs the steady state will not. The mean is a poor summary here because it is dragged upward by one-directional outliers like trial 4 — interference only ever makes a run *slower*, never faster, so the minimum is the more principled floor. The discipline, then: monotonic clock, warm up, many trials, report the minimum and the spread. A benchmark that reports one number from one run is a benchmark you cannot trust.

THE SAME IDEA THREE WAYS: A MEASUREMENT IS ONLY AS HONEST AS ITS METHOD

1. In words. Optimize only after measuring, and only the dominant cost (Amdahl). Make the timed work observable so the optimizer cannot delete it; time it at the shipping optimization level with a monotonic clock; run many trials and report the minimum and spread, not one number. Then profile to find the dominant cost before you touch anything.

2. As a picture. Figure 33.1: a pipeline — *profile* (find the hot spot) → *benchmark* the hot spot honestly (observable result, `-O2`, many trials) → *optimize* → *re-measure* → stop when good enough — with two red "lies" branching off: a deleted loop reporting 0.0000 s, and a single run reporting an outlier.

3. As numbers/contrast. The same billion-iteration loop reported `0.0000 s` (deleted) vs. `1.5592 s` (real) depending only on whether its result was used; the same kernel run five times reported `0.6694–0.8198 s`, a 22% spread. Two different methods, two different "answers" for identical code — which is why the method, not the stopwatch, is the measurement.

(Answer to the §33.1 quick check: the two halves are (1) do not optimize on a hunch — measure, because bottlenecks are rarely where you guess — and (2) optimize only in proportion to a part's share of runtime, per Amdahl's law. Perfectly optimizing a part that is 5% of runtime speeds the whole program by at most 5% (100 units become 95), even if that part becomes infinitely fast. That is why finding the dominant cost is the first job: effort on a small part cannot pay off no matter how good the optimization, so you must locate the part that dominates before optimizing anything.)

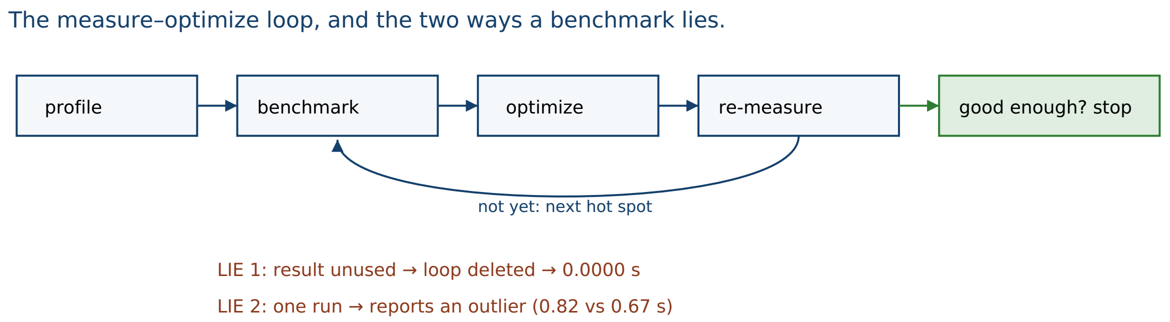


Figure 33.1: The measure–optimize loop — profile to find the hot spot, benchmark it honestly, optimize, re-measure, and stop when good enough — with the two classic benchmark lies: a deleted loop reporting `0.0000 s` (§33.2) and a single run reporting an outlier (§33.3).

33.4 Profiling: where the time actually goes

Benchmarking compares two versions of a known piece of code; **profiling** answers the prior question — *which* piece of code to look at. A profiler attributes runtime to the functions (and lines) that spend it, so Amdahl's dominant cost stops being a guess. There are two families. An **instrumenting** profiler (the classic `gprof`) inserts counting and timing code at function entry and exit; it is exact about call counts but perturbs the very timing it measures and can distort the picture for tiny, hot functions (Graham, Kessler & McKusick, 1982). A **sampling** profiler (Linux `perf`, and most modern tools) instead interrupts the running program many times per second and records where the program counter is; over enough samples this builds a statistical map of where time is spent, with negligible overhead and no recompilation (Gregg, 2020). The output is often drawn as a **flame graph** — stacked bars whose width is time spent — so the widest box is, quite literally, where to optimize. To make the payoff concrete, here is a two-phase program timed by hand, the poor engineer's profile, compiled and run here (gcc 11.4.0, `-O2`):

```
t0 = now(); parse(...);   t1 = now();     // a setup phase
            compute(...); t2 = now();     // the main work

$ gcc -O2 profile.c -o profile && ./profile
parse   : 0.1577 s  ( 6.4% of runtime)
compute : 2.2904 s  (93.6% of runtime)
```

The profile is unambiguous: `compute` is 93.6% of runtime, `parse` only 6.4%. Amdahl now dictates strategy. Every hour spent optimizing `parse` buys at most a 6.4% whole-program improvement even if you make it vanish; a modest 30% cut to `compute` saves 0.30 × 93.6% = 28% of the whole. A programmer who *guessed* — who found `parse`'s string handling ugly and spent the afternoon there — would have optimized the part that cannot matter, which is exactly the failure §33.1 warned of. This is why the loop is *profile first*: the profiler replaces the guess with a measurement, points you at the 93.6% box, and only then do you benchmark candidate improvements to it honestly (§33.2– §33.3). A real profiler does this across hundreds of functions you would never think to time by hand — but the logic is the one above, scaled up: measure where the time is, attack the widest box, re-measure, stop.

TRAP: OPTIMIZING WITHOUT A PROFILE, AND TRUSTING A BENCHMARK THAT MEASURED NOTHING

Two mistakes, both the “measure first” rule violated from opposite ends. The first is optimizing on intuition: rewriting the function that *looks* expensive — a clever-looking loop, some string formatting — without a profile showing it is expensive. Nine times in ten it is not the bottleneck, and you have added complexity and risk to code that was 3% of runtime while the real 90% sits untouched; worse, you now believe the problem is “handled.” Always get a profile first, then optimize the box the profile makes wide. The second mistake is trusting a benchmark that never ran the work: the `0.0000 s` of §33.2 (dead-code elimination), a loop over a constant the compiler folded, a measurement taken at `-O0` that has nothing to do with the shipped build, or a single run that happened to be an outlier. Both mistakes feel like measurement — you ran something, you got a number — which is what makes them dangerous. The number is only meaningful if the method behind it (a profile to choose the target, an observable result, the shipping optimization level, many trials) is sound. A confident optimization built on a lying benchmark is worse than no optimization, because you will defend it.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I profiled nothing, but I'm sure the JSON parsing is our bottleneck — it just *looks* slow. I rewrote it with a hand-tuned loop and my microbenchmark says it's 100× faster. Ship it?" Cover the paragraph and respond in four or five sentences, drawing on §33.1–§33.4.

Model answer. "Two problems, and both are 'measure first.' First, 'looks slow' isn't a measurement — we need a profile of the real workload showing parsing is actually a big share of runtime, because if it's 5% of the total, Amdahl says even your 100× win is at most a 5% whole-program improvement, and you've added a hand-tuned loop we now have to maintain. Second, a 100× microbenchmark speedup is exactly the number I distrust: it usually means the compiler deleted the work because the result was unused, or you benchmarked a constant, or you compared a `-O0` build to a `-O2` one — make the result observable, benchmark at the shipping optimization level, and run it many times. So before we ship: profile the real path, confirm parsing is the dominant cost, then re-run your benchmark honestly and show the whole-program time moved. If the profile says parsing is 3%, the right call is to revert and go look at the 90% box instead." Refusing to ship an optimization without a profile that justifies the target and a benchmark that survives scrutiny is the senior register.

(Answer to the §33.2 quick check: version A reported `0.0000 s` because the compiler deleted the loop — with `s` never used, the loop had no observable effect, and by the Chapter 10 rule the optimizer may remove computations whose results are never observed. Making `s` observable — printing it as a checksum, or writing it to a `volatile`/opaque sink — turns the loop back on. Two further rules: benchmark at the optimization level you will ship (`-O2`, not `-O0`), and ensure the input is not a compile-time constant the optimizer can fold away.)

33.5 What to carry forward

Performance engineering starts with a refusal: do not optimize what you have not measured. The governing rule is **measure before you optimize**, and Amdahl's law (Chapter 26) makes it quantitative — a part that is 5% of runtime caps its own whole-program payoff at 5%, so the *dominant* cost is the only target worth your effort, and only a **profile** tells you which part that is. Profiling comes in two forms — **instrumenting** (exact counts, perturbs timing, e.g. `gprof`) and **sampling** (statistical, low overhead, e.g. `perf`, drawn as flame graphs) — and its job is to replace a guess with the widest box. **Benchmarking** then compares candidate fixes, but only honestly: make the timed result observable so the optimizer cannot delete it (the `0.0000 s` vs. `1.5592 s` of §33.2), time at the shipping optimization level, use a monotonic clock, and run many trials reporting the minimum and the spread (the 0.6694–0.8198 s of §33.3), not one number. Put together, these form the loop: profile, benchmark, optimize, re-measure, and stop when good enough.

What the profile does *not* tell you is *why* the widest box is wide — and that is the rest of this part. When `compute` is 93.6% of runtime, the question becomes where *its* time goes, and the answer is usually not the instruction count you can read in the source but the costs the earlier volumes named: cache misses (Chapter 3), branch mispredictions, memory bandwidth, allocation, the layout of your data. The next chapters take the dominant cost the profiler hands you and open it up — **cache-aware and data-oriented design**, then wider techniques like **SIMD** — always returning to this chapter's loop to prove that a change actually moved the number. Safety was the compiler's job in the last part; speed is yours, and it begins not with a clever rewrite but with a measurement honest enough to bet on.

CAN YOU… WITHOUT LOOKING?

- State “measure before you optimize” and use Amdahl's law to say why the dominant cost is the only target.
- Explain why a loop whose result is unused can report `0.0000 s`, and the Chapter 10 rule behind it.
- Give the fix (observable result) and two more rules for an honest benchmark.
- Say which clock to use and why one run is untrustworthy; explain why you report the minimum.
- Distinguish sampling from instrumenting profilers, with a cost of each.
- State the measure–optimize loop end to end.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What are the two halves of “measure before you optimize,” and how does Amdahl's law make the second quantitative? (§33.1)
2. **R2.** Why can a microbenchmark report `0.0000 s`, and what makes the work run again? (§33.2)
3. **R3.** Why is a single timing untrustworthy, and why report the minimum of many trials rather than the mean? (§33.3)
4. **R4.** What is the difference between sampling and instrumenting profilers, and what does each cost? (§33.4)
5. **R5.** Given a profile showing one function at 90%+ of runtime, what does Amdahl say to do, and why? (§33.1, §33.4)

FURTHER READING

- **Brendan Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. (Addison-Wesley, 2020), ISBN 978-0-13-682015-4.** The standard modern reference for performance methodology, sampling profilers (`perf`, BPF-based tools), and flame graphs of §33.4; the source for the measure-first methodology of this chapter. Author, edition, and year verified via the publisher.
- **Donald E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys* 6(4), 261–301 (1974), DOI 10.1145/356635.356640.** The origin (p. 268) of "premature optimization is the root of all evil" quoted in §33.1 — and the fuller argument that measurement should find the critical few percent of code worth tuning. First author Donald Knuth; DOI verified via the ACM Digital Library.
- **Susan L. Graham, Peter B. Kessler & Marshall K. McKusick, "gprof: a Call Graph Execution Profiler," *ACM SIGPLAN Notices* 17(6), 120–126 (1982), DOI 10.1145/872726.806987.** The classic instrumenting profiler of §33.4 — call-graph attribution of runtime, and the source of its counts-vs-perturbation trade-off. First author Susan Graham; DOI verified via the ACM Digital Library.
- **The Linux `perf` tool (`perf_events`) documentation (`perfwiki.github.io`, and `man perf`).** Reference for the sampling profiler named in §33.4 — how it counts hardware events and samples the program counter with low overhead. Not installed in this environment, so discussed from its documentation, not run; verify against your distribution's version.

Exercises

33.1 WARM-UP State the rule "measure before you optimize" in one sentence, and say what a profiler is for versus what a benchmark is for.

33.2 WARM-UP Why can a microbenchmark of a loop report `0.0000 s` even though the loop appears to do real work?

33.3 WARM-UP Why do you run a benchmark many times and report the minimum (or median) rather than a single run's time?

33.4 CORE Take the two-phase profile of §33.4 (`parse` 6.4%, `compute` 93.6%). (a) Using Amdahl's law, what is the most you can improve total runtime by perfectly optimizing `parse`? (b) What whole-program improvement does a 30% cut to `compute` give? (c) A colleague, without profiling, spends a day optimizing `parse` because its code "looks slow." Explain precisely why this violates §33.1.

33.5 CORE Explain the dead-code-elimination benchmark bug of §33.2. (a) Under what condition is the compiler allowed to delete the timed loop, and which Chapter 10 principle is that? (b) Name two distinct ways to make the result observable so the work runs. (c) Why must you also benchmark at the optimization level you ship rather than at `-O0`?

33.6 CORE Contrast sampling and instrumenting profilers. (a) How does each obtain its data? (b) What is the cost or distortion of each, especially for many tiny, hot functions? (c) For a first look at a large program you do not want to recompile, which would you reach for, and why?

33.7 COST Reason about measurement error. (a) The five trials of §33.3 spanned 0.6694–0.8198 s for identical work; list three sources of that run-to-run variation. (b) Why is the *minimum* a more principled estimate of intrinsic cost than the mean, given that interference only ever slows a run? (c) When would the median be a better summary than the minimum, and what should you always report alongside either?

33.8 FADED *Worked, with the last step for you.* A developer benchmarks a hashing routine: `t0 = now(); for (i=0;i<N;i++) hash(key); t1 = now();` and reports `0.0000 s`, concluding the hash is "essentially free."
Step 1 (given): the return value of `hash(key)` is never used, so the calls have no observable effect.
Step 2 (given): at `-O2` the compiler may therefore remove the entire loop (Chapter 10), so nothing was timed.
Step 3 (given): the input `key` is also loop-invariant, which the optimizer can hoist or fold, compounding the problem.
Step 4 (you): Rewrite the benchmark so it honestly times the hash — make the result observable and vary the input across iterations — and state what you would additionally do (clock, trials) before trusting the number.

33.9 MIXED For each situation, say whether you would *profile*, *benchmark*, or *neither yet*, and why: (a) a service is "slow" but you have no idea which part; (b) you have two candidate implementations of the confirmed hot function and must pick one; (c) a teammate proposes rewriting a module that is 2% of runtime; (d) you changed the hot function and want to know if total runtime actually dropped.

33.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "A benchmark reporting `0.0000 s` means the code is extremely fast." (b) "Benchmarking at `-O0` tells you the shipped build's performance." (c) "A sampling profiler requires recompiling the program." (d) "Optimizing a part that is 4% of runtime can, at best, speed the program by 4%." (e) "One timing run is enough if the machine is otherwise idle."

33.11 STRETCH Tie this chapter to the cost model. (a) The profiler in §33.4 says `compute` is 93.6% of runtime but not *why*; list three costs from Chapters 3–5 that could be the real reason a loop is slow even at a low instruction count. (b) Explain how you would use the §33.3 timing method to test whether a proposed cache-friendlier layout actually helps — what exactly must you hold constant, and what must you make observable? (c) Argue why "profile → hypothesize a cause → change one thing → re-measure" is the scientific method applied to performance.

33.12 LAB On your own machine (a C toolchain suffices): (a) Reproduce §33.2 — write a loop that sums into a variable, compile at `-O2` without using the variable and time it, then print the variable and time it again; report both times and explain the difference. (b) Reproduce §33.3 — run an honest version five times with `clock_gettime(CLOCK_MONOTONIC, ...)`, report the five times and the minimum, and compute the spread. (c) If your platform has `perf` (or `gprof`), profile a small two-function program like §33.4's and confirm the profiler agrees with hand timing about which function dominates; if not, do the hand timing and state which tool you would use and what it would add. Note your toolchain versions and CPU.

Chapter 34 — Cache-Aware Design: Programming for Locality

Before you start: Chapter 3 (the memory hierarchy and caches — this chapter turns that model into code you can time), Chapter 5 (mechanical sympathy — writing with the grain of the machine), Chapter 4 (virtual memory and the TLB — the other table a strided access thrashes), Chapter 33 (measure first — every number here is measured, and you should distrust the ones that are not). · The profiler of Chapter 33 tells you *where* the time goes; this chapter opens the most common answer — the loop is waiting on memory — and shows the code changes that make it wait less.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 3 — roughly how many times slower is a main-memory access than an L1 cache hit, order of magnitude? (2) Chapter 3 — what is a *cache line*, and what is the unit the hardware moves between memory and cache? (3) *Predict:* you have an 8192×8192 matrix of `double` stored row by row, and you sum every element two ways — walking along each row, then walking down each column. Same arithmetic, same element count. How many times slower do you expect the column walk to be: about the same, 2×, or something far larger? Write your guess; §34.2 and §34.3 answer all three.

Chapter 3 gave you the memory hierarchy as a set of facts: registers, then L1, L2, L3 caches, then DRAM, each larger and an order of magnitude slower than the last. This chapter makes those facts *actionable*. The RAM model of Chapter 1 says every memory access costs the same; the real machine says an access that hits L1 is perhaps a hundred times cheaper than one that goes to DRAM, and which one you get is decided by **locality** — whether your accesses reuse recent data (temporal) and touch neighbors of recent data (spatial). The striking consequence is that two loops doing the *identical* arithmetic on the *identical* data can differ in wall-clock time by more than an order of magnitude, decided entirely by the order in which they touch memory. We build this up in five steps: the hierarchy as a cost model, with this machine's real cache sizes (§34.1); spatial locality and why the *cache line*, not the byte, is the unit you pay for (§34.2); the canonical demonstration — row-major versus column-major traversal, measured at 19× here (§34.3); temporal locality, working sets, and blocking to keep reuse in cache (§34.4); and what to carry into the layout and vectorization chapters that follow (§34.5).

On the code in this chapter. As in Chapter 33 the programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H, timed with `clock_gettime(CLOCK_MONOTONIC)` and reported as the minimum of several trials (Chapter 33's method). The cache sizes quoted are read from this machine's `/sys/devices/system/cpu/.../cache`, so they are real for this CPU; the *times* are illustrative of the effect and hardware-specific — your machine will differ. A hardware-counter profiler (`perf`) is *not* installed here, so cache-miss counts are not quoted; the effects below are shown through wall-clock timing, which is enough to see an order-of-magnitude gap, and this is stated wherever it matters.

34.1 The hierarchy as a cost model, in real numbers

The reason locality governs performance is that the levels of the hierarchy differ not by a little but by orders of magnitude, and the fast levels are small. On the machine these programs run on — an Intel Core i7-12700H, values read from `/sys` — each core has a 48 KiB L1 data cache and a 1.25 MiB L2, and all cores share a 24 MiB L3; the line size at every level is 64 bytes. Representative published access latencies (order-of-magnitude teaching figures, not measured here; see Drepper 2007 and CS:APP) run about 4 cycles to hit L1, ~12 to L2, ~40 to L3, and *hundreds* of cycles — often quoted near 100 ns — to reach DRAM. Two facts follow that decide everything else. First, the gap is enormous: a DRAM access can cost more than a hundred L1 hits, so a loop that misses cache on most accesses is not a little slower, it is *stalled*, the CPU idling while data crawls in. Second, the fast memory is *small* — 48 KiB of L1, not megabytes — so you cannot keep everything hot; you must arrange for the data you are about to use to already be near the core. That arrangement is what “cache-aware” means, and the hardware gives you two levers, spatial and temporal locality, which the next sections make concrete.

> **QUICK CHECK**
>
> Name the levels of the hierarchy from fastest to slowest and say, to an order of magnitude, how much slower a DRAM access is than an L1 hit. Why does the smallness of L1 (tens of KiB) mean you cannot simply “keep the data in cache” and must instead control locality? (Answer after the next section.)

34.2 Spatial locality: you pay by the cache line, not the byte

The hardware never fetches a single byte or a single `int` from memory. It fetches a whole **cache line** — 64 bytes on this machine — and caches that. So when you read one `int`, the fifteen `int`s next to it in the same line come along for free; if your next access is one of them, it hits cache. This is **spatial locality**: code that touches memory in contiguous, small strides gets many accesses per line fetched, while code that jumps far between accesses pays a full line fetch to use just one element and throws the other 60 bytes away. You can see the line size directly by sweeping the stride of a walk over an array far larger than L3 and measuring time per access — compiled and run here (gcc 11.4.0, `-O2`, a 256 MiB `int` array, a fixed 2^{24} accesses at each stride):

```
for (long k=0, idx=0; k < touches; k++) { s += a[idx]; idx += stride; if (idx >= N) idx =
0; }

$ gcc -O2 stride.c -o stride && ./stride
stride  1 ints (  4 B):  2.93 ns/access
stride  2 ints (  8 B):  2.71 ns/access
stride  4 ints ( 16 B):  3.53 ns/access
stride  8 ints ( 32 B):  5.82 ns/access
stride 16 ints ( 64 B): 10.61 ns/access   <- one access per 64-byte line
stride 32 ints (128 B): 18.03 ns/access
stride 64 ints (256 B): 20.91 ns/access
```

Read the shape, not the exact figures. At stride 1 each 64-byte line serves 16 accesses, so the line-fetch cost is amortized 16 ways and each access is cheap (~2.9 ns here, with the hardware prefetcher helping). As the stride grows the number of useful accesses per line falls — 8, then 4, then 2 — and the per-access cost climbs, because the same line fetch now serves fewer of them. At a stride of 16 `int`s — exactly 64 bytes, one cache line — *every* access lands in a fresh line and uses only 4 of its 64 bytes; the cost has jumped to ~10.6 ns, roughly 3.6× the stride-1 cost. The knee at 64 bytes is the cache line making itself visible in wall-clock time. Beyond it the cost keeps rising as larger strides also defeat the prefetcher and stress the TLB (Chapter 4), but the lesson is already fixed: **the cache line is the unit you pay for**, so packing the data a loop uses into as few lines as possible — contiguous, small-stride, no wasted bytes per line — is the first and cheapest performance lever you have.

> **QUICK CHECK**
>
> Why does a stride-1 walk cost far less per access than a stride-16-`int` walk over the same array, when both read the same number of elements? What is special about a stride of 64 bytes on this machine? (Answer below the communicate box.)

34.3 The canonical case: row-major versus column-major

Now the demonstration that every performance course reaches for, because it isolates spatial locality with nothing else changed. C stores a two-dimensional array in **row-major** order: `A[i][j]` and `A[i][j+1]` are adjacent in memory, while `A[i][j]` and `A[i+1][j]` are a full row apart. Summing the matrix by walking along rows therefore strides by one element (stay in the line, spatial locality is perfect); walking down columns strides by a whole row — 8192 `double`s, 64 KiB — so every single access lands in a new line, uses 8 of its 64 bytes, and evicts a line you will need again later. Same elements, same additions, opposite locality. Compiled and run here (gcc 11.4.0, `-O2`, an 8192×8192 `double` matrix, 512 MiB, minimum of 5 trials):

```
for (i) for (j) s += A[i*N + j];   // row-major: stride 1 element
for (j) for (i) s += A[i*N + j];   // col-major: stride N elements (one row apart)

$ gcc -O2 rowcol.c -o rowcol && ./rowcol
row-major (stride 1): 0.1269 s
col-major (stride N): 2.4744 s
ratio col/row       : 19.50x
```

The column walk is **19.5× slower** for arithmetic that is byte-for-byte identical. Nothing about the computation changed — the same 67 million additions run in both — only the order of memory access, and that order alone cost a factor of nineteen. This is the whole chapter in one measurement: at the RAM-model level (Chapter 1) the two loops are the same program, and a beginner reasoning from instruction counts predicts they run in the same time; the real machine charges an order of magnitude for the difference in locality, exactly the "where does the time go" question Chapter 33 said the instruction count cannot answer. The fix is almost always to reorder the loops or transpose the data so the innermost loop strides by one — and note that the compiler generally may *not* do this for you, because swapping loop nesting can change results under floating-point rounding and is not always legal, so the choice of traversal order is yours to make and to measure.

> **THE SAME IDEA THREE WAYS: LOCALITY DECIDES THE COST OF IDENTICAL WORK**
>
> **1. In words.** Memory is a hierarchy whose levels differ by orders of magnitude, and the hardware moves whole cache lines. Touch memory contiguously (spatial locality) and reuse it before it is evicted (temporal locality) and most accesses hit a fast level; stride far or revisit after eviction and most accesses stall on DRAM. The arithmetic is the same either way; the access order sets the price.
>
> **2. As a picture.** Figure 34.1: the hierarchy as a pyramid (registers → L1 48 KiB → L2 1.25 MiB → L3 24 MiB → DRAM), each level wider and slower; beside it, a 64-byte cache line with a row-major walk using all of it and a column-major walk using one element and discarding the rest.
>
> **3. As numbers/contrast.** Same array, cost per access rose from ~2.9 ns at stride 1 to ~10.6 ns at a 64-byte stride as line utilization fell (§34.2); the same 8192×8192 sum ran in 0.1269 s row-major and 2.4744 s column-major — **19.5×** — for byte-identical arithmetic (§34.3). Identical work, an order-of-magnitude spread, decided only by access order.

(Answer to the §34.1 quick check: fastest to slowest — registers, L1, L2, L3, DRAM — and a DRAM access is roughly two orders of magnitude slower than an L1 hit (tens of nanoseconds versus a fraction of one). L1 is only tens of KiB, far too small to hold a working set of any size, so you cannot "just keep it in cache"; you must arrange your accesses so the small hot data you need next is already near the core — that arrangement is locality, spatial and temporal.)

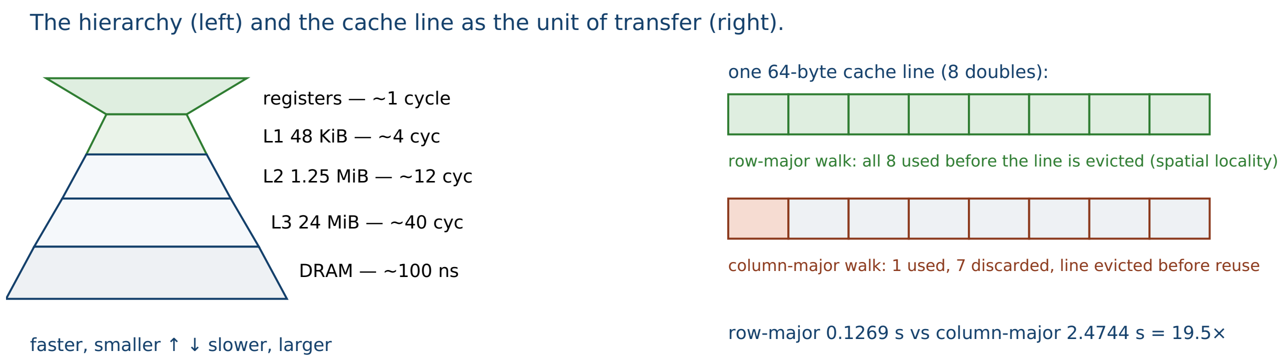


Figure 34.1: The memory hierarchy (left) charges orders of magnitude more for a DRAM access than an L1 hit, and the hardware moves whole 64-byte lines (right). A row-major walk uses every element of each line it fetches; a column-major walk uses one and discards the rest, evicting lines before their reuse — measured here at a 19.5× gap for identical arithmetic (§34.3).

34.4 Temporal locality, working sets, and blocking

Spatial locality is about the *neighbors* of what you touch; **temporal locality** is about revisiting the *same* data soon enough that it is still cached. The governing quantity is the **working set** — the amount of data a phase of the program touches repeatedly. If the working set fits in a cache level, the second and later passes over it hit that level; if it overflows, each pass evicts what the next pass needs, and you pay DRAM latency on every access even though you are reusing data. The classic example is dense matrix multiplication: the naive triple loop streams an entire N×N matrix for each output element, so once N is large enough that a row plus a column no longer fit in cache, nearly every access misses. The standard cure is **blocking** (tiling): restructure the loops to work on small sub-blocks that *do* fit in cache, compute everything that block contributes before moving on, so each loaded line is reused many times before eviction (CS:APP, ch.6; Hennessy & Patterson, ch.2). The block size is chosen so the active tiles fit a target cache level, turning a memory-bound computation into a compute-bound one. The general principle generalizes far past matrices: **size your hot working set to a cache level and reuse each loaded line as much as possible before it is evicted**. When a profile (Chapter 33) shows a loop stalled on memory, the two questions are always the same — am I striding contiguously (spatial), and does my reused data fit in cache (temporal) — and the fixes are reorder the accesses, or shrink the working set until it does.

TRAP: THE INNOCENT TRANSPOSE — STRIDING ACROSS CACHE LINES BY ACCIDENT

The column-major sum of §34.3 was 19× slower than the row-major one, and nobody wrote a “slow” loop — they wrote `for (j) for (i) A[i][j]`, which looks as reasonable as the other order and is a one-token difference. This is the trap: **an access pattern that strides across cache lines instead of along them is invisible in the source**. It hides in iterating a row-major array in column order; in an array of large structs where you touch one field per element (Chapter 35); in chasing pointers through a linked list or tree whose nodes were allocated all over the heap; in a hash table walked in key order. Each looks like ordinary code and each turns a fast level into DRAM latency on nearly every access. Two defenses. First, *know your data's memory layout* and make the innermost, most frequent loop stride by one along it — transpose the data, or swap the loops, so the hot access is contiguous. Second, *measure*: the gap does not show up in a code review or an instruction count, only in a timed run (Chapter 33) or a cache-miss counter (`perf`, where available). The order-of-magnitude cost of getting this wrong is precisely why it is worth a deliberate look at any hot loop's access pattern.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: “Our image filter is too slow. The inner loop is just `out[x][y] = f(in[x][y])` over a big 2-D array — it's the simplest possible code, so I don't think we can do better without rewriting it in assembly.” Cover the paragraph and respond in four or five sentences, drawing on §34.1–§34.4.

Model answer. “Before assembly, let's check the access pattern, because ‘simplest code’ and ‘fastest code’ aren't the same thing here. If the arrays are stored row-major and the loops are nested `for x { for y }` with `x` the row index, then `[x][y]` is striding by a whole row on the inner loop — every access lands in a new cache line and uses a fraction of it, which on the matrix case cost us about 19×. Swapping the loop nesting so the inner index runs along the contiguous dimension, or transposing one array, could give most of that back for a one-line change, no assembly. Let's measure it the Chapter 33 way — same input, `-O2`, many trials, minimum — with the loops each way and compare; if the profile then still shows us memory-bound, we look at blocking the working set to fit L2, and only after that consider vectorizing the kernel.” Reaching for the access pattern and a measurement before reaching for assembly is the senior register: the cheap, high-leverage fix first.

(Answer to the §34.2 quick check: a stride-1 walk reuses each fetched 64-byte line for all 16 `int`s in it, so the line-fetch cost is amortized across many accesses and each is cheap; a stride-16-`int` walk strides exactly one 64-byte line per access, so every access triggers a fresh line fetch and uses only 4 of its 64 bytes, and the cost climbs to the per-line fetch cost. A 64-byte stride is special because it equals the cache line: it is the smallest stride at which no two consecutive accesses ever share a line, so spatial locality is completely defeated.)

34.5 What to carry forward

The memory hierarchy is not a detail under the RAM model; on real hardware it is often *the* cost. Its levels differ by orders of magnitude and the fast ones are small (48 KiB of L1 on this machine, 24 MiB of shared L3), so performance is decided by **locality**: touch memory contiguously so each 64-byte line you fetch serves many accesses (**spatial locality**, §34.2), and reuse data while it is still cached by sizing your hot working set to a cache level (**temporal locality** and **blocking**, §34.4). The cost of ignoring this is not marginal: identical arithmetic ran **19.5×** slower in column-major than row-major order (§34.3), a factor that no instruction-count reasoning predicts and that a profile (Chapter 33) surfaces only as "stalled on memory." The cardinal rule of the chapter: the cache line, not the byte, is the unit you pay for, so pack what a loop uses into as few lines as possible and stride along them, not across them.

This chapter took the data layout as given — a matrix, an array — and controlled the *order* of access. The next chapter, Chapter 35, takes the complementary lever: controlling the *layout itself*, so that the bytes a loop needs are contiguous by construction. That is **data-oriented design** — array-of-structs versus struct-of-arrays, hot/cold splitting, packing — the operational form of the mechanical sympathy Chapter 5 introduced, and it turns the "stride along, not across" rule from something you arrange at the loop into something guaranteed by the data structure. After that, Chapter 36 shows how a cache-friendly, contiguous layout is also the precondition for the CPU's other lever of throughput, **SIMD**: the vector unit can only stream data it can reach quickly. Locality first, because a vector unit starved for memory is no faster than a scalar one.

CAN YOU… WITHOUT LOOKING?

- State the levels of the memory hierarchy and, to an order of magnitude, the L1-to-DRAM latency gap.
- Define spatial and temporal locality, and say why the cache line (not the byte) is the unit you pay for.
- Explain why row-major and column-major traversal of the same matrix can differ by an order of magnitude.
- Explain what a working set is and how blocking keeps reuse in cache.
- Name three ordinary-looking code patterns that secretly stride across cache lines.
- Describe how you would measure whether a proposed locality fix actually helped.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why do the orders-of-magnitude latency gaps between cache levels make locality, not instruction count, the thing to optimize? (§34.1)
2. **R2.** What is a cache line, and why does striding by exactly one line size defeat spatial locality? (§34.2)
3. **R3.** Why is column-major traversal of a row-major matrix an order of magnitude slower than row-major, for identical arithmetic? (§34.3)
4. **R4.** What is a working set, and how does blocking (tiling) turn a memory-bound loop into a compute-bound one? (§34.4)
5. **R5.** A profile says a loop is stalled on memory. What two locality questions do you ask, and what are the corresponding fixes? (§34.4)

FURTHER READING

- **Ulrich Drepper, "What Every Programmer Should Know About Memory" (Red Hat, Inc., 2007).** The definitive long-form treatment of caches, cache lines, prefetching, and the locality techniques of this chapter, published as a Red Hat whitepaper and as an LWN.net series (`lwn.net/Articles/250967`). Author Ulrich Drepper; a whitepaper, not a journal article, so no DOI — cited as such. The 19× effect of §34.3 is exactly what its cache chapters predict.
- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2015), ISBN 978-0-13-409266-9.** Chapter 6 (the memory hierarchy) is the source for the working-set and blocking discussion of §34.4 and for the representative latency figures of §34.1. First author Randal Bryant; edition, publisher, and ISBN verified via the publisher.
- **John L. Hennessy & David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017), ISBN 978-0-12-811905-1.** Chapter 2 develops the memory hierarchy quantitatively — cache organization, miss penalties, and the arithmetic behind the cost model of §34.1. First author John Hennessy; edition and ISBN verified via the publisher.
- **Linux kernel sysfs cache topology (`/sys/devices/system/cpu/cpu*/cache/`).** The source of the real cache sizes and 64-byte line size quoted for this machine in §34.1–§34.2 (`size`, `coherency_line_size`, `level`). Read directly from this environment; your machine's values will differ, so read your own before quoting them.

Exercises

34.1 WARM-UP Define spatial locality and temporal locality in one sentence each, and give a small code example that has good spatial locality and one that has poor spatial locality.

34.2 WARM-UP The cache line on this machine is 64 bytes. If you read one `double` (8 bytes) and then read the next 7 `double`s in memory, how many cache-line fetches does that cost in the best case, and why?

34.3 WARM-UP In one sentence, why is a DRAM access being ~100× slower than an L1 hit the reason we care about locality at all, rather than just counting instructions?

34.4 CORE The §34.3 measurement: row-major 0.1269 s, column-major 2.4744 s, on an 8192×8192 `double` matrix. (a) Both loops do the same number of additions — so what, precisely, differs between them to produce a 19.5× gap? (b) A `double` is 8 bytes and the line is 64 bytes; how many elements of each fetched line does the column-major walk actually use before the line is (much later) needed again? (c) Name two ways to fix the column-major loop.

34.5 CORE The stride sweep of §34.2 rose from ~2.9 ns/access at stride 1 to ~10.6 ns at a 64-byte stride. (a) Explain the climb in terms of accesses-served-per-line. (b) Why is there a recognizable knee at 64 bytes specifically? (c) Costs kept rising past 64 bytes; name two mechanisms other than the L1 line that explain the further increase.

34.6 CORE A colleague's hot loop chases a linked list: each node is `malloc`'d separately and holds one `int` plus a `next` pointer. Profiling shows it stalled on memory. (a) Why does this pattern have poor spatial locality even though each node is small? (b) What layout change would improve it, and why? (c) What does this share with the column-major case of §34.3?

34.7 COST Reason about working sets and blocking. A program makes repeated passes over an array of size S, and the last-level cache holds C bytes. (a) Qualitatively, what happens to the miss rate as S grows from below C to well above C? (b) Explain how blocking the computation into chunks of size ≤C changes the picture. (c) Why can blocking turn a “memory-bound” loop into a “compute-bound” one without reducing the number of arithmetic operations at all?

34.8 FADED *Worked, with the last step for you.* You must sum a 4096×4096 `float` matrix stored row-major.
Step 1 (given): row-major storage means `A[i][j]` and `A[i][j+1]` are adjacent; `A[i][j]` and `A[i+1][j]` are one row (16 KiB) apart.
Step 2 (given): the loop `for (j) for (i) sum += A[i][j]` strides by a full row on its inner loop, so nearly every access misses.
Step 3 (given): the loop `for (i) for (j) sum += A[i][j]` strides by one element on its inner loop, so each 64-byte line serves 16 accesses.
Step 4 (you): State which loop to ship and why, and describe exactly how you would measure that the reordered loop is faster on this matrix (clock, optimization level, trials, what to hold constant).

34.9 MIXED For each, say whether the problem is primarily *spatial* locality, *temporal* locality, or *neither*, and why: (a) iterating a row-major 2-D array in column order; (b) repeatedly scanning a 100 MiB table that does not fit in a 24 MiB L3; (c) a tight loop summing a 1 KiB array a million times; (d) chasing pointers through a tree whose nodes are scattered across the heap.

34.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “Reading one byte from memory fetches exactly one byte into cache.” (b) “Two loops with the same instruction count always take the same time.” (c) “The compiler will always reorder loops to fix a bad traversal order for you.” (d) “If the working set fits in L2, repeated passes over it mostly hit L2.” (e) “Blocking reduces the number of arithmetic operations.”

34.11 STRETCH Tie locality back to the profiler. (a) A profile (Chapter 33) shows one loop at 85% of runtime but a low instruction count per iteration; give the locality hypothesis and how you would test it. (b) Without `perf` or a cache-miss counter, describe a purely wall-clock experiment that distinguishes “memory-bound because of poor locality” from “genuinely compute-bound.” (c) Why is “transpose the data / swap the loops, then re-measure” a stronger move than “rewrite the loop in assembly” as a first response to a memory-bound loop?

34.12 LAB On your own machine (a C toolchain suffices): (a) Read your real cache sizes and line size from `/sys/devices/system/cpu/cpu0/cache/index*/` (or your platform's equivalent) and record them. (b) Reproduce §34.3 — sum a large row-major matrix both ways at `-O2`, time each as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials, and report the ratio; explain it using your line size. (c) Reproduce the stride sweep of §34.2 over an array larger than your L3 and find the knee; check that it falls at your line size. Note your toolchain version and CPU, and state that these are wall-clock measurements (no hardware counters) if you lack `perf`.

Chapter 35 — Data-Oriented Design: Laying Out Data for the Cache

Before you start: Chapter 5 (mechanical sympathy and data-oriented design — this chapter is that idea made operational and measured), Chapter 34 (locality and the cache line — here we control the layout, not just the access order), Chapter 26 (false sharing — its cause is cache-line layout, and we measure it here), Chapter 33 (measure first). · Chapter 34 took the data layout as given and reordered the accesses; this chapter changes the *layout itself*, so the bytes a loop needs are contiguous by construction.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 34 — what is the unit the hardware moves between memory and cache, and how big is it on typical hardware? (2) Chapter 26 — what is *false sharing*, in one line? (3) *Predict:* you have 16 million particles, each a struct of eight `doubles` (position, velocity, mass, charge), and a loop that sums just the `x` field. Compare that to storing all the `x`s in their own separate array and summing those. Same 16 million additions. How much faster do you expect the separate-array version to be: the same, a little, or several times? Write your guess; §35.2 answers it.

Object-oriented habit says: model a thing as a struct with all its attributes, make an array of those structs, and write loops over them. **Data-oriented design** inverts the priority. It asks first what *transformations* the hot loops perform on the data — which fields they read, in what order, how often — and then lays the data out so those loops touch memory contiguously, with no wasted bytes per cache line. The object is a convenience for the programmer; the layout is a contract with the cache, and Chapter 34 showed that contract can be worth an order of magnitude. Mike Acton put the discipline bluntly in his 2014 keynote: the purpose of the program is to transform data, so “if you don't understand the data, you don't understand the problem,” and the hardware you run on has one overriding cost — memory (Acton, 2014; Fabian, 2018). This chapter makes that concrete in four steps: the core idea and why the object is not the natural unit (§35.1); array-of-structs versus struct-of-arrays, measured at 4× here (§35.2); hot/cold splitting, field ordering, and packing to fit more useful data per line (§35.3); and the place where layout collides with concurrency — **false sharing**, measured at nearly 8× (§35.4). Then what to carry into the vectorization chapter, which needs exactly the layout this one produces (§35.5).

On the code in this chapter. As before the programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H, timed with `clock_gettime(CLOCK_MONOTONIC)` and reported as the minimum of several trials (Chapter 33). The times are illustrative of the effect and hardware-specific; a hardware-counter profiler (`perf`) is not installed here, so the effects are shown through wall-clock timing rather than cache-miss counts, and this is stated where it matters.

35.1 The object is not the natural unit

Start from the mismatch. A natural object-oriented model of a particle bundles everything about it — position, velocity, mass, charge — into one struct, and a simulation holds an array of them. But a physics update loop that only needs the `x` coordinate now strides through memory in jumps the size of the *whole struct*, dragging velocity, mass, and charge into cache on every step only to ignore them. The cache line is spent on bytes the loop never reads. Data-oriented design starts from the loop, not the object: it asks which fields the hot transformation actually touches, and arranges memory so *those* bytes are contiguous and the rest are elsewhere. The design question flips from "what is a particle?" to "what does the update loop read, and how do I give it that with no wasted line?" This is the same "stride along, not across" rule as Chapter 34, but achieved by *choosing the layout* rather than by reordering accesses over a fixed one — and because you own the layout, you can guarantee the good pattern instead of hoping the access order preserves it. The rest of the chapter is the concrete moves this principle licenses: splitting one array of fat structs into several arrays of fields (§35.2), separating hot fields from cold (§35.3), and being deliberate about which fields share a cache line when threads are involved (§35.4).

> **QUICK CHECK**
>
> Why does a loop that reads only the `x` field of an array of large structs waste most of every cache line it fetches? State the data-oriented reframing: which question replaces "what is a particle?" (Answer after the next section.)

35.2 Array-of-structs versus struct-of-arrays

The canonical data-oriented transform has a name. **Array-of-structs (AoS)** is the object-oriented default: `Particle a[N]`, each element holding all its fields together. **Struct-of-arrays (SoA)** stores each field in its own contiguous array: `double x[N], y[N], vx[N], ...`. For a loop that reads one field across all elements, SoA is pure spatial locality — every fetched line is full of the field you want — while AoS strides by the struct size and wastes the rest of each line. Here is the exact case from the opener: 16 million particles, an 8-field struct that happens to be 64 bytes (one whole cache line), summing only `x`. Compiled and run here (gcc 11.4.0, `-O2`, minimum of 7 trials):

```
struct Particle { double x,y,z,vx,vy,vz,mass,charge; };  // 64 bytes = one cache line
for (i) s += aos[i].x;    // AoS: stride 64 B, uses 8 of every 64 bytes fetched
for (i) s += soa_x[i];    // SoA: stride 8 B, every line full of x's

$ gcc -O2 aossoa.c -o aossoa && ./aossoa
AoS  sum of .x (64 B stride): 0.1514 s
SoA  sum of x  ( 8 B packed): 0.0371 s
ratio AoS/SoA               : 4.08x
```

SoA is **4.08× faster** for the same 16 million additions. The reason is the same accounting as Chapter 34's cache line: the AoS loop fetches a 64-byte line and uses 8 bytes of it — one `double` — before moving on, so 7 of every 8 bytes it pulls from memory are thrown away; the SoA loop uses all 64 bytes of every line, and the ratio of useful-to-fetched bytes (8× better) is most of the speedup. Notice this is not a made-up struct: 64 bytes is an entirely ordinary object size, and the “sum one field” loop is what half of data processing looks like. The tradeoff is real and worth stating plainly: SoA wins decisively when loops sweep *one or a few fields across many elements*; AoS can win when a loop touches *all* fields of *one* element at a time (then the whole struct in one line is exactly what you want). Data-oriented design is choosing per the dominant access pattern the profile shows — and when the hot loop reads a column, you store a column.

> **QUICK CHECK**
>
> Define AoS and SoA. For a loop summing one field across all elements, why is SoA faster, and roughly by what factor did the byte accounting predict here? When would AoS be the better layout instead? (Answer below the communicate box.)

35.3 Hot/cold splitting, field order, and packing

SoA is the extreme; between it and AoS lies a spectrum of cheaper wins from the same principle: *get more useful bytes into each fetched line*. Three moves recur. First, **hot/cold splitting**: if a struct has a few fields the hot loop reads every time (hot) and many it touches rarely (cold), split them into two structs — a dense array of hot fields the loop streams efficiently, and a parallel array of cold fields it visits only when needed. This keeps AoS convenience for the cold data while giving the hot loop SoA-like density. Second, **field ordering and padding**: the compiler inserts padding to satisfy each field's alignment, so a careless field order can bloat a struct with holes — ordering fields large-to-small typically minimizes that padding and shrinks the struct, so more elements fit per line (CS:APP, ch.3). A 24-byte struct that packs to 16 bytes with reordered fields is 50% more elements per line for free. Third, **shrinking the data itself**: using a 32-bit index instead of a 64-bit pointer, or a `float` instead of a `double` where precision allows, or a smaller enum, all raise the number of useful elements per cache line and so the effective bandwidth. None of these change the algorithm or the operation count; they change how many of the bytes you drag through the cache are ones you actually use. The unifying question, again, is the one Chapter 34 posed at the loop and this chapter poses at the type: **of every 64-byte line this code fetches, how many bytes does it use?** Every layout move here raises that fraction.

THE SAME IDEA THREE WAYS: RAISE THE USEFUL BYTES PER FETCHED LINE

1. In words. The hardware fetches whole cache lines; design the data so the bytes a hot loop reads are packed into as few lines as possible and the bytes it ignores are elsewhere. Store the column the loop sweeps (SoA), split hot fields from cold, order and shrink fields to cut padding. The operation count is unchanged; the wasted-bytes-per-line falls.

2. As a picture. Figure 35.1: AoS as an array of fat boxes with one thin stripe (the read field) per box, the rest shaded "fetched but unused"; SoA as one tight array of just those stripes, every line full. A cache line drawn over each shows AoS using 1 slot of 8, SoA using 8 of 8.

3. As numbers/contrast. Summing one field of 16M 64-byte structs: AoS 0.1514 s, SoA 0.0371 s — **4.08×**, tracking the 8× better useful-bytes-per-line (§35.2). And the same layout lever, mishandled across threads, produced a **7.9×** *slowdown* from false sharing (§35.4). Layout gives and layout takes away.

(Answer to the §35.1 quick check: a loop reading only `x` from an array of large structs fetches a full 64-byte line per element but uses only the few bytes of `x` in it, so most of every line — and most of the memory bandwidth it consumes — is wasted on fields it never reads. The data-oriented reframing replaces "what is a particle (and what are all its attributes)?" with "what does the hot loop read, and how do I lay memory out so it reads only that, contiguously?")

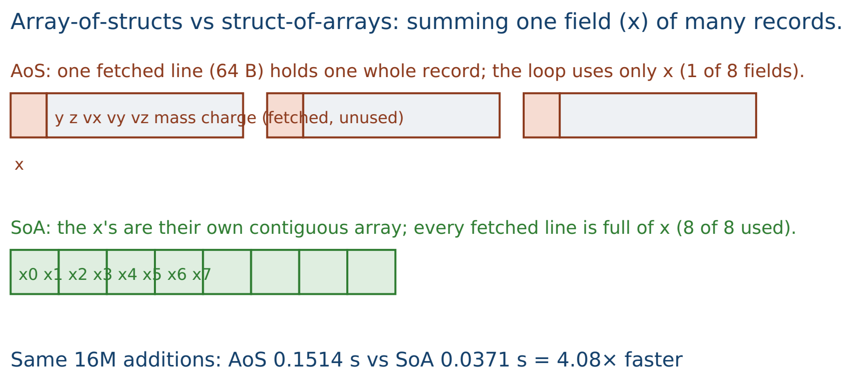


Figure 35.1: Summing one field across many records. In AoS each fetched 64-byte line holds one whole record and the loop uses a single field, discarding the rest; in SoA the field has its own contiguous array and every fetched line is fully used. Measured here at 4.08× for identical arithmetic (§35.2).

35.4 When layout meets concurrency: false sharing

Layout can also cost you a factor when threads are involved, and the mechanism is again the 64-byte line. Chapter 26 named **false sharing**: two threads write to two *different* variables that happen to sit in the *same* cache line, and the cache-coherence protocol, which tracks ownership at line granularity, ping-pongs that line between the cores' caches on every write — even though the threads never touch each other's data. The variables are logically independent; the hardware sees one shared line and serializes access to it. This is a pure layout bug: nothing is logically contended, yet the line is. The fix is layout too — **pad or align each thread's data to its own cache line** so no two threads' hot variables share one. Here are four threads each incrementing their own counter 200 million times, first with the counters packed into one line, then padded to a line each. Compiled and run here (gcc 11.4.0, `-O2`, four threads on this machine, minimum of 3 trials):

```
struct { volatile long v; } shared[4];                   // 4 counters in ONE 64-byte line
struct { volatile long v; char pad[56]; } padded[4];     // each counter on its OWN line

$ gcc -O2 falseshare.c -o falseshare -lpthread && ./falseshare
false-shared (one line) : 1.3925 s
padded (one line each)  : 0.1757 s
slowdown from sharing    : 7.92x
```

Packing the counters into one line made the program **7.9× slower**, and not one byte of data was actually shared between threads — each thread only ever touched its own counter. All 7.9× is the coherence protocol bouncing the single contested line between four cores. The 56 bytes of padding that fixed it look like waste, and in single-threaded terms they are; across threads they buy back a factor of eight, because they guarantee each thread's line is its own. This is the same lesson as SoA from the other direction: in §35.2 *packing* data together helped a single loop; here *separating* data onto distinct lines helped concurrent threads. Both are the same principle — control which bytes share a cache line — applied to opposite access patterns. The general rule for concurrent code: **per-thread hot data should live on its own cache line**, achieved by padding or by `alignas(64)`, and you confirm it the Chapter 33 way, by measuring the padded and unpadded versions.

TRAP: THE ARRAY OF FAT OBJECTS (AND ITS MIRROR, THE SHARED COUNTER LINE)

The default reflex — model a thing as a struct, make an array of them, loop over the array — is invisible as a performance bug because it is *good object-oriented style*. Yet a hot loop that reads one field of a 64-byte struct wastes 7 of every 8 bytes it fetches, which cost **4×** in §35.2, and no code review flags “array of structs” as slow because it is textbook design. The mirror-image trap is its concurrent form: put each thread's counter in an adjacent array slot — the obvious, tidy layout — and false sharing costs **8×** (§35.4), again with nothing in the source that looks wrong. Both are layout bugs hiding behind reasonable-looking code, and both are invisible to instruction counting; they show up only in a timed run (Chapter 33) or a cache/coherence counter. Two defenses. First, at any hot loop, ask the useful-bytes-per-line question of §35.3 — is this loop reading a column out of an array of rows? If so, store the column (SoA), or split hot from cold. Second, for per-thread hot data, pad or align to a cache line and measure both ways. The connecting insight is that *the layout is a performance decision the type system lets you make silently* — so make it deliberately, and measure it, rather than inheriting whatever the object model happened to produce.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our entity system has an `Entity` struct with 20 fields, and we store them in one big `Entity[]` array. The movement system, which runs every frame over all entities, only reads position and velocity, and it's our hottest loop. It's clean OOP — should we leave it?" Cover the paragraph and respond in four or five sentences, drawing on §35.1–§35.4.

Model answer. "The clean-OOP layout is probably costing us several times on that loop, and it won't show up in a review. The movement system reads maybe 4 of 20 fields, so every cache line it fetches is mostly the other 16 fields it never touches — in a comparable case, summing one field of a 64-byte struct, storing it as its own array was about 4× faster. I'd hot/cold split: put position and velocity in their own tightly packed arrays (struct-of-arrays for the hot fields) that the movement system streams, and leave the cold fields in a parallel array visited only when needed. Let's measure it the Chapter 33 way — same entity count, `-O2`, many trials, minimum — with the current layout and the split, and confirm the frame time actually dropped. And while we're at it, if any per-entity data is written by multiple worker threads, we should pad it to a cache line so we don't add false sharing on top." Choosing the layout from the hot loop's access pattern, and proving it with a measurement, is the senior register — not defending a layout because it is tidy.

(Answer to the §35.2 quick check: AoS is `Particle a[N]` with all fields of each element together; SoA is a separate contiguous array per field. For a loop summing one field across all elements, SoA is faster because every fetched 64-byte line is full of that field while AoS wastes the bytes of the other fields — here the useful-bytes-per-line was 8× better and the measured speedup 4.08×. AoS is better when a loop touches all fields of one element at a time, since then the whole struct in one line is exactly what it uses.)

35.5 What to carry forward

Data-oriented design is the operational form of the mechanical sympathy Chapter 5 named: decide the memory layout from what the hot loops actually read, not from what makes a tidy object. Because the hardware fetches whole 64-byte lines (Chapter 34), the governing metric is **useful bytes per fetched line**, and every move in this chapter raises it — **struct-of-arrays** to store the column a loop sweeps (4.08× here, §35.2), **hot/cold splitting** and **field ordering and packing** to cut wasted bytes and padding (§35.3), and, across threads, cache-line **padding to avoid false sharing** (7.9× here, §35.4). None of these change the algorithm or the operation count; they change how much of the memory traffic is useful. And all of them are invisible to instruction counting and to code review — an array of fat structs and a row of adjacent counters both look like good code — so they are found and fixed only by measurement (Chapter 33).

The layout this chapter produces is not only cache-friendly; it is the precondition for the next lever of throughput. A vector unit (Chapter 36, **SIMD**) processes 4, 8, or 16 elements per instruction, but only if those elements are contiguous in memory and identically laid out — which is exactly what struct-of-arrays gives it and what array-of-structs denies it. The auto-vectorizer that Chapter 36 leans on typically cannot vectorize a strided AoS field access but happily vectorizes a packed SoA array. So the two chapters compose: Chapter 34 got the access order right, this chapter got the layout right, and

Chapter 36 turns that contiguous, well-laid-out data into vector throughput. Lay the data out for the cache first, because the vector unit, like the cache, rewards contiguity and punishes strides.

CAN YOU… WITHOUT LOOKING?

- State the data-oriented reframing: which question about the data replaces “what is this object?”
- Define AoS and SoA, and say for which access pattern each wins.
- Explain the “useful bytes per fetched line” metric and how SoA, hot/cold splitting, and packing each raise it.
- Explain false sharing as a layout bug and give its fix.
- Say why an array of fat structs and a row of adjacent per-thread counters are both invisible performance bugs.
- Describe how you would measure whether a layout change actually helped.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What question does data-oriented design ask first, and why is the object not the natural unit of layout? (§35.1)
2. **R2.** What are AoS and SoA, why is SoA faster for a one-field sweep, and when does AoS win? (§35.2)
3. **R3.** Name three layout moves that raise useful-bytes-per-line without changing the algorithm. (§35.3)
4. **R4.** What is false sharing, why did it cost ~8× here, and what is the fix? (§35.4)
5. **R5.** Why are these layout issues invisible to code review and instruction counting, and what makes them visible? (§35.3, §35.4)

FURTHER READING

- **Richard Fabian, *Data-Oriented Design: Software Engineering for Limited Resources and Short Schedules* (2018), ISBN 978-1-916478-70-1.** The book-length treatment of the discipline of this chapter — designing around data transformations and cache-friendly layout, with SoA, hot/cold splitting, and existence-based processing. Author Richard Fabian; year and ISBN verified via the publisher listing. Also available online at `dataorienteddesign.com/dodbook`.
- **Mike Acton, "Data-Oriented Design and C++," keynote, CppCon 2014.** The influential talk (Acton was then engine director at Insomniac Games) that popularized the "understand the data, then design the code" framing of §35.1 and the AoS-vs-SoA argument of §35.2. A recorded conference talk, not a paper, so cited as such; slides in the CppCon2014 GitHub archive.
- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2015), ISBN 978-0-13-409266-9.** Chapter 3 covers struct layout, alignment, and padding — the basis for the field-ordering and packing of §35.3. First author Randal Bryant; edition and ISBN verified via the publisher.
- **Ulrich Drepper, "What Every Programmer Should Know About Memory" (Red Hat, Inc., 2007).** Its cache- coherency sections explain the line-granularity ownership that makes false sharing (§35.4) cost what it does, and the layout techniques that avoid it. A Red Hat whitepaper / LWN.net series (`lwn.net/Articles/250967`), so no DOI — cited as such.

Exercises

35.1 WARM-UP Define array-of-structs (AoS) and struct-of-arrays (SoA) in one sentence each, with a small code sketch of each for a particle with fields `x, y, vx, vy`.

35.2 WARM-UP State the "useful bytes per fetched line" metric in one sentence, and use it to explain why summing one field of a 64-byte AoS struct wastes most of each cache line.

35.3 WARM-UP What is false sharing, in one or two sentences, and what one-word layout change fixes it?

35.4 CORE The §35.2 measurement: AoS 0.1514 s, SoA 0.0371 s (4.08×) summing one field of 16M 64-byte structs. (a) Compute the useful-bytes-per-line ratio between the two layouts and explain why it roughly predicts the speedup. (b) Why is the measured 4.08× a little less than the 8× byte ratio? Give one plausible reason. (c) Describe a loop for which AoS would be the *faster* layout, and why.

35.5 CORE Hot/cold splitting. A `Player` struct has `x, y` (read every frame) and `name, inventory, achievements` (read rarely). (a) Sketch the hot/cold split. (b) Why does this help the per-frame loop even though you did not remove any fields? (c) What is the cost of the split — what becomes slower or more awkward, and when is it not worth it?

35.6 CORE Field ordering and packing. A struct is declared `{ char a; double b; char c; }`. (a) Explain why alignment makes this larger than the 10 bytes of its fields, and estimate its actual size on a typical 64-bit target. (b) Reorder the fields to minimize padding and give the new size. (c) In an array of a million of these, why does the smaller struct improve a loop that sweeps them?

35.7 COST Reason about the false-sharing measurement (§35.4): packed 1.3925 s, padded 0.1757 s, 7.92×. (a) Given no bytes are logically shared between threads, where does the entire 7.9× come from? (b) Why does the padding (56 wasted bytes per counter) buy back the factor, and what is it trading? (c) Would the gap grow or shrink if you ran *one* thread instead of four, and why?

35.8 FADED *Worked, with the last step for you.* A ray tracer stores scene objects as `Sphere{ double cx,cy,cz,r; Material m; char name[48]; }` in a `Sphere[]`, and its hottest loop is an intersection test that reads only `cx,cy,cz,r`.
Step 1 (given): the loop reads 32 bytes (four `double`s) of a struct that is well over 100 bytes, so it wastes most of every fetched line.
Step 2 (given): the profile (Chapter 33) confirms the intersection loop dominates runtime and is stalled on memory.
Step 3 (given): two candidate layouts — full SoA (separate `cx[],cy[],cz[],r[]` arrays) and hot/cold split (a packed `{cx,cy,cz,r}` array plus a parallel array of the cold fields).
Step 4 (you): Choose a layout, justify it from the access pattern, and describe exactly how you would measure that it beats the original (clock, optimization level, trials, what to hold constant).

35.9 MIXED For each, say whether the better layout is *AoS*, *SoA*, or *hot/cold split*, and why: (a) a loop that sweeps one field across millions of records; (b) a loop that processes one record fully (all its fields) then moves to the next; (c) a struct with 3 hot fields and 30 cold ones, swept every frame on the hot fields; (d) two threads each updating their own per-thread counter in a tight loop.

35.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "SoA is always faster than AoS." (b) "Reordering struct fields can shrink the struct without removing any field." (c) "False sharing means two threads are writing the same variable." (d) "Padding a struct to a cache line is pure waste with no benefit." (e) "A layout change that keeps the same operation count cannot change runtime."

35.11 STRETCH Tie layout to the whole part. (a) Chapter 34 reordered accesses over a fixed layout; this chapter changed the layout. Give one problem each is the right tool for, and one where you would use both together. (b) Explain why SoA is also the layout a vectorizer (Chapter 36) wants, referencing contiguity. (c) A colleague proposes converting the entire codebase from AoS to SoA up front, before profiling. Argue, from Chapters 33–35, why that is the wrong order.

35.12 LAB On your own machine (a C toolchain suffices; a threads library for part c): (a) Reproduce §35.2 — a struct of several `double`s in an array, versus a separate array of one field; sum that field both ways at `-O2`, time each as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials, and report the ratio, explaining it from your struct size and line size. (b) Vary the struct size (16, 32, 64, 128 bytes) and observe how the AoS/SoA ratio changes; explain the trend. (c) If you have threads, reproduce the false-sharing experiment — counters packed into one line versus padded to a line each — and report the slowdown. State that these are wall-clock measurements (no hardware counters) if you lack `perf`, and note your toolchain and CPU.

Chapter 36 — SIMD and Vectorization

Before you start: Chapter 2 (the CPU up close — the vector unit is another execution resource on the same core), Chapter 35 (struct-of-arrays — the contiguous layout the vectorizer needs), Chapter 34 (locality and memory bandwidth — the ceiling that decides whether SIMD helps), Chapter 10 (the optimizing compiler — the auto-vectorizer is one of its transforms), Chapter 33 (measure first). · Chapters 34 and 35 got the data close to the core and contiguous; this chapter turns that data into wide, parallel arithmetic — when the memory system can keep up.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 35 — why does a struct-of-arrays layout stream one field faster than an array-of-structs? (2) Chapter 34 — what does it mean for a loop to be “memory-bandwidth bound” rather than compute bound? (3) *Predict:* a modern vector register holds 8 `float`s (256-bit AVX), so one vector instruction does 8 multiplies at once. If you take a scalar arithmetic-heavy loop over an array that fits in L1 and let the compiler vectorize it, what speedup do you expect: about 8×, less, or possibly more? And would a loop that streams a 64 MiB array instead see the same? Write your guesses; §36.2 and §36.3 answer them.

A scalar instruction operates on one pair of operands: one add, one result. A **SIMD** instruction — Single Instruction, Multiple Data — operates on a whole vector of them at once: with 256-bit AVX registers, eight `float`s added to eight `float`s in one instruction, on the same core, in about the same time as the scalar add. This is **data-level parallelism** (Hennessy & Patterson, ch.4): not more cores, but wider instructions on one core, and it is free throughput *if* your data is laid out so the CPU can load a vector's worth at once and your loop has no dependency forcing the elements to be done one at a time. Both conditions point back: contiguity is Chapter 35's struct-of-arrays, and independence is a property of the loop. This chapter proceeds in five steps: what SIMD is and where the lanes come from (§36.1); letting the compiler vectorize for you, measured here from 3.4 to 45 Gflop/s (§36.2); what blocks the vectorizer — dependencies, reductions and floating-point associativity, aliasing (§36.3); intrinsics, and the memory-bandwidth ceiling that decides whether any of this helps (§36.4); and what to carry forward as the performance part closes (§36.5).

On the code in this chapter. As before the programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (which supports AVX2, i.e. 256-bit / 8-wide `float` vectors, not AVX-512), timed with `clock_gettime(CLOCK_MONOTONIC)` and reported as the minimum of several trials (Chapter 33). Throughputs are in Gflop/s (billions of floating-point operations per second) and are illustrative of the effect, not a benchmark of the machine. Where a compiler decision is claimed — that a loop did or did not vectorize — it is confirmed with gcc's own `-fopt-info-vec` report, quoted inline, not asserted. A hardware-counter profiler (`perf`) is not installed here, so the evidence is wall-clock throughput plus the compiler's vectorization report.

36.1 One instruction, many lanes

The vector unit is extra width on the ordinary core, not a separate processor. A modern x86 core has vector registers — 128-bit (SSE), 256-bit (AVX/AVX2), and on some parts 512-bit (AVX-512) — each holding several elements: a 256-bit register holds 8 `float`s or 4 `double`s, and a single AVX instruction like `vmulps` multiplies all 8 lanes at once. The elements travel through the pipeline (Chapter 2) together, so eight multiplies retire in roughly the time of one, giving up to an 8× throughput gain on 32-bit data for arithmetic that can be done in parallel across the lanes. Two requirements make that gain real, and both were set up by the previous chapters. First, **contiguity**: to fill a vector register in one load, the 8 elements must be adjacent in memory — precisely the struct-of-arrays layout of Chapter 35; a strided array-of-structs field forces the CPU to gather elements one at a time and throws the advantage away. Second, **independence**: the 8 lanes are computed simultaneously, so no lane may depend on another's result within the vector — a loop where each iteration needs the previous one's output cannot be vectorized as written. When both hold, the same loop that issued one scalar operation per element now issues one vector operation per 8 elements, and the arithmetic throughput multiplies. The next section shows how little you often have to do to get this, and how much it is worth.

> **QUICK CHECK**
>
> How many `floats` fit in a 256-bit AVX register, and how many does one AVX arithmetic instruction process at once? Name the two properties a loop's data and computation must have for the vector unit to deliver its throughput. (Answer after the next section.)

36.2 Auto-vectorization: let the compiler do it

You rarely write vector instructions by hand first. The compiler's **auto-vectorizer** (Chapter 10's optimizer, with the loop transforms of §36.3) recognizes a vectorizable loop and emits vector instructions for it — but only when you ask for the right optimization level and target. gcc does not vectorize at `-O2` in this version; `-O3` enables it against the conservative baseline instruction set (SSE2, 128-bit), and `-march=native` lets it use the widest vectors the actual CPU supports (AVX2, 256-bit here). Here is one compute-heavy, per-element-independent kernel — a Horner-form polynomial over an L1-resident 32 KiB array — compiled three ways and run here (gcc 11.4.0, minimum of 7 trials):

```
for (i) y[i] += ((((v*0.11f+0.22f)*v+0.33f)*v+0.44f)*v+0.55f)*v+0.66f;   // v = x[i], ~11
flops/elem

$ gcc -O2 -fno-tree-vectorize simd.c ...   scalar (no vectors) :  3.43 Gflop/s   (1.049 s)
$ gcc -O3 simd.c ...                       -O3 (SSE2, 4-wide)  : 24.89 Gflop/s   (0.145 s)
$ gcc -O3 -march=native simd.c ...         -O3 native (AVX2)   : 45.14 Gflop/s   (0.080 s)

$ gcc -O3 -march=native -fopt-info-vec-optimized simd.c
simd.c:10: optimized: loop vectorized using 32 byte vectors      # AVX2: 8 floats/vector
```

The measured jump is **13×** from scalar to AVX2 (3.43 to 45.14 Gflop/s), and the compiler's own report confirms it vectorized “using 32 byte vectors” — 256-bit AVX2, 8 `float`s per instruction. Two things deserve care in reading this. First, the speedup *exceeds* the 8-lane width, and honestly so: vectorizing also lets the compiler use fused multiply-add (one instruction for `a*b+c`), unroll the loop, and cut per-iteration overhead, so the win is lanes-times-other-improvements, not lanes alone — which is exactly why you *measure* the speedup rather than assume it equals the vector width. Second, the flags matter enormously: the identical source ran at 3.4, 25, or 45 Gflop/s depending only on `-O2` versus `-O3` versus `-O3 -march=native`, so “did it vectorize?” is a question about the build, and Chapter 33's rule to benchmark the build you ship is what keeps you honest. The practical lesson: reach for auto-vectorization first — right flags, contiguous data, an independent loop — and confirm it happened with `-fopt-info-vec` rather than hoping.

QUICK CHECK

Why did the same source run at 3.4, 25, and 45 Gflop/s? Why is the 13× scalar-to-AVX2 speedup *more* than the 8× you would get from lane count alone? How would you confirm a loop actually vectorized rather than assume it did? (Answer below the communicate box.)

36.3 What blocks the vectorizer

Auto-vectorization is not magic; a few common properties defeat it, and knowing them lets you either fix the loop or understand why the report says “not vectorized.” The first is a **loop-carried dependency**: if iteration `i` reads a value iteration `i-1` wrote (`a[i] = a[i-1] + x[i]`, a running prefix), the lanes cannot be computed in parallel because each needs the one before, and the loop is inherently serial as written. The second is subtle and important — a **reduction under floating-point non-associativity**. Summing an array into one accumulator (`s += a[i]`) is a reduction, and vectorizing it means adding the elements in a different grouping (8 partial sums, combined at the end), which under floating-point rounding can give a *different result* than the strict left-to-right order the source specifies. Because the standard forbids the compiler from silently changing your numerical result, gcc will *not* vectorize a floating-point reduction at `-O3` unless you permit reassociation with `-ffast-math` (or `-funsafe-math-optimizations`). You can see this in a dot product streamed over a 64 MiB array pair here (gcc 11.4.0):

```
float dot = 0; for (i) dot += a[i]*b[i];      // a reduction: reassociating changes FP
rounding

$ gcc -O2 -fno-tree-vectorize dot.c ...   scalar              : 0.96 Gflop/s
$ gcc -O3 -march=native dot.c ...         -O3 native, no fast : 1.12 Gflop/s   # reduction
NOT vectorized
$ gcc -O3 -march=native -ffast-math ...   with -ffast-math    : 2.30 Gflop/s   # now
vectorized
```

Without `-ffast-math` the reduction stays essentially scalar (1.12 vs 0.96 Gflop/s — the small gain is other optimization, not vectorization); allowing reassociation unlocks the vector reduction and roughly doubles throughput. Two lessons follow. One, **a floating-point reduction needs your explicit permission to vectorize**, because the vector version is not bit-identical — so decide deliberately whether your problem tolerates the rounding difference rather than sprinkling `-ffast-math` blindly (it also enables other unsafe assumptions). Two, the dot-product speedup here is only ~2.4×, far short of the 13× of §36.2 on the same machine, and the reason is the whole point of the next section: this loop is limited by memory bandwidth, not arithmetic. Other blockers worth naming: **pointer aliasing** (the compiler must assume two pointers might overlap and serialize to be safe — `restrict`, used in the §36.2 kernel, promises they do not), data-dependent branches inside the loop, and non-contiguous (strided or gathered) access — the array-of-structs pattern of Chapter 35, which is why layout came first.

> **THE SAME IDEA THREE WAYS: SIMD MULTIPLIES ARITHMETIC, NOT MEMORY**
>
> **1. In words.** A vector instruction does 8 (or 4, or 16) operations at once on contiguous, independent data, so an arithmetic-bound loop can speed up by roughly the lane count (often more, with FMA and unrolling). But if the loop is limited by how fast memory can feed it, widening the arithmetic does little — you were waiting on bytes, not on the adder.
>
> **2. As a picture.** Figure 36.1: on the left, a scalar lane doing one multiply per step; on the right, 8 lanes doing 8 at once from one contiguous load. Below, two bars: a compute-bound kernel rising from 3.4 to 45 Gflop/s (13×), and a bandwidth-bound dot product barely moving (0.96 to 2.3), capped by a “memory bandwidth” line.
>
> **3. As numbers/contrast.** Same machine, same compiler: an L1-resident compute kernel went 3.43 → 24.89 → 45.14 Gflop/s across scalar, SSE2, and AVX2 — **13×** (§36.2); a 64 MiB-streaming dot product went 0.96 → 2.30 Gflop/s even vectorized — **2.4×**, and only after `-ffast-math` let the reduction vectorize at all (§36.3). Vector width is a throughput ceiling on arithmetic; memory bandwidth is a different ceiling entirely.

(Answer to the §36.1 quick check: a 256-bit AVX register holds 8 `floats` (or 4 `doubles`), and one AVX arithmetic instruction processes all 8 lanes at once. The two required properties are contiguity — the 8 elements must be adjacent so one load fills the register, i.e. Chapter 35's struct-of-arrays — and independence — no lane may depend on another's result within the vector, so a loop-carried dependency defeats it.)

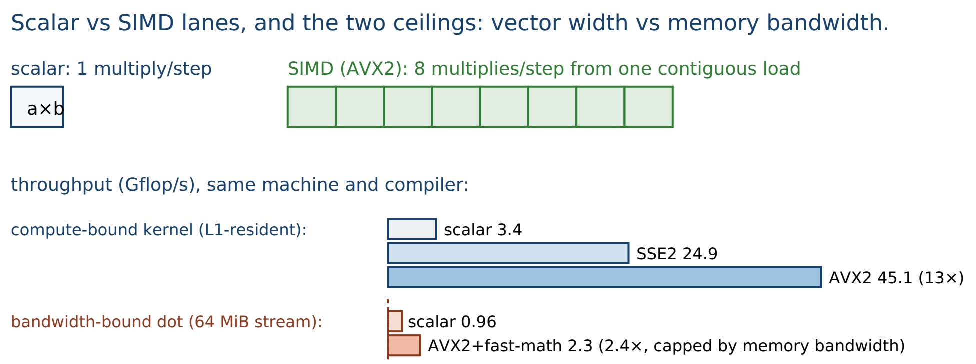


Figure 36.1: SIMD does many operations per instruction on contiguous, independent lanes. On an L1-resident, arithmetic-heavy kernel this multiplied throughput 13× (scalar → AVX2); on a 64 MiB-streaming dot product it gained only ~2.4×, because that loop is limited by memory bandwidth, not by the width of the adder (§36.2–§36.4).

36.4 Intrinsics, and the memory-bandwidth ceiling

When the auto-vectorizer cannot see the parallelism — an awkward data layout, a shuffle it will not synthesize, an algorithm that needs hand-mapping to lanes — you can write the vector instructions yourself with **intrinsics**: C functions like `_mm256_mul_ps` that map one-to-one to AVX instructions, giving you the lanes explicitly at the cost of portability and readability (Intel Intrinsics Guide; Agner Fog's optimization manuals). Intrinsics are the right tool when the payoff is proven and the compiler will not deliver it, but they are a last resort after auto-vectorization, because they are verbose, CPU-specific, and easy to get wrong — and, crucially, they cannot beat the ceiling the previous section exposed. That ceiling is **memory bandwidth**. The dot product of §36.3 gained only 2.4× from vectorization not because the vector unit was idle but because the loop streams 64 MiB pair of arrays that do not fit in any cache, so it is limited by how fast DRAM can deliver bytes, not by how fast the core can multiply them. This is the **roofline** idea (Williams, Waterman & Patterson, 2009): a loop's achievable performance is capped by the lower of two ceilings — peak compute (raised by SIMD) and peak memory bandwidth (raised only by better locality) — and which one binds depends on the loop's *arithmetic intensity*, its ratio of flops to bytes moved. A dot product does ~2 flops per 8 bytes read: low intensity, bandwidth-bound, so SIMD barely helps and the real lever is Chapters 34–35 (locality and layout, to move fewer bytes). The L1 kernel of §36.2 does ~11 flops per element already in cache: high intensity, compute-bound, so SIMD delivered 13×. The discipline: **vectorize the compute-bound loops; for the bandwidth-bound ones, fix locality first** — and let a measurement, not the lane count, tell you which kind you have.

> **TRAP: COUNTING LANES INSTEAD OF MEASURING — THE SIMD THAT MEMORY BANDWIDTH EATS**
>
> The seductive arithmetic is “8 lanes, so 8× faster,” and it fails in both directions. It *over*-predicts on bandwidth-bound loops: the dot product of §36.3 has 8 lanes available and gained only 2.4×, because it waits on DRAM, not on the adder — hand-writing AVX intrinsics for it would add complexity and CPU-specific code for a win the memory system has already capped. And it *under*-predicts on compute-bound loops: the §36.2 kernel gained 13×, *more* than 8×, because vectorization also unlocked FMA and unrolling. Both errors come from reasoning about the hardware instead of measuring it. Two defenses. First, decide whether the loop is compute- or bandwidth-bound before reaching for SIMD — a rough arithmetic-intensity estimate (flops per byte moved) or, decisively, the Chapter 33 experiment of timing scalar versus vectorized; if widening the arithmetic does not move the number, you are bandwidth-bound and the fix is locality and layout (Chapters 34–35), not more lanes. Second, if you do vectorize, confirm it happened (`-fopt-info-vec`) and measure the actual speedup rather than assuming the lane count — because the true number, high or low, is the only one you can ship a decision on. A hand-written SIMD kernel that ignores the bandwidth ceiling is effort spent past the point where it can pay, the vector-unit version of optimizing the 5% part.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I hand-wrote AVX2 intrinsics for our vector-add loop `c[i] = a[i] + b[i]` over big arrays — 8 lanes, so it should be 8× faster — but my benchmark only shows about 1.2×. The intrinsics must be wrong; help me debug them." Cover the paragraph and respond in four or five sentences, drawing on §36.1–§36.4.

Model answer. "The intrinsics are probably fine — 1.2× is what I'd expect, because that loop is bandwidth-bound, not compute-bound. Vector-add does one add per two loads and a store, so its arithmetic intensity is tiny: you're limited by how fast DRAM can stream the arrays, not by how fast the core adds, and widening the add from 1 lane to 8 doesn't help when the bottleneck is the memory bus. The 8× rule only applies to loops that are actually compute-bound and cache-resident — a math-heavy kernel on data that fits in L1 can hit or exceed 8×, which we've measured. So before debugging the intrinsics, let's confirm the diagnosis: time the plain `-O3 -march=native` auto-vectorized version and check `-fopt-info-vec` — I bet it matches your 1.2×, which means the memory system is the ceiling and the real win is in layout and locality (fewer bytes moved), not more lanes. If we truly need this faster, the lever is Chapters 34–35, not more SIMD." Diagnosing the ceiling before optimizing against it — and preferring the auto-vectorizer plus a measurement to hand intrinsics — is the senior register.

(Answer to the §36.2 quick check: the same source ran at 3.4/25/45 Gflop/s because vectorization depends on the build — `-O2` did not vectorize, `-O3` used 128-bit SSE2, and `-O3 -march=native` used 256-bit AVX2. The scalar-to-AVX2 gain exceeds 8× because vectorization also enabled fused multiply-add, loop unrolling, and lower per-iteration overhead, so the win is lane count times those other improvements. You confirm a loop vectorized with the compiler's own report, `-fopt-info-vec-optimized`, which named "32 byte vectors," rather than assuming it did.)

36.5 What to carry forward

SIMD is data-level parallelism on a single core: one instruction operating on a whole vector — 8 `float`s in 256-bit AVX2 — for up to lane-count throughput, and often more once vectorization unlocks fused multiply-add and unrolling (13× on the compute-bound kernel of §36.2). It is not something you usually write by hand: reach for **auto-vectorization** first, with the right flags (`-O3 -march=native`), contiguous data (Chapter 35's struct-of-arrays), and an independent loop, and *confirm* it with `-fopt-info-vec`. Know what blocks it — loop-carried dependencies, floating-point reductions that need `-ffast-math` to reassociate, pointer aliasing that `restrict` resolves, strided or branchy access — and drop to **intrinsics** only when the payoff is proven and the compiler will not deliver it. Above all, respect the ceiling: SIMD raises the *compute* roof, so it multiplies arithmetic-bound loops but does almost nothing for bandwidth-bound ones (2.4× on the streaming dot product, §36.4). Which ceiling binds is a question of arithmetic intensity, and the honest way to answer it is a measurement, not a lane count.

This chapter closes the arc the part has been building. Chapter 33 said measure first and let a profile find the dominant cost. Chapter 34 opened the most common cause of that cost — the memory hierarchy — and got the access order right. Chapter 35 got the data layout right, contiguous and dense. And this chapter turned that well-placed, well-laid-out data into wide parallel arithmetic, while insisting that the memory system, not the vector unit, often sets the limit — which is why locality and layout came first, and why the whole part keeps returning to a timed run to tell truth from hope. That is the through-line to carry out of Performance Engineering: the machine has several ceilings — instruction count, cache, bandwidth, vector width — and engineering performance is finding, by measurement, which one you are actually under, and then raising *that* one. The next part turns from the CPU and its memory to the boundary where programs meet the outside world — **I/O and networking** — where a new set of costs, and a new set of "where does the time go" questions, take over.

CAN YOU… WITHOUT LOOKING?

- Explain what SIMD is and where the lanes come from (register width), with the count for 256-bit AVX2.
- Name the two properties a loop needs to auto-vectorize, and the flags that enable and confirm it.
- Explain why a floating-point reduction will not vectorize without `-ffast-math`, and the risk of that flag.
- Explain why the scalar-to-AVX2 speedup can exceed the lane count.
- Distinguish a compute-bound from a bandwidth-bound loop, and say which SIMD helps.
- State when you would drop to intrinsics, and why it is a last resort.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is SIMD, how many `float`s does a 256-bit AVX2 instruction process, and what two loop properties does it require? (§36.1)
2. **R2.** What flags enable and confirm auto-vectorization in gcc, and why did the same source run at 3.4/25/45 Gflop/s? (§36.2)
3. **R3.** Why won't gcc vectorize a floating-point reduction without `-ffast-math`, and what does that flag risk? (§36.3)
4. **R4.** Why did the dot product gain only ~2.4× from SIMD while the L1 kernel gained 13×? Use arithmetic intensity. (§36.4)
5. **R5.** When do you reach for intrinsics rather than the auto-vectorizer, and why is it a last resort? (§36.4)

FURTHER READING

- **John L. Hennessy & David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017), ISBN 978-0-12-811905-1.** Chapter 4 ("Data-Level Parallelism in Vector, SIMD, and GPU Architectures") is the architectural source for §36.1 — vector registers, lanes, and why width is throughput. First author John Hennessy; edition and ISBN verified via the publisher.
- **Samuel Williams, Andrew Waterman & David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM* 52(4), 65–76 (2009), DOI 10.1145/1498765.1498785.** The roofline model of §36.4 — arithmetic intensity, the compute and bandwidth ceilings, and why a low-intensity loop like the dot product is bandwidth-bound. First author Samuel Williams; DOI verified via the ACM Digital Library.
- **Agner Fog, optimization manuals, *Optimizing software in C++* and *Instruction tables* (`agner.org/optimize`, continuously updated).** The practical reference for vectorization, intrinsics, and per-instruction latencies/throughputs behind §36.3–§36.4. Author Agner Fog; living documents, not a dated publication, so cited as such — consult the current versions.
- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2015), ISBN 978-0-13-409266-9.** Chapter 5 covers program optimization including vectorization, loop-carried dependencies, and the reduction/associativity issue of §36.3 from the programmer's side. First author Randal Bryant; edition and ISBN verified via the publisher.

Exercises

36.1 WARM-UP Define SIMD in one sentence. How many `float`s and how many `double`s fit in a 256-bit AVX register, and how many does one such arithmetic instruction process at once?

36.2 WARM-UP Name the two properties a loop's data and computation must have for the vector unit to deliver its throughput, and say which chapter's layout supplies the first.

36.3 WARM-UP Which gcc flags (a) enable vectorization at all, (b) let it use the widest vectors your CPU supports, and (c) let you confirm that a given loop was vectorized?

36.4 CORE The §36.2 kernel went 3.43 → 24.89 → 45.14 Gflop/s across scalar, `-O3` (SSE2), and `-O3 -march=native` (AVX2). (a) Why does the same source produce three different throughputs? (b) The scalar-to-AVX2 gain is 13×, more than the 8-lane width — give two reasons vectorization can beat the raw lane count. (c) Why is "confirm with `-fopt-info-vec`" better engineering than assuming the loop vectorized?

36.5 CORE Floating-point reductions. (a) Why does vectorizing `s += a[i]` change the numerical result, and why does that stop gcc from doing it at `-O3`? (b) What flag permits it, and what else does that flag allow that might be unsafe? (c) The dot product gained only ~2.4× even after vectorizing — name the reason, and say why it is *not* the reduction issue.

36.6 CORE For each loop, say whether it can be auto-vectorized as written, and if not, why: (a) `c[i] = a[i] * b[i];` (b) `a[i] = a[i-1] + x[i];` (c) `if (x[i] > 0) y[i] = f(x[i]);` with a heavy data-dependent branch; (d) `s += a[i];` summed into one `float` at plain `-O3`.

36.7 COST Arithmetic intensity and the roofline. (a) Define arithmetic intensity in one sentence. (b) Estimate it (flops per byte moved) for the dot product (2 flops per two 4-byte loads) and argue why it is bandwidth-bound. (c) For the §36.2 kernel (~11 flops per 4-byte element, data in L1), argue why it is compute-bound. (d) State the rule for which loops SIMD helps.

36.8 FADED *Worked, with the last step for you.* A colleague reports that hand-written AVX2 intrinsics for `c[i]=a[i]+b[i]` over 100 MiB arrays give only 1.2× over scalar, and concludes the intrinsics are buggy.
Step 1 (given): vector-add does 1 add per 2 loads + 1 store — very few flops per byte moved (low arithmetic intensity).
Step 2 (given): 100 MiB arrays do not fit in any cache, so the loop streams from DRAM every pass.
Step 3 (given): therefore the loop is bandwidth-bound; the memory bus, not the adder, sets the ceiling.
Step 4 (you): Explain why 1.2× is expected (not a bug), how you would confirm the diagnosis with the auto-vectorizer and a measurement, and where the real speedup would have to come from instead.

36.9 MIXED For each, say whether SIMD is likely to give a large speedup, a small one, or none, and why: (a) a math-heavy per-pixel shader on an image tile that fits in L1; (b) copying a 1 GiB buffer; (c) summing a cache-resident array of 10,000 `float`s in a tight loop; (d) a loop with a loop-carried dependency computing a running total sequentially.

36.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “8 vector lanes always means 8× faster.” (b) “gcc auto-vectorizes floating-point reductions at `-O3` by default.” (c) “`-march=native` lets the compiler use the widest vectors the build machine supports.” (d) “SIMD helps a memory-bandwidth-bound loop as much as a compute-bound one.” (e) “An array-of-structs field is as easy to vectorize as a packed struct-of-arrays field.”

36.11 STRETCH Tie the whole part together. (a) Explain why Chapters 34 and 35 (locality and layout) are prerequisites for Chapter 36 (SIMD), not independent tricks. (b) A profile (Chapter 33) shows one kernel at 80% of runtime; describe the sequence of questions — bound by what? fixable how? — you would ask, in order, drawing on all four chapters. (c) Argue why “the machine has several ceilings; find the one you're under and raise that one” is a better mental model than “make the code do fewer instructions.”

36.12 LAB On your own machine (a C toolchain with a vectorizing compiler): (a) Write a compute-heavy, per-element-independent kernel over an array that fits in L1; compile it at `-O2 -fno-tree-vectorize`, `-O3`, and `-O3 -march=native`, time each as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials, and report the throughputs and the speedup; confirm vectorization with `-fopt-info-vec-optimized`. (b) Write a dot product over arrays far larger than your last-level cache; show it barely vectorizes without `-ffast-math` and gains little even with it, and explain via arithmetic intensity. (c) State your toolchain version, CPU, and widest vector width, and note that these are wall-clock throughputs (no hardware counters) if you lack `perf`.

Chapter 37 — Branch Prediction and Speculative Execution

Before you start: Chapter 2 (the CPU up close — the pipeline that a misprediction flushes), Chapter 10 (the optimizing compiler — *if-conversion* is one of its transforms, and it changes everything here), Chapter 36 (SIMD and masks — how the vectorizer turns a branch into arithmetic), Chapter 33 (measure first — this chapter's central claim is that you must, because the effect appears and vanishes with the build). · The last three chapters raised the memory and compute ceilings; this one opens a different cost the profiler can hand you — a branch the CPU guessed wrong — and shows why the honest number is, again, a measured one.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 2 — what is an instruction *pipeline*, and why does a modern core have several instructions in flight at once rather than finishing one before starting the next? (2) Chapter 33 — why do you report the minimum of several timed trials rather than a single run? (3) *Predict:* you sum only the elements `>= 128` of a 32768-element array of random bytes, with a plain `if (data[i] >= 128) sum += data[i];`. You then *sort* the array first and run the identical loop again. Same arithmetic, same element count. How much faster do you expect the sorted run: about the same, 2×, or something far larger? And does your answer depend on the compiler flags? Write your guesses; §37.2 and §37.4 answer them.

A branch — an `if`, a loop test, a `switch`, a function return — is a fork in the instruction stream, and a modern CPU cannot afford to wait to learn which way it goes. The pipeline (Chapter 2) has a dozen or more instructions in flight at once, so by the time a branch's condition is actually computed the core has already fetched and begun executing the instructions after it — but *which* instructions? The ones on the taken path or the not-taken path? The core does not know yet, so it **guesses**, using a hardware **branch predictor**, and speculatively runs ahead down the predicted path. When the guess is right, the work is already done and the branch cost essentially nothing. When it is wrong, everything speculatively executed after the branch must be thrown away and the pipeline refilled from the correct target — a **misprediction penalty** of, on published figures, roughly 15–20 cycles on modern x86 (Agner Fog's microarchitecture manual). That is the cost this chapter is about: not the work the branch guards, but the guess it forces, and what happens when the data makes the guess unpredictable. This chapter proceeds in five steps: why a branch can cost more than the work it guards (§37.1); the predictor, and the sorted-array experiment that made this famous, measured here (§37.2); making the branch cheap — predication, `cmov`, and branchless code (§37.3); what the compiler already did to your branch, and why the effect appears and vanishes with the build, so you must measure (§37.4); and what to carry forward (§37.5).

On the code in this chapter. As before the programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H, timed with `clock_gettime(CLOCK_MONOTONIC)` and reported as the minimum of several trials (Chapter 33). Times are wall-clock seconds for a fixed amount of work and are illustrative of the effect, not a benchmark of the machine. Where a compiler decision is claimed — that a loop kept a real branch, used a conditional move, or vectorized with a mask — it is confirmed by reading gcc's own assembly output (`-S`) or its `-fopt-info-vec` report, quoted inline, not asserted. A hardware-counter profiler (`perf`, which would report branch-misprediction counts directly) is *not* installed here, so the evidence for a misprediction is wall-clock time plus the emitted instructions — which, as §37.4 argues, is exactly the evidence you can actually ship a decision on. Published cycle figures (pipeline depth, misprediction penalty) are labelled as such and attributed; they are not measured here.

37.1 Why a branch can cost more than the work it guards

A pipelined core overlaps instructions: while one is executing, the next is being decoded and the one after that fetched, so several are in flight at once (Chapter 2). This is **instruction-level parallelism**, and it is most of why a modern core is fast. A branch threatens it. Consider `if (cond) A; else B;`: the core must keep fetching to keep the pipeline full, but the target address — the address of `A` or of `B` — is not known until `cond` is evaluated, several stages later. Stalling until then would drain the pipeline and throw away the parallelism, so instead the core **predicts** the outcome and speculatively executes down the guessed path, holding the results provisionally. If the prediction was right, those results are committed and the branch was, in effect, free. If it was wrong, the speculated instructions are **squashed** — their results discarded — and the front end restarts fetching from the correct target, leaving a bubble of a dozen-plus cycles while the pipeline refills. That refill is the misprediction penalty, and its size is roughly the depth of the pipeline: published figures put modern x86 pipelines at on the order of 14–19 stages and the misprediction penalty at roughly 15–20 cycles (Agner Fog; Hennessy & Patterson, ch.3). The crucial consequence is that the cost of a branch is *not* the cost of the comparison — a `cmp` is one cheap instruction — but the cost of guessing wrong, paid only on mispredictions. A branch the predictor gets right almost every time is nearly free; a branch it gets wrong half the time can dominate a loop, costing far more per iteration than the handful of arithmetic instructions inside it. Whether a branch is cheap or ruinous therefore depends not on the code but on the *data* — on whether the pattern of taken and not-taken outcomes is something the predictor can learn.

> **QUICK CHECK**
>
> In one sentence each: what does a CPU do at a branch rather than wait for the condition, and what is thrown away when the guess is wrong? Is the expensive part the comparison or the misprediction? (Answer after the next section.)

37.2 The predictor, and the array that made branches famous

The branch predictor is a small hardware table that records, per branch, the recent history of which way it went, and bets it will go the same way again; modern predictors correlate on the pattern of surrounding branches too (the classic strategies are surveyed by J. E. Smith, ISCA 1981). It is startlingly good on *regular* patterns: a loop that runs a million iterations takes its back-edge a million times and is mispredicted essentially once, on the final exit; a branch that is almost always taken is learned in a few iterations. Where it fails is on branches whose outcome is **data-dependent and irregular** — where taken and not-taken alternate with no learnable pattern. The canonical demonstration is a single data-dependent branch inside a hot loop, run first over unsorted data and then over the same data sorted. Here is that experiment, run in this environment (gcc 11.4.0, minimum of several trials), summing the elements `>= 128` of a 32768-element array of random bytes, 20000 passes:

```
for (i) if (data[i] >= 128) sum += data[i];     // one data-dependent branch per element

# built so the compiler keeps a REAL branch (see §37.4 for why this flag is needed):
$ gcc -O2 -fno-if-conversion -fno-if-conversion2 -fno-tree-loop-if-convert branch.c
$ ./a.out unsorted     random order, branch ~50% taken :  7.64 s
$ ./a.out sorted       sorted first, branch predictable :  0.50 s      # ~15x faster,
identical arithmetic
```

Same instructions, same additions, same 32768 elements summed 20000 times — and the sorted run is about **15×** faster. Nothing about the arithmetic changed; what changed is the *predictability* of the branch. On the random array, whether `data[i] >= 128` is true is a coin flip with no pattern, so the predictor is wrong on roughly half of 655 million branches, and each miss costs a pipeline refill; on the sorted array the branch is false for the whole first stretch (all the values below 128) and then true for the whole rest, so it flips exactly once and is predicted correctly essentially always. This is the famous “why is processing a sorted array faster” effect, and it is real — but read the flag on the compile line, because it is doing something essential that §37.4 returns to: we had to *forbid* the compiler from removing the branch in order to see the effect at all. The lesson of the raw phenomenon stands on its own, though: **an unpredictable data-dependent branch in a hot loop can be the dominant cost**, dwarfing the work it guards, and the profiler that pointed you at this loop (Chapter 33) is telling you about mispredictions, not arithmetic.

> **QUICK CHECK**
>
> Why is the sorted run ~15× faster when the arithmetic is identical? On which array is the branch predictable, and why? What single change to the data — not the code — turned a ruinous branch into a nearly free one? (Answer below the communicate box.)

37.3 Making the branch cheap: predication, cmov, and branchless code

If the misprediction is the cost, the cure is to stop mispredicting — and there are three ways, in rough order of preference. The first is to **make the branch predictable**: if sorting or partitioning the data groups the outcomes (as above), or if the branch is almost always one way, the predictor handles it and you need do nothing. The second, when the branch is inherently a coin flip, is to **remove the branch** and compute both sides, selecting the result with data — *branchless* code. The value `data[i] >= 128 ? data[i] : 0` can be produced with no control flow at all: subtract 128, use the sign bit as a mask, and AND it against the value, so the loop always does the same instructions regardless of the data and there is nothing to mispredict. The hardware form of this is the **conditional move** (`cmov`): an instruction that selects one of two values based on a flag without branching, so the pipeline never has to guess. Here is the branchless mask version, run over the same data:

```
int m = (data[i] - 128) >> 31;   // 0 if >=128, -1 if <128 (arithmetic shift = sign bit)
sum += data[i] & ~m;             // adds data[i] when >=128, else 0 — no branch

$ ./a.out unsorted branchless :  0.87 s      # data order no longer matters
$ ./a.out sorted   branchless :  1.01 s      # ~same as unsorted: nothing to mispredict
```

The branchless version runs in about the same time — near 0.9 s — *regardless of whether the data is sorted*, because it has no data-dependent branch: it is immune to the 15× swing entirely. Notice the trade it makes. On the unsorted data it is a large win (0.9 s versus the 7.64 s of the mispredicting branch), because it converts an unpredictable branch into unconditional arithmetic. But on the *sorted* data it is actually a little *slower* than the well-predicted branch (about 0.9–1.0 s versus 0.50 s), because the branchless code always does the work of both paths — the mask, the AND, the add — whereas a correctly predicted branch does the work of only the taken path and skips the rest for free. That is the fundamental tension: **branchless code trades a possible misprediction for guaranteed work on both paths**. It wins when the branch is unpredictable (the misprediction you avoid costs more than the extra arithmetic) and loses when the branch is predictable (you pay for both paths to avoid a misprediction that was not going to happen). Which regime you are in is, once again, a property of the data, and the honest way to decide is to measure both — not to assume branchless is always faster.

THE SAME IDEA THREE WAYS: A BRANCH IS A BET ON THE DATA

1. In words. To keep the pipeline full, the CPU guesses which way each branch goes and runs ahead speculatively. A correct guess is free; a wrong guess throws away a pipeline's worth of work (~15–20 cycles). So a branch is cheap when its outcomes are predictable and expensive when they are a data-dependent coin flip — and the fix is either to make the pattern learnable or to remove the branch and always do the work of both sides.

2. As a picture. Figure 37.1: a pipeline with several instructions in flight; at a branch, a predictor picks a path and speculation fills the pipe. Right guess → commit; wrong guess → squash and refill (the bubble). Below, three bars for the same summing loop: a mispredicting branch on unsorted data (tall), the same branch on sorted data (short), and branchless code (medium, and the same height whatever the order).

3. As numbers/contrast. Same machine, same arithmetic: a real data-dependent branch ran **7.64 s unsorted vs 0.50 s sorted** — a 15× swing driven only by predictability (§37.2). Branchless mask code ran ~0.9 s *either way* (§37.3): a huge win over the mispredicting branch, a slight loss to the well-predicted one. Predictability, not instruction count, decided the cost.

(Answer to the §37.1 quick check: at a branch the CPU predicts the outcome and speculatively executes down the guessed path rather than stalling until the condition is known; on a wrong guess it squashes — discards — everything speculatively executed after the branch and refills the pipeline from the correct target. The expensive part is the

misprediction, not the comparison: the `cmp` is one cheap instruction, while a wrong guess costs a pipeline-depth's worth of cycles.)

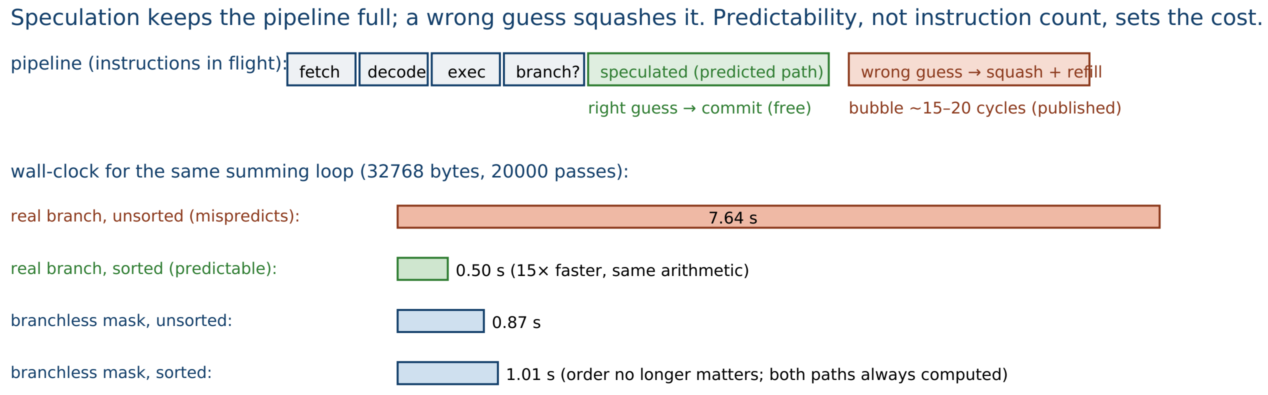


Figure 37.1: The CPU speculates past a branch to keep the pipeline full; a misprediction squashes and refills it. The same summing loop swung 15× (7.64 → 0.50 s) purely on whether the data made the branch predictable; branchless code ran ~0.9 s either way, immune to order but always doing both paths' work (§37.2–§37.3). Wall-clock, illustrative.

37.4 What the compiler already did, and why you must measure

Now the flag on the §37.2 compile line, which has been waiting. The 15× effect required `-fno-if-conversion -fno-if-conversion2 -fno-tree-loop-if-convert` — flags that *forbid* the compiler from transforming the branch. Left to its own devices, gcc at plain `-O2` does not emit a branch for this loop at all: it applies **if-conversion** (Chapter 10's optimizer), turning `if (data[i] >= 128) sum += data[i];` into a branchless conditional move. Reading the emitted assembly confirms it — the inner loop contains a `cmovg`, not a conditional jump — and the timings show the effect has evaporated:

```
$ gcc -O2 branch.c                    # compiler free to choose; emits cmov, not a branch
$ ./a.out unsorted :  1.09 s          # (asm: inner loop uses cmovg, no conditional jump)
$ ./a.out sorted   :  0.95 s          # ~same as unsorted: there is no branch to
mispredict

$ gcc -O3 -march=native branch.c      # now it VECTORIZES the loop with a mask
  branch.c:18: optimized: loop vectorized using 32 byte vectors    # -fopt-info-vec
$ ./a.out unsorted :  0.23 s
$ ./a.out sorted   :  0.21 s          # AVX2 masked add: order irrelevant, and fastest of
all
```

Three builds, three stories. At `-O2` the compiler already de-branched the loop into a `cmov`, so both orders run in ~1 s and the sorted-array effect is *gone* — the compiler did the §37.3 branchless transform for you. At `-O3 -march=native` the vectorizer (Chapter 36) goes further and computes the sum with a masked AVX2 add — eight elements per instruction, no branch — and both orders run in ~0.2 s, faster than any scalar version. The famous 15× effect only appeared because we *forced* the compiler to keep a real branch; the code most people actually ship, compiled at `-O2` or above, never had the branch in the first place. This is the chapter's real lesson, and it is a Chapter 33 lesson: **the cost of a branch is a property of the compiled build, not of the source**. You cannot read a misprediction penalty off the C code, because whether there is even a branch to mispredict depends on if-conversion and vectorization decisions the compiler makes at your optimization level and target. Counting branches in the source and multiplying by a published cycle penalty gives a number that is often simply wrong — the branch may not exist in the binary. The only reliable evidence is to look at what the compiler emitted (read the assembly, or check `-fopt-info-vec`) and to *time the build you ship*. Where a real, unpredictable, un-convertible branch does survive — a data-dependent call through a function pointer, a `switch` the compiler keeps as a jump, a branch guarding side-effecting work it cannot speculate — the §37.2 penalty is real and the §37.3 fixes apply; but you learn which case you are in by measuring, not by reasoning from the source.

TRAP: COUNTING BRANCHES IN THE SOURCE — THE MISPREDICTION THAT THE COMPILER ALREADY REMOVED

The seductive move is to read a hot loop, spot a data-dependent `if`, multiply by "~15 cycles per misprediction," and rewrite it branchless — *without* checking whether the branch survives compilation. It usually does not. At `-O2` gcc turned the §37.2 `if` into a `cmovg` (confirmed in the assembly), and at `-O3 -march=native` it vectorized the whole loop with a masked add; in both builds there is no branch to mispredict and the "15× effect" does not exist. Hand-writing branchless mask code on top of that buys nothing — the compiler already did it — and can even lose, because your version may block the vectorizer that would have done better. Worse, the branchless rewrite is a *net loss* whenever the branch was actually predictable (sorted or almost-always-one-way data): it forces the work of both paths to dodge a misprediction that was not happening, and the §37.3 numbers show it running slower than the well-predicted branch. Two defenses. First, before optimizing a branch, confirm it exists in the binary — read the assembly for a conditional jump versus a `cmov`, or check the vectorization report — because the source is not the build (Chapter 10). Second, if a real branch does survive, decide branch-vs-branchless by *measuring both on representative data* (Chapter 33), since the right choice flips with predictability. Reasoning from a cycle penalty in a table, against a branch the compiler already deleted, is optimizing a cost that is not there — the control-flow version of timing a loop whose result is unused.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "I read that unpredictable branches cost ~15 cycles each, and our hot filter loop has a data-dependent `if` that's basically a coin flip on the input. I rewrote it branchless with bit-masks and I'm about to send the PR — should speed up the whole pipeline." Cover the paragraph and respond in four or five sentences, drawing on §37.1–§37.4.

Model answer. "The instinct is right that unpredictable branches are expensive, but before we merge, let's check the branch is actually in the binary — at `-O2` or `-O3` gcc very often converts a small `if` like that into a `cmov` or vectorizes it with a mask, in which case there's no branch to mispredict and your rewrite is doing by hand what the compiler already did (and it might even block the auto-vectorizer). So step one is read the assembly or the `-fopt-info-vec` report for the current build. Step two, if a real branch does survive, measure both versions on representative data, because branchless only wins when the branch is genuinely unpredictable — if the data is sorted or the condition is almost always one way, the predicted branch is faster and your always-both-paths version is a regression. Let's get those two numbers before the PR, so we're shipping a measured win and not a plausible-sounding rewrite of a cost that may not exist." Checking that the branch survives the compiler, then measuring branch-vs-branchless on real data, is the senior register.

(Answer to the §37.2 quick check: the sorted run is ~15× faster because the branch is predictable — false for the whole run of values below 128, then true for the rest, so it flips once and the predictor is right almost always; on the unsorted array the outcome is a coin flip with no pattern, so the predictor is wrong on ~half the branches and each miss costs a pipeline refill. The single change was to the data — sorting it — not to the code; the arithmetic is identical. And this only shows up when the compiler is forbidden to remove the branch, per §37.4.)

37.5 What to carry forward

A **branch** forces the CPU to guess: to keep its deep pipeline full, the core predicts each branch's outcome and speculatively executes down the guessed path, committing the work if the guess was right and squashing a pipeline's worth of it if the guess was wrong — a **misprediction penalty** of roughly 15–20 cycles (published). So a branch is nearly free when its outcomes are predictable and can be the dominant cost when they are an irregular, data-dependent coin flip: the same summing loop swung **15×** (7.64 → 0.50 s) purely on whether the data made the branch learnable (§37.2). The fixes, in order: make the branch predictable (sort or partition the data so the pattern is learnable); or remove it with **branchless** code or a **conditional move**, which is immune to data order but always pays for both paths — a win only when the branch is genuinely unpredictable (§37.3). Above all, remember the twist that makes this a measurement discipline, not a cycle-counting one: **the compiler has usually already removed your branch**. At `-O2` gcc if-converted the example into a `cmov`; at `-O3 -march=native` it vectorized it away entirely; the famous effect only survived because we forbade both. So the cost of a branch is a property of the build, and the honest way to find it is to read the emitted instructions and time the binary you ship (§37.4) — not to count `if`s in the source.

This is the same through-line the part has followed. Chapter 33 said measure first; Chapters 34 and 35 raised the memory ceiling with locality and layout; Chapter 36 raised the compute ceiling with SIMD; and this chapter opened the *control* cost — the branch the CPU guessed wrong — only to find that the compiler often eliminates it before you can, so the discipline is to check the build rather than reason from the source. Two costs the profiler routinely hands you remain before the part closes. The next chapter opens the one Chapter 33 named alongside cache misses and mispredictions — **the cost of dynamic allocation**, where the time goes not into your loop at all but into `malloc` and `free` and the scattered memory they hand back (Chapter 38). Then a final chapter (Chapter 39) ties the whole part into one workflow: given a profile, which ceiling are you under, and how do you raise it.

CAN YOU… WITHOUT LOOKING?

- Explain why a branch threatens pipeline parallelism, and what speculation and squashing are.
- State what a misprediction costs (order of magnitude), and that it is the penalty, not the comparison.
- Explain why the sorted array ran ~15× faster with identical arithmetic.
- Say when branchless code (or a `cmov`) wins and when it loses, in terms of predictability.
- Explain why the effect appeared only with if-conversion disabled, and what that says about source vs build.
- State how you would confirm whether a branch even exists in the binary.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why does a modern CPU predict branches and execute speculatively, and what is thrown away on a wrong guess? (§37.1)
2. **R2.** In the sorted-vs-unsorted experiment, why is the sorted run ~15× faster when the arithmetic is identical? (§37.2)
3. **R3.** What does branchless code (or `cmov`) trade away, and in which data regime does it win versus lose? (§37.3)
4. **R4.** Why did the 15× effect require disabling if-conversion, and what does that prove about reading branch cost off the source? (§37.4)
5. **R5.** How do you confirm whether a data-dependent `if` is actually a branch in the shipped binary? (§37.4)

FURTHER READING

- **John L. Hennessy & David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017), ISBN 978-0-12-811905-1.** Chapter 3 ("Instruction-Level Parallelism and Its Exploitation") is the architectural source for §37.1–§37.2 — pipelines, speculation, dynamic branch prediction, and the misprediction penalty. First author John Hennessy; edition and ISBN verified via the publisher.
- **Agner Fog, *The microarchitecture of Intel, AMD, and VIA CPUs: An optimization guide for assembly programmers and compiler makers* (`agner.org/optimize`, continuously updated).** The practical reference for the per-microarchitecture branch-prediction algorithms and the published pipeline-depth and misprediction-penalty figures used in §37.1. Author Agner Fog; a living document, cited as such — consult the current version.
- **James E. Smith, "A Study of Branch Prediction Strategies," in *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA '81)*, pp. 135–148 (1981).** The classic survey of branch-prediction strategies behind §37.2 — the history of how predictors learn patterns. First author James E. Smith; conference and pages verified; this legacy paper carries no resolvable DOI, so it is cited by venue and pages.
- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2015), ISBN 978-0-13-409266-9.** Chapter 4 (processor architecture and pipelining) and Chapter 5 (optimizing program performance, including the effect of branches and the compiler's transformations) cover this chapter from the programmer's side. First author Randal Bryant; edition and ISBN verified via the publisher.

Exercises

37.1 WARM-UP In two sentences, explain why a deeply pipelined CPU cannot simply wait at a branch for the condition to be computed, and what it does instead.

37.2 WARM-UP What is a branch-misprediction penalty, roughly how many cycles is it on modern x86 (order of magnitude, and label it as a published figure), and why is it about the depth of the pipeline?

37.3 WARM-UP The comparison in `if (data[i] >= 128)` is a single cheap instruction. Explain why, then, the branch can still be the most expensive thing in the loop.

37.4 CORE The summing loop ran 7.64 s on unsorted data and 0.50 s on the same data sorted, with a real branch. (a) Explain the ~15× difference in terms of the predictor and the misprediction penalty. (b) Why is the branch predictable on the sorted array and not on the unsorted one? (c) The arithmetic and instruction count are identical in both runs — what, precisely, differs?

37.5 CORE Branchless code. (a) Rewrite `if (x >= 128) sum += x;` as branchless arithmetic using the sign bit as a mask, and explain why it has nothing to mispredict. (b) The branchless version ran ~0.9 s on both sorted and unsorted data, while the real branch ran 0.50 s on sorted data — why is branchless *slower* here than the well-predicted branch? (c) State the rule for when branchless wins.

37.6 CORE The 15× effect only appeared with `-fno-if-conversion -fno-if-conversion2 -fno-tree-loop-if-convert`. (a) What did the compiler do to the branch at plain `-O2`, and how would you confirm it from the binary? (b) What did it do at `-O3 -march=native`? (c) What does this say about estimating branch cost by reading the C source?

37.7 COST A hot loop runs K iterations, each with one data-dependent branch, and a misprediction costs p cycles. (a) Write the extra cycles spent on mispredictions if the branch is mispredicted a fraction f of the time. (b) Contrast $f = 0.5$ (coin flip) with f near 0 (predictable). (c) A branchless version does w extra cycles of both-paths work per iteration and never mispredicts; write the condition on f, p, w under which branchless is faster.

37.8 FADED *Worked, with the last step for you.* A colleague benchmarks a data-dependent `if` in a hot loop, sees no difference between sorted and unsorted input, and concludes "branch prediction is a myth on this CPU."
Step 1 (given): they compiled at `-O2`, the default for their build.
Step 2 (given): reading the assembly shows the inner loop uses `cmovg`, not a conditional jump.
Step 3 (given): a conditional move is branchless — there is no prediction and nothing to mispredict.
Step 4 (you): Explain why they saw no sorted-vs-unsorted difference, what single change would let them observe the effect, and why the correct conclusion is about the *build*, not the CPU.

37.9 MIXED For each branch, say whether the predictor is likely to handle it well or badly, and why: (a) a loop back-edge running 10 million iterations; (b) `if (rand() & 1)` inside a hot loop; (c) `if (ptr != NULL)` where `ptr` is almost never null; (d) a `switch` on the next byte of compressed data with a uniform distribution over 8 cases.

37.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "A data-dependent `if` in C always costs a misprediction penalty at run time." (b) "Branchless code is always faster than a branch." (c) "A correctly predicted branch is nearly free." (d) "`-O3 -march=native` can turn a scalar `if`-loop into a masked vector loop with no branch." (e) "You can read a loop's misprediction cost off the C source without looking at the build."

37.11 STRETCH Tie this chapter to the rest of the part. (a) A profile (Chapter 33) shows a hot loop, and you suspect branch misprediction; describe the sequence of checks — build? assembly? measurement? — you would run before rewriting anything. (b) Explain how §37.4's masked vectorization connects branch elimination to Chapter 36 (SIMD). (c) Argue why "a branch's cost is a property of the build and the data, not the source" is a better mental model than "branches are slow, so avoid them."

37.12 LAB On your own machine (a C toolchain with a vectorizing compiler): (a) Write the summing loop `if (data[i] >= 128) sum += data[i];` over a 32768-element random-byte array; compile once at plain `-O2` and once with if-conversion disabled (`-fno-if-conversion -fno-if-conversion2 -fno-tree-loop-if-convert`), and time each on unsorted and on sorted data as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials. (b) Report which build shows the sorted-vs-unsorted swing and which does not, and confirm by reading the assembly (look for `cmov` vs a conditional jump). (c) Add a branchless mask version and show it is order-independent. (d) State your toolchain and CPU, and note these are wall-clock times (no hardware counters) if you lack `perf`.

Chapter 38 — The Cost of Dynamic Allocation

Before you start: Chapter 9 (manual allocation — what `malloc` and `free` are and what goes wrong), Chapter 12 (custom allocators — arenas, pools, and free-lists, built there for correctness and reused here for speed), Chapter 3 (the memory hierarchy — why *where* the allocator puts your objects matters as much as how long the call took), Chapter 34 (locality — the ceiling scattered allocation runs you into), Chapter 33 (measure first). · The last chapter opened the control cost — a branch guessed wrong; this one opens the cost the profiler blames on nothing in your loop at all, because the time went into `malloc`, `free`, and the scattered memory they hand back.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 9 — what does a general-purpose `malloc` have to *do* on a call beyond returning a pointer: how does it find a free block of the right size, and what does `free` have to track? (2) Chapter 12 — what is an *arena* (bump) allocator, and why is its allocation just a pointer increment? (3) *Predict:* you allocate a million 32-byte objects, first with a `malloc` per object and then by bump-allocating them out of one big block (an arena). Two questions: how much faster is the arena's *per-object* cost — about the same, 2×, or more? And after you have them, you walk a linked list through all million objects: how much does it matter whether the objects are contiguous or scattered across the heap — a little, or a lot? Write your guesses; §38.2 answers both.

A call to `malloc` looks like it costs nothing — it returns a pointer and you move on — but it is a real computation, and sometimes an expensive one. A general-purpose allocator must, on each call, find a free block of at least the requested size in a heap shared by every other allocation in the program: it searches size-segregated free-lists, may split a larger block or coalesce freed neighbours, updates per-block bookkeeping (the size and status headers of Chapter 9), keeps it all thread-safe, and occasionally asks the kernel for more memory with a system call (Chapter 13). None of that is your program's work; it is the tax of asking a shared, general mechanism for a block of a general size at a general time. And there is a second cost that does not show up in the call at all: `malloc` is free to hand back blocks scattered anywhere in the heap, so a data structure built from many small allocations ends up spread across memory — and walking it then pays a cache miss per node (Chapter 3), which the profiler charges to your traversal loop, not to the allocator that scattered the data. This chapter is about both costs and their cure. It proceeds in five steps: why an allocation can cost more than the memory it returns (§38.1); the two costs measured here — the call and the cache (§38.2); making allocation cheap with arenas, pools, and free-lists (§38.3); what the allocator already does for you, and why you must measure rather than assume (§38.4); and what to carry forward (§38.5).

On the code in this chapter. As before the programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H, against the system allocator (glibc 2.35's `ptmalloc2`), timed with `clock_gettime(CLOCK_MONOTONIC)` and reported as the minimum of several trials (Chapter 33). Times are wall-clock and are illustrative of the effect, not a benchmark of the allocator. The cache sizes quoted (L1d 48 KiB, L2

1.25 MiB, L3 24 MiB) are read from `/sys/devices/system/cpu/cpu0/cache` on this machine, so the pointer-chase working set below is chosen to exceed them. A hardware-counter profiler (`perf`, which would report cache-miss counts directly) is *not* installed here, so the evidence for the locality cost is wall-clock time plus the arithmetic that the scattered working set cannot fit in cache; the numbers depend on this allocator and this machine and are meant to show the *shape* of the cost, not a portable constant.

38.1 Why an allocation can cost more than the memory it returns

The stack costs nothing to allocate on: a function's locals appear by decrementing the stack pointer, one instruction, and vanish when the frame pops (Chapter 8). The heap is the opposite: it is a single pool of memory shared by every allocation in the program, with no implied order of freeing, so the allocator must actively *manage* it. On a `malloc(n)` it has to locate a free block of at least `n` bytes — typically by walking a **free-list** of available blocks, often segregated into size classes so the search is bounded — and if the block it finds is bigger than needed it **splits** it, returning part and keeping the remainder free. Each block carries a header recording its size and whether it is in use, so the allocator can find its neighbours; on `free` it reads that header, marks the block available, and often **coalesces** it with adjacent free blocks to fight fragmentation (Wilson et al.'s survey catalogues the strategies). All of this must be safe against other threads, so a shared allocator takes a lock or manipulates per-thread caches. And when the heap has no block to give, `malloc` crosses the **system-call boundary** (Chapter 13) — `brk` or `mmap` — to get more from the kernel, which is orders of magnitude more expensive than the common case. The point is that a general-purpose allocation is a small algorithm over shared mutable state, not a free handout: the classic account (Bryant & O'Hallaron, ch.9) shows a real `malloc` is dozens of instructions of list-walking and bookkeeping in the good case and a syscall in the bad one. Whether that matters depends, as always, on how often you do it — a handful of allocations at startup is invisible; millions inside a hot loop are not.

> **QUICK CHECK**
>
> Name three things a general-purpose `malloc` may have to do on a single call beyond returning a pointer. Why is a stack allocation essentially free while a heap allocation is not? (Answer after the next section.)

38.2 The two costs, measured: the call and the cache

There are two distinct costs, and it is worth separating them because they have different cures. The first is the **per-call cost** of allocation and deallocation itself. Here, allocating a million 32-byte objects one `malloc` at a time and then freeing them all, versus bump-allocating them out of a single arena block and freeing the arena in one call, run in this environment (gcc 11.4.0, minimum of several trials):

```
1,000,000 objects of 32 bytes:
                          allocate       free            per object (alloc+free)
malloc / free (glibc) :  9.6 ns/op   +   8.6 ns/op    =   ~18 ns/object
arena (bump pointer)  :  3.6 ns/op   +   ~0           =   ~3.6 ns/object   (one free() for
all 1,000,000)
```

The arena is about **5× cheaper per object** on allocation-plus-free, and the asymmetry is even sharper than that average: arena allocation is a pointer increment (~3.6 ns), and arena *deallocation* is a single `free()` of the whole block no matter how many objects it held, so the per-object free cost is essentially zero, versus ~8.6 ns each for a million individual `free`s. That is the first cost, and it is real but bounded — a few nanoseconds per object. The second cost is larger and sneakier: **locality**. Because `malloc` can return blocks from anywhere in the heap, a structure built from many separate allocations is scattered across memory, and walking it pays a cache miss per node. Here is the same set of objects linked into a list and traversed, once when the nodes are contiguous (an arena, list order = memory order) and once when each node was `malloc`'d separately and linked in a shuffled order so consecutive nodes are far apart — a working set of ~244 MiB, far larger than this machine's 24 MiB L3:

```
traverse 4,000,000 nodes (64 bytes each, ~244 MiB >> 24 MiB L3), summing one field:
contiguous (arena)      :   8.1 ns/node       (hardware prefetcher hides the latency; near-
sequential)
scattered (malloc+shuffle): 177 ns/node       (a cache miss, often a TLB miss, per node)
                                              ~22x slower, identical pointer-chasing loop
```

Same loop, same number of nodes, same arithmetic — and the scattered layout is about **22× slower**, at ~177 ns per node against ~8 ns. Nothing in the traversal code changed; what changed is *where the allocator put the nodes*. On the contiguous arena the accesses march sequentially through memory, so the hardware prefetcher (Chapter 3) fetches the next line before the loop asks for it and most accesses hit cache; on the scattered heap each `p = p->next` jumps to an unpredictable address that is not in cache and not prefetchable, so the core stalls on DRAM — hundreds of cycles — for essentially every node. This is the cost that hides: a profiler points at the *traversal* loop as hot, and it is, but the reason is a memory layout decided long ago by how you allocated. The 22× here dwarfs the ~5× per-call cost, which is the chapter's real lesson — **the expensive thing about dynamic allocation is usually not the call, it is the scattered memory the call hands back**, and that cost is paid not at allocation time but on every later traversal.

> **QUICK CHECK**
>
> Which was the larger cost here — the per-call overhead of `malloc/free` (~5×), or the locality penalty of scattered nodes (~22×)? Why does the contiguous traversal hit cache while the scattered one misses, when the loop is identical? (Answer below the communicate box.)

38.3 Making allocation cheap: arenas, pools, and free-lists

The fixes are the custom allocators of Chapter 12, now motivated by speed rather than only control. Each attacks a specific cost, in rough order of how much structure you need.

An **arena** (or *bump*, or *region*) allocator serves allocations by incrementing a pointer through one large block and frees nothing individually — you throw away the whole region at once when the phase that owns it ends. It wins on both costs at once: allocation is a pointer add (the ~3.6 ns above), deallocation is one call, and because the objects are handed out consecutively they are *contiguous*, so later traversals get the ~8 ns locality instead of 177. Its constraint is its power: you cannot free one object, only the whole arena, so it fits phase- or request-scoped work — parse a document, serve a request, run a frame — where everything allocated in the phase dies together. A **pool** (or *slab*) allocator handles the case where objects are all the *same size* and are individually freed and reused: it carves a block into fixed-size slots and keeps a free-list of them, so both allocation and deallocation are an O(1) push/pop and the objects stay packed and cache-friendly. A general **free-list** allocator is the same idea for mixed sizes and is essentially what a general-purpose `malloc` already is — which is the hint that a hand-rolled allocator only helps when it can exploit something the general one cannot: a known lifetime (arena), a known fixed size (pool), or a known usage pattern. The measured payoff — the arena's ~5× on the call and ~22× on traversal — comes precisely from discarding generality the shared allocator has to keep: it does not need to support freeing individual objects, arbitrary sizes, or interleaving with the rest of the program, and it buys speed and contiguity with those constraints (Berger, Zorn & McKinley studied exactly this trade in real programs). The discipline is to reach for a custom allocator when the allocation pattern has such a structure to exploit, and to keep the general `malloc` when it does not.

THE SAME IDEA THREE WAYS: ALLOCATION IS MANAGEMENT, AND SCATTERING IS THE HIDDEN COST

1. In words. A general `malloc` manages one shared heap: it searches free-lists, splits and coalesces blocks, locks, and sometimes calls the kernel — so a call is a small algorithm, not a handout. Worse, it can put your objects anywhere, so a structure of many small allocations is scattered, and every later walk of it pays a cache miss per node. A custom allocator that knows the lifetime (arena) or the size (pool) drops the generality, making the call a pointer bump and the objects contiguous.

2. As a picture. Figure 38.1: on the left a heap with objects scattered among free gaps, a traversal arrow zig-zagging across memory (miss, miss, miss); on the right an arena with the same objects packed consecutively, the arrow marching straight through (hit, hit, hit). Below, two bar pairs: per-object cost (malloc/free ~18 ns vs arena ~3.6 ns) and per-node traversal (scattered ~177 ns vs contiguous ~8 ns).

3. As numbers/contrast. Same machine, one million 32-byte objects: `malloc/free` cost ~18 ns/object, an arena ~3.6 ns with an O(1) bulk free — **~5×** (§38.2). Then walking four million linked nodes: contiguous ~8 ns/node, scattered ~177 ns/node — **~22×** (§38.2). The call is a few nanoseconds; the scattered memory it returns is the dominant cost, paid on every traversal.

(Answer to the §38.1 quick check: on a single call a general `malloc` may walk a size-segregated free-list to find a block, split a larger block or coalesce freed neighbours, update per-block size/status headers, take a lock or use a per-thread cache for thread-safety, and occasionally make a `brk/mmap` system call for more memory. A stack allocation is essentially free because it is a single decrement of the stack pointer with a statically

known lifetime — the frame — whereas the heap is a shared pool with no implied freeing order, so it must be actively managed.)

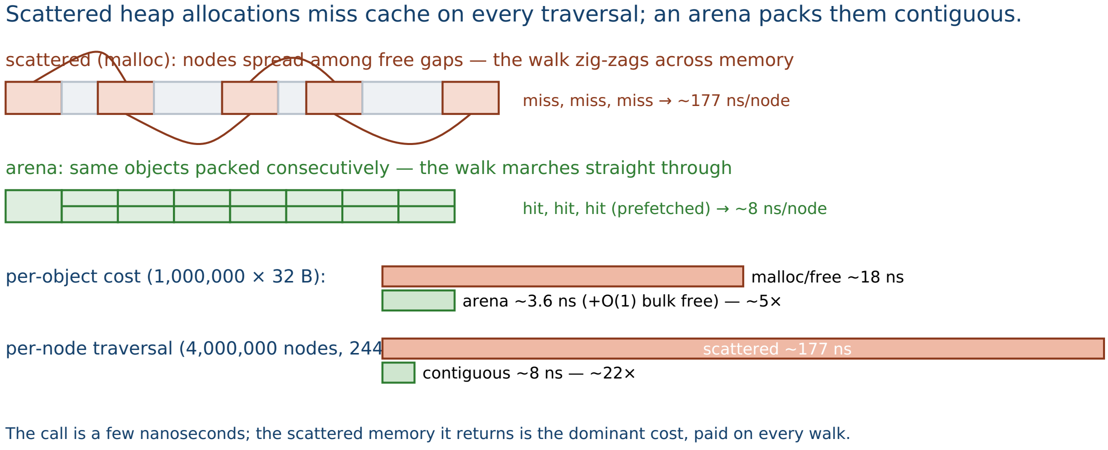


Figure 38.1: Dynamic allocation has two costs. The per-call overhead of `malloc/free` (~18 ns/object) is real but bounded; an arena cuts it ~5× and makes deallocation O(1). The larger cost is *locality*: because `malloc` scatters objects, traversing a structure built from many small allocations misses cache on every node (~177 ns) versus a contiguous arena (~8 ns) — ~22× on an identical loop (§38.2). Wall-clock, illustrative.

38.4 What the allocator already does, and why you must measure

Before hand-rolling an allocator, the same caution the last two chapters raised applies here: the general `malloc` is far better than its worst case, and much of the naive win may already be taken. Modern allocators keep **per-thread caches** (glibc's `tcache`, and the arena-per-thread design that gives `ptmalloc2` its name) so a common-size allocation that was recently freed is served from a thread-local free-list with no lock and no search — which is exactly why the §38.2 `malloc/free` came in at ~18 ns and not the hundreds a syscall would cost. When your allocation pattern is allocate-and-free-in-a-tight-loop of one size, the allocator's fast path is already close to a pool, and a hand-written pool may buy little; when it is many long-lived small objects of mixed size, freed in no particular order, the general allocator scatters them and the arena's contiguity win (§38.2's 22×) is large. You cannot tell which regime you are in by reading the code — it depends on sizes, lifetimes, thread counts, and the allocator's own heuristics — so the rule is the part's rule: **measure the allocation cost in the workload you actually run**. A profiler that shows time in `malloc/free` tells you the per-call cost is real; a profiler that shows time in a traversal loop over freshly allocated data, with a high cache-miss rate, is pointing at the *locality* cost even though the allocator's name never appears. And there is a measurement trap specific to allocation: a microbenchmark that allocates a million objects and immediately frees each one will hit the `tcache` fast path and look cheap, hiding the fragmentation and scattering that the same allocations cause in a long-running program where they interleave with everything else. As always, the number you can act on comes from the representative workload, not the convenient microbenchmark — and the fix you reach for (cheaper calls, or better locality) depends on *which* of the two costs the measurement shows.

TRAP: BLAMING THE ALLOCATOR'S SPEED — THE COST THAT IS REALLY THE SCATTERED MEMORY

The seductive move is to see time in a hot loop that walks a linked structure, conclude "allocation is slow," and either swap in a faster `malloc` or cache allocated objects to avoid the calls — attacking the ~5× per-call cost while ignoring the ~22× one. The per-call cost is rarely where the time is: at ~18 ns an allocation, a program doing millions of them per second spends a percent or two in `malloc`. The time is in the *traversals*, and it is there because the objects are scattered — a cache miss per node (§38.2), which no faster allocator fixes if it still hands back non-contiguous blocks. Two defenses. First, distinguish the two costs with the profile: is the time in `malloc`/`free` themselves (a per-call problem — batch, pool, or arena the calls) or in a loop over the allocated data with a high miss rate (a locality problem — the fix is a contiguous layout, Chapters 34–35, which an arena gives for free)? Second, remember that the largest allocation win is usually not a faster allocator but *fewer, more contiguous* allocations — an arena or a flat array instead of a graph of separately-`malloc`'d nodes — because it converts hundreds-of-cycles pointer chases into prefetchable sequential access. Optimizing the allocator's call speed against a workload whose real cost is scattered traversal is the allocation version of raising a ceiling you are not under.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our graph traversal is the hot loop in the profile, and it's full of pointer-chasing over nodes we `malloc`'d one at a time. I'm going to drop in a faster third-party `malloc` — that should speed up the traversal." Cover the paragraph and respond in four or five sentences, drawing on §38.1–§38.4.

Model answer. "A faster `malloc` probably won't move the traversal, because the traversal doesn't call `malloc` — the allocation already happened. What's slow is that each node was allocated separately, so they're scattered across the heap and every `p = p->next` is a cache miss; on our numbers a scattered walk runs ~20× slower than a contiguous one over the same nodes, and that gap is memory latency, not allocator speed. The faster allocator would cut the *allocation* time, which is likely a percent or two of the run, not the traversal. The real fix is to make the nodes contiguous — allocate them out of an arena, or flatten the graph into arrays with indices instead of pointers — so the walk becomes prefetchable sequential access. Let's confirm with the profile that the time is in the traversal's cache misses, not in `malloc`, and then change the layout, not the allocator." Separating the per-call cost from the locality cost, and fixing layout rather than swapping allocators, is the senior register.

(Answer to the §38.2 quick check: the locality penalty of scattered nodes was far larger — ~22× on the traversal versus ~5× on the per-call cost. The contiguous traversal hits cache because it walks memory sequentially, so the hardware prefetcher loads the next cache line before the loop needs it; the scattered traversal misses because each `next` pointer jumps to an unpredictable address that is neither in cache nor prefetchable, stalling on DRAM for essentially every node — the loop is identical, but the memory layout the allocator produced is not.)

38.5 What to carry forward

Dynamic allocation has two costs, and they need different cures. The first is the **per-call cost**: a general-purpose `malloc` manages a shared heap — searching free-lists, splitting and coalescing, locking, and occasionally calling the kernel — so an allocation is a small algorithm, not a handout (Chapter 9). Measured here it is ~18 ns per allocate-plus-free of a small object, which an arena cuts to ~3.6 ns with an O(1) bulk free — about **5×** (§38.2). The second, usually larger, cost is **locality**: because `malloc` scatters objects across the heap, a structure built from many small allocations is spread through memory, and traversing it pays a cache miss per node — here ~177 ns/node scattered versus ~8 ns/node contiguous, about **22×** on an identical loop (§38.2), a cost charged to the traversal but caused by the allocation. The fixes are Chapter 12's custom allocators, now for speed: an **arena** when everything in a phase dies together (pointer-bump allocation, bulk free, and contiguity for free), a **pool** when objects are one fixed size and individually reused, and the general `malloc` when the pattern has no such structure to exploit (§38.3). Above all, the part's discipline: distinguish the two costs by measuring — time in `malloc` is a per-call problem, time in a high-miss traversal of allocated data is a locality problem — and remember that the biggest allocation win is usually not a faster allocator but *fewer, more contiguous* allocations (§38.4).

This is the same through-line the part has followed, now applied to memory you asked for by hand. Chapter 33 said measure first; Chapters 34 and 35 raised the memory ceiling with locality and layout; Chapter 36 raised the compute ceiling with SIMD; Chapter 37 opened the control cost of a mispredicted branch; and this chapter showed that dynamic allocation lands you right back on Chapter 34's ceiling — its dominant cost is the cache misses of the scattered memory it returns, so the cure is contiguity, the same lever as locality and layout. One chapter remains, and it does not add a new cost but ties all of them together: given a profile, the machine has several ceilings — instruction count, cache misses, branch mispredictions, allocation, bandwidth, vector width — and the engineering question is which one you are actually under and how you raise *that* one (Chapter 39). That capstone turns the part's scattered techniques into a single, ordered workflow.

CAN YOU… WITHOUT LOOKING?

- List three things a general-purpose `malloc` does on a call beyond returning a pointer, and why the stack is free.
- State the two distinct costs of dynamic allocation, and which one is usually larger.
- Explain why a scattered linked structure traverses far slower than a contiguous one, with the order of magnitude.
- Say what an arena and a pool each exploit, and when each applies.
- Explain why swapping in a faster `malloc` often does nothing for a pointer-chasing hot loop.
- State how you would tell the per-call cost from the locality cost using a profile.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does a general-purpose `malloc` have to do on a call, and why can that cost more than returning the memory? (§38.1)
2. **R2.** What are the two costs of dynamic allocation, and roughly how big was each here (per-call vs locality)? (§38.2)
3. **R3.** Why did the scattered traversal run ~22× slower than the contiguous one on an identical loop? (§38.2)
4. **R4.** What does an arena exploit that a general `malloc` cannot, and what does a pool exploit? (§38.3)
5. **R5.** How do you distinguish a per-call allocation cost from a locality cost in a profile, and why does that decide the fix? (§38.4)

FURTHER READING

- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2015), ISBN 978-0-13-409266-9.** Chapter 9 ("Virtual Memory," including dynamic memory allocation) is the source for §38.1 — implicit and explicit free-lists, splitting and coalescing, headers, and what a real `malloc` costs. First author Randal Bryant; edition and ISBN verified via the publisher.
- **Emery D. Berger, Benjamin G. Zorn & Kathryn S. McKinley, "Reconsidering Custom Memory Allocation," in *Proceedings of OOPSLA 2002*, pp. 1–12, DOI 10.1145/582419.582421.** The empirical study behind §38.3 — when custom (region/arena) allocators actually beat a good general-purpose `malloc`, and when they do not. First author Emery Berger; DOI verified via the ACM Digital Library.
- **Paul R. Wilson, Mark S. Johnstone, Michael Neely & David Boles, "Dynamic Storage Allocation: A Survey and Critical Review," in *Memory Management (IWMM '95)*, LNCS 986, Springer, 1995, DOI 10.1007/3-540-60368-9_19.** The comprehensive survey behind §38.1 — free-list strategies, fragmentation, splitting and coalescing across three decades of allocator design. First author Paul Wilson; DOI verified via SpringerLink.
- **Ulrich Drepper, *What Every Programmer Should Know About Memory* (Red Hat technical report, 2007).** The practical reference for §38.2's locality cost — why a pointer-chase across scattered memory stalls on DRAM and defeats the prefetcher, and how cache and TLB behaviour dominate. Author Ulrich Drepper; a technical report (also serialized on LWN.net), carrying no DOI, so cited by author and year.

Exercises

38.1 WARM-UP In two sentences, name three things a general-purpose `malloc` may have to do on a single call beyond handing back a pointer, and say why a stack allocation does none of them.

38.2 WARM-UP What are the *two* distinct costs of dynamic allocation described in this chapter? For each, say whether it is paid at allocation time or later.

38.3 WARM-UP An arena allocator's per-object deallocation cost was reported as essentially zero. Explain why, in terms of how an arena is freed.

38.4 CORE Traversing four million linked nodes ran ~8 ns/node contiguous and ~177 ns/node scattered. (a) Explain the ~22× difference in terms of the cache and the hardware prefetcher. (b) Why does the working set (~244 MiB) exceeding the 24 MiB L3 matter to the result? (c) The traversal loop is byte-for-byte identical in both runs — what, precisely, differs, and who decided it?

38.5 CORE The per-call cost was ~18 ns/object for `malloc`/`free` and ~3.6 ns for an arena. (a) Give two things the arena does *not* do that a general `malloc` must, which account for the difference. (b) The arena's allocation win was ~2.7× but its allocate-plus-free win was ~5× — why is the free side a bigger win than the allocation side? (c) State the constraint an arena imposes in return for this speed, and one kind of workload it therefore suits.

38.6 CORE For each allocation pattern, say which custom allocator (arena, pool, or none — keep general `malloc`) fits best, and why: (a) all objects allocated while parsing one request and all freed when the response is sent; (b) a stream of same-sized objects created and individually destroyed in no particular order over a long run; (c) a handful of long-lived objects of wildly varying sizes allocated at startup.

38.7 COST A program allocates N small objects, each costing c nanoseconds to allocate, then traverses them t times, each traversal costing m nanoseconds per node. (a) Write the total time in terms of N, c, t, m. (b) Using the chapter's numbers ($c{\approx}18$, $m{\approx}8$ contiguous or 177 scattered) with $t{=}10$, show that the layout choice dominates the allocator choice. (c) For what value of t would the per-call cost and the extra scattered-traversal cost be equal? Interpret the answer.

38.8 FADED *Worked, with the last step for you.* A colleague microbenchmarks their allocator by allocating a million objects and immediately freeing each one in a tight loop, sees ~18 ns/object, and concludes “allocation is cheap, no need for an arena.”
Step 1 (given): allocate-then-immediately-free hits the allocator's per-thread cache (`tcache`) fast path, so it is served from a thread-local free-list with no lock and no search.
Step 2 (given): in the real program the objects are long-lived and freed in no particular order, interleaved with other allocations.
Step 3 (given): so in the real program they are scattered across the heap, and later traversals miss cache per node.
Step 4 (you): Explain why the microbenchmark understates the real cost, what the colleague should measure instead, and which of the two costs the arena would actually fix here.

38.9 MIXED For each, say whether the dominant cost is the per-call allocation overhead or the locality of the resulting layout, and why: (a) a server that `malloc`s and frees millions of tiny request buffers per second but never traverses them in bulk; (b) a graph of ten million nodes, each separately `malloc`'d, walked repeatedly; (c) a parser that allocates thousands of AST nodes per file and walks the tree many times; (d) a program that allocates one 1 GiB buffer at startup and streams through it.

38.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "A faster `malloc` speeds up a pointer-chasing traversal over already-allocated nodes." (b) "An arena can free individual objects." (c) "The largest cost of dynamic allocation is usually the time inside `malloc`." (d) "An arena makes allocated objects contiguous, which helps later traversals." (e) "A microbenchmark that allocates then immediately frees each object faithfully represents allocation cost in a long-running program."

38.11 STRETCH Tie this chapter to the part. (a) Explain why "the cost of dynamic allocation" is really, for most programs, a Chapter 34 (locality) problem in disguise — and what that implies about the fix. (b) A profile shows 60% of runtime in one traversal loop over freshly built data structures; describe the checks — is the time in `malloc`? in cache misses? — you would run before changing anything. (c) Argue why "allocate fewer, more contiguous objects" is a better mental model than "find a faster allocator."

38.12 LAB On your own machine (a C toolchain): (a) Allocate N small objects one `malloc` at a time and free them, versus bump-allocating them from one arena block and freeing the block once; time both as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials and report the per-object costs. (b) Link N nodes (choose N so the working set exceeds your last-level cache, which you can read from `/sys/devices/system/cpu/cpu0/cache`) into a list once contiguously and once in a shuffled order, traverse each, and report the ns/node and the ratio. (c) State your toolchain version, CPU, last-level cache size, and allocator, and note these are wall-clock times (no hardware counters) if you lack `perf`.

Chapter 39 — Putting It Together: A Performance Workflow

Before you start: Chapter 33 (measure first — the discipline this chapter turns into a full workflow), Chapter 34 (locality — the cache ceiling), Chapter 36 (SIMD and the roofline — the compute and bandwidth ceilings), Chapter 37 (branch misprediction — the control ceiling), Chapter 38 (allocation — the ceiling hiding in your traversals). · The last six chapters each raised one ceiling; this one is the map — given a profile, which ceiling are you actually under, and how do you raise *that* one without wasting effort on the others.

BEFORE YOU READ ON

From memory, reaching back across the part: (1) Chapter 33 — why do you profile before optimizing rather than guess, and why report the minimum of several timed trials? (2) Chapter 36 — what is the roofline idea: what two ceilings cap a loop, and what decides which one binds? (3) *Predict:* a profile says one function is 80% of runtime. You have five tools in hand — better locality, better layout, SIMD, branchless code, a custom allocator. Before you reach for any of them, what is the *first* question you should answer about that function, and why can the wrong first move make things slower? Write your guess; §39.1 and §39.2 answer it.

Every chapter in this part measured one cost and raised one ceiling: cache misses (Chapters 34–35), compute throughput and memory bandwidth (Chapter 36), branch mispredictions (Chapter 37), and dynamic allocation (Chapter 38), all under the founding rule to measure first (Chapter 33). Taken singly, each is a technique; taken together, they are a **decision problem**, because a real program is under exactly one dominant ceiling at a time, and the whole art of performance engineering is finding which one and raising only that. This is not optional discipline — it is forced by arithmetic. A machine's speed is capped by whichever resource saturates first, so effort spent raising a ceiling you are not under buys nothing: vectorizing a memory-bound loop, de-branching a predictable branch, or replacing an allocator whose cost is really scattered traversal all fail for the same reason — they raise the wrong ceiling. This capstone assembles the part into one workflow. It proceeds in five steps: the loop that is the whole method — measure, find the dominant cost, identify its ceiling, raise that one, re-measure (§39.1); reading a profile to name the ceiling, with the part's symptoms in one table (§39.2); knowing when to stop, via Amdahl's law and the cost of premature optimization (§39.3); the traps of the part collected — the ways each ceiling is misdiagnosed (§39.4); and what to carry out of Performance Engineering (§39.5).

On the code in this chapter. This is a synthesis chapter and introduces no new benchmark; every number it cites was measured earlier in this part, in this environment (gcc 11.4.0 on an Intel Core i7-12700H, wall-clock `clock_gettime(CLOCK_MONOTONIC)` as the minimum of several trials, with the caches and allocator read from the machine as noted in each chapter). A hardware-counter profiler (`perf`) is *not* installed here, which is itself part of the lesson: the workflow below is written to run on wall-clock time plus the compiler's own reports (assembly, `-fopt-info-vec`) — the evidence you can always get —

and it names where percentage-level cache-miss and misprediction counters would sharpen a diagnosis if you have them. The published figures it references (pipeline-depth and misprediction penalties, Amdahl's law) are labelled as such and attributed; they are not measured here.

39.1 The workflow: find the ceiling, raise that one, re-measure

The entire method is a short loop, and its order is the point. **First, measure** — profile the real workload to find where time actually goes, and establish a baseline as the minimum of several trials (Chapter 33), because every later step is judged against it. **Second, find the dominant cost** — the one function, loop, or phase that is the largest share of runtime; Amdahl's law (§39.3) says nothing else is worth touching yet. **Third, identify the ceiling** that dominant cost is under: is it compute-bound (saturating the arithmetic units), memory-bound (waiting on cache misses or DRAM bandwidth), control-bound (mispredicting a branch), or allocation-bound (in `malloc/free` or the scattered memory they returned)? This is the crux, and §39.2 is a table for it. **Fourth, raise that ceiling** with the matching tool — locality and layout for cache misses (Chapters 34–35), SIMD for compute (Chapter 36), predictability or branchless code for mispredictions (Chapter 37), an arena or pool for allocation (Chapter 38) — and only that tool. **Fifth, re-measure** against the baseline: confirm the change moved the number, and then repeat, because raising one ceiling exposes the next. The ordering is not bureaucracy; it is what stops the two failure modes the part kept meeting. Skipping step one and optimizing by intuition rewrites cold code (Chapter 33's whole warning). Skipping step three — reaching for a favourite tool before naming the ceiling — is worse, because the tools are not interchangeable: SIMD does almost nothing for a bandwidth-bound loop (Chapter 36's 2.4× versus 13×), a branchless rewrite loses to a predictable branch (Chapter 37), and a faster allocator does nothing for a scattered traversal (Chapter 38). Each tool raises one specific ceiling, so you must name the ceiling first, and by measurement, not guess.

QUICK CHECK

State the five steps of the workflow in order. Which step is the one the part warns is most often skipped, and what goes wrong when you skip it? (Answer after the next section.)

39.2 Reading a profile: which ceiling are you under?

Naming the ceiling is a matter of matching a symptom to a cause, and the part supplies the symptoms. A loop that is **compute-bound** already touches data resident in cache and does substantial arithmetic per element; its throughput scales when you widen the arithmetic, so SIMD helps (the §36.2 kernel: 13× scalar to AVX2), and the tell is that vectorizing moves the number. A loop that is **memory-bound** stalls waiting on data: if it streams arrays larger than cache, it is *bandwidth*-bound (the §36.3 dot product: only 2.4× from SIMD, because it waits on DRAM), and if it chases pointers across scattered memory it is *latency*-bound (the §38.2 traversal: 177 versus 8 ns/node) — in both cases widening arithmetic does nothing and the fix is locality and layout, to move fewer bytes or touch them in order. A loop that is **control-bound** spends its time in mispredictions:

the tell is a data-dependent branch that swings with input order (the §37.2 loop: 15× sorted versus unsorted) — but only if the branch survived the compiler, which you confirm in the assembly before believing it. A phase that is **allocation-bound** shows time inside `malloc`/`free` (a per-call problem) or, more often, time in a traversal of freshly allocated data with a high miss rate (a locality problem in disguise, §38.4). The discipline is that these are distinguished by *measurement* — varying input size to find cache cliffs, timing scalar versus vectorized to see if compute binds, reading the assembly to see if a branch exists, checking whether time is in the allocator or in the walk of what it returned — not by staring at source, because the same C can be any of them depending on data sizes and what the compiler emitted. Where hardware counters are available (`perf`) they name the ceiling directly — cache-miss rate, misprediction rate, instructions-per-cycle — and are worth using; where they are not, as here, wall-clock experiments and the compiler's reports still resolve the question, which is the whole reason the part insisted the honest number is a measured one.

THE SAME IDEA THREE WAYS: RAISE THE CEILING YOU ARE UNDER, NOT THE ONE YOU KNOW BEST

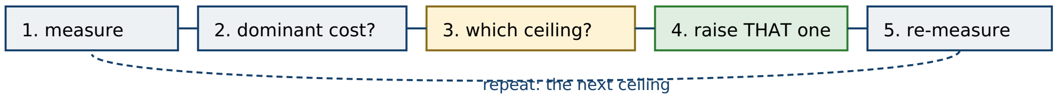
1. In words. A program runs at the speed of its slowest resource, so performance work is diagnosis before treatment: profile to find the dominant cost, identify which ceiling it is under (compute, cache, bandwidth, control, allocation), raise exactly that one, and re-measure. Reaching for a familiar tool before naming the ceiling raises the wrong roof and buys nothing — or loses.

2. As a picture. Figure 39.1: a flow — *measure → dominant cost? → which ceiling? → matching fix → re-measure → repeat* — beside a table mapping each symptom (scales with SIMD; stalls on a streamed array; stalls on a pointer chase; swings with branch order; time in `malloc` or in a walk of fresh data) to its ceiling and its one right tool.

3. As numbers/contrast. The same part, five ceilings, one machine: SIMD gave **13×** on a compute-bound kernel but only **2.4×** on a bandwidth-bound one (Chapter 36); a real branch swung **15×** on predictability (Chapter 37); a scattered traversal ran **22×** slower than a contiguous one (Chapter 38). Each number is large only when it matches the ceiling — the proof that naming the ceiling first is the whole game.

(Answer to the §39.1 quick check: the five steps are (1) measure and baseline, (2) find the dominant cost, (3) identify its ceiling, (4) raise that ceiling with the matching tool, (5) re-measure and repeat. The most-often-skipped step is (3), identifying the ceiling: skip it and you reach for a favourite tool — SIMD, branchless code, a new allocator — that may raise a ceiling you are not under, doing nothing on a memory-bound loop or even losing on a predictable branch, because the tools are not interchangeable.)

The workflow, and the symptom-to-ceiling table it turns on.



symptom	ceiling	the one right tool
throughput scales when you widen arithmetic (cache-resident, math-heavy)	compute	SIMD (Ch 36) — 13×
stalls streaming an array larger than cache	bandwidth	locality / layout (Ch 34–35)
stalls chasing pointers across scattered memory	latency	contiguity / arena (Ch 38) — 22×
time swings with branch order; a surviving data-dependent branch	control	predictable / branchless (Ch 37) — 15×
time in malloc/free, or in a walk of freshly allocated data	allocation	arena / pool (Ch 38) — 5×

Wrong first move: raise a ceiling you are not under → SIMD on a bandwidth-bound loop = 2.4×, not 13×.

Each large number in this part appeared only when the tool matched the ceiling. Diagnosis before treatment.

Figure 39.1: The performance workflow (top) and the symptom-to-ceiling table it turns on (bottom). A program runs at the speed of its slowest resource, so the method is diagnosis before treatment: profile, name the ceiling, raise *that* one, re-measure. The part's large speedups (13×, 15×, 22×) each appeared only when the tool matched the ceiling; the wrong first move raises a roof you are not under (§39.1–§39.2).

39.3 Knowing when to stop: Amdahl's law and the 5% part

The workflow's second step — optimize the dominant cost, nothing else yet — is not a preference; it is **Amdahl's law** (Amdahl, 1967). If a part of the program takes a fraction p of the runtime and you speed *that part* up by a factor s, the whole program speeds up by only $1 / ((1-p) + p/s)$. The consequence is brutal and clarifying: the overall speedup is bounded by $1 / (1-p)$ no matter how large s is, so a part that is 5% of runtime yields at most a $1 / 0.95 \approx 1.05\times$ overall *even if you make it infinitely fast*. This is the arithmetic behind the phrase the part has used repeatedly — “optimizing the 5% part” — and behind Knuth's older warning that *premature optimization is the root of all evil* (Knuth, 1974): effort spent on code that is not the dominant cost is capped at a rounding error before it begins, and it is not free — it adds complexity, risks bugs, and can block the compiler (a hand-written branchless loop that defeats the vectorizer, Chapter 37). So the discipline has two edges. One edge says *start* at the dominant cost, because that is where the ceiling of achievable speedup is highest. The other says *stop* when the remaining costs are small: once the dominant cost is raised and the profile flattens — no single part is a large share — Amdahl's law says the next change is bounded low, and the right engineering move is often to stop optimizing and spend the complexity budget elsewhere. Knowing when to stop is as much a part of the workflow as knowing where to start: the goal is not the fastest possible code but code fast enough that the dominant cost no longer dominates, reached by the fewest changes that measurably move the baseline.

> **QUICK CHECK**
>
> A function is 10% of runtime and you make it 100× faster. What is the overall speedup, roughly? What does Amdahl's law say is the ceiling on optimizing a part that is 5% of runtime, and what does that imply about where to start? (Answer below the communicate box.)

39.4 The traps of the part, in one place

Each chapter of this part ended with a trap, and collected they are one trap wearing five costumes: **acting on a model of the hardware instead of a measurement of it**. Chapter 33's trap was optimizing without profiling — rewriting code that a profile would have shown to be cold. Chapter 34's was assuming an algorithm with fewer operations is faster when a cache-hostile access pattern makes it slower — the model counted operations, the machine counted misses. Chapter 36's was counting vector lanes — “8 lanes, so 8×” — when the loop was bandwidth-bound and gained 2.4×, or compute-bound and gained more than 8× via FMA; the lane count is a model, the ceiling is measured. Chapter 37's was counting branches in the source and multiplying by a cycle penalty, against a branch the compiler had already if-converted away — the source is a model, the binary is the fact. Chapter 38's was blaming the allocator's speed for time that was really the scattered memory it returned — the call is a small model, the traversal's cache misses are the fact. The unifying lesson is that the machine has too many interacting layers — compiler, cache hierarchy, predictor, allocator, memory bus — for a mental model to reliably predict where time goes; the layers routinely conspire to make the intuitive answer wrong (the branch that was compiled away, the SIMD the memory bus ate, the algorithm the cache punished). The defense is the same in every case: before optimizing, get evidence of what the machine actually does — a profile, a timed experiment, the emitted assembly, the vectorization report — and after optimizing, re-measure to confirm the change helped. Every large number in this part came from measuring the machine; every trap came from trusting a model of it.

TRAP: OPTIMIZING TO A STORY — THE CHANGE THAT IS PLAUSIBLE, MEASURED AGAINST NOTHING

The final and most general trap is the one under all the others: shipping an optimization because it *sounds* right — fewer instructions, more lanes, no branch, a faster allocator — without a before-and-after measurement of the workload it is supposed to help. Every specific trap in this part is an instance: the vectorized memory-bound loop, the de-branched predictable branch, the swapped allocator on a scattered traversal, the rewritten cold function. Each is plausible; each is measured against nothing; each raises a ceiling the program was not under, so the profile does not move — and worse, the added complexity stays. Two defenses close the part. First, tie every optimization to a number: a profile that names the dominant cost before, and a re-measured baseline that shows the change moved it after — no number, no merge (Chapter 33). Second, respect Amdahl's law as a gate: if the part you are optimizing is not the dominant cost, the best case is a rounding error, so the change has to justify its complexity against a ceiling that is provably low. Optimizing to a story rather than a measurement is how fast-sounding code that is not actually faster — and is now harder to read — gets written; the whole part exists to replace the story with the number.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate opens a PR: "Rewrote the hot path — vectorized the inner loop with intrinsics, made the filter branchless, and swapped in a faster allocator. Should be a big speedup across the board." No before/after numbers are attached. Cover the paragraph and respond in four or five sentences, drawing on §39.1–§39.4.

Model answer. "Three real techniques, but I can't tell which one is actually helping without numbers — and they don't all raise the same ceiling, so some may be doing nothing or losing. Can you attach the profile that identified the hot path and a before/after on the representative workload? Specifically: is the inner loop compute-bound (then SIMD earns its keep) or memory-bound (then it's ~1× and the real fix is layout); did the branch even survive the old build, or did the compiler already if-convert it, in which case branchless can be a regression on predictable data; and is the allocator time in `malloc` itself or in traversing what it returned, because a faster allocator only helps the former. Let's land the measured win — the one or two changes the profile actually moves — and drop the rest, so we're not carrying complexity for speedups that aren't there." Asking which ceiling each change raises, and gating the merge on a measured before/after, is the senior register — and the whole part in one review.

(Answer to the §39.3 quick check: a function that is 10% of runtime made 100× faster gives an overall speedup of 1 / (0.90 + 0.10/100) ≈ 1.11× — the 90% you did not touch dominates. Amdahl's law caps optimizing a 5%-of-runtime part at 1 / 0.95 ≈ 1.05× even at infinite speedup, which is why you start at the dominant cost — the part with the highest ceiling on achievable speedup — and why optimizing a small part is bounded to a rounding error before you begin.)

39.5 What to carry forward

Performance engineering is a **diagnosis discipline**, and this part is one method repeated against five ceilings. The method: **measure** the real workload and baseline it (Chapter 33); find the **dominant cost**, because Amdahl's law caps everything else low (§39.3); **identify the ceiling** that cost is under — compute, cache, bandwidth, control, or allocation (§39.2); **raise that one** with its matching tool and only that tool — locality and layout for the memory ceilings (Chapters 34–35), SIMD for compute (Chapter 36), predictability or branchless code for control (Chapter 37), an arena or pool for allocation (Chapter 38); and **re-measure**, then repeat on the ceiling that surfaces next. The part's large numbers — 13× from SIMD, 15× from branch predictability, 22× from contiguity — were each large only because the tool matched the ceiling; the same tools against the wrong ceiling gave 2.4×, a regression, or nothing. That is the through-line to carry out: the machine has several ceilings, and engineering performance is finding, by measurement, which one you are actually under, and raising *that* one — never reasoning from a model of the hardware when you can get evidence of what it does, and never shipping an optimization that is not tied to a before-and-after number.

This closes Volume 1's turn toward the CPU and its memory. The part began by refusing to guess and ends with a workflow that never has to: given a profile, you can now name the ceiling and reach for the one tool that raises it. But every cost this part measured lived *inside* the machine — in caches, pipelines, vector units, and the heap — where the program runs alone against silicon. The next part crosses the boundary where the program meets the outside world: **I/O and networking**, where the costs are no longer cycles and cache lines but system calls, disk and network latency, and the blocking that stalls a whole thread while it waits — a new cost model, and a new set of "where does the time go" questions, with the same measure-first discipline carried across. The machine is understood; now it must talk to the world.

CAN YOU… WITHOUT LOOKING?

- State the five steps of the performance workflow in order, and why the order matters.
- Match each symptom (scales with SIMD, stalls on a stream, chases scattered pointers, swings with branch order, time in the allocator) to its ceiling and tool.
- State Amdahl's law and compute the ceiling on optimizing a 5%-of-runtime part.
- Explain why each tool in the part fails against the wrong ceiling, with an example.
- Name the single trap under all five chapters' traps, and the defense against it.
- Say when to *stop* optimizing, in Amdahl terms.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What are the five steps of the workflow, and which is most often skipped with what consequence? (§39.1)
2. **R2.** Give the symptom that identifies each of the compute, bandwidth, latency, control, and allocation ceilings. (§39.2)
3. **R3.** State Amdahl's law and the ceiling it puts on optimizing a 5%-of-runtime part. (§39.3)
4. **R4.** What single trap underlies all five chapters' traps, and what is the defense? (§39.4)
5. **R5.** Why did the part's tools give large speedups in some cases and near-nothing in others — what decides it? (§39.2, §39.5)

FURTHER READING

- **Gene M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *AFIPS '67 Spring Joint Computer Conference*, pp. 483–485 (1967), DOI 10.1145/1465482.1465560.** The origin of §39.3's law bounding overall speedup by the fraction you do not optimize — the arithmetic behind "start at the dominant cost." First author Gene Amdahl; DOI verified via the ACM Digital Library.
- **Donald E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys* 6(4), pp. 261–301 (1974), DOI 10.1145/356635.356640.** The source of "premature optimization is the root of all evil" (§39.3) — and its less-quoted second half, that the critical few percent are worth optimizing well. First author Donald Knuth; DOI verified via the ACM Digital Library.
- **Samuel Williams, Andrew Waterman & David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM* 52(4), 65–76 (2009), DOI 10.1145/1498765.1498785.** The compute-vs-bandwidth ceiling model of §39.2 — arithmetic intensity deciding which roof binds. First author Samuel Williams; DOI verified via the ACM Digital Library.
- **Brendan Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. (Addison-Wesley, 2020), ISBN 978-0-13-682015-4.** A practitioner's reference for the whole workflow of §39.1–§39.2 — methodologies (including the USE method), tools, and reading profiles across the stack. First author Brendan Gregg; edition and ISBN verified via the publisher.
- **John L. Hennessy & David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. (Morgan Kaufmann, 2017), ISBN 978-0-12-811905-1.** The architectural ground under every ceiling in this part — pipelines, caches, memory, and data-level parallelism. First author John Hennessy; edition and ISBN verified via the publisher.

Exercises

39.1 WARM-UP State the five steps of the performance workflow in order, in one line each.

39.2 WARM-UP Why must you identify the ceiling *before* choosing a tool? Give one example from the part of a tool that fails against the wrong ceiling.

39.3 WARM-UP For each symptom, name the ceiling: (a) throughput scales when you widen the arithmetic; (b) a loop stalls streaming an array far larger than cache; (c) time swings 15× with the sort order of the input; (d) a pointer-chasing walk over separately-allocated nodes is slow.

39.4 CORE Amdahl's law. (a) State it in terms of the optimized fraction p and the local speedup s. (b) A function is 20% of runtime; you make it 4× faster. Compute the overall speedup. (c) What is the maximum overall speedup from optimizing that same 20% function, no matter how fast you make it, and what does that imply about where to look next?

39.5 CORE A profile shows one loop at 70% of runtime, chasing pointers through a graph of separately-`malloc`'d nodes, over a working set far larger than L3. (a) Which ceiling is it under, and how would you confirm? (b) Which tool from the part raises that ceiling, and which tools would *not* help here? (c) After the fix the same loop is now 30% of runtime and the profile is flat — what does Amdahl's law advise?

39.6 CORE For each, name the ceiling and the one tool, and say which of the part's numbers it echoes: (a) a cache-resident, math-heavy kernel with independent lanes; (b) a 64 MiB-streaming dot product; (c) a data-dependent branch that survived compilation and swings with input order; (d) a phase spending its time in a traversal of freshly built, scattered data structures.

39.7 COST A program has two hot parts: part A is 60% of runtime, part B is 30%. You can raise either ceiling for the same effort, achieving a 5× local speedup on whichever you pick. (a) Compute the overall speedup from optimizing A alone, and from B alone. (b) Which should you do first, and why does Amdahl's law make the choice obvious? (c) After doing the better one, recompute the fractions and say whether the other is now worth doing.

39.8 FADED *Worked, with the last step for you.* A colleague ships a PR that hand-vectorizes an inner loop with AVX2 intrinsics and reports "no measurable speedup, but the code is more advanced."
Step 1 (given): the loop streams a pair of arrays far larger than last-level cache.
Step 2 (given): its arithmetic intensity is tiny (about one add per two loads), so it is bandwidth-bound.
Step 3 (given): SIMD raises the compute ceiling, not the bandwidth ceiling.
Step 4 (you): Explain why the speedup is ~1× (not a bug), what the profile would have shown before the change, and why the PR should be dropped despite the code being "more advanced."

39.9 MIXED Each of these is a version of the same underlying trap. Name the trap they share, and for each say what evidence would have caught it: (a) rewriting a function that turns out to be 2% of runtime; (b) vectorizing a bandwidth-bound loop; (c) de-branching a branch the compiler already if-converted; (d) swapping in a faster `malloc` to speed up a scattered traversal.

39.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "The fastest program is the one with the fewest instructions." (b) "Optimizing a part that is 5% of runtime can, at best, give about 1.05× overall." (c) "SIMD, branchless code, and custom allocators are interchangeable ways to make code faster." (d) "You should re-measure after an optimization even if the change obviously should help." (e) "A program is usually under several ceilings at once, so you should raise them all together."

39.11 STRETCH Synthesize the part. (a) Explain why every chapter's trap is the same trap — reasoning from a model of the hardware instead of measuring it — and give the one defense common to all of them. (b) The part reported speedups of 13×, 15×, and 22×, and also 2.4× and a regression, using the *same* techniques — explain in one paragraph what decides which you get. (c) Argue why “find the ceiling you're under and raise that one” is a more durable skill than knowing any individual technique.

39.12 LAB On your own machine, run the full workflow once end to end. (a) Take a small program of your own with a plausible hot loop; profile or time it to find the dominant cost and baseline it as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials. (b) Identify the ceiling it is under using the §39.2 tests (vary input size for cache cliffs; time scalar vs vectorized for compute; read the assembly for a surviving branch; check whether time is in the allocator or in a walk of its output), raise that one ceiling with the matching tool, and re-measure. (c) Report the before/after, the ceiling you identified and how, and — honestly — whether the change moved the number or taught you the ceiling was elsewhere; state your toolchain, CPU, and that these are wall-clock times if you lack `perf`.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART VII

I/O & Networking, Low-Level

Files, sockets, blocking vs async, epoll/io_uring, zero-copy — the systems substrate under every data pipeline and service.

Chapter 40 — The I/O Stack: System Calls, Buffering, and the Page Cache

Before you start: Chapter 13 (the system-call boundary — why crossing into the kernel costs more than a function call), Chapter 3 (the memory hierarchy — a fast layer in front of a slow one, which is exactly what the page cache is), Chapter 17 (demand paging — the same page-fault machinery the page cache rides on), Chapter 33 (measure first — carried across the boundary into I/O). · The last part measured costs *inside* the machine — cycles, cache lines, the heap. This part crosses the boundary to where the program meets storage and the network, and the first cost it meets is the one this chapter is about: the system call, and the two layers of buffering that hide it.

BEFORE YOU READ ON

From memory, reaching across the earlier parts: (1) Chapter 13 — what is a *system call*, and why does it cost more than an ordinary function call: what has to happen at the boundary? (2) Chapter 3 — what is a cache, and why does putting a fast layer in front of a slow one speed up *repeated* access to the same data? (3) *Predict:* you copy a 4 MiB file two ways — once reading and writing one byte at a time with the `read/write` system calls, and once through a 64 KiB buffer. How large is the gap — 2×, 20×, more? And separately: you read a 256 MiB file once when it is not in memory and once when it already is. How large is *that* gap? Write both guesses; §40.2 and §40.3 answer them.

Reading a file looks like the simplest thing a program does — call `read`, bytes appear — but underneath that call is a stack of layers, and where your time goes depends on which of them you hit. Every ordinary file access is a **system call** (Chapter 13): `read` and `write` are not library routines that touch the disk directly but requests to the kernel, which owns the hardware. Crossing into the kernel and back has a fixed cost — a mode switch, argument checking, the kernel's own work — that is paid *per call* regardless of how many bytes the call moves, so the first rule of I/O performance is that the number of system calls, not the number of bytes, often decides the speed. That is why **buffering** exists, and it exists twice: the C library buffers your bytes in user space so that many small `fputc`s become one `write`, and the kernel buffers file data in a region of DRAM called the **page cache** so that a read already in memory never touches the disk at all. This chapter is the map of that stack. It proceeds in five steps: the layers a `read` passes through, from the call to the device (§40.1); why buffering is the difference between fast and unusably slow, measured (§40.2); the page cache — DRAM standing in front of the disk — and what it does to your numbers (§40.3); what this means for how you do and how you *measure* I/O, with the trap that catches everyone (§40.4); and what to carry forward (§40.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35, 4 KiB pages) against an `ext4` filesystem, timed with `clock_gettime(CLOCK_MONOTONIC)` and reported as the minimum of several trials (Chapter 33). Times are wall-clock and illustrate the *shape* of each cost, not a storage benchmark. A hardware-counter profiler (`perf`) is *not* installed here, so the evidence is wall-clock time plus what can be read from the system — the page size from `getconf`, cache eviction forced with `posix_fadvise`. The “cold” read below is made cold by asking the kernel to drop this file's pages with `POSIX_FADV_DONTNEED`, not by a privileged cache flush; on this machine the backing store is a virtual disk, so the cold number reflects *this* storage and would differ on bare-metal SSD or spinning disk — published device latencies, where quoted, are labelled as such.

40.1 The I/O stack: from read() to the device

When a program calls `read(fd, buf, n)`, the request descends through several layers before any bytes come back, and knowing them tells you where a given cost lives. At the top is the **file descriptor**: a small integer that indexes a per-process table, each entry pointing at an open-file description that holds the current offset and a reference to the underlying object (Chapter 13). The call traps into the kernel and reaches the **virtual filesystem** (VFS), the abstraction layer that lets one `read` work uniformly across `ext4`, a pipe, a socket, or a device — “everything is a file” is this layer keeping its promise. For a regular file the VFS asks the **page cache** whether the requested bytes are already resident in DRAM; if they are, they are copied straight to your buffer and the call returns without any device I/O at all. If they are not, the request descends to the **filesystem** proper, which maps the file offset to physical block numbers, and then to the **block layer**, which queues and schedules the request to the **device driver** and the hardware — where the real latency lives, milliseconds for a spinning disk, tens of microseconds for an SSD, against tens of *nanoseconds* to reach DRAM. Two facts fall out of this picture and organize the rest of the chapter. First, the cost of the *call itself* — the trap, the VFS dispatch, the copy into your buffer — is paid every time regardless of size, so making the call move more bytes amortizes it; that is §40.2. Second, the page cache means a “disk read” is usually not a disk read: the same file read twice hits DRAM the second time, so the number you measure depends entirely on whether the data was resident, which is §40.3. The layers are the same for `write` in reverse, with one twist that Chapter 41 turns into its whole subject: a `write` that reaches the page cache returns *before* the data is on the device.

> **QUICK CHECK**
>
> Name the layers a `read` of a regular file passes through, from the file descriptor down to the device. At which layer does a read return *without* touching the disk, and why does that make “how long does a read take” a question with two very different answers? (Answer after the next section.)

40.2 Why buffering matters: the syscall is the cost

The cheapest way to see the cost of the system call is to pay it as many times as possible. Copying a 4 MiB file one byte per `read` and one byte per `write` — two system calls for every byte — against copying it through a single 64 KiB buffer, so each call moves 65,536 bytes, runs in this environment (gcc 11.4.0, minimum of several trials):

```
copy a 4 MiB file:
                                      total       per byte      throughput
per-byte read()/write() (2 syscalls/byte):  6.73 s     1604 ns/byte    0.59 MiB/s
buffered 64 KiB read()/write()           :  0.0022 s    0.53 ns/byte   1812 MiB/s
stdio getc/putc (libc buffers ~4 KiB)    :  0.078 s     18.6 ns/byte   51 MiB/s
                                            buffered vs per-byte: ~3000x faster
```

The per-byte version is not a little slower — it is about **3000× slower**, and the reason is entirely the system calls. Each byte costs two round trips into the kernel, and on this machine a trivial system call is on the order of a few hundred nanoseconds (measured directly in Chapter 42), so at ~1600 ns/byte the copy is doing almost nothing *but* crossing the boundary. The buffered version pays the same per-call cost but amortizes it over 65,536 bytes, dropping the per-byte cost below a nanosecond and running at memory speed. Between the two sits **stdio**: `getc/putc` read and write one character at a time in your source, but the C library holds a buffer (about 4 KiB by default) and issues a real `read/write` only when it fills or drains — so a million `getc`s become a few hundred system calls, and the 18.6 ns/byte here is the cost of the per-character function calls and library bookkeeping, not of the syscall, which stdio has already amortized. This is the crux of I/O performance and it is worth stating plainly: **there are two independent buffers in the stack** — the kernel's page cache, which you get for free, and the user-space buffer (stdio's, or your own), which turns many small logical operations into few large system calls. The per-byte disaster above is what happens when you defeat the second one by calling `read/write` yourself with a size of 1. The fix is never faster hardware; it is a bigger buffer, so the fixed per-call cost is spread across more bytes (Kerrisk, ch.13; Bryant & O'Hallaron, ch.10).

QUICK CHECK

Why is copying a file one byte per `read/write` about 3000× slower than through a 64 KiB buffer, when both move the same bytes? Where does stdio's `getc/putc` sit between the two, and what is it buffering that the raw per-byte version is not? (Answer below the communicate box.)

40.3 The page cache: DRAM in front of the disk

The second buffer is the kernel's, and it changes the meaning of every I/O number you will ever measure. The **page cache** is a region of otherwise-free DRAM in which the kernel keeps recently used file pages: a `read` whose bytes are already there is served from memory with a copy and no device access, and a `write` lands in the cache (marking the page *dirty*) and is flushed to the device later by kernel writeback threads (Love, ch.16). It is a cache in exactly the Chapter 3 sense — a fast layer standing in front of a slow one, betting that data used once will be used again — and its effect is large. Reading a 256 MiB file once when its pages have been evicted from the cache, and once when they are resident, runs here:

```
read a 256 MiB file, summing bytes:
cold (pages evicted with POSIX_FADV_DONTNEED): ~0.59-0.68 s     ~375-440 MiB/s   (goes to the
device)
warm (pages resident in the page cache)      :  0.047 s        ~5450 MiB/s      (a copy from
DRAM)
                                              cold/warm: ~13x on this machine
```

Same code, same file, same bytes summed — and the warm read is about **13× faster** because it never leaves DRAM, while the cold read waits on the storage device. The gap is this machine's; on a spinning disk it would be far larger (a seek is milliseconds), on a fast NVMe SSD smaller, but the *shape* is universal: the first touch pays the device, later touches pay only a memory copy. Two behaviours of the cache are worth knowing because they shape your numbers. The kernel does **readahead** — on a sequential read it fetches pages ahead of your position, so a streaming scan overlaps device latency with your computation and runs faster than random access would. And on the write side it does **write-back**, not write-through: `write` returns as soon as the bytes are in the cache, so writes appear far faster than the device could sustain, with the real device cost paid asynchronously later — which is convenient for speed and, as Chapter 41 will show, a genuine hazard for durability, because data that a `write` call has "accepted" is not yet on disk. The page cache is also why free memory is never wasted on a busy machine: the kernel fills it with file pages and evicts them under pressure, so a long-running server's I/O gets faster as its working set warms.

THE SAME IDEA THREE WAYS: THE SYSCALL IS THE COST, AND THE CACHE HIDES THE DISK

1. In words. A file access descends through the file descriptor, the VFS, the page cache, the filesystem, the block layer, and the device. Two costs dominate. The *system call* is a fixed per-call price, so moving one byte per call is catastrophic and moving 64 KiB per call is nearly free per byte — buffer to amortize it. The *page cache* is DRAM in front of the device, so a read that hits it never touches the disk, and a read that misses pays the device — the same code times differently depending only on what is resident.

2. As a picture. Figure 40.1: the stack as a vertical column — `read()` → file descriptor → VFS → page cache (hit: return from DRAM) → filesystem → block layer → device (miss) — with two cost bars beside it: per-byte vs 64 KiB buffering, and cold vs warm reads.

3. As numbers/contrast. Same machine, one file. Per-byte `read`/`write`: ~1604 ns/byte (0.59 MiB/s); a 64 KiB buffer: ~0.53 ns/byte (1812 MiB/s) — **~3000×** (§40.2). Cold read ~400 MiB/s; warm read ~5450 MiB/s — **~13×** (§40.3). Neither gap is the disk getting faster; one is the syscall amortized, the other is the device skipped.

(Answer to the §40.1 quick check: a `read` passes through the file descriptor and its open-file description (holding the offset), the VFS, the page cache, and — on a miss — the filesystem, the block layer, the device driver, and the hardware. It returns without touching the disk at the page cache, when the requested pages are already resident in DRAM; that is why "how long does a read take" has two answers — tens of nanoseconds per byte for a cache hit, and a device access of microseconds to milliseconds for a miss — and why an I/O measurement is meaningless unless you say which case it is.)

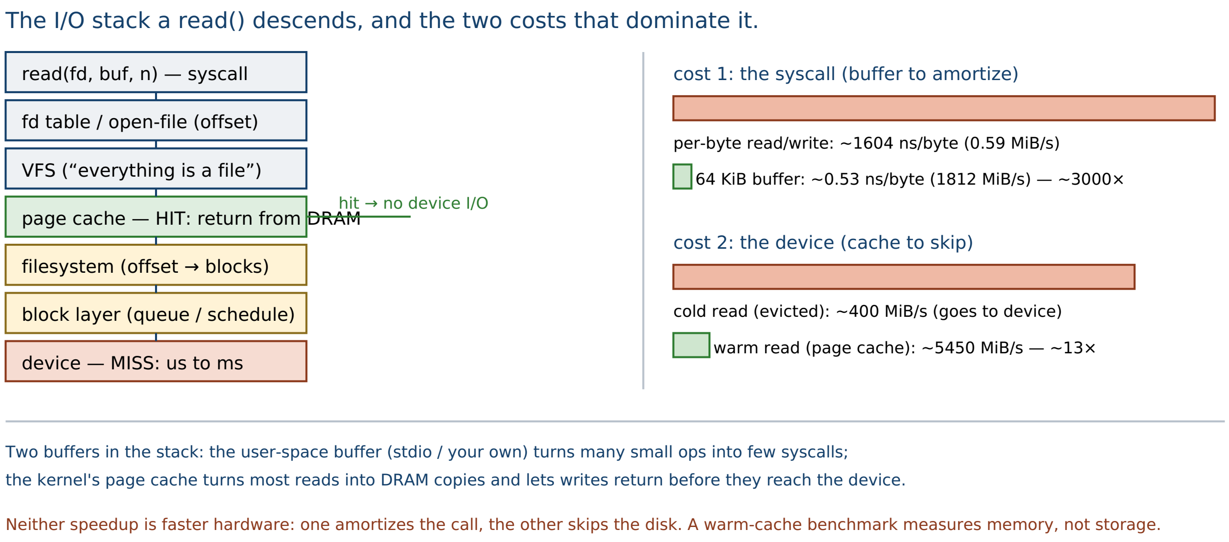


Figure 40.1: The I/O stack (left) and its two dominant costs (right). A `read` descends from the file descriptor through the VFS to the page cache, returning from DRAM on a hit and reaching the device only on a miss. The *system call* is a fixed per-call cost — buffering amortizes it (~3000×, §40.2) — and the *page cache* is DRAM in front of the device — hitting it skips the disk (~13×, §40.3). Wall-clock, illustrative; cold reads reflect this machine's virtual disk.

40.4 Doing and measuring I/O: buffer sizes, short counts, and the warm-cache trap

Three practical facts follow from the stack, and each is a place engineers go wrong. First, **buffer size has a sweet spot, not a maximum**: the per-byte cost falls steeply as the buffer grows from 1 byte toward the tens of kilobytes that amortize the syscall and match the kernel's readahead, then flattens — a 64 KiB buffer already ran at memory speed above, and a 64 MiB one would not be meaningfully faster while wasting cache and memory. The right size is “large enough that the syscall is amortized,” typically tens to hundreds of KiB, not “as large as possible.” Second, **`read` and `write` may move fewer bytes than you asked**: a `read` can return a *short count* at end-of-file or when interrupted, and a `write` to a pipe or socket can accept only part of your buffer, so correct I/O code loops until the requested count is done or an error occurs — the buffered copy above does exactly this, advancing a pointer through the buffer on each partial `write`. Code that assumes one `read` returns everything, or one `write` consumes everything, is not merely slow but wrong on the inputs where it matters. Third, and most damaging to your numbers, is the **page cache in your measurements**: run an I/O benchmark twice and the second run is served from the warm cache, so it reports DRAM throughput (the ~5450 MiB/s above) while presenting itself as a disk number. This is the trap the “On the code” note guards against — the cold figures here required explicitly evicting the file's pages first. As always the honest number is the one that names its conditions (Chapter 33): a storage benchmark must state whether it ran cold or warm, because those are different systems, and a warm-cache time is a measurement of memory wearing a disk's name. Where hardware counters exist, they let you see cache-hit rates directly; here, the discipline is to control the cache state yourself and to report which state you measured.

TRAP: BENCHMARKING I/O AGAINST A WARM CACHE — MEASURING DRAM, REPORTING IT AS DISK

The seductive mistake is to time a read or a copy, see a big number, and believe it describes your storage — when the file was already in the page cache from a previous run or from the write that created it, so what you measured was a copy from DRAM. On this machine the same read is ~13× faster warm than cold; on a spinning disk the gap can be hundreds of times, so a warm-cache benchmark can overstate real throughput by orders of magnitude and send you optimizing a bottleneck that will not exist in production, where the data arrives cold. Two defenses. First, *control the cache state*: to measure cold storage, evict the file's pages first — `posix_fadvise` with `POSIX_FADV_DONTNEED`, or on a machine you administer a cache drop — and to measure warm-cache behaviour, pre-warm deliberately; either is fine, but you must know and state which. Second, *report the conditions*: every I/O number should say cold or warm, the buffer size, and the filesystem and device, because those determine it far more than the code does. A throughput figure with no cache state is not a measurement of anything — it is the “optimizing to a story” trap of the last part (Chapter 39) in an I/O costume: a plausible number measured against an uncontrolled variable.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: “Our log processor reads the file a character at a time and it is painfully slow — I think we are disk-bound, so let us put it on faster SSDs and add a few reader threads.” Cover the paragraph and respond in four or five sentences, drawing on §40.1–§40.4.

Model answer. “I do not think it is the disk — a character-at-a-time reader is spending its time in system calls, not waiting on storage. Every `getc` or one-byte `read` is a trip across the kernel boundary, and on our numbers per-byte I/O runs about 3000× slower than the same copy through a 64 KiB buffer, which is nearly free per byte. Faster SSDs and more threads will not help, because the bottleneck is the call frequency, not device bandwidth or parallelism — we would just be making more syscalls in parallel. The fix is one line of buffering: read into a large buffer and parse from memory, or use buffered stdio instead of raw one-byte calls. Let us confirm with a quick before/after — buffer the reads and re-time — and I expect the ‘disk’ problem to vanish without touching the hardware.” Naming the syscall as the cost, and reaching for a buffer before hardware or threads, is the senior register.

(Answer to the §40.2 quick check: per-byte I/O is ~3000× slower because each byte pays two system calls, and the fixed per-call cost — the trap into the kernel and back — dwarfs the work of moving one byte; the 64 KiB buffer pays the same per-call cost once per 65,536 bytes, amortizing it to under a nanosecond each. Stdio's `getc/putc` *sit in between: they present a per-character interface but buffer in user space (~4 KiB), so they issue few real system calls — what they buffer is exactly what the raw per-byte version does not, which is why they run ~86× faster than it while still paying per-character function-call overhead.)*

40.5 What to carry forward

An I/O operation is a **descent through layers**, and its cost depends on which layer answers. A file access is a **system call** (Chapter 13) into the kernel's VFS, and the call has a fixed per-invocation price paid regardless of size — so the dominant lever in I/O performance is **buffering**, amortizing that price over many bytes. Measured here, per-byte `read/write` ran ~1604 ns/byte against ~0.53 ns/byte through a 64 KiB buffer — about **3000×** (§40.2) — and stdio sits between by buffering in user space so that many small operations become few syscalls. Below the VFS sits the **page cache**, DRAM standing in front of the device (Chapter 3's caching idea, applied to storage): a read that hits it is a memory copy, a read that misses pays the device, and here a warm read ran ~13× faster than a cold one (§40.3) — the same code timing differently based only on what was resident. The cache also does readahead for sequential reads and write-back for writes, which makes writes look instant. From all of this the working discipline: buffer your I/O to amortize the syscall, handle short counts by looping, and — above all — control and report the cache state when you measure, because a warm-cache number is a measurement of memory, not storage (§40.4).

This is the same measure-first discipline the last part built, now carried across the boundary into I/O, where the cost model is new — syscalls and device latency instead of cycles and cache lines — but the method is unchanged. One feature of the page cache, though, is a loose thread this chapter deliberately left hanging: a `write` returns as soon as the bytes reach the cache, *before* they are on the device, which is wonderful for speed and a real problem the moment you care whether the data survives a crash. That is the subject of the next chapter, Chapter 41: files and durability — how to force data to stable storage with `fsync` and what it costs, how `O_DIRECT` bypasses the cache, and how memory-mapped I/O maps file pages straight into your address space. The stack is mapped; now we make it durable, and pay for it.

CAN YOU… WITHOUT LOOKING?

- Name the layers a `read` of a regular file descends, and where a cache hit returns without device I/O.
- Explain why per-byte `read/write` is ~3000× slower than a 64 KiB buffer, with the order of magnitude.
- Distinguish the two buffers in the stack — user-space (stdio) and the kernel's page cache — and what each does.
- State what the page cache is and why a warm read is much faster than a cold one.
- Explain why `read/write` can move fewer bytes than requested, and what correct code does about it.
- Say why a warm-cache benchmark is a misleading measurement of storage, and how to run a cold one.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What layers does a `read` pass through, and at which one can it return without touching the device? (§40.1)
2. **R2.** Why is per-byte I/O ~3000× slower than buffered I/O, and where does stdio fit? (§40.2)
3. **R3.** What is the page cache, and roughly how much faster was a warm read than a cold one here, and why? (§40.3)
4. **R4.** Why can `read/write` return a short count, and what must correct code do? (§40.4)
5. **R5.** Why is a warm-cache benchmark a misleading storage number, and how do you measure a cold read? (§40.4)

FURTHER READING

- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2015), ISBN 978-0-13-409266-9.** Chapter 10 ("System-Level I/O") is the source for §40.1–§40.2 — Unix I/O, file descriptors, the difference between unbuffered system-call I/O and buffered stdio, and short counts. First author Randal Bryant; edition and ISBN verified via the publisher.
- **Michael Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), ISBN 978-1-59327-220-3.** Chapter 4 ("File I/O") and Chapter 13 ("File I/O Buffering") are the detailed reference for this chapter — the `read/write` system calls, the two levels of buffering (stdio and the kernel), and how buffer size affects performance. First author Michael Kerrisk; edition and ISBN verified via the publisher.
- **Robert Love, *Linux Kernel Development*, 3rd ed. (Addison-Wesley, 2010), ISBN 978-0-672-32946-3.** Chapter 16 ("The Page Cache and Page Writeback") is the source for §40.3 — how the kernel caches file pages in DRAM, readahead, and the write-back of dirty pages. First author Robert Love; edition and ISBN verified via the publisher.
- **Remzi H. Arpaci-Dusseau & Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (Arpaci-Dusseau Books, 2018).** The "Persistence" chapters — especially "I/O Devices" (ch.36) — ground §40.1's device layer: how a request reaches the hardware, interrupts, and DMA. Freely available online; first author Remzi Arpaci-Dusseau; cited by author and title (an open-access book carrying no ISBN for the online edition).

Exercises

40.1 WARM-UP List, in order, the layers a `read` of a regular file passes through from the file descriptor down to the device, and mark the one at which a cache hit can return without any device I/O.

40.2 WARM-UP In two sentences, explain why copying a file one byte per `read/write` is roughly 3000× slower than copying it through a 64 KiB buffer, when both move the same number of bytes.

40.3 WARM-UP There are two distinct buffers in the I/O stack. Name each, say who owns it, and say what problem each one solves.

40.4 CORE The page cache. (a) What is it, physically, and what does it hold? (b) Explain why the same `read` ran ~13× faster warm than cold here. (c) What is *readahead*, and why does it make a sequential scan faster than random access over the same file?

40.5 CORE Short counts. (a) Give two situations in which `read` returns fewer bytes than requested, and one in which `write` does. (b) Sketch, in pseudocode, the loop that correctly writes an entire buffer despite partial `write`s. (c) Why is code that assumes one `read` returns the whole file not just slow but incorrect?

40.6 CORE Buffer size. Given that per-byte I/O was ~1604 ns/byte and a 64 KiB buffer was ~0.53 ns/byte, explain (a) why the per-byte cost falls steeply as the buffer grows from 1 byte to tens of KiB, (b) why it then flattens rather than continuing to fall, and (c) what this implies about choosing "as large as possible" versus "large enough."

40.7 COST A program issues k system calls to move a total of B bytes, each call costing a fixed s nanoseconds plus b nanoseconds per byte moved. (a) Write the total time in terms of k, B, s, b. (b) With per-byte I/O, $k = B$; with a buffer of size W, $k = B/W$. Show how the total time depends on W, and identify the W beyond which increasing it barely helps.

40.8 FADED *Worked, with the last step for you.* A colleague benchmarks a file reader, gets 5 GiB/s, and reports "our storage does 5 GiB/s."
Step 1 (given): the benchmark reads the same file the test harness just wrote.
Step 2 (given): the write left every page of the file resident in the page cache.
Step 3 (given): so the read was served entirely from DRAM, never reaching the device.
Step 4 (you): Explain why 5 GiB/s is a memory number, not a storage number; what the colleague should do to measure cold-storage throughput; and why the difference could be an order of magnitude in production.

40.9 MIXED For each, say whether the dominant cost is *system-call frequency* or *device latency*, and how you would confirm: (a) a parser that calls `getc` in a tight loop over a 1 GiB file already in cache; (b) a program that opens ten thousand small files scattered across a cold disk and reads each once; (c) a copy that reads and writes through a 1 MiB buffer between two files on the same warm filesystem; (d) a logger that does a one-byte `write` per event.

40.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "`read` always returns exactly the number of bytes you asked for unless the file ends." (b) "A `write` that returns success means the data is on the disk." (c) "stdio's `getc` issues one system call per character." (d) "A read served from the page cache does not touch the storage device." (e) "The bigger the I/O buffer, the faster, without limit."

40.11 STRETCH Tie the chapter together. (a) Explain why both of this chapter's speedups — buffering (~3000×) and the warm cache (~13×) — are described as "not faster hardware," and what each one actually removes. (b) A server's I/O throughput improves over the first few minutes of running and then stabilizes; explain this in terms of the page cache warming. (c) Argue why "how many system calls, and is the data resident" is a better first question about I/O performance than "how fast is the disk."

40.12 LAB On your own machine (a C toolchain): (a) Copy a several-MiB file one byte per `read`/`write` and again through a 64 KiB buffer; time both as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials and report the per-byte cost and the ratio. (b) Read a large file twice — once after evicting its pages (`posix_fadvise` with `POSIX_FADV_DONTNEED`, or a cache drop if you administer the machine) and once warm — and report cold vs warm throughput. (c) State your toolchain, CPU, filesystem, and buffer size, and note that these are wall-clock times (no hardware counters) if you lack `perf`, and which reads were cold versus warm.

Chapter 41 — Files and Durability: fsync, Direct I/O, and Memory Mapping

Before you start: Chapter 40 (the page cache and write-back — why a `write` returns before the data is on the device), Chapter 13 (the system-call boundary), Chapter 4 (virtual memory and address translation — the machinery `mmap` reuses), Chapter 17 (demand paging — a mapped file page is faulted in on first touch). · The last chapter mapped the I/O stack and left one thread hanging: a `write` reaches only the page cache, not the disk. This chapter pulls that thread — how you force data to stable storage and what it costs, how to bypass the cache, and how to map a file into memory.

BEFORE YOU READ ON

From memory: (1) Chapter 40 — when a program calls `write` and it returns success, where is the data, and when does it actually reach the storage device? (2) Chapter 4 and Chapter 17 — what does the MMU do on a load to an address whose page is not resident, and what is *demand paging*? (3) *Predict:* you append two thousand small records to a file. In one version you call `fsync` (or `fdatasync`) after every record to make it durable; in the other you do not. How much slower is the durable version — 2×, 100×, more? And separately: you sum every 64th byte of a large cached file, once by `read`-ing it into a buffer and once by `mmap`-ing it and reading memory directly. Which is faster, and why? Write your guesses; §41.2 and §41.4 answer them.

A `write` that returns success has not written anything to your disk. As Chapter 40 showed, it copied your bytes into the kernel's page cache and marked the page dirty; the kernel flushes dirty pages to the device on its own schedule, seconds later, so a crash or power loss in the interval loses data the program believes it saved. For a scratch file this is exactly the behaviour you want — it makes writes fast. For anything whose survival matters — a database's log, a user's document, a financial record — it is a correctness problem, and closing it requires an explicit, expensive act: telling the kernel to force the data all the way to stable storage and to wait until it is there. That act is `fsync`, and its cost is the central number of this chapter, because it is what separates “fast” from “durable” and drives the design of every storage system in Volume 2. Around it sit two more tools for controlling how file data moves: `O_DIRECT`, which bypasses the page cache entirely for programs that manage their own, and memory-mapped I/O, which maps a file into your address space so you read and write it with ordinary loads and stores. This chapter proceeds in five steps: why a `write` is not durable and what a crash can lose (§41.1); `fsync`, and the price of durability, measured (§41.2); bypassing the cache with `O_DIRECT` (§41.3); memory-mapped I/O and when it beats `read`/`write` (§41.4); and what to carry forward (§41.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35, 4 KiB pages) against an `ext4` filesystem, timed with `clock_gettime(CLOCK_MONOTONIC)` as the minimum of several trials (Chapter 33). Times are wall-clock. A hardware-counter profiler (`perf`) is *not* installed here. The storage on this machine is a *virtual disk*, so the absolute `fsync` cost below reflects this device's flush path and is not a portable constant — published flush latencies for real SSDs and spinning disks, quoted for comparison, are labelled as such. The `fsync` number is stable across runs and shows the *shape* of the durability cost; the `mmap`-vs-`read` comparison was deliberately reported with its measured variance, because the two are close and the winner depends on the access pattern and page-fault behaviour rather than on a fixed ratio.

41.1 A write() is not on disk: the durability gap

The page cache's write-back policy (Chapter 40) is the source of both the speed and the hazard. When you call `write`, the kernel copies your bytes into a page in the cache, marks that page **dirty**, and returns — it does *not* wait for the device. Dirty pages are flushed later by kernel writeback threads, governed by tunables (how many dirty pages may accumulate, how old a dirty page may get) that typically let data sit in memory for seconds before it reaches stable storage. In that window the data exists only in volatile DRAM, so a power failure, a kernel panic, or a hard reset loses it, even though every `write` call returned success and the program moved on believing the data saved. This is not a bug; it is the deliberate trade that makes the §40.3 write-back cache fast, and for most files it is correct — a temporary file, a rebuildable cache, a log you can afford to lose the tail of. The problem is specifically **durability**: the guarantee that once an operation is acknowledged, its effect survives a crash. A plain `write` does not provide it, and no amount of retrying or checking the return value adds it, because the return value only tells you the kernel accepted the bytes into the cache, not that the device has them. Worse, the order in which dirty pages reach the disk is not the order you wrote them, so after a crash a file can be left in a state that never existed in the program's logic — the last record half-present, or metadata updated but data not — which is why crash-consistency is a genuine and studied difficulty (Pillai et al., OSDI 2014). Everything in the next section exists to close this gap on demand, for the specific data whose survival you are willing to pay for.

> **QUICK CHECK**
>
> After a `write` returns success, where exactly is the data, and what event would lose it? Why does checking the return value of `write` not tell you the data is durable? (Answer after the next section.)

41.2 fsync: the price of durability

The system call that closes the gap is `fsync(fd)` — and its variant `fdatasync(fd)` — which tells the kernel to flush all dirty pages of that file to the storage device and to wait until the device confirms they are stable before returning (`fdatasync` skips non-essential metadata like the modification time, so it can avoid an extra metadata write). It converts the deferred, asynchronous write-back into a synchronous, durable barrier, and the wait is the cost. Appending two thousand 256-byte records to a file, once with no sync and once calling `fdatasync` after every record, runs here:

```
append 2,000 records of 256 bytes:
                              per record      throughput
write only (no sync)        :   ~0.9 us       ~1,100,000 records/s   (lands in the page cache)
write + fdatasync each      :   ~8.6 ms       ~116 records/s         (waits for the device to
flush)
                              durability tax: ~9000x slower per record
```

The difference is not marginal — it is about **9000× per record**, because a plain `write` returns after a memory copy (~0.9 µs) while `fdatasync` must push the data through the block layer to the device and wait for the device to report it durably stored, which on this machine's virtual disk takes about 8.6 ms — a real device-flush round trip, not a CPU cost. That absolute number is this device's; a fast NVMe SSD with a power-loss-protected cache can flush in tens to hundreds of microseconds, a spinning disk takes on the order of ten milliseconds (a rotation plus a seek) — published figures, cited for scale — but on every device the flush is *orders of magnitude* more expensive than the buffered write, because it is bounded by physical storage, not by memory. This one number explains a great deal of systems design. It is why you never `fsync` per small write in a hot path: at ~116 durable records per second, a naive per-record sync is a throughput catastrophe. It is why durable systems **batch** — a database groups many transactions into one log flush (*group commit*), amortizing a single `fsync` across hundreds of records, so the per-record durability cost falls back toward the memory write while the guarantee is kept. And it is why the write-ahead log exists at all: append the change to a sequential log and `fsync` that once, rather than syncing scattered updates to the data files — a design Volume 2 builds in full. The rule to carry now: durability has a fixed, large per-flush price, so you pay it as seldom as correctness allows, by batching, not by syncing more.

> **QUICK CHECK**
>
> What does `fsync` do that `write` does not, and why is it roughly 9000× slower per record here? Given that cost, how do durable systems like databases keep throughput up while still guaranteeing that acknowledged data survives a crash? (Answer below the communicate box.)

41.3 Bypassing the cache: O_DIRECT

The page cache is a benefit you can decline. Opening a file with the `O_DIRECT` flag makes reads and writes bypass the page cache and transfer data directly between the storage device and your user-space buffer by DMA, with no kernel copy and no caching of the file's pages. The cost of this power is strict constraints, verified here against this `ext4` filesystem: the buffer address, the file offset, and the transfer length must all be aligned to the device's block size (commonly 512 or 4096 bytes), so a direct read needs a `posix_memalign`'d buffer, not an arbitrary `malloc`'d one — an unaligned direct I/O fails with `EINVAL`. Why would anyone give up the free cache? Because sometimes the kernel's cache is redundant or actively harmful. A database maintains its own **buffer pool** (Volume 2) — its own cache of pages, sized and evicted by its own policy — and the page cache underneath would just hold a second, uncoordinated copy of the same data, doubling memory use and defeating the database's careful replacement decisions; with `O_DIRECT` it manages its cache directly and the kernel stays out of the way. Direct I/O also avoids

cache pollution: a one-pass backup or a large sequential scan that will never reread its data would otherwise evict the working set of every other process to cache bytes it does not need, so bypassing the cache keeps the shared cache useful for programs that benefit from it. The trade is real and cuts both ways: `O_DIRECT` gives control and predictability but forfeits readahead, write-back batching, and the enormous win of a cache hit (the §40.3 13×), so it helps only programs that genuinely manage their own caching or deliberately stream past it, and hurts everything else. It is a specialist's tool — the man page (`open(2)`) is unusually blunt that most applications should not use it — and the discipline is the part's discipline: reach for it only when you have measured that the page cache is a cost rather than a benefit for your access pattern.

> **THE SAME IDEA THREE WAYS: DURABILITY IS A DEVICE ROUND-TRIP, SO YOU PAY IT RARELY**
>
> **1. In words.** A `write` is cheap because it is deferred — the bytes sit dirty in the page cache and the call returns. Making them *durable* means `fsync`: force the dirty pages to the device and wait for the device to confirm. That wait is a physical round trip to storage, orders of magnitude slower than the memory write, so durable systems batch many writes behind one flush rather than syncing each.
>
> **2. As a picture.** Figure 41.1: a `write` arrow stopping at the page cache (dirty page, call returns), with a dashed “later” arrow drifting to the device; beside it an `fsync` arrow going all the way to the device and a clock showing the caller blocked until it confirms. Below, two bars: buffered write ~0.9 µs/record vs write+`fdatasync` ~8.6 ms/record.
>
> **3. As numbers/contrast.** Same machine, 256-byte records: a buffered `write` costs ~0.9 µs and returns ~1.1 million/s; a `write` plus `fdatasync` costs ~8.6 ms and returns ~116/s — **~9000×** (§41.2). Group-commit a batch of, say, 1000 records behind one flush and the per-record durability cost falls back toward the memory write while the guarantee is kept — which is why write-ahead logs exist.

(Answer to the §41.1 quick check: after `write` returns, the data is in a dirty page in the kernel's page cache in volatile DRAM, scheduled for write-back later; a power loss, kernel panic, or hard reset before the flush loses it. Checking `write`'s return value does not help because it reports only that the kernel accepted the bytes into the cache — not that the device has them — so durability requires the separate, synchronous `fsync`.)

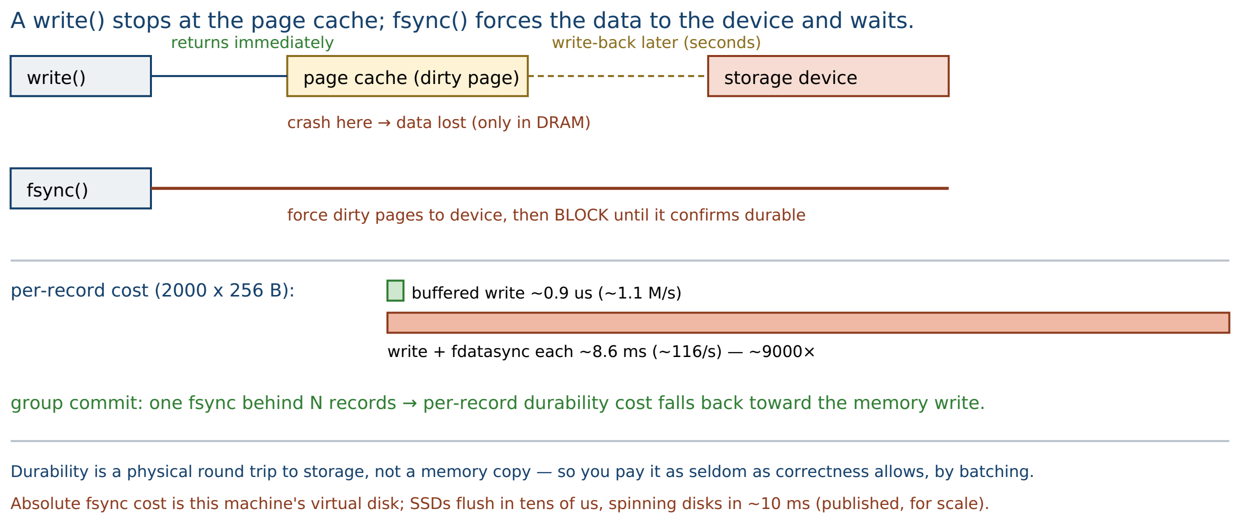


Figure 41.1: The durability gap and its price. A `write` returns once the data is a dirty page in the page cache (lost on a crash); `fsync/fdatasync` forces those pages to the device and blocks until it confirms — here ~8.6 ms versus ~0.9 µs for the buffered write, about **9000×** (§41.2). Because durability is a device round trip, systems batch many writes behind one flush (group commit). Wall-clock; absolute flush cost reflects this machine's virtual disk.

41.4 Memory-mapped I/O: mapping a file into your address space

The third tool changes not *when* data is durable but *how* you touch it. `mmap` maps a file (or part of one) into the process's virtual address space, so the file's contents appear as an array of bytes in memory: you read the file with ordinary loads and write it with ordinary stores, and no `read/write` system call is made per access. It reuses the virtual memory machinery of Chapters 4 and 17 directly — the mapping is lazy, so pages are brought in by **demand paging** on first touch (the MMU faults, the kernel fetches the page from the file into the page cache and maps it), and dirtied pages are written back to the file by the same write-back path, with `msync` available to force them like `fsync`. Its headline advantage is avoiding the copy: a `read` copies file data from the page cache into your buffer, whereas a mapped access reads the page-cache page in place, so `mmap` eliminates that user-space copy. Whether that makes it faster is genuinely a “measure it” question. Summing every 64th byte of a 256 MiB cached file, once via `read` into a buffer and once via `mmap`, ran here:

```
sum every 64th byte of a 256 MiB warm file:
read() into a buffer + sum : ~3,000-4,000 MiB/s   (a page-cache copy into the buffer, then
the sum)
mmap() + sum in place      : comparable           (no copy, but a minor page fault per 4 KiB
page on first touch)
                             across runs mmap ranged from ~2x faster to ~3x slower — the copy
it saves
                             trades against the page-fault cost; the winner depends on the
access pattern
```

The honest result is that the two are **close**, and which wins depends on the workload — exactly the discipline of the last part carried into I/O. `mmap` removes the `read` copy, which helps most when you touch the data many times or randomly (a large index, a shared read-only table), so the one-time page-fault cost of mapping it in is amortized and you get zero-copy random access through the address space. `read`/`write` tends to win for a single sequential streaming pass, where kernel readahead is already optimal and `mmap`'s per-page minor faults add overhead the streaming copy avoids — and it gives you explicit control over exactly when I/O happens, which mapped access hides behind faults. `mmap` also enables things `read`/`write` cannot: multiple processes can map the same file and share one set of physical pages (Chapter 19's shared memory is the anonymous case of this), and a large file can be treated as a random-access array without a manual seek-and-read. The costs are equally real: a page fault on an unmapped page can stall unpredictably, a hard I/O error becomes a `SIGBUS` at an ordinary memory access rather than an error return you can check, and durability still requires an explicit `msync` plus the same `fsync` physics as §41.2. So the choice is by access pattern, measured: map when you reuse or randomly access data and want it as memory; use buffered `read`/`write` when you stream once and want control over the I/O (Kerrisk, ch.49; Stevens & Rago, ch.14).

TRAP: TRUSTING A RETURNED WRITE() FOR DURABILITY — THE DATA THAT IS ONLY IN THE PAGE CACHE

The most dangerous I/O bug is invisible in testing: treating a successful `write` (or a flushed stdio buffer, or a `close`) as if the data were on disk. It is not — it is a dirty page in the page cache (§41.1), and it survives every normal test because the machine does not crash during a test; it is lost only in the power failure or kernel panic you were writing durable code to survive in the first place. `fflush` does not help: it moves data from the C library's user-space buffer into the kernel's page cache, which is still volatile — one buffer down, the durable barrier not yet crossed. Even `fsync` on the file is not always enough: creating or renaming a file also modifies its *directory*, and if the directory entry is not itself synced, a crash can leave the data durable but the file unreachable (the classic write-to-temp-then-rename pattern requires syncing the directory too). Two defenses. First, decide explicitly which data must be durable and cross the barrier for exactly that data with `fsync`/`fdatasync` (and `msync` for mapped writes), accepting the §41.2 cost and batching to amortize it — durability is never implicit. Second, test it honestly: a durability claim is only credible if it survives a real crash — kill the power (or the VM) mid-write and check what remains — because the whole failure mode is defined by a crash, and code that only ever ran to completion has never been tested against it. Assuming write-back is durability is the I/O version of shipping an optimization measured against nothing (Chapter 39): a guarantee asserted, never paid for.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our service writes each event to a file and returns 200 OK, so we are durable — and it is fast, a million events a second. If we ever need more safety we can `fsync` every write." Cover the paragraph and respond in four or five sentences, drawing on §41.1–§41.4.

Model answer. "We are fast precisely because we are *not* durable yet — the `write` returns when the data is in the page cache, so a power loss or kernel panic loses every event we have acknowledged since the last write-back, seconds of data. Returning 200 promises the client we kept it, which today is not true across a crash. But `fsync`-per-write is not the fix either: on our numbers a durable flush is about 8.6 ms against under a microsecond for the buffered write — roughly 9000× — so syncing every event would drop us from a million a second to about a hundred. The right design is to batch: append events to a log and `fsync` once per group (group commit), which amortizes the one expensive flush across many events and keeps both the guarantee and most of the throughput. Let us decide the actual durability requirement — how much data we may lose on a crash — and size the batch to it, then test it by pulling the plug mid-write and seeing what survives." Naming write-back as non-durable, quoting the flush cost, and reaching for batching over per-write sync is the senior register — and the seed of the write-ahead log.

*(Answer to the §41.2 quick check: `fsync` forces the file's dirty pages out of the page cache to the storage device and waits until the device confirms they are durably stored, which a plain `write` never does; it is ~9000× slower per record here because that wait is a physical round trip to storage (~8.6 ms on this device) versus a memory copy (~0.9 µs). Durable systems keep throughput up by **batching** — grouping many writes behind a single `fsync` (group commit, the write-ahead log), so the fixed per-flush cost is amortized across hundreds of records while every acknowledged record is still on the device before the acknowledgement.)*

41.5 What to carry forward

Durability is not free and not implicit. A `write` only reaches the page cache (Chapter 40), so it is fast but volatile: a crash before write-back loses acknowledged data, and the return value tells you nothing about the device (§41.1). The barrier that makes data durable is **`fsync`** (or `fdatasync`), which forces the file's dirty pages to storage and waits — here ~8.6 ms per flush against ~0.9 µs for the buffered write, about **9000×** (§41.2) — because it is bounded by the physical device, not by memory. That single fact drives durable-system design: you pay the flush as rarely as correctness allows by **batching** many writes behind one sync (group commit, the write-ahead log). Two more tools control how file data moves: **`O_DIRECT`** bypasses the page cache for programs that manage their own (databases with a buffer pool, one-pass scans that would only pollute the cache), at the cost of alignment constraints and forfeited readahead and cache hits (§41.3); and **memory-mapped I/O** maps a file into your address space so you touch it with loads and stores, avoiding the `read` copy — a win for reuse and random access, roughly a wash for a single streaming pass, and chosen by measured access pattern (§41.4). Above all, the trap: never mistake a returned `write` for durable data, and test a durability claim against a real crash (§41.4).

These are the mechanics Volume 2 builds a database on: the write-ahead log is exactly "append and `fsync` once" made into a discipline, the buffer pool is exactly "manage your own cache with `O_DIRECT`," and crash recovery is exactly "reason about what a `write` had and had not made durable when the power failed." The last two chapters have been about a program moving bytes to and from storage, one operation at a time, blocking until each finishes. But a server does not talk to one file — it talks to thousands of clients at once, and it cannot afford to stop and wait on each. The next chapter, Chapter 42, turns to that problem: **blocking versus non-blocking I/O** — what happens to a thread that waits on a slow read, why one thread per connection does not scale to ten thousand of them, and how a single thread can watch thousands of descriptors and act only on the ones that are ready. The cost model shifts again, from the latency of one operation to the concurrency of many.

CAN YOU… WITHOUT LOOKING?

- Explain why a successful `write` is not durable, and what event loses the data.
- State what `fsync` does and, to an order of magnitude, how much slower it is than a buffered write.
- Explain why durable systems batch writes behind one flush, and name the design that does so.
- Say what `O_DIRECT` does, its main constraint, and one kind of program that wants it.
- State what `mmap` avoids versus `read`, and when to prefer each.
- Name the durability trap and how you would actually test a durability claim.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** After a `write` returns, where is the data, and what loses it? (§41.1)
2. **R2.** What does `fsync` do, why is it ~9000× slower per record here, and how do systems keep throughput? (§41.2)
3. **R3.** What does `O_DIRECT` do, what constraint does it impose, and who wants it? (§41.3)
4. **R4.** What does `mmap` avoid versus `read`, and when does each win? (§41.4)
5. **R5.** Why is trusting a returned `write` for durability a trap, and how do you test the claim? (§41.4)

FURTHER READING

- **Michael Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), ISBN 978-1-59327-220-3.** Chapter 13 (kernel buffering, `fsync`/`fdatasync`, and `O_DIRECT`) and Chapter 49 ("Memory Mappings") are the reference for this entire chapter — the durability barrier, direct I/O, and `mmap`. First author Michael Kerrisk; edition and ISBN verified via the publisher.
- **W. Richard Stevens & Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. (Addison-Wesley, 2013), ISBN 978-0-321-63773-4.** Chapter 3 (file I/O and synchronized writes) and Chapter 14 (memory-mapped I/O) ground §41.1–§41.4 in the POSIX interfaces. First author W. Richard Stevens; edition and ISBN verified via the publisher.
- **Thanumalayan S. Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau & Remzi H. Arpaci-Dusseau, "All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications," in *Proceedings of OSDI 2014*, pp. 433–448.** The empirical study behind §41.1's crash-consistency warning — how applications get durability and ordering wrong, and why `fsync` placement matters. First author Thanumalayan Pillai; USENIX open-access proceedings (no DOI), cited by venue and page range.
- **Marshall K. McKusick, William N. Joy, Samuel J. Leffler & Robert S. Fabry, "A Fast File System for UNIX," *ACM Transactions on Computer Systems* 2(3), pp. 181–197 (1984), DOI 10.1145/989.990.** The classic behind the filesystem layer that `fsync` must push data through — allocation, locality, and on-disk layout. First author Marshall Kirk McKusick; DOI verified via the ACM Digital Library.

Exercises

41.1 WARM-UP In two sentences, say where the data is after a `write` returns success, and what single event would cause it to be lost.

41.2 WARM-UP What does `fsync` do that `write` does not? What does `fdatasync` skip that `fsync` does, and why can that make it slightly cheaper?

41.3 WARM-UP Why is `fflush` on a stdio stream *not* the same as making the data durable? Which buffer does it empty, and into what?

41.4 CORE Durability cost. (a) Explain why `fdatasync`-per-record ran ~9000× slower than the buffered write here. (b) Why is the absolute ~8.6 ms a property of the device rather than the CPU? (c) A system needs durable acknowledgements but 100 records/s is far too slow — describe how group commit gets both the guarantee and the throughput, and what it trades.

41.5 CORE `O_DIRECT`. (a) What does it bypass, and how is the data moved instead? (b) State the alignment constraint it imposes and why an ordinary `malloc`'d buffer can fail. (c) Give one program that genuinely wants `O_DIRECT` and one that would be harmed by it, with the reason in each case.

41.6 CORE Memory-mapped I/O. (a) What copy does `mmap` avoid that `read` pays? (b) Why did the two come out close in the §41.4 measurement — what does `mmap` pay instead of the copy? (c) Name an access pattern for which `mmap` clearly wins and one for which buffered `read`/`write` is the better choice.

41.7 COST A durable log appends records and must `fsync` to acknowledge them. A buffered write costs w and an `fsync` costs f, with $f >> w$. (a) Write the per-record cost when you `fsync` after every record, and when you `fsync` once per batch of k records. (b) Using $w \approx 0.9$ µs, $f \approx 8.6$ ms, compute the per-record cost at $k=1$ and $k=1000$. (c) What does the batch cost the application in the event of a crash, and how does that set an upper bound on k?

41.8 FADED *Worked, with the last step for you.* A colleague's service writes each record and calls `fsync` after every one; it is correct but only sustains ~100 records/s, and they propose buying faster disks.
Step 1 (given): each acknowledgement is bounded by one device-flush round trip (~8.6 ms here), not by CPU or the buffered write.
Step 2 (given): the flush cost is a fixed per-`fsync` price largely independent of how many bytes it covers.
Step 3 (given): so one `fsync` can be shared across many records appended before it (group commit).
Step 4 (you): Explain why batching, not faster disks, is the right first move; estimate the throughput at a batch of 1000; and state precisely what durability the batch still guarantees and what it does not.

41.9 MIXED For each, say whether the data is durable at the marked point, and if not, what to add: (a) `write(fd, rec, n)` returns success, then the machine loses power; (b) the same, but `fdatasync(fd)` returned before the power loss; (c) a record is written to a new temp file, `fsync`'d, and the temp file renamed over the target, then power is lost — but the directory was never synced; (d) a mapped-file store dirties pages via memory writes and the process exits cleanly without `msync`, then power is lost.

41.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “If `write` returns success, the data is on the disk.” (b) “`fsync` is slow mainly because copying the bytes is expensive.” (c) “`O_DIRECT` makes I/O faster for every workload.” (d) “`mmap` lets several processes share one set of physical pages for the same file.” (e) “A durability guarantee that passes all your tests without a crash is proven.”

41.11 STRETCH Look ahead to Volume 2. (a) Explain why the huge cost of `fsync` is the reason databases keep a *write-ahead log* rather than syncing their data files directly. (b) Explain why a database using its own buffer pool often opens data files with `O_DIRECT`, in terms of avoiding a redundant second cache. (c) Argue why “how often must I cross the durability barrier” is the first design question for any storage system, tying it to the ~9000× number.

41.12 LAB On your own machine (a C toolchain): (a) Append many small records to a file, timing a version with no sync against one with `fdatasync` after each record; report per-record cost and the ratio. (b) Repeat with an `fsync` once per batch of k records for a few values of k, and plot per-record cost against k. (c) Sum a large cached file via `read` into a buffer and via `mmap`, reporting both and their variance. State your toolchain, CPU, filesystem, and device, and note these are wall-clock times (no hardware counters) if you lack `perf`; if your storage is a virtual disk, say so, since it sets the absolute `fsync` cost.

Chapter 42 — Blocking, Non-Blocking, and Event-Driven I/O

Before you start: Chapter 41 (a read that must reach the device blocks the thread until it finishes), Chapter 40 (the I/O syscalls), Chapter 13 (the cost of a system call), Chapter 15 and Chapter 16 (threads and scheduling — what a thread costs and what a blocked one does), Chapter 25 (thread pools — where blocking work belongs). · The last two chapters moved bytes one operation at a time, each blocking until it finished. A server does not have that luxury: it faces thousands of connections at once and cannot stop on any one. This chapter is how a single thread watches thousands of descriptors and acts only on the ready ones.

BEFORE YOU READ ON

From memory: (1) Chapter 16 — when a thread calls a blocking `read` on data that has not arrived, what does the scheduler do with that thread, and what is a context switch? (2) Chapter 15 and Chapter 25 — what does a thread cost in memory (its stack) and in scheduler pressure, and why does "just add threads" stop scaling? (3) *Predict:* a server holds 10,000 connections, almost all idle at any instant. Version A dedicates one thread to each, each blocked in `read`. Version B uses one thread that asks the kernel "which of these 10,000 are ready?" and handles only those. Two questions: what does version A cost when 10,000 threads sit blocked? And to answer "which are ready," how does the cost of `select` (which scans all of them) compare to `epoll` as the number grows? Write your guesses; §42.1 and §42.3 answer them.

Every I/O call so far has been **blocking**: a `read` on data that is not ready does not return — it parks the calling thread, which the scheduler deschedules until the data arrives (Chapter 16). For a program that talks to one file or one connection, this is exactly right and beautifully simple: the code reads as if the data were always there, and the kernel handles the waiting. The trouble begins when one program must serve *many* slow connections at once. If a blocked thread can wait on only one descriptor, then handling N connections concurrently needs N threads — and a thread is not free: it holds a stack (often megabytes of address space), it adds to the scheduler's run queue, and switching between thousands of them costs context switches that do no useful work (Chapters 15–16). At ten thousand connections — the famous **C10K problem** (Kegel) — the thread-per-connection model spends its resources on threads that are almost all just waiting. The escape is to stop dedicating a thread to each wait and instead ask the kernel a different question: not "block me until *this one* descriptor is ready," but "tell me which of *these thousands* are ready right now," and then do work only for those. That is **event-driven I/O**, and this chapter builds it in five steps: blocking I/O and why thread-per-connection does not scale (§42.1); non-blocking I/O and the busy-poll trap it creates (§42.2); readiness notification — `select`, `poll`, and `epoll` — and how their costs differ, measured (§42.3); the event loop and its own characteristic trap (§42.4); and what to carry forward (§42.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35, 20 hardware threads), timed with `clock_gettime(CLOCK_MONOTONIC)` as the minimum of several trials (Chapter 33). Times are wall-clock. A hardware-counter profiler (`perf`) is *not* installed here, so the syscall cost below is measured as wall-clock time per call, and the `select`-vs-`epoll` comparison is measured by monitoring many idle pipe descriptors with exactly one ready and timing the readiness scan as the count grows — the absolute nanoseconds are this machine's (on WSL2 the syscall floor is higher than on bare metal), but the *scaling* — `select` growing with the descriptor count, `epoll` staying flat — is the portable, architectural result the chapter turns on.

42.1 Blocking I/O and the thread-per-connection model

A blocking call is the default and, for one stream, the ideal. When a thread calls `read` on a socket with no data yet, the kernel puts the thread to sleep — off the run queue entirely — and wakes it when data arrives, so the thread consumes no CPU while waiting and the code reads as a simple sequence (Chapter 16). The natural way to serve many connections with blocking calls is therefore one thread (or process) per connection: each thread blocks in `read` on its own socket, wakes when its client sends something, handles it, and blocks again. This model is easy to reason about and fine at small scale. It fails at large scale for reasons that are all about the *thread*, not the I/O. Each thread reserves a stack — commonly a megabyte or more of address space by default — so ten thousand threads reserve gigabytes before doing any work. Each runnable thread is an entry the scheduler must consider, and when many wake at once the kernel pays **context switches** — saving and restoring registers, switching address-space state, polluting caches — none of which moves a byte (Chapters 15–16). And the system call itself is not free: measured here, a minimal system call that must enter the kernel costs about **199 ns**, against about 40 ns for a `clock_gettime` that the kernel services in user space (the vDSO) without a real kernel entry — so the boundary crossing alone is a real per-operation tax, paid on every blocking call that has to sleep and wake. The result is that thread-per-connection spends most of its resources holding threads that are, at any instant, almost all asleep: ten thousand connections that are individually idle 99% of the time still cost ten thousand stacks and the scheduler churn of waking whichever few become ready. The problem is not that blocking is wrong — it is that *one thread per wait* ties a heavy, scheduler-visible resource to each connection, and connections vastly outnumber the work actually ready at any moment. This is the C10K problem, and its solution is to break the one-thread-per-connection assumption while keeping the machine busy only with connections that have something to do.

> **QUICK CHECK**
>
> Why does a blocking `read` consume no CPU while it waits, yet thread-per-connection still fails to scale to ten thousand mostly idle connections? Name two costs of a thread that are paid whether or not the thread is doing work. (Answer after the next section.)

42.2 Non-blocking I/O and the busy-poll trap

The first tool for escaping thread-per-connection is to change how the descriptor behaves. Setting the `O_NONBLOCK` flag on a file descriptor makes its I/O calls **non-blocking**: a `read` with no data ready returns immediately with the error `EAGAIN` (equivalently `EWOULDBLOCK`) instead of sleeping, and a `write` that would block because the socket's send buffer is full returns having accepted only part, or none, of the data. This means a single thread can attend to many descriptors without any of them parking it: try each in turn, handle the ones that return data, and skip the ones that return `EAGAIN`. But non-blocking I/O alone creates a new problem as bad as the one it solved. If the thread simply loops over all N descriptors calling `read` and mostly collecting `EAGAIN`, it is **busy-polling**: it burns 100% of a CPU asking “anything yet? anything yet?” across thousands of descriptors that are overwhelmingly not ready, turning idle waiting (which cost nothing under blocking) into a spinning, power-hungry scan. And it does so at the system-call cost of §42.1 — each `read` that returns `EAGAIN` is a ~199 ns kernel round trip that accomplished nothing — so polling 10,000 idle descriptors in a tight loop is millions of wasted syscalls per second. Non-blocking I/O is necessary — an event-driven server must never let a single descriptor block its one thread — but it is not sufficient: it removes the blocking, but it does not tell you *which* descriptors are worth calling. What is missing is a way to ask the kernel, in one call, which of a large set of descriptors are ready, so the thread can sleep when nothing is ready (consuming no CPU, like blocking) and wake with the exact list of descriptors that now have work (handling only those, like non-blocking). That is readiness notification, and it is the mechanism the whole model turns on.

> **QUICK CHECK**
>
> What does `O_NONBLOCK` change about a `read` that finds no data? Why is looping over thousands of non-blocking descriptors calling `read` on each — busy-polling — nearly as bad as thread-per-connection, and what capability is still missing? (Answer below the communicate box.)

42.3 Readiness notification: select, poll, and epoll

The missing capability is a system call that takes a set of descriptors and returns which are ready, sleeping the thread until at least one is. Three generations of it exist, and the difference between them is the whole performance story. `select` (and its near-twin `poll`) takes the entire set of descriptors you care about on *every* call, and the kernel scans all of them to find the ready ones — so its cost grows with the *total* number of descriptors watched, ready or not. `epoll` (Linux; `kqueue` is the BSD equivalent) splits this into two steps: you register your interest in each descriptor *once* with `epoll_ctl`, and then each `epoll_wait` call returns only the descriptors that have *become* ready, so its cost grows with the number of *active* descriptors, not the total watched. Timing a readiness scan over many idle pipe descriptors with exactly one ready, as the count grows, shows the difference sharply in this environment:

```
readiness scan, M descriptors watched, exactly 1 ready (min of several trials):
   M         select()        epoll_wait()
   10          ~0.7 us          ~0.4 us
  100          ~2.6 us          ~0.4 us
  500         ~14.6 us          ~0.4 us
 1000     n/a (> FD_SETSIZE)     ~0.4 us
 4000     n/a (> FD_SETSIZE)     ~0.4 us
               select grows ~linearly with M; epoll_wait stays flat
```

The pattern is unmistakable: `select`'s cost climbs roughly **linearly** with the number of descriptors watched — about 0.7 µs at 10, 2.6 at 100, 14.6 at 500 — because it rescans the whole set on every call, while `epoll_wait` stays essentially **flat** at a few hundred nanoseconds no matter how many descriptors are registered, because it returns only the one that is ready. This is the O(N)-versus-O(active) distinction made concrete, and it is exactly the cost that matters for a server where N is huge and the active fraction per wakeup is tiny. `select` carries a second, hard limit visible in the table: the `fd_set` it uses is a fixed-size bitmap capped at `FD_SETSIZE` (1024 on this system), so it simply cannot watch a descriptor numbered 1024 or above — a wall the C10K problem hits immediately, and one `epoll` does not have. One refinement matters when you build on `epoll`: it offers **level-triggered** notification (the default: a descriptor is reported ready as long as it has data, matching `select`'s semantics and forgiving of partial reads) and **edge-triggered** notification (reported only on the transition to ready, which is faster but requires you to drain the descriptor fully on each event or you will miss data). Level-triggered is the safer default; edge-triggered is an optimization for the last drop of performance. The lesson to carry is the scaling, not the nanoseconds: to watch many descriptors, register interest once and let the kernel hand you the ready ones — never rescan the whole set on every call (Kerrisk, ch.63; Banga, Mogul & Druschel, 1999).

THE SAME IDEA THREE WAYS: DON'T WAIT PER CONNECTION, AND DON'T RESCAN EVERYONE — ASK ONCE WHO'S READY

1. In words. Thread-per-connection ties a heavy thread to each idle wait, so it drowns at C10K. Non-blocking I/O lets one thread attend many descriptors but, alone, forces a wasteful busy-poll. Readiness notification supplies the missing piece — one call that sleeps until some descriptor is ready and returns which — and `epoll` makes that call scale by registering interest once and returning only the active descriptors, instead of `select` rescanning the whole set every time.

2. As a picture. Figure 42.1: on the left, N threads each blocked on one socket (N stacks, scheduler churn); on the right, one thread in an event loop calling `epoll_wait`, which returns the two ready descriptors out of thousands, dispatched to non-blocking handlers. Below, a line plot: `select` rising with M, `epoll_wait` flat, and a vertical wall at `FD_SETSIZE` = 1024.

3. As numbers/contrast. Same machine, one descriptor ready. `select`: ~0.7 µs at 10 descriptors, ~14.6 µs at 500 — and impossible past 1024. `epoll_wait`: ~0.4 µs at 10, at 500, and at 4000 — **flat** (§42.3). A minimal kernel-entering syscall costs ~199 ns here (§42.1), so a busy-poll of 10,000 idle descriptors is millions of those per second doing nothing; one `epoll_wait` replaces the whole scan.

(Answer to the §42.1 quick check: a blocking `read` consumes no CPU because the kernel takes the thread off the run queue entirely and wakes it only when data arrives — waiting

*is free of CPU. Thread-per-connection still fails to scale because the cost is in the **threads**, not the waiting: each thread reserves a stack (often a megabyte or more of address space) and is a scheduler-visible entity, so ten thousand mostly-idle connections cost ten thousand stacks plus the context-switch churn of waking whichever few become ready — two costs paid regardless of whether any thread is doing work.)*

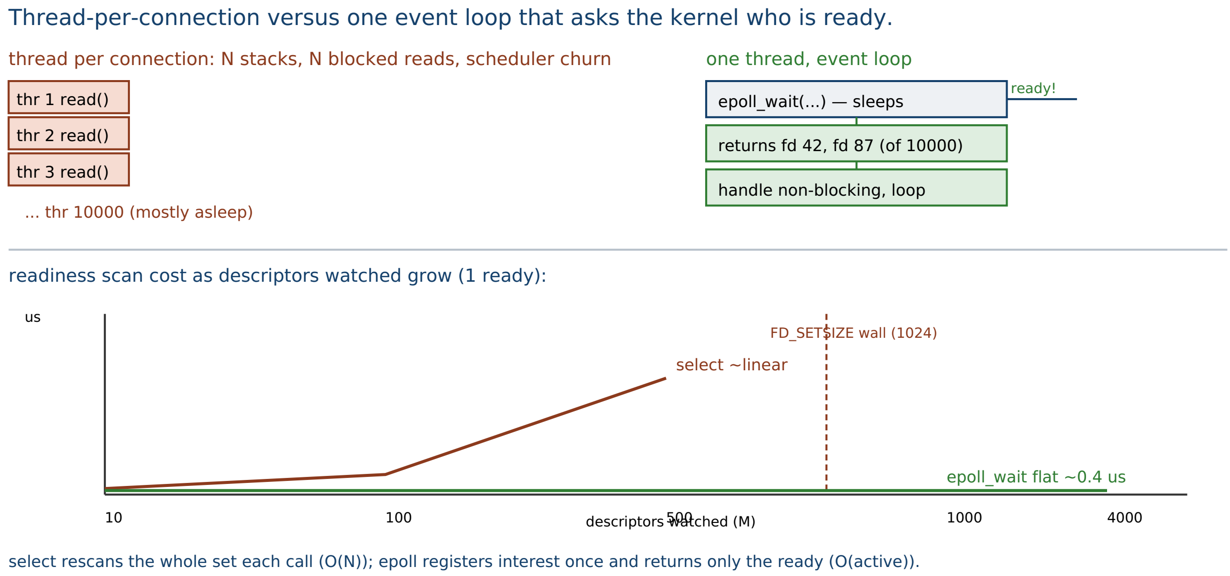


Figure 42.1: Thread-per-connection (left) ties a stack and a scheduler entry to each mostly-idle connection and drowns at C10K; an event loop (right) uses one thread that calls `epoll_wait` and handles only the ready descriptors. `select`'s cost grows linearly with the number watched and hits the `FD_SETSIZE`=1024 wall; `epoll_wait` stays flat at ~0.4 µs to thousands of descriptors (§42.3). Wall-clock; the scaling shape, not the absolute nanoseconds, is the portable result.

42.4 The event loop, and the blocking call that stalls everything

Put the pieces together and you have the **event loop** that underlies nearly every high-concurrency server (nginx, Redis, Node.js, and the async runtimes of most languages): one thread — or a small pool, one per core — repeatedly calls `epoll_wait`, receives the set of ready descriptors, and for each dispatches a short, non-blocking handler that does the available work (read what is there, write what fits) and returns immediately to the loop. The thread sleeps in `epoll_wait` when nothing is ready (no busy CPU, like blocking) and wakes with exactly the ready descriptors (handling only those, like non-blocking) — the two halves the previous sections were missing, joined. A handful of threads on this model can hold hundreds of thousands of connections, because a connection now costs only a descriptor and a little state, not a thread and a stack. But the model has a sharp and characteristic failure mode that is the mirror image of thread-per-connection's: because *one* thread drives thousands of connections, a single **blocking operation inside a handler stalls every connection that thread owns**. If a handler calls a blocking `read` from disk (an un-cached file — recall from Chapter 41 that this can wait milliseconds), performs a synchronous DNS lookup, holds a lock, calls `fsync` (the ~8.6 ms of §41.2!), or simply runs a long CPU computation, the event loop cannot make progress on any other descriptor for that entire duration — one slow client, or one careless call, freezes them all. The discipline the model demands is therefore absolute: **nothing in an event-loop**

handler may block. Everything the loop touches — sockets, and increasingly files — must be non-blocking, and any operation that must block or compute at length is handed off to a separate **thread pool** (Chapter 25), whose result comes back to the loop as another ready event. This is the seam where the two concurrency models meet: the event loop for the vast number of waiting connections, a bounded pool of threads for the genuinely blocking or CPU-heavy work, connected by the readiness mechanism. (There is a further step this part will reach — *completion*-based asynchronous interfaces such as `io_uring`, where you submit an operation and are notified when it has finished rather than when a descriptor is ready to be tried — which extends the same idea to disk I/O that has no clean readiness signal; the readiness model of this chapter is the foundation it builds on.)

TRAP: THE BLOCKING CALL INSIDE THE EVENT LOOP — ONE SLOW HANDLER FREEZES EVERY CONNECTION

The event loop's strength is its weakness: one thread serving thousands of connections means any single blocking or long-running call in a handler halts *all* of them for its full duration. The trap is that this is invisible in light testing — with one client and warm caches every operation returns instantly, so the accidental blocking call (a disk read that is usually cached, an occasional `fsync`, a synchronous library call, a lock held under contention, a regex or JSON parse over a huge input) never actually blocks — and it surfaces only in production as mysterious, correlated latency spikes across unrelated connections whenever that one path is hit cold. The tell is exactly that correlation: in thread-per-connection a slow operation hurts one client, but in an event loop it hurts every client the thread holds, so a single cold disk read can spike the tail latency of thousands of requests at once. Two defenses. First, make it a rule that *no* syscall in a handler may block: every socket is `O_NONBLOCK`, and anything that can wait — disk I/O, DNS, `fsync`, a large computation — is dispatched to a thread pool (Chapter 25) and its completion delivered back to the loop as an event, never awaited inline. Second, measure the handler, not just the throughput: the number that matters is the maximum time a handler spends before returning to `epoll_wait`, because that is the latency floor for every other connection — a mean that looks fine can hide a handler that occasionally blocks for milliseconds. Assuming a handler is non-blocking because it usually is fast is the event-loop version of the part's recurring trap: a property asserted from the common case and never measured against the one that hurts.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "For our new gateway we'll spawn a thread per connection — threads are cheap now, and the code is so much simpler than an event loop. We expect maybe fifty thousand mostly-idle long-lived connections." Cover the paragraph and respond in four or five sentences, drawing on §42.1–§42.4.

Model answer. "Thread-per-connection is simpler and it is the right call at small scale, but fifty thousand mostly-idle connections is exactly where it breaks — that is fifty thousand stacks (tens of gigabytes of address space at a megabyte each) and a scheduler holding fifty thousand entities to wake a handful at a time, when almost none have work at any instant. The cost is in the threads, not the waiting, so it does not matter that they are idle. The scalable shape is an event loop: a few threads, one per core, each calling `epoll_wait` and handling only the ready descriptors — `epoll` stays flat as connections grow, where `select` would climb linearly and hit the 1024-descriptor wall. The catch we must respect is that every handler has to be strictly non-blocking: one blocking disk read or `fsync` in a handler stalls every connection on that thread, so anything that can wait goes to a thread pool and comes back as an event. If we want the simple code, let us use a language runtime or library that gives us the event loop underneath — but let us not put fifty thousand blocking threads in the kernel." Naming the thread cost, reaching for the event loop, and flagging the no-blocking-in-handlers rule is the senior register.

(Answer to the §42.2 quick check: `O_NONBLOCK` makes a `read` with no data return immediately with `EAGAIN` instead of parking the thread. Looping over thousands of non-blocking descriptors calling `read` on each is busy-polling: it burns a full CPU issuing millions of ~199 ns syscalls per second that mostly return `EAGAIN`, turning the free idle-wait of blocking into a spinning scan — nearly as wasteful as thread-per-connection, in CPU instead of memory. What is still missing is a single call that sleeps until some descriptor is ready and returns ***which*** *ones — readiness notification, i.e. `epoll`.)*

42.5 What to carry forward

Serving many connections is a **concurrency** problem, not a latency one, and the cost model is the thread. A **blocking** call is simple and consumes no CPU while it waits (Chapter 16), but it can wait on only one descriptor, so serving N connections this way needs N threads — and threads cost stacks and scheduler churn whether idle or not, which is why thread-per-connection drowns at the **C10K** scale (§42.1). **Non-blocking** I/O (`O_NONBLOCK`) lets one thread attend many descriptors without any parking it, but alone it forces a wasteful **busy-poll** — millions of `EAGAIN` syscalls (~199 ns each here) doing nothing (§42.2). The missing piece is **readiness notification**: one call that sleeps until some descriptor is ready and returns which. `select`/`poll` rescan the whole set each call, so their cost grows linearly with the number watched (and `select` caps at `FD_SETSIZE`=1024); `epoll` registers interest once and returns only the active descriptors, staying flat — here ~0.4 µs from 10 to 4000 descriptors, against `select`'s climb to ~14.6 µs at 500 (§42.3). Assembled, these give the **event loop**: one thread (or one per core) calling `epoll_wait` and dispatching short non-blocking handlers, holding hundreds of thousands of connections cheaply — with the absolute rule that no handler may block, or it stalls every connection the thread owns, with the genuinely blocking work handed to a thread pool (§42.4).

This is the substrate under every scalable service, and it is where the part's threads converge: the OS gave us blocking calls and cheap scheduling (Volume 1's Part III), concurrency gave us threads and the pools to bound them (Part IV), and this chapter spends both wisely — an event loop for the many waits, a pool for the few blocks. The remaining chapters of this part put this substrate to work on the network itself. The next builds the connection the event loop has been watching — **sockets**: how a program opens a network endpoint, accepts connections, and moves bytes across the wire, and how the blocking-versus-readiness choice of this chapter plays out on a real server. From there the part reaches the completion-based interfaces (`io_uring`) that extend readiness to disk, and the *zero-copy* techniques that remove even the page-cache copy this part has taken for granted — the last cost between the file and the network card. The waiting is solved; now we wire it to the world.

CAN YOU… WITHOUT LOOKING?

- Explain why a blocking `read` uses no CPU, yet thread-per-connection fails to scale to C10K.
- Name two costs of a thread paid whether or not it is doing work.
- Say what `O_NONBLOCK` does and why non-blocking I/O alone leads to a wasteful busy-poll.
- State how `epoll` scales differently from `select`, and why, with the order of magnitude.
- Describe the event loop and the one rule its handlers must obey.
- Explain why a single blocking call in a handler is worse in an event loop than in thread-per-connection.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why does thread-per-connection fail at C10K, when blocking waits are CPU-free? (§42.1)
2. **R2.** What does `O_NONBLOCK` change, and what trap does non-blocking I/O create on its own? (§42.2)
3. **R3.** How do `select` and `epoll` scale differently, and why — and what hard limit does `select` have? (§42.3)
4. **R4.** What is the event loop, and what is the absolute rule for its handlers? (§42.4)
5. **R5.** Why does one blocking call in a handler hurt every connection, and where should blocking work go? (§42.4)

FURTHER READING

- **Michael Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), ISBN 978-1-59327-220-3.** Chapter 63 ("Alternative I/O Models") is the primary reference for this chapter — non-blocking I/O, I/O multiplexing with `select/poll`, the `epoll` API, and level- versus edge-triggered notification. First author Michael Kerrisk; edition and ISBN verified via the publisher.
- **W. Richard Stevens, Bill Fenner & Andrew M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. (Addison-Wesley, 2003), ISBN 978-0-13-141155-5.** Chapter 6 (I/O multiplexing) and Chapter 16 (non-blocking I/O) ground §42.2–§42.3 in the sockets context that Chapter 43 will build. First author W. Richard Stevens; edition and ISBN verified via the publisher.
- **Gaurav Banga, Jeffrey C. Mogul & Peter Druschel, "A Scalable and Explicit Event Delivery Mechanism for UNIX," in *Proceedings of the USENIX Annual Technical Conference*, 1999.** The paper behind §42.3's shift from `select`'s rescan-everything model to explicit, scalable readiness delivery — the idea `epoll` and `kqueue` implement. First author Gaurav Banga; USENIX open-access proceedings (no DOI), cited by venue and year.
- **Dan Kegel, "The C10K problem" (online, 1999, updated through the 2000s).** The essay that named the scaling problem of §42.1 and surveyed the event-driven techniques that answer it. Author Dan Kegel; a web resource carrying no DOI, cited by author and title.

Exercises

42.1 WARM-UP In two sentences, explain why a blocking `read` consumes no CPU while it waits, and name two costs a thread carries whether or not it is doing work.

42.2 WARM-UP What does setting `O_NONBLOCK` change about a `read` that finds no data ready? What error does it return in that case?

42.3 WARM-UP In one sentence each, state the essential difference between how `select` and `epoll` find the ready descriptors, and which one's cost grows with the total number watched.

42.4 CORE The C10K problem. (a) Why does dedicating one thread per connection not scale to ten thousand mostly-idle connections — name the two dominant costs. (b) Why does it *not* help that the threads are idle almost all the time? (c) What does the event-loop model tie to each connection instead of a thread, and why is that so much cheaper?

42.5 CORE Busy-polling. (a) Explain why looping over thousands of non-blocking descriptors calling `read` on each burns a whole CPU. (b) Using the ~199 ns syscall cost, estimate the syscalls per second for a tight poll of 10,000 idle descriptors and comment. (c) What does readiness notification add that turns this back into a CPU-free wait?

42.6 CORE `select` versus `epoll`. (a) From the measured table, describe how each scales as the number of descriptors grows from 10 to 500. (b) Explain the mechanism behind each scaling — what does `select` do on every call that `epoll_wait` does not? (c) What is `FD_SETSIZE`, and why is its limit fatal for a C10K server using `select`?

42.7 COST A server watches M descriptors, of which a are ready per wakeup. Model the readiness-scan cost as $c_s = p \cdot M$ for `select` and $c_e = q \cdot a + r$ for `epoll`. (a) For which regime of M and a is `epoll` dramatically better, and for which are they comparable? (b) Using the measured numbers (`select` ~14.6 µs at M=500; `epoll_wait` ~0.4 µs), estimate the ratio at M=500, a=1. (c) Explain why a workload where nearly all watched descriptors are ready every wakeup narrows `epoll`'s advantage.

42.8 FADED *Worked, with the last step for you.* A team's event-loop server shows fine mean latency in load tests but occasional spikes in production where thousands of unrelated requests slow down together.
Step 1 (given): one thread drives all those connections via `epoll_wait` and non-blocking handlers.
Step 2 (given): one handler reads a config file that is usually in the page cache, so in testing the read never blocks.
Step 3 (given): in production the file is occasionally cold, and that `read` blocks for milliseconds.
Step 4 (you): Explain why a single occasionally-blocking handler produces correlated spikes across thousands of connections, why the load test missed it, and the fix.

42.9 MIXED For each operation inside an event-loop handler, say whether it is safe or must be moved to a thread pool, and why: (a) a non-blocking `read` from a socket returning `EAGAIN`; (b) an `fsync` on a log file; (c) parsing a 200-byte JSON body already in memory; (d) a synchronous DNS lookup; (e) a CPU-bound image resize on an uploaded photo.

42.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “A blocking `read` burns CPU while it waits.” (b) “`epoll_wait`'s cost grows with the total number of descriptors registered.” (c) “Non-blocking I/O by itself is enough to serve ten thousand connections efficiently.” (d) “One blocking call in an event-loop handler can stall every connection that thread serves.” (e) “`select` can watch a descriptor with any file-descriptor number.”

42.11 STRETCH Synthesize the chapter. (a) Explain why blocking, non-blocking, and readiness notification are not three competing choices but three parts of one design — what each contributes and what it lacks alone. (b) The event loop and thread-per-connection have mirror-image failure modes; describe both and the single principle (match the resource to the work) that explains each. (c) Argue why “how many descriptors, and how many are active per wakeup” is the right first question when choosing an I/O model, tying it to the `select`-vs-`epoll` scaling.

42.12 LAB On your own machine (a C toolchain): (a) Measure the cost of a minimal kernel-entering system call (e.g. `syscall(SYS_getpid)`) as ns/call, the minimum of several trials, and contrast it with a vDSO call like `clock_gettime`. (b) Create many pipe descriptors, mark exactly one ready, and time a readiness scan with `select` and with `epoll_wait` as the count grows (10, 100, 500, and — for `epoll` only — 1000+); report how each scales and where `select` hits `FD_SETSIZE`. (c) State your toolchain and CPU, and note these are wall-clock times (no hardware counters) if you lack `perf`, and that the absolute syscall cost is higher under some virtualized environments while the scaling shape is portable.

Chapter 43 — Sockets, TCP, and UDP: The Connection Lifecycle

Before you start: Chapter 42 (the event loop that watches many connections, and the blocking-versus-readiness choice), Chapter 40 (the I/O syscalls and buffering), Chapter 13 (the cost of a system call), Chapter 16 (what a blocked thread does). · The last chapter watched thousands of descriptors without asking what they were. Here they get a name: a **socket** is the descriptor that connects a program to the network, and this chapter is how one is opened, connected, and fed bytes — and what the two transport protocols underneath, TCP and UDP, promise and refuse.

BEFORE YOU READ ON

From memory: (1) Chapter 40 — a `read` returns however many bytes are available *now*, not a fixed record; what does that imply for data that arrives in pieces? (2) Chapter 13 and Chapter 42 — a system call that must enter the kernel costs a few hundred nanoseconds; keep that number in mind. (3) *Predict:* a client sends a 40-byte request as two small `write` calls and then waits for the reply; the server reads the whole request, then answers. On a plain TCP connection the round trip takes tens of milliseconds; flip one socket option and it drops to under a tenth of a millisecond. Two questions: what could make a localhost round trip take *40 ms* when the network latency is microseconds? And what one option removes it? Write your guesses; §43.4 answers them.

Every network program is built from the same object: a **socket**, a file descriptor (Chapter 40) that instead of naming a file names an endpoint of a communication channel. Because it is a descriptor, everything the I/O chapters taught applies unchanged — `read`, `write`, `close`, `O_NONBLOCK`, and the `epoll` of Chapter 42 all work on sockets exactly as on files — which is the whole point of the design: the network is just another I/O device behind the descriptor abstraction. What is new is how a socket is *set up* (a file you just `open`; a connection must be negotiated between two machines), and what the transport protocol underneath guarantees about the bytes that flow through it. This chapter builds that in five steps: the socket API and the connection lifecycle, server and client (§43.1); TCP as a reliable, ordered byte stream — and why it has no message boundaries (§43.2); UDP as datagrams that keep boundaries but drop every other promise (§43.3); the small-write stall where Nagle's algorithm and delayed acknowledgements collide, measured (§43.4); and what to carry forward (§43.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35, 20 hardware threads), timed with `clock_gettime(CLOCK_MONOTONIC)` as the minimum of several trials (Chapter 33). Times are wall-clock; a hardware-counter profiler (`perf`) is *not* installed here. The client and server run as two threads in one process talking over the **loopback** interface (`127.0.0.1`), so there is no real network latency — which makes the §43.4 result striking: the tens of milliseconds measured there are not the wire, they are a protocol timer firing, and they reproduce on localhost precisely because the wire is not the cause. Every socket call and flag below was checked against the Linux man pages (`socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `connect(2)`, `send(2)`, `setsockopt(2)`, `tcp(7)`) on this machine.

43.1 The socket API and the connection lifecycle

A socket is created with `socket(domain, type, protocol)`, which returns a descriptor. The *domain* selects the address family (`AF_INET` for IPv4, `AF_INET6` for IPv6); the *type* selects the transport semantics — `SOCK_STREAM` for a TCP byte stream, `SOCK_DGRAM` for UDP datagrams. From there the server and client follow different sequences. A **server** runs four calls: `bind` attaches the socket to a local address and port so clients can find it; `listen` marks it passive and gives it a queue of pending connections (the *backlog*); and `accept` blocks until a client connects, then returns a *new* descriptor for that one connection — the listening socket stays open to accept the next, while the returned socket carries this client's bytes. A **client** runs one: `connect` to the server's address, which does not return until the connection is established (or fails). Establishing it is TCP's **three-way handshake** (RFC 793): the client sends a `SYN` (synchronise) segment, the server replies `SYN+ACK`, the client replies `ACK`, and now both sides agree on starting sequence numbers and the channel is open — one full round trip of latency spent before a single byte of data moves, which is why connection setup is a cost you pay per connection and why long-lived or pooled connections matter. Once open, both sides call `read/recv` and `write/send` on their connected socket exactly as on a pipe. Closing is also negotiated: `close` starts a `FIN/ACK` exchange, and the side that closes first (the *active* close) leaves its socket in the **TIME_WAIT** state for twice the maximum segment lifetime, holding the port so that stray delayed segments from the old connection cannot be mistaken for a new one — the reason a just-killed server sometimes cannot immediately rebind its port without `SO_REUSEADDR`. The lifecycle, then, is: create, (server) bind/listen/accept or (client) connect, exchange bytes, close — and every step but the byte exchange is machinery TCP runs to turn two unreliable hosts into one reliable pipe.

> **QUICK CHECK**
>
> Name the four calls a TCP server makes before it can exchange bytes, and the one a client makes. What does `accept` return, and why is it a *different* descriptor from the listening socket? (Answer after the next section.)

43.2 TCP: a reliable, ordered byte stream — with no message boundaries

TCP (RFC 793) takes two hosts connected by a network that can lose, reorder, duplicate, and delay packets, and presents the application a channel that behaves as if none of that happened: a **reliable, ordered stream of bytes**. It achieves this with machinery the application never sees. Every byte is numbered by a **sequence number**; the receiver returns **acknowledgements** for bytes received, and the sender **retransmits** anything not acknowledged within a timeout — that is the reliability. The receiver uses the sequence numbers to deliver bytes in order even if packets arrive out of order — that is the ordering. A **sliding window** advertised by the receiver bounds how much unacknowledged data the sender may have outstanding, so a fast sender cannot overrun a slow receiver — that is **flow control** — and a separate **congestion-control** algorithm slows the sender when the network itself is loaded, the problem RFC 896 was written to solve. All of this is why TCP is the default: you get a pipe that just works. But the abstraction has one edge that trips nearly every beginner, and it follows directly from "stream of bytes": **TCP does not preserve message boundaries.** If the sender makes three `write` calls of 100 bytes, the receiver may get all 300 in a single `read`, or 60 then 240, or one byte at a time — TCP guarantees the bytes arrive in order, not that they arrive grouped the way they were sent. The transport is free to coalesce small writes into one segment (indeed Nagle's algorithm, §43.4, tries to) and to split large ones across segments, and a `read` returns whatever bytes have arrived at that instant (Chapter 40), which need not align with any `write`. The consequence is a rule: **if your protocol has messages, you must frame them yourself** — prefix each with a length, or use a delimiter, and loop `read` until a whole message is assembled. Assuming "one `send`, one `recv`" is the most common networking bug there is; it appears to work in testing (small messages on a fast link usually do arrive in one piece) and fails in production under load or across a real network, which is the same common-case-testing trap that runs through this whole part.

> **QUICK CHECK**
>
> A sender issues three 100-byte `write` calls on a TCP socket. How many bytes might a single `read` on the other end return, and why does TCP not promise you get them in three pieces of 100? What must a protocol with real messages do about it? (Answer below the communicate box.)

43.3 UDP: datagrams without the promises

UDP (RFC 768) is the other transport, and it is almost the complement of TCP: it keeps the one guarantee TCP drops and drops every guarantee TCP keeps. UDP is **connectionless** — there is no handshake; you create a `SOCK_DGRAM` socket and `sendto` a datagram to an address, and it is sent — and it is **message-oriented**: each `sendto` is one datagram, and the receiver's `recvfrom` returns exactly one datagram per call, boundaries preserved. What you give up is everything TCP's machinery provided: UDP does **not** guarantee delivery (a lost datagram is simply gone — no retransmission), does **not** guarantee ordering (datagrams can arrive in any order), does **not** provide flow or congestion control (send too fast and you overrun the receiver or the network, and nothing slows you down), and offers only a weak optional checksum. Its header is a mere 8 bytes against TCP's 20-byte minimum, and it costs no setup round trip. So UDP is the right tool exactly when TCP's guarantees are either unneeded or actively in the way: DNS queries (one small request, one small reply — a whole TCP handshake would cost more than the query); real-time audio and video and games (a late packet is worse than a lost one, so TCP's retransmit-and-deliver-in-order would *add* harmful latency — you would rather skip the lost frame); and modern protocols such as QUIC that build their own reliability, ordering, and congestion control on top of UDP because they want control TCP's fixed implementation does not give. The trade is explicit and total: with UDP you get boundaries, low overhead, and no waiting, and in exchange you either do not need reliability and ordering, or you build exactly the version of them you want. Choosing between them is therefore not about speed in the abstract but about which guarantees your application needs and which it would rather implement or skip.

THE SAME CHOICE THREE WAYS: TCP GIVES YOU THE GUARANTEES; UDP GIVES YOU THE BOUNDARIES AND THE CONTROL

1. In words. TCP turns an unreliable network into a reliable, ordered byte stream — at the cost of a setup round trip, per-byte sequencing and acknowledgement, retransmission delay, and no message boundaries. UDP hands you the raw datagram: message boundaries preserved, no setup, minimal header — and no delivery, ordering, flow, or congestion guarantee at all. TCP when you want a pipe that just works; UDP when you need boundaries and low latency and will supply (or skip) reliability yourself.

2. As a picture. Figure 43.1: on the left, the TCP three-way handshake (SYN, SYN-ACK, ACK) then a numbered byte stream where three writes coalesce into whatever segments the transport chooses; on the right, UDP, where each `sendto` is one datagram delivered whole or not at all, in any order, with no handshake.

3. As numbers/contrast. TCP header minimum 20 bytes and one RTT of handshake before any data; UDP header 8 bytes and zero setup. And the guarantee has a price measured in §43.4: a small-write request/response on default TCP can stall **~40 ms** per round on the interaction of Nagle's algorithm with delayed acknowledgements — versus **~0.09 ms** with `TCP_NODELAY` — an artifact of TCP's machinery that UDP, having no such machinery, cannot produce.

*(Answer to the §43.1 quick check: a TCP server calls `socket`, `bind`, `listen`, and `accept`; a client calls `socket` then `connect`. `accept` returns a **new** descriptor dedicated to the one accepted connection, while the original listening socket stays open to accept further clients — so a server holds one listening socket plus one connected socket per client, which is exactly the set of descriptors the Chapter 42 event loop watches.)*

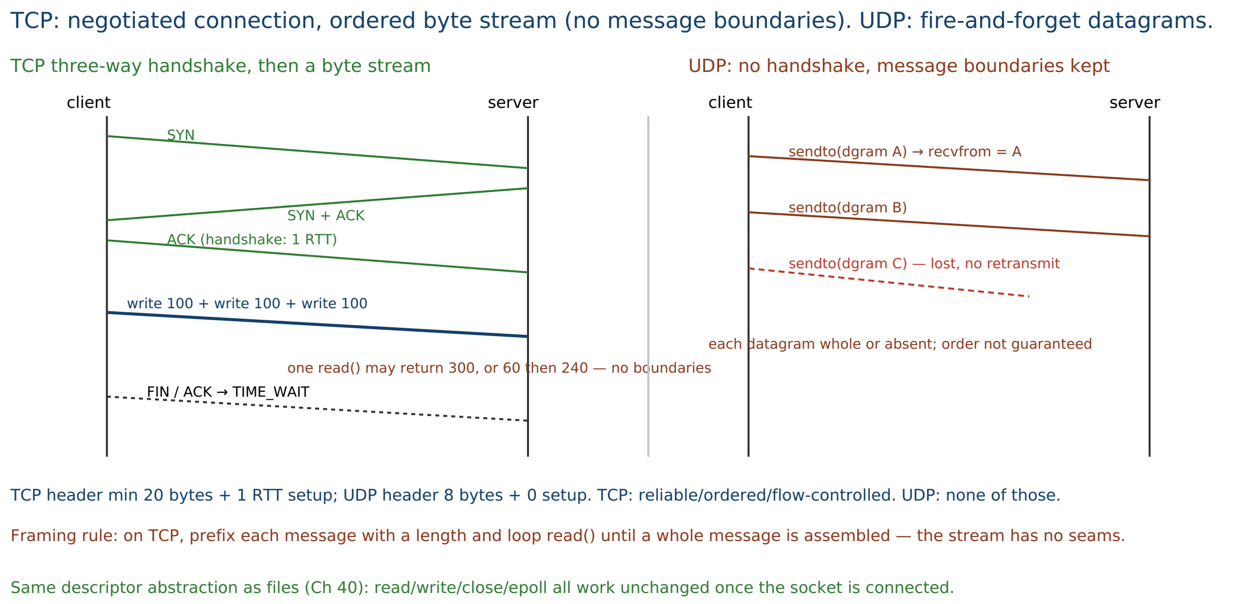


Figure 43.1: TCP (left) negotiates a connection with a three-way handshake, then delivers a reliable ordered byte stream in which message boundaries are *not* preserved — you must frame messages yourself. UDP (right) sends independent datagrams with boundaries intact but no delivery, ordering, or congestion guarantee. TCP costs a setup RTT and a 20-byte header; UDP costs nothing to set up and an 8-byte header.

43.4 Small writes, Nagle's algorithm, and the delayed-ACK stall

TCP's default configuration hides a latency trap that is invisible until it is catastrophic, and it is worth building precisely because it teaches how two independent optimisations can collide. **Nagle's algorithm** (RFC 896) reduces the number of tiny packets on the wire: when an application writes a small chunk, TCP will *hold* it rather than send a nearly-empty segment, waiting either until enough data accumulates to fill a segment or until all previously sent data has been acknowledged — the goal being to coalesce a flurry of small writes into fewer, fuller packets, which is good for the network. Independently, TCP's receiver uses **delayed acknowledgements** (RFC 1122): rather than send a bare ACK for every segment, it waits briefly (up to a fixed timer — about 40 ms on Linux) hoping to piggyback the ACK on a data segment it is about to send back, or to acknowledge two segments at once — which also reduces packet count. Each optimisation is sensible alone. Together, on a request/response pattern with small writes, they deadlock into a timer: the client writes a request in pieces; Nagle holds the second piece because the first is not yet acknowledged; the server cannot acknowledge yet because delayed-ACK is waiting to piggyback the ACK on the response — but the server has no response to send because it has not received the whole request, which is stuck in the client's Nagle buffer. Nothing moves until the delayed-ACK timer expires (~40 ms), forcing the ACK, which releases Nagle, which sends the rest. Every round trip pays that timer. Measured here on loopback, with a client sending a 40-byte request as two small writes and waiting for a one-byte reply:

```
write-write-read round trip on loopback, mean per round (min of several trials):
  Nagle ON  (TCP default)   :   ~43,800 us/round   (43.8 ms  — the delayed-ACK timer)
  TCP_NODELAY (Nagle off)   :       ~90 us/round
  ratio: ~488x
```

The number is unambiguous: the default connection spends about **43.8 ms** per round trip, essentially all of it the ~40 ms delayed-ACK timer firing, while the same code with Nagle disabled runs in **~90 µs** — a ratio near **500×**, on a loopback link with no real latency at all. The fix is one socket option: `setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof one)` disables Nagle's algorithm on that socket, so small writes are sent immediately. The lesson is not "always set `TCP_NODELAY`" as a superstition; it is to understand the interaction. For latency-sensitive request/response protocols (RPC, databases, interactive services) with small messages, `TCP_NODELAY` is almost always correct, because you care about round-trip latency far more than about saving a few packets. For bulk transfer where you write large buffers, Nagle never engages (your segments are already full) and disabling it changes nothing. The deeper fix, when you control the write pattern, is to avoid many small writes in the first place — assemble the whole request in one buffer and issue one `write` — which sidesteps the interaction regardless of the option. What you must never do is ship a request/response protocol without knowing which of these you did, and then wonder why every call takes 40 ms.

TRAP: THE 40-MILLISECOND RPC — NAGLE'S ALGORITHM MEETS DELAYED ACK

A request/response protocol that writes its request in more than one small `write` and then blocks for the reply can stall for the delayed-ACK timer (~40 ms on Linux) on *every single call*, on the fastest possible link, because Nagle is holding the client's last small segment until the server acknowledges the first, and the server's delayed-ACK is holding that acknowledgement waiting for a response the server cannot produce until it has the whole request. The trap is that it is invisible in the obvious tests: a single small message often fits in one segment and never triggers the hold, and throughput benchmarks that stream large buffers never engage Nagle at all, so the bug surfaces only as a suspiciously round **40 ms** (or a small multiple) added to interactive latency in production — a number so specific it is a fingerprint. The tell is exactly that: per-request latencies clustered near 40 ms with the CPU idle and the network unloaded means a timer, not work, is the cost. Two fixes, in order of preference. First, do not split a logical message across several small writes — build it in one buffer and write once, which never triggers Nagle regardless of the option. Second, for small-message request/response protocols, set `TCP_NODELAY` to disable Nagle on the socket. Do *not* reach for `TCP_NODELAY` reflexively on bulk-transfer sockets, where Nagle does no harm and coalescing helps; and understand that the real defect is a chatty write pattern colliding with two reasonable protocol optimisations, not either optimisation being wrong. Diagnosing a fixed-latency stall as a timer rather than as slow code is the systems-level move here.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate reports: "Our new internal RPC service is weirdly slow — every call takes about 40 ms even though the servers are in the same rack and CPU is near zero. Under load throughput is fine, it is just that each individual call is slow. Maybe the network is bad?" Cover the paragraph and respond in four or five sentences, drawing on §43.2 and §43.4.

Model answer. "A flat ~40 ms with idle CPU and an unloaded network is not a slow network — it is a timer, and the classic one is TCP's delayed-ACK interacting with Nagle's algorithm. If the client writes its request in more than one small `write` and then waits for the reply, Nagle holds the last piece until the first is acknowledged, and the server's delayed-ACK holds that acknowledgement for ~40 ms because it has no reply to piggyback it on yet — so every call eats the delayed-ACK timer, which is exactly the 40 ms you are seeing. Throughput looks fine because under load the pipeline stays full and the timer overlaps other work; it is the isolated round-trip latency that shows it. The clean fix is to build each request in one buffer and issue a single `write` so Nagle never engages; if the write pattern is hard to change, set `TCP_NODELAY` on the socket to disable Nagle for this small-message protocol. Let me confirm with a capture that the request is going out in two segments — that will nail it in a minute." Naming the timer, explaining the Nagle/delayed-ACK collision, and reaching first for the one-write fix rather than a reflexive option flip is the senior register.

*(Answer to the §43.2 quick check: a single `read` after three 100-byte writes might return anywhere from 1 to 300 bytes — all 300 at once, 60 then 240, one byte at a time, any split. TCP guarantees only that the bytes arrive **in order and complete**, not that they arrive grouped as they were written, because TCP is a byte stream and the transport freely coalesces and splits writes into whatever segments it chooses. A protocol with real messages must therefore **frame** them itself — length prefix or delimiter — and loop `read` until a whole message has been assembled.)*

43.5 What to carry forward

A **socket** is a descriptor that names a network endpoint, so once connected it obeys every I/O lesson of this part — `read`, `write`, `close`, `O_NONBLOCK`, and `epoll` (Chapter 42) all apply unchanged; what is new is the setup. A **server** runs `socket→bind→listen→accept`, where `accept` returns a fresh per-connection descriptor and the listening socket stays open; a **client** runs `socket→connect`, whose three-way handshake spends one round trip before any data moves, and the active closer pays **TIME_WAIT** afterward (§43.1). **TCP** (`SOCK_STREAM`, RFC 793) turns an unreliable network into a reliable, ordered byte stream via sequence numbers, acknowledgements, retransmission, and a sliding window for flow control — but it preserves **no message boundaries**, so a protocol with messages must frame them and loop `read` until each is complete (§43.2). **UDP** (`SOCK_DGRAM`, RFC 768) keeps message boundaries and adds nothing else: no delivery, ordering, flow, or congestion guarantee, an 8-byte header, and no setup — the right choice when TCP's guarantees are unneeded (DNS) or harmful (real-time media) or when you will build your own (QUIC) (§43.3). And TCP's default configuration hides the **Nagle–delayed-ACK** collision: a small-write request/response can stall on the ~40 ms delayed-ACK timer every round — measured here at ~43.8 ms versus ~90 µs with `TCP_NODELAY`, a ~500×

artifact of two reasonable optimisations meeting — fixed by writing each message once, or by disabling Nagle for small-message protocols (§43.4).

This is the endpoint the event loop of Chapter 42 has been watching, now built and understood: a socket you can create, connect, feed, and frame, over a transport whose guarantees you can name. Two threads of this part remain, and both are about spending the descriptor's bytes more cheaply. The next chapter returns to the *readiness*-versus-*completion* question Chapter 42 opened and follows it to its conclusion — **io_uring**, an interface where you submit a batch of operations and are told when each has *finished*, collapsing the per-operation system-call boundary this part has paid at every step and extending the asynchronous model from sockets to disk. The chapter after removes the last copy: the data path from a file to a socket still runs every byte through a user-space buffer, and **zero-copy** transfer (`sendfile`) eliminates it. The connection is open and its bytes are ordered; what remains is to move them with the fewest crossings and copies the machine allows.

CAN YOU… WITHOUT LOOKING?

- List the four calls a TCP server makes and the one a client makes, and say what `accept` returns.
- Describe the three-way handshake and why connection setup costs a round trip.
- Explain why TCP preserves no message boundaries, and what a protocol with messages must therefore do.
- State what UDP keeps and what it gives up, and name two workloads where UDP is the right choice.
- Explain how Nagle's algorithm and delayed ACK combine to add ~40 ms per round trip, and two ways to fix it.
- Say why the Nagle stall reproduces on loopback with no real network latency.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What are the server-side and client-side socket-call sequences, and what does `accept` return? (§43.1)
2. **R2.** What does the three-way handshake accomplish, and what is TIME_WAIT for? (§43.1)
3. **R3.** Why does TCP preserve no message boundaries, and what must a message protocol do about it? (§43.2)
4. **R4.** What does UDP guarantee and not guarantee, and when is it the right transport? (§43.3)
5. **R5.** How do Nagle and delayed ACK collide, and what are the two fixes? (§43.4)

FURTHER READING

- **W. Richard Stevens, Bill Fenner & Andrew M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. (Addison-Wesley, 2003), ISBN 978-0-13-141155-5.** The definitive reference for this chapter: Chapters 4–5 (elementary TCP sockets and the connection lifecycle), Chapter 7 (socket options including `TCP_NODELAY`), and Chapter 8 (elementary UDP sockets). First author W. Richard Stevens; edition and ISBN verified via the publisher.
- **Michael Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), ISBN 978-1-59327-220-3.** Chapters 56–61 cover sockets end to end — the API, TCP stream sockets, UDP datagram sockets, and socket options — as implemented on Linux, the system these programs run on. First author Michael Kerrisk; edition and ISBN verified via the publisher.
- **J. Postel, RFC 793, "Transmission Control Protocol" (September 1981), and RFC 768, "User Datagram Protocol" (August 1980).** The original specifications of the two transports of §43.2–§43.3: the byte-stream service model and its handshake, and the datagram service. RFC 793 has since been consolidated and obsoleted by RFC 9293 (2022); both are worth reading. Cited by RFC number, author, and date via the RFC Editor.
- **J. Nagle, RFC 896, "Congestion Control in IP/TCP Internetworks" (January 1984), and R. Braden (ed.), RFC 1122, "Requirements for Internet Hosts — Communication Layers" (October 1989).** RFC 896 defines the small-packet algorithm that bears Nagle's name; RFC 1122 specifies delayed acknowledgements — the two mechanisms whose collision §43.4 measures. Cited by RFC number, author, and date via the RFC Editor.

Exercises

43.1 WARM-UP List, in order, the four system calls a TCP server makes before it can exchange bytes with a client, and the one call the client makes. In one sentence, say what `accept` returns.

43.2 WARM-UP What socket *type* selects TCP, and which selects UDP? Name one guarantee TCP provides that UDP does not, and one thing UDP preserves that TCP does not.

43.3 WARM-UP In one sentence, state which socket option disables Nagle's algorithm, and on what kind of protocol you would set it.

43.4 CORE The byte stream. (a) A sender makes three `write` calls of 100 bytes each on a TCP socket. Give three different ways the bytes might be split across `read` calls on the receiver, and say which of them TCP permits. (b) State precisely what TCP *does* guarantee about those 300 bytes. (c) Describe a concrete framing scheme that lets the receiver recover the three original messages, and the receive loop it implies.

43.5 CORE The connection lifecycle. (a) Walk through the three-way handshake segment by segment and say what each side knows after it. (b) Why does connection setup cost a full round trip, and what does that imply for a client that opens a fresh connection per request? (c) What is TIME_WAIT, which side enters it, and why does it exist?

43.6 CORE TCP versus UDP by workload. For each, say which transport you would choose and why: (a) a bulk file transfer; (b) a DNS query and reply; (c) live voice-over-IP; (d) a bank transaction; (e) a new protocol that wants its own congestion control and 0-RTT reconnection. For (c) and (e), explain what TCP would *add* that you do not want.

43.7 COST The Nagle stall. (a) Explain step by step why a write-write-read pattern deadlocks Nagle against delayed ACK until the ~40 ms timer fires. (b) Using the measured numbers (~43.8 ms with Nagle, ~90 µs with `TCP_NODELAY`), a service does 25 sequential such round trips per user request; estimate the added latency per user request in each case. (c) Explain why a throughput benchmark that streams large buffers would completely miss this bug.

43.8 FADED *Worked, with the last step for you.* A team's RPC library shows ~40 ms per call in production between two hosts in the same rack, with CPU idle; a load test on the same code showed fine throughput.
Step 1 (given): the client serialises each request by writing a 4-byte length prefix, then the body, as two separate `write` calls, then reads the reply.
Step 2 (given): the socket uses TCP defaults, so Nagle is on.
Step 3 (given): the server delays its acknowledgement of the length-prefix segment, waiting to piggyback it on the reply, which it cannot send until it has the whole request.
Step 4 (you): Explain why every call stalls ~40 ms, why the throughput load test missed it, and give the two fixes in order of preference.

43.9 MIXED For each statement, say safe/correct or buggy and why: (a) “I called `send` once with my whole message, so the peer's `recv` will return it in one call.” (b) “I set `TCP_NODELAY` on my bulk-upload socket to make it faster.” (c) “I use UDP and rely on it to redeliver any datagram that gets lost.” (d) “I read into a buffer in a loop until I have a full length-prefixed message before parsing it.” (e) “My server accepts a connection and keeps using the listening socket to talk to that client.”

43.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “A socket is a special object that does not support `read`/`write`.” (b) “UDP guarantees datagrams arrive in the order they were sent.” (c) “TCP's three-way handshake costs one round trip before any data is sent.” (d) “Disabling Nagle speeds up a socket that only ever sends large buffers.” (e) “On a TCP connection, one `write` always corresponds to exactly one `read` on the peer.”

43.11 STRETCH Synthesize the chapter. (a) Explain why "a socket is just a descriptor" is both true (the I/O API applies unchanged) and incomplete (setup and transport semantics are new) — what does each half buy you? (b) TCP and UDP are often framed as "reliable versus fast," which is misleading; give the accurate framing in terms of *which guarantees you get versus build*, and show how QUIC-on-UDP proves the point. (c) The Nagle stall comes from two individually correct optimisations. Argue why this is a recurring shape in systems (local optima composing into a global pathology) and what it implies about testing.

43.12 LAB On your own machine (a C toolchain): (a) Write a TCP echo server and client over loopback; confirm that sending a message in two small `write` calls and reading the reply is subject to the Nagle stall, and measure the per-round latency with Nagle on and with `TCP_NODELAY` set. (b) Demonstrate the byte-stream property: have the client send three distinct small messages back-to-back and show that a single `read` on the server may return more than one of them; then add a length-prefix framing loop that recovers them correctly. (c) State your toolchain and CPU, note the times are wall-clock (no hardware counters), and explain why the ~40 ms you measure is a protocol timer rather than network latency, given that loopback has essentially none.

Chapter 44 — io_uring and Completion-Based Asynchronous I/O

> **Before you start:** Chapter 42 (readiness notification — `epoll` tells you *which descriptors are ready*, then you do the operation), Chapter 41 (a disk read that misses the page cache blocks on the device — and a regular file has no useful readiness signal), Chapter 40 (the I/O syscalls), Chapter 13 and Chapter 42 (a system call costs a few hundred nanoseconds to enter the kernel). · Chapter 42 asked the kernel *which* descriptors are ready and then still made a syscall per operation. This chapter changes the question one more time: submit a batch of operations and be told when each has *finished* — the completion model, and its Linux realisation, `io_uring`.

> **BEFORE YOU READ ON**
>
> From memory: (1) Chapter 42 — `epoll` reports a socket *ready*, after which you call `read` yourself; how many kernel crossings is one handled operation, and what does readiness tell you that a regular disk file cannot answer? (2) Chapter 41 — when a `read` misses the page cache, what does the calling thread do, and why can't `epoll` help you avoid that wait on a file? (3) *Predict:* a program must issue 65,536 small reads. Version A calls `pread` 65,536 times. Version B submits them in batches of 256 through a shared ring and reaps completions. Two questions: how many kernel crossings does each pay? And on data already in the page cache, will version B's *wall-clock* time be dramatically lower, about the same, or is the syscall count the real story? Write your guesses; §44.3 answers them.

Chapter 42 solved *waiting*: instead of a thread per blocked call, one thread asks `epoll` which descriptors are ready and handles only those. But readiness has two limits this chapter removes. First, it is still one system call per operation: `epoll_wait` tells you a socket is readable, and then you cross the kernel boundary again to actually `read` it — so a busy server pays the few-hundred-nanosecond syscall tax (Chapter 42) on every single operation, and at millions of operations per second that boundary becomes the cost. Second, and deeper, readiness does not work for **disk**. A regular file is *always* reportable as ready — there is no “the data has arrived” event for a file the way there is for a socket — yet a `read` that misses the page cache still blocks the thread on the device for milliseconds (Chapter 41). `epoll` has nothing useful to say about a file; the whole readiness model assumes an operation that *would* block until a descriptor becomes ready, and disk I/O simply is not shaped that way. The answer to both is to stop notifying *readiness* and start notifying *completion*: you hand the kernel a description of the operation you want done, the kernel does it (blocking on the device on its own, not on your thread), and later tells you it is *finished*, with the result. This chapter builds that model in five steps: readiness versus completion and why disk forces the change (§44.1); the `io_uring` interface — two shared ring buffers between application and kernel (§44.2); why the boundary shrinks, with the syscall count measured (§44.3); the hazard the completion model introduces — asynchronous buffer lifetime (§44.4); and what to carry forward (§44.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35, 20 hardware threads), timed with `clock_gettime(CLOCK_MONOTONIC)`; times are wall-clock and a hardware-counter profiler (`perf`) is *not* installed. The kernel here is 6.x on WSL2, which supports `io_uring`; the helper library `liburing` is *not* installed, so the programs drive the interface through the raw system calls `io_uring_setup(2)` and `io_uring_enter(2)` directly — setting up the shared rings by hand with `mmap` — which is exactly what makes the mechanism visible. A minimal `IORING_OP_NOP` and batched `IORING_OP_READ` submissions were run here to confirm the interface works and to count kernel crossings; the honest wall-clock result of §44.3 is reported as measured, including where it is *not* a speed-up.

44.1 Readiness versus completion, and why disk forces the change

There are two ways for the kernel to help a program do asynchronous I/O, and the difference is the whole chapter. The **readiness** model (Chapter 42: `select`, `poll`, `epoll`) answers the question "which descriptors *can* I operate on without blocking?" — the kernel watches, and when a socket has data or space you go and do the `read` or `write` yourself. The **completion** model answers a different question: "here is the operation I want; tell me when it is *done*." You submit a read — descriptor, buffer, length, offset — and the kernel performs it, waiting on the device itself if it must, and later hands you a completion record saying how many bytes it read (or what error). The two are not just cosmetically different. Readiness fits sockets, where "ready" is a real, observable state and the operation, once ready, completes instantly from a buffer. It does *not* fit **disk files**, and this is the crucial point: a regular file is always "ready" in the readiness sense — you can always *attempt* the read — but the attempt can still block for milliseconds on a cache miss (Chapter 41), and there is no event to wait for that means "now the read will not block." So `epoll` cannot make disk I/O asynchronous; historically Linux had a separate, awkward, partial mechanism (POSIX AIO) for that, and sockets and files needed different tools. The completion model unifies them, because "tell me when it is done" is meaningful for *any* operation — a socket read, a disk read, an `fsync`, an `accept`, a `send` — the kernel just does the work and reports completion. That generality is why the completion interface (`io_uring` on Linux; `IOCP` on Windows long before) is the model modern high-performance I/O converges on: one asynchronous interface for the whole descriptor world, sockets and disk alike.

> **QUICK CHECK**
>
> In one sentence each: what does the readiness model ask the kernel, and what does the completion model ask instead? Why can `epoll`'s readiness model not make a disk-file `read` asynchronous? (Answer after the next section.)

44.2 The io_uring interface: two shared rings

`io_uring` (merged into Linux 5.1 in 2019) implements the completion model with a design built to make the kernel boundary cheap: two ring buffers, shared directly between the application and the kernel in memory, so that submitting work and collecting results need not copy arguments across the boundary or, in the limit, make a system call at all. You create the instance with `io_uring_setup(entries, params)`, which returns a ring file descriptor and, through its `params`, the offsets of the shared structures; you then `mmap` them into your address space. There are two queues. The **submission queue** (SQ) is an array of **submission queue entries** (SQEs); each SQE describes one operation — an `opcode` (`IORING_OP_READ`, `IORING_OP_WRITE`, `IORING_OP_SEND`, `IORING_OP_ACCEPT`, `IORING_OP_FSYNC`, and dozens more), the descriptor, a buffer address, a length, an offset, and a 64-bit `user_data` field you choose and the kernel echoes back so you can tell which completion belongs to which request. The **completion queue** (CQ) is an array of **completion queue entries** (CQEs); each CQE carries the `user_data` you set and a `res` field holding the operation's result — the byte count on success, or a negative error code — exactly the value the synchronous call would have returned. The flow is: fill one or more SQEs, advance the submission ring's tail, and call `io_uring_enter(fd, to_submit, min_complete, flags)` *once* to submit that whole batch and optionally wait for completions; then walk the completion ring, reading a CQE per finished operation, and advance its head. The decisive property is that *one* `io_uring_enter` can submit many operations and reap many completions — the system-call boundary is crossed per *batch*, not per operation. And it can be pushed further: in **polled submission** mode (`SQPOLL`) a kernel thread watches the submission ring, so the application submits work by writing shared memory with *no system call at all*. The rings are the mechanism; batching across them is the payoff.

> **QUICK CHECK**
>
> Name the two rings `io_uring` shares between application and kernel, and what one entry in each carries. What does the `user_data` field on a submission accomplish, and how many system calls does it take to submit a batch of 100 reads and collect their results? (Answer below the communicate box.)

44.3 Why the boundary shrinks — the syscall count, measured

The point of the shared rings is to stop paying the kernel-entry tax per operation. To see it plainly, here is the same work done two ways on this machine: 65,536 reads of 4 KiB each (256 MiB total) from a file already warm in the page cache, issued once as 65,536 synchronous `pread` calls, and once as `io_uring` submissions batched 256 at a time. The metric that matters first is *kernel crossings*:

```
65,536 random 4 KiB reads from the page cache (this machine):
  synchronous pread loop      :  65,536 syscalls   ~87-94 ms
  io_uring, batched QD=256    :     256 io_uring_enter calls   ~77-104 ms
  kernel crossings: 256x fewer     wall-clock: roughly break-even (~0.8-1.2x, noisy)
```

Two things must be read honestly here. The first is a clean, large result: `io_uring` cut the number of system calls by **256×**, from 65,536 to 256, because each `io_uring_enter` submitted and reaped a whole batch. That is the architectural win, and at a few hundred nanoseconds per crossing (Chapter 42) it is real work removed. The second is a result the book will not inflate: on this workload the **wall-clock time was roughly break-even**, not a speed-up. The reason is exactly the cost model of this part. These reads *hit* the page cache, so each one is a 4 KiB memory copy inside the kernel — and that copy, done 65,536 times, dominates the total, dwarfing the syscall-entry cost that `io_uring` removed. When the thing you eliminated (the boundary crossing) is small next to the thing you did not (the per-read copy), collapsing the crossings barely moves the clock. This is the chapter's most important lesson and it is the part's recurring one: **a technique's benefit is only as large as the cost it removes is of the total.** `io_uring`'s boundary-batching is decisive precisely where the boundary *is* the cost — enormous numbers of tiny operations (high-IOPS storage, packet processing) where per-syscall overhead dominates, and, above all, cases the wall-clock here cannot show at all: genuine *device* I/O, where the kernel does the blocking on its own threads while yours submits the next batch and never sleeps — the completion model's real prize is that a single thread keeps thousands of disk and network operations in flight without a thread blocked on any of them, which readiness could never do for disk (§44.1). The syscall count is the honest, measured headline; the throughput win is workload-dependent, and you must measure it, not assume it.

THE SAME IDEA THREE WAYS: DON'T CROSS THE KERNEL BOUNDARY PER OPERATION — SUBMIT A BATCH, REAP COMPLETIONS

1. In words. Readiness (Chapter 42) still costs one syscall per operation and cannot describe disk. The completion model lets you submit many operations at once and be told when each finishes; `io_uring` realises it with two shared rings so a single `io_uring_enter` submits and reaps a whole batch — and the kernel, not your thread, does any blocking on the device.

2. As a picture. Figure 44.1: on the left, the synchronous loop — one kernel crossing per read, thousands of them; on the right, the application filling many SQEs in the shared submission ring, one `io_uring_enter` for the batch, and completions appearing in the shared completion ring, each tagged with its `user_data`.

3. As numbers/contrast. 65,536 cache-hot 4 KiB reads: 65,536 syscalls the synchronous way, **256** the `io_uring` way (256× fewer crossings) — but on cache-warm data the per-read copy dominates, so wall-clock is **roughly break-even** here (§44.3). The win becomes a real speed-up when the boundary is the cost (huge counts of tiny ops) or when the operations touch a real device, where the kernel blocks and your thread does not.

(Answer to the §44.1 quick check: the readiness model asks the kernel “which descriptors can I operate on without blocking?” and then you do the operation yourself; the completion model asks “do this operation and tell me when it is finished.” `epoll` cannot make a disk-file `read` asynchronous because a regular file is always “ready” in the readiness sense — you can always attempt the read — yet the attempt still blocks on the device on a cache miss, and there is no readiness event that means “this read will not block,” so readiness has nothing to watch for a file.)

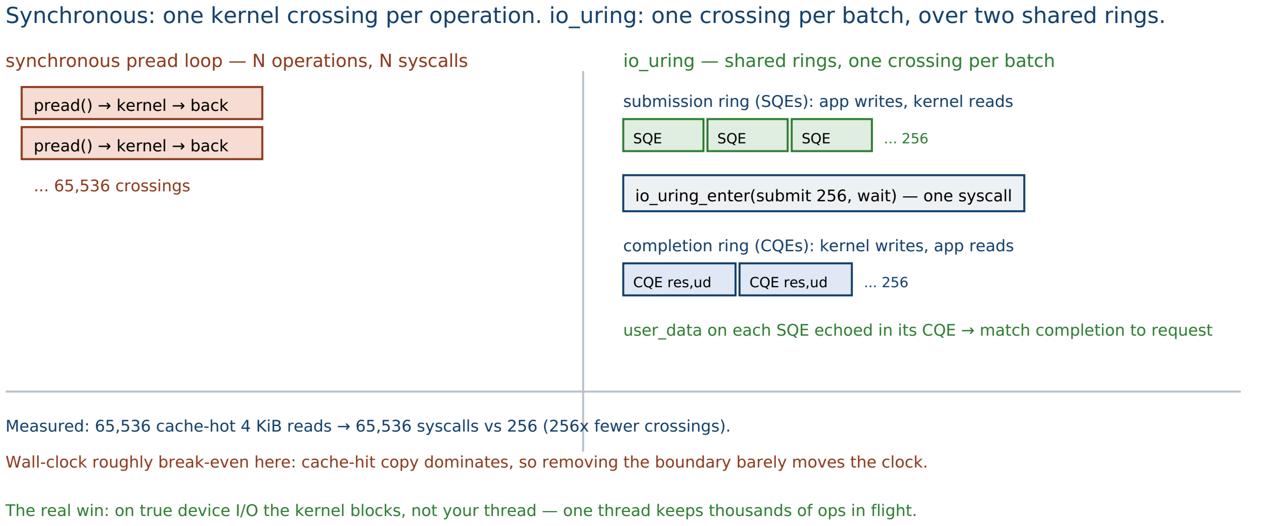


Figure 44.1: The synchronous loop (left) crosses the kernel boundary once per operation. `io_uring` (right) shares a submission ring and a completion ring with the kernel, so one `io_uring_enter` submits and reaps a whole batch — 256× fewer crossings for 65,536 reads. On cache-warm data the per-read copy dominates and wall-clock is roughly break-even; the model's real payoff is on device I/O, where the kernel does the blocking and one thread keeps thousands of operations in flight.

44.4 The completion model's hazard: asynchronous buffer lifetime

The completion model buys its power by decoupling the moment you *submit* an operation from the moment it *completes*, and that gap is where its characteristic bug lives. When you fill an SQE for a read, you give the kernel the address of a buffer to read into; when you fill one for a write or send, you give it the address of a buffer to read *from*. In the synchronous world those buffers are only in use for the duration of the call, which returns before your next line runs, so their lifetime is trivial. Under `io_uring` the kernel may touch that buffer at *any* time between your submission and the matching completion — which can be milliseconds later, while your thread has run on to submit more work or handle other completions. The rule this imposes is absolute and easy to violate: **every buffer referenced by an in-flight SQE must remain valid and unchanged until its CQE arrives.** Free it, let it go out of scope, reuse it for a different operation, or overwrite it with the next request, and you have handed the kernel a dangling or racing pointer — a read that lands in freed memory, a send that transmits half-overwritten garbage, an intermittent corruption that appears only under the timing that makes the completion late. It is the asynchronous version of a use-after-free, and it is invisible in exactly the tests where completions come back fast: a lightly loaded run completes each operation almost immediately, so the buffer is still alive by luck, and the bug hides until load or a slow device stretches the gap. The same decoupling creates a second discipline: completions arrive in *whatever order the kernel finishes*, not submission order, so you must use the `user_data` tag to match each completion to its request and its buffer — assuming completions come back in order is a sibling bug. Both hazards are the price of asynchrony made explicit: the interface that lets one thread keep thousands of operations in flight also makes you, not the call stack, responsible for keeping each operation's memory alive across the gap.

TRAP: THE BUFFER FREED BEFORE ITS COMPLETION — ASYNCHRONOUS USE-AFTER-FREE

Under a completion interface, submitting an operation does not finish it: the kernel may read from or write to the buffer you named at any point up to the completion event, potentially milliseconds later. The trap is to treat submission like a synchronous call and reuse, free, or let the buffer go out of scope as soon as `io_uring_enter` returns — which works perfectly whenever the completion happens to come back fast (light load, warm cache, fast device), and corrupts memory intermittently exactly when it does not (heavy load, cold cache, slow device stretch the submit-to-complete gap). Because the failure is timing-dependent it survives unit tests and casual load tests and surfaces as rare, unreproducible data corruption in production — the worst kind of bug. Two disciplines defend against it. First, tie each in-flight buffer's lifetime to its completion, not to the submitting call: keep it owned by the request state you look up via `user_data` when the CQE arrives, and only then free or reuse it (registered/fixed buffers make this ownership explicit). Second, never assume completion order — match every CQE to its SQE through `user_data`, because the kernel completes operations in whatever order they finish, not the order you submitted. The mental shift is the whole point of the chapter: a submitted operation is a promise outstanding, and everything it references must outlive the promise. Reasoning about a buffer's lifetime as if submission were completion is the completion-model version of a use-after-free, and it is the first bug every io_uring program must be designed against.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: “Let's rewrite our storage service's hot read path on `io_uring` — it's the fastest I/O interface Linux has, so we should get a big throughput win basically for free.” Cover the paragraph and respond in four or five sentences, drawing on §44.1, §44.3, and §44.4.

Model answer. “`io_uring` is the right long-term direction, but let's be precise about *why* it wins, because ‘fastest, for free’ will burn us. Its concrete gains are two: it batches the kernel boundary — many operations per `io_uring_enter` instead of one syscall each — and it makes *disk* I/O truly asynchronous, so one thread keeps thousands of device operations in flight without blocking, which `epoll` cannot do for files. But the throughput win is workload-dependent: for reads that hit the page cache, the per-read copy dominates and batching the syscall barely moves the wall clock — I measured that as roughly break-even — so the payoff is real only where the syscall boundary or actual device latency is the bottleneck. And it is not free to write: a completion interface means every buffer must stay alive until its completion, or we get an asynchronous use-after-free that only shows under load. So let's prototype it on the path that is actually device-bound, measure against the current code, and design the buffer ownership carefully — not rewrite the cache-hot path expecting magic.” Naming the two real gains, refusing the free-lunch framing with a measured break-even, and flagging the buffer-lifetime hazard is the senior register.

*(Answer to the §44.2 quick check: the two rings are the **submission queue** (SQEs, which the application writes and the kernel reads) and the **completion queue** (CQEs, which the kernel writes and the application reads); an SQE describes one operation (opcode, descriptor, buffer, length, offset, `user_data`) and a CQE carries the result (`res`) and the echoed `user_data`. The `user_data` field lets you match each completion to the request that produced it, since completions can arrive out of order. Submitting 100 reads and*

*collecting their results takes **one** `io_uring_enter` — the boundary is crossed per batch, not per operation.)*

44.5 What to carry forward

There are two ways the kernel helps with asynchronous I/O, and the difference is the chapter. **Readiness** (Chapter 42: `epoll`) says which descriptors can be operated on without blocking, then you do the operation — one syscall per operation, and no help at all for disk, since a regular file is always "ready" yet a cache-miss `read` still blocks on the device (§44.1). **Completion** says "do this operation and tell me when it is done," which is meaningful for any operation — socket or disk — and so unifies the two worlds. **`io_uring`** (Linux 5.1, 2019) implements it with two memory-shared ring buffers: a submission queue of SQEs describing operations and a completion queue of CQEs carrying results, so one `io_uring_enter` submits and reaps a whole batch — and `SQPOLL` can remove even that (§44.2). Measured here, 65,536 cache-hot reads cost **256× fewer kernel crossings** through `io_uring` (256 syscalls versus 65,536) — but roughly **break-even wall-clock**, because on cache-warm data the per-read copy dominates the boundary cost that was removed; the throughput win is real where the boundary or the device is the bottleneck, and must be measured, not assumed (§44.3). And the completion model's power — decoupling submit from complete — creates its signature hazard: **every buffer must stay valid until its completion**, or you get an asynchronous use-after-free that hides until load stretches the gap, and completions must be matched to requests by `user_data` because they arrive out of order (§44.4).

This is where the I/O part's central thread — the cost of the kernel boundary, traced from the first `read` syscall of Chapter 40 through the readiness of Chapter 42 — reaches its conclusion: submit in batches, be told on completion, and let the kernel, not your thread, do the blocking. One cost remains, and it is the last one in the part. Even with the boundary batched away, every byte moved from a file to a socket still travels through a buffer in your process — read into user space, written back out — a copy the CPU makes that does no useful transformation. The final chapter removes it: **zero-copy** transfer with `sendfile` and `splice`, where the data goes from the page cache to the network card without ever entering your address space. The waiting is solved and the boundary is batched; the last thing to eliminate is the copy itself.

CAN YOU… WITHOUT LOOKING?

- State the question the readiness model asks the kernel and the different one the completion model asks.
- Explain why `epoll`'s readiness model cannot make disk-file I/O asynchronous.
- Describe the two `io_uring` rings and what one SQE and one CQE each carry.
- Say how many system calls it takes to submit and reap a batch of 100 operations, and why.
- Explain why 65,536 cache-hot reads were 256× fewer syscalls yet roughly break-even in wall-clock — and where io_uring does win.
- State the buffer-lifetime rule the completion model imposes and the bug that violating it causes.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does readiness ask versus completion, and why does disk force the change? (§44.1)
2. **R2.** What are the two rings, and what do SQEs and CQEs carry? (§44.2)
3. **R3.** How many syscalls submit a batch of N operations, and what did the measured 65,536-read comparison show — in syscalls and in wall-clock? (§44.3)
4. **R4.** Why was the wall-clock roughly break-even on cache-hot data, and where does `io_uring` actually win? (§44.3)
5. **R5.** What is the buffer-lifetime rule, why is the bug intermittent, and how do you match completions to requests? (§44.4)

FURTHER READING

- **Jens Axboe, "Efficient IO with io_uring" (kernel.dk, 2019).** The design document by `io_uring`'s author: the submission and completion rings, SQE and CQE layout, the `io_uring_setup`/`io_uring_enter` system calls, and the polled-submission mode — the primary reference for §44.2. Author Jens Axboe (also the author of the kernel implementation); a freely-available technical document cited by author, title, and year.
- **"Ringing in a new asynchronous I/O API," *LWN.net* (2019).** LWN's walkthrough of `io_uring` as it was merged into Linux 5.1, motivating the completion model against the limits of readiness and POSIX AIO that §44.1 draws on. Cited by title, venue, and year; LWN articles carry no DOI.
- **Michael Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), ISBN 978-1-59327-220-3.** Chapter 63 ("Alternative I/O Models") covers the readiness-based predecessors — `select`/`poll`/`epoll`, signal-driven I/O, and POSIX AIO — that motivate `io_uring`; note that `io_uring` postdates the book (2019 vs 2010), so it is context for §44.1, not a reference for the interface itself. First author Michael Kerrisk; edition and ISBN verified via the publisher.
- **W. Richard Stevens, Bill Fenner & Andrew M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. (Addison-Wesley, 2003), ISBN 978-0-13-141155-5.** Chapter 6's taxonomy of I/O models — blocking, non-blocking, multiplexing, signal-driven, and asynchronous (completion) — is the classic framing that §44.1's readiness-versus-completion distinction extends. First author W. Richard Stevens; edition and ISBN verified via the publisher.

Exercises

44.1 WARM-UP In one sentence each, state the question the readiness model asks the kernel and the question the completion model asks instead.

44.2 WARM-UP Name the two ring buffers `io_uring` shares with the kernel, and say which side (application or kernel) writes each.

44.3 WARM-UP How many `io_uring_enter` calls does it take to submit a batch of 200 operations and wait for their completions, and how does that compare to doing 200 synchronous `pread` calls?

44.4 CORE Readiness versus completion. (a) Explain why the readiness model needs one system call per operation even after `epoll_wait` returns. (b) A regular disk file is always reportable as “ready”; explain why that makes `epoll` useless for making a cache-missing `read` asynchronous. (c) Why is “tell me when it is done” a question that works for both sockets and disk, where “which are ready” does not?

44.5 CORE The rings. (a) Describe the fields of a submission queue entry and of a completion queue entry, and what `user_data` is for. (b) Walk through the steps to submit three reads and collect their results. (c) What does `SQPOLL` (polled submission) change about the number of system calls, and how?

44.6 CORE The measured result. (a) Why did 65,536 cache-hot reads cost 256 syscalls through `io_uring` instead of 65,536? (b) Given that result, why was the wall-clock time only roughly break-even rather than 256× faster? (c) Name two workloads where `io_uring` would show a real wall-clock win, and say what is the dominant cost in each that batching or async submission removes.

44.7 COST Model the cost of N operations as $N(b + w)$ for the synchronous loop, where b is the per-syscall boundary cost and w is the per-operation work (e.g. the copy), versus $N{\cdot}w + (N/k){\cdot}b$ for `io_uring` batching k at a time. (a) Write the ratio of the two and simplify. (b) For which relative sizes of b and w is the speed-up large, and for which is it near 1? (c) Explain how the §44.3 measurement (cache-hot reads: w is a 4 KiB copy, b a few hundred ns) lands in the “near 1” regime, and how a real device (w includes milliseconds of latency the kernel absorbs) changes the picture entirely.

44.8 FADED *Worked, with the last step for you.* A service ported its read path to `io_uring` and now returns corrupted data — but only occasionally, and never in the test suite.
Step 1 (given): for each request it fills an SQE pointing at a stack buffer, calls `io_uring_enter` to submit, and returns from the function.
Step 2 (given): the completion for that read arrives later, on a separate reaping loop.
Step 3 (given): by the time the kernel writes the read data, the function has returned and its stack frame — including the buffer — has been reused.
Step 4 (you): Explain the bug precisely, why it is intermittent and passes tests, and how to fix the buffer's lifetime.

44.9 MIXED For each claim, say correct or wrong and why: (a) “After `io_uring_enter` returns, my read is done and the buffer holds the data.” (b) “Completions come back in the order I submitted, so I can ignore `user_data`.” (c) “`io_uring` can make disk reads asynchronous, unlike `epoll`.” (d) “One `io_uring_enter` can submit a hundred operations.” (e) “Because it batches syscalls, `io_uring` always beats synchronous I/O on wall-clock.”

44.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “A regular file has a readiness event that tells you a `read` will not block.” (b) “An SQE's `user_data` is echoed back in the matching CQE.” (c) “The completion model requires the caller to keep each operation's buffer valid until its completion.” (d) “`io_uring`'s rings live only in kernel memory and the application cannot see them.” (e) “Batching cut the 65,536-read example to 256 kernel crossings.”

44.11 STRETCH Synthesize the chapter with the part. (a) Trace the “cost of the kernel boundary” thread from the blocking `read` of Chapter 40, through `epoll` readiness in Chapter 42, to `io_uring` completion here — what does each stage remove and what remains? (b) Argue why the honest break-even result of §44.3 is *more* instructive than a speed-up would have been, tying it to the part's cost-model discipline. (c) The completion model unifies sockets and disk under one interface but shifts buffer-lifetime responsibility to the programmer; discuss this as a general pattern — power purchased with explicitness — and one other place in this book where the same trade appears.

44.12 LAB On your own machine (a recent Linux kernel with `io_uring`; `liburing` optional): (a) Submit a single `IORING_OP_NOP` through `io_uring_setup`/`io_uring_enter` and confirm you reap its completion, to verify the interface works. (b) Read a large cache-warm file two ways — a synchronous `pread` loop and `io_uring` batched at some queue depth — and report the *kernel-crossing count* for each and the wall-clock time; comment on why the wall-clock difference is small on cache-hit data. (c) State your kernel version, toolchain, and CPU; note the times are wall-clock (no hardware counters); and, if you can, sketch how the picture would change for uncached reads from a real disk, where the kernel absorbs the device latency and your thread stays free.

Chapter 45 — Zero-Copy: sendfile, splice, and the Data Path

> **Before you start:** Chapter 40 (the page cache, and how `read` copies from it into a user buffer), Chapter 44 (batching away the per-operation kernel boundary), Chapter 43 (the socket the bytes go to), Chapter 13 (the cost of a system call), Chapter 3 (memory bandwidth — a copy is not free). · The part has solved the waiting and batched the boundary. One cost is left: every byte sent from a file to a socket is copied through a buffer in your process for no reason. This chapter removes it — the last copy between the file and the wire — and closes the part.

BEFORE YOU READ ON

From memory: (1) Chapter 40 — when you `read` a file, where does the data physically move, and whose CPU makes that copy? (2) Chapter 3 — copying a large buffer is bounded by memory bandwidth; is moving 512 MiB through a user buffer free? (3) *Predict:* a program sends a 512 MiB file (already warm in the page cache) over a socket two ways — a `read`-into-buffer-then-`write` loop, and a single `sendfile`. Two questions: how many times does the CPU copy each byte in the first version, and how many of those copies does the second remove? And what will happen to the number of system calls? Write your guesses; §45.1–§45.2 answer them.

Consider the most ordinary server task there is: send the contents of a file over a network connection — a static web asset, a video chunk, a database page to a replica. The obvious code reads the file into a buffer and writes that buffer to the socket, in a loop. It works, and for most of this part it is exactly what we have done. But trace where the bytes actually go and you find the CPU copying every one of them twice, through a user-space buffer that exists only to be a middleman — the data is copied out of the kernel's page cache into your buffer, and then straight back into the kernel's socket buffer, unchanged. Those copies cost memory bandwidth (Chapter 3), and at scale they are the bottleneck: a file server bottlenecked on `memcpy` is spending its cycles shuffling bytes it never looks at. **Zero-copy** is the set of techniques that remove that pointless round trip through user space, letting the kernel move the data from the page cache to the socket directly. This final chapter of the part builds it in five steps: the copies in the normal file-to-socket path, counted (§45.1); `sendfile`, which sends a file to a socket without ever touching user space, measured (§45.2); `splice` and the pipe-as-conduit generalisation, and what “zero” really means once the hardware helps (§45.3); the trap that zero-copy and touching-the-bytes are mutually exclusive — you cannot encrypt or transform what you never copy (§45.4); and what to carry forward, out of this part and this volume (§45.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35, 20 hardware threads), timed with `clock_gettime(CLOCK_MONOTONIC)` as the best of several trials; times are wall-clock and a hardware-counter profiler (`perf`) is *not* installed. The file (512 MiB) is sent over the **loopback** interface with its contents warm in the page cache, so the measurement isolates the *user-space copy* — loopback has no network card, so the saving here is the CPU copies and the system calls removed, not DMA to a NIC; the “how far can zero-copy go” discussion of §45.3 names the hardware step loopback cannot show. Every call and flag below was checked against the Linux man pages (`sendfile(2)`, `splice(2)`, `tcp(7)`) on this machine.

45.1 The copies in the ordinary file-to-socket path

Follow one byte from a file on disk to a network connection under the ordinary `read`-then-`write` loop, and count the copies. When the file is served, the byte makes four moves. First, the disk controller copies it from the device into the kernel's page cache by **DMA** (direct memory access — the device's own engine moves it, not the CPU), which happens once and stays cached (Chapter 40). Then `read` copies it from the page cache into your user-space buffer — a **CPU copy**, your process's cycles spent on `memcpy`. Then `write` copies it from your user-space buffer into the kernel's socket send buffer — a second **CPU copy**, straight back into the kernel it just came from. Finally the network card copies it from the socket buffer onto the wire by DMA. Of those four, the two DMA copies are unavoidable and cheap (the hardware does them); the two **CPU copies through user space are the waste** — the byte went kernel → user → kernel without being changed at all, and your process paid memory bandwidth for the privilege of being a pass-through. On top of the copies, the loop crosses the system-call boundary twice per block (one `read`, one `write`), the per-syscall tax this part has tracked since Chapter 40. The realization that drives zero-copy is simple: if the user-space buffer is never read or modified — if it is purely a relay — then routing the data through it is pointless work, and the kernel, which has the data in the page cache and controls the socket buffer, could move it directly. Every zero-copy technique is a way to say to the kernel: take these bytes from that file and put them on that socket; do not bring them through my address space, because I am not going to touch them.

> **QUICK CHECK**
>
> In the ordinary `read`-then-`write` loop that sends a file to a socket, how many times does the CPU copy each byte, and between which places? Which of the four total copies are done by DMA and are unavoidable, and which are the waste zero-copy removes? (Answer after the next section.)

45.2 sendfile: the file to the socket, without user space

The direct tool is `sendfile(out_fd, in_fd, offset, count)`: it copies up to `count` bytes from the file `in_fd` to the socket `out_fd` *inside the kernel*, so the data never enters your address space. As the `sendfile(2)` man page puts it, “because this copying is done within the kernel, `sendfile()` is more efficient than the combination of `read(2)` and `write(2)`, which would require transferring data to and from user space.” One call replaces the whole loop: no user buffer, no two CPU copies through it, and — because a single `sendfile` can move a large range — far fewer system calls. Measured here, sending a 512 MiB cache-warm file over loopback:

```
send 512 MiB, file warm in page cache, over loopback TCP (best of several trials):
  read()+write() loop (64 KiB buffer) :  ~1.73 GB/s    16,384 syscalls   (2 CPU copies/byte)
  sendfile()                          :  ~4.04 GB/s         1 syscall    (no user-space
copy)
  sendfile: ~2.3x throughput, ~16,000x fewer syscalls
```

The `sendfile` version moved the same bytes at about **2.3×** the throughput — 4.04 against 1.73 GB/s — and did it in **one** system call instead of 16,384, because it removed the two things the loop was spending on: the CPU copies into and out of the user buffer (the bandwidth), and the per-block `read`/`write` boundary crossings (the syscall tax). Note what the number is *not*: it is not infinite speed-up, because `sendfile` still does work — on ordinary hardware it still performs one in-kernel copy from the page cache into the socket buffer, and of course the DMA to move the data still happens; what it removed was specifically the user-space round trip. On this loopback test there is no network card, so the win you see is purely the eliminated user-space copies and syscalls; on a real server sending to a real NIC the win is larger, because the hardware can go further (§45.3). The practical rule is clean: whenever you are relaying a file to a socket unchanged — static file serving, video streaming, proxying, replication — `sendfile` is the right call, and it is exactly why high-performance web and file servers (nginx, Kafka, and the like) are built on it. The user-space buffer in the ordinary loop was never doing anything but relaying; `sendfile` is the kernel doing the relay without you.

THE SAME IDEA THREE WAYS: DON'T RELAY BYTES THROUGH USER SPACE YOU ARE NOT GOING TO TOUCH

1. In words. The ordinary loop copies each byte from the page cache into a user buffer and straight back into the socket buffer — two CPU copies through an address space that never reads or changes the data. `sendfile` tells the kernel to move the file's bytes to the socket directly, removing the user-space round trip and collapsing the per-block syscalls into one.

2. As a picture. Figure 45.1: on the left, the four-copy path — DMA disk→page cache, CPU page cache→user buffer, CPU user buffer→socket buffer, DMA socket buffer→NIC — with the two middle CPU copies highlighted as waste; on the right, `sendfile` going page cache→socket buffer inside the kernel, the user buffer gone entirely.

3. As numbers/contrast. 512 MiB, cache-warm, loopback: the `read`+`write` loop at **~1.73 GB/s** with 16,384 syscalls and two CPU copies per byte; `sendfile` at **~4.04 GB/s** with one syscall and no user-space copy — ~2.3× the throughput and ~16,000× fewer boundary crossings (§45.2), and more still once a scatter-gather NIC removes the last in-kernel copy (§45.3).

*(Answer to the §45.1 quick check: the CPU copies each byte **twice** — once from the page cache into the user-space buffer (by `read`) and once from that buffer into the socket send buffer (by `write`). Of the four total copies, the two **DMA** copies — disk into the page cache, and socket buffer out to the network card — are done by the hardware and are unavoidable; the two **CPU** copies through user space are the waste, because the buffer is a pure relay that never reads or changes the data, and that is exactly what `sendfile` removes.)*

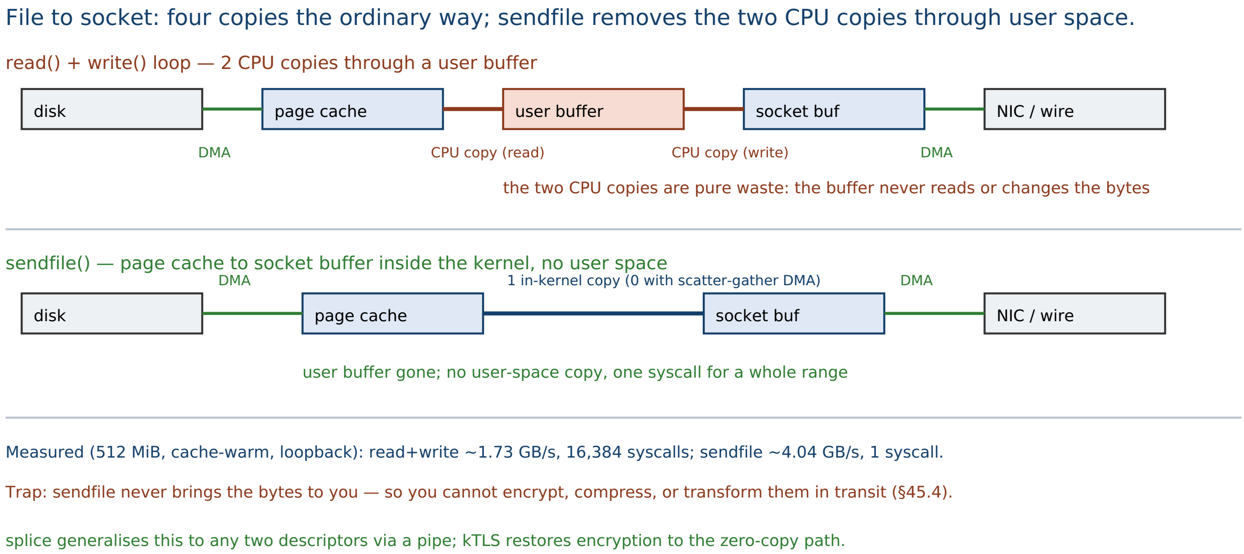


Figure 45.1: The ordinary loop (top) copies each byte through a user-space buffer twice (CPU copies) between two DMA transfers. `sendfile` (bottom) moves the data from the page cache to the socket buffer inside the kernel — no user-space copy, one system call for a whole range, and zero CPU copies at all when a scatter-gather NIC can send straight from the page cache.

45.3 splice, the pipe conduit, and how far “zero” goes

`sendfile` is specialised: a file in, a socket out. `splice(fd_in, off_in, fd_out, off_out, count, flags)` generalises the same idea — its man page describes it as moving data “between two file descriptors without copying between kernel address space and user address space” — with one constraint: one of the two descriptors must be a **pipe**. The pipe acts as a kernel-space conduit: rather than copy bytes, `splice` moves *references* to the kernel buffers (page-cache pages) through the pipe, so data can flow file→pipe→socket, or socket→pipe→file, or between two sockets, all without a user-space copy. That pipe requirement looks odd but it is the mechanism: the pipe is the kernel's internal handle for a queue of buffer references, and splicing is re-pointing them, not memcpy-ing them. This is the general zero-copy primitive; `sendfile` is essentially a convenient special case. It matters because the “which two descriptors” question is broader than file-to-socket: a proxy splicing one socket to another, a logger splicing a socket to a file, all get the same no-user-copy path. Now, how far does “zero” actually go? On ordinary hardware, even `sendfile` does *one* in-kernel copy, from the page cache into the socket buffer, because the network card needs the data laid out where it can DMA it. But modern NICs support **scatter-gather DMA** and checksum offload: given a list of page-cache pages, the card's DMA engine gathers them directly from the page cache onto the wire, computing the checksum in hardware — so that last in-kernel copy disappears too, and the byte goes disk → page cache (DMA) → wire (DMA) with the CPU never copying it at all. *That* is true zero-copy, and it is the endpoint the loopback measurement of §45.2 cannot show (there is no card), but it is why the technique matters so much on real servers: the CPU is removed from the data path entirely, freeing it to do work that actually requires a CPU. The progression is the whole part's lesson in miniature — first stop blocking (Chapter 42), then stop crossing the boundary per operation (Chapter 44), then stop copying the bytes at all (here) — each step removing a cost the previous one left standing.

QUICK CHECK

What does `splice` generalise about `sendfile`, and why does it require one of the two descriptors to be a pipe? On ordinary hardware `sendfile` still makes one in-kernel copy — what hardware feature removes even that, and what does the data path look like once it is gone? (Answer below the communicate box.)

45.4 The trap: zero-copy means you never see the bytes

Zero-copy's power is exactly its limitation, and the two are the same fact stated twice: the data never enters your address space. If your process never touches the bytes, then your process *cannot* touch the bytes — you cannot inspect, modify, compress, or **encrypt** them on the way through, because there is no point in the path where they are in memory you can run code over. This is the trap that catches teams reaching for `sendfile` to speed up a server that also needs to do something to the data. The most common collision is TLS: a secure server must encrypt every byte before it goes on the wire, and encryption is a transformation of the bytes — which requires having them in a buffer to transform — so a naive “`sendfile` for speed” and “TLS for security” are directly incompatible, because the first says “never bring the bytes to user space” and the second says “you must transform every byte.” The same conflict appears with on-the-fly compression, watermarking, transcoding, content inspection, or any per-request modification: if you must change the bytes, zero-copy is off the table by definition, and the ordinary `read`/transform/`write` path is not a failure to optimise — it is the only path that can do the job. There are real answers when you need both, and they work by relocating the transformation, not by wishing the conflict away: **kernel TLS** (kTLS) moves the encryption *into* the kernel, so the kernel can encrypt page-cache data on its way to the socket and `sendfile` can once again feed a TLS connection without a user-space copy; hardware offload can push crypto or compression onto the NIC. But absent such a mechanism, the rule is firm and worth stating plainly: **you can move bytes with zero copies, or you can transform them, but not both in the same step.** Choosing `sendfile` for a path that must encrypt, without kTLS, does not make encryption fast — it makes it impossible, and the bug is discovering that your “optimised” path silently cannot meet a requirement the ordinary path met for free.

TRAP: REACHING FOR SENDFILE ON DATA YOU MUST TRANSFORM — ZERO-COPY AND TOUCHING-THE-BYTES ARE EXCLUSIVE

`sendfile` and `splice` are fast precisely because the data never enters your address space — which means any operation that must *read or change* the bytes (encryption for TLS, on-the-fly compression, transcoding, watermarking, content inspection) is fundamentally incompatible with them: there is no moment in the zero-copy path where the bytes are in memory your code can run over. The trap is to treat zero-copy as a free speed-up you can bolt onto any file-serving path, then discover it cannot coexist with a transformation the product requires — most often TLS, where "encrypt every byte" and "never copy the bytes to user space" are contradictory by construction. This surfaces late and badly: a static-file benchmark shows `sendfile`'s beautiful numbers, the path ships, and then the requirement to serve over HTTPS (or to compress, or to inspect) reveals that the fast path structurally cannot do it. The defenses are to decide up front whether the data must be transformed in transit: if it must and you have no in-kernel offload, use the ordinary `read`/transform/`write` path and do not pretend zero-copy is available; if you want both, use **kernel TLS** (kTLS) or NIC offload, which relocate the transformation into the kernel or hardware so the zero-copy path can carry it. What you must not do is choose `sendfile` for its throughput and assume the transformation can be added later — the whole point of the technique is that the bytes are never yours to transform. "It is faster because we skip the copy" and "we encrypt every byte in the app" cannot both be true of the same step.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our media server is CPU-bound just shoveling files to clients — let's switch the send path to `sendfile` and get the copies out of user space. Also, product wants everything served over HTTPS now." Cover the paragraph and respond in four or five sentences, drawing on §45.2 and §45.4.

Model answer. "`sendfile` is exactly right for shoveling files unchanged — it removes the two user-space CPU copies and collapses the per-block syscalls, and I measured about 2× throughput on a plain relay, more on a real NIC that can DMA straight from the page cache. But there is a hard conflict with the HTTPS requirement: TLS means we must encrypt every byte before it hits the wire, and encryption is a transformation, so it needs the bytes in a buffer — which is exactly what `sendfile` refuses to give us. We cannot have both 'never copy to user space' and 'encrypt every byte in the app' in the same step. The clean answer is **kernel TLS** (kTLS): it moves the encryption into the kernel so the kernel encrypts page-cache data on the way to the socket and `sendfile` keeps working over TLS. So let's enable kTLS if the platform supports it; if it does not, the encrypted path has to go through user space and we optimise it differently — but we should not ship a `sendfile` fast path that structurally can't do HTTPS." Naming the throughput win, the encrypt-versus-copy contradiction, and kTLS as the specific reconciliation is the senior register.

*(Answer to the §45.3 quick check: `splice` generalises `sendfile` from "file to socket" to "any two descriptors," still without a user-space copy; it requires one end to be a **pipe** because the pipe is the kernel's internal conduit — a queue of references to kernel buffers — and splicing moves those references rather than copying bytes. On ordinary hardware `sendfile` still does one in-kernel copy into the socket buffer; a NIC with **scatter-gather DMA** and checksum offload removes even that, gathering the page-cache pages straight onto the wire, so the byte travels disk → page cache → wire by DMA alone with the CPU never copying it.)*

45.5 What to carry forward

Sending a file to a socket the ordinary way copies every byte through a user-space buffer twice — page cache → user buffer → socket buffer — two **CPU copies** that do nothing but relay, framed by two unavoidable DMA copies and paid for in memory bandwidth and per-block syscalls (§45.1). **`sendfile`** moves the data file-to-socket inside the kernel, removing the user-space round trip: measured here at ~4.04 GB/s against the loop's ~1.73 GB/s (~2.3×) and one syscall against 16,384, and it is why nginx, Kafka, and every serious file server use it (§45.2). **`splice`** generalises it to any two descriptors via a pipe conduit, and on a scatter-gather NIC even `sendfile`'s last in-kernel copy vanishes, giving *true* zero-copy where the CPU never touches the data — disk → page cache → wire by DMA alone (§45.3). But zero-copy's power is its constraint: because the bytes never enter your address space, you **cannot transform them in transit** — no TLS encryption, compression, or modification — so reaching for `sendfile` on a path that must encrypt makes the transformation impossible, not fast, and the reconciliation when you need both is to relocate the transformation into the kernel (**kTLS**) or hardware (§45.4).

This closes the low-level I/O and networking part, and with it the first volume. The arc is one idea pursued to the bottom: a program does not run on an abstract machine, and neither does its I/O. We started with the syscall boundary and the page cache (Chapter 40), learned when bytes are durable and when the OS is lying about it (Chapter 41), stopped dedicating a thread to every wait and built the event loop (Chapter 42), gave the connection a name and a transport whose guarantees we can state (Chapter 43), batched the kernel boundary away with completion-based submission (Chapter 44), and here removed the last needless copy between the file and the wire. The through-line was a single discipline, the same one that opened the volume: know where the bytes physically are, know who copies them and who waits, and measure the cost you claim to be removing rather than assume it — the honest break-even of Chapter 44 and the real 2.3× here are both results of that habit. This substrate — files, the page cache, durability, sockets, and the copies and crossings between them — is the ground the next volume builds a database on: the buffer pool that manages the page cache by hand, the write-ahead log that turns `fsync` into a durability guarantee, the storage engine that lays pages on disk. Everything a database does about storage is a specific, disciplined answer to the costs this part made concrete. The machine is real; now we build something serious on top of it.

CAN YOU… WITHOUT LOOKING?

- Count the copies in the ordinary file-to-socket path and say which are CPU copies and which are DMA.
- Explain what `sendfile` removes and why, and state its measured effect on throughput and syscall count.
- Say what `splice` generalises and why it needs a pipe.
- Explain what “true” zero-copy needs from the hardware, and what the data path looks like then.
- State why zero-copy and transforming the bytes (e.g. TLS) are mutually exclusive, and how kTLS reconciles them.
- Trace the part's arc: stop blocking, stop crossing per operation, stop copying — what each step removed.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** How many CPU copies does the ordinary file-to-socket loop make, between where and where, and which copies are DMA? (§45.1)
2. **R2.** What does `sendfile` do, and what were its measured throughput and syscall effects? (§45.2)
3. **R3.** What does `splice` generalise, and why the pipe? (§45.3)
4. **R4.** What hardware feature yields true zero-copy, and what is the resulting data path? (§45.3)
5. **R5.** Why can't you encrypt bytes you send with `sendfile`, and what reconciles zero-copy with TLS? (§45.4)

FURTHER READING

- **Linux `sendfile(2)` man page (Linux man-pages project).** The primary reference for §45.2: the `sendfile(out_fd, in_fd, offset, count)` interface and its rationale — "because this copying is done within the kernel, `sendfile()` is more efficient than the combination of `read(2)` and `write(2)`" — verified against the page installed on this machine.
- **Linux `splice(2)` man page (Linux man-pages project).** The reference for §45.3: `splice` "moves data between two file descriptors without copying between kernel address space and user address space," with one end a pipe — verified against the page installed on this machine.
- **Michael Kerrisk, *The Linux Programming Interface* (No Starch Press, 2010), ISBN 978-1-59327-220-3.** Section 61.4.4, "Transferring a file to a socket: `sendfile()`," explains the copy the ordinary `read`/`write` loop wastes and how `sendfile` avoids it — the source for §45.1–§45.2's copy accounting. First author Michael Kerrisk; edition and ISBN verified via the publisher.
- **Vivek S. Pai, Peter Druschel & Willy Zwaenepoel, "IO-Lite: A Unified I/O Buffering and Caching System," in *Proc. 3rd USENIX OSDI* (1999); expanded in *ACM Transactions on Computer Systems* 18(1), 2000, DOI 10.1145/332799.332895.** The research foundation of zero-copy I/O: a single physical copy of data shared safely across the file cache, IPC, and network subsystem — the idea §45.3's page-reference splicing and scatter-gather sending descend from. First author Vivek S. Pai; venue, year, and DOI verified via the ACM Digital Library.

Exercises

45.1 WARM-UP In the ordinary `read`-then-`write` loop that sends a file to a socket, how many times does the CPU copy each byte, and between which two pairs of places?

45.2 WARM-UP In one sentence, what does `sendfile` remove from the ordinary loop, and roughly what was its measured effect on the number of system calls in this chapter?

45.3 WARM-UP Name one operation on the data that `sendfile` makes impossible, and say why.

45.4 CORE Counting copies. (a) List all four copies a byte undergoes from disk to wire in the ordinary loop, labeling each as CPU or DMA. (b) Which two does `sendfile` remove, and why are they waste? (c) On ordinary hardware `sendfile` still does one in-kernel copy — between where and where, and what removes it?

45.5 CORE The measurement. (a) The loop ran at ~1.73 GB/s and `sendfile` at ~4.04 GB/s for the same 512 MiB; name the two distinct costs `sendfile` removed that account for the difference. (b) Why is the win only ~2.3× and not, say, 100×? (c) Why would the same comparison likely show a *larger* win on a real server with a network card than on this loopback test?

45.6 CORE `splice` and the pipe. (a) How does `splice` generalise `sendfile`? (b) Why does it require one descriptor to be a pipe — what is the pipe actually doing? (c) Give two useful data paths `splice` enables that `sendfile` does not.

45.7 COST A server sends files at rate R bytes/s. Model per-byte CPU-copy cost as $1/B$ seconds, where B is memory copy bandwidth. (a) The ordinary loop does two CPU copies per byte; write the CPU time per second it spends just copying at rate R. (b) `sendfile` on ordinary hardware does one in-kernel copy; write the same. (c) With a scatter-gather NIC (zero CPU copies), what is that CPU time, and what does the freed CPU now get to do? Comment on why a file server can become copy-bound.

45.8 FADED *Worked, with the last step for you.* A team switched their static-asset server to `sendfile`, benchmarked a 3× throughput improvement, and shipped it. Two months later, product mandates HTTPS for all traffic.
Step 1 (given): TLS requires encrypting every byte before it is sent on the socket.
Step 2 (given): encryption is a transformation of the bytes and needs them in a buffer to transform.
Step 3 (given): `sendfile` moves bytes from the page cache to the socket without ever bringing them to user space.
Step 4 (you): Explain why the fast `sendfile` path structurally cannot serve TLS, why the benchmark did not reveal this, and the two ways forward (one that keeps zero-copy, one that does not).

45.9 MIXED For each task, say whether zero-copy (`sendfile`/`splice`) applies, and why: (a) serving a static image file over plain HTTP; (b) serving the same file over HTTPS with no kernel TLS; (c) proxying bytes from an upstream socket to a client socket unchanged; (d) gzip-compressing a log file as you stream it to a client; (e) serving a video file over HTTPS on a host with kTLS enabled.

45.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "`sendfile` copies the file into a user buffer and then to the socket." (b) "On a scatter-gather NIC, a sent byte need never be copied by the CPU." (c) "You can encrypt data in your application and still send it with `sendfile` in the same step." (d) "`splice` needs one end to be a pipe." (e) "The two DMA copies in the ordinary path are the waste zero-copy removes."

45.11 STRETCH Synthesize the part. (a) The part removed three costs in sequence — blocking (Chapter 42), the per-operation kernel boundary (Chapter 44), and the user-space copy (this chapter). State what each step removed and what it left for the next. (b) Zero-copy's benefit and its limitation are the same fact; explain that identity and why it means the "fastest" path is not always the usable one. (c) Argue why "know where the bytes physically are and who copies them" is the single question that unifies this entire part, using the four-copy diagram to make the case.

45.12 LAB On your own machine (a C toolchain): (a) Send a large cache-warm file over a loopback TCP connection two ways — a `read`/`write` loop with a fixed buffer, and a single `sendfile` — and report the throughput and the number of system calls for each. (b) Explain, from your syscall counts, how much of the difference is the removed copies versus the removed boundary crossings. (c) State your toolchain and CPU, note the times are wall-clock (no hardware counters), and explain why a loopback test understates the win a real network card would show, naming the hardware feature (scatter-gather DMA) that removes the last copy.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART VIII

Storage & the Disk

Pages, files, the buffer pool, and the storage manager — the foundation you build a database on.

Chapter 46 — Storage & the Disk: How Devices Really Work

Before you start: Chapter 3 (the memory hierarchy and the cost cliff between a cache hit and DRAM), Chapter 40 (the page cache — how the OS keeps disk blocks in memory), Chapter 41 (durability and `fsync`), Chapter 17 (demand paging and page replacement — the same problem a buffer pool will solve by hand), Chapter 5 (mechanical sympathy — knowing the device you run on). · This chapter opens the second volume. The first built the machine and its I/O; the second builds a database on top, and a database is, before anything else, a program that is honest about where its data physically lives. That place is the disk, and the disk is nothing like RAM. This chapter is the floor the whole volume stands on.

BEFORE YOU READ ON

From memory: (1) Chapter 3 — a cache miss to DRAM costs on the order of a hundred nanoseconds against a hit's fraction of one; storage sits one more cliff below DRAM. Roughly how much slower is a disk access than a DRAM access? (2) Chapter 40 — when you `read` a file the OS may serve it from the page cache in memory; what has to happen the first time, when it is *not* cached? (3) *Predict:* a program reads a 1 GiB file with the page cache bypassed, two ways — streaming it in large sequential blocks, and fetching 4 KiB pages at random offsets. Two questions: which is faster, and by what factor — 2×, 10×, 50×? And would a spinning hard disk and a flash SSD give the *same* ratio? Write your guesses; §46.2–§46.4 answer them.

Everything in Volume 1 could pretend the machine had one kind of memory. The RAM model of Chapter 1 says any location costs the same to touch, and even after Chapter 3 punctured that with caches, the whole hierarchy — registers, L1, L2, L3, DRAM — was still *memory*: electrically addressed, uniform within a level, measured in nanoseconds. A database cannot live in that world, because its data outlives the process and outgrows the RAM, so it must live on **persistent storage** — a disk — and the disk breaks every assumption the memory hierarchy let you keep. It is slower than DRAM by three to five orders of magnitude; it is addressed not in bytes but in fixed **blocks**; and, most consequentially, its cost depends violently on *access pattern* — reading the same amount of data sequentially versus at random can differ by fifty times. Every structure a database builds — the page, the B-tree, the log, the buffer pool — is a specific, disciplined answer to those three facts. This chapter establishes them in five steps: the storage hierarchy and why the disk is the floor (§46.1); the spinning hard disk, its geometry, and why a seek is the enemy (§46.2); the flash SSD, its erase blocks, the translation layer, and why it has no seek but hates small writes (§46.3); the block layer and the sequential-versus-random gap, measured on this machine (§46.4); and the storage discipline the rest of the volume inherits (§46.5).

On the code in this chapter. The program is C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35), timed with `clock_gettime(CLOCK_MONOTONIC)` as the best of several trials; times are wall-clock and a hardware-counter profiler (`perf`) is *not* installed. Reads use `O_DIRECT` to bypass the page cache so the measurement reaches the storage stack rather than DRAM. This machine runs under **WSL2**: the working directory is an `ext4` filesystem on a virtual disk (`/dev/sdd`), whose `/sys/block/sdd/queue/physical_block_size` reads **4096** and whose `rotational` flag reads **1** — a virtio artifact, not a real platter; the host backing store is an NVMe SSD. So the absolute numbers below reflect a *virtualised SSD through a hypervisor at queue depth one*, not a bare device, and the published hard-disk and SSD latencies quoted in §46.1–§46.3 are labeled **published/typical**, not measured here. Device values were read from `/sys/block` on this machine.

46.1 The storage hierarchy and why the disk is the floor

Extend the memory hierarchy of Chapter 3 downward past DRAM and you reach a different kind of device. The numbers, as *published/typical* figures (Bryant & O'Hallaron, CS:APP, Chapter 6; the orders of magnitude, not the exact digits, are the point): an L1 cache hit is about one nanosecond; a miss to DRAM is on the order of a hundred nanoseconds; a read from a flash **SSD** is on the order of tens to a hundred **microseconds** — a thousand times slower than DRAM; and a read from a spinning **hard disk** that must physically move its head is on the order of **ten milliseconds** — a hundred thousand times slower than DRAM, and roughly ten million times slower than the L1 hit. If a DRAM access were one second, a hard-disk seek would be about four months. That is not a slower level of the same hierarchy; it is a cliff, and a database spends its life on the wrong side of it. The consequence is the founding principle of storage engineering: **an access to disk is so expensive relative to computation that the entire design is organized around avoiding, batching, and amortizing it.** A B-tree is shaped to reach any record in three or four disk accesses instead of thousands; a log is written sequentially so that durability costs one kind of I/O instead of the worst kind; a buffer pool (Chapter 48) keeps hot pages in DRAM so that most accesses never reach the cliff at all. You cannot make the disk fast. You can only touch it rarely, in bulk, and in the pattern it prefers — and to do that you must know what pattern it prefers, which is the subject of the next three sections.

> **QUICK CHECK**
>
> Order these by access latency and give the published/typical magnitude of each: an L1 cache hit, a DRAM access, a flash-SSD read, a hard-disk seek. Roughly how many DRAM accesses fit in the time of one hard-disk seek? (Answer after the next section.)

46.2 The spinning hard disk: geometry, seek, and rotation

A mechanical hard disk stores bits as magnetic patterns on the surfaces of rigid spinning **platters**. Each surface is divided into concentric **tracks**, and each track into fixed-size **sectors** (historically 512 bytes, now commonly 4096); a **head** on a moving arm reads and writes whichever sector passes beneath it. To read a given sector, two mechanical motions must finish: the arm must move the head to the right track — a **seek** — and the platter must rotate until the target sector arrives under the head — **rotational latency**. Both are governed by physics, not electronics, and both are slow. Published/typical figures (OSTEP, "Hard Disk Drives"): an average seek is on the order of several to ten milliseconds, and a 7200-RPM drive turns once every about 8.3 ms, so the average rotational wait — half a turn — is roughly 4 ms. Only after both finish does the actual data **transfer** begin, and the transfer itself is fast (100+ MB/s). This decomposition is the whole story of the hard disk: the time to read a sector is dominated by *getting to it*, not by reading it. Now consider what that means for access pattern. A **sequential** read walks sectors that are already adjacent — same track, then the next — so after one initial seek the head barely moves and the platter simply feeds sector after sector; the per-sector cost collapses toward the raw transfer rate. A **random** read jumps to an unrelated sector each time, paying a fresh seek plus rotational latency — call it roughly 10 ms — for every single access. The difference is not a constant factor; it is the difference between paying mechanical positioning *once* and paying it *every time*. This is why, on a hard disk, a few hundred random reads per second is a realistic ceiling while sequential throughput is hundreds of megabytes per second, and it is the physical reason databases were built, from the beginning, to turn random work into sequential work wherever they possibly can (McKusick's fast file system, Chapter 47's on-disk layout, and the log-structured designs of the next part all descend from this one fact).

THE SAME IDEA THREE WAYS: THE DISK IS NOT RAM — RANDOM ACCESS COSTS FAR MORE THAN SEQUENTIAL

1. In words. On storage, the cost of reaching data dominates the cost of reading it. On a spinning disk the reach is a mechanical seek plus rotation (~10 ms); even on an SSD, small random I/O forfeits the streaming efficiency of large sequential I/O. So the same number of bytes costs far more fetched at random than fetched in order — the opposite of the uniform-cost RAM model.

2. As a picture. Figure 46.1: the latency cliff from L1 (~1 ns) down to DRAM (~100 ns), SSD (~100 µs), and hard disk (~10 ms) on a log scale; beside it, two bars for the same 1 GiB moved through this machine's storage stack — large sequential streaming versus small random 4 KiB reads — the random bar a sliver of the sequential one.

3. As numbers/contrast. Measured here (1 GiB, `O_DIRECT`, queue depth 1): sequential 1 MiB-block streaming at **~916 MB/s** against random 4 KiB reads at **~19 MB/s** and ~221 µs per read — about **49×** more throughput for the same device when the access is sequential (§46.4).

*(Answer to the §46.1 quick check: fastest to slowest — L1 hit ~1 ns, DRAM ~100 ns, flash-SSD read ~tens–100 µs, hard-disk seek ~10 ms, all published/typical. A hard-disk seek is roughly **100,000×** a DRAM access, so on the order of a hundred thousand DRAM accesses fit in one seek — the cliff a database spends its life avoiding.)*

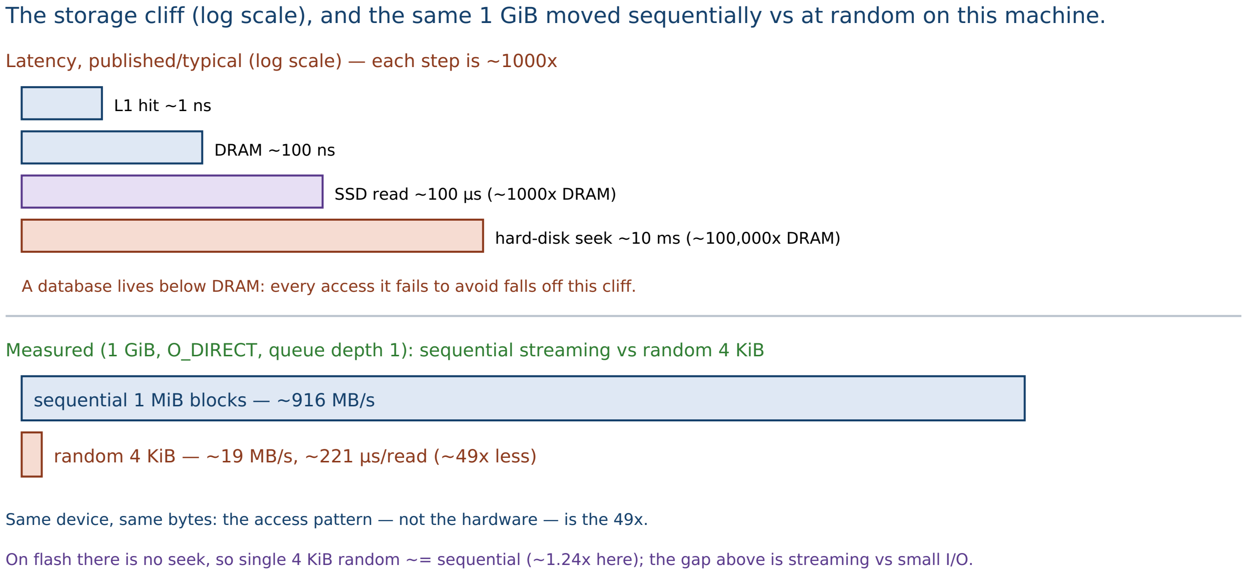


Figure 46.1: Storage sits one to five orders of magnitude below DRAM (top, log scale). For the same 1 GiB on this machine (bottom), large sequential streaming reaches ~916 MB/s while small random 4 KiB reads reach ~19 MB/s — a ~49× gap set by the *access pattern*, not by the device.

46.3 The flash SSD: erase blocks, the FTL, and wear

A solid-state drive has no moving parts: it stores bits as trapped charge in **NAND flash** cells, and reads them electronically, so there is no seek and no rotation. That single fact changes the cost model — random reads on an SSD are cheap, close to sequential, because there is nothing mechanical to reposition. But flash has its own awkward physics, and a database that assumes an SSD is just “a fast disk” will be surprised by writes. Flash is organized into **pages** (typically a few kilobytes, the unit of read and *program*/write) grouped into **erase blocks** (typically hundreds of pages, the unit of *erase*). The awkward rule is this: you can read any page and you can program an *empty* page, but you **cannot overwrite a page in place** — to rewrite it you must first erase its entire surrounding block, and erase is slow and can only be done a limited number of times before the cells wear out. No sane system erases a whole block to change one page, so every SSD hides a **Flash Translation Layer** (FTL): a controller that maps logical block addresses (what the OS sees) to physical flash pages (where the data actually is), writes updated data to a fresh already-erased page and remaps the pointer, garbage-collects blocks whose pages are mostly stale, and spreads writes across all blocks so no cell wears out first — **wear leveling** (Agrawal et al., “Design Tradeoffs for SSD Performance,” USENIX ATC 2008; OSTEP, “Flash-based SSDs”). The consequences a database designer must keep in mind: reads are fast and pattern-insensitive; a random small *write* can trigger far more physical work than it seems (a read-erase-reprogram cycle plus garbage collection), an effect called **write amplification**; and the device has a finite write lifetime. So the sequential-write preference that the hard disk taught for mechanical reasons survives on the SSD for a completely different reason — large sequential writes are friendlier to the FTL's erase-block structure and cause less amplification. As a small confirmation that flash has no seek, the single-4 KiB reads measured on this machine came out only about **1.24×** slower at random offsets than at sequential ones (§46.4) — a hard disk would show that same comparison as a chasm, because each random read there would pay a fresh ~10 ms seek.

> **QUICK CHECK**
>
> On an SSD, why is a random *read* nearly as cheap as a sequential one, when on a hard disk it is far more expensive? Why is a small random *write* the operation an SSD dislikes, and what is the layer that hides in-place-overwrite from the OS called? (Answer below the communicate box.)

46.4 The block layer and sequential vs random, measured

Above the device, the operating system presents storage as the **block layer**: a linear array of fixed-size blocks, addressed by number, that you read and write a whole block at a time. On this machine, `/sys/block/sdd/queue/logical_block_size` is **512** (the smallest addressable unit the device advertises) and `physical_block_size` is **4096** (the size the device actually operates on internally); a database will choose a page size that is a multiple of that physical block — 4, 8, or 16 KiB — precisely so that one page is one natural unit of device I/O. The block abstraction is why a database thinks in *pages*, not bytes (Chapter 47): the smallest thing you can fetch or store is a block, so making the byte the unit of I/O would waste almost all of every transfer. To measure the sequential-versus-random gap directly, the program reads a 1 GiB file with `O_DIRECT` — which bypasses the page cache so each read reaches the storage stack — two ways: streaming the whole file in 1 MiB blocks, and fetching 4 KiB pages at uniformly random offsets. Best of several trials on this machine:

```
read 1 GiB, O_DIRECT (page cache bypassed), single thread, queue depth 1:
  sequential, 1 MiB blocks :  ~916 MB/s
  random,     4 KiB pages   :  ~19 MB/s   (~221 µs/read, ~4,500 IOPS)
  sequential streaming is ~49x the throughput of small random I/O
  (single 4 KiB reads: random only ~1.24x slower than sequential -> flash has no seek)
```

The sequential stream moved bytes at about **49×** the throughput of the small random reads — 916 against 19 MB/s — on the exact same device, in the same run. Two distinct effects combine to produce that gap, and it is worth separating them because they teach different lessons. The first is **I/O size**: the sequential test asks for 1 MiB at a time and the random test for 4 KiB, so the sequential test amortizes each request's fixed overhead over 256× more data; big requests are efficient, tiny requests are not. The second is **access pattern**: the random test scatters its reads, forfeiting any readahead or locality the stack could exploit. That the *single*-4 KiB comparison (sequential vs random at the same small size) came out only ~1.24× apart tells you that on this flash-backed device the pattern penalty alone is modest — there is no seek — and that most of the 49× is the I/O-*size* effect, the reward for fetching in bulk. On a real spinning disk the same experiment would show the pattern penalty exploding instead, because each random read would pay a mechanical seek. Either way the engineering conclusion is identical, and it is the reason a database is built the way it is: **read and write in large, sequential, block-aligned units, and avoid a storm of tiny random accesses.** Hold the WSL2 caveat firmly, though — 221 µs per random read and 4,500 IOPS are the numbers for *this virtualised* *disk at queue depth one*; a bare NVMe SSD driven with deep request queues does far more, and a spinning disk far less. The *shape* of the result — sequential vastly beats small random — is the portable lesson; the digits are this machine's.

TRAP: TREATING THE DISK AS RANDOM-ACCESS MEMORY — ASSUMING ANY ACCESS COSTS THE SAME

The most expensive mistake in storage design is to carry the RAM model (Chapter 1) down to the disk: to lay data out, or write access code, as if fetching a record from a random offset costs the same as streaming the next one. It does not — measured here, small random I/O ran at 1/49 of sequential throughput, and on a spinning disk the ratio would be far worse because each random access pays a mechanical seek. The trap surfaces as designs that scatter related data across the file and then chase it with a pointer per record, or that update rows in place at random offsets, and then are baffled that the system is slow despite “not doing much I/O” — it is doing the *worst kind* of I/O. Two secondary traps live here too. First, do not trust a single flag or number: this machine's `/sys/block/sdd/rotational` reads **1** though the backing store is an NVMe SSD (a WSL2 virtio artifact), so a design that branched on that flag would be wrong; and the 221 µs random-read latency is a queue-depth-one virtualised figure, not a device ceiling. Second, do not assume an SSD's cheap random *reads* mean cheap random *writes* — the FTL makes small random writes amplify (§46.3). The defense is the discipline this part builds: think in block-sized pages, prefer large sequential I/O, keep related data physically together, and turn unavoidable random work into sequential work (the log, the buffer pool, the B-tree's fanout) rather than pretending the disk is flat.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: “Let's store each record at whatever offset in the file, and keep a hash table from record ID to byte offset. To read a record we just `seek` there and read it. Disks are random-access anyway, so it'll be fast.” Cover the paragraph and respond in four or five sentences, drawing on §46.2 and §46.4.

Model answer. “The premise is the trap — a disk is not random-access like RAM. I measured small random 4 KiB reads on our storage at about 19 MB/s and ~221 µs each, against ~916 MB/s streaming sequentially: roughly 49× slower for the same bytes, and on a spinning disk it would be far worse because every seek is mechanical, on the order of 10 ms. A seek-per-record design turns every lookup and especially every scan into a storm of the most expensive I/O there is. The reason databases use fixed-size **pages** and pack related records together is exactly to fetch in large, block-aligned, mostly-sequential units and to keep hot pages in a buffer pool so most reads never touch the disk at all. So let's not scatter records at arbitrary offsets; let's lay them out in pages, and if we need lookup by ID, that's what an index is for — it's built to reach a record in a few page reads, not one seek per record.” Naming the measured sequential-vs-random gap, the page as the I/O unit, and the buffer pool as the thing that avoids the cliff is the senior register.

(Answer to the §46.3 quick check: a random read is cheap on an SSD because flash is addressed electronically — there is no head to move or platter to rotate, so reaching any page costs the same; on a hard disk a random read pays a mechanical seek plus rotational latency each time. A small random ***write*** *is the SSD's weak spot because flash cannot be overwritten in place — the surrounding erase block must be erased first — so the controller must write elsewhere, remap, and later garbage-collect, causing* ***write***

***amplification**; the layer that hides in-place overwrite and does the remapping, garbage collection, and wear leveling is the **Flash Translation Layer**, the FTL.)*

46.5 What to carry forward

Storage sits one to five orders of magnitude below DRAM, and a database spends its life avoiding, batching, and amortizing the trip down the cliff (§46.1). A spinning hard disk pays a mechanical **seek** plus rotational latency to reach data — on the order of 10 ms — so sequential access, which pays that once, beats random access, which pays it every time; this is the physical origin of every “keep related data together” rule in the volume (§46.2). A flash **SSD** has no seek, so random reads are cheap, but it cannot overwrite in place, so a **Flash Translation Layer** remaps writes, garbage-collects, and levels wear, making small random writes amplify — the sequential-write preference survives for a new reason (§46.3). Above the device the **block layer** presents fixed-size blocks (physical block 4096 on this machine), which is why a database thinks in *pages*; and measured here, large sequential streaming beat small random I/O by **~49×** on the same device — the portable lesson being *read and write in large, sequential, block-aligned units* (§46.4). Multiple disks can be combined for throughput and redundancy — the RAID idea (Patterson, Gibson & Katz, 1988) — but even then the per-device physics above is what each disk contributes.

That physics is the foundation the rest of this volume builds on, and the shape of what follows is already visible. The next chapter (Chapter 47) takes the block layer's fixed-size unit and makes it the database's currency: the **page**, laid out to hold variable-length records, and the file as a linear array of pages — the on-disk format a storage manager reads and writes, with the durability barrier (`fsync`) that Chapter 41 introduced now costed page by page. Chapter 48 then confronts the cliff head-on with the **buffer pool**: the database's own hand-managed cache of pages in DRAM, so that most accesses hit memory and only misses fall to the disk — the same demand-paging problem as Chapter 17, solved deliberately because the database knows its own access patterns. And beyond this part, the two great storage-engine families — B-trees and log-structured merge trees — are each an answer to this chapter's cost model: the B-tree minimizes the number of disk accesses per lookup, and the LSM tree turns random writes into sequential ones. The disk is the floor; know what it costs to touch it, and every structure above it stops being arbitrary and starts being *forced*.

CAN YOU… WITHOUT LOOKING?

- Place storage on the memory hierarchy and give the published/typical latency of DRAM, an SSD read, and a hard-disk seek.
- Decompose a hard-disk read into seek, rotational latency, and transfer, and explain why sequential beats random.
- Explain why an SSD has no seek, what an erase block is, and why the FTL exists.
- Define write amplification and say why small random writes are an SSD's weak spot.
- State what the block layer presents, why a database thinks in pages, and the physical block size of this machine's disk.
- Quote the measured sequential-vs-random gap here (~49×) and separate the I/O-size effect from the access-pattern effect.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Roughly how much slower is a hard-disk seek than a DRAM access, and why does that ratio organize all of storage design? (§46.1)
2. **R2.** What two mechanical delays must finish before a hard disk transfers data, and why does that make random access far costlier than sequential? (§46.2)
3. **R3.** Why can't flash be overwritten in place, and what three jobs does the FTL do about it? (§46.3)
4. **R4.** What does the block layer present, and why does a database choose a page size that is a multiple of the physical block size? (§46.4)
5. **R5.** What was the measured sequential-vs-random throughput ratio here, and which part of it is I/O size versus access pattern? (§46.4)

FURTHER READING

- **Remzi H. Arpaci-Dusseau & Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, "Hard Disk Drives" (free online, pages.cs.wisc.edu/~remzi/OSTEP).** The clearest short treatment of the source for §46.2: platters, tracks, sectors, and the seek + rotation + transfer decomposition of a disk access. First authors the Arpaci-Dusseaus; chapter title verified against the online edition.
- **Arpaci-Dusseau & Arpaci-Dusseau, *OSTEP*, "Flash-based SSDs."** The reference for §46.3: NAND pages and erase blocks, the read/program/erase asymmetry, the Flash Translation Layer, garbage collection, and wear leveling — chapter title verified against the online edition.
- **Randal E. Bryant & David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. (Pearson, 2016), ISBN 978-0-13-409266-9, Chapter 6 ("The Memory Hierarchy").** The source for §46.1's hierarchy and its published latency magnitudes, including disk geometry from a programmer's viewpoint. First author Randal E. Bryant; edition and ISBN verified via the publisher.
- **Nitin Agrawal, Vijayan Prabhakaran, Ted Wobber, John D. Davis, Mark Manasse & Rina Panigrahy, "Design Tradeoffs for SSD Performance," *USENIX Annual Technical Conference* (2008).** The design-space study behind §46.3: FTL policies, write amplification, and how workload shapes SSD performance and lifetime. First author Nitin Agrawal; venue and year verified via the USENIX proceedings.
- **David A. Patterson, Garth Gibson & Randy H. Katz, "A Case for Redundant Arrays of Inexpensive Disks (RAID)," *Proc. ACM SIGMOD* (1988), DOI 10.1145/50202.50214.** The origin of combining many disks for throughput and redundancy, referenced in §46.5. First author David A. Patterson; venue, year, and DOI verified via the ACM Digital Library.

Exercises

46.1 WARM-UP Give the published/typical latency magnitude of each and order them: an L1 cache hit, a DRAM access, a flash-SSD read, a hard-disk seek.

46.2 WARM-UP Name the two mechanical delays a spinning hard disk pays before it can transfer a sector, and say in one sentence why they make random reads far costlier than sequential ones.

46.3 WARM-UP Why can a flash SSD not overwrite a page in place, and what is the unit it must erase before rewriting?

46.4 CORE The hard-disk cost model. (a) Write the total time to read one sector as the sum of three terms and name each. (b) A drive has a 9 ms average seek, spins at 7200 RPM, and transfers at 150 MB/s. Estimate the time to read one 4 KiB sector at a random location, and identify which term dominates. (c) Estimate the time to read 1 MiB laid out sequentially, and explain why the per-byte cost is now completely different.

46.5 CORE Flash asymmetry. (a) Explain why a random *read* on an SSD is nearly as fast as a sequential one, but a random small *write* can be much more expensive. (b) Define write amplification and give the sequence of physical operations a single small overwrite can trigger. (c) State the sequential-write preference that both a hard disk and an SSD share, and why the *reason* differs between them.

46.6 CORE The measurement. (a) On this machine sequential streaming reached ~916 MB/s and random 4 KiB reads ~19 MB/s; name the two distinct effects that together produce the ~49× gap. (b) The *single*-4 KiB comparison was only ~1.24× — what does that tell you about which of the two effects dominates on this flash-backed device? (c) Why would the same experiment on a spinning disk shift the balance between the two effects?

46.7 COST A workload issues N record reads. Under design A each record is at a random offset (one random I/O per read); under design B records are packed 100 to a page and reads have good locality, so on average one page read serves 100 records. Let a random page read cost t_r. (a) Write the total I/O time for each design. (b) On a hard disk with $t_r \approx 10$ ms and $N = 10^6$, compute both. (c) Explain what the 100× factor physically represents and why it is the whole argument for pages.

46.8 FADED *Worked, with the last step for you.* A team benchmarks their storage by reading a 1 GiB file with a plain buffered `read` loop, sees 6 GB/s, and concludes "random and sequential are the same on our disk, so we don't need to worry about layout."
Step 1 (given): a plain `read` loop hits the OS page cache, so a warm file is served from DRAM, not the device.
Step 2 (given): to measure the device you must bypass the cache (`O_DIRECT` or an uncached file) and vary the access pattern and I/O size.
Step 3 (given): doing so on this machine gave ~916 MB/s sequential vs ~19 MB/s random 4 KiB.
Step 4 (you): Explain why the team's 6 GB/s measured DRAM rather than the disk, why their conclusion is exactly backwards, and what they should measure instead.

46.9 MIXED For each, say whether it is limited mainly by *seek/positioning*, by *I/O size*, by *flash write amplification*, or by *the DRAM ceiling*, and why: (a) 10,000 random 4 KiB reads from a cold spinning disk; (b) streaming a 10 GiB file sequentially from an SSD; (c) 10,000 random 4 KiB reads from a warm page cache; (d) a benchmark doing millions of tiny random overwrites on an SSD; (e) reading 4 KiB pages one at a time versus 1 MiB blocks from the same SSD.

46.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "An SSD read is about as fast as a DRAM read." (b) "On a hard disk, reading data sequentially is faster than reading it at random because the transfer rate is higher for sequential data." (c) "Because an SSD has no seek, random writes are as cheap as random reads." (d) "A database reads and writes in fixed-size pages because the block layer's smallest unit is a block." (e) "`/sys/block/sdd/rotational` is 1, so this machine has a spinning disk."

46.11 STRETCH Synthesize the cost model. (a) State the one sentence about disks that forces the shape of B-trees, logs, and buffer pools, and explain in one line each how those three structures answer it. (b) The chapter measured a ~49× sequential-vs-random gap yet a ~1.24× single-read gap on the *same* device; explain why both numbers are true and what each one is actually measuring. (c) Argue why “know where the bytes physically are and what it costs to reach them” is the same discipline that unified Volume 1's I/O part, now applied one level down at the device.

46.12 LAB On your own machine (a C toolchain): (a) Read `/sys/block/<dev>/queue/rotational`, `logical_block_size`, and `physical_block_size` for the disk your files live on, and report them. (b) Read a large file two ways with `O_DIRECT` — streaming in large blocks and fetching small pages at random offsets — and report throughput, per-read latency, and IOPS for each. (c) State your toolchain and CPU, note the times are wall-clock (no hardware counters), say whether your machine is virtualised, and explain which part of your sequential-vs-random gap is I/O size and which is access pattern — and how you could tell them apart with a third measurement.

Chapter 47 — Files and Pages: The Database on Disk

Before you start: Chapter 46 (the block layer and why sequential, block-aligned I/O wins), Chapter 41 (durability, `fsync`, and what "written" really means), Chapter 40 (the page cache the OS keeps under your `write`), Chapter 7 (memory layout and alignment — we lay records out in bytes the same way), Chapter 11 (buffer bounds — an on-disk page is a fixed-size buffer you must not overrun). · The last chapter established what the device costs. This one gives the database its on-disk shape: a file treated as an array of fixed-size *pages*, each page a little heap of records, each page write made durable at a price we now measure.

BEFORE YOU READ ON

From memory: (1) Chapter 46 — the block layer hands you fixed-size blocks and rewards large, block-aligned, sequential I/O; what unit should a database therefore read and write in? (2) Chapter 41 — after `write` returns, is your data on the disk platter yet, and what call forces it there? (3) *Predict:* a program appends 2,000 small pages to a file two ways — calling `fsync` once at the end, and calling `fsync` after *every* page (as a paranoid "make each record durable immediately" loop might). Two questions: how many durable writes per second do you think the per-page version can sustain, and how many times slower is it than the batched version — 3×, 10×, 100×? Write your guesses; §47.4 answers them.

A database has to turn the block layer's flat array of numbered blocks into something it can store rows in, look rows up in, and update without rewriting the world. The abstraction it chooses, almost universally, is the **page**: a fixed-size chunk — 4, 8, or 16 KiB, a multiple of the device's physical block (Chapter 46) — that is the unit of everything. The database file is one large array of pages; the storage manager reads and writes a whole page at a time; the buffer pool (Chapter 48) caches whole pages; the B-tree and the log are built out of pages. Inside a page lives a small, self-contained data structure that packs variable-length records and lets you add, find, and delete them without disturbing their neighbors. And because a page only becomes durable when you pay the barrier of Chapter 41, the cost of *persisting* a page — not just computing it — becomes a first-class number the design must respect. This chapter builds the on-disk database in five steps: the file as a linear space of pages, and page-number-to-offset arithmetic (§47.1); the slotted-page layout that holds variable-length records (§47.2); how a single record is laid out in bytes, with fixed and variable fields and a null bitmap (§47.3); the cost of making a page durable, measured, and the torn-write hazard (§47.4); and what the storage manager carries into the buffer pool and beyond (§47.5).

On the code in this chapter. The program is C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35), timed with `clock_gettime(CLOCK_MONOTONIC)`; times are wall-clock and a hardware-counter profiler (`perf`) is *not* installed. The durability measurement writes 4 KiB pages to an `ext4` file on this machine's WSL2 virtual disk (`/dev/sdd`, `physical_block_size` 4096) and calls `fsync`/ `fdatasync`; because this is a **virtualised** disk behind a hypervisor, the per-`fsync` cost reflects that stack, not a bare NVMe SSD with power-loss-protected cache — the *ratio* between per-page and batched syncing is the portable lesson, and the run-to-run variance (noted below) is real. The slotted-page and record layouts are described in structural terms and cross-checked against the standard references (Hellerstein et al.; Ramakrishnan & Gehrke, Chapter 9).

47.1 The database file: a linear space of pages

The storage manager takes one (or a few) ordinary OS files and treats each as a contiguous array of fixed-size **pages**, numbered 0, 1, 2, and so on. That is the entire on-disk model, and its power is in the arithmetic: page number p lives at byte offset $p \times pagesize$, so to read page p the manager issues `pread(fd, buf, pagesize, p*pagesize)` and to write it, `pwrite` at the same offset — a whole page, always, never a partial byte range. Choosing *pagesize* a multiple of the device's physical block (4096 on this machine) means one page is a clean unit of device I/O, so a page read is a single well-aligned block transfer and a page write does not force the device to read-modify-write a block it only partly touched. This is the direct payoff of Chapter 46: the page is the granularity at which the database pays the disk, chosen to match the granularity at which the disk charges. A page identifier — the pair (file, page number) — is the database's fundamental address, the equivalent of a pointer but for data that lives on disk and outlives the process; everything higher up, from a row's location to a B-tree node's children, is ultimately expressed as page numbers. Growing the file is just appending pages at the end (a sequential write, the pattern the disk likes); the manager keeps track of which pages are in use, often with a small allocation structure or a free-space map, but the essential picture stays simple: **the database on disk is an array of pages, addressed by number, read and written whole.**

> **QUICK CHECK**
>
> Given a page size of 8 KiB, at what byte offset does page number 5 begin, and how many bytes does the storage manager read to fetch it? Why is choosing the page size to be a multiple of the device's physical block size (4096 here) worth doing? (Answer after the next section.)

47.2 Inside a page: the slotted-page layout

A page has to hold records that are *variable-length* — a row with a short name and a row with a long one — and it has to let you insert, delete, and occasionally grow them without shifting every other record or leaving unusable gaps. The structure that solves this, used by essentially every row-store, is the **slotted page** (Ramakrishnan & Gehrke, Chapter 9; Hellerstein et al., the storage-manager section). The page is a fixed-size buffer with two regions that grow toward each other from opposite ends. At the top is a small **header** (how many slots, where the free space begins) followed by a **slot directory** — an array of slots that grows *downward*; at the bottom is the record data, which grows *upward*. Each **slot** is a tiny fixed-size entry holding the (offset, length) of one record within the page. To find a record you index into the slot directory and follow its offset; to insert one you place its bytes just above the current record data, add a slot pointing at them, and advance both boundaries; to delete one you mark its slot free (and can later compact the page to reclaim the hole). This indirection — you address a record by *slot number*, and the slot holds the real offset — is what makes the layout flexible: a record can be moved within the page (during compaction) and only its slot's offset changes, so every external reference, which names the slot, stays valid. That external reference is the **record identifier**, the pair (page number, slot number), and it is how an index or a scan names a specific row: the page number gets you the page from disk (or the buffer pool), and the slot number gets you the record within it. Two design points fall out. First, the slot directory means records need not be stored in any particular order, yet can be scanned in slot order and addressed stably — order on disk and order of reference are decoupled. Second, when a record must grow beyond the free space of its page, the layout offers an escape: leave a **forwarding** pointer in the original slot to the record's new home on another page, so its record identifier still resolves — at the cost of one extra page access, a small echo of Chapter 46's “one more disk trip” being the unit of expense.

THE SAME IDEA THREE WAYS: ADDRESS RECORDS BY SLOT, NOT BY RAW OFFSET, SO THEY CAN MOVE WITHIN A PAGE

1. In words. A slotted page keeps a directory of (offset, length) slots at one end and the record bytes at the other. You name a record by its slot number; the slot holds where the bytes actually are. So the page can compact or rearrange records and only slots change — every outside reference, which uses the slot number, keeps working.

2. As a picture. Figure 47.1: a page with a header and a slot directory growing down from the top, record data growing up from the bottom, free space in the middle; arrows from two slots to their records, and a third slot marked free (a deleted record's hole awaiting compaction).

3. As numbers/contrast. A record identifier is (page number, slot number): the page number costs one page fetch — a buffer-pool hit (~tens of ns) or a disk miss (~hundreds of µs, Chapter 48) — and the slot number costs an array index. Storing a raw byte offset instead would save the indirection but freeze every record in place; the slot's one extra lookup buys the freedom to compact and reclaim space.

(Answer to the §47.1 quick check: page 5 begins at byte offset 5 × 8192 = 40,960, and the manager reads the whole page — 8,192 bytes — to fetch it. Making the page size a multiple of the physical block (4096) means each page read is one aligned block transfer and each page write covers whole device blocks, so the device never has to read-modify-write a block the page only partly filled — the page is charged as one clean unit of I/O.)

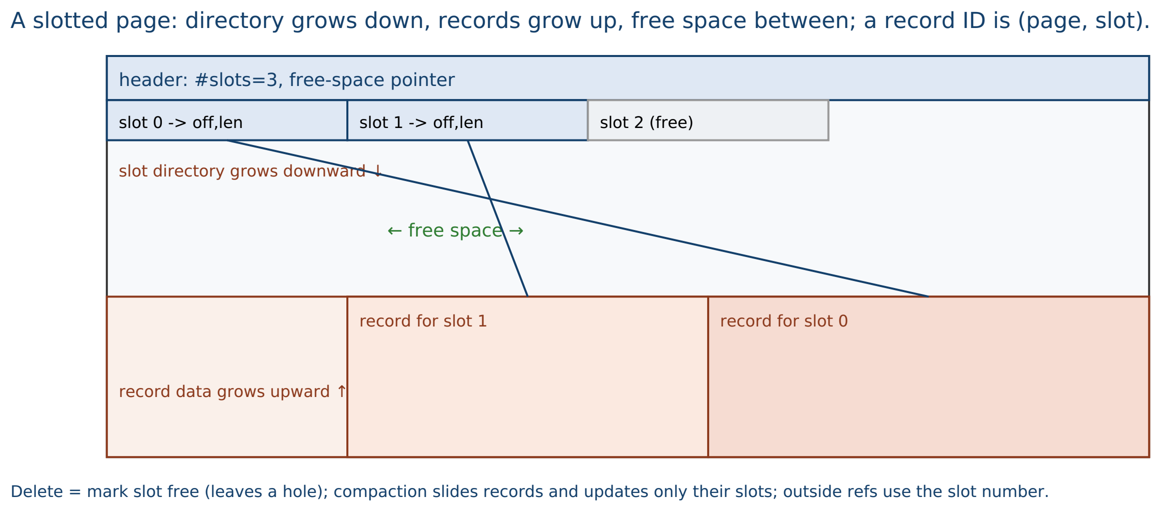


Figure 47.1: A slotted page. The header and slot directory grow down from the top; record bytes grow up from the bottom; free space sits between. A record is addressed by its slot number, so compaction can move the bytes and only the slot's offset changes — every reference by (page, slot) stays valid.

47.3 A record in bytes: fixed fields, variable fields, and nulls

Inside a slot's bytes is one **record** — a row — and laying it out is the same discipline as laying out a C struct (Chapter 7), with two complications the struct did not have: fields can be variable-length, and fields can be null. The standard layout splits the record into a fixed-length part and a variable-length part. Fixed-width columns (an `int`, a `double`, a timestamp) are stored inline at known offsets, so reading column three is a constant-offset load, exactly like a struct member — and alignment matters here for the same reason it did in Chapter 7, though on disk the tradeoff is space versus access convenience rather than a hardware fault. Variable-width columns (a `varchar`) cannot sit at a fixed offset because their length is not known at schema time, so the record stores, in the fixed part, an **offset** (or length) pointing into a variable-length area at the end of the record where the actual bytes live; reading such a column is a two-step indirection within the record, the same pattern as the slot directory one level up. Nullability is handled with a **null bitmap**: a small run of bits in the record header, one per nullable column, marking which columns are null so their storage can be omitted or ignored — checking a bit is cheaper and denser than storing a sentinel value in the field itself. Turning a record into these bytes is **serialization**; turning the bytes back into typed columns on read is **deserialization**; and the format is a contract the storage manager and everything above it must agree on, versioned carefully because a page written under one layout may be read years later. The through-line from Chapter 7 is exact: a record on disk is a struct whose layout you control byte for byte, extended to carry its own variability (offsets for variable fields) and its own optionality (the null bitmap) because, unlike an in-memory struct, it must be self-describing enough to survive the round trip to storage and back.

QUICK CHECK

In a record with both fixed-width and variable-width columns, why can a fixed-width column be read at a constant offset while a variable-width column needs an offset stored in the record? What does the null bitmap buy over storing a special "null" value in each field? (Answer below the communicate box.)

47.4 Making a page durable: the cost of the barrier, measured

Writing a page and *persisting* a page are different acts, and the gap between them is where Chapter 41's lesson becomes a number a storage engine must design around. When the manager calls `pwrite` to store a page, the data lands in the OS page cache and the call returns almost immediately — but the bytes are not yet on the device, and a crash now loses them (Chapter 40, Chapter 41). Durability requires the barrier: `fsync` (or `fdatasync`), which does not return until the device reports the data persisted. That barrier is expensive because it forces the one thing the whole stack is built to defer — an actual, acknowledged trip to the device — and worse, it cannot be pipelined away, so calling it per record serializes the workload behind the disk. To put a number on it, the program appends 2,000 4 KiB pages two ways: one `fsync` at the very end, versus an `fsync` after *every* page. On this machine:

```
append 2,000 x 4 KiB pages to an ext4 file (WSL2 virtual disk), representative run:
  fsync once at the end   :  ~0.06-0.07 s total   (~30-35 µs/page amortized)
  fsync after every page  :  ~6-8 s total         (~3-4 ms/page, ~250-310 durable writes/s)
  -> per-page fsync is on the order of 100x slower than batching one sync at the end
  (fdatasync per page is in the same 3-4 ms range; run-to-run variance is significant)
```

The batched version amortizes a single barrier over all 2,000 pages and finishes in tens of milliseconds; the per-page version pays a full device round trip 2,000 times and takes *seconds*, sustaining only a few hundred durable writes per second. The ratio — on the order of **100×** here — is the measured cost of confusing "`write` returned" with "data is safe," and it is exactly why a database does not `fsync` after every row. Instead it *batches* durability: many changes are made in memory and to a sequential log, and one `fsync` makes a whole batch durable at once — group commit and the write-ahead log, which the durability part of this volume builds, are the disciplined answer to this measurement. There is a second, subtler hazard lurking in a page write: **atomicity**. A database page (say 8 or 16 KiB) may be larger than the unit the device writes atomically, so a crash in the middle of persisting a page can leave it **torn** — half old bytes, half new — which corrupts the page rather than merely losing the update. Matching the page size to the device's physical block (4096 here) reduces the exposure, but the general defense, again, is the log: a technique like full-page logging or double-write records the page's intended contents somewhere durable first, so a torn page can be reconstructed after a crash. Both facts — the barrier's cost and the torn-write risk — point the same direction: durability is not free and not automatic, it is a specific expensive operation you batch and protect with a log, and pretending otherwise is how a "safe" storage layer becomes either unusably slow or quietly corrupt.

TRAP: FSYNC PER RECORD — PAYING THE FULL DURABILITY BARRIER ON EVERY WRITE

The intuitive way to make data safe is to force it to disk immediately after each change: `write` the record, `fsync`, move on. It is correct, and it is catastrophically slow — measured here, an `fsync` per 4 KiB page cost ~3–4 ms and capped the system at a few hundred durable writes per second, roughly **100×** slower than syncing once at the end, because each sync is a full device round trip that cannot be pipelined. The trap is seductive because it passes every correctness test — the data really is durable after each call — so the bug is not wrongness but a throughput ceiling that only appears under load, when the system that benchmarked fine on one insert falls over at a few hundred inserts a second. Two related mistakes travel with it. First, believing `write` alone is durable (Chapter 41): it is not, the data sits in the page cache, so a crash loses it — under-syncing corrupts, over-syncing crawls, and both are failures to think about the barrier. Second, assuming a page write is *atomic*: a page larger than the device's atomic unit can be **torn** by a crash, leaving corruption, not just a lost update. The defense is the discipline the durability part formalizes: do not `fsync` per record — batch many changes behind one barrier, make durability a sequential-log write that one `fsync` commits in a group, and protect page atomicity by logging a page's contents before overwriting it in place. “It's safe because we fsync every write” and “it sustains real write throughput” are, without batching, not both true.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: “To guarantee we never lose data, I made the storage layer `fsync` after every single row insert. It's rock-solid — I tested a crash and nothing was lost.” Cover the paragraph and respond in four or five sentences, drawing on §47.4.

Model answer. “The correctness is right, but that design has a hard throughput ceiling we'll hit fast. An `fsync` is a full device round trip that can't be pipelined — I measured about 3–4 ms per sync on our storage, which caps us at only a few hundred durable inserts per second, roughly 100× slower than syncing once for a batch. The fix isn't to sync less safely; it's to batch durability: make the changes in memory and to a sequential write-ahead log, then one `fsync` commits a whole group of inserts at once — that's what group commit is, and it keeps the same crash guarantee. We should also make sure a page write can't be torn across a crash — if our page is bigger than the device's atomic unit, we log the page before overwriting it. So let's keep the durability guarantee but move it to a batched log-and-sync instead of a sync per row; same safety, orders of magnitude more throughput.” Naming the per-sync cost, the group-commit fix, and the torn-page risk is the senior register.

(Answer to the §47.3 quick check: a fixed-width column has a known size, so its position within the record is a compile-time constant and reading it is a constant-offset load; a variable-width column's length is not known in advance, so it cannot occupy a fixed position — the record stores an offset/length in its fixed part pointing to the bytes in a variable-length area, and reading it takes that extra indirection. The null bitmap packs one bit per nullable column, so testing or marking null is a dense bit check and the column's storage can be omitted entirely, rather than reserving space in every row for a special sentinel value.)

47.5 What to carry forward

A database turns the block layer into an array of fixed-size **pages** addressed by number, where page p lives at offset $p \times pagesize$ and is read and written whole, the page size a multiple of the device's physical block so each page is a clean unit of I/O (§47.1). Inside a page, the **slotted-page** layout keeps a directory of (offset, length) slots at one end and record bytes at the other, so records are addressed by slot number and can be compacted or moved while every (page, slot) record identifier stays valid (§47.2). A **record** is laid out like a struct that must describe itself — fixed-width fields inline, variable-width fields via stored offsets into a trailing area, optionality via a null bitmap — serialized on write and deserialized on read (§47.3). And persisting a page is a distinct, expensive act: measured here, an `fsync` per page cost ~3–4 ms and ~250–310 durable writes/s, on the order of **100×** slower than one batched barrier, while a page larger than the device's atomic unit risks a **torn write** — so durability must be batched and protected with a log (§47.4).

This is the storage manager's on-disk world, and it hands two things forward. The immediate one is the subject of the next chapter: pages are the unit of caching, and because a disk miss is hundreds of microseconds while a memory hit is tens of nanoseconds (Chapter 46), the database keeps hot pages in DRAM in its own hand-managed cache — the **buffer pool** of Chapter 48 — pinning pages in use, tracking which are dirty, and choosing which to evict when memory fills. The deferred one is the answer to this chapter's durability measurement: the batching-and-log discipline sketched in §47.4 becomes, in the durability part of this volume, the write-ahead log and group commit that turn the expensive `fsync` barrier into a throughput a real system can sustain, and the torn-write defense into crash-consistent recovery. The page is the currency; the slot makes it flexible; the barrier makes durability real but costly. Everything the storage engines build next — B-tree nodes and LSM runs alike — is made of these pages, laid on this file, persisted through this barrier.

CAN YOU… WITHOUT LOOKING?

- Give the offset of page p and explain why the page size is a multiple of the physical block size.
- Draw a slotted page and explain why records are addressed by slot number rather than raw offset.
- Define a record identifier and say what each of its two parts costs to resolve.
- Lay out a record with fixed and variable fields and a null bitmap, and say why variable fields need stored offsets.
- Quote the measured per-page vs batched `fsync` cost here and explain why per-record syncing is a throughput trap.
- Explain what a torn write is and why matching page size to the physical block, plus logging, defends against it.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** How does the storage manager turn a file into pages, and how does it compute where page *p* lives? (§47.1)
2. **R2.** What are the parts of a slotted page, and why does addressing records by slot number allow compaction? (§47.2)
3. **R3.** How are fixed-width, variable-width, and nullable columns stored in a record? (§47.3)
4. **R4.** What were the measured per-page and batched `fsync` costs, and what does the ratio tell you to do? (§47.4)
5. **R5.** What is a torn write, and what two defenses reduce or repair it? (§47.4)

FURTHER READING

- **Raghu Ramakrishnan & Johannes Gehrke, *Database Management Systems*, 3rd ed. (McGraw-Hill, 2003), ISBN 978-0-07-246563-1, Chapter 9 ("Storing Data: Disks and Files").** The standard textbook treatment behind §47.1–§47.3: pages, the slotted-page layout, record formats, and record identifiers. First author Raghu Ramakrishnan; edition and ISBN verified via the publisher.
- **Joseph M. Hellerstein, Michael Stonebraker & James Hamilton, "Architecture of a Database System," *Foundations and Trends in Databases* 1(2), 2007, DOI 10.1561/1900000002.** The storage-manager sections frame §47.1–§47.4: the file and page abstraction, the buffer manager it feeds, and where durability sits. First author Joseph M. Hellerstein; venue, year, and DOI verified via the ACM Digital Library.
- **Marshall K. McKusick, William N. Joy, Samuel J. Leffler & Robert S. Fabry, "A Fast File System for UNIX," *ACM Transactions on Computer Systems* 2(3), 1984, pp. 181–197, DOI 10.1145/989.990.** The classic on-disk-layout paper behind §47.1: block sizes, allocation for locality, and turning random file access into sequential I/O — the file-system ancestor of a database's page file. First author Marshall K. McKusick; venue, year, and DOI verified via the ACM Digital Library.
- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, "File System Implementation" (free online).** The OS-level counterpart to §47.1: how a file becomes blocks on a device, inodes, and allocation — the abstraction the database file sits on. First authors the Arpaci-Dusseaus; chapter title verified against the online edition.

Exercises

47.1 WARM-UP With a 4 KiB page size, at what byte offset does page number 10 begin, and how many bytes does one page read transfer?

47.2 WARM-UP Name the two regions of a slotted page and the direction each grows, and say what one slot contains.

47.3 WARM-UP In one sentence each: why does `write` returning not mean the data is durable, and what does `fsync` do about it?

47.4 CORE Record identifiers. (a) What are the two components of a record identifier, and what does each cost to resolve? (b) Explain why addressing a record by slot number (not raw byte offset) lets the page compact away deletion holes without breaking outside references. (c) What is a forwarding pointer, when is it needed, and what does it cost on read?

47.5 CORE Record layout. (a) For a row with an `int`, a `double`, and two `varchar` columns (one nullable), sketch the byte layout: fixed part, variable-length area, and null bitmap. (b) Explain why the two `varchar`s need stored offsets while the `int` and `double` do not. (c) Why is a null bitmap denser and cheaper than a sentinel value per field?

47.6 CORE The durability measurement. (a) Per-page `fsync` ran ~100× slower than one `fsync` at the end here; explain physically why, in terms of what a sync forces and why it can't be pipelined. (b) What is the batched alternative a database actually uses, and how does it keep the same crash guarantee? (c) Why does the per-record-`fsync` design pass a correctness test yet fail under load?

47.7 COST A workload commits N records. Let one `fsync` cost s seconds and let non-sync per-record work be negligible. (a) Write the total durability time if you `fsync` per record, and if you `fsync` once per batch of k records. (b) With s = 3 ms, N = 100,000, compute both for $k = 1$ and $k = 1000$. (c) What is the sustainable durable-commit rate in each case, and what does this say about why group commit exists?

47.8 FADED *Worked, with the last step for you.* A storage layer uses 16 KiB pages on a device whose atomic write unit is 4 KiB. It overwrites pages in place and does no logging. After a crash during a page overwrite, a page is found half-updated and the database won't open.
Step 1 (given): a 16 KiB page spans four 4 KiB atomic units, written non-atomically.
Step 2 (given): a crash between the units leaves the page torn — part old, part new — which is corruption, not a lost update.
Step 3 (given): in-place overwrite with no prior durable copy means the old contents are gone too.
Step 4 (you): Explain why this is worse than merely losing the last write, and describe a logging scheme that would let recovery reconstruct the correct page.

47.9 MIXED For each, say whether it belongs in the *fixed part*, the *variable-length area*, or the *null bitmap* of a record, and why: (a) a 4-byte integer key; (b) a 200-character product description; (c) the fact that the description column is absent for this row; (d) an 8-byte timestamp; (e) the offset that locates the description within the record.

47.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “A slotted page stores records in sorted order so scans are fast.” (b) “A record identifier is a raw byte offset into the file.” (c) “After `pwrite` returns, the page is safely on the device.” (d) “Calling `fsync` once per batch gives weaker durability than calling it per record.” (e) “A torn write means the last update was lost.”

47.11 STRETCH Synthesize the on-disk design. (a) Trace the two levels of indirection between “I want row R” and its bytes on disk: record identifier → page → slot → record, and say what each level buys. (b) The chapter measured a ~100× penalty for per-page `fsync`; connect that number to why the write-ahead log (built later) is the shape durability must take, not an optional optimization. (c) Argue that a record on disk is “a struct that must describe itself,” using the variable-field offsets and null bitmap of §47.3 as evidence, and say what the round trip to storage adds that an in-memory struct never needs.

47.12 LAB On your own machine (a C toolchain): (a) Append many 4 KiB pages to a file two ways — one `fsync` at the end, and one `fsync` per page — and report total time, per-page time, and sustainable durable writes per second for each. (b) Repeat with `fdatasync` and note any difference. (c) State your toolchain and CPU, note the times are wall-clock and whether your disk is virtualised, and explain why the *ratio* between the two is the portable result while the absolute per-sync time is your machine's.

Chapter 48 — The Buffer Pool

Before you start: Chapter 40 (the OS page cache — the buffer pool is the database doing this itself), Chapter 17 (demand paging and page replacement — the same problem, LRU and clock), Chapter 46 (a disk miss is hundreds of microseconds; a memory hit is tens of nanoseconds), Chapter 47 (the page is the unit that gets cached, pinned, and evicted), Chapter 3 (the memory hierarchy — a hit is DRAM, a miss falls off the cliff). · Chapters 46 and 47 gave the device and the page. This chapter puts a layer of DRAM between them and the rest of the database — the pool of pages that turns most accesses into memory hits, and confronts the cliff head-on.

BEFORE YOU READ ON

From memory: (1) Chapter 46 — a random page read from this machine's disk cost on the order of hundreds of microseconds; how does that compare to reading a page already sitting in DRAM? (2) Chapter 17 — when physical memory fills and a new page is needed, the OS must choose a victim to evict; what were the policies it used, and which one did the clock algorithm approximate? (3) *Predict:* a database keeps a pool of pages in memory. Accessing a page that is *in* the pool (a hit) is a memory operation; accessing one that is not (a miss) is a disk read. Two questions: by roughly what factor do you think a miss is slower than a hit here — 10×, 100×, 10,000×? And if a full-table scan reads more distinct pages than the pool holds, what happens to the pages that were hot before the scan? Write your guesses; §48.3–§48.4 answer them.

The disk is the floor and it is a cliff (Chapter 46); the page is the currency (Chapter 47); and no database can afford to touch the disk for most of what it does. The escape is a cache — a region of DRAM holding pages so that repeated accesses hit memory instead of the device — and a database builds its own, called the **buffer pool**, rather than leaning solely on the OS page cache of Chapter 40. It does so deliberately: the database knows things the OS cannot, and a hand-managed cache lets it act on that knowledge — pin a page so a query using it cannot have it yanked away, control the order dirty pages are written so the write-ahead log's rules are honored, prefetch along an access pattern it can predict, and choose its own eviction policy. The buffer pool is, structurally, the same demand-paging problem the OS solved in Chapter 17 — a set of frames, a map from what-you-want to where-it-is, and a replacement policy for when memory fills — solved again on purpose, at the database's level, where the database's knowledge can be applied. This chapter builds it in five steps: why the database manages its own cache instead of trusting the OS (§48.1); the anatomy — frames, the page table, pin counts, and dirty bits (§48.2); the hit-versus-miss gap, measured here at four orders of magnitude, and why hit rate is everything (§48.3); replacement — LRU, the clock approximation, and the sequential scan that defeats them, simulated (§48.4); and what the buffer pool carries into the storage engines and the log (§48.5).

On the code in this chapter. The programs are C, compiled with gcc 11.4.0 and run in this environment on an Intel Core i7-12700H (glibc 2.35), timed with `clock_gettime(CLOCK_MONOTONIC)`; times are wall-clock and a hardware-counter profiler (`perf`) is *not* installed. The hit measurement is a `memcpy` from a resident 4 KiB frame; the miss measurement is an `O_DIRECT` `pread` of a 4 KiB page at random offsets from a 1 GiB file on this machine's WSL2 virtual disk (`/dev/sdd`) — so the miss cost is the virtualised-disk figure of Chapter 46, not a bare device, and its run-to-run variance is real (it landed near ~350 µs here, within the same hundreds-of-microseconds band as Chapter 46's ~221 µs). The replacement results are from a **deterministic simulation** (a fixed pseudo-random reference string, seeded), reported as hit rates, not timings — they compare policies, and are exactly reproducible rather than measured against a clock.

48.1 Why a database manages its own cache

The operating system already caches disk blocks in memory — the page cache of Chapter 40 — so it is fair to ask why a database bothers to build a second cache of its own on top of it. The answer is that the OS page cache is generic by necessity: it serves every process, it does not know what a database page *means*, and it makes eviction and writeback decisions on information the database could improve on. A database, managing its own **buffer pool**, can do four things the OS cache cannot. First, it can **pin** a page: while a query is reading or modifying a page, the database marks it un-evictable, guaranteeing the page stays in memory and at a stable address for the duration — something the OS gives no direct control over. Second, it can **control writeback order**: the write-ahead log (built later in this volume) demands that a page's log records reach disk *before* the page itself, and only a cache the database controls can enforce “do not write this dirty page until its log is durable.” Third, it can **prefetch** along access patterns it understands — the next leaf of a B-tree, the next run of a scan — because it knows the structure of its own data. Fourth, it can choose a **replacement policy** suited to database workloads, which as §48.4 shows differ from the generic case (a large scan should not be allowed to evict everything). The cost of this control is a known awkwardness: the same page can now sit in both the OS page cache and the database's buffer pool, wasting memory and copying twice — **double buffering** — which is exactly why databases often open their files with `O_DIRECT` (Chapter 41) to bypass the OS cache and let the buffer pool be the single authority on which pages are in memory. The principle is the recurring one of this volume: the layer that has the most specific knowledge should make the decision, and a database knows more about its own pages than the OS ever can.

> **QUICK CHECK**
>
> Name two things a database's own buffer pool can do that it cannot ask the OS page cache to do for it. What is double buffering, and why do databases sometimes use `O_DIRECT` to avoid it? (Answer after the next section.)

48.2 Anatomy: frames, the page table, pins, and dirty bits

A buffer pool is a fixed-size array of **frames** — each frame a slot of exactly one page's size (Chapter 47) — plus the bookkeeping to find, protect, and reclaim them. Finding a page uses a **page table** (a hash table) mapping page identifier → frame index: given a page number, a lookup says either “it is in frame 7” (a hit) or “it is not resident” (a miss). On a miss the pool picks a free or evictable frame, reads the page into it from disk (Chapter 47's `pread`), and records the mapping. Two per-frame flags govern the frame's life. The

pin count (or reference count) tracks how many callers are currently using the page: `fetchPage` looks the page up, brings it in if absent, increments the pin count, and returns a pointer into the frame; the caller works on it and then calls `unpin` to decrement. A frame with a nonzero pin count is **in use and cannot be evicted** — this is the mechanism promised in §48.1, and it is essential for correctness: evicting a page a query is mid-read on would pull memory out from under it. The **dirty bit** records whether the page has been modified since it was read in; when a dirty page is chosen for eviction, it *must be written back* to disk first (respecting, later, the log-before-page rule), whereas a clean page can simply be dropped because the on-disk copy is already current. So the lifecycle of a page in the pool is: fetched (pinned, mapped into a frame) → used (read or written, setting the dirty bit) → unpinned → eventually chosen as a victim → written back if dirty → frame reused for another page (Effelsberg & Härder, "Principles of Database Buffer Management," formalize exactly this — the fixing of pages, the frame allocation, and the replacement search as the buffer manager's three tasks). This anatomy is small, but every field earns its place: without the page table you cannot find a resident page; without the pin count you can corrupt an in-flight read; without the dirty bit you either lose updates or waste writes.

THE SAME IDEA THREE WAYS: A BUFFER-POOL HIT IS A MEMORY ACCESS; A MISS IS A FALL OFF THE DISK CLIFF

1. In words. The page table turns a page number into either "in frame N" (a hit — read it from DRAM) or "not resident" (a miss — read it from disk into a frame first). A hit is a memory operation; a miss pays the device latency of Chapter 46. So the fraction of accesses that hit — the *hit rate* — governs the effective speed of the whole database.

2. As a picture. Figure 48.1: a page-number request entering a hash table; a hit arrow going straight to a frame in DRAM (~tens of ns); a miss arrow going out to the disk, back into an evicted frame (~hundreds of µs), then to the caller — the two paths drawn to scale on a log axis so the miss path dwarfs the hit.

3. As numbers/contrast. Measured here: a hit (memcpy from a resident 4 KiB frame) ~**36 ns**; a miss (`O_DIRECT pread`) ~**350 µs** — about **10,000×**. So effective access time is $h \times 36$ ns $+ (1-h) \times 350$ µs: at $h = 0.99$ the misses still dominate the average, which is why databases chase the last few percent of hit rate (§48.3).

*(Answer to the §48.1 quick check: a database buffer pool can, among other things, **pin** a page so an in-use page is never evicted, **order writeback** so a dirty page is not flushed before its log records are durable, **prefetch** along access patterns it predicts, and pick a database-suitable **replacement policy** — none of which the generic OS page cache exposes. Double buffering is the same page being held in both the OS page cache and the buffer pool, wasting memory and an extra copy; opening files with `O_DIRECT` bypasses the OS cache so the buffer pool is the single authority, avoiding the duplication.)*

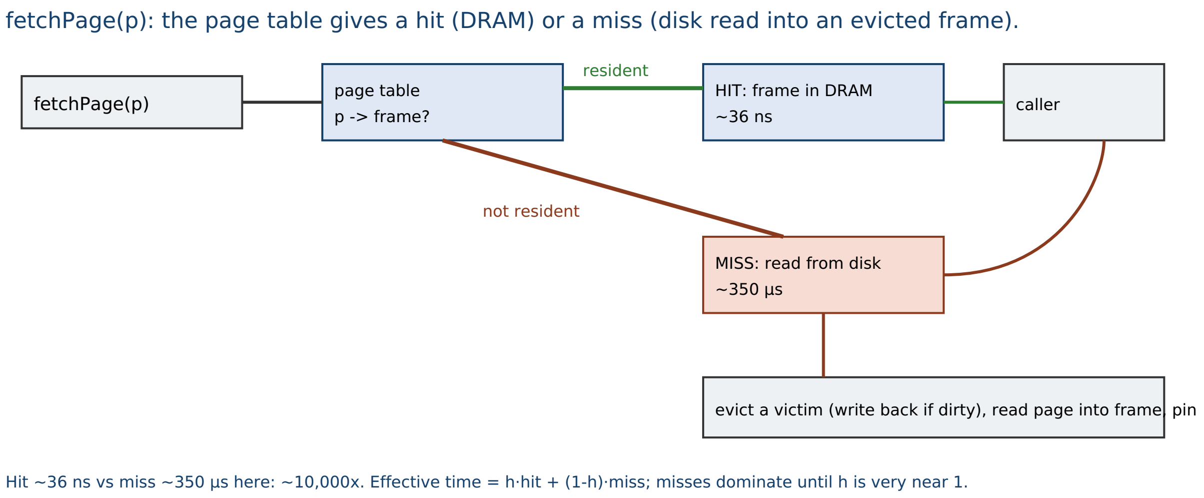


Figure 48.1: `fetchPage` resolves a page number through the page table. A hit reads a resident frame from DRAM (~36 ns here); a miss evicts a victim (writing it back if dirty), reads the page from disk into the frame (~350 µs here), pins it, and returns — about 10,000× the cost of a hit.

48.3 The hit that saves the cliff, measured

The whole justification for a buffer pool is the size of the gap between a hit and a miss, so it is worth measuring rather than asserting. A **hit** — the page is already in a frame — costs a memory access: here, copying a resident 4 KiB page out of its frame took about **36 nanoseconds**. A **miss** — the page is not resident and must be read from disk — costs a device I/O: here, an `O_DIRECT` read of a 4 KiB page landed around **350 microseconds** (the Chapter 46 virtualised-disk figure, in the same hundreds-of-microseconds band as the ~221 µs measured there; the exact value wanders with the virtual disk). The ratio is about **10,000×**:

```
4 KiB page access on this machine:
  HIT  (memcpy from a resident frame)      : ~36 ns
  MISS (O_DIRECT pread from virtual disk)  : ~350 µs   (~hundreds of µs, cf. Ch.46)
  miss / hit  ~ 10,000x
```

Four orders of magnitude is a different kind of number from the 49× of Chapter 46, and it changes how you must think about the average. The **effective access time** of a page request is a weighted average of the two outcomes: if a fraction h of accesses hit, the mean cost is $h \times 36$ ns $+ (1-h) \times 350$ µs. The startling consequence of the 10,000× gap is that the misses dominate the average until the hit rate is *extremely* close to one. At a 90% hit rate the average is about 35 µs — essentially all misses, the hits contributing nothing; even at 99% it is about 3.5 µs, still a hundred times the hit cost. Only as h approaches 99.9% and beyond do the hits begin to set the pace. This is why a database treats its hit rate as a first-class health metric and why the last fractions of a percent are worth fighting for: raising the hit rate from 99% to 99.9% cuts the average access time roughly tenfold, because it removes nine-tenths of the remaining ruinously expensive misses. It is also why buffer-pool *sizing* — how much DRAM you devote to frames — is among the highest-leverage tuning decisions in operating a database: more frames means more of the working set stays resident, which means a higher hit rate, which, on this curve, means a disproportionate drop in effective latency. The buffer pool does not make the disk faster; it makes you visit the disk rarely, and the arithmetic of the hit/miss gap is what turns "rarely" into "fast."

> **QUICK CHECK**
>
> With a hit at ~36 ns and a miss at ~350 µs, why does a 90% hit rate still give an average access time almost entirely made of miss cost? Roughly what does raising the hit rate from 99% to 99.9% do to the average, and why is that the reason to fight for the last fractions of a percent? (Answer below the communicate box.)

48.4 Replacement: LRU, the clock approximation, and the scan that breaks them

When every frame is full and a miss needs one, the buffer pool must evict a resident, unpinned page — and which one it picks determines the hit rate the previous section showed to be everything. The ideal is to evict the page that will be needed furthest in the future, which is unknowable, so the standard proxy is **least-recently-used** (LRU): evict the page unused for the longest time, betting that the recent past predicts the near future (Chapter 17 met exactly this bet for OS pages). True LRU means updating a page's position on *every* access, which is bookkeeping the pool would rather not do on a hot path, so real systems approximate it with the **clock** (second-chance) algorithm: frames sit in a circle, each with a reference bit set on access; a hand sweeps the circle, and when it needs a victim it clears the reference bit of a page it finds set (giving it a "second chance") and evicts the first page it finds already clear — approximating LRU at a fraction of the cost (OSTEP, "Beyond Physical Memory: Policies"). A deterministic simulation on a skewed access pattern (1,000 pages, a 100-frame pool, 200,000 references) shows clock tracking LRU closely and both beating FIFO:

```
replacement simulation (skewed access; 1000 pages, pool=100, 200000 refs):
  FIFO  hit rate : 17.2 %
  CLOCK hit rate : 18.1 %     (approximates LRU cheaply)
  LRU   hit rate : 18.6 %

looping sequential scan of 110 pages through a 100-frame pool:
  FIFO  hit rate : 0.0 %
  CLOCK hit rate : 0.0 %
  LRU   hit rate : 0.0 %
```

The first block is the reassuring result: clock (18.1%) sits between FIFO (17.2%) and true LRU (18.6%), confirming it is a cheap, faithful LRU approximation — the reason it is the workhorse of real buffer pools. The second block is the warning, and it is the named trap of this chapter. A **looping sequential scan** over slightly more pages than the pool holds — 110 pages cycling through 100 frames — produces a **0% hit rate under every policy**, because by the time the scan comes back around to a page, LRU (and clock, and FIFO) has just evicted it as the least-recently-used to make room for the pages in between: each page is thrown out moments before it is needed again. This is **sequential flooding**, and it is worse than its own zero hit rate suggests, because a single large scan does not just fail to benefit from the cache — it *evicts everyone else's hot pages* to fill the pool with scan pages it will never reuse, collapsing the hit rate for the whole system while it runs. It is exactly why databases do not use naive LRU for the buffer pool: they use scan-resistant policies — LRU-K, which tracks the last K references so a page touched once by a scan is not treated as "recently used," or the query-locality approach of DBMIN (Chou & DeWitt, "An Evaluation of Buffer Management Strategies for Relational Database Systems"), which gives each query its own locality set — or they simply refuse to cache scan pages, reading them through a small dedicated ring buffer so a big scan cannot flood the pool. The lesson generalizes the whole chapter: the replacement policy is not a detail but a defense, because the access patterns that break the simplest good-enough policy (the sequential scan) are common in exactly the systems that most need the cache to hold.

TRAP: LETTING A BIG SEQUENTIAL SCAN FLOOD THE BUFFER POOL AND EVICT THE WORKING SET

Naive LRU (or clock) has a specific, common failure mode: a sequential scan that touches more distinct pages than the pool holds gets a **0% hit rate** on its own pages — measured here, a 110-page loop through a 100-frame pool hit nothing under FIFO, clock, and LRU alike — because each page is evicted just before the scan wraps back to it. The trap is not merely that the scan is uncached; it is that the scan *pulls every other query's hot pages out of the pool* to make room for pages it will use exactly once, so a single analytics scan can tank the hit rate — and therefore the latency (§48.3) — of the entire system while it runs. This surfaces as the classic operational mystery: the transactional workload was fast, someone runs a nightly full-table report, and suddenly unrelated queries slow to a crawl and blame lands on "the cache being broken" when the cache is working exactly as a naive policy dictates. The defenses are to make the policy scan-resistant rather than to make the pool bigger (a scan larger than *any* pool still floods): track multiple references so a once-touched page is not deemed hot (LRU-K), give each query its own locality budget (DBMIN), or read large scans through a small ring buffer that never displaces the general pool. "We use LRU, so the hot pages stay hot" and "a full scan runs concurrently" are, without a scan-resistant policy, not both safe to assume.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate reports: "Every night our reporting job does a full scan of the orders table, and while it runs, the normal customer queries — which were fast all day — get slow. The buffer pool must be too small or broken." Cover the paragraph and respond in four or five sentences, drawing on §48.3 and §48.4.

Model answer. "The pool isn't broken — this is sequential flooding, and a bigger pool won't fix it. A full scan reads more distinct pages than any reasonable pool holds, and with plain LRU each scan page evicts one of the customer queries' hot pages to make room, so the scan fills the pool with pages it uses exactly once and pushes out the working set everyone else depends on. Because a miss here is about 10,000× a hit, dropping the hit rate even a little sends effective latency up sharply — that's the daytime slowdown. The fix is a scan-resistant policy, not more memory: don't let a single-touch scan page count as 'hot' (LRU-K), or give the reporting query its own small locality budget, or run big scans through a dedicated ring buffer so they can't displace the shared pool. So let's make the scan not poison the cache rather than chase pool size." Naming sequential flooding, the miss/hit ratio's leverage on latency, and a scan-resistant fix is the senior register.

(Answer to the §48.3 quick check: at a 90% hit rate the average is 0.9 × 36 ns + 0.1 × 350 µs ≈ 35 µs — the 10% of misses, each ~10,000× a hit, contribute essentially the entire average while the hits are rounding error. Raising the hit rate from 99% to 99.9% removes nine-tenths of the remaining misses, cutting the average from ~3.5 µs to ~0.4 µs — roughly tenfold — which is why the last fractions of a percent, not the first, are where the effective latency is won.)

48.5 What to carry forward

A database builds its own **buffer pool** rather than trusting the OS page cache because it can pin pages, order writeback for the log, prefetch on patterns it predicts, and choose a database-suitable replacement policy — often bypassing the OS cache with `O_DIRECT` to avoid double buffering (§48.1). The pool is an array of **frames** plus a **page table** (page number → frame), a per-frame **pin count** (a pinned page cannot be evicted), and a **dirty bit** (a dirty victim must be written back before reuse) (§48.2). A hit is a memory access and a miss a disk read — measured here ~36 ns against ~350 µs, about **10,000×** — so effective access time is a hit/miss weighted average in which misses dominate until the hit rate is very near one, making hit rate and pool sizing the highest-leverage knobs (§48.3). Replacement approximates the unknowable optimum with **LRU**, cheaply realized by the **clock** algorithm (simulated at 18.1% against LRU's 18.6% and FIFO's 17.2%), but a **sequential scan** larger than the pool defeats them all — 0% hit rate and, worse, it floods out everyone's working set — so real pools use scan-resistant policies like LRU-K or DBMIN (§48.4).

With this, the storage foundation of the volume is complete: the device and its cost model (Chapter 46), the page and its on-disk file (Chapter 47), and the buffer pool that keeps hot pages in DRAM (here). Everything the rest of the volume builds sits on these three. The buffer pool's two loose threads point straight ahead. Its **writeback ordering** — the rule that a dirty page must not reach disk before its log is durable — is precisely the hook the write-ahead log needs, so the durability part of this volume will formalize the buffer pool's dirty-bit lifecycle into the log-before-page protocol and group commit that make Chapter 47's expensive `fsync` a throughput a real system can sustain. And the pool's role as the layer that turns disk pages into memory the query can touch is what lets the storage engines built next — the B-tree, whose nodes are pages fetched through this pool, and the log-structured merge tree, whose runs are read and merged through it — assume that “fetch a page” is usually cheap. The disk is the floor; the page is the currency; the buffer pool is the layer of DRAM that means you rarely have to spend it. On that foundation, the database becomes something you can build.

CAN YOU… WITHOUT LOOKING?

- Give two things a database buffer pool can do that the OS page cache cannot, and explain double buffering.
- Name the parts of a buffer pool — frames, page table, pin count, dirty bit — and what each is for.
- Quote the measured hit vs miss cost here and the ratio, and write the effective-access-time formula.
- Explain why a 90% hit rate is still miss-dominated, and why the last fractions of a percent matter most.
- Describe the clock algorithm and why it is used instead of true LRU.
- Explain sequential flooding, why it defeats LRU, and name one scan-resistant defense.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Why does a database keep its own buffer pool instead of relying on the OS page cache? (§48.1)
2. **R2.** What does the pin count protect against, and what must happen to a dirty page before its frame is reused? (§48.2)
3. **R3.** What were the measured hit and miss costs, and why do misses dominate the average until the hit rate is near one? (§48.3)
4. **R4.** How does the clock algorithm approximate LRU, and how did it compare to LRU and FIFO in the simulation? (§48.4)
5. **R5.** What is sequential flooding, why does it give a 0% hit rate, and how is it defended against? (§48.4)

FURTHER READING

- **Wolfgang Effelsberg & Theo Härder, "Principles of Database Buffer Management," *ACM Transactions on Database Systems* 9(4), 1984, pp. 560–595, DOI 10.1145/1994.2022.** The foundational treatment behind §48.1–§48.2: how a database buffer manager differs from OS paging, the fixing (pinning) of pages, frame allocation, and the replacement search. First author Wolfgang Effelsberg; venue, year, and DOI verified via the ACM Digital Library.
- **Hong-Tai Chou & David J. DeWitt, "An Evaluation of Buffer Management Strategies for Relational Database Systems," *Proc. 11th VLDB* (1985), pp. 127–141.** The source for §48.4's scan problem and its fix: the query-locality-set model and the DBMIN algorithm, which allocate buffers per query rather than by a single global LRU. First author Hong-Tai Chou; venue and year verified via the VLDB proceedings.
- **Joseph M. Hellerstein, Michael Stonebraker & James Hamilton, "Architecture of a Database System," *Foundations and Trends in Databases* 1(2), 2007, DOI 10.1561/1900000002.** Its buffer-manager section situates §48.1–§48.3 in the whole storage stack, including the writeback ordering the log depends on. First author Joseph M. Hellerstein; venue, year, and DOI verified via the ACM Digital Library.
- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, "Beyond Physical Memory: Policies" (free online).** The OS-level companion to §48.4: optimal replacement, LRU, and the clock (second-chance) approximation — the same algorithms the buffer pool reuses. First authors the Arpaci-Dusseaus; chapter title verified against the online edition.

Exercises

48.1 WARM-UP In one sentence, what does the page table in a buffer pool map, and what are the two possible outcomes of a lookup?

48.2 WARM-UP What does a nonzero pin count on a frame mean, and why must a dirty victim page be written back before its frame is reused?

48.3 WARM-UP Roughly what were the measured hit and miss costs for a 4 KiB page on this machine, and what is their approximate ratio?

48.4 CORE Effective access time. (a) Write the effective access time as a function of hit rate h using this chapter's ~36 ns hit and ~350 µs miss. (b) Evaluate it at h = 0.90, 0.99, and 0.999. (c) Explain why the improvement from 0.99 to 0.999 is so much more valuable than from 0.90 to 0.99, and what that implies about buffer-pool sizing.

48.5 CORE The clock algorithm. (a) Describe the reference bit and the hand's sweep, including what happens when the hand finds a set bit versus a clear one. (b) Why is clock preferred over true LRU in a buffer pool? (c) In the simulation clock scored 18.1% against LRU's 18.6% and FIFO's 17.2% — interpret what that ordering confirms about clock.

48.6 CORE Sequential flooding. (a) Explain why a loop over 110 pages through a 100-frame pool gave a 0% hit rate under LRU. (b) Why is this worse than the scan simply being uncached — what does it do to other queries? (c) Name two defenses and say how each prevents the flood.

48.7 COST A query repeatedly scans a working set of W distinct pages, uniformly, through a pool of F frames under LRU. (a) Give the steady-state hit rate when $W \le F$ and when $W > F$ in the looping-scan case, and explain the cliff between them. (b) Using this chapter's hit/miss costs, write the effective access time on each side of the cliff. (c) Explain why this cliff is the quantitative core of the sequential-flooding trap.

48.8 FADED *Worked, with the last step for you.* A team adds a large analytics scan to a database that serves fast point queries. The point queries were at ~99.5% hit rate; while the scan runs, they drop to ~85% and latency spikes.
Step 1 (given): the scan reads far more distinct pages than the pool holds, each used once.
Step 2 (given): under LRU each scan page evicts a hot point-query page to make room.
Step 3 (given): a miss here is ~10,000× a hit, so a hit-rate drop from 99.5% to 85% raises effective latency sharply.
Step 4 (you): Explain why the scan causes the point queries' hit rate to fall, estimate the effect on their average access time using the chapter's numbers, and give a fix that does not require enlarging the pool.

48.9 MIXED For each, say which buffer-pool mechanism is responsible — *pin count*, *dirty bit*, *page table*, *replacement policy*, or *writeback ordering* — and why: (a) a page a query is mid-read on is never evicted; (b) a modified page is written to disk before its frame is reused; (c) a page number is resolved to a frame index; (d) a full scan is prevented from evicting the working set; (e) a dirty page is not flushed until its log records are durable.

48.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "A buffer-pool hit and a miss cost about the same order of magnitude." (b) "The clock algorithm gives exactly the same evictions as true LRU." (c) "A clean victim page must be written to disk before its frame is reused." (d) "Enlarging the buffer pool fixes sequential flooding." (e) "At a 90% hit rate, hits contribute most of the average access time."

48.11 STRETCH Synthesize the storage foundation. (a) Trace a single record read from "query wants row R" down through record identifier (Chapter 47), buffer-pool lookup (here), and possibly disk (Chapter 46), naming the cost at each step and where the cliff is. (b) The buffer pool reuses Chapter 17's demand-paging machinery; state one thing the database knows that the OS does not, and how it changes the design. (c) Argue that hit rate, not raw disk speed, is the number that governs a database's storage performance, using the ~10,000× gap and the effective-access-time curve to make the case.

48.12 LAB On your own machine (a C toolchain): (a) Measure the cost of a 4 KiB page "hit" (a `memcpy` from a resident buffer) against a "miss" (an `O_DIRECT pread` from a large file), and report both and their ratio. (b) Write a small deterministic replacement simulation over a fixed reference string and report FIFO, clock, and LRU hit rates, including a looping scan larger than your pool. (c) State your toolchain and CPU, note the hit/miss times are wall-clock and whether your disk is virtualised, and explain which of your results are measurements and which are a reproducible simulation — and why that distinction matters for what you can claim.

Chapter 49 — Heap Files: Organizing a Table's Pages

Before you start: Chapter 47 (the file is a linear space of pages and a page is a slotted collection of records — this chapter organizes those pages into a table), Chapter 48 (every page a heap file touches is fetched, pinned, and evicted through the buffer pool — a page touch is a buffer-pool lookup that may fall to disk), Chapter 46 (sequential reads are cheap and random reads pay the seek/latency cliff — the difference a table scan lives or dies by), Chapter 40 (the page is the unit of I/O). · Chapter 47 gave you one page and the records inside it; this chapter answers the next question — how thousands of those pages become a *table* you can insert into, delete from, and scan, and where finding room for one more record quietly becomes the cost that dominates.

BEFORE YOU READ ON

From memory: (1) Chapter 47 — inside a slotted page, what does the slot array let you do that a packed array of records cannot, and why does that matter when a record is deleted? (2) Chapter 46 — reading a run of adjacent pages versus reading the same number of pages at random offsets: which was dramatically cheaper on this machine, and by roughly what factor? (3) *Predict:* a table is a pile of pages, and to insert a record you must find a page with enough free bytes. If the table's pages are threaded together as a simple linked list and you walk it from the front looking for room, how many pages do you think you touch per insert once the table has grown to thousands of pages — a constant few, or a number that grows with the table? And when you scan the whole table, is that read sequential or random? Write your guesses; §49.2 and §49.4 answer them.

A table is not a page; it is a great many pages, and the layer that turns "a pile of pages" into "a thing you can insert into, delete from, and read back" is the **heap file** — the simplest and most common physical organization for a table's rows. It is a *heap* in the plain English sense: an unordered collection. The records live wherever there was room; the file makes no promise about the order in which a scan returns them, only that every record inserted and not yet deleted will be returned exactly once. That modest contract is enough to build a database on, because ordering and fast lookup are the job of the *indexes* built in the next part — the heap file's job is to store the rows compactly and hand them back. Everything it does reduces to three operations over the pages of Chapter 47, fetched through the buffer pool of Chapter 48: **insert** a record (find a page with room, place it, record its identifier), **delete** a record (mark its slot free), and **scan** the file (read every page and yield its live records). The whole chapter is the observation that the hard part of that list is the parenthetical in *insert* — **find a page with room** — because how the file tracks free space decides whether an insert touches one page or thousands. This chapter builds it in five steps: the heap file as an unordered collection of pages and the record identifier that names a row (§49.1); the two classic ways to find free space — a linked list of pages versus a page directory — and the enormous gap between them (§49.2); insert, delete, and the update that grows a record past its slot, with its named trap (§49.3); the full table scan and why the heap file is built for sequential I/O (§49.4); and what the heap file hands up to the indexes and the storage manager (§49.5).

On the code in this chapter. The free-space result is a **deterministic simulation**, not a timing: a C program (compiled with gcc 11.4.0 on this machine's Intel Core i7-12700H, glibc 2.35) inserts 200,000 variable-size records — sizes drawn from a fixed, seeded pseudo-random sequence so the run is exactly reproducible — into a heap of 4 KiB pages, and *counts* how many data-page fetches each free-space scheme performs. It reports counts, not wall-clock, precisely because the point is a structural comparison that does not depend on the disk's speed; where the cost of those page fetches matters, it is the Chapter 48 buffer-pool hit/miss cost and the Chapter 46 sequential-versus-random figures measured earlier, which this chapter cites rather than re-measures. The sequential-scan argument of §49.4 rests on those earlier measurements (a hardware-counter profiler, `perf`, is *not* installed here, and the disk is a WSL2 virtual disk, both noted where relevant).

49.1 The heap file: an unordered collection of pages

A **heap file** is a set of pages holding the records of one table, with no ordering imposed on the records: a row goes wherever there is space, and a scan returns rows in whatever physical order they happen to sit. This is the default table storage in real systems — a PostgreSQL table with no clustering, an unindexed table anywhere — because it is the cheapest thing that works: inserts do not have to maintain any sorted position, so they are fast, and the ordering that queries need is supplied separately by indexes (the next part) that point *into* the heap. The unit the file deals in is the page of Chapter 47; the unit a query deals in is the record; and the bridge between them is the **record identifier** (the **RID**, also called a tuple id or row id): the pair *(page number, slot number)* that names exactly where a record lives — which page in the file, and which slot within that page's slot array. The RID is the address a heap file hands back when you insert a record and the address an index stores so it can find the record again; it is stable as long as the record does not move, and §49.3 is about the moment it might. Because the page number is an index into the file's linear page space and the slot number is an index into the page's slot array (both from Chapter 47), resolving a RID to bytes is two array indexings and one page fetch: compute the byte offset of page *p*, fetch that page through the buffer pool, read slot *s* of its slot array to get the record's offset and length within the page, and there is the row. No search — a RID is a direct address. That directness is what makes indexes possible: an index does the searching in its own structure and stores a RID at the leaf, so following an index entry to the actual row is one page fetch, not a scan. The heap file, then, is deliberately dumb: it stores rows and names them, and delegates *order* and *fast lookup* upward. Its one genuine problem is the one the plain-English word “heap” hides — when you insert, where do you put the row?

> **QUICK CHECK**
>
> What is a record identifier (RID), what two numbers is it made of, and why does resolving a RID to the actual record bytes require no searching? Why does a heap file make no promise about the order a scan returns rows? (Answer after the next section.)

49.2 Finding free space: the linked list versus the page directory

To insert a record of *b* bytes, the heap file must find a page with at least *b* bytes free (plus a slot). How it finds one is the single decision that separates a heap file that scales from one that does not, and there are two classic schemes. The first is the **linked list of pages**: the file's header points at a chain of pages, and to find room you walk the chain from the front, fetching each page and inspecting its header's free-space count, stopping at the first page that fits (or appending a new page if none does). It is trivial to implement and needs no extra structure — but every page it inspects is a *page fetch* through the buffer pool (§48), and as the file fills with mostly-full pages the walk grows longer, so the cost of an insert climbs with the size of the file. The second scheme is the **page directory**: the file keeps a small separate structure — an array with one compact entry per page recording that page's free space — and to find room you scan the *directory* (a sequence of a few bytes per page, held in memory and cached) rather than the pages themselves, then fetch exactly one data page: the one you have chosen to write into. The directory turns an *O(pages)* chain of expensive page fetches into an in-memory scan plus a single write. The difference is not subtle. In the simulation — 200,000 variable-size records inserted into a heap that grew to 7,075 pages of 4 KiB — the two schemes cost:

```
insert 200,000 records into a heap of 7,075 4-KiB pages (deterministic sim):
  linked-list heap : ~708,000,000 DATA-PAGE fetches to locate room  (~3,542 per insert)
  page-directory   : 0 data-page fetches to locate room; the search is an
                     in-memory directory scan, then ONE page fetch to write
```

Read the first line carefully: the linked-list heap averaged about **3,542 data-page fetches per insert**, and that number is not a constant — it grew as the file grew, because a longer chain of fuller pages means a longer walk before finding room. Each of those fetches is a buffer-pool lookup that, on a miss, is the Chapter 46 disk read; multiply 3,542 by even a small miss rate and an insert that should be nearly free becomes hundreds of disk I/Os. The page directory does **zero** data-page fetches to *locate* free space — the entire search happens in the compact directory array in memory — and then fetches the one page it decided to write. This is why real heap files are directory-organized (or use an equivalent free-space map): the naive linked list is correct but its insert cost is a function of table size, which is unacceptable for a table that grows. The lesson is the recurring storage lesson: keep the metadata you search often small and separate from the data, so that searching it is a memory operation and touching the data is a single, deliberate act.

THE SAME IDEA THREE WAYS: FINDING ROOM TO INSERT IS A SEARCH, AND WHERE YOU SEARCH DECIDES ITS COST

1. In words. An insert must find a page with enough free space. A linked-list heap searches by *fetching pages* and reading each one's free-space header, so its search cost is measured in page fetches and grows with the file. A page-directory heap searches a small in-memory array of per-page free-space counts, so its search cost is a cheap memory scan and it touches exactly one data page — the one it writes.

2. As a picture. Figure 49.1: on the left, an insert arrow walking a chain of pages P0→P1→P2→..., each a buffer-pool fetch, until one has room; on the right, an insert arrow reading a compact directory array (bytes of free-space per page) in memory, landing on the chosen page, and fetching only that one.

3. As numbers. Simulated here (200,000 inserts, 7,075 pages): linked list ~**3,542 data-page fetches per insert** and rising with file size; page directory **0** data-page fetches to locate room plus one write. The difference is not a constant factor — it is *O(pages)* page fetches versus *O(1)*, which is why the directory (or a free-space map) is what real systems use.

(Answer to the §49.1 quick check: a RID is a record identifier, the pair ***(page number, slot number)*** *— the page number indexes the file's linear page space and the slot number indexes that page's slot array. Resolving it needs no search because both are direct array indices: fetch page p, read slot s for the record's in-page offset and length, done. A heap file promises no scan order because records are placed wherever there was free space, not in any key order — ordering is the indexes' job, not the heap's.)*

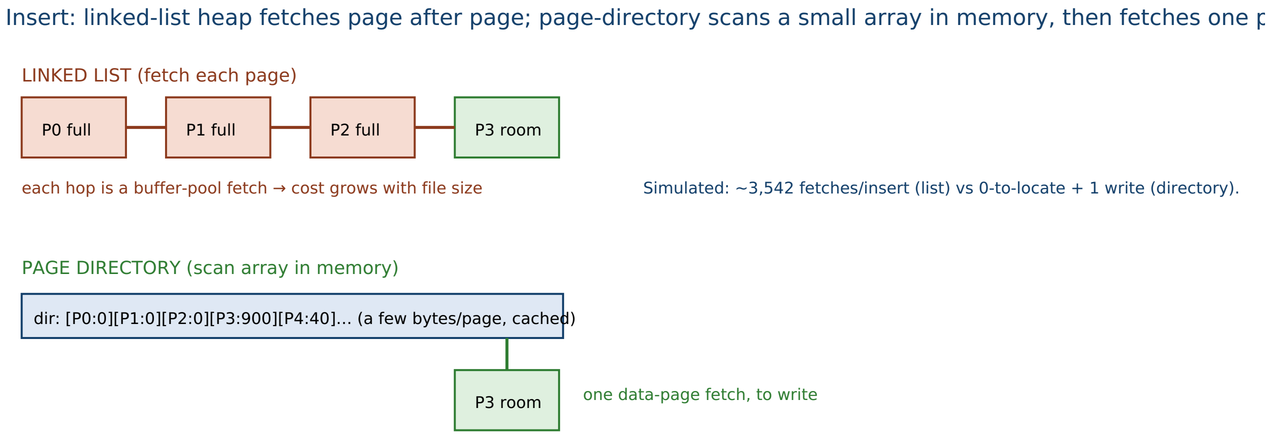


Figure 49.1: finding room for an insert. The linked-list heap fetches page after page until one fits, so its cost grows with the file; the page directory scans a compact in-memory array of per-page free space and then fetches exactly the one page it will write. Simulated here at ~3,542 data-page fetches per insert for the list versus zero-to-locate for the directory.

49.3 Insert, delete, and the update that no longer fits

With free space handled, the three operations are almost simple. **Insert**: find a page with room (§49.2), write the record into free space, add a slot pointing at it, update the page's and directory's free-space counts, and return the RID. **Delete**: fetch the page, mark the record's slot free (a slotted page can leave a hole and compact later — Chapter 47), and add the reclaimed bytes back to the free-space count. A subtlety: the deleted record's slot is usually *not* immediately reused for a different record, because some index may still hold its RID; systems either leave the slot as a tombstone until a later cleanup confirms no references remain, or reuse it and accept that indexes must be maintained in lockstep. The genuinely awkward operation is **update**. If the new version of the record is the same size or smaller, it is easy: overwrite in place, adjust the slot's length, keep the same RID. But if the update makes the record *larger* — a variable-length field grew, a null became a long string — and the page no longer has room for it, the record cannot stay where it is, and the heap file faces a dilemma with no free answer. Moving the record to a page that does have room changes its physical location, which would change its RID — and every index entry pointing at the old RID would now be dangling. The standard escape is a **forwarding pointer** (a **tombstone** or forwarding RID): the record's original slot is left in place but, instead of the record, it now holds a pointer to the record's new location on another page. The old RID stays valid — indexes are undisturbed — but reading the record now costs *two* page fetches: one to the original page to find the forwarding pointer, a second to the page where the record actually lives. This is the named trap of the chapter, and it compounds: a workload that repeatedly grows records can leave a heap file littered with forwarding pointers, so that rows an index should reach in one fetch take two, and a table that was once dense with data becomes a maze of redirections — **row migration**, and the read amplification it causes, is a real and common source of a heap file quietly getting slower over its lifetime.

> **TRAP: THE GROWING UPDATE THAT MIGRATES A ROW AND LEAVES A FORWARDING POINTER BEHIND**
>
> An update that makes a record too big for its current page cannot move the record freely, because its RID is an address stored in every index that covers the table — move the record and those index entries dangle. The near-universal fix is a **forwarding pointer**: leave a tombstone at the old RID pointing to the record's new home. Correct, but not free — **every read of that row now costs two page fetches** instead of one (old page → forwarding pointer → new page), and a fetch that misses the buffer pool is the Chapter 46 disk cliff. The trap is that this degradation is *invisible until it accumulates*: a table loaded once and read is fine; a table whose rows are updated to grow — a status field that lengthens, a JSON blob that gains keys — slowly fills with forwarding pointers, and reads that were one I/O become two with no schema change and no obvious cause. Worse, a page can forward to a record that is itself later updated and forwarded again; naive implementations allow chains of forwards, turning one logical read into several. Real systems bound this by never forwarding a forward (they update the original tombstone to point to the final location) and by *compacting* or *reorganizing* the heap (PostgreSQL's `VACUUM` and `pg_repack`, Oracle's row-chaining rebuilds) to collapse the redirections. “Updates are in place and cheap” and “rows can grow” are, in a heap file, not both true at once — a growing update is a move in disguise.

> **QUICK CHECK**
>
> Why can't a heap file simply relocate a record that has outgrown its page? What does a forwarding pointer buy, and what does it cost on every subsequent read of that row? (Answer below the communicate box.)

49.4 The full table scan: why heap files love sequential I/O

The operation a heap file is *best* at is the one that touches everything: the **full table scan**, which reads every page of the file in order and yields each page's live records. Because a heap file's pages sit in the file's linear page space (§47) and a scan reads them front to back, a table scan is a **sequential** read — exactly the access pattern Chapter 46 measured as dramatically cheaper than random access, because it lets the device stream (no per-page seek on a spinning disk; large, prefetchable requests on an SSD) and lets the buffer pool and OS read ahead. This is the heap file's structural strength and the reason it survives as the default organization: for any query that must examine most of a table — an aggregate over all rows, a filter with no useful index, the build side of a large join — reading the heap sequentially is the *cheapest possible* way to get the bytes off the device, and the total cost is close to (number of pages) × (sequential page read), with the device running near its streaming bandwidth. Two consequences follow. First, **tuple density matters**: the fewer bytes of overhead and dead space per page, the more live records each sequential page read returns, so the same scan bandwidth yields more useful rows — which is exactly why a heap littered with tombstones and half-empty pages (from the deletes and migrations of §49.3) scans slowly even though the device is streaming, and why periodic compaction pays off. Second, the scan is the fallback the whole system leans on: when no index helps, the planner scans the heap, and it can afford to because the scan is sequential. The heap file thus embodies a clean division of labor with the indexes of the next part: the *index* is what you use to find a few rows without reading everything (random access, one RID at a time); the *heap scan* is what you use to read many rows cheaply (sequential access, every page once). A database that understands its own cost model chooses between them per query — and it can, because the heap file makes the “read everything” path as cheap as the device allows.

> **SAY IT LIKE A SYSTEMS ENGINEER**
>
> *Try it first.* A teammate says: “Our `orders` table has gotten slow. Point look-ups by id are fine, but the reads that fetch a single order jumped from about one disk read to two over the last few months, and nobody changed the schema or the indexes.” Cover the paragraph and respond in four or five sentences, drawing on §49.3.
>
> *Model answer.* “That two-fetch pattern with no schema change is the signature of row migration. When an update makes a row too big for its page, the heap can't move it freely — its RID is stored in the indexes — so it leaves a forwarding pointer at the old location and puts the row on another page; every read then costs two fetches, the tombstone plus the real row. If your orders grow after insert — a status field that lengthens, line items appended, a JSON column that gains keys — you'll accumulate forwarding pointers steadily, which is exactly a gradual one-to-two I/O drift with no code change. The fix isn't a new index; it's to reduce migration and collapse the redirects: reorganize or compact the table (`VACUUM FULL` / `pg_repack` in Postgres) to rewrite rows into fresh pages, and if rows are known to grow, leave fill-factor headroom on each page so an update stays in place. So let's measure the forwarding-pointer rate and reorganize, rather than assume the storage got mysteriously slow.” Naming row migration, tying it to the two-fetch cost, and prescribing compaction plus fill-factor headroom is the senior register.

*(Answer to the §49.3 quick check: a heap file cannot freely relocate an outgrown record because its RID — its (page, slot) address — is stored in every index that covers the table, so moving it would leave those index entries dangling. A forwarding pointer keeps the old RID valid by leaving a tombstone that redirects to the record's new page, so indexes are undisturbed; the cost is that every subsequent read of that row is **two page fetches** — the original page to find the pointer, then the page where the record now lives — and each miss is a disk read.)*

49.5 What to carry forward

A **heap file** stores a table's rows as an unordered collection of pages, promising only that every live record is returned once by a scan; order and fast lookup are delegated upward to indexes (§49.1). Each record is named by a **record identifier (RID)** — the pair (page number, slot number) — which resolves to bytes with no search: two array indices and one page fetch, which is exactly what makes an index's "follow the pointer to the row" a single fetch (§49.1). The one hard operation is finding free space to insert: a **linked list of pages** searches by fetching pages and so costs *O(pages)* fetches per insert (~3,542 per insert in the simulation, and rising with file size), while a **page directory** (or free-space map) searches a compact in-memory array and touches one data page — zero fetches to locate room — which is why real systems use it (§49.2). Insert and delete are cheap, but an **update that grows a record past its page** must leave a **forwarding pointer** to avoid changing the RID, turning every later read of that row into two fetches — **row migration**, a slow, invisible degradation defended against by compaction and fill-factor headroom (§49.3). The heap file's great strength is the **full table scan**: reading every page in order is *sequential* I/O, the cheap access pattern of Chapter 46, which is why scanning is the affordable fallback when no index helps and why tuple density is worth defending (§49.4).

The heap file is the second of the storage foundation's two data organizations — the page and its records (Chapter 47) turned into a table you can insert into, scan, and address by RID. Its two loose threads point straight ahead. The free-space directory is the first hint of a recurring pattern: a small, searchable structure kept beside the data so you do not have to scan the data to find something — which is precisely what an *index* is, generalized from "which page has room" to "which page holds key *k*," and the next part builds exactly that. And the RID — a stable address that indexes store and the heap resolves in one fetch — is the currency those indexes trade in: every index leaf ultimately holds a RID, and following it lands on the row through the heap file built here. The heap gives you cheap storage and cheap full scans; the index will give you cheap selective lookups; and the storage manager built next is the module that ties the pages, the buffer pool, the heap file, and the values too big to fit a page into the single interface the rest of the database calls. On the heap file, a table becomes real; on the index, it becomes fast.

CAN YOU… WITHOUT LOOKING?

- Define a heap file and state the one promise it makes about a scan.
- Say what a RID is made of and why resolving it needs no search.
- Explain why a linked-list heap's insert cost grows with the file, and how a page directory fixes it (with the simulated numbers).
- Describe what happens when an update grows a record past its page, and what a forwarding pointer costs on every later read.
- Explain row migration, why it is invisible until it accumulates, and two defenses.
- Explain why a full table scan is sequential I/O and why that makes it the affordable fallback.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a heap file, and how does it divide labor with indexes? (§49.1)
2. **R2.** What is a RID, and why is resolving one to a record a single page fetch with no search? (§49.1)
3. **R3.** Why does a linked-list heap's insert cost grow with the table, and what did the page directory cost instead in the simulation? (§49.2)
4. **R4.** Why can't a heap relocate an outgrown record, and what does a forwarding pointer cost on every later read? (§49.3)
5. **R5.** Why is a full table scan sequential, and why does that make it the affordable fallback when no index helps? (§49.4)

FURTHER READING

- **Raghu Ramakrishnan & Johannes Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill, 2003 (ISBN 978-0072465631), Chapter 9, "Storing Data: Disks and Files."** The direct textbook treatment of §49.1–§49.2: heap files, the two free-space schemes (linked list of pages versus page directory), and page and record formats. First authors Raghu Ramakrishnan and Johannes Gehrke; edition, publisher, year, and ISBN verified via the publisher and retailer listings.
- **Hector Garcia-Molina, Jeffrey D. Ullman & Jennifer Widom, *Database Systems: The Complete Book*, 2nd ed., Pearson/Prentice Hall, 2008 (ISBN 978-0131873254).** Its storage chapters cover how records, blocks, and files are represented on disk, including variable-length records and the record-address (RID) abstraction underpinning §49.1. First author Hector Garcia-Molina; edition, year, and ISBN verified via the publisher listing.
- **Abraham Silberschatz, Henry F. Korth & S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019 (ISBN 978-0078022159).** Its storage-and-file-structure chapter treats variable-length records, free-space management, and the update that no longer fits — the source material behind §49.3's migration problem. First author Abraham Silberschatz; edition, year, and ISBN verified via the publisher listing.
- **Joseph M. Hellerstein, Michael Stonebraker & James Hamilton, "Architecture of a Database System," *Foundations and Trends in Databases* 1(2), 2007, DOI 10.1561/1900000002.** Its storage-and-access-methods sections situate the heap file within the whole engine — how the storage manager exposes pages and records to the layers above. First author Joseph M. Hellerstein; venue, year, and DOI verified via the ACM Digital Library.

Exercises

49.1 WARM-UP In one sentence, what is the only promise a heap file makes about a full scan, and what does it deliberately *not* promise?

49.2 WARM-UP A RID is the pair (page number, slot number). Explain in one or two sentences why resolving a RID to the actual record bytes requires no search, naming the two array indexings and the one page fetch involved.

49.3 WARM-UP Roughly how many data-page fetches per insert did the linked-list heap cost in the simulation, and how many did the page directory cost to *locate* free space? Why is the linked-list number not a constant?

49.4 CORE Free-space schemes. (a) Explain why a linked-list heap file's insert cost grows with the number of pages in the file. (b) Explain how a page directory reduces the cost of *locating* free space to zero data-page fetches. (c) State the asymptotic insert cost (in page fetches to find room) of each scheme as a function of the number of pages, and why that difference is decisive for a growing table.

49.5 CORE The growing update. (a) Why can a heap file overwrite a record in place when an update keeps it the same size or smaller, but not when the update makes it larger than the page's remaining room? (b) Why can't it simply move the record to a page with room? (c) What is a forwarding pointer, and what does it cost on every subsequent read of that row?

49.6 CORE Row migration over time. (a) Describe a workload that steadily fills a heap file with forwarding pointers. (b) Explain why the resulting slowdown is invisible until it accumulates and appears without any schema or index change. (c) Give two defenses and say how each reduces the two-fetch cost.

49.7 COST A table has P pages, each holding on average r live records. (a) Give the cost of a full table scan in page reads, and state whether those reads are sequential or random and why that matters (cite the Chapter 46 result qualitatively). (b) A point look-up by RID costs how many page fetches when the row has not migrated, and how many when it has? (c) If a fraction m of rows have migrated, write the average page-fetch cost of a random point look-up and explain why raising tuple density (compaction) helps both the scan and the look-up.

49.8 FADED *Worked, with the last step for you.* A reporting query aggregates over an entire 2-million-row table and is slower than its raw data size suggests it should be; the table has been heavily updated in place over a year.
Step 1 (given): the query has no useful index, so the planner does a full heap scan — sequential I/O over every page.
Step 2 (given): a year of growing updates has left many pages half-empty (deleted/ migrated rows) and scattered forwarding pointers.
Step 3 (given): the scan reads every page whether or not it is dense, so low tuple density means more pages read per live row.
Step 4 (you): Explain why the scan is slow despite the device streaming sequentially, and give one operation that would speed it up without adding an index — and say what it does to the pages.

49.9 MIXED For each, name the heap-file mechanism or property responsible — *RID*, *page directory*, *forwarding pointer*, *slot array*, or *sequential scan* — and why: (a) an index finds a row in one page fetch after its search; (b) an insert locates free space without touching data pages; (c) a grown record keeps its old address valid after moving; (d) a deleted record can leave a hole that is compacted later; (e) reading every row of an unindexed table is affordable.

49.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “A heap file returns rows in insertion order.” (b) “A page directory makes the cost of finding free space independent of the number of data pages fetched.” (c) “An in-place update that shrinks a record must relocate it.” (d) “A forwarding pointer makes a row's reads cheaper.” (e) “A full table scan is random I/O.”

49.11 STRETCH Synthesize the storage layers. (a) Trace an insert of one record end to end: finding free space via the directory (§49.2), writing the record and slot into a page (§47), through the buffer pool (§48), and what RID is returned. (b) Argue that the free-space directory and an index are the same idea — a small searchable structure beside the data — applied to different questions, and state each question. (c) Explain why a heap file plus indexes is a better division of labor than a single always-sorted file, using the insert cost of §49.2 and the scan cost of §49.4.

49.12 LAB On your own machine (a C toolchain): (a) Write a deterministic simulation that inserts many variable-size records into a heap of fixed-size pages under two free-space schemes — a linked-list walk from the head and a page-directory scan — and count the data-page fetches each performs to locate room; report both and their ratio, and how the linked-list number changes as the file grows. (b) Extend it with a fraction of updates that grow a record past its page, inserting a forwarding pointer, and report the resulting two-fetch read rate. (c) State your toolchain and CPU, and explain which of your results are exact counts from a deterministic simulation (reproducible, hardware-independent) versus anything you timed — and why the counts are the claim that travels.

Chapter 50 — The Storage Manager: The Interface Under Everything

Before you start: Chapter 46 (the device and its cost model — the floor everything rests on), Chapter 47 (the page and the records inside it — the unit and the currency), Chapter 48 (the buffer pool — how a page becomes memory you can touch, pinned and evictable), Chapter 49 (the heap file — pages organized into a table, records named by RID). · The four chapters before this each built one slab of the storage foundation. This chapter names the seam that joins them — the *storage manager*, the single interface the rest of the database calls — and handles the one thing a fixed-size page cannot: a value bigger than the page itself.

BEFORE YOU READ ON

From memory: (1) Chapter 48 — what does `fetchPage(p)` do on a hit versus a miss, and what does the caller promise to do when it is finished with the page? (2) Chapter 49 — a record is named by a RID of (page number, slot number); what happens to that RID when a record grows too big for its page? (3) *Predict:* a page is 4 KiB, but a column might hold a 1 MiB document or a 10 MiB image. It plainly cannot fit in one page. Two questions: roughly how many 4 KiB pages would a 1 MiB value need if you chain overflow pages, and — the subtler one — what happens to a *table scan* if you instead try to cram that big value *inline* in the row, next to the small columns queries actually read? Write your guesses; §50.3 answers them.

Four chapters have each built one part of a storage engine: the device and its cost model (Chapter 46), the page and its records (Chapter 47), the buffer pool that turns pages into memory (Chapter 48), and the heap file that turns pages into a table (Chapter 49). What has not yet been named is the thing that joins them — the **storage manager**, the module that presents all of this to the rest of the database as one coherent interface. The layers above it — the indexes and access methods of the next part, the query executor after that — must not each know about disk geometry, buffer frames, slot arrays, and free-space directories; they need a small, stable vocabulary: *give me page P*, *here is a page, make it durable*, *allocate a new page*, *free this page*, *read the record at this RID*. The storage manager is the layer that provides exactly that vocabulary and hides everything below it, so that a B-tree can be written as "a tree whose nodes are pages" without the B-tree code ever issuing a `pread` or choosing an eviction victim. This chapter does two jobs. It makes the interface explicit — what the storage manager exposes and what it hides (§50.1), and how it names a page across many files with a **page identifier** so the address space scales beyond one file (§50.2). And it handles the one case the fixed-size page of Chapter 47 cannot: a **value larger than a page**, stored out-of-line in a chain of overflow pages, with the named trap of storing it inline instead (§50.3). Then it composes the whole foundation into a single end-to-end trace of one record read (§50.4), and hands the finished interface up to the storage engines the next part builds on it (§50.5). This is the capstone of the storage part: after it, "fetch a page" and "read a record" are primitives the rest of the book may simply assume.

On the code in this chapter. The overflow-page counts of §50.3 are **exact arithmetic**, not measurements: a short program computes the ceiling-division page counts for values of various sizes given a 4 KiB page and a fixed per-page overhead, so the numbers are reproducible and hardware-independent (they depend only on the page size and overhead stated, not on this machine's disk). Where those page counts turn into a cost, the cost is the Chapter 48 buffer-pool fetch and the Chapter 46 device figures measured earlier in this part, which this chapter cites rather than re-measures. The arithmetic itself was run on this machine's Intel Core i7-12700H (glibc 2.35, gcc 11.4.0 available). The C in §50.1 is an *interface sketch* — function signatures illustrating the vocabulary, with types like `PageId` left abstract — not a compiled, timed program; no wall-clock claim is made about it, and a hardware-counter profiler (`perf`) is not installed here.

50.1 What the storage manager is: the interface between access methods and pages

The **storage manager** is the database module that owns physical storage and exposes it to everyone above as a narrow interface. Its whole reason to exist is *encapsulation*: the layers above — indexes, the table/heap layer, the executor — should reason about pages and records, never about how a page reaches memory or which frame it lands in. Concretely, the storage manager sits on top of the buffer pool (Chapter 48) and the on-disk files (Chapter 47) and offers roughly this vocabulary. To read, `fetchPage(pid)` returns a pinned pointer to the page's frame (a buffer-pool hit or a disk read, invisibly), and `unpinPage(pid, dirty)` releases it, flagging whether it was modified. To grow and shrink storage, `allocatePage()` extends a file and returns a new page identifier, and `freePage(pid)` returns a page to the free list. To find records, `readRecord(rid)` resolves a RID to bytes (the heap-file resolution of Chapter 49), and `insertRecord(bytes)` finds room (the free-space directory of Chapter 49), writes, and returns a new RID. In C the shape is small:

```
/* the storage-manager interface the layers above see -- pages and records,
   nothing about disks, frames, or eviction */
Page *fetchPage(PageId pid);              /* pin + return frame pointer (hit or disk read) */
void  unpinPage(PageId pid, bool dirty);  /* release; 'dirty' -> must be written back before
reuse */
PageId allocatePage(FileId f);            /* extend file f, return new page id */
void  freePage(PageId pid);               /* return page to the free list */
Record readRecord(Rid rid);              /* (page,slot) -> bytes: one fetchPage + slot index
*/
Rid    insertRecord(FileId f, Bytes rec); /* find room (free-space dir), write, return rid */
```

Everything the previous four chapters built is *behind* these six calls. `fetchPage` hides the buffer pool's page table, pin counts, dirty bits, and replacement policy, and hides the device beneath them — the caller sees only a pointer that is valid until it unpins. `insertRecord` hides the free-space directory and the slotted-page write. The layers above are written entirely in this vocabulary: a B-tree node *is* a page fetched with `fetchPage`; a table scan *is* a loop over page identifiers calling `fetchPage` and yielding records; an index leaf stores a *RID* and calls `readRecord` to reach the row. This is the layering that makes a database buildable at all: each layer depends only on the interface below it, so the B-tree does not break when the buffer pool changes its eviction policy, and the executor does not break when the heap file changes its free-space scheme. The storage manager is where "the database on disk" stops being about disks and starts being about pages and records — the exact abstraction the rest of the book stands on.

> **QUICK CHECK**
>
> Name two things the storage manager's `fetchPage` hides from the layers above it. Why does this encapsulation let a B-tree be written without any knowledge of disks or buffer frames? (Answer after the next section.)

50.2 The page identifier and the address space

Chapter 49's RID named a record by (page number, slot number), taking "page number" for granted. The storage manager must make that concrete, because a real database is not one file — it is many: one or more per table, more for indexes, still more for the log. So a page is named not by a bare number but by a **page identifier** (a **PageId**): typically a pair *(file identifier, page number within that file)*, sometimes packed into a single integer. The PageId is the address the whole storage manager turns on: `fetchPage(pid)` uses the file id to pick the file and the page number to compute the byte offset within it (page number × page size), then reads that page through the buffer pool. This is the storage layer's *address space* — a two-level address, file then offset, exactly analogous to the virtual-memory address translation of the operating-system part, and for the same reason: a flat global page number would tie every page to one file and one growth path, whereas (file, page) lets each table and index grow in its own file independently while still being named by a single, comparable identifier. The RID of Chapter 49 is thus properly a *(PageId, slot)*: file, page, slot — enough to locate any record in the database with no search, across any number of files. Two properties make the PageId the right currency for everything above. It is **small and comparable**: a few bytes, cheap to store by the million in index leaves and cheap to compare and sort, which is what lets an index hold RIDs densely. And it is **stable**: a page keeps its identifier for its lifetime, so a stored PageId remains valid — the property Chapter 49's forwarding pointers worked so hard to preserve for records is guaranteed for pages by construction. The address space is deliberately boring, and that is its virtue: a stable, small, comparable name for every page means every layer above can refer to storage by identifier and let the storage manager do the translation to bytes.

THE SAME IDEA THREE WAYS: A VALUE BIGGER THAN A PAGE MUST LIVE OUT-OF-LINE, IN A CHAIN OF PAGES

1. In words. A page is fixed at 4 KiB, so a value larger than a page cannot be a single slotted record. The storage manager stores it *out-of-line*: it writes the value across a chain of dedicated overflow pages and leaves, in the row itself, only a small pointer to the head of that chain. The row stays small; the big value lives elsewhere and is fetched only when actually needed.

2. As a picture. Figure 50.1: a heap page with a normal-sized row whose large column is not the bytes but a pointer, an arrow leading to overflow page O0, which chains O0→O1→O2→... each holding a slice of the value; the small columns stay inline, dense on the heap page.

3. As numbers. Computed here with a 4 KiB page and ~24 bytes of per-page overhead (usable ~4,072 B): a 8 KiB value needs **3** overflow pages; 64 KiB needs **17**; 1 MiB needs **258**; 10 MiB needs **2,576**. A linked chain also means random access to a byte deep in the value may walk up to the whole chain — which is why large values are read whole or sequentially, not seeked into.

*(Answer to the §50.1 quick check: `fetchPage` hides, among other things, the **buffer pool** — its page table, pin counts, dirty bits, and eviction policy — and the **device** beneath it: the caller cannot tell a hit from a disk read, and never issues a `pread` or picks a victim. This encapsulation lets a B-tree be written as "a tree of pages" using only `fetchPage`/`unpinPage`, so it neither knows nor cares how a page reaches memory — the storage manager owns that, and can change it without touching the B-tree.)*

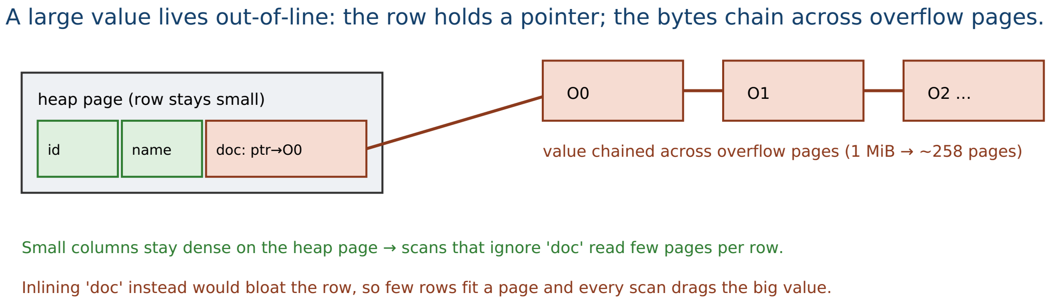


Figure 50.1: out-of-line storage for a large value. The row keeps small columns inline and stores only a pointer to a chain of overflow pages holding the big value, so scans that do not read the big column stay dense and cheap; inlining the value instead would bloat every row and slow every scan.

50.3 Values too big for a page: overflow and out-of-line storage

A page is fixed-size — 4 KiB here — but a column is not: a text document, a JSON blob, an image, a serialized vector can be kilobytes or megabytes, far larger than any single page. The storage manager cannot store such a value as one slotted record, so it stores it **out-of-line (overflow storage)**: the large value is written across a chain of dedicated **overflow pages**, and the row keeps only a small pointer to the head of the chain in place of the value. The arithmetic is a ceiling division — a value of S bytes, on pages of P bytes with per-page overhead h, needs $\lceil S / (P-h) \rceil$ overflow pages — and it grows exactly as you would expect:

```
out-of-line storage, 4-KiB pages, ~24 B/page overhead (usable ~4072 B):
  value    8 KiB  ->    3 overflow pages
  value   64 KiB  ->   17 overflow pages
  value    1 MiB  ->  258 overflow pages
  value   10 MiB  -> 2576 overflow pages
```

Out-of-line storage buys two things. It keeps the **row small**, so many rows still fit a heap page and scans stay dense (§49.4); and it makes the big value **pay-per-use**, fetched only when a query actually reads that column, not dragged through memory every time the row is touched for its small fields. The cost is that reading the value is a walk of the overflow chain — sequential and fine when you want the whole value, but poor for random access into the middle of it, since a linked chain must be walked to reach a deep offset (real systems that need random access into large values use a tree or a chunk index over the overflow pages rather than a bare chain). This is exactly the mechanism behind PostgreSQL's TOAST ("The Oversized-Attribute Storage Technique"), Oracle's and SQL Server's out-of-line LOB storage, and every system's BLOB handling: a large value is moved off the row and pointed at. The decision of *when* to push a value out-of-line is usually a threshold — a value beyond some fraction of the page (a common heuristic is roughly a quarter of the page, so several rows still fit) becomes a candidate for out-of-line storage — and getting it wrong is the named trap: storing a large value *inline* bloats every row so that only one or a few fit per page, which does not just waste space but destroys the tuple density that made the heap scan of §49.4 cheap. Every full scan then drags the big value through the buffer pool even for queries that never read it, turning a cheap scan of the small columns into an expensive scan of megabytes of document — a self-inflicted collapse of the storage engine's central advantage.

TRAP: STORING A LARGE VALUE INLINE AND COLLAPSING TUPLE DENSITY

A page is fixed-size, so the number of rows it holds is *(usable page bytes) / (row size)*. Put a large value inline — a multi-kilobyte JSON document, a serialized blob, a base64 image — and the row size balloons, so a page holds **one or a handful of rows instead of dozens**. The damage is not mainly wasted space; it is that **every full scan now reads that big value for every row, even queries that never touch that column**. A scan that should stream the small columns densely (Chapter 49) instead reads megabytes of document per row, so a report over `customer_id` and `total` is dragging the 1 MiB `notes` field of every row through the buffer pool — the tuple density that made the scan cheap is gone, and with it the storage engine's central advantage. The trap is seductive because inline "just works" on small data and only bites when values grow: a `text` column that held short strings in testing holds novels in production, and scans that were fast quietly become I/O-bound with no query change. The fix is to store large values out-of-line above a size threshold (roughly a quarter of a page is a common cutoff) so the row stays small and the big value is fetched only when read — exactly what TOAST/LOB storage does automatically, and what a hand-rolled engine must do deliberately. "It's just a column" and "scans are cheap" are not both true once that column holds a value larger than a fraction of the page.

QUICK CHECK

Why does storing a 1 MiB value inline in a row hurt a scan that never reads that value? Roughly how many 4 KiB pages does a 1 MiB value need out-of-line, and what does keeping only a pointer in the row preserve? (Answer below the communicate box.)

50.4 Composing the layers: one record read, end to end

The storage part is now complete enough to trace a single operation through every layer it built — the clearest way to see the interface doing its job. Suppose the executor holds a RID (from an index leaf or a scan) and wants the row. It calls the storage manager's `readRecord(rid)`, and here is the whole descent. The RID is *(PageId, slot)* = (file id, page number, slot) (§50.2). The storage manager calls `fetchPage(pid)` (§50.1); the buffer pool looks the PageId up in its page table (§48): on a **hit** the page is already in a frame — a memory access, about 36 ns measured earlier — and on a **miss** the pool evicts a victim (writing it back if dirty), computes the byte offset (page number × page size) in the file, and reads the 4 KiB page from the device (§46) — about 350 µs on this machine's virtual disk, four orders of magnitude slower. The page is pinned; the storage manager indexes its slot array at *slot* to get the record's in-page offset and length (§47, §49), and reads the record's bytes. If the record is a forwarding pointer (the row migrated, §49.3), it follows it with a second `fetchPage`. If a column is stored out-of-line and the query reads it, the pointer in the row is followed into the overflow chain (§50.3) with further fetches. Finally the storage manager `unpinPage`s and returns the bytes; the executor never saw a frame, a slot, or a byte offset — only “here is the record.” That single trace is the entire storage foundation in motion: the RID and page identifier that name the data (§49, §50.2), the heap file and slotted page that lay it out (§47, §49), the buffer pool that turns a page into memory and decides hit or miss (§48), the device that is the floor and the cliff (§46), and the storage manager that composes them into one call. Everything the next part builds — every B-tree node, every index lookup, every log write — is made of exactly this call.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: “Let's store each document's full JSON — often a megabyte — directly in a `data` column on the main table, so we don't need a second lookup. Simpler, and one row is one read.” The table is also scanned nightly for a report that reads only two small columns. Cover the paragraph and respond in four or five sentences, drawing on §50.3 and §49.4.

Model answer. “One row being one read is only true for the point look-up; for the nightly scan, inlining a megabyte per row is the expensive choice. A page is fixed at a few kilobytes, so if the row carries a 1 MiB JSON blob, a page holds essentially one row, and the scan that needs only two small columns now reads a megabyte per row through the buffer pool anyway — the tuple density that makes a scan cheap is gone. If we store the JSON out-of-line and keep a pointer in the row, the row stays small, dozens fit per page, the nightly scan stays dense and fast, and we fetch the megabyte only for the queries that actually read it — which is exactly what TOAST or LOB storage does for us automatically. So the blob should live out-of-line above a size threshold; the point look-up still gets its data in a follow-the-pointer fetch, and the scan keeps its density.” Naming tuple density, the scan-versus-point-lookup asymmetry, and out-of-line storage as the standard fix is the senior register.

*(Answer to the §50.3 quick check: storing a 1 MiB value inline balloons the row so a fixed-size page holds only one or a few rows, and a full scan then reads that megabyte for every row even when the query never touches that column — the tuple density that made the scan cheap collapses. A 1 MiB value needs about **258** 4 KiB overflow pages out-of-line; keeping only a pointer in the row preserves the row's small size, so scans of the other columns stay dense and the big value is fetched only when actually read.)*

50.5 What to carry forward

The **storage manager** is the module that owns physical storage and exposes it as a narrow interface — `fetchPage`/ `unpinPage`, `allocatePage`/`freePage`, `readRecord`/`insertRecord` — hiding the buffer pool, the slotted page, the free-space directory, and the device, so the layers above reason only about pages and records (§50.1). A page is named by a **page identifier (PageId)** — (file id, page number) — a small, comparable, stable address; the RID of Chapter 49 is properly (PageId, slot), enough to locate any record across any number of files with no search (§50.2). A value larger than a page is stored **out-of-line** across a chain of overflow pages with only a pointer left in the row — a 1 MiB value needing ~258 4 KiB pages — which keeps rows small and makes big values pay-per-use; storing them **inline** instead collapses tuple density and makes every scan drag the big value, the named trap (§50.3). One `readRecord` composes the whole foundation: RID → PageId → buffer-pool fetch (hit ~36 ns / miss ~350 µs) → slot index → bytes, following forwarding pointers and overflow chains as needed (§50.4).

With this, the storage part is finished, and finished as an *interface*, not just a pile of mechanisms. The device, the page, the buffer pool, the heap file, and the values too big to fit have all been folded behind a handful of calls that speak only of pages and records — and that interface is precisely the ground the rest of the volume stands on. The next part builds the **storage engines** — the access methods that give a table fast, selective access rather than only the sequential scan of the heap. The first, the **B-tree**, is nothing but a tree whose nodes are pages fetched through `fetchPage`, whose leaves store the RIDs this part defined, and whose shallow height turns a key look-up into a handful of page fetches; the second family, the **log-structured merge tree**, is runs of sorted pages written and merged through this same interface. Neither needs to know a thing about disks or frames, because the storage manager built here has already hidden them. The heap gave you a table; the storage manager gave you the interface; the access methods will give you speed. From here up, “fetch a page” is a primitive — and on that primitive, a database is built.

CAN YOU… WITHOUT LOOKING?

- State what the storage manager exposes and name three things it hides.
- Say what a PageId is made of and why (file, page) beats a flat global page number.
- Explain why the RID is properly (PageId, slot) and why that locates any record with no search.
- Explain out-of-line storage, why it keeps rows small, and roughly how many pages a 1 MiB value needs.
- Explain why storing a large value inline collapses tuple density and slows scans that never read it.
- Trace one `readRecord` from RID to bytes, naming the layer and cost at each step.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What interface does the storage manager expose, and what does it hide from the layers above? (§50.1)
2. **R2.** What is a PageId, and why is a two-level (file, page) address better than a flat global page number? (§50.2)
3. **R3.** How is a value larger than a page stored, and roughly how many 4 KiB pages does a 1 MiB value need? (§50.3)
4. **R4.** Why does storing a large value inline collapse tuple density and slow scans? (§50.3)
5. **R5.** Trace one `readRecord` from a RID to the record bytes, naming each layer and the hit/miss cost. (§50.4)

FURTHER READING

- **Joseph M. Hellerstein, Michael Stonebraker & James Hamilton, "Architecture of a Database System," *Foundations and Trends in Databases* 1(2), 2007, DOI 10.1561/1900000002.** Its storage-manager and access-method sections are the direct source for §50.1–§50.2: the interface the storage manager presents and the layering above it. First author Joseph M. Hellerstein; venue, year, and DOI verified via the ACM Digital Library.
- **Jim Gray & Andreas Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1992 (ISBN 978-1558601901).** Its storage-structures chapters cover the file/page abstraction, large-object handling, and how the storage layer presents pages and records to the transaction manager above — the framing behind §50.1 and §50.3. First author Jim Gray; publisher, year, and ISBN verified via the publisher and library listings.
- **Abraham Silberschatz, Henry F. Korth & S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019 (ISBN 978-0078022159).** Its storage-and-file-structure chapter covers large-object (LOB) storage and out-of-line representation of oversized values — the mechanism of §50.3. First author Abraham Silberschatz; edition, year, and ISBN verified via the publisher listing.
- **Raghu Ramakrishnan & Johannes Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill, 2003 (ISBN 978-0072465631), Chapter 9, "Storing Data: Disks and Files."** The page/record formats and file API that the storage manager wraps (§50.1–§50.2). First authors Raghu Ramakrishnan and Johannes Gehrke; edition, publisher, year, and ISBN verified via the publisher and retailer listings.

Exercises

50.1 WARM-UP In one sentence, what is the storage manager's job, and name two of the six interface calls it exposes.

50.2 WARM-UP What is a PageId made of, and why is the RID of Chapter 49 properly a (PageId, slot) rather than just (page number, slot)?

50.3 WARM-UP Roughly how many 4 KiB overflow pages does a 1 MiB value need out-of-line, and what does the row keep in place of the value?

50.4 CORE Encapsulation. (a) Name three things `fetchPage` hides from the layer that calls it. (b) Explain why this lets a B-tree be written without any knowledge of disks or buffer frames. (c) Give one concrete change to a lower layer (e.g. the buffer pool's eviction policy or the heap's free-space scheme) that the layers above do *not* have to know about, and why the interface protects them.

50.5 CORE Out-of-line storage. (a) Write the formula for the number of overflow pages a value of S bytes needs on pages of P bytes with per-page overhead h. (b) Evaluate it for S = 64 KiB and 1 MiB with P = 4096, h = 24. (c) Explain the two benefits of out-of-line storage (small rows, pay-per-use) and the one cost (walking the chain for deep random access).

50.6 CORE The inline trap. (a) If a page has U usable bytes and rows are R bytes, how many rows fit a page, and what happens to that count when a 1 MiB value is inlined into each row? (b) Explain why a scan that reads only two small columns is nonetheless slowed by an inlined large column. (c) State the threshold rule that avoids the trap and what it preserves.

50.7 COST A table has N rows; a query reads only two small columns via a full scan. (a) Give the number of pages the scan reads when the rows are small (density d rows/page) versus when a 1 MiB value is inlined (one row/page). (b) Using the Chapter 46/48 sequential-read cost per page qualitatively, express how many times more data the inlined scan moves. (c) Explain why out-of-line storage makes the scan cost depend on the columns read, not on the size of the unread large column.

50.8 FADED *Worked, with the last step for you.* An index look-up returns a RID for a row whose large `body` column is stored out-of-line; the query selects only `id` and `title`.
Step 1 (given): the RID is (PageId, slot); one `fetchPage` brings in the heap page (hit ~36 ns, miss ~350 µs).
Step 2 (given): the slot array gives the row's offset; `id` and `title` are inline, so they are read directly.
Step 3 (given): `body` is a pointer to an overflow chain; the query does not read it.
Step 4 (you): State how many page fetches this look-up performs and why the size of `body` does not affect it, then say what would change if the query *did* select `body`.

50.9 MIXED For each, name the storage-manager concept responsible — *fetchPage*, *PageId*, *out-of-line storage*, *RID*, or *the interface (encapsulation)* — and why: (a) a B-tree is written without a single `pread`; (b) a row keeps only a pointer to its 1 MiB document; (c) a page is named across many files by a small comparable value; (d) an index leaf stores a row's address; (e) the caller cannot tell a buffer-pool hit from a disk read.

50.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) “The storage manager exposes disk offsets and frame indices to the executor.” (b) “A PageId is stable for a page's lifetime.” (c) “A 10 MiB value fits in one page if the page is compressed.” (d) “Inlining a large value speeds up scans that never read it.” (e) “Out-of-line storage lets a big value be fetched only when the query reads that column.”

50.11 STRETCH Synthesize the storage foundation. (a) Trace one `readRecord(rid)` from RID to bytes, naming every layer (Chapters 46–50) and the cost at each. (b) Argue that the storage manager's interface is what makes the next part's B-tree buildable, using the fact that a B-tree node is a page and a leaf holds a RID. (c) Explain why the two-level address space (file, page) and out-of-line storage are the same design instinct — put a small stable name in front of variable, growing data — applied to pages and to values respectively.

50.12 LAB On your own machine (a C toolchain): (a) Sketch, in code or pseudocode, the six-call storage-manager interface over a buffer pool and a heap file, and implement `readRecord(rid)` as one `fetchPage` plus a slot index. (b) Compute (exact arithmetic) the overflow-page count for several value sizes given your page size and per-page overhead, and confirm it is a ceiling division. (c) Optionally measure the density effect: build one table with a large value inline and one with it out-of-line, scan both reading only the small columns, and report pages read; state your toolchain and CPU and which results are exact arithmetic versus timed measurements.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART IX

Storage Engines

B-trees and LSM-trees implemented and analyzed — the two families and when each wins.

Chapter 51 — The B-Tree: A Search Tree on Disk

> **Before you start:** Chapter 46 (a random page read is the disk cliff, ~350 µs here; the number of page fetches, not comparisons, is what a disk structure pays), Chapter 47 (the page — a B-tree node *is* a page), Chapter 48 (nodes are fetched through the buffer pool; a shallow tree keeps its upper levels resident), Chapter 49 (the RID a leaf stores to point at a heap row), Chapter 50 (the storage-manager interface a B-tree is built entirely on — `fetchPage` and nothing lower). · The storage part gave you a table you can store and scan. This chapter opens the storage-engines part with the structure that gives a table *selective* access — find a few rows out of billions in a handful of page fetches — and it does so by being shaped for the disk, not the CPU.

> **BEFORE YOU READ ON**
>
> From memory: (1) Chapter 46 — a random 4 KiB page read cost about how long on this machine, and how does that compare to an in-memory comparison of two keys (nanoseconds)? (2) Chapter 49 — the only way a heap file finds a specific row without an index is what, and what does it cost on a large table? (3) *Predict:* a balanced binary search tree over a billion keys is about 30 levels deep, and on disk each level is a separate page fetch. If instead you pack *hundreds* of keys into each node — one node per page — so the tree branches hundreds of ways at each level, roughly how many levels deep is the tree over a billion keys: still ~30, or a single-digit number? And what does that do to the number of disk reads for one look-up? Write your guesses; §51.3 answers them.

The heap file gives a table cheap storage and a cheap full scan, but only one way to find a specific row without reading everything: it has none — an unindexed look-up is a full scan (Chapter 49). What a database needs is **selective** access: find the handful of rows matching a key, or the range of rows between two keys, without touching the rest. That is the job of an **index**, and the dominant index structure in every relational database for fifty years is the **B-tree** — more precisely its variant the **B+-tree** — invented by Bayer and McCreight in 1972 and so pervasive that Comer's 1979 survey could already call it "ubiquitous." The reason a B-tree, and not the balanced binary search tree every algorithms course teaches, is the entire point of this chapter: a binary tree is optimal when a node access is a memory operation, but on disk a node access is a *page fetch*, and the structure that wins is the one that minimizes page fetches, not comparisons. The B-tree wins by being **wide and shallow** — each node is a full page holding hundreds of keys, so the tree branches hundreds of ways per level and reaches billions of keys in three or four levels, which means three or four page fetches per look-up instead of the thirty a binary tree would pay. This chapter builds it in five steps: why the disk rules out the binary tree and demands high fanout (§51.1); the B+-tree's shape — nodes are pages, internal nodes route, leaves hold the data and link together (§51.2); the fanout-and-height arithmetic that makes three or four levels index billions, reproduced here (§51.3); search, insert, and the node split that keeps the tree balanced, with its named trap of monotonically increasing keys (§51.4); and what the B-tree carries into the rest of the storage-engines part (§51.5).

On the code in this chapter. Two C programs, compiled with gcc 11.4.0 on this machine's Intel Core i7-12700H (glibc 2.35). The first computes **fanout and height** arithmetically — exact functions of the page size, key size, and pointer size stated — so its numbers are reproducible and hardware-independent (they describe the structure, not this machine). The second is a **working in-memory B-tree** that inserts keys in a fixed, seeded order and reports the resulting tree *height* and split count; the height is a structural result of the insertions, not a timing, and it is reported to confirm the arithmetic of §51.3, not to clock anything. Where a page fetch's cost appears, it is the Chapter 48 buffer-pool hit/miss and Chapter 46 device figures measured earlier in this volume, cited here rather than re-measured (a hardware-counter profiler, `perf`, is not installed on this machine).

51.1 Why not a binary tree: the disk changes the calculus

Every algorithms course teaches the balanced binary search tree as the way to search: *O(log N)* comparisons, halving the search space at each step. That analysis is correct and, on disk, nearly useless — because it counts *comparisons*, and on disk the cost that matters is *page fetches*. A binary tree over a billion keys is about **30 levels** deep ($\log_2$ of a billion), and if the tree does not fit in memory, each level you descend is a pointer to a different node on a different page — a separate page fetch, each one potentially the ~350 µs disk read of Chapter 46. Thirty page fetches to find one key is a catastrophe: it is the disk cliff paid thirty times over. The comparisons themselves — thirty of them, tens of nanoseconds each — are free by comparison; the entire cost is in the page fetches, and a binary tree maximizes them by putting only one key (one branch decision) in each node, so each node visited advances the search by a single comparison. The fix is to invert that ratio: put *as many keys as fit in a page* into each node, so that a single page fetch advances the search by a *whole page's worth* of branching. If a node holds 254 keys and so has 255 children, then one page fetch narrows the search 255-fold instead of 2-fold — and the number of fetches to reach any key falls from $\log_2(N)$ to $\log_{255}(N)$. This is the governing insight of on-disk data structures and the reason the B-tree exists: **match the node to the page, and make the fanout as high as the page allows**, because the unit of cost is the page fetch and a fat node amortizes each fetch over hundreds of branch decisions. The binary tree is not wrong; it is tuned for the wrong cost model — the RAM model of Chapter 1, where every access is unit cost — and the disk's cost model demands a wide, shallow tree instead.

> **QUICK CHECK**
>
> On disk, what is the unit of cost a search structure must minimize, and why does that make a binary search tree a poor choice despite its optimal comparison count? What single change turns it into a B-tree? (Answer after the next section.)

51.2 The B+-tree: nodes are pages, keys route, leaves hold the data

A **B+-tree** is a balanced search tree in which **every node is one page** (Chapter 47) and the tree is kept both sorted and balanced. It has two kinds of node. **Internal nodes** hold sorted keys and child pointers, and they exist only to *route*: a node with m sorted keys has m+1 child pointers, and to search for a key you find which of the m+1 gaps between the keys it falls into and follow that child. Internal nodes store no data, only keys-as-signposts and pointers to lower nodes. **Leaf nodes** sit at the bottom, hold the actual index entries — each a key together with a **RID** pointing at the heap row (Chapter 49), or the row itself in a clustered index — and, crucially, are **linked together** in key order: each leaf points to the next, so once a search reaches the correct leaf, a *range scan* can walk the linked leaves in sorted order without ever returning to the root. (This leaf-linking and the “all data in the leaves” rule are exactly what distinguish the B+-tree from the original B-tree, which stored data in internal nodes too; the B+-tree's design is what makes range scans efficient, which is why it, not the plain B-tree, is what real databases build — the two names are often used interchangeably, and this chapter means the B+-tree.) The whole structure obeys the **B-tree balance invariant**: every node except the root is kept at least half full, and *all leaves are at the same depth* — the tree is perfectly height-balanced at all times, so every look-up costs exactly the height in page fetches, no matter which key. That invariant is not free; it is maintained by the splitting of §51.4. But its payoff is total: a B+-tree gives you exact-match look-up (descend to a leaf), range scan (descend once, then walk linked leaves), and ordered iteration (walk the leaf chain from the start) — all three of the accesses a heap file could not give cheaply — on a structure whose height, the next section shows, is astonishingly small.

THE SAME IDEA THREE WAYS: HIGH FANOUT MAKES THE TREE SHALLOW, SO A LOOK-UP IS A HANDFUL OF PAGE FETCHES

1. In words. Each node is a page holding hundreds of keys, so the tree branches by its fanout at every level; the height is therefore the fanout-base logarithm of the key count, which for a fanout in the hundreds is a single-digit number even for billions of keys. Height equals page fetches per look-up, so a shallow tree is a fast one.

2. As a picture. Figure 51.1: a three-level B+-tree — one root, a middle level fanning out by ~254, a leaf level fanning out again — drawn beside a tall thin binary tree of the same key count, the B+-tree three boxes tall and the binary tree running off the page at ~30.

3. As numbers. Computed here for a 4 KiB page with 8-byte keys and 8-byte pointers: fanout **254**. Height to index a billion keys: **4 levels** (so 4 page fetches per look-up), against a binary tree's **~30** — the upper levels of the B-tree stay cached in the buffer pool (§48), so in practice only the leaf fetch usually touches the disk.

*(Answer to the §51.1 quick check: on disk the unit of cost is the **page fetch**, not the comparison. A binary search tree minimizes comparisons but puts one key per node, so it has ~$\log_2(N)$ levels and pays one page fetch per level — ~30 for a billion keys, i.e. the disk cliff thirty times. The single change is to make each node a full page holding*

hundreds of keys, so one fetch branches hundreds of ways and the height collapses to $\log_{255}(N)$ *— that is a B-tree.)*

A B+-tree: each node is a page of ~254 keys, so 3-4 levels reach billions. Internal nodes route; linked leaves hold

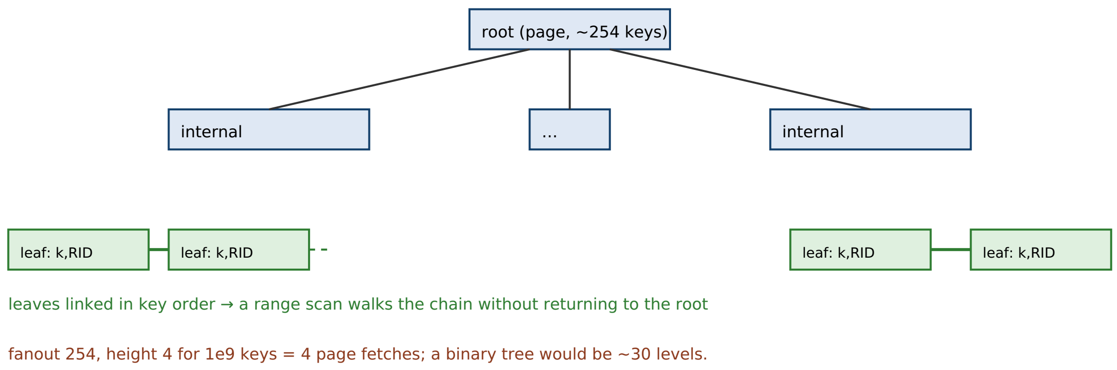


Figure 51.1: a B+-tree. Every node is a page; internal nodes hold sorted keys and child pointers and only route; leaves hold (key, RID) entries and are linked in key order so a range scan walks the chain. With fanout ~254, four levels index a billion keys — four page fetches per look-up against a binary tree's ~30.

51.3 Fanout and height: why three or four levels index billions

The B-tree's whole advantage is quantitative, so it is worth deriving. The **fanout** of an internal node is how many children it has, which is how many (key + child-pointer) pairs fit in a page: fanout ≈ (page size − header) / (key size + pointer size). For a 4 KiB page with a small header, 8-byte keys, and 8-byte child pointers, that is (4096 − 24) / 16 ≈ **254**. The **height** — the number of levels, which equals the page fetches per look-up — is then the fanout-base logarithm of the number of keys: a leaf holds some entries, and each level up multiplies the reach by the fanout, so height ≈ ceil(log-base-fanout of (N / leaf-entries)). Computed here across a few node sizes:

```
4-KiB page, exact fanout/height arithmetic:
  8B key + 8B ptr : fanout=254, leaf entries=254
     N = 1,000             -> height 2
     N = 1,000,000         -> height 3
     N = 1,000,000,000     -> height 4
     N = 1,000,000,000,000-> height 5
  16B key + 8B ptr: fanout=169 ->  N=1e9 -> height 5
  32B key + 8B ptr: fanout=101 ->  N=1e9 -> height 5

contrast: a balanced BINARY tree over 1e9 keys is ceil(log2) = 30 levels
```

This is the number that makes databases possible: with a fanout of 254, **a billion keys are indexed in four levels** — four page fetches to find any key, against a binary tree's thirty. And it is better than even four fetches suggests, because the tree is fetched through the buffer pool (Chapter 48): the root is one page and is essentially always resident; the second level is a few hundred pages and easily cached; so in steady state only the *leaf* fetch — the last level — actually reaches the disk, and a B-tree look-up on a billion-row table is, in practice, often a single disk read. The height grows with the logarithm, so it is extraordinarily flat: going from a billion keys to a trillion adds *one* level (height 5). This is why the B-tree scales — the cost of a look-up grows like the logarithm of the data with a base in the hundreds, which is to say it barely grows at all. A working B-tree confirms the arithmetic. An in-memory B-tree of order 8 (max 7 keys per node), inserting keys in a scrambled seeded order, produced these heights, each bracketed by the theoretical bounds ceil(log-base-*m* of N) ≤ height ≤ ceil(log-base-⌈*m*/2⌉ of N) that a B-tree of order *m* guarantees:

```
working B-tree, order m=8 (max 7 keys/node), seeded insert order:
  N =     1,000 : measured height 4   (bounds: log_8=4  .. log_4=5)
  N =   100,000 : measured height 7   (bounds: log_8=6  .. log_4=9)
  N = 10,000,000: measured height 10  (bounds: log_8=8  .. log_4=12)
```

Every measured height lands inside the bracket the invariant guarantees — the tree is provably balanced, its height logarithmic in the key count with the fanout as the base. (The measured heights sit above the full-node bound because randomly inserted nodes are on average only about 69% full, so the effective fanout is a little below the maximum — which is exactly why real systems care about node occupancy, the subject of the next section's trap.) Change *m* from 8 to 254 and the same logarithm gives the four-levels-for-a-billion result: the working tree and the arithmetic are the same claim at different fanouts.

QUICK CHECK

With a fanout of 254, why does a billion-key B-tree have height 4, and why in practice does a look-up often cost only one disk read rather than four? Roughly how many extra levels do you add going from a billion keys to a trillion? (Answer below the communicate box.)

51.4 Search, insert, and the split that keeps it balanced

Search is the easy operation and the one everything is arranged for: start at the root, in each internal node binary-search the sorted keys to find which child covers the target, follow it, and repeat until a leaf; then binary-search the leaf for the key and read its RID (or report absent). Exactly *height* page fetches, every time. **Insert** begins the same way — descend to the leaf where the key belongs — and then must add the entry while preserving the balance invariant, which is where the B-tree earns its keep. If the target leaf has room, the entry is inserted in sorted position and the job is done. If the leaf is **full**, it **splits**: the leaf's entries are divided into two half-full leaves, the new leaf is linked into the leaf chain, and the *separator key* (the first key of the new right leaf) is pushed **up** into the parent as a new routing key with a pointer to the new leaf. If that push fills the parent, the parent splits too, by the same rule, and the split can propagate up the tree. If it propagates all the way to the root and the root splits, a **new root** is created above the old one — and this is the only way a B-tree's height ever increases. That single detail is the source of the balance invariant: the tree grows *at the root*, upward, not at the leaves, so all leaves always stay at the same depth and the tree is never lopsided. Splitting is what pays for search's simplicity: because inserts do the work of keeping every path the same length, search never has to handle an unbalanced tree. Deletion is the mirror image — remove the entry, and if a node falls below half full, borrow from or merge with a sibling — though many real systems simplify deletion by tolerating under-full nodes and reclaiming them lazily. The costly case is the one where inserts arrive in a pattern that makes splitting pathological, and that is the named trap.

TRAP: MONOTONICALLY INCREASING KEYS CREATE A RIGHT-EDGE HOTSPOT AND HALF-EMPTY PAGES

Insert keys in **monotonically increasing order** — an auto-increment id, an appended timestamp, a sequence — and every insert lands in the *same* place: the rightmost leaf, because each new key is the largest so far. Two costs follow. First, a **write hotspot**: all inserts hit one leaf and its path to the root, so under concurrency every inserter contends on the same few pages and their latches, and the buffer pool churns that one hot path — the opposite of spreading load. Second, and subtler, **space waste from naive splitting**: when the rightmost leaf fills and splits 50/50, the left half is left half-full and, because all future keys are larger, it is *never inserted into again* — so a naive B-tree built by sequential insertion tends toward **~50% page utilization**, nearly doubling the tree's size and halving the effective fanout (which adds height, the very thing §51.3 worked to avoid). Real systems defend against both: they detect append patterns and do an *unbalanced right-edge split* (keep the full left page, start a fresh nearly-empty right page) so sequential loads reach ~100% fill, and they bulk-load sorted data bottom-up rather than by repeated insert. The counter-trap is over-correcting: switching a primary key to a *random* value (a UUID v4) to spread inserts destroys the locality that made range scans and cache residency cheap — now every insert lands on a random leaf, so the working set of dirty leaves is the whole tree instead of one hot page, and inserts miss the buffer pool constantly. “Just use an auto-increment key” and “the index packs densely and inserts spread out” are not both true under naive splitting; and “use a random key to spread load” trades a write hotspot for a cache-thrashing insert pattern. The honest answer is append-aware splitting (or sorted bulk-load), keeping both density and locality.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: “Our primary key is an auto-increment integer and the index is fine, but someone suggested switching to random UUIDs so inserts spread across the tree and we avoid a hotspot on the last page. Should we?” Cover the paragraph and respond in four or five sentences, drawing on §51.3 and §51.4.

Model answer. “Be careful — that trades one problem for a worse one. An auto-increment key does put all inserts on the rightmost leaf, which is a contention hotspot and, with naive 50/50 splits, leaves the left pages half-full; but the fix for that is append-aware splitting, which most engines already do, so sequential keys actually pack densely and stay cache-friendly. Switching to random UUIDs scatters every insert to a random leaf, so instead of one hot page you now dirty leaves all over the tree — the working set of pages you're writing becomes the whole index rather than one page, and inserts start missing the buffer pool constantly, which is often slower overall. Random keys also throw away range-scan locality: rows inserted together no longer sit together. So keep the sequential key and rely on append-aware splitting (or bulk-load sorted), or if you genuinely need to spread writes, use a scheme that keeps locality — not a fully random key.” Naming the right-edge hotspot, the density cost of naive splitting, and the cache-thrashing counter-trap of random keys is the senior register.

(Answer to the §51.3 quick check: with fanout 254 the tree's height is $\lceil \log_{254}(1e9) \rceil = 4$, so four page fetches reach any key. In practice a look-up often costs one disk read because the root and the second level are few enough pages to stay resident in the buffer pool, so

only the leaf fetch actually reaches the disk. Going from a billion to a trillion keys adds just one level — height 5 — because the height grows as the logarithm base 254.)

51.5 What to carry forward

On disk the unit of cost is the **page fetch**, not the comparison, so the balanced binary tree — optimal in comparisons but ~30 levels and thus ~30 fetches over a billion keys — is the wrong structure; the **B-tree** makes each node a full page of hundreds of keys so one fetch branches hundreds of ways (§51.1). A **B+-tree** is a balanced tree of pages: internal nodes hold sorted keys and child pointers and only route, leaves hold (key, RID) entries and are linked in key order for range scans, and the balance invariant keeps every node at least half full with all leaves at the same depth (§51.2). **Fanout** ≈ page/(key+pointer) ≈ 254 for an 8-byte key on a 4 KiB page, giving **height 4 for a billion keys** — four page fetches, often just one disk read after caching, and only one extra level per thousandfold more data; a working B-tree's measured heights sit inside the guaranteed logarithmic bracket (§51.3). **Search** is *height* fetches; **insert** descends and, if the leaf is full, **splits**, pushing a separator key up and growing the tree only at the root — which is what keeps it balanced — while **monotonic keys** create a right-edge hotspot and, under naive splitting, ~50% fill, defended by append-aware splitting rather than random keys (§51.4).

The B-tree is the first storage engine, and it is the workhorse: the default index in PostgreSQL, MySQL/InnoDB (whose tables *are* a B+-tree on the primary key), Oracle, SQL Server, and SQLite, and the structure most look-ups in most databases go through. It is built entirely on the storage manager of the previous part — a node is a page from `fetchPage`, a leaf stores a RID from the heap file — which is exactly why that interface was worth building. Its one structural bias is worth carrying forward as the setup for the next engine: the B-tree updates *in place*, splitting and rewriting pages where the keys belong, which makes look-ups and range scans excellent but makes a write a random page update — and a workload of many small random writes pays the random-write cost of Chapter 46 on every one. That bias is precisely the opening for the other great storage-engine family, the **log-structured merge tree**, which inverts the trade: it turns random writes into sequential ones by buffering in memory and flushing sorted runs, paying instead at read time to merge them. The B-tree optimizes reads and range scans; the LSM-tree optimizes writes; and the choice between them is the central storage-engine decision the rest of this part unfolds. The heap gave you a table; the storage manager gave you the interface; the B-tree gives you the first fast, selective, ordered access to a table — a handful of page fetches to find any row among billions.

CAN YOU… WITHOUT LOOKING?

- Explain why a binary search tree is the wrong structure on disk despite its optimal comparison count.
- Describe a B+-tree's two node types, what each holds, and why leaves are linked.
- State the fanout formula and compute it for an 8-byte key on a 4 KiB page.
- Explain why a billion-key B-tree has height 4 and often costs one disk read per look-up.
- Describe a node split, why it pushes a key up, and why the tree grows only at the root.
- Explain the monotonic-key trap — hotspot and ~50% fill — and why random keys are the wrong fix.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** On disk, what cost must a search structure minimize, and why does that rule out the binary search tree? (§51.1)
2. **R2.** What do a B+-tree's internal nodes and leaves each hold, and why are the leaves linked? (§51.2)
3. **R3.** Write the fanout formula and explain why height 4 indexes a billion keys with fanout ~254. (§51.3)
4. **R4.** Walk through a leaf split: what is divided, what is pushed up, and how does the tree's height ever grow? (§51.4)
5. **R5.** Why do monotonically increasing keys cause a hotspot and ~50% fill, and why is a random key the wrong fix? (§51.4)

FURTHER READING

- **Rudolf Bayer & Edward M. McCreight, "Organization and Maintenance of Large Ordered Indexes,"** ***Acta Informatica*** **1(3), 1972, pp. 173–189, DOI 10.1007/BF00288683.** The original paper introducing the B-tree and the split-and-grow-at-the-root maintenance of §51.4. First author Rudolf Bayer; venue, volume, pages, year, and DOI verified via SpringerLink.
- **Douglas Comer, "The Ubiquitous B-Tree,"** ***ACM Computing Surveys*** **11(2), 1979, pp. 121–137, DOI 10.1145/356770.356776.** The classic survey behind §51.1–§51.2: the B-tree versus B+-tree distinction, the leaf-linking that makes range scans efficient, and why the structure is everywhere. Sole author Douglas Comer; venue, volume, pages, year, and DOI verified via the ACM Digital Library.
- **Goetz Graefe, "Modern B-Tree Techniques,"** ***Foundations and Trends in Databases*** **3(4), 2011, pp. 203–402, DOI 10.1561/1900000028.** The comprehensive modern survey: fanout and occupancy, splitting and bulk-loading (the §51.4 trap and its fixes), and the transactional and concurrency techniques real B-trees use. Sole author Goetz Graefe; venue, volume, pages, year, and DOI verified via the ACM Digital Library and publisher.
- **Raghu Ramakrishnan & Johannes Gehrke,** ***Database Management Systems*****, 3rd ed., McGraw-Hill, 2003 (ISBN 978-0072465631), Chapter 10, "Tree-Structured Indexing."** The textbook development of the B+-tree, its search/insert/split, and the fanout/height arithmetic of §51.3. First authors Raghu Ramakrishnan and Johannes Gehrke; edition, publisher, year, and ISBN verified via the publisher and retailer listings.

Exercises

51.1 WARM-UP In one sentence, what is the unit of cost a disk search structure minimizes, and why does that favor a wide, shallow tree over a binary one?

51.2 WARM-UP In a B+-tree, what do internal nodes hold and what do leaves hold, and why are the leaves linked together in key order?

51.3 WARM-UP For a 4 KiB page with 8-byte keys and 8-byte pointers, roughly what is the fanout, and how many levels index a billion keys? How many page fetches is a look-up?

51.4 CORE Fanout and height. (a) Write the fanout formula in terms of page size, header, key size, and pointer size. (b) Compute the fanout for 16-byte keys and 8-byte pointers on a 4 KiB page. (c) Using that fanout, estimate the height needed to index a billion keys, and explain why doubling the key size does not double the height.

51.5 CORE The split. (a) Describe what happens when an insert lands in a full leaf: what is divided, what is created, and what is pushed up. (b) Explain why a split can propagate up the tree, and what happens when it reaches the root. (c) Explain how "grow only at the root" keeps all leaves at the same depth.

51.6 CORE Binary tree versus B-tree on disk. (a) How many levels does a balanced binary tree over a billion keys have, and how many page fetches would a look-up cost if the tree does not fit in memory? (b) How many does a fanout-254 B-tree cost? (c) Explain why the comparison count (both ~30 for the binary tree's descent) is irrelevant to the on-disk cost.

51.7 COST Caching the upper levels. (a) A fanout-254 B-tree over a billion keys has height 4; estimate how many pages the root and second level together occupy, and argue they stay resident in a reasonable buffer pool. (b) Given that, how many of the 4 page fetches typically reach the disk? (c) Using the Chapter 48 hit (~36 ns) and miss (~350 µs) costs, estimate the effective cost of a look-up when only the leaf misses.

51.8 FADED *Worked, with the last step for you.* A team loads a billion rows into a table whose primary-key index is a B+-tree, inserting rows in increasing key order, and finds the index is nearly twice the size they expected.
Step 1 (given): increasing keys always land in the rightmost leaf, which fills and splits.
Step 2 (given): a naive 50/50 split leaves the left half half-full, and no future (larger) key is ever inserted into it.
Step 3 (given): so pages settle near 50% utilization, roughly doubling the page count.
Step 4 (you): Explain why the index is about twice the expected size and what effect the lower fill has on height and look-up cost, then name the fix that keeps sequential loads dense.

51.9 MIXED For each, name the B+-tree concept responsible — *fanout*, *leaf linking*, *node split*, *the balance invariant*, or *the RID in the leaf* — and why: (a) a range scan walks in sorted order without returning to the root; (b) every look-up costs the same number of fetches regardless of key; (c) hundreds of keys fit in one node; (d) the tree stays balanced as it grows; (e) a leaf entry points at the actual heap row.

51.10 MIXED Classify each as *true* or *false* and fix the false ones: (a) "A B+-tree stores data in its internal nodes." (b) "A B-tree's height grows when a leaf splits." (c) "Doubling the number of keys roughly doubles a B-tree's height." (d) "Sequential key insertion, split naively, tends toward ~50% page fill." (e) "On disk, the binary search tree is preferred because it makes the fewest comparisons."

51.11 STRETCH Synthesize the access path. (a) Trace a point look-up by key end to end: descend the B-tree (§51.4) fetching each node via the storage manager (§50) through the buffer pool (§48), reach the leaf, read the RID, and fetch the heap row (§49) — count the page fetches and say which usually hit versus miss. (b) Explain why the B-tree is built entirely on the previous part's `fetchPage` interface and what that bought. (c) Argue that the B-tree's in-place update bias is exactly what motivates the log-structured alternative, using the Chapter 46 random-write cost.

51.12 LAB On your own machine (a C toolchain): (a) Write a program that computes fanout and B-tree height for a page size, key size, and pointer size, and tabulate the height for N from a thousand to a trillion. (b) Implement a working in-memory B-tree (choose an order *m*), insert keys in a seeded order, and report the measured height and split count, checking it against the bracket ceil(log-base-*m* of N) ≤ height ≤ ceil(log-base-⌈*m*/2⌉ of N). (c) State your toolchain and CPU, and explain which results are exact arithmetic and which are structural outcomes of a deterministic insert order — and why neither is a wall-clock timing.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART X

Durability & Recovery

Write-ahead logging, ARIES, checkpoints, and crash recovery — how a database keeps its promises across failures.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XI

Transactions & Concurrency Control

ACID, isolation levels and their anomalies, 2PL, MVCC, and OCC — correctness under concurrency, implemented.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XII

Query Processing & Execution

Parser, planner, physical operators, and execution models (Volcano/vectorized/codegen) — turning SQL into work.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XIII

Query Optimization

Statistics, cardinality estimation, cost models, and join ordering — the optimizer, honestly.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XIV

Indexing & Access Methods

Secondary/clustered/covering indexes, and specialized structures — the access-method layer.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XV

[BUILD] A Working Database

Assemble the pieces into a small but real relational engine — storage → txn → query → execute — with tests.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XVI

Toward Distributed Databases

Sharding, replication, and the distributed-transaction problem — the bridge to Volume 3.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XVII

Models & Foundations

System and failure models, the FLP impossibility, time and clocks, causality and logical time — the formal ground.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XVIII

Replication & Consistency

Consistency models, linearizability, CAP/PACELC, and quorums — what 'consistent' actually means.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XIX

Consensus

Paxos, Raft, and view-stamped replication (with a note on Byzantine fault tolerance) — agreement under failure.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XX

Coordination & Transactions

Leader election, distributed transactions, 2PC/3PC, and sagas — coordinating across nodes.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXI

Replicated Data & Convergence

Eventual consistency, anti-entropy, gossip, and CRDTs — converging without coordination.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXII

Partitioning & Scale

Consistent hashing, rebalancing, and hot-spot handling — spreading state and load.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXIII

Distributed Algorithms & Patterns

Broadcast/ordering, failure detectors, distributed snapshots, and the patterns real systems use.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXIV

Formal Methods

Specifying and model-checking a protocol with TLA+ — finding the bug before production does.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXV

Building & Operating Distributed Systems

From spec to a real, observable, testable distributed system — and operating it.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXVI

Security Foundations & Threat Modeling

The security mindset, threat modeling, trust boundaries, and reasoning about adversaries.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXVII

Applied Cryptography

Symmetric/asymmetric crypto, hashing, MACs, signatures, TLS, and key management — used correctly, not invented.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXVIII

Authentication & Authorization

Identity, sessions, OAuth2/OIDC, access-control models, and zero-trust — who can do what.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXIX

Software & Systems Security

Memory-safety bugs, injection, the supply chain, and secure-by-design coding — the common failure modes and their defenses.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXX

Data Security

Encryption at rest and in transit, secrets management, tokenization, and secure key handling for data platforms.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXXI

Privacy Engineering

Differential privacy, anonymization vs pseudonymization, GDPR/CCPA, and data governance — privacy as an engineering discipline.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXXII

Secure Data Pipelines & Platforms

Applying the above to real data systems: lineage, access, multi-tenancy, and compliance-by-design.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXXIII

Detection, Response & Operations

Logging, monitoring, detection, and incident response — assuming breach and limiting blast radius.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXIV

The Staff+ Role & Scope

Archetypes (tech lead, architect, solver, right-hand), what 'impact' means above senior, and the shift from output to leverage.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXV

Technical Leadership & Strategy

Technical vision, strategy, and roadmaps — setting direction others can follow.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXVI

Architecture & Design at Scale

Design review, ADRs, trade-off analysis, and evolving systems without a rewrite — the architect's judgment, applied.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXVII

Influence Without Authority

Building alignment and buy-in, driving decisions across teams, and disagree-and-commit.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXVIII

Communication & Writing

The written word as a staff engineer's main tool: design docs, RFCs, and making complex ideas land.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXIX

Mentoring & Growing Engineers

Mentorship, sponsorship, code/design review as teaching, and multiplying the people around you.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXX

Execution & Judgment

Leading projects and incidents, making decisions under uncertainty, and knowing what NOT to do.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXXI

Career & the Organization

Navigating the org, the promotion case, sustainability, and a durable definition of technical leadership.

BACK MATTER

Solutions to Exercises

Worked solutions to the end-of-chapter exercises. Try each before reading its solution.

Chapter 1 — Solutions

1.1 **Uniform cost:** every basic operation — arithmetic, comparison, and any memory read or write — costs the same fixed unit of time, independent of which address is accessed. The three false assertions about real hardware: (1) all memory accesses cost the same (they vary by orders of magnitude with which tier of the hierarchy holds the data); (2) one instruction takes one unit of time (real CPUs overlap, reorder, and speculate, retiring several per cycle or stalling for hundreds — Chapter 2); (3) operations are independent and equally priced (some ops cost more than others, and a dependent instruction must wait for its input). *Method note:* the trap is listing symptoms ("caches exist") without naming the underlying assumption they violate — always tie the fact back to *uniform cost*.

1.2 Fastest to slowest: **register** (<1 ns) < **L1** (~1 ns) < **L2** (a few ns) < **L3** (~10 ns) < **DRAM** (~100 ns) < **NVMe SSD** (tens of µs) < **same-datacenter network round trip** (hundreds of µs to ~1 ms). All figures are orders of magnitude, machine- dependent (§1.3).

1.3 The memory wall is the large and historically widening gap between how fast a CPU can execute instructions and how long DRAM takes to return a requested byte, because processor throughput improved much faster than DRAM latency for decades (Wulf & McKee, 1995). It makes uniform cost worse over time because the ratio between "one arithmetic op" and "one DRAM access" grew, so pricing a DRAM access at the same "one unit" as an add gets more wrong every hardware generation, not less.

1.4 (a) A few billion ops/second is $\sim 3\times10^9$/s, so in 100 ns = 10^{-7} s the core could do on the order of **a few hundred operations** ($3\times10^9 \times 10^{-7} = 300$). (b) A program dominated by random pointer-following spends most of its time *waiting* for DRAM, leaving the arithmetic units idle for hundreds of potential operations per miss — so it can run at a small fraction of the core's peak throughput even though its big-O looks fine. *Order-of-magnitude flags:* both the 100 ns and the few-billion-ops/s are approximate and machine-dependent; the point is the ratio ($\sim 10^2$), not the exact count. **Feed-forward:** if this surprised you, revisit §1.2.

1.5 Two explanations: (1) **Constant factors dominate at this size.** Big-O ignores the constant, and the O(n) bucketing pass may do far more *slow* memory traffic per element (e.g. scattering writes across buckets spread through DRAM) than the O(n log n) sort, which may stream memory in a cache-friendly order (§1.4). (2) **Wrong regime.** The asymptotic advantage only shows up once n is large enough; on this dataset n may be small enough that the log n factor is tiny and the bucketing's larger constant wins for the wrong reasons (§1.4). The single settling action: **measure both on the real data** — big-O cannot answer a "how fast" question. *Tempting wrong answer:* "the O(n) version must be faster, the benchmark is flawed" — this is trusting the RAM model for a speed question, exactly the §1.4 trap. **Feed-forward:** §1.4 and the warning box.

1.6 **Step 4:** The binary search over a small contiguous sorted array is likely to make *fewer trips to slow memory* per lookup — its handful of accesses are clustered and cache-friendly and the small array may live largely in cache, whereas the hash map's single lookup typically lands on a random heap address and pays a full cache/DRAM miss. So the asymptotically worse structure can win on real hardware because *the metric that matters here is costly-memory-accesses per lookup, not operation count.* The one thing to do before trusting the argument: **benchmark both on the real data and access pattern.** *Why this final step and not "pick the lower big-O":* big-O measures scaling and is silent about the constant factor — where the data lives — which is the entire basis of the argument; only measurement can confirm the memory-access reasoning holds for your actual sizes and hardware. **Feed-forward:** §1.4; the array-vs-pointer effect is measured in Chapter 3.

1.7 (a) **RAM/big-O** — the tables grow without bound, so the deciding factor is scaling (quadratic vs linear), which is exactly what big-O captures. (b) **Real cost model** — same size, same O(n); the 8× gap can only come from constant factors, i.e. memory-access patterns. (c) **RAM/big-O** — "1000× bigger" is a scaling question. (d) **Real cost model** — identical length and O(n); the difference is contiguous streaming vs random pointer-chasing through the hierarchy. *Method-selection cue:* if the question mentions "grows / bigger / scales," reach for big-O; if it fixes the size and asks "why is one faster," reach for the real cost model.

1.8 The claims are **not in contradiction**. Claim (i) is about the RAM/big-O model (optimal *scaling*); claim (ii) is about the real cost model (*speed* at a given size, set by constant factors the memory hierarchy dominates). Both can be true simultaneously: two O(n) algorithms are equally optimal asymptotically yet differ 10× in wall-clock time. To choose for production you still need: the actual input sizes you will run at, and a **measurement** of both on representative data and hardware. *Tempting wrong answer:* "'optimal' means fastest, so (i) and (ii) conflict" — conflating asymptotic optimality with speed, the core confusion of §1.4.

1.9 **Mechanical sympathy** is writing code that works *with* the grain of the hardware — its memory hierarchy, pipeline, and I/O costs — rather than against it, by understanding enough of how the machine works to get the most from it. The ordering rule: choose a good *algorithm* first (a bad big-O cannot be rescued by micro-optimization — no cache trick fixes an accidental quadratic), *then* apply mechanical sympathy to win the constant factor. Failure modes: knowing only big-O → asymptotically optimal code that needlessly runs 10× slow; knowing only micro-optimization → a lovingly hand-tuned O(n^2) that dies at scale.

1.10 Three reasons the absolute numbers can't be trusted: (1) they are **microarchitecture-specific** — cache sizes, latencies, and cycle counts differ across CPU models and generations; (2) they **drift with hardware** over time, so any printed figure is dated; (3) measured latency depends on **context** — contention, frequency scaling, what else is resident in cache, and access pattern — so there is no single "the" latency. The ratios are more stable because they reflect the *structural* reason each tier exists (each is a deliberate size/speed tradeoff a step down from the last), and that layering persists even as every absolute number moves. To get trustworthy numbers: **measure on the actual target machine** under representative load (a microbenchmark or a profiler), which is the discipline the performance-engineering chapters build. **Feed-forward:** §1.3; measurement technique in Chapter 3.

1.11 Answers are machine-specific by design; a correct write-up (i) reports real cache-line size (commonly 64 bytes on x86-64) and your actual L1/L2/L3 sizes from `getconf`/`lscpu`, (ii) reports two measured per-element times — the in-L1 loop should be markedly faster per element than the exceeds-L3 loop, which is bottlenecked on DRAM bandwidth — and (iii) states the ratio you observed. The learning goal is the habit: on a "how fast" question you *generated your own numbers* rather than quoting the book's orders of magnitude. Exact values depend entirely on your hardware; the *shape* (in-cache faster than out-of-cache) is what must appear. **Feed-forward:** the mechanism behind the ratio is §1.2–1.3 and all of Chapter 3.

Chapter 2 — Solutions

2.1 In order: (1) **pipelining** — overlap the stages so different instructions occupy different units in the same cycle; (2) **superscalar** — duplicate execution units and issue several instructions per cycle; (3) **out-of-order** — reorder execution to run later independent instructions while an earlier one is stalled; (4) **speculation / branch prediction** — guess a branch's direction and execute down the predicted path so the pipeline need not wait for the branch to resolve.

2.2 **Latency** is the number of cycles for one instruction to pass through all stages; **throughput** is how often a new instruction *completes*. Because the stages overlap — while one instruction is in "execute," the next is in "decode," and so on — a new instruction can finish every cycle even though each individually spends several cycles traversing the pipeline. The two numbers measure different things: time-through-one vs. completions-per-time.

2.3 A **data hazard**: an instruction needs a result that an earlier, not-yet-finished instruction will produce, so it cannot proceed until that value is ready. A **control hazard**: the CPU does not yet know which instruction comes next because an unresolved branch determines it. **Branch prediction addresses the control hazard** — it guesses the branch direction so fetching can continue instead of stalling.

2.4 In loop A each add depends on the previous add's result (they all target one accumulator), forming a single dependency chain: the out-of-order engine can never have more than one add in flight, so the superscalar units sit mostly idle and the loop runs at roughly one add per dependency latency. In loop B the four partial sums are *independent*, so up to four adds can be in flight at once, keeping multiple execution units busy and finishing in a fraction of the time — same number of additions, more ILP. The enabling condition: floating-point addition is not associative, so B only computes the "same" result if you accept that the grouping (and thus rounding) differs, or the compiler is allowed to reassociate — B's parallelism is possible precisely because its adds have no data dependency between them. *Method-selection note:* the tell that ILP is the lever is "same operation count, one version serializes them." **Feed-forward:** §2.3 and the threeways box.

2.5 On P the data-dependent branch resolves in long predictable runs, so the branch predictor is almost always right, speculation is nearly free, and the pipeline stays full. On Q the branch is a patternless coin flip: the predictor is wrong about half the time, and each misprediction flushes the speculatively executed work and refills the pipeline — a penalty on the order of tens of cycles (machine-specific) paid on ~50% of iterations. The extra time on Q is spent flushing and refilling the pipeline, not doing arithmetic. *Tempting wrong answer:* "Q must be doing more comparisons" — no, the arithmetic is identical; the cost is misprediction, not extra work. **Feed-forward:** §2.4.

2.6 (a) Average cost per iteration ≈ $b + p \cdot k$ cycles (caveat: k is an order-of-magnitude, microarchitecture-specific figure). (b) The misprediction term dominates when $p \cdot k > b$, i.e. roughly $p > b/k$; for a cheap body and a ~tens-of-cycles penalty, even a modest p (a branch that misses a nontrivial fraction of the time) makes misprediction the largest term. (c) Making the branch predictable lowers p (the predictor is right more often); a branchless rewrite removes the branch so there is no k to pay at all. You measure first because on an already-predictable branch p is already tiny, so a branchless rewrite adds complexity for no gain — and may even be slower if it does more arithmetic unconditionally. **Feed-forward:** §2.4 warning box.

2.7 **Step 4:** The miss stalls the CPU *only if there is no independent work to do while it is serviced.* The deciding property is the amount of **instruction-level parallelism in the surrounding code** — whether there are nearby instructions whose inputs are ready and do not depend on the missing load. If there is enough such work, the out-of-order engine hides the miss latency behind it; if the code is a tight dependency chain that all waits on the loaded value, the machine has nothing else to do and stalls for the full miss. It is that property, not the miss latency alone, that decides the outcome, because the latency is a fixed cost of reaching DRAM — what varies, and what you control by how you write the code, is whether the CPU can *overlap* useful work with it. **Feed-forward:** §2.3 (out-of-order) and Chapter 1 §1.2 (miss latency).

2.8 (a) **Branch prediction** — sorting changed only predictability, not arithmetic. (b) **Dependency chain / ILP** — splitting an accumulator exposes independent work. (c) **Memory hierarchy** — a linked list scatters accesses to random addresses (cache misses, Chapter 1) where an array streams contiguously. (d) None of the three is a bottleneck — no branches, no memory pressure, presumably enough ILP — so the loop is already near peak; the lesson is that when none of the mechanisms is being violated, there is little constant-factor slack left to recover. *Method-selection cue:* match the symptom — "sorting helped" → branches; "extra accumulators helped" → ILP; "layout/pointers hurt" → memory.

2.9 The **real cost model** (Chapter 1 §1.4) resolves it: equal big-O guarantees equal *scaling*, never equal *speed*; a 5× gap lives entirely in the constant factor big-O discards. Three independent mechanisms that could each cause it: (1) **memory access pattern** — one version streams cache-friendly memory, the other chases pointers into DRAM (Chapter 1, Chapter 3); (2) **branch misprediction** — one has an unpredictable data-dependent branch flushing the pipeline (§2.4); (3) **dependency chains / low ILP** — one serializes its work and starves the superscalar units (§2.3). The single identifying action: **profile / measure** (e.g. with a profiler reporting cache misses and branch mispredictions) — the mechanism is an empirical question, not one big-O can answer. *Tempting wrong answer:* "it's impossible / the benchmark is wrong," which is the Chapter 1 trap of trusting big-O for a speed question. **Feed-forward:** §2.5 and Chapter 1 §1.4.

2.10 (a) On typical code branches are highly predictable, so speculation is almost always correct and the CPU keeps working instead of stalling several cycles at every branch — the occasional wasted work is far cheaper than stalling on every branch. (b) On an unpredictable branch the predictor is wrong a large fraction of the time, so much speculative work is discarded and the misprediction penalty dominates — a net loss versus not having to speculate at all. (c) At a high level: because speculatively executed instructions can affect microarchitectural state (such as what ends up in the cache) even when their results are later discarded, that residual state can, in principle, be observed and used to infer data that should not have been accessible — this is the basis of the real speculative-execution vulnerability class. The precise mechanisms and mitigations are a genuine, separate topic (touched on in Volume 4); describing them accurately requires primary sources, and this answer deliberately does not invent specifics. *No-fabrication note:* full credit requires the high-level reasoning *and* the acknowledgement that the details need real sources, not invented ones.

2.11 Machine- and compiler-specific by design. A correct write-up reports two measured times (single vs. four accumulators) and their ratio, with the four-accumulator version typically faster when the compiler has not already reassociated or vectorized the single-accumulator loop. It must state which numbers are the reader's own and note the ratio depends on CPU and compiler. Crucially, if optimization collapses the difference (the compiler reassociates or auto-vectorizes both), the correct response is to *report that* and describe what was observed — never to quote a ratio that was not actually measured. *Method note:* the exercise is as much about honest measurement discipline (no fabricated numbers) as about ILP. **Feed-forward:** §2.3 threeways box.

Chapter 3 — Solutions

3.1 64 / 8 = **8 doubles** per line. Reading one from a cold line brings in the whole line, so the **7 neighbors** become cheap hits. This rewards **spatial locality** — you get the nearby elements for free because memory arrives a full line at a time.

3.2 **Temporal:** a loop accumulator `sum += x[i]` — `sum` is reused every iteration, so after its first access it stays resident (retention of recently used lines). **Spatial:** walking `x[0], x[1], x[2], …` — consecutive elements share lines (line fill) and the sequential pattern lets the *prefetcher* fetch ahead. Line fill and prefetching exploit spatial locality; keeping recent lines exploits temporal.

3.3 (a) **Compulsory (cold)** — first-ever reference to those lines. (b) **Capacity** — the 4 GB working set vastly exceeds the 8 MB cache, so lines are evicted before reuse. (c) **Conflict** — a specific power-of-two row length aligns many accesses onto the same cache sets, so they evict each other despite free space elsewhere.

3.4 (a) **Inner loop over columns (row-major order) is faster**: consecutive accesses are adjacent in memory (stride 1), so each 64-byte line is fully used and the prefetcher runs ahead. Inner loop over rows strides by N elements, touching a new line almost every access. (b) Same big-O and same additions govern *scaling*, not *speed*; the constant factor — memory moved and misses incurred — differs by roughly an order of magnitude because only one version has spatial locality (this is the Chapter 1 §1.4 point, now concrete). (c) The slow version suffers mostly **capacity misses** (the whole matrix far exceeds cache, and its poor locality means it re-fetches lines it effectively wasted) — with a real **conflict** component too, since the power-of-two dimension (N = 8192) gives a power-of-two stride that aliases into a few cache sets (the very symptom §3.4 described); its lack of spatial reuse is what turns each fetch into wasted bandwidth. *Tempting wrong answer:* "they're both O(N^2) so the difference must be a measurement error" — the §3.6 trap. **Feed-forward:** §3.6.

3.5 Replace the array-of-pointers with a contiguous **array of the objects themselves** (array of structs, or a struct-of-arrays if you only touch some fields), laid out in traversal order. Why it reduces misses: the objects now share cache lines, so walking them in order has spatial locality — each fetched line yields several objects instead of one random-heap object per pointer (fewer misses). Why it helps hide the rest: sequential, contiguous access is prefetchable and, unlike pointer-chasing, does not force a dependency chain where each object's address depends on reading the previous one — so the out-of-order engine (Chapter 2 §2.3) can overlap independent accesses and hide the misses that remain. *Method note:* the two wins are distinct — locality (fewer misses) and independence (hide the misses) — and the pointer layout loses on both. **Feed-forward:** §3.5.

3.6 (a) 8 bytes used of 64 fetched = **1/8** of each line. (b) A stride-1 traversal uses all 8 values per line, so the stride-8 traversal moves roughly **8× more memory** for the same values. (c) That ~8× is a floor on the slowdown from wasted bandwidth alone; the measured row-major/column-major ratio was larger (~24–26×) because real effects compound it — two that push the measured number above the naive estimate: (i) the hardware **prefetcher** helps the sequential (row-major) loop even more, widening the gap; (ii) **loss of memory-level parallelism / stalling** on the strided loop, and possibly **TLB misses** from touching many pages, add cost the simple bytes-moved model ignores. *No-fabrication note:* the 8× is arithmetic; the ~25× is measured — do not present the estimate as the measurement. **Feed-forward:** §3.6.

3.7 **Step 4:** In this particular experiment the dominant lever is the **number of misses (locality)**, not the inability to hide each one. The column-major loop touches a new line almost every iteration and uses only 1 of 8 values per line, so it issues roughly 8× as many line fetches and moves many times more memory than the row-major loop; that sheer volume of DRAM traffic is the main cost. The dependency argument is secondary here because both versions are simple predictable summations with similar per-iteration dependency structure — what differs overwhelmingly is how many expensive lines each one demands. The number of misses is primary *because the two loops differ in locality but not meaningfully in their dependency chains* — so the variable that changed is misses. To test the claim: measure cache misses and memory traffic for both (e.g. with a profiler's cache-miss counters) and confirm the column-major loop's miss count is the multiple that tracks its slowdown. **Feed-forward:** §3.5–3.6, Chapter 2 §2.3.

3.8 (a) **Capacity miss fixed by blocking** — tiling keeps each block's working set inside cache so it is reused before eviction. (b) **Branch prediction** — sorting makes the filter's data-dependent branch predictable (Chapter 2 §2.4). (c) **Spatial locality / cache lines** — consuming each line fully raises useful-bytes-per-line. (d) **ILP / dependency chain** — independent accumulators expose parallelism (Chapter 2 §2.3). *Method-selection cue:* "tiling/ blocking" → capacity; "sorting helped a branch" → prediction; "layout/stride" → spatial locality; "extra accumulators" → ILP.

3.9 Candidate causes across Chapters 1–3 (at least three): **poor spatial locality / high miss rate** (§3.1–3.5); **capacity misses** from a working set exceeding cache, fixable by blocking (§3.3); **branch mispredictions** from a data-dependent branch (Ch 2 §2.4); **a serial dependency chain / low ILP** (Ch 2 §2.3); pointer-chasing combining bad locality *and* dependencies (§3.5). The first measurement: **run a profiler that reports cache-miss rate and branch-misprediction rate** — it is most informative because it directly distinguishes the two biggest mechanism families (memory vs. branches) in one shot, telling you which chapter's toolkit to reach for before you change any code. *Tempting wrong answer:* "start rewriting the layout" — that guesses the cause; measure first (the discipline of this whole Part). **Feed-forward:** §3.6 and Chapter 1 §1.4.

3.10 (a) Three properties and their effect on the ratio: **cache/last-level-cache size** — a much larger cache (or a smaller matrix) shrinks the gap by keeping more of the working set resident; **memory bandwidth and prefetcher quality** — a stronger prefetcher and higher bandwidth help the sequential loop more, tending to *raise* the ratio; **compiler behavior** — if the compiler vectorizes or reorders one loop, the ratio can move either way. (b) The effect's existence and rough size are robust because they follow from a structural fact true of essentially all cache-based machines: memory moves in lines, and a strided traversal wastes most of each line — so a large (not small) multiplier is guaranteed even though its exact value tracks the specific hardware. (c) Reporting "about 25× on this specific CPU, reproduce it yourself" is honest because the number was measured on one machine and genuinely varies; it is more useful because it teaches the reader the *method* (measure on your own hardware) rather than a false universal constant they might quote as fact — which is exactly the no-fabrication discipline of this book. **Feed-forward:** §3.6 and the warning box.

3.11 Machine-specific by design. A correct write-up records the reader's real cache-line size (commonly 64 bytes) and L1/L2/L3 sizes; confirms both loops produce the identical sum (proving equal work); reports two measured times and the observed ratio; and compares that ratio to the book's ~25× and to the ~8× stride estimate from §3.6, expecting it to sit somewhere around or above the estimate and to vary with hardware. The essential discipline: report only measured numbers, and if the compiler elides a loop, change the code (consume the result / use a `volatile` sink) and say what was done — never quote a number you did not observe. **Feed-forward:** §3.6; deeper measurement in the performance-engineering chapters.

Chapter 4 — Solutions

4.1 (a) **12 bits** (2^{12} = 4096 = 4 KiB). (b) **One translation**: `a[0]` and `a[1]` sit on the same 4 KiB page (unless `a[0]` happens to be the last element of a page), so the same page number translates both. (c) A 4 KiB page holds 4096/8 = **512 `longs`**, so up to ~512 consecutive elements share one translation before you cross into the next page. This is the TLB analogue of the cache-line reuse from §3.1 — one translation amortized over many accesses.

4.2 **Isolation** — each process names only its own frames, so it cannot read or corrupt another's memory (this is the security/isolation boundary). **A simple, uniform programming/layout model** — every process sees the same clean virtual layout regardless of where it lands in physical RAM. **OS flexibility** — a virtual page can map to a frame, a file, nothing yet (lazy allocation), or be shared read-only. The isolation one is the protection boundary.

4.3 A data cache stores recently used **cache lines** so a repeated data access is a cheap hit; the TLB stores recently used **translations (virtual-page→physical-frame mappings)** so a repeated access to the same **page** skips the **page-table walk**. Same hit/miss/eviction logic (§3.3), different key and value: address→line for the data cache, page→frame for the TLB.

4.4 A multi-level walk reads one table to learn the address of the next table, reads that to learn the next, and so on — **each read's address depends on the value returned by the previous read**, so the reads cannot be issued in parallel or overlapped; they are strictly serial. That matters because a dependency chain defeats the CPU's latency-hiding (Chapter 2 §2.3): there is no independent work to run during each memory access, so the walk pays the full latency of every step in sequence. This is exactly the **pointer-chasing** structure of a linked-list traversal (§3.5): in both, you cannot compute the next address until the current access completes, so both are serial-latency-bound rather than bandwidth-bound. *Method note:* recognizing "the next address depends on this read" is the general tell for a latency-bound pattern. **Feed-forward:** §3.5, Chapter 2 §2.3.

4.5 (a) A random probe pays (i) a **data-cache miss** to fetch the line holding the bucket (~100 ns to DRAM) and (ii) a **TLB miss / page-table walk** to translate the bucket's address before the fetch — several dependent memory accesses. (b) With 4 KiB pages, TLB reach is (entries × 4 KiB), typically only a few megabytes; a 400 MiB table spans far more pages than the TLB can hold, so random probes keep landing on pages whose translations were evicted — heavy TLB misses (a capacity miss at the translation level, §4.4). (c) To cut translation cost: map the table with **huge pages** (raises TLB reach ~512×), assuming a profiler confirms page-walk cost. To cut the data-miss cost: improve **locality** of the buckets (e.g. keys and values stored together, open addressing with good probe locality, or a more cache-friendly layout) so more probes hit. *Tempting wrong answer:* "it's all just DRAM latency, nothing to do" — the §4.4 trap; the surplus over ~100 ns is the translation cost. **Feed-forward:** §4.4–4.5.

4.6 (a) **reach = E × 4 KiB**. (b) Expect heavy thrashing when the randomly-touched footprint exceeds reach: **W > E × 4 KiB** (more precisely, when the number of distinct pages touched, W / 4 KiB, far exceeds E), so translations are evicted before reuse. (c) Switching to 2 MiB pages multiplies reach by **2 MiB / 4 KiB = 512×**, so the threshold in (b) becomes W > E × 2 MiB — a working set 512× larger now fits within reach, which is exactly why huge pages relieve TLB thrashing. *No-fabrication note:* the 512× is arithmetic from real, documented page sizes; the actual E is a chip-specific number this book does not invent. **Feed-forward:** §4.5.

4.7 **Step 4:** The random-by-page walker needs a *new* translation on essentially **every jump**, because each jump targets a different page and it does not stay on any page long enough to reuse a translation. Since the TLB's reach (a few MiB) is far smaller than the 512 MiB footprint, the translation for a page it jumps to was almost certainly evicted since the last visit — so nearly every random access **misses the TLB and pays a page-table walk**, on top of the data-cache miss. The sequential walker, by contrast, amortizes one translation over ~512 accesses (Step 3), so its TLB-miss *rate* is ~1 per 512 accesses versus ~1 per access for the random walker. To confirm the TLB — not just the data cache — is the culprit, measure the **dTLB-load-miss (or page-walk) count/rate** for both runs (e.g. `perf stat -e dTLB-load-misses`): the random run should show dramatically more, isolating translation cost from data-fetch cost. **Feed-forward:** §4.4, and the Chapter 6 capstone.

4.8 (a) **TLB miss / page-walk** — huge pages helping is the signature; they raise reach, not data-cache capacity. (b) **Data-cache capacity miss** — 64 MiB exceeds the 24 MiB L3 so it lives in DRAM, while 1 MiB fits in L2; same accesses, different tier (Ch 3 §3.3). (c) **Dependency chain / low ILP** combined with poor locality — pointer-chasing a list is serial and scattered (§3.5); the standout distinguishing feature here is the serial dependency. (d) **Branch misprediction** — sorting makes the data-dependent branch predictable (Ch 2 §2.4). *Method-selection cue:* "huge pages helped" → TLB; "bigger set, re-read" → capacity; "linked list" → dependency/locality; "sorting fixed a branch" → prediction.

4.9 (a) Huge pages help specifically when **TLB misses / page-walk cost are a measured bottleneck** — i.e. a large, scattered working set whose page footprint exceeds TLB reach. (b) Downsides: internal **fragmentation** / wasted memory (a 2 MiB page backing a few live KiB); difficulty **finding contiguous physical memory** once RAM is fragmented (allocation can fail or stall); and no benefit — pure overhead — for workloads that were never TLB-bound. (c) First measurement: the **TLB-miss / page-walk rate** (e.g. `perf stat -e dTLB-load-misses`); if it is low, huge pages will not help. *Discipline:* this is the "measure before you optimize" rule of Part 1 — don't flip huge pages on faith. **Feed-forward:** §4.5 and the Chapter 6 lab.

4.10 (a) **Compulsory**: yes — the first access to a page's translation always misses the TLB (it was never cached), just as the first touch of a line is a cold miss. **Capacity**: yes — when the page footprint exceeds TLB reach, translations are evicted before reuse; this is TLB thrashing (§4.4). **Conflict**: an analogue exists for set-associative TLBs (translations colliding on the same TLB set), though it is secondary and hardware-specific; the dominant effect for large working sets is capacity. (b) Huge pages attack the *capacity* analogue: they do not add TLB entries but make each entry cover far more memory, so the same footprint needs fewer translations and fits within reach. (c) Larger base pages are not a free system-wide win because programs with small, sparse footprints would waste memory to internal fragmentation (rounding every allocation up to a huge page) and the OS would find it harder to place them; the 4 KiB default is a compromise, and huge pages are opt-in for the workloads that benefit. *Tempting wrong answer:* "just make all pages 2 MiB" — ignores fragmentation and allocation cost. **Feed-forward:** §4.5.

4.11 Machine-specific by design. A correct write-up records the real page size (commonly 4 KiB) and cites the architecture's documented page-table depth and huge-page sizes (do not guess — for x86-64, a four- level table with 2 MiB and 1 GiB huge pages); confirms both loops touch the same data the same number of times; and reports two measured ns-per-access figures showing the random-by-page run slower. Where TLB counters are available, the random run should show far more dTLB-load-misses / page-walks than the sequential run, isolating translation cost. The essential discipline: report only measured numbers, attribute the architecture facts to real documentation, and if the compiler elides a loop, consume the result (a `volatile` sink) and say so. **Feed-forward:** §4.4, and the Chapter 6 capstone which measures exactly this latency-bound gap.

Chapter 5 — Solutions

5.1 **Array-of-structs (AoS):** one array whose elements are whole records, so a record's own fields are adjacent in memory. **Struct-of-arrays (SoA):** one array per field, so the same field across different records is contiguous. **SoA** stores "the same field across different records" contiguously.

5.2 (a) **8 of 64 bytes** — one `double` per fetched line. (b) SoA uses all 8 values per line, so AoS moves roughly **8× more memory** for the same field. (c) SoA restores **spatial locality**: consecutive values of the scanned field are adjacent, so each line is full of values the loop uses (§3.1–3.2).

5.3 A hot loop reads only a couple of a record's many fields, so most of every fetched cache line is fields the loop never touches — dead weight that lowers useful-bytes-per-line. (Split the few hot fields into their own compact record; move the rest to a cold side-structure.)

5.4 (a) **AoS** — reads/writes all fields of one particle before moving on, so a record's adjacent fields are all used from one or two fetched lines. (b) **SoA** — summing one column over ten million rows is a stride-1 scan of that field; contiguous storage uses every byte of every line. (c) **SoA** — a SIMD kernel over one field wants that field contiguous so a vector load grabs consecutive values. (d) **AoS** — position, color, and size of one object are used together, so keeping a record's fields adjacent brings them in one fetch. *Method-selection cue:* "one field across records / SIMD one channel" → SoA; "whole record at a time" → AoS.

5.5 (a) The `double` needs 8-byte alignment, so the compiler inserts **padding** after `flag` to place `value` at an 8-byte offset, and adds tail padding so an array keeps every element aligned — pushing the size above 13 bytes (commonly to 24). (b) Order large/strictly-aligned first: `struct S { double value; int id; char flag; };` — `value` at offset 0, `id` at 8, `flag` at 12, then a little tail padding to the struct's alignment; this minimizes internal gaps. (c) A smaller struct means **more records per cache line and per page**, so a scan over ten million of them fetches fewer lines and touches fewer pages — better spatial locality and less TLB pressure, for free, from field ordering alone. *Tempting wrong answer:* "field order doesn't affect layout in C" — it does, through alignment padding. **Feed-forward:** §5.4.

5.6 (a) The scan uses **8 useful bytes of each 64-byte line** (assuming one field per record on a line), a fraction of **8/64 = 1/8** when $S \geq 64$; for smaller S a line may hold more than one record's copies but you still touch one field's worth per record. (b) As S grows past the line size, each record's scanned field sits on its own line, so the AoS scan moves ~$S/8$ lines' worth where SoA moves 1 — the bandwidth ratio grows with S (bounded by one line fetched per record). (c) The measured speedup can be smaller than the bandwidth ratio because the SoA loop may be limited by something other than bandwidth — e.g. a **single-accumulator dependency chain** (Ch 2 §2.3) that caps how fast it can consume values, so it cannot fully cash in the saved bandwidth. That is exactly why this chapter's demo measured ~4×, not the naive 8×; removing the accumulator ceiling widens it. *No-fabrication note:* 8× is the arithmetic ceiling; ~4× is what was measured — keep them distinct. **Feed-forward:** §5.2 and Chapter 6.

5.7 **Step 4:** Decide empirically. **Measure** where the runtime actually goes — profile both loops (or time them separately) to learn which dominates and by how much; the layout should favor whichever loop owns the most wall-clock time. If loop A (scan) dominates, lean SoA; if loop B (whole-record) dominates, lean AoS. A **hybrid** can serve both partly: hot/cold-split so the field loop A scans lives in its own contiguous array (SoA for that field) while the fields loop B updates together stay grouped — or duplicate/derive the scanned field into a packed side-array refreshed when needed. The posture "pick the layout that helps the hotter loop, then re-measure" is right because the interaction of layout with the prefetcher, the accumulator dependency, and the two loops' real proportions is hard to predict on paper (the demo's ~4×-not-8× is proof), so the paper argument only picks the *candidate* and the measurement decides. This reflects the **measure-before-you-optimize** discipline of Part 1. **Feed-forward:** §5.2–5.3, Chapter 6.

5.8 (a) **Hot/cold splitting** — moving a cold field out raises useful-bytes-per-line for the hot loop. (b) **SoA / whole-line use** — column arrays make the column scan stride-1. (c) **Capacity miss fixed by blocking** — tiling keeps each block resident (Ch 3 §3.3). (d) **TLB / huge-page effect** — 2 MiB pages raise TLB reach for the random index (Ch 4 §4.5). (e) **ILP / multiple accumulators** — independent accumulators break the dependency chain (Ch 2 §2.3). *Method-selection cue:* "cold field moved" → hot/cold; "rows→columns" → SoA; "tiling" → capacity/blocking; "huge pages" → TLB; "more accumulators" → ILP.

5.9 Candidate causes across Chapters 2–5 (at least three, incl. a layout cause and an earlier one): **AoS-when-scanning-one-field / poor useful-bytes-per-line** (§5.2); **cold fields bloating hot records**, fixable by hot/cold splitting (§5.3); **a capacity miss** from a working set exceeding cache (Ch 3 §3.3); **TLB misses** on a large scattered footprint (Ch 4); **a single-accumulator dependency chain / low ILP** (Ch 2 §2.3); a **data-dependent branch** misprediction (Ch 2 §2.4). First measurement: run a **profiler reporting cache-miss rate (and, if available, TLB-miss and branch-miss rates)** — it distinguishes the big mechanism families in one shot, telling you whether to reach for a layout change, a blocking change, or a branch/ILP fix before touching code. *Tempting wrong answer:* "rewrite the layout first" — that guesses; measure first. **Feed-forward:** §5.2 and Chapter 6.

5.10 (a) Both follow from one fact: **a whole cache line moves as a unit**. Single-threaded, you want the bytes you use together on one line (packing) so one fetch serves them all. Multi-threaded, if two cores write different variables on one line, that same all-or-nothing line movement forces the line to bounce between caches on every write (false sharing). Same mechanism, opposite consequence depending on who writes. (b) Deliberately pad a struct up to a full line for **per-thread counters/state written by different threads** — pad so each thread's variable sits on its own line, eliminating false sharing; you trade away memory (and some single-thread locality) to stop the coherence ping-pong. (c) The choice depends on core count because false sharing only occurs when *multiple cores write* the shared line; with one writer, packing is pure win — so "how many cores write this?" decides whether to pack or spread. *Tempting wrong answer:* "packing is always good for cache performance" — true single-threaded, false under concurrent writers. **Feed-forward:** §5.5 and the concurrency Part.

5.11 Machine-specific by design. A correct write-up records the cache-line size, defines a line-sized record, confirms both sums are identical (equal work), reports two measured times and the observed SoA/AoS ratio, and compares it to the book's ~3.7–4× and to the ~8× bandwidth ceiling — expecting a several-fold win that varies with hardware. The second part should show the ratio widening toward 8× once the accumulator dependency is removed (multiple accumulators / vectorization), confirming the demo's explanation for why it measured ~4×. Essential discipline: report only measured numbers, and if a loop is elided, consume the result and say what was done. **Feed-forward:** §5.2, §5.6, and the Chapter 6 capstone.

Chapter 6 — Solutions

6.1 **Latency** is the time to complete *one* access you must wait for; **bandwidth** is the throughput when streaming *many* contiguous items. Use **latency** for (a) fetching one random element (a single wait) and **bandwidth** for (b) streaming a huge array in order (the prefetcher keeps many accesses in flight).

6.2 (a) **Memory-bound** — `memcpy` does no arithmetic per byte; it is pure data movement at memory bandwidth. (b) **Compute-bound** — SHA-256 does much arithmetic per byte and the data is already in L1, so the execution units are the limit. (c) **I/O-bound** — the time is dominated by per-row network/SSD latency, not CPU or memory. *Method-selection cue:* "work per byte" — near zero → memory; high → compute; waiting on storage/ network → I/O.

6.3 **Estimate** (predict the class and rough cost from the cost model), **measure** (run it and observe the real time / counters), **reconcile** (explain any gap and update the model). A gap is useful because it usually reveals a wrong assumption — most often using latency where bandwidth applies (or vice versa) — so the discrepancy points directly at the true bottleneck.

6.4 (a) 512 MiB of `long` is 2^{26} = ~67 million elements. Latency estimate: 67M × 100 ns ≈ **~6.7 s**. Bandwidth estimate: 512 MiB / 6 GB/s ≈ **~0.09 s**. (b) The **bandwidth** estimate is right for a sequential sum: the prefetcher streams the data, so accesses overlap rather than each incurring the full latency (this matches the ~0.08–0.09 s measured in §6.3). (c) They differ by roughly **~70–100×** — the cost of using the latency model for a bandwidth-bound workload is being wrong by about two orders of magnitude, exactly the §6.3 surprise. *Tempting wrong answer:* the ~6.7 s latency figure — tempting because ~100 ns is the "known" DRAM number, but it is the wrong number for streaming. **Feed-forward:** §6.3.

6.5 (a) The sum is stride-1, so the hardware prefetcher recognizes the pattern and fetches lines ahead of the loop, keeping many independent accesses in flight; the loop is then limited by how fast memory can *stream* (bandwidth), not by any single access's latency. (b) Even the L2 sum is capped by the **single-accumulator dependency chain** (§2.3): each `sum +=` depends on the previous, so the additions serialize at roughly one add-latency apart, limiting throughput below what the cache could supply. (c) Sum into **several independent accumulators** (or let the compiler vectorize), breaking the dependency chain so more values are consumed per cycle and the loop can approach memory bandwidth. *Method note:* the two limiters — prefetchable bandwidth and the add dependency — are distinct; naming both is the full explanation. **Feed-forward:** §6.3 and Chapter 2 §2.3.

6.6 (a) **compute-time ≈ F / 10^{10} s**; **memory-time ≈ B / 10^{10} s**; the runtime floor is the larger of the two. (b) Compute-bound when compute-time > memory-time, i.e. **F / B > 1** op per byte for these rates (more generally, when work per byte exceeds the machine's ops-per-byte balance point). (c) A sum does 1 add per 8 bytes, so F/B = 1/8 < 1 → **memory-bound** by this test. §6.3 agrees: the sum's speed tracked memory (GB/s), not the add rate — though the single-accumulator dependency kept it from reaching full bandwidth, which is why the measured GB/s was modest. *No-fabrication note:* the 10^{10} rates are round orders of magnitude for estimation, not a spec of any particular chip. **Feed-forward:** §6.2–6.3.

6.7 **Step 4:** ~150 ns/step exceeds the ~100 ns bare DRAM latency because the random chase over 512 MiB also **thrashes the TLB** (Chapter 4): each jump lands on a new page whose translation has been evicted, so most steps pay a **page-table walk** — extra dependent memory accesses to translate the address — *on top of* the data miss. So the ~150 ns is roughly the DRAM data latency plus the translation cost. To confirm the surplus is translation and not data, measure the **dTLB-load-miss / page-walk rate** (e.g. `perf stat -e dTLB-load-misses`) and check it is high for the random run and negligible for the sequential one. The pair teaches the rule: estimate with **bandwidth** for prefetchable sequential access (§6.3's ~1.7×) and with **latency** — plus translation cost — for scattered, dependent access (this ~25×); the access pattern, not just the data's tier, chooses the model. **Feed-forward:** §6.3–6.4, Chapter 4 §4.4.

6.8 (a) **Memory-bound**, memory bandwidth saturated — a single core's sequential scan already uses much of the available bandwidth, so more threads add little (the bottleneck is the memory pipe, not cores). (b) **Compute-bound-ish / control**, branch misprediction — sorting made the data-dependent branch predictable (Ch 2 §2.4), cutting flushes. (c) **Memory-bound**, TLB miss / page-walk — the ~160 ns over ~100 ns and the huge-page fix are the TLB signature (Ch 4). (d) **Compute-bound**, execution throughput — the multiply-add issue rate limits an in-cache matmul (Ch 2). *Method-selection cue:* "adding threads doesn't help a scan" → bandwidth-bound; "sorting fixed it" → branch; "huge pages fixed it" → TLB; "issue rate limited" → compute.

6.9 (a) The 100× is a *latency* ratio, but a sequential scan is prefetchable and therefore **bandwidth-bound**, so latency is the wrong model — the actual slowdown vs. cache is a small factor, not 100× (§6.3). (b) Rough class: **memory-bound (streaming)**; rough slowdown vs. an in-cache version: on the order of a **~2×**, since DRAM streaming bandwidth is within a small factor of cache (and it may be further gated by the file/I-O layer for a file-backed array). (c) Confirm by measuring (i) the scan's **effective GB/s** (is it near memory bandwidth?) and (ii) its **cache-miss / prefetch behavior** (or simply compare to the same scan on an in-cache slice) before optimizing. *Tempting wrong answer:* accept the 100× and build a cache — that optimizes a bottleneck that isn't there. **Feed-forward:** §6.3 and the warning box.

6.10 (a) The **random** ratio is more portable because it is bounded by DRAM *latency* (plus translation), which is set by physics and the memory subsystem and varies modestly across machines; the **sequential** ratio is bounded by the *bandwidth* gap between L2 and DRAM and by loop details (accumulators, vectorization, prefetcher quality), which vary a lot — so the sequential number is the fragile one. (b) Raise the sequential ratio: a stronger prefetcher / higher L2 bandwidth relative to DRAM (widens the streaming gap), or removing the accumulator dependency so the L2 loop speeds up more than the DRAM loop. Lower the random ns/step: faster DRAM / lower memory latency, a larger TLB or huge pages (cuts page-walk cost), or a bigger last-level cache that keeps more of the footprint on-chip. (c) Reporting both ratios with their access patterns is honest because there is no single "DRAM is N× slower" — the answer depends entirely on whether access is streaming or scattered; a lone headline number invites readers to apply it to the wrong pattern and mis-estimate by orders of magnitude, which is exactly the §6.3 trap. *Tempting wrong answer:* "just quote the bigger, scarier ratio" — that mispredicts every bandwidth-bound workload. **Feed-forward:** §6.3–6.4.

6.11 Machine-specific by design, and the write-up matters as much as the numbers. A correct submission (1) records real L2/L3 and cache-line sizes; (2) states a *written-first* prediction with the expected bound (sequential → bandwidth-bound, small ratio; DRAM chase → latency-bound, large ratio); (3) reports measured ns/element and GB/s for both sums (identical sums verified); (4) reports ns/step for both pointer-chases; and (5) reconciles each measurement against the prediction — expecting a modest sequential ratio (bandwidth) and a large random one (latency plus TLB), ideally backed by `perf` cache-miss and dTLB-miss counters showing the random DRAM run dominated by misses and page-walks. The essential discipline: predict before measuring, report only observed numbers, explain surprises with latency-vs-bandwidth and the TLB rather than hand-waving, and if a loop is elided, consume the result and say so. *Tempting wrong answer:* skipping the written prediction and only reporting measurements — you lose the reconciliation that is the whole point. **Feed-forward:** §6.3–6.4 and the performance-engineering Part.

Chapter 7 — Solutions

7.1 (a) A pointer variable stores a **memory address** (a virtual address, per Ch 4). (b) `*p` **dereferences** — goes to the address in `p` and reads or writes the object there, using `p`'s type to know how many bytes and how to interpret them. (c) `&x` produces the **address of** `x` (a pointer to `x`).

7.2 As a `double *`: `q + 4` = 2000 + 4×`sizeof(double)` = 2000 + 4×8 = **2032**. As an `int *`: 2000 + 4×`sizeof(int)` = 2000 + 4×4 = **2016**. The rule: `p + n` advances by `n × sizeof(*p)` bytes, so the byte step is the element size. *Method note:* always multiply the integer by the pointee's `sizeof`, never treat it as a byte offset. **Feed-forward:** §7.2.

7.3 `sizeof(a)` = **40** (10 ints × 4 bytes — the whole array); `sizeof(p)` = **8** (just the pointer, on this x86-64 machine). Difference: `a` names the array object, so `sizeof` measures all its storage; `p` is a separate pointer variable, so `sizeof` measures the pointer, not what it points at. *Tempting wrong answer:* "both are 40 because `p` points at the array" — no, `sizeof` looks at the operand's *type*, and `p`'s type is `int *`. **Feed-forward:** §7.3.

7.4 **Array-to-pointer decay:** in most expression contexts an array name is automatically converted to a pointer to its first element (C §6.3.2.1), which is why `arr` can be assigned to an `int *`, indexed, or passed to a function taking `int *`. In `size_t len(int a[]) { return sizeof(a)/sizeof(a[0]); }` the parameter `a` is *already a pointer* (a function parameter of "array" type is adjusted to pointer type; the array decayed at the call), so `sizeof(a)` is the **pointer size** (8 here), not the array size. The returned value is `8 / sizeof(int)` = **2** on this machine, regardless of the caller's real length. Correct approach: pass the length explicitly. *Tempting wrong answer:* "it returns the element count" — it cannot; the length was lost at the call boundary. **Feed-forward:** §7.3.

7.5 `struct S { char a; short b; char c; int d; };` with align = size (char 1, short 2, int 4): `a` at offset 0 (byte 0); `b` (align 2) needs an even offset, next is 2, so 1 byte padding at 1, `b` occupies 2–3; `c` at offset 4 (byte 4); `d` (align 4) needs a multiple of 4, next after byte 4 is 8, so 3 bytes padding at 5–7, `d` occupies 8–11. Struct alignment = max(1,2,1,4) = 4; running size 12 is already a multiple of 4, so no tail padding. **`sizeof(struct S) = 12`**; real data = 1+2+1+4 = 8, so **4 bytes padding**. Verify with `offsetof(struct S, member)` for each member and `sizeof(struct S)`. *Tempting wrong answer:* "1+2+1+4 = 8" — ignores alignment padding. **Feed-forward:** §7.5.

7.6 Order largest-alignment first: `struct S { int d; short b; char a; char c; };` — `d` at 0 (0–3), `b` at 4 (4–5), `a` at 6, `c` at 7; running size 8, alignment 4, 8 is a multiple of 4, so no tail padding. **`sizeof = 8`** — the size of the real data, zero padding. Rule applied: placing members in decreasing alignment order minimizes the internal gaps needed to re-align each successive member (and packs the small fields into what would otherwise be padding). *Tempting wrong answer:* "field order can't matter in C" — it does, via alignment padding. **Feed-forward:** §7.5, and §5.4.

7.7 (a) 10 M × 24 bytes = **240 MB**. (b) 10 M × 16 = **160 MB**. A 64-byte line holds `64/24` = 2 whole 24-byte records vs. `64/16` = 4 whole 16-byte records — **2 more per line**; a 4 KiB page holds 4096/24 = 170 vs. 4096/16 = 256 — **86 more per page**. (c) The smaller record raises useful-bytes-per-line and records-per-page, so a scan fetches fewer cache lines and touches fewer pages — better spatial locality and less TLB pressure (Ch 3–5), for free, from field ordering. *Tempting wrong answer:* ignoring that a line/page holds a *whole* number of records (partial records straddle boundaries), which is why we floor. **Feed-forward:** §7.5, §5.4.

7.8 **Step 4:** The struct's alignment is max(8,1,4) = **8**. After `b` the running size is 16, which *is* a multiple of 8, so **no tail padding** is needed. Final **`sizeof(struct T) = 16`**; real data = 8+1+4 = 13, so **3 padding bytes** (the ones at offsets 9–11). Confirm each member's offset with `offsetof(struct T, member)` (and `sizeof` for the total). *Method note:* always finish by rounding the size up to the struct's alignment — here it happened to already be aligned. **Feed-forward:** §7.5.

7.9 (a) **Array-to-pointer decay** (Ch 7) — the parameter is a pointer, so `sizeof` is 8; length was lost at the call. (b) **TLB miss / page-walk** (Ch 4) — huge pages helping is the signature; they raise TLB reach. (c) **Struct padding/alignment** (Ch 7/5) — the compiler inserts padding to align members. (d) **Endianness** (Ch 7) — the other machine's byte order differs, so integers appear byte-swapped. *Method-selection cue:* "sizeof of a parameter is 8" → decay; "huge pages helped" → TLB; "bigger than the fields" → padding; "byte-swapped across machines" → endianness. **Feed-forward:** §7.3–7.6.

7.10 (a) A **virtual** address (Ch 4) — the hardware translates it to physical on access. (b) `&arr[5] - &arr[2]` = **3** (an element count of type `ptrdiff_t`; pointer subtraction divides out the element size). (c) **Not stable** — a PIE build is relocated by ASLR each run, so absolute addresses change; the relative layout is stable. (d) **No** — the C standard guarantees only `sizeof(char)==1` and minimum ranges (`int` ≥ 16 bits); 4 bytes is this ABI's implementation-defined value, verified, not guaranteed. *Tempting wrong answer:* on (b), answering "12 bytes" — subtraction yields elements, not bytes. **Feed-forward:** §7.2, §7.4–7.5, and Ch 4.

7.11 (a) For `int arr[10]`, `&arr[10]` is the special "one past the last element" address, which the standard explicitly permits you to *form* and compare against (useful as a loop end), but the object there does not exist, so *dereferencing* it is undefined (C §6.5.6). (b) Making it undefined (rather than "wraps" or "reads adjacent memory") lets the optimizer assume in-bounds access — enabling optimizations and diagnostics — and reflects that beyond the array there may be padding, another object, an unmapped page, or a segment boundary (Ch 4), so no single "reasonable" behavior exists to define. (c) This is exactly the **buffer overflow** bug of Ch 9: an out-of-bounds access is UB, and an out-of-bounds write is the classic memory-corruption vulnerability. *Tempting wrong answer:* "one-past-the-end is illegal to compute" — it is legal to compute and compare, only illegal to dereference. **Feed-forward:** §7.2 and §9.3.

7.12 Machine-specific by design. A correct write-up reports measured `sizeof`/`_Alignof` for the scalar types (on x86-64 LP64: char 1, short 2, int 4, long 8, double 8, pointer 8, alignment = size), a struct whose hand-computed offsets and total `sizeof` match `offsetof`/`sizeof` exactly, and an endianness check (x86-64 prints the low byte first — little-endian). The essential discipline: label the machine/ABI, report only measured numbers, and note that a different platform (16-bit, or big-endian, or a different ABI) would legitimately differ — which is the whole reason to measure rather than assume. *Tempting wrong answer:* hard-coding "int is 4 bytes" as universal — it is implementation-defined. **Feed-forward:** §7.5–7.6.

Chapter 8 — Solutions

8.1 (a) A **stack frame** (activation record) is the block of storage for one function call — its locals, arguments, saved return address, and bookkeeping. (b) Subtracting the frame's size from the **stack-pointer register** (moving it down) allocates it — one arithmetic operation. (c) A local's lifetime ends when execution **leaves its block/function** and the frame is popped.

8.2 Heap reason (any one): the object must **outlive the function that creates it**, its size is only known at runtime, or it is too large for the stack. Cost the heap imposes: the allocator does real work (searching/splitting free lists), and — the big one — **you must free it exactly once**, an obligation the stack does not have (frames free automatically on return). **Feed-forward:** §8.2, §8.5.

8.3 (1) **Runaway/unbounded recursion** (missing base case or too deep for the input) — fix by bounding the recursion or converting to iteration with an explicit (heap-backed) stack. (2) **Very large automatic arrays/locals** — fix by allocating large or runtime-sized buffers on the **heap** instead. *Tempting wrong answer:* "increase the stack limit" — a band-aid that does not fix unbounded recursion. **Feed-forward:** §8.3.

8.4 `int *f(void){ int x=7; return &x; }` returns the address of the local `x`, which lives in `f`'s stack frame. On `return`, that frame is **popped** — the stack pointer moves back — so `x`'s storage is no longer reserved for it and can be reused by the next call. The returned pointer therefore **dangles**, and reading/writing through it is **undefined behavior** (C §6.2.4: using a pointer to an automatic object past its lifetime). gcc emits `warning: function returns address of local variable [-Wreturn-local-addr]`; clang emits `-Wreturn-stack-address`. *Tempting wrong answer:* "it returns 7 because the value is still in memory" — not guaranteed; UB imposes no such promise. **Feed-forward:** §8.4.

8.5 (a) **Caller owns the storage:** `void f(int *out){ *out = 7; }`, called `int x; f(&x);` — `x` lives in the caller's frame and outlives `f`. (b) **Heap:** `int *f(void){ int *p = malloc(sizeof(int)); if(!p) return NULL; *p = 7; return p; }` — the block outlives the call; **the caller must `free` it** exactly once, after its last use (ownership transfers to the caller). *Method note:* the choice is "who owns the memory?" — make it explicit. **Feed-forward:** §8.4–8.5.

8.6 "It worked" is not evidence of correctness because the bug is **undefined behavior**: the C standard imposes no requirements, so the program is free to *appear* to work — the dead frame's bytes may simply still hold the old value until something reuses them. Correctness is a property of the code against the standard, not of one run's observed output. The danger of a bug that appears to work is that it is **latent**: it hides through testing and surfaces later when the optimization level, compiler, or call pattern changes — often in production, and possibly as data corruption rather than an obvious crash. *Tempting wrong answer:* "if it passes tests it's fine" — UB can pass tests and still be a defect. **Feed-forward:** §8.4.

8.7 (a) A stack frame's locals are reserved by **subtracting the frame size from the stack pointer** — a single register adjustment, with no per-variable work and nothing to track. (b) One `malloc` may **search a free list** for a fitting block, **split** it, update bookkeeping, and, if the pool is exhausted, **ask the OS for more memory** — orders of magnitude more work. (c) Rule of thumb: for a small scratch buffer needed each iteration, use a **stack local (or one buffer reused across iterations)**, not a per-iteration `malloc`/`free` — the stack version is nearly free and avoids allocator traffic and fragmentation in the hot loop. *Tempting wrong answer:* "malloc a fresh buffer each iteration for cleanliness" — needless cost in a hot loop. **Feed-forward:** §8.1–8.2.

8.8 **Step 4:** The nodes must be allocated on the **heap** (they outlive the helper and their count is dynamic). The **caller becomes responsible for freeing** every node — exactly once each, after it is done traversing the list (walking and freeing node by node). If that responsibility is forgotten, the result is a **memory leak** (Ch 9 preview): the nodes are never reclaimed. The design principle is **ownership** — decide and document who owns the memory and therefore who frees it. *Method note:* "outlives its creator" + "size known only at runtime" is the standard signal for heap allocation with transferred ownership. **Feed-forward:** §8.5 and §9.1, §9.3.

8.9 (a) **Stack overflow** (Ch 8) — deep recursion only on large inputs exhausts the bounded stack. (b) **Dangling pointer to a local** (Ch 8) — correct in tests, garbage after unrelated code changes the call pattern that reuses the frame; classic UB signature. (c) **TLB miss / page-walk** (Ch 4) — huge pages helping points at translation reach. (d) **Struct padding/alignment** (Ch 7) — the compiler pads to align members. *Method-selection cue:* "deep recursion, large input" → overflow; "works then garbage after unrelated change" → dangling/UB; "huge pages helped" → TLB; "bigger than its fields" → padding. **Feed-forward:** §8.3–8.4.

8.10 (a) **Well-defined** — using `&local` within the same function is fine; the object is alive. (b) **Undefined behavior** — the local's frame is popped on return, so the pointer dangles (§8.4). (c) **Performance/robustness issue tending to undefined behavior** — `int big[8000000]` is ~32 MB, likely overflowing a few-MB stack; if it overflows, the access faults (typically a crash). (d) **Undefined behavior** — storing `&local` in a global and reading it after return is the same dangling-pointer bug as (b), just via a longer-lived stash. *Tempting wrong answer:* calling (d) safe "because a global is permanent" — the global outlives the object, which is exactly the problem. **Feed-forward:** §8.3–8.4.

8.11 (a) Both violate the property that **a pointer must not be used outside its target's lifetime**. (b) The stack version is easier to catch because the direct pattern — returning `&local` — is statically visible, and compilers warn (`-Wreturn-local-addr` / `-Wreturn-stack-address`); the heap version (use-after-free) generally is not statically detectable, since the `free` and the later use can be far apart and flow-dependent, so it usually needs a runtime tool (Ch 9's sanitizers). (c) Both stem from "the storage is reclaimed whether or not a pointer still refers to it" — the frame pop (stack) and `free` (heap) both end the object's lifetime without invalidating the pointers that name it, leaving a dangling handle. *Tempting wrong answer:* "they're unrelated bugs" — they are the same lifetime violation in different regions. **Feed-forward:** §8.4 and §9.3.

8.12 Machine-specific by design. A correct write-up records the exact warning (on gcc, `-Wreturn-local-addr`; on clang, `-Wreturn-stack-address`); reports what the run actually did on that machine (on the book's machine the plain build **segfaulted**, but the write-up must note this is *not* guaranteed by the standard — a different build could print a value or garbage); and shows the caller-owned fix (`void f(int *out){ *out = 7; }`) compiling warning-free and behaving correctly. Essential discipline: never rely on the observed (b) outcome — UB is not obligated to crash — and reason about lifetime instead. *Tempting wrong answer:* "it segfaults, so it's at least safe to detect" — the crash is not reliable. **Feed-forward:** §8.4, and Ch 9's tools.

Chapter 9 — Solutions

9.1 (a) **No** — `malloc` returns uninitialized (indeterminate) bytes. (b) **Yes** — `calloc` returns all-zero bytes. (c) **Yes** — `free(NULL)` is an explicit no-op, always safe.

9.2 Safe idiom: `void *tmp = realloc(p, n); if (tmp) p = tmp; else { /* handle failure; p still valid */ }`. With `p = realloc(p, n);`, if `realloc` fails it returns `NULL` and leaves the original block *valid but unreferenced* — you have overwritten your only pointer to it, so it is **leaked** (and you have lost the data). *Tempting wrong answer:* "assign directly, it's cleaner" — cleaner and leaky. **Feed-forward:** §9.1.

9.3 **Use-after-free:** accessing a block after `free` — *UB*. **Double-free:** freeing the same block twice — *UB*. **Memory leak:** losing the last pointer without freeing — *not UB* (a defined defect). **Buffer overflow:** accessing outside an allocation's bounds — *UB*. **Uninitialized read:** using memory before writing it — *indeterminate value; unreliable, UB in general*. Four of five are undefined behavior; the leak is the exception. **Feed-forward:** §9.3.

9.4 "Returns the old value, so it's harmless" is wrong because use-after-free is **undefined behavior** (C §3.4.3): the standard imposes *no requirements*, so there is no guaranteed value and no guarantee it is harmless. Two realistic other outcomes: (1) the freed slot has been handed to another allocation, so you read (or clobber) **unrelated live data** — a correctness and security hole; (2) the access **corrupts allocator metadata or crashes**, possibly far from the buggy line. It can also differ by build or run, and be exploitable. *Tempting wrong answer:* the premise itself — treating UB as having a defined outcome. **Feed-forward:** §9.3.

9.5 (a) **Use-after-free** — reading `p[0]` after `free(p)`; UB. (b) **Buffer overflow** — `p[4]` on a 4-element (indices 0–3) block; UB. (c) **Double-free** — freeing the same block twice; UB. (d) **Uninitialized read** — reading a `malloc`'d `int` before writing it; indeterminate value, unreliable (UB in general). (e) **Memory leak** — overwriting the only pointer to a 1024-byte block without freeing; a defect, but *not* UB. *Method-selection cue:* "touched after free" → UAF; "index past the end" → overflow; "freed twice" → double-free; "read before write" → uninitialized; "pointer lost, not freed" → leak. **Feed-forward:** §9.3.

9.6 A **leak** frees *too late* — never; the block stays allocated but unreachable, so memory is wasted but no invalid access occurs, hence **not undefined behavior** (the program stays well-defined, just wasteful). A **use-after-free** frees *too early* — while a pointer still needs the block; the later access touches storage whose lifetime has ended, which **is undefined behavior**. The asymmetry: accessing outside a lifetime is undefined; merely retaining allocated memory forever is defined (if undesirable). *Tempting wrong answer:* "both are UB because both are memory bugs" — a leak is a defect but not UB. **Feed-forward:** §9.3.

9.7 (a) A heap allocation may search a free list, split a block, update bookkeeping, and sometimes call the OS for more memory — real work — whereas a stack allocation is a single stack-pointer adjustment (Ch 8). (b) **External fragmentation** is free space chopped into many small, non-adjacent pieces by interleaved allocate/free of varying sizes; a large request can fail because *no single free block* is big enough, even though the *total* free memory exceeds the request. (c) **Not undefined behavior** — fragmentation is a *performance/capacity* problem, not a language-rule violation. *Tempting wrong answer:* "fragmentation corrupts memory" — it does not; it just wastes/scatters it. **Feed-forward:** §9.2.

9.8 **Step 4:** Correct allocation size: `malloc(strlen(s) + 1)` — room for the string *and* its terminating `'\0'`. Missing check: after `malloc`, verify it did not return `NULL` before writing (`if (!r) return NULL;`). Corrected function: `char *dup(const char *s){ char *r = malloc(strlen(s)+1); if(!r) return NULL; strcpy(r,s); return r; }`. The tool from §9.4 that catches the original one-byte overflow is **AddressSanitizer** (`-fsanitize=address`), which would report a `heap-buffer-overflow` on the `strcpy` line (Valgrind would too). *Tempting wrong answer:* "`strlen` already counts the null terminator" — it does not; it counts the characters before it. **Feed-forward:** §9.3–9.4.

9.9 (a) **Leak** — steadily growing memory until the OOM killer acts is the leak signature. (b) **Uninitialized read** — a different value each run, only in release builds, is the classic "garbage bytes" symptom (build-dependent). (c) **Buffer overflow** — an out-of-bounds write corrupting an adjacent variable. (d) **Use-after-free / double-free** — crashing inside `free` with heap corruption after releasing the same pointer twice is a double-free. (e) **Non-memory performance issue** (a TLB/huge-page effect, Ch 4) — *not* a memory *bug*; the trap in this item. *Method-selection cue:* "unbounded growth" → leak; "differs per run/release" → uninitialized; "adjacent corruption" → overflow; "crash in free after two frees" → double-free; "huge pages help" → TLB, not a bug. **Feed-forward:** §9.3, Ch 4.

9.10 (a) **Well-defined and fine** — `free(NULL)` is a guaranteed no-op. (b) **A defined-but-real defect** — a memory leak; wasteful but not UB. (c) **Undefined behavior** — `a[n]` on an `n`-element array is one past the last valid index (valid indices 0..n-1); dereferencing it is out of bounds. (d) **Well-defined and fine** — a `calloc`'d block is zeroed, so reading it yields 0, a defined value. (e) **Reads an indeterminate value (at best unspecified, in general undefined behavior)** — a `malloc`'d block is uninitialized, so reading before writing uses an *indeterminate value*: on this ABI an `int` has no trap representation, so you get an unspecified (garbage) value rather than a crash, but the standard permits it to be full UB in general (matching §9.3), so never rely on it. *Tempting wrong answer:* conflating (d) and (e) — `calloc` zeroes, `malloc` does not. **Feed-forward:** §9.1, §9.3.

9.11 (a) The three obligations: use a pointer only within its target's **lifetime**; access only within the allocation's **bounds**; and **free each block exactly once** (not zero, not twice). (b) **Use-after-free** violates *lifetime*; **double-free** violates *free-exactly-once*; a **leak** also violates *free-exactly-once* (frees zero times) — while use-after-free additionally implies the free happened too early. (c) A compiler that checks these before the program runs would **reject**, at compile time, any program that could use a pointer past its target's lifetime, index out of bounds, or mismanage a free — turning a whole class of runtime UB into type errors. That is more reliable than a runtime sanitizer because a sanitizer only catches a bug on *inputs/paths you actually execute* during testing, whereas a static guarantee holds for *all* executions — which is exactly Rust's ownership/borrowing promise. *Tempting wrong answer:* "a sanitizer is just as good" — it is path-dependent, not a proof. **Feed-forward:** §9.5 and Part 5 (Rust).

9.12 Machine-specific by design. A correct write-up reports, for each of the four bugs, ASan's exact error type and the source line — on the book's machine (gcc 11, x86-64): `heap-use-after-free` (a READ on the line reading the freed block), `heap-buffer-overflow` (a WRITE "0 bytes to the right of" the allocated region), `attempting double-free`, and (via LeakSanitizer) `Direct leak of N byte(s)` pinned to the allocation line. The fifth program (uninitialized read) should show that ASan does **not** flag it — reach for Valgrind or MemorySanitizer, or catch the simple case with `gcc -Wall` (`-Wuninitialized`). Essential discipline: label compiler/tool versions, report only observed output (addresses and wording vary), and note that ASan detects addressability bugs, not uninitialized-value bugs. *Tempting wrong answer:* "ASan catches everything" — it misses uninitialized reads. **Feed-forward:** §9.4.

Chapter 10 — Solutions

10.1 **Undefined behavior:** the standard imposes *no requirements* (C §3.4.3) — the program has no meaning on the triggering input. **Unspecified behavior:** the standard offers two or more valid possibilities and does not say which you get (e.g. argument evaluation order). **Implementation- defined:** unspecified but the implementation must *document* its choice (e.g. `sizeof(int)`). Only **undefined** behavior grants the optimizer license to assume the situation never happens. **Feed-forward:** §10.1.

10.2 **Unsigned** integer overflow is *not* UB — it is defined to wrap modulo 2^n. **Signed** integer overflow *is* undefined behavior. The defined case (unsigned) yields the mathematically correct value reduced modulo 2^n (i.e. the low n bits). *Tempting wrong answer:* "both wrap, it's two's complement" — the hardware may wrap, but the *language* leaves signed overflow undefined, which is what the optimizer exploits. **Feed-forward:** §10.3.

10.3 Most likely explanation: the program contains **undefined behavior** that was harmless-looking at `-O0` but that the optimizer exploited at `-O2` (it assumed the UB never happens and transformed the code). The two flags to hunt it: `-fsanitize=undefined` (UBSan) and `-fsanitize=address` (ASan), ideally with `-Wall -Wextra`. *Tempting wrong answer:* "the optimizer has a bug" — far more often the UB was already there. **Feed-forward:** §10.2, §10.4.

10.4 Signed overflow is UB, so the compiler may assume `x + 1` never overflows; if it never overflows, `x + 1 > x` is always true, so the function folds to `return 1` (on this machine, `movl $1, %eax; ret`). What breaks: a programmer using `if (x + 1 < x)` to detect overflow relies on the overflow *happening* and wrapping — but that is UB, so the compiler assumes it cannot happen and deletes the check. Correct detection: compile with `-fwrapv` (defines signed overflow as wraparound), or do the arithmetic in an unsigned type, or use `__builtin_add_overflow`, which reports overflow without invoking UB. *Tempting wrong answer:* "test after the add" — testing *after* signed overflow is testing after UB. **Feed-forward:** §10.3–10.4.

10.5 (a) The UB is a **null-pointer dereference**: `t->sk` dereferences `t` before it is checked. (b) Because dereferencing null is UB, the compiler may assume `t` was non-null at the dereference, so `if (!t)` is provably false and is deleted as dead code. (c) Fix: move the check before the dereference — `if (!t) return -1; int *sk = t->sk; return *sk;`. (d) The flag that preserves the check without moving code: `-fno-delete-null-pointer-checks`. The pattern is the real Linux kernel bug **CVE-2009-1897** (`tun_chr_poll`). *Tempting wrong answer:* "the check is there, so it's fine" — it is there in the source, not in the binary. **Feed-forward:** §10.3.

10.6 The compiler is behaving correctly under the **as-if rule**: it must preserve the observable behavior of a *well-defined* program, but a program that dereferences a possibly-null pointer has undefined behavior on the null input, so there is no observable behavior it must preserve there. Given the dereference `t->sk`, the compiler infers `t != NULL` and legally removes the redundant `if (!t)`. The code, not the compiler, is wrong: the check comes after the dereference it is meant to guard. *Tempting wrong answer:* "the source clearly has the check, so the compiler dropped correct code" — to the optimizer, a check after a dereference is a statement the pointer was already non-null. **Feed-forward:** §10.2.

10.7 (a) Mandating checks would insert a bounds check on every array access, an overflow check on every signed add, and a null check on every dereference — runtime cost on the hot path of essentially every program. (b) One optimization signed-overflow UB enables: treating a loop counter as unable to wrap, so the compiler can prove loop termination, promote the counter to a wider type, or strength-reduce — none guaranteed if overflow were defined to wrap. (c) The bargain: the programmer gives up "defined behavior in the bad case" (and takes on the duty never to trigger UB); the compiler gains the freedom to generate fast code across all hardware without pervasive safety checks. *Tempting wrong answer:* "UB is just a mistake in the standard" — it is a deliberate performance/portability choice. **Feed-forward:** §10.4.

10.8 **Step 4:** Fix: the bounds check must come *before* the access — check `if (i < 0 || i >= n) return;` (or handle the error) *then* touch `a[i]`, including for logging. A sanitizer that would have flagged the original out-of-bounds access at runtime: **AddressSanitizer** (`-fsanitize=address`) for a heap/stack array, or UBSan's bounds check for a known-size array. Moving the log line is not "just reordering output": once `a[i]` executes, the compiler may *prove* `0 <= i < n` (else UB) and delete the later check — so the position of the access changes what the compiler is allowed to assume about `i`, not merely when a byte is printed. *Tempting wrong answer:* "it's just a logging line, order doesn't matter" — the access is what licenses the deletion. **Feed-forward:** §10.2–10.3.

10.9 (a) **Implementation-defined** — `sizeof(int)` varies but must be documented. (b) **Unspecified** — the order of evaluating `a()` and `b()` is one of two valid, undocumented choices. (c) **Undefined** — `INT_MAX + 1` is signed overflow. (d) **Implementation-defined** — right-shift of a negative signed integer is implementation-defined (the standard lets the implementation choose arithmetic vs. logical shift and document it). (e) **Undefined behavior** (in general) — reading a `malloc`'d block before writing it uses an indeterminate value (the §9.3 memory bug). *Method-selection cue:* "varies but documented" → impl-defined; "two valid orders" → unspecified; "no meaning at all" → undefined. **Feed-forward:** §10.1, §9.3.

10.10 Least to most effective: (a) `-O0` — *hides* the bug (the UB remains; a future build or a different path re-exposes it); (b) `volatile` — also *hides* it (it defeats some optimizer assumptions but does not remove the UB, and is fragile and slow); (c) `-fsanitize=undefined,address` plus fixing what it reports — *removes* the bug by finding and eliminating the actual UB. So the order is (a) < (b) < (c), and only (c) removes rather than hides. *Tempting wrong answer:* "compile at `-O0` in production to be safe" — you keep the bug and lose performance. **Feed-forward:** §10.4.

10.11 (a) The standard says a program that executes UB has undefined behavior for that *execution* — not just from the buggy line onward — so the compiler need not preserve observable effects that occur before the UB either; it may hoist, sink, or delete surrounding work ("UB can time-travel," in Regehr's phrasing). (b) Concrete scenario: an upstream `t->sk` lets the compiler delete a downstream `if (!t) return ERR;`; a caller that would have safely returned an error on a null `t` now falls through into code that trusts `t`, and an attacker who maps memory at address 0 reaches it — a would-be crash becomes a silent privilege escalation (the CVE-2009-1897 shape). (c) A static guarantee is stronger because a runtime sanitizer only catches UB on the *inputs and paths you actually execute* during testing, whereas "the safe subset has no UB" holds for *all* executions — a proof, not a spot check. *Tempting wrong answer:* "sanitizers make UB safe" — they detect it on exercised paths only. **Feed-forward:** §10.2, §10.5, Part 5.

10.12 Machine-specific by design. A correct write-up reports: at `-O2`, `x + 1 > x` folds to `return 1` (on the book's machine, `movl $1, %eax; ret`), and adding `-fwrapv` makes the addition and comparison reappear; for `poll`, the default `-O2` omits the `testq %rdi,%rdi` null check while `-fno-delete-null-pointer-checks` restores it. The signed-overflow program should be flagged by `-fsanitize=undefined` as a runtime error (`signed integer overflow`). Essential discipline: report only observed assembly/output and label the compiler/version (gcc 11.4.0, x86-64 here), since exact instructions and wording vary. *Tempting wrong answer:* "the assembly will be the same as mine" — it depends on compiler and version. **Feed-forward:** §10.3–10.4.

Chapter 11 — Solutions

11.1 (a) A single **null byte** `'\0'` (value 0) marks the end. (b) `strlen` is O(n) because the length is *not* stored — it must scan byte by byte until it finds the terminator. (c) The function learns **nothing** about capacity: the array decays to a bare `char *`, an address with no length attached. **Feed-forward:** §11.1.

11.2 `gets` — it read a line with no bound at all and was **removed from the language in C11** (deprecated in C99). Use `fgets(s, size, stdin)`, which takes the buffer size. *Tempting wrong answer:* "`scanf("%s")` is fine" — unbounded `%s` has the same overflow problem; use a width, e.g. `%63s`. **Feed-forward:** §11.3.

11.3 (a) **False** — `strncpy` does not terminate when the source is at least `n` bytes long; it leaves the destination unterminated. (b) **False** — a canary *detects* a linear stack overflow and aborts; the bug (the out-of-bounds write) is still there. (c) **True** — `snprintf` always null-terminates when its size argument is nonzero. **Feed-forward:** §11.2–11.4.

11.4 On a downward-growing stack, a local array sits *below* the saved frame pointer and saved **return address**, while a copy into the array writes toward *higher* addresses — so an overrun climbs over the frame pointer onto the return address. Overwriting it means that when the function executes `ret`, the CPU jumps to the attacker-supplied value: control-flow hijack. An out-of-bounds *read* (usually) cannot redirect execution — it leaks adjacent bytes but does not write control data. *Tempting wrong answer:* "reads are as dangerous as writes" — reads leak (serious, e.g. Heartbleed) but do not seize control. **Feed-forward:** §11.2.

11.5 (a) **Overflows** whenever `user_input` is longer than `buf` — no destination bound; rewrite `snprintf(buf, sizeof buf, "%s", user_input);`. (b) **Overflows** if the formatted result exceeds `buf` — `sprintf` is unbounded; rewrite `snprintf(buf, sizeof buf, "%s-%s", a, b);`. (c) **Over-reads** — `strncpy(d, "abcd", 4)` fills all 4 bytes with no terminator, so `printf("%s", d)` runs off the end; fix by terminating (`d[3] = '\0';`, losing a char) or, better, `snprintf(d, sizeof d, "%s", "abcd")`. *Method-selection cue:* "no size argument" → can overflow; "size argument but no guaranteed terminator" → can over-read. **Feed-forward:** §11.2–11.3.

11.6 Three reasons a canary + ASLR do not "handle" overflows: (1) a **canary only catches a linear stack overflow and only at return** — it does nothing for heap overflows or overwrites of non-control data, and it acts after the corruption (turning a hijack into a crash/DoS); (2) **ASLR only randomizes addresses** — it raises the difficulty of a working exploit but is defeated by info-leaks or techniques that do not need fixed addresses; (3) both are **mitigations, not fixes** — the out-of-bounds write is still undefined behavior, still present. The actual fix: remove the unbounded writes (use `snprintf`/bounded copies) so the overflow cannot occur. *Tempting wrong answer:* "mitigations on = bug fixed" — they contain blast radius, they do not remove the bug. **Feed-forward:** §11.4.

11.7 (a) `strlen` is O(n) because a C string stores no length — it scans to the terminator — whereas a length-carrying string stores the count, so reading the length is O(1). (b) Stack canaries add a random-value write on entry and a compare on return per protected frame; `_FORTIFY_SOURCE` adds size-checked variants of the string/memory calls — small overhead in exchange for turning many overflows into detected aborts or compile errors. (c) Not free: the canary compare is on the return path of every protected function, a real (if usually small) cost. Trade-off: **safety vs. speed** — you pay a little runtime to detect a class of memory corruption. *Tempting wrong answer:* "canaries are free" — they add work on every protected return. **Feed-forward:** §11.1, §11.4.

11.8 **Step 4:** Rewrite with a single bounded format: `void greet(const char *name){ char msg[16]; snprintf(msg, sizeof msg, "Hello, %s", name); puts(msg); }`. `snprintf` writes at most `sizeof msg` bytes *including* the terminator and always null-terminates (when the size is nonzero), so no `name`, however long, can overflow `msg` — it truncates instead; `strcat` guarantees neither a bound nor (relative to the destination capacity) safety. A §11.4 defense that turns the original overflow into a crash rather than a silent hijack: a **stack canary** (`-fstack-protector`), which aborts with "stack smashing detected" instead of returning through a corrupted return address (`_FORTIFY_SOURCE` would also catch it, at compile or run time). *Tempting wrong answer:* "use `strncat` with `sizeof msg`" — `strncat`'s bound is the bytes to *append*, not the buffer size, and it is easy to miscompute. **Feed-forward:** §11.3–11.4.

11.9 (a) **Buffer overflow (over-write)** is possible, but "garbage appended *after* the expected text" most cleanly matches an **unterminated string (over-read)** — the print continues past the intended end until it hits a stray zero. (b) **A mitigation firing** — `*** stack smashing detected ***` is the stack canary aborting on a detected overflow. (c) **Use-after-free** — reading a freed block that was reallocated to another request (Ch 9). (d) **Unterminated string (over- read)** — `printf("%s")` reading far past the data and leaking adjacent memory (Heartbleed class). *Method-selection cue:* "garbage after expected text / reads too far" → missing terminator; "stack smashing detected" → canary; "freed block reused" → UAF. **Feed-forward:** §11.1–11.4, §9.3.

11.10 (a) **Undefined behavior** — `strncpy` with `strlen(s) >= sizeof d` leaves `d` unterminated, so reading it as a string over-reads out of bounds. (b) **Well-defined and fine** — `snprintf` always terminates, so reading `d` is safe (possibly truncated, but valid). (c) **Undefined behavior** — a one-byte heap overflow is an out-of-bounds write. (d) **Well-defined** (a defined defensive action) — a canary deliberately aborting the process on a detected overflow is defined behavior, not UB (it is the mitigation working). *Tempting wrong answer:* conflating (a) and (b) — the difference is the guaranteed terminator. **Feed-forward:** §11.2–11.4.

11.11 (a) A **stack canary** defeats a linear overflow that reaches the return address (detected at return); **NX/DEP** marks the stack/heap non-executable, defeating "inject shellcode and jump to it"; **ASLR** randomizes segment base addresses, defeating attacks that need to predict them. **Return-oriented programming (ROP)** was developed to route around NX by chaining fragments of existing executable code ("gadgets") instead of injecting new code. (b) Layering is worthwhile because each mitigation raises the cost and reliability barrier for a different technique; an attacker must defeat all of them, and many real exploits fail against the stack. (c) Rust makes the bug "not happen" by carrying the length with every slice/string and bounds-checking indexing: an out-of-bounds access is a defined **panic**, not undefined memory corruption — categorically different because a panic has a defined, contained outcome (the program stops safely) whereas UB has no defined outcome and can corrupt control flow. *Tempting wrong answer:* "a panic is just a crash, same as UB" — a panic is defined and cannot be leveraged into memory corruption. **Feed-forward:** §11.4–11.5, Part 5.

11.12 Machine-specific by design. A correct write-up reports: (a) with `-fstack-protector-all` the 8-byte overflow aborts with `*** stack smashing detected ***` (on the book's machine, exit status 134, SIGABRT); (b) with `-D_FORTIFY_SOURCE=2 -O2` a compile-time diagnostic like `__builtin___strcpy_chk / __memcpy_chk writing N bytes into a region of size M` (and gcc's `-Wstringop-overflow` may also fire); (c) `strncpy(d, "abcd", 4)` leaves the four bytes `97 98 99 100` with *no* terminating zero. Essential discipline: report only observed output and label compiler/version (gcc 11.4.0, x86-64 here), since exact messages and status vary. *Tempting wrong answer:* "the canary means the overflow was harmless" — it converted a hijack into a crash; the bug is the same. **Feed-forward:** §11.2–11.4.

Chapter 12 — Solutions

12.1 (a) An arena exploits **shared lifetime** — many objects that die together. (b) A pool exploits **shared size** — many objects of one fixed size. (c) The **pool** can free an individual object (push its slot back on the free list); the arena cannot. **Feed-forward:** §12.1–12.3.

12.2 `arena_alloc` is **O(1)** (round up the offset, bump it, return the address); `arena_reset` is **O(1)** (set the offset to 0). The one thing an arena cannot do that `malloc` can: **free an individual object** — it frees only the whole region at once. *Tempting wrong answer:* "reset is O(n) because it frees n objects" — it frees them all with a single offset assignment, touching none of them. **Feed-forward:** §12.2.

12.3 The "next free slot" pointer is stored **inside the free slot itself** (its first bytes). It costs no extra memory because a free slot holds no live object — its space is unused — so the free list is threaded through the slots rather than kept in a separate structure. *Tempting wrong answer:* "you need a separate array of next-pointers" — unnecessary; the free slots' own bytes hold them. **Feed-forward:** §12.3.

12.4 `typedef struct { unsigned char *base; size_t cap, off; } Arena;` with `void *arena_alloc(Arena *a, size_t n){ size_t p = (a->off + 15) & ~(size_t)15; if (p + n > a->cap) return NULL; a->off = p + n; return a->base + p; }` and `void arena_reset(Arena *a){ a->off = 0; }`. `arena_alloc` is O(1) because it does a fixed amount of work — align, bounds-check, bump — with no search or coalescing. `arena_reset` frees everything without touching individual objects because "free" here just means "the offset no longer points past them," so resetting the offset makes all of that space available again in one step. *Tempting wrong answer:* forgetting the alignment round-up — unaligned returns are UB for some types (Ch 7). **Feed-forward:** §12.2.

12.5 `void *pool_alloc(Pool *p){ if (!p->free_list) return NULL; void *slot = p->free_list; p->free_list = *(void **)slot; return slot; }` and `void pool_free(Pool *p, void *slot) { *(void **)slot = p->free_list; p->free_list = slot; }`. Both are O(1): allocation pops the free-list head and freeing pushes onto it — a couple of pointer writes, no search. There is no external fragmentation within the size class because every slot is identical and interchangeable, so any free slot satisfies any request — free space is never "too scattered to fit." *Tempting wrong answer:* "you still need to coalesce" — with one fixed size there is nothing to coalesce. **Feed-forward:** §12.3.

12.6 Reason 1 (inflexibility): an arena cannot free individual objects, so any object that outlives its batch pins the *whole* region until reset — used "everywhere," including for long-lived mixed-lifetime state, it can inflate memory badly. Reason 2 (evidence): Berger, Zorn & McKinley (OOPSLA 2002) found a good general- purpose allocator matched or beat hand-written custom allocators for most programs; the arena/region case was the exception, not the rule. Correct procedure: **profile first** to confirm allocation is the bottleneck, then apply an arena only where the shared-lifetime pattern fits (e.g. per-request scratch), keeping `malloc` elsewhere. *Tempting wrong answer:* "arenas are always faster, so use them everywhere" — faster only for the right pattern, and costly in memory otherwise. **Feed-forward:** §12.1, §12.4.

12.7 (a) The arena skips, per allocation: the **free-list search** (it just bumps an offset) and the **per-block header / coalescing bookkeeping** (it keeps none) — both of which the general allocator does (Ch 9). (b) The ~4–5× measurement used "N objects live at once, then freed": the arena frees all N with one reset while `malloc` pays N individual `free` calls, so the comparison rewards the arena's bulk free. A pattern that narrows or erases the gap: freeing each object immediately after allocating it, where glibc's `malloc` reuses one hot block and the per-object cost nearly vanishes (as the chapter's first, discarded measurement showed). (c) Memory risk: because it never frees individual objects, an arena holding a long-lived object pins the entire region — it bites when object lifetimes are mixed rather than shared. *Tempting wrong answer:* "the arena is always ~5× faster" — only for its pattern. **Feed-forward:** §12.2, §12.4.

12.8 **Step 4:** The bug is a **use-after-free** (a use-after-reset): the cached pointer dangles into arena storage whose lifetime `arena_reset` ended, and the next request reuses those bytes under it. It **is undefined behavior** (Ch 10). ASan/Valgrind will likely *not* catch it by default because they instrument `malloc/free`, not *your* arena's reset — to them the arena's backing block is still one big live allocation, so a read into it looks legal (you would have to add ASan's manual poisoning hooks for the tool to see the reset). Fix: do not let the cache point into the arena — **copy** the string into memory with a lifetime that matches the cache (e.g. `strdup/malloc` owned by the cache), so nothing the cache holds lives in per-request arena storage. *Tempting wrong answer:* "the sanitizer would have caught it" — not for a custom allocator's reset, by default. **Feed-forward:** §12.4–12.5, §9.3–9.4.

12.9 (a) **Arena** — many small objects sharing one lifetime (the request), freed together at response time. (b) **Pool** — constant create/destroy of same-size nodes throughout the run; O(1) both ways, no fragmentation, individual frees. (c) **General `malloc`** — a library cannot predict its callers' patterns, so the general allocator is the safe default. (d) **Arena** — a burst of AST nodes freed together when the pass ends (the classic region/compiler-pass case Berger et al. highlighted). *Method-selection cue:* "share a lifetime, die together" → arena; "same size, repeated alloc/free" → pool; "unknown pattern" → general. **Feed-forward:** §12.1, §12.4.

12.10 (a) **Undefined behavior** — reading through a pointer into an arena after `arena_reset` is a use-after-reset (a use-after-free). (b) **A defined-but-real defect** — a long-lived object pinning the whole region wastes memory but is not a language-rule violation (analogous to a leak: defined, undesirable). (c) **Undefined behavior** — `pool_free`-ing the same slot twice corrupts the free list (a double-free in your allocator). (d) **Well-defined and fine** — resetting an arena with no live pointers into it is exactly how it is meant to be used. *Tempting wrong answer:* calling (b) UB — over-retaining memory is a defect, not undefined behavior. **Feed-forward:** §12.2–12.5, §9.3.

12.11 (a) In C the programmer must guarantee, by hand, that no pointer is used outside its target's lifetime, no access falls outside its bounds, and every block is freed exactly once — with no undefined behavior triggered. (b) A custom allocator *relocates* the obligation: it does not free individual objects for you (arena) or it recycles slots you must not double-free (pool), so *you* — the allocator's author and its caller — now enforce "free exactly once" and "no use after the lifetime ends" for arena/pool memory, and the system allocator's tooling no longer helps. (c) A language that ties an allocation's lifetime to a scope (Rust's ownership/borrowing) can make "use after reset" a **compile-time error** — the borrow checker refuses to let a reference outlive the region it points into — which is stronger than a runtime sanitizer because it holds for all executions and needs no test input to trigger, and because the sanitizer does not even understand your custom allocator's reset. *Tempting wrong answer:* "custom allocators remove the obligation" — they move it onto you. **Feed-forward:** §12.5, Part 5.

12.12 Machine-specific by design. A correct write-up reports: (a) three allocations return independent, 16-byte- aligned pointers with the expected values, and `arena_reset` sets the offset back to 0; (b) for large *N* with all objects live then freed, the arena (one bulk free) beats `malloc` (N frees) by a several-fold ratio — on the book's machine roughly 4–5×, varying run-to-run; (c) when each object is freed immediately after allocation, the arena's advantage shrinks or disappears, because the general allocator reuses one hot block and per-object cost nearly vanishes — showing the win depends entirely on the lifetime pattern. Essential discipline: report only measured numbers and label compiler/hardware (gcc 11.4.0, x86-64 here), since figures vary. *Tempting wrong answer:* "the arena is faster in every test" — not for the free-immediately pattern. **Feed-forward:** §12.2, §12.4.

Chapter 13 — Solutions

13.1 The two modes are **user mode** (ring 3, where your code runs) and **kernel mode** (ring 0, where the OS runs). A user-mode program may not, for example, execute privileged instructions such as halting the CPU, disabling interrupts, loading the page tables (Ch 4), or accessing device registers directly. *Tempting wrong answer:* "user mode is just slower than kernel mode" — the difference is authority, not speed; the same instruction runs at the same rate, but privileged ones simply trap in user mode. **Feed-forward:** §13.1.

13.2 On x86-64 Linux the system-call number goes in **`rax`**, and the return value comes back in **`rax`** as well (a negative small value being `-errno`). *Tempting wrong answer:* "the fourth argument goes in `rcx`" — it goes in `r10`, because the `syscall` instruction clobbers `rcx` (and `r11`). **Feed-forward:** §13.2.

13.3 No — `printf` is **not** a system call; it is a C library function that parses the format string, converts values to text, and **buffers** the result in user-space memory. It eventually produces output through the **`write`** system call, but only when the buffer flushes (fills, hits a newline for a line-buffered terminal, or the program exits normally). *Tempting wrong answer:* "every output function is a system call" — most output is buffered in user space precisely to avoid crossing the boundary per call. **Feed-forward:** §13.3.

13.4 The C library's `write` wrapper puts the `write` system-call number in `rax`, the file descriptor `1` in `rdi`, the buffer's address in `rsi`, and the length `5` in `rdx`; it executes the `syscall` instruction, which switches to kernel mode and jumps to the fixed kernel entry point; the kernel validates the arguments, does the I/O, and returns the number of bytes written (5 on success) in `rax`. *Tempting wrong answer:* "the program jumps directly into the kernel function" — it cannot choose the target; the trap lands only at the kernel's fixed entry point. **Feed-forward:** §13.2.

13.5 The raw kernel interface signals failure by returning a **negative error code** (e.g. `-EBADF`) in `rax`. The C library wrapper detects that, stores the code in the global **`errno`**, and returns **`-1`** to the caller. So a raw `-EBADF` becomes: the wrapper returns `-1` and sets `errno == EBADF`. *Tempting wrong answer:* "`errno` is set by the kernel" — the kernel returns a negative number; the library *translates* it into `errno` and `-1`. **Feed-forward:** §13.3.

13.6 The thousands of `read(3, ..., 1) = 1` lines mean the program is reading the file **one byte per system call** — one boundary crossing per byte, which is the most expensive way possible. The fix is to **read in large blocks** (a big buffer, or buffered I/O such as `fread`/a `FILE*`), turning thousands of crossings into a handful. *Tempting wrong answer:* "nothing is wrong; it's reading the file" — it is, but at ~100× the necessary cost. **Feed-forward:** §13.4.

13.7 (a) A system call must **switch privilege mode** (save/restore state, enter and leave the kernel) and it **disturbs caches/TLB** and runs argument validation — none of which a plain function call does. (b) Reading 1 MB one byte at a time is ~1,000,000 boundary crossings, and each costs ~100× a function call; buffering (read a large block, or use a `FILE*`) cuts it to a handful of crossings. (c) `clock_gettime` is served by the **vDSO** — a page the kernel maps read-only into every process — so it runs entirely in user mode with no trap, and the "call" makes no system call at all. *Tempting wrong answer:* "reading byte-by-byte is slow because the disk is slow" — the dominant cost here is the boundary crossings, not the device (the data may already be cached). **Feed-forward:** §13.4.

13.8 **Step 4:** When the program died, the diagnostic bytes were still in the **C library's user-space output buffer** — they had never crossed the boundary, because `printf` buffers and only calls `write` on a flush. The abnormal termination skipped the normal exit-time flush, so the line was lost: it never reached the terminal. Two ways to guarantee it appears before a possible crash: call **`fflush(stdout)`** immediately after the `printf` (force the crossing), or use **unbuffered output** — write with the `write` system call directly, or set the stream unbuffered / line-buffered. *Tempting wrong answer:* "the `printf` never executed" — it did; its output was simply stranded in the buffer. **Feed-forward:** §13.3, §13.4.

13.9 (a) **Neither / user space** — an integer add is a plain instruction. (b) **Boundary crossing** — opening a file needs the kernel (`openat`). (c) **User space** — sorting an in-memory array is pure computation. (d) **Boundary crossing** — sending a packet touches the network device, so it needs the kernel. (e) **User space** — `sprintf` formats into a memory buffer; no I/O, no crossing (contrast `printf`, which will eventually `write`). *Method-selection cue:* touch a device, another process, or kernel-managed state → crossing; compute in your own memory → user space. **Feed-forward:** §13.2–13.4.

13.10 (a) **Undefined behavior** — reading one past a stack array is an out-of-bounds access (Ch 11). (b) **A boundary crossing** — `read` on a valid descriptor is a system call. (c) **Undefined behavior** — signed integer overflow is UB (Ch 10). (d) **Neither** — a buffered `printf` that has not flushed is well-defined; the bytes are simply still in user space and no crossing has happened yet. *Tempting wrong answer:* calling (d) a system call — the crossing happens only at the flush, not at the `printf`. **Feed-forward:** §13.3, and Ch 10–11.

13.11 (a) A process's pointers are **virtual addresses translated through its own page tables** (Ch 4); a pointer value that names another process's or the kernel's memory does not map to anything in *this* process's address space, so the kernel's validation rejects it (or it simply is not this process's memory to read). (b) Without validation, a process could pass the kernel a pointer into *kernel* memory as the "buffer" for a `read`, tricking privileged code into copying secrets out, or into *another* process's memory — a confused-deputy attack; validation is what keeps the kernel from acting on addresses the caller has no right to. (c) Per-process address translation confines each process to its own memory (it cannot even name another's), and the user/kernel mode bit prevents it from rewriting the translation tables or reading devices to get around that — together, one process can neither address nor reach another's memory. *Tempting wrong answer:* "the kernel trusts the pointer because the process passed it" — the kernel must never trust a user pointer; validating it is essential to isolation. **Feed-forward:** §13.1–13.2, Ch 4.

13.12 Machine-specific by design. A correct write-up reports: (a) `strace` shows the `write` call with its arguments and return value (e.g. `write(1, "...", n) = n`); (b) a trivial hello-world makes a couple dozen system calls, most of them process startup (dynamic linker `mmap`/`openat`/ `mprotect`, `brk`, etc.) with only one or two being the program's own output; (c) the `getpid` system call costs on the order of ~100× a trivial function call (on the book's machine ~150–190 ns vs. ~1–2 ns), varying run to run and with mitigations; (d) one million `clock_gettime` calls produce *zero* `clock_gettime` system calls under `strace -c`, because the vDSO answers in user space. Essential discipline: report only measured numbers and label compiler/kernel/ hardware, since figures vary. *Tempting wrong answer:* "the syscall and the function call cost about the same" — the whole point is that they differ by roughly two orders of magnitude. **Feed-forward:** §13.2, §13.4.

Chapter 14 — Solutions

14.1 A process bundles (1) an **address space** (its private virtual memory — text, data, bss, heap, stack), (2) one or more **threads of execution** (the running context: program counter, stack pointer, registers), and (3) a set of **kernel resources** (open file descriptors, PID/parent PID, credentials, working directory). *Tempting wrong answer:* "a process is just the program's code" — the code on disk is inert; the process is the running instance with memory, execution state, and resources. **Feed-forward:** §14.1.

14.2 `fork` returns the **child's PID in the parent** and **0 in the child** (or `-1` on failure). That difference is how each of the two otherwise-identical processes learns which one it is, so it can branch to different code. *Tempting wrong answer:* "`fork` returns 0 to whichever called it" — both continue from the return; the caller becomes the parent and gets the child's PID, while the new child gets 0. **Feed-forward:** §14.2.

14.3 No — on success `exec` does **not return** (there is no original program to return to). It **replaces the entire process image** — text, data, heap, stack — with a new program, while **keeping the PID** (and the open file descriptors, unless marked close-on-exec). *Tempting wrong answer:* "`exec` starts a new process" — it does not; it transforms the *current* process into a different program, same PID. **Feed-forward:** §14.3.

14.4 `pid_t pid = fork(); if (pid == 0) { /* child */ } else if (pid > 0) { int st; waitpid(pid, &st, 0); /* parent, after child exits */ } else { /* fork failed */ }`. The `pid == 0` branch is the child; the `pid > 0` branch is the parent, which waits for the child; each knows which it is solely from `fork`'s return value. *Tempting wrong answer:* forgetting the parent must `wait` — without it the finished child becomes a zombie (§14.4). **Feed-forward:** §14.2, §14.4.

14.5 At `fork`, the kernel does **not** copy the parent's pages; it maps the same physical pages into both processes and marks them **read-only**. A real copy is triggered only when *either* process **writes** a page: the hardware faults, the kernel makes a private copy of that one page for the writer, and both continue. So the cost is proportional to the number of pages actually modified, not to the size of the address space — forking a 1 GB process is fast because almost none of that gigabyte is written before use. *Tempting wrong answer:* "`fork` duplicates all of memory immediately" — that would make forking large processes prohibitively slow; copy-on-write avoids it. **Feed-forward:** §14.2, Ch 4.

14.6 The shell `fork`s a child; the child `exec`s the requested program (replacing itself with it); the parent (shell) `wait`s for the child to finish before prompting again. The gap between `fork` and `exec` is where the child, still running the shell's code, adjusts its inherited resources — e.g. **redirecting standard output to a file** or wiring its input/output to a **pipe** — so that the program it then `exec`s starts up already plumbed into that environment. *Tempting wrong answer:* "the shell `exec`s the command directly" — then the shell would replace *itself* and never return to the prompt; it forks first so a child does the `exec`. **Feed-forward:** §14.3.

14.7 (a) `fork` is cheap because of **copy-on-write**: it shares the parent's physical pages read-only rather than copying them, so it does work proportional to pages later modified, not to address-space size. (b) A workload where the child promptly **writes to nearly every inherited page** (e.g. it mutates a large data structure it inherited) gets little from COW — almost every page faults and is copied anyway, so you pay close to a full copy. (c) Process-per-request pays **process-creation and context-switch overhead** and needs IPC to share data; thread-per-request pays the **loss of isolation** (a crash or memory bug in one thread takes down all requests in the process) and the burden of synchronizing shared state. *Tempting wrong answer:* "COW makes `fork` free regardless of workload" — a write-heavy child erodes the benefit. **Feed-forward:** §14.2, and Ch 15.

14.8 Step 4: The accumulating state is **zombies** — terminated children whose exit status the kernel is still holding because the parent never collected it. They eventually stop new `fork`s because each zombie occupies a **process-table slot**, and once the table (a finite resource) fills, the kernel cannot allocate a slot for a new process, so `fork` fails. This is a **defined resource leak, not undefined behavior** — the program violates no language rule; it simply fails to clean up (like a memory leak). Fix: the parent must **reap** its children — call `wait`/`waitpid` (e.g. from a `SIGCHLD` handler, or a reaping loop) so each child's status is collected and its slot freed. *Tempting wrong answer:* "the zombies are using up memory/CPU" — a zombie holds essentially no memory or CPU, only a table slot and its exit status; the leak is of slots. **Feed-forward:** §14.4.

14.9 (a) **`fork` then `exec`** — fork a child, have it `exec /bin/ls`, and the parent waits and continues (a lone `exec` would replace your program permanently). (b) **`fork`** alone — you want a second process running the *same* code, which is exactly what fork gives (no `exec` needed). (c) **`fork` then `exec`** — the shell forks and the child execs the typed command, while the shell waits. *Method-selection cue:* same program, new process → `fork`; different program, replace this one → `exec`; new process running a different program → both. **Feed-forward:** §14.2–14.3.

14.10 (a) **A defined-but-real defect** — the expectation is simply wrong: after `fork` the two processes have independent address spaces, so the child's changes are invisible to the parent (not UB, just a mistaken mental model that will produce bugs). (b) **A defined-but-real defect** — never reaping children leaks zombies (a resource leak, defined). (c) **Well-defined and fine (though usually unintended)** — both flushing a buffer copied across `fork` is defined behavior; it prints the line twice, which is a correctness surprise to fix with `fflush` before `fork`, but not UB. (d) **Well-defined and fine** — a child `exec`ing a new program while the parent runs is the normal pattern. *Tempting wrong answer:* calling (a) or (c) undefined behavior — both are defined; the issues are a wrong assumption and a duplicated buffer, not language-rule violations. **Feed-forward:** §14.2–14.4.

14.11 (a) A buffer overflow in process A writes through A's pointers, which are virtual addresses translated by **A's page tables** (Ch 4); they cannot name B's physical memory, and A cannot rewrite the translation tables to escape because installing page tables is a **privileged instruction** that traps in user mode (Ch 13). So the two mechanisms — per-process address translation and the user/kernel mode bit — confine the corruption to A. (b) Because a thread shares its process's address space, a compromised or crashing thread can corrupt the whole process; a separate *process* has its own address space and authority, so untrusted code in it cannot reach the host's memory — making the process the right sandbox unit. (c) Putting each tenant in its own process means one tenant's crash, memory corruption, or compromise cannot read or damage another tenant's data — fault and security isolation come for free from the process boundary. *Tempting wrong answer:* "a thread is isolated enough because it has its own stack" — a private stack does not isolate memory; all threads share the heap and globals and one bug corrupts them all. **Feed-forward:** §14.1, Ch 11, Ch 13, and Ch 15.

14.12 Machine-specific by design. A correct write-up reports: (a) the demo prints two lines with different PIDs, the child's `ppid` equal to the parent's `pid`, and independent values of the variable (e.g. child 101, parent 110) — confirming `fork` returned twice into two separate address spaces; (b) with a `printf`ed line *before* the `fork` and output redirected to a file (block buffering), the line appears **twice**, because `fork` copied the unflushed buffer into both processes and both flushed it at exit; (c) adding `fflush(stdout)` before the `fork` makes the line appear once, because the bytes crossed to the kernel before the buffer was duplicated. Essential discipline: report exactly what you observed and label your compiler/system. *Tempting wrong answer:* "the double line means `fork` ran the `printf` twice" — it ran once; `fork` duplicated the *buffer*, not the print. **Feed-forward:** §14.2, §14.4, and Ch 13.

Chapter 15 — Solutions

15.1 Threads in a process **share** the address space — code (text), the heap, global/static data, and open file descriptors. Each thread keeps **private** its own stack, its own registers (program counter and stack pointer), and its thread-local storage. *Tempting wrong answer:* "each thread has its own heap" — no; the heap is shared, which is exactly why threads can pass data by pointer and also why they race. **Feed-forward:** §15.1.

15.2 `pthread_create(&tid, attr, fn, arg)` takes a pointer to a `pthread_t` to fill in, optional attributes, the function `fn` (of type `void *(*)(void *)`) the new thread will run, and the `arg` passed to it; it starts `fn(arg)` running concurrently. `pthread_join(tid, &ret)` blocks until that thread finishes and collects its return value. *Tempting wrong answer:* "`pthread_create` runs the function and returns its result" — it starts the function on a *new* thread and returns immediately; `pthread_join` is what waits. **Feed-forward:** §15.2.

15.3 Linux uses the **one-to-one (1:1)** model — each `pthread` is its own kernel-scheduled thread (implemented by NPTL). `pthread_create` uses the **`clone`** system call underneath, with flags including `CLONE_VM` to share the address space. *Tempting wrong answer:* "Linux threads are M:N green threads scheduled in user space" — that was never NPTL; Linux threads are 1:1 kernel threads (language runtimes may add their own user-space scheduling above them). **Feed-forward:** §15.3.

15.4 `pthread_t t1, t2; pthread_create(&t1, NULL, worker, NULL); pthread_create(&t2, NULL, worker, NULL); pthread_join(t1, NULL); pthread_join(t2, NULL);` with `void *worker(void *a) { ... return NULL; }`. `pthread_create` is the thread analog of `fork` (start a new flow of control) and `pthread_join` of `wait` (wait for it and collect its result). The joins are necessary because if `main` returned first, the process would exit and tear down the shared address space the workers are still using. *Tempting wrong answer:* skipping the joins "because the threads will finish anyway" — `main` returning ends the process and can kill still-running threads. **Feed-forward:** §15.2, and Ch 14.

15.5 `counter++` compiles to a **read-modify-write**: (1) load `counter` into a register, (2) add one, (3) store it back. With two threads and no synchronization, an interleaving such as — A loads 5, B loads 5, A computes 6, B computes 6, A stores 6, B stores 6 — has both threads increment from the same starting value, so two increments produce a net change of only **one**; one update is lost. Repeated across millions of iterations, many updates vanish. *Tempting wrong answer:* "`++` is a single atomic operation" — in C it is not; it is three separate steps that can interleave. **Feed-forward:** §15.4.

15.6 **Address space:** processes have separate address spaces; threads share one. **Isolation:** processes are isolated (a crash is contained); threads are not (a crash or memory bug kills the whole process). **Cost to create:** a thread is cheaper (no address-space copy / copy-on-write setup); a process is heavier. **Cost of sharing data:** processes need explicit IPC (pipes, shared memory); threads share by pointer for free — but that "free" sharing costs you the obligation to synchronize (data races). *Tempting wrong answer:* "threads are strictly better because they're cheaper" — they trade away isolation, which is sometimes exactly what you need. **Feed-forward:** §15.1, and Ch 14.

15.7 (a) Thread creation skips duplicating the address space — no copy-on-write page setup, no new page tables — because the new thread *shares* the existing address space; it just needs a stack and a kernel-scheduled context. (b) In exchange you take on the **data race** hazard (and the loss of isolation): every piece of shared mutable state must now be synchronized or it is undefined behavior. (c) The loss is **not deterministic** — it depends on how the threads' read-modify-writes happen to interleave, which varies with scheduling, load, core count, and compiler, so the demo gave a different wrong total each run. Its nondeterminism means **tests cannot be trusted to catch it**: a racing program can pass repeatedly (even print the exactly-right answer) and still be broken. *Tempting wrong answer:* "if the test passes, there's no race" — a race is defined by the rules (unsynchronized conflicting access), not by whether a run misbehaved. **Feed-forward:** §15.4.

15.8 **Step 4:** The bug is a **data race** on `hits` — multiple threads performing unsynchronized read-modify-writes (`hits++`) on shared memory — and in C/C++ it is **undefined behavior**. The unit tests missed it because they exercised requests **one at a time**, so no two increments ever overlapped and nothing raced; the bug only manifests under genuine concurrency, which the tests never created. The fix category is **synchronization**: make the update atomic — e.g. protect it with a **mutex** (`pthread_mutex_lock/unlock` around `hits++`) or use an **atomic** increment (a `_Atomic`/ `std::atomic` counter, or an atomic fetch-add). *Tempting wrong answer:* "add more tests" — single-threaded tests, however many, cannot exercise a race; you need concurrency plus a race detector, and the real fix is synchronization. **Feed-forward:** §15.4.

15.9 (a) **Separate process** — isolation is the whole point; an untrusted plugin in its own address space cannot corrupt or crash the host, whereas a thread shares memory and could take everything down. (b) **Thread** — the workers all read one large in-memory array, so shared-address-space threads let them access it directly with no copy or IPC (and reads alone do not race). (c) **Separate process** (via `fork`+`exec`) — running a typed command means launching a different program, which is a process, not a thread. *Method-selection cue:* need containment/a different program → process; need cheap shared-memory work → thread. **Feed-forward:** §15.1, and Ch 14.

15.10 (a) **Undefined behavior** — two threads writing the same non-atomic variable with no synchronization is a data race. (b) **Well-defined and fine** — writing two *different* variables with no shared location is not a race (barring false-sharing performance effects, which are a cost, not UB). (c) **Undefined behavior** — reading through a freed pointer is a use-after-free (Ch 9), regardless of threads. (d) **Well-defined and fine** — concurrent *reads* of a shared value, with no writer, do not race. *Tempting wrong answer:* calling (b) or (d) a race — a data race requires a conflicting access (at least one write) to the *same* location; disjoint writes and read-only sharing are fine. **Feed-forward:** §15.4, and Ch 9.

15.11 (a) The added clause: **no unsynchronized concurrent access to shared mutable memory** (any two threads' accesses to the same location, at least one a write, must be ordered by synchronization). (b) Violating it is a data race, which is **undefined behavior** in C/C++; unlike a use-after-free — which reliably touches freed memory every run — a race depends on interleaving, so it **may never reproduce** under testing even though the program is incorrect for some schedule. (c) A language can move this to compile time with a type system that tracks sharing and mutation: Rust's `Send`/ `Sync` traits mark which data may cross or be shared between threads, and the borrow checker forbids aliasing a mutable reference — so an unsynchronized shared mutation is a *compile error*, not a runtime gamble. *Tempting wrong answer:* "a thorough test suite proves there's no race" — no finite set of runs can, because the offending interleaving may simply never occur; only reasoning from the rules (or a checker/ detector) settles it. **Feed-forward:** §15.4, §15.5, and Part 5 (Rust).

15.12 Machine-specific by design. A correct write-up reports: (a) the four-thread, one-million-each unsynchronized counter prints totals well below 4,000,000 that differ every run (on the book's machine ~1.1–1.8 million across five runs) — lost updates from interleaved read-modify-writes; (b) adding a mutex around the increment (or using an atomic increment) restores exactly 4,000,000 every run; (c) the bug's visibility depends on build and variable: at `-O2` without `volatile` the optimizer may collapse each loop into a single add, shrinking the race window so the "wrong" answer hides (it printed 4,000,000 for the author), while `-O0` or a `volatile` counter forces per-iteration memory writes and exposes the loss — demonstrating that a correct-looking run does not mean the code is correct. Essential discipline: report exactly what you saw and label compiler/core count/ hardware. *Tempting wrong answer:* "the version that printed 4,000,000 is the correct one" — it is the same racy code; it merely failed to manifest. **Feed-forward:** §15.4.

Chapter 16 — Solutions

16.1 The states are **running** (on a CPU now), **ready** (runnable but no free CPU), and **blocked** (waiting for an event — I/O, a child, a lock — and unable to run until it arrives). The scheduler chooses the next runner from the **ready** threads only; a blocked thread is not a candidate until the event wakes it. *Tempting wrong answer:* "the scheduler picks among all threads" — it cannot run a blocked thread; picking one would accomplish nothing until its event fires. **Feed-forward:** §16.1.

16.2 **Turnaround time** = completion − arrival; **response time** = first-run − arrival. For the job: response = 8 − 0 = **8**; turnaround = 40 − 0 = **32**. *Tempting wrong answer:* reporting response as the time it *finished* (40) — response measures when the job first got the CPU, not when it completed. **Feed-forward:** §16.1.

16.3 The **convoy effect**: short jobs pile up behind a long one (like cars behind a truck), so they inherit its runtime and average turnaround balloons. It afflicts **FIFO** (first-come-first-served, no preemption). *Tempting wrong answer:* attributing it to round-robin — RR's small slices are exactly what break the convoy. **Feed-forward:** §16.2.

16.4 **FIFO (X, Y, Z):** X completes at 8, Y at 12, Z at 16 → average turnaround (8+12+16)/3 = **12 ms**. **SJF (Y, Z, X):** Y at 4, Z at 8, X at 16 → average (4+8+16)/3 = **9.33 ms**. SJF is lower, and it is no accident: when all jobs arrive together, running shorter jobs first minimizes the sum of completion times, which is exactly what average turnaround measures — SJF is provably optimal for it. *Tempting wrong answer:* assuming the last-completion time (16 ms in both) decides the average — it is the same, yet the averages differ because ordering changes the *earlier* completions. **Feed-forward:** §16.2.

16.5 SJF minimizes average turnaround because, with simultaneous arrivals, a job's turnaround includes the runtime of everything before it; ordering shortest-first makes each long job's runtime fall on the fewest later jobs (any swap putting a longer job earlier only adds its length to more waits). It is unusable in general because (1) it needs an **oracle** — you must know each job's length in advance, which for interactive and server work you never do — and (2) even given the oracle it gives poor **response time**: a job arriving after a long one starts must wait (or, with STCF, only if shorter). *Tempting wrong answer:* "just estimate the length" — MLFQ does something like this by observing behavior, but a pure SJF that mis-estimates loses its optimality and can starve. **Feed-forward:** §16.2, §16.3.

16.6 An MLFQ keeps several priority queues and always runs from the highest non-empty one (round-robin within a level). A job's priority *adapts*: it enters at the top; if it uses its entire time slice (CPU-bound), it is demoted a level; if it yields before the slice ends (blocks on I/O — interactive), it keeps its level. So interactive jobs, which block frequently, stay near the top and are scheduled quickly (low response time), while CPU-bound jobs sink and share the leftovers — the scheduler *learns* each job's character from behavior rather than being told it. *Tempting wrong answer:* "the programmer sets the priority" — MLFQ derives it from observed CPU/blocking behavior; a boost and anti-gaming accounting keep it honest. **Feed-forward:** §16.4.

16.7 (a) Shrinking the slice improves **response time** (jobs cycle onto the CPU sooner) and hurts **turnaround/throughput** (more interleaving, more context switches). (b) If the slice equals the switch cost, then for every unit of switching the CPU does one unit of useful work — roughly **half** the CPU is wasted on switching (and it only gets worse as the slice drops below the switch cost). (c) A few milliseconds is large enough that a few-microsecond switch is a tiny percentage overhead, yet small enough that interactive jobs still feel instant; seconds would wreck response time, microseconds would drown in switch overhead. *Tempting wrong answer:* "smaller is always more responsive, so make it tiny" — below a threshold, switch overhead dominates and total throughput collapses. **Feed-forward:** §16.3.

16.8 **Step 4:** Lower the dashboard's `nice` value (or, better, put the two workloads in separate **cgroups** and give the dashboard a larger **CPU weight**). Under Linux's proportional-share scheduler this grants the dashboard a bigger *share* of the CPU when the two contend, while the batch job still runs freely whenever the dashboard is idle. It is safer than `SCHED_FIFO` because it changes the *proportion*, not the absolute priority: a low-weight job still gets *some* CPU, so nothing starves and a runaway loop cannot freeze the machine, whereas a CPU-bound `SCHED_FIFO` thread can starve everything on its core, kernel threads included. *Tempting wrong answer:* "give the dashboard real-time priority" — that fixes the starvation by creating a worse one and risks wedging the box. **Feed-forward:** §16.4.

16.9 (a) **Response time** — the editor must react to each keystroke immediately; the user feels lag directly. (b) **Turnaround time** — the batch job's only requirement is that the whole thing finishes by a deadline; no user is watching it start. (c) **Throughput** — a build farm is judged by jobs completed per hour in aggregate, not any one job's latency. *Method-selection cue:* a human waiting on each action → response; a job that must complete → turnaround; a fleet maximizing aggregate work → throughput. **Feed-forward:** §16.1.

16.10 (a) **False** — a process blocked on `read` is off the CPU entirely; it consumes no CPU until the data arrives and it becomes ready. (b) **False** — Linux threads are 1:1 kernel-scheduled entities (Chapter 15), so the scheduler picks among *threads*. (c) **False** — a context switch costs register save/restore plus cold cache/TLB; the slice size trades responsiveness against that overhead. (d) **True** — a lower `nice` value means a larger weight and thus a larger CPU share under contention. *Tempting wrong answer:* calling (d) false because "nice" sounds like it should lower priority — the naming is inverted: *lower* nice is *higher* priority. **Feed-forward:** §16.1, §16.4, and Ch 15.

16.11 (a) Strict SJF always runs the shortest ready job; if short jobs keep arriving, a long job is never the shortest and **never runs** — indefinite starvation. (b) An MLFQ periodically **boosts** every job back to the top priority, guaranteeing even a long, demoted job eventually gets CPU; without the boost, CPU-bound jobs stuck at the bottom could starve behind a steady stream of interactive work. (c) A proportional-share scheduler gives every job CPU in proportion to its weight, so a low-weight job gets a *small* share, never *zero* — "smaller share" is not "no share." That is exactly why `nice` cannot starve a task (it only shrinks the proportion) while `SCHED_FIFO`, being strict priority, can (a higher-priority runnable thread gets *all* the CPU). *Tempting wrong answer:* "a low nice value can starve a process" — no; only strict-priority classes can starve, which is the whole safety argument for using nice/weights instead. **Feed-forward:** §16.4.

16.12 Machine-specific by design. A correct write-up reports: (a) a loop that spins for N wall-seconds and prints the CPU-seconds it accrued; (b) pinned to one shared core, the `nice 0` copy gets far more CPU than the `nice 19` copy — on the book's machine (Linux 6.18, EEVDF) roughly 4.66 vs 0.09 CPU-seconds over a five-second window, about fifty to one; (c) on separate cores the two no longer compete for the same CPU, so each gets a full core and the nice difference nearly vanishes — `nice` only matters *under contention for the same CPU*. The discipline: report exactly what you saw and label kernel/core count/scheduler. *Tempting wrong answer:* "nice 19 was paused/slept" — it was not; it ran continuously but was granted a much smaller *share* of the contended core. **Feed-forward:** §16.4.

Chapter 17 — Solutions

17.1 A **virtual page** is a fixed-size chunk (4 KiB on x86-64) of a process's virtual address space; a **physical frame** is an equal-size chunk of physical RAM. A page-table entry's **present bit** records whether that virtual page currently occupies a frame (present) or does not (not present). *Tempting wrong answer:* "pages and frames are the same thing" — they are the same size, but a page is virtual and a frame is physical; the page table is exactly the mapping between them. **Feed-forward:** §17.1.

17.2 A **page fault** is a hardware exception raised when a process accesses a virtual page whose PTE is not present; it traps into the kernel's fault handler. The two broad outcomes: (1) the access is **legal**, so the handler materializes the page — finds a frame, zeroes it (anonymous) or reads it from a file/swap, sets present, and restarts the instruction; or (2) the access is **illegal**, so the handler delivers `SIGSEGV`. *Tempting wrong answer:* "a page fault means a crash" — most page faults are the normal, successful mechanism of demand paging; only the illegal case becomes a fault the program sees. **Feed-forward:** §17.2.

17.3 A **minor** fault is satisfied without disk I/O — the page comes from a zeroed frame or one already in memory (e.g. the page cache). A **major** fault must read the page in from a file or swap, costing a disk access. *Tempting wrong answer:* "all page faults hit the disk" — anonymous demand-zero and cache-hit faults are minor and never touch the disk. **Feed-forward:** §17.2, §17.3.

17.4 The `mmap` only recorded a 1 GiB mapping in the page table with the present bits clear — no frames spent — so RSS was just the program itself immediately afterward. As the program touched pages, each first touch raised a page fault, the handler gave that page a frame and set present, and RSS climbed one page at a time toward 1 GiB. The allocation did not consume 1 GiB up front because **demand paging** makes allocation a promise; *residency* is paid per touched page. *Tempting wrong answer:* "the OS was slow to account for the memory" — nothing was resident to account for; the frames genuinely did not exist until the touches. **Feed- forward:** §17.2.

17.5 Reference 1 2 3 1 4 2 5, 3 frames. **FIFO** = **5** faults: [1]→[1,2]→[1,2,3]→hit 1→evict 1 for 4 [2,3,4]→hit 2→evict 2 for 5 [3,4,5]. **LRU** = **6** faults: [1]→[1,2]→[1,2,3]→hit 1 (1 now MRU) [2,3,1]→evict LRU 2 for 4 [3,1,4]→evict LRU 3 for 2 [1,4,2]→evict LRU 1 for 5 [4,2,5]. Here **FIFO does better** (5 vs 6) — a deliberate reminder that LRU is a *heuristic* that pays off when the reference stream has locality; on a short string engineered without it, LRU can lose. Only OPT is guaranteed optimal. *Tempting wrong answer:* "LRU always beats FIFO" — LRU usually wins on real (local) workloads, but not on every string; this one is a counterexample. **Feed-forward:** §17.4.

17.6 **Belady's anomaly** is that giving a program more page frames can *increase* its fault count. It is exhibited by **FIFO** (on 1 2 3 4 1 2 5 1 2 3 4 5, FIFO takes 9 faults with 3 frames but 10 with 4). LRU and OPT cannot exhibit it because they are *stack algorithms*: their **resident set with more frames is a superset of the resident set with fewer**, so any page that would have been a hit with N frames is still resident with N+1 — adding frames can only remove faults, never add them. FIFO lacks this property because a larger queue reorders evictions and can eject a soon-to-be-reused page it would have kept. *Tempting wrong answer:* "more memory always means fewer faults" — true for LRU/OPT, false for FIFO; that surprise is the whole point of the anomaly. **Feed-forward:** §17.4.

17.7 (a) A minor fault costs a handful of instructions plus a zeroed or already-resident frame — sub-microsecond; a major fault waits on a disk. Using Chapter 3's hierarchy, DRAM access is tens of nanoseconds while an SSD read is tens of microseconds and a spinning disk seek is milliseconds — so a major fault is roughly a thousand to a million times slower than a minor one. (b) A *clean* page is an unmodified copy of data still on disk (or a file), so the OS can drop it and re-read later; a *dirty* page has been modified, so it must be **written out** to swap before its frame is reused — an extra disk write. (c) The likely cause is paging/swap pressure: the working set no longer fits, so the service takes major faults that stall it for disk-access times. The fix is *not* in the CPU-bound code — it is to shrink the memory footprint or add RAM so the working set fits. *Tempting wrong answer:* "profile and optimize the hot function" — the CPU is not the bottleneck; the cores are idle waiting on disk. **Feed-forward:** §17.3, §17.4, and Ch 3.

17.8 **Step 4:** The phenomenon is **thrashing** — the combined working set exceeds physical memory, so nearly every eviction removes a page another thread is about to need, producing a storm of **major** page faults. CPU utilization is near *zero* because the cores are almost always *blocked waiting on disk I/O* (paging pages in and out), not computing — the machine is I/O-bound on the swap device, not CPU-bound. The fix is to shrink the working set, not "optimize the loop": go back to one (or a few) sequential passes so the active window fits in RAM, or add memory. *Tempting wrong answer:* "add more threads / cores to speed it up" — more concurrent passes enlarge the working set and make the thrashing worse; the original single sequential pass was faster precisely because its working set fit. **Feed-forward:** §17.4.

17.9 (a) **Demand paging / overcommit** — the allocation only reserved mappings; the slowdown is the pages faulting in as they are first touched. Measure: RSS climbing over time (vs the reserved VSZ). (b) **Thrashing** — the dataset just crossed the size where the working set stopped fitting in RAM, so the system is now swapping. Measure: major-fault rate and swap I/O. (c) **Page cache / demand paging of a file mapping** — the first pass takes major faults reading the file from disk; the second pass hits the now-cached pages (minor or no faults). Measure: major faults on first pass vs near-zero on the second. *Method-selection cue:* "allocate huge/fast then slow" → demand paging; "small growth, huge slowdown" → crossed the working-set cliff (thrashing); "slow first, fast second" → cache warming. **Feed-forward:** §17.2, §17.3, §17.4.

17.10 (a) **False** — a successful `malloc` records mappings; physical RAM is consumed only as pages are touched (demand paging/overcommit). (b) **True** — a segmentation fault is a page fault the handler found no legal mapping for, so it delivered `SIGSEGV`. (c) **False** — `fork` uses *copy-on-write*: it copies page tables, not data; frames are shared read-only until a write faults and copies one page. (d) **False** — each process has its own page table, so the same virtual address in two processes normally maps to *different* frames (that is the isolation); they share a frame only through explicit shared memory or a shared file mapping. *Tempting wrong answer:* calling (d) true by analogy to threads — threads share one page table; distinct processes do not. **Feed-forward:** §17.1, §17.2, and Ch 4, Ch 14.

17.11 (a) `fork` copies the parent's *page tables*, mapping the child's pages to the same physical frames as the parent's, but marks every shared writable page **read-only** — so the "copy" is instantaneous and shares all data. (b) When the child (or parent) writes to a read-only shared page, the write raises a **protection page fault**; the handler sees a copy-on-write page, allocates a fresh frame, copies *that one page*'s contents into it, remaps the writer's PTE to the new frame with write permission, and restarts the instruction — exactly **one** page is copied per first write. (c) This makes `fork`-then-`exec` nearly free even for a large parent: `exec` immediately replaces the address space, so almost none of the shared pages are ever written before being discarded — copying them eagerly would be wasted work. A single write to a shared page costs one page fault plus one page copy. *Tempting wrong answer:* "the first write copies the whole address space" — copy-on-write copies only the single page touched, on demand. **Feed-forward:** §17.5, and Ch 14.

17.12 Machine-specific by design. A correct write-up reports: (a) touching one byte per 4 KiB page across the region yields about one minor fault per page (on the book's machine, 25,672 minor faults and 0 major faults to touch 100 MiB), shown via `/usr/bin/time -v`; (b) `VmRSS` is tiny right after a large `mmap` and grows to about the mapping size after touching it all (on the book's machine, 1,364 kB → 525,976 kB for 512 MiB) — reservation vs residency; (c) if you dare to exceed RAM, major-fault and swap-I/O rates climb while CPU utilization drops, the signature of thrashing — stop before the box wedges or the OOM killer fires. Label kernel/page size/RAM. *Tempting wrong answer:* "RSS should equal the mapping size right after `mmap`" — it should not; that is exactly what demand paging prevents. **Feed-forward:** §17.2, §17.4.

Chapter 18 — Solutions

18.1 A **signal** is an asynchronous notification the kernel delivers to a process to indicate an event (a software interrupt). It differs from a normal function call in that the program never *calls* the handler: the kernel *injects* it into the control flow at an arbitrary instant, suspending whatever was running, then resumes that code when the handler returns. *Tempting wrong answer:* "a signal is just a callback you invoke" — you register it, but delivery is forced by the kernel, not called by your code. **Feed-forward:** §18.1, §18.2.

18.2 **Terminate** (`SIGTERM`, `SIGINT`); **Terminate and dump core** (`SIGSEGV`, `SIGABRT`); **Ignore** (`SIGCHLD`'s default); and **Stop/Continue** (`SIGSTOP`/`SIGCONT`). *Tempting wrong answer:* "the default is always to terminate" — several signals default to ignore or to stop/continue, not termination. **Feed- forward:** §18.1.

18.3 **`SIGKILL`** and **`SIGSTOP`** cannot be caught or ignored. If a process could catch them, it could refuse to die or to be stopped — a runaway or malicious program would be unkillable, so the OS would lose its last-resort control. Keeping these two unblockable guarantees the kernel can always terminate or halt any process. *Tempting wrong answer:* "you can catch `SIGKILL` to clean up first" — you cannot; that is exactly why `SIGTERM` (catchable) exists for graceful shutdown and `SIGKILL` for the forced kill. **Feed-forward:** §18.1.

18.4 When `Ctrl-C` sends `SIGINT`, the kernel suspends the blocked `read`, runs the handler, and then the `read` returns `-1` with `errno == EINTR` ("interrupted system call"). It did not resume waiting because the kernel had to unblock the process to deliver the signal, and the default is to report the interruption rather than silently re-block. The two standard fixes: (1) **retry** the call when `errno == EINTR` (a loop), or (2) install the handler with **`SA_RESTART`** so the kernel auto-restarts many interrupted calls — though not all, so robust code still handles `EINTR`. *Tempting wrong answer:* "treat `EINTR` as a real read error and abort" — it is not an error about the data; the correct response is to retry. **Feed-forward:** §18.3.

18.5 It is unsafe because a handler can run in the middle of arbitrary code, including inside `printf`/ `malloc` themselves; those functions hold internal locks / mutable state and are not reentrant, so the handler's re-entry can deadlock or corrupt the heap. A function callable from a handler must be **async-signal-safe** (essentially reentrant — the `signal-safety(7)` list, e.g. `write`, `_exit`). The safe pattern: the handler sets a `volatile sig_atomic_t` flag (or does one `write` to a self-pipe) and returns; the main loop notices and does the real work synchronously. *Tempting wrong answer:* "it's fine if you only `printf` occasionally" — it only has to interrupt `printf` once, under load, to hang; rarity is what makes the bug dangerous, not safe. **Feed-forward:** §18.4.

18.6 Standard signals (1–31) are represented by a single pending bit each, so they do **not queue**: if the same signal is generated multiple times while pending/blocked, the process sees it once when it is delivered; the extra instances coalesce. So if several children exit close together, the parent may receive a single `SIGCHLD` for all of them — reaping exactly one child per signal would leave the others as zombies (Chapter 14). A correct handler therefore loops: `while (waitpid(-1, &status, WNOHANG) > 0) ;` reaps *all* currently-exited children per delivery. *Tempting wrong answer:* "one `SIGCHLD` per child, so reap one" — signals coalesce, so the count of deliveries does not match the count of events. **Feed-forward:** §18.4, and Ch 14.

18.7 (a) Setting a `volatile sig_atomic_t` flag is a single, atomic, async-signal-safe store — it cannot deadlock or corrupt anything — whereas doing the work directly risks re-entering unsafe libc; and it is cheap (one write). (b) The self-pipe / `signalfd` approach adds a little latency (a `write` then the main loop's next `read`) but buys correctness: the real work runs in the synchronous main flow where all functions are safe, and it integrates signals into an ordinary event loop. (c) It can never be a reliable counter because standard signals do not queue — multiple occurrences while pending collapse into one delivery, so the number of handler runs under-counts the number of events. *Tempting wrong answer:* "increment a counter in the handler to count events" — coalescing makes that count wrong under load. **Feed-forward:** §18.4.

18.8 **Step 4:** The bug is an **async-signal-safety violation**: the `SIGHUP` handler calls non-reentrant functions (`malloc`, stdio `fprintf`), so when the signal interrupts a request thread that is already inside one of them, the handler re-enters it and **deadlocks** (or corrupts the heap). It appears only under load because it requires the signal to land *during* an unsafe call; light testing rarely hits that window, but heavy request traffic keeps threads inside `malloc`/stdio a large fraction of the time, so the odds rise — the same schedule-dependent, "passes tests / breaks in production" pathology as a data race. The fix: make the handler trivial (set `volatile sig_atomic_t reload_requested = 1;` or `write` a self-pipe byte and return), and perform the config re-read, `malloc`, and logging in the **main loop** when it observes the flag, where libc is safe to call. *Tempting wrong answer:* "add a lock around the handler's `malloc`" — the handler may already hold or be blocked on that very lock via the interrupted call; locking cannot make a handler reentrant. **Feed-forward:** §18.4.

18.9 (a) **Signal** (`SIGHUP` by convention) — a content-free "reload" notification is exactly what a signal is for. (b) **Real IPC — a pipe** — a signal carries no data; a byte stream needs a pipe (or socket). (c) **Signal** — this *is* a signal (`SIGCHLD`); the parent then `wait`s. (d) **Real IPC — a socket** (or a pipe pair) — structured request/reply is a conversation, and a signal is only a doorbell. *Method-selection cue:* a content-free notification → signal; moving data or a request/reply → pipe/socket/shared memory. **Feed-forward:** §18.1, §18.5.

18.10 (a) **False** — the kernel injects the handler asynchronously; the program does not call it. (b) **False** — `SA_RESTART` restarts *many* but not all interrupted calls, so robust code still handles `EINTR`. (c) **False** — `SIGKILL` cannot be caught, so no cleanup runs; use `SIGTERM` for graceful shutdown. (d) **True** — an invalid memory access causes the kernel to deliver `SIGSEGV`, whose default disposition terminates (and cores) the process. *Tempting wrong answer:* calling (d) false because "a segfault is a hardware thing" — the hardware fault is *translated* by the kernel into the `SIGSEGV` signal. **Feed-forward:** §18.1, §18.2, §18.3, and Ch 11.

18.11 (a) `volatile` stops the compiler from caching the flag in a register or optimizing the read away, so the main loop actually re-reads memory the handler wrote; `sig_atomic_t` guarantees the read/write is a single uninterruptible operation, so the handler cannot catch a half-written value. Without both, the compiler might never observe the change, or the main loop might see a torn value. (b) It is *similar* to a data race: two flows of control touch the same memory with no ordering, so the shared access needs care. It is *different* in that there is one thread — the handler cannot run truly simultaneously with the main flow, only interrupt it — so the concern is atomicity of a single access and compiler visibility, not simultaneous multi-core writes (which is why `sig_atomic_t` suffices here where full synchronization is needed between threads). (c) Because the handler is a second, asynchronously-scheduled flow of control sharing memory with the main flow, a single-threaded program with signals already has concurrency — the interruption can happen between any two instructions. *Tempting wrong answer:* "a single-threaded program can't have concurrency bugs" — signals introduce asynchronous control flow, which is concurrency. **Feed-forward:** §18.4, and Ch 15.

18.12 Machine-specific by design. A correct write-up reports: (a) with a plain handler (no `SA_RESTART`), the `read` interrupted by `SIGALRM` returns `-1` with `errno == EINTR` (on the book's machine, "errno=4 (Interrupted system call)"); reinstalling with `SA_RESTART` makes the kernel restart the `read` so it keeps blocking (no `EINTR` surfaced). (b) The counting handler runs **once**, not five times, because the five blocked `SIGUSR1`s coalesce into one pending bit ("handler count so far = 0" while blocked, then "ran 1 time(s)" after unblock). (c) The first shows a caught signal interrupts a blocking call; the second shows standard signals do not queue. Label compiler/kernel. *Tempting wrong answer:* "the handler should run five times" — non-queuing standard signals collapse the repeats. **Feed- forward:** §18.3, §18.4.

Chapter 19 — Solutions

19.1 The toolkit splits into channels the kernel **copies** bytes through and arrangements that **share** the same physical memory. Copy side: a **pipe** (or FIFO, or socket) — bytes go sender → kernel buffer → receiver. Share side: **shared memory** (`MAP_SHARED`) — both processes map the same frames and touch them directly, moving no data. *Tempting wrong answer:* "sockets are the sharing mechanism" — a socket copies through the kernel like a pipe; only shared memory truly shares. **Feed-forward:** §19.1.

19.2 A bare pipe usefully connects processes that share its descriptors, which happens because file descriptors survive `fork` — so a parent creates the pipe and forks, and parent and child inherit the ends. A **FIFO** (named pipe, made with `mkfifo`) adds a *filesystem name*, so *unrelated* processes with no common ancestor can meet by opening the same path. *Tempting wrong answer:* "two unrelated processes can just call `pipe()` and connect" — a plain pipe's descriptors are private to the creator and its descendants; unrelated processes need a FIFO (or a socket). **Feed-forward:** §19.2.

19.3 When all write ends are closed, a `read` returns **0** — end-of-file — which is how the reader learns the writer is done. Writing to a pipe whose read ends are all closed raises **`SIGPIPE`** (default: terminate the writer), or, if that signal is ignored, fails the `write` with `errno == EPIPE`. *Tempting wrong answer:* "`read` blocks forever when the writer is gone" — it returns 0; it blocks only while a writer still holds the write end but has sent nothing. **Feed-forward:** §19.2.

19.4 A pipe is a byte stream: the kernel concatenates all writes and does not record how many `write`s produced the bytes, so two 3-byte writes are indistinguishable from one 6-byte write — measured as one `read` returning `"ABCDEF"`. For discrete messages you must add **framing**. Two schemes: (1) a fixed-size **length prefix** before each message (read the prefix, then read exactly that many bytes); (2) a **delimiter** (e.g. newline-terminated records), reading until the delimiter. *Tempting wrong answer:* "just do one `write` per message and one `read` per message" — the stream may coalesce or split regardless of how you wrote it, so the boundary is not preserved. **Feed-forward:** §19.2, §19.4.

19.5 Fastest because it **copies nothing**: both processes map the same physical frames (Chapter 17's shared mapping), so a store by one is a load by the other with no kernel round-trip. Most hazardous because it provides sharing and *nothing else* — no ordering, no mutual exclusion. `MAP_SHARED`: the child's write of 42 is visible to the parent (`shared == 42`). `MAP_PRIVATE`: copy-on-write splits the page on the child's write, so the parent still sees 0 (`private == 0`). Two processes racing a shared counter lost ~700,000 of two million updates — the Chapter 15 data race across processes. *Tempting wrong answer:* "shared memory is safe because there is only one copy of the data" — one copy is exactly why concurrent unsynchronized writes corrupt it. **Feed-forward:** §19.3, and Ch 15.

19.6 A **stream socket** (`SOCK_STREAM`, TCP) is a reliable, ordered byte stream with no message boundaries; a **datagram socket** (`SOCK_DGRAM`, UDP) preserves message boundaries but may lose, duplicate, or reorder datagrams. (a) reliable in-order request/response → **stream** (TCP), which gives you reliability and ordering for free (add framing for message boundaries). (b) low-latency telemetry tolerating loss → **datagram** (UDP), whose per-message delivery and lack of retransmission fit lossy, latest-value data. `AF_UNIX` makes the socket a *local* (same-host) channel — faster, and able to pass file descriptors — while `AF_INET` uses the same API to cross machines. *Tempting wrong answer:* "UDP is just faster TCP, always prefer it" — UDP drops reliability and ordering; choosing it means signing up to handle loss and reordering yourself. **Feed-forward:** §19.4.

19.7 (a) Through a pipe a byte is copied twice — sender → kernel buffer → receiver — plus system-call overhead; through shared memory it is copied **zero** times, because both processes read and write the same frames. Fewer copies and no per-message trap into the kernel make shared memory faster. (b) The 65536-byte buffer bounds how far ahead a writer can get: when it fills, the next `write` *blocks* until the reader drains it, so a fast producer is paced to the slow consumer — flow control (backpressure) for free. (c) The cost is that you must supply your own synchronization and a crash-recovery story: shared memory hands you unsynchronized concurrency, so you carry, by hand, the mutual exclusion and ordering the copying channels gave you automatically. *Tempting wrong answer:* "shared memory is strictly better because it is faster" — it trades the kernel's built-in serialization and flow control for engineering burden you now own. **Feed-forward:** §19.2, §19.3.

19.8 **Step 4:** The bug is a **data race** on the ring buffer's shared write index (and the slots): multiple producers perform an unsynchronized read–modify–write on the same index, so two can grab the same slot or advance the index inconsistently, garbling and duplicating lines. It appears only under load because the corruption requires two producers to touch the index at *overlapping* instants; light traffic rarely overlaps, heavy traffic overlaps constantly — the same schedule-dependent, "passes tests / breaks in production" pathology as any race. Fix 1 (synchronize in the shared region): place a mutex or semaphore *inside* the shared memory (a process-shared `pthread_mutex` or a POSIX semaphore both processes map) and hold it while advancing the index — or make the index a lock-free atomic. Fix 2 (change the channel): give each producer its own pipe or socket to the consumer, so there is no shared write index to race on at all; the kernel serializes each channel for you. *Tempting wrong answer:* "add a lock in each producer's private memory" — a lock only excludes if all racers take the *same* lock, which must therefore live in the shared region. **Feed-forward:** §19.3, and Ch 15.

19.9 (a) **Pipe/FIFO** — a one-way local byte stream is exactly a pipe; the shell builds it with `fork`. (b) **Shared memory** — 60 large frames a second on one host demands zero-copy; a socket's per-frame copies would waste bandwidth. (c) **Network socket** — different machines rules out every local mechanism; only a socket reaches across the network. (d) **Unix-domain socket** — bidirectional local exchange plus descriptor passing (`SCM_RIGHTS`) is a capability only Unix-domain sockets offer. *Method-selection cue:* local one-way stream → pipe; local high-volume data → shared memory; crosses machines → network socket; local bidirectional or fd-passing → Unix-domain socket. **Feed-forward:** §19.1–§19.4.

19.10 (a) **False** — TCP is a byte stream with no message boundaries; a `send` may be split or coalesced, so you must frame. (b) **False** — the pointer is meaningless in the other private address space; pass data through IPC, not a pointer. (c) **False** — shared memory provides no synchronization; concurrent writes race. (d) **False** — a standard signal carries only its number (real-time signals add a small payload); it is a doorbell, not a data channel (Chapter 18). (e) **True** — a `read` returns 0 once all write ends close, signalling end-of-file. *Tempting wrong answer:* calling (a) true because "TCP is reliable" — reliability (no loss, in order) is orthogonal to message framing; TCP is reliable *and* boundary-less. **Feed-forward:** §19.2, §19.4, and Ch 18.

19.11 (a) The producer fills the pipe's 65536-byte buffer, and its next `write` **blocks** until the consumer reads some out; it cannot run ahead or exhaust memory because the kernel buffer is bounded and a full pipe suspends the writer. It ends up waiting inside `write`. (b) This is **backpressure**: the slow stage throttles the fast one automatically. Over a raw shared-memory channel you get no such thing — you would have to build the bounded buffer and the block-when-full/block-when-empty logic yourself (which is precisely the producer–consumer problem of the next chapters). (c) When the consumer dies, all read ends close; the producer's next `write` raises **`SIGPIPE`** (default terminate) or, if ignored, returns `-1` with `errno == EPIPE`. The OS notifies rather than discards so the writer stops promptly instead of doing useless work forever pouring bytes to nobody. *Tempting wrong answer:* "the producer keeps buffering until it runs out of memory" — the bounded pipe blocks it long before that. **Feed-forward:** §19.2.

19.12 Machine-specific by design. A correct write-up reports: (a) one `read` after `"ABC"` and `"DEF"` returns six bytes, `"ABCDEF"`, showing the erased boundary. (b) The non-blocking pipe fills near **65536** bytes (on the book's machine, exactly 65536) before `write` returns `EAGAIN`; `fcntl(fd, F_SETPIPE_SZ, n)` changes the capacity. (c) With `MAP_SHARED` the parent sees the child's write (42); with `MAP_PRIVATE` it does not (0), because the private page is copy-on-write. Label compiler/kernel. *Tempting wrong answer:* "the two 3-byte writes should come back as two reads" — a stream erases the boundary; expecting message preservation is the classic framing bug. **Feed-forward:** §19.2, §19.3.

Chapter 20 — Solutions

20.1 A **critical section** is a region of code that touches shared state and produces a wrong result if two threads execute it concurrently. **Mutual exclusion** is the property that at most one thread is inside such a region at any instant. `counter++` needs it because it is really load–add–store: without exclusion, two threads can load the same old value and each store back a value one greater, losing an update. *Tempting wrong answer:* "`counter++` is a single instruction, so it's atomic" — it compiles to several steps, and even a single instruction is not atomic across cores without special guarantees. **Feed-forward:** §20.1.

20.2 **Lock** (acquire) and **unlock** (release). Acquiring a mutex already held by another thread **blocks** the caller until the holder releases it, after which one waiter proceeds. *Tempting wrong answer:* "lock returns an error if the mutex is held" — a plain `lock` blocks; it is `trylock` that returns immediately without acquiring. **Feed-forward:** §20.2.

20.3 **Mutual exclusion**, **hold-and-wait**, **no preemption**, and **circular wait** — all four must hold. The most practical to break in application code is **circular wait**, by imposing a *global lock ordering* so every thread acquires multiple locks in the same sequence. *Tempting wrong answer:* "break mutual exclusion by not using locks" — you usually cannot drop exclusion without a race; ordering is the practical lever. **Feed-forward:** §20.4.

20.4 "If free, take it" is itself a read–modify–write: read the mutex's state, decide, write "held." Two threads can both read "free" before either writes "held," so both believe they acquired the lock and both enter the critical section — the exact data race of Chapter 15, now on the lock word itself. A hardware atomic (test-and-set, compare-and-swap, exchange) performs the read-and-conditional-write as one indivisible step no other core can split, so exactly one thread wins. Without it, the lock cannot be race-free, and a racy lock provides no exclusion. *Tempting wrong answer:* "disabling interrupts is enough" — that works on a uniprocessor for the kernel, but not across multiple cores in user space; you need the atomic. **Feed-forward:** §20.2, and Ch 15.

20.5 (a) Thread 1 locks A; thread 2 locks B; thread 1 tries to lock B (held by 2) and blocks; thread 2 tries to lock A (held by 1) and blocks — permanent circular wait. (b) Making both acquire `A then B` removes **circular wait**: whoever gets A first will get B before releasing A, so no cycle of waiters can form. (c) It requires the two paths to interleave at just the wrong instant, which is rare under light test load but common under production concurrency — the schedule-dependent "passes tests / breaks in production" pathology. *Tempting wrong answer:* "add a timeout and retry" — `trylock`/back-off can help but risks livelock and masks the design flaw; a global lock order fixes it outright. **Feed-forward:** §20.4.

20.6 Exclusion: at most one thread runs the guarded code, so the load–add–store cannot interleave. **Memory synchronization:** the POSIX unlock in one thread and the next lock in another establish a happens-before ordering, so writes made under the lock are guaranteed visible, in order, to the next holder. The second role is what makes the non-atomic `counter++` *well-defined*: the C/C++ memory model calls an unsynchronized conflicting access a data race, which is undefined behavior; the lock's ordering removes the "conflicting, unsynchronized" property, so the access is legal. Without a lock (or atomics), a plain shared read–write is UB — the compiler and hardware may reorder and cache it freely. *Tempting wrong answer:* "exclusion alone makes it correct" — exclusion without the memory ordering still leaves visibility/reordering undefined; POSIX locks supply both, which is why they work. **Feed-forward:** §20.2, and Ch 15.

20.7 (a) The ~10× is the atomic read-modify-write, the surrounding bookkeeping, and the memory-ordering fence the lock and unlock impose — each far more expensive than a bare register add. (b) Serialization (only one thread in the critical section), cache-line bouncing (the lock word and guarded data migrate between cores' caches, a cross-core miss per handoff), and context switches (a blocking mutex sleeps and wakes waiters via the scheduler). Cache-line bouncing tends to dominate when the critical section is tiny and threads spin/retry; context-switch cost dominates when threads actually sleep. (c) By Amdahl, the serial (locked) fraction caps speedup regardless of core count, so reducing the fraction of work under the lock lifts the ceiling more than shaving nanoseconds off each acquisition. *Tempting wrong answer:* "a faster lock implementation is the main win" — if a large fraction of work is serialized, no lock speed helps; shrink the critical section. **Feed-forward:** §20.3.

20.8 Step 4: The bug is a **deadlock** from inconsistent lock ordering: transfer `X→Y` and `Y→X` acquire the two account locks in opposite orders, so concurrent opposite-direction transfers form a circular wait and both threads block in `lock` forever. It is load- and timing-dependent because it needs the two transfers to start close enough in time to each grab one lock before the other grabs the second — rare in tests, common under production traffic. Fix: impose a **global lock order** on accounts — e.g. always lock the account with the smaller id first, regardless of transfer direction — so both a `X→Y` and a `Y→X` transfer lock the same account first and no cycle can form. This removes the **circular-wait** condition. *Tempting wrong answer:* "lock a single global bank mutex for every transfer" — correct but serializes all transfers, killing throughput; ordering per-account locks keeps concurrency. **Feed-forward:** §20.4.

20.9 (a) **Single mutex** — one shared counter, all threads contend on the same word; one lock suffices (or an atomic). (b) **Fine-grained per-bucket mutexes** — operations on different buckets are independent, so per-bucket locks let them proceed in parallel; one big lock would needlessly serialize them. (c) **No lock** — thread-local storage is not shared, so there is no race to prevent. (d) **No lock (or a read-optimized scheme)** — a rarely-written, frequently-read value can use no lock if updates are done safely (e.g. atomic pointer swap / RCU-style publish); a plain mutex on every read would serialize all requests. *Method-selection cue:* is the state shared and mutated? If not shared → no lock; if shared and independent partitions → fine-grained; if a single hot datum → one lock (or atomic). **Feed-forward:** §20.3.

20.10 (a) **False** — mutual exclusion requires racers to take the *same* lock; different locks do not exclude each other. (b) **False** — deadlocked threads are *blocked*, so CPU is near zero; 100% CPU is a spin/livelock, not a classic deadlock. (c) **False** — a mutex adds overhead and serializes; it makes code *correct*, not faster. (d) **False** — holding a lock longer increases contention and deadlock exposure; hold it for the smallest correct region. (e) **False** — Peterson/Dekker use only loads and stores but assume ordering guarantees; real, portable locks need a hardware atomic. *Tempting wrong answer:* calling (e) true by citing Peterson's algorithm — it is correct only under a memory model with sufficient ordering, which on real hardware you get via atomics/fences, so "no atomic at all" is misleading. **Feed-forward:** §20.2, §20.3, §20.4.

20.11 (a) **Priority inversion**: H is blocked on the lock L holds, so H can proceed only when L runs and releases it. (b) Under strict priority scheduling, medium M is runnable and higher priority than L, so it preempts L and never yields; L therefore never gets the CPU to release the lock, and H — though highest priority — waits behind both. (c) **Priority inheritance**: while L holds a lock a higher-priority thread (H) is waiting on, L temporarily inherits H's priority, so L now outranks M, runs, releases the lock, and H proceeds. It breaks the link where M starves L. (d) Holding a lock while doing unbounded work under real-time scheduling can stall higher-priority threads for that whole duration — bounded, minimal critical sections are essential when priorities are strict. *Tempting wrong answer:* "just raise H's priority" — H is already highest; the problem is L is too low to run, which inheritance, not H's priority, fixes. **Feed-forward:** §20.4, and Ch 16.

20.12 Machine-specific by design. A correct write-up reports: (a) the unlocked four-thread total is below 4,000,000 and differs each run (on the book's machine, e.g. 1,187,737 / 1,144,730 / 2,766,329), while the mutex version is exactly 4,000,000 every run. (b) A lock+increment+unlock is roughly an order of magnitude slower than a bare increment (the book measured ~1 ns vs ~12 ns, ~12×) — illustrative and machine-dependent. (c) Opposite lock orders hang (a deadlock; kill it after a timeout), while a consistent order always completes. Label compiler/kernel. *Tempting wrong answer:* "re-running the unlocked version until it prints 4,000,000 proves it's fine" — a lucky correct run does not make a racing program correct; correctness must come from the rules. **Feed-forward:** §20.1, §20.3, §20.4.

Chapter 21 — Solutions

21.1 Because a spinning thread stays *runnable*, so the scheduler keeps giving it a CPU while it does no useful work — it burns a core (and, if it spins holding the lock, prevents the very progress it waits for). A blocking wait takes the thread *off* the CPU until it is woken, freeing the core for threads with work. *Tempting wrong answer:* "spinning is faster because there's no context switch" — true only for waits shorter than a context switch; for any appreciable wait, spinning wastes a whole core. **Feed-forward:** §21.1.

21.2 `wait`, `signal`, and `broadcast`. `wait` (called with the mutex held) **atomically releases the mutex and sleeps**; when later woken it re-acquires the mutex before returning. `signal` wakes one waiter; `broadcast` wakes all. *Tempting wrong answer:* "`wait` keeps the mutex so no one else can interfere" — it must release it, or the signalling thread could never acquire the lock to change the state. **Feed-forward:** §21.2.

21.3 At most **N** threads can be inside the guarded section at once: the semaphore starts with N permits, each `wait` takes one (blocking at zero), each `post` returns one. *Tempting wrong answer:* "it lets exactly N threads run and blocks the rest forever" — blocked threads proceed as permits are posted back; N is a concurrency cap, not a hard total. **Feed-forward:** §21.3.

21.4 Two reasons. (1) **Spurious wakeups**: POSIX permits `cond_wait` to return with no signal, so the predicate may not hold on waking. (2) **Stolen condition**: even after a real signal, between the signal and this thread re-acquiring the mutex, another thread can run and consume the item, so the predicate is false again. Two-consumer example: buffer has one item; producer signals; consumer C1 and C2 both were waiting; C1 wakes, re-acquires, pops the item; C2 wakes next, re-acquires — with `if` it does not re-check and pops from the now-empty buffer, reading garbage or crashing. With `while`, C2 re-tests `count == 0` and waits again. *Tempting wrong answer:* "one signal per item means one wakeup per item, so `if` is fine" — spurious wakeups and stolen conditions both break that count. **Feed-forward:** §21.2, §21.4.

21.5 (a) A condition variable's `signal` with no waiter is lost — it wakes no one and leaves no trace. So if a producer signals in the window after a consumer checked the predicate but before it called `cond_wait`, the consumer misses the signal and sleeps forever though the item is present. (b) Discipline: update the shared state *under the mutex* and then signal (still holding, or immediately after, the lock), and have the waiter loop `while (!predicate) cond_wait(...)`; holding the lock means the waiter cannot be mid-check-and-not-yet-asleep when you signal, and the loop re-checks the state on waking. (c) A semaphore's count *remembers* the post: a `sem_post` with no waiter increments the count, so a later `sem_wait` succeeds immediately — the "signal before wait" ordering is harmless. *Tempting wrong answer:* "signal before locking to be fast" — that is exactly the lost-wakeup race. **Feed-forward:** §21.3, §21.4.

21.6

```
producer(x):                          consumer():
  lock(m)                               lock(m)
  while (count == CAP)                  while (count == 0)
    wait(not_full, m)                     wait(not_empty, m)
  buf[in]=x; in=(in+1)%CAP; count++     x=buf[out]; out=(out+1)%CAP; count--
  signal(not_empty)                     signal(not_full)
  unlock(m)                             unlock(m)
```

It cannot overflow because the producer waits on `not_full` while `count == CAP`, so it never enqueues into a full buffer; it cannot underflow because the consumer waits on `not_empty` while `count == 0`. The loops are necessary because a wakeup may be spurious or the slot/item may have been taken by another producer/consumer before this thread re-acquires the mutex, so the predicate must be re-checked. *Method-selection cue:* "wait until a condition over shared state, then act" is precisely mutex + condition variable in a loop. **Feed-forward:** §21.2.

21.7 (a) The spinner stays runnable, so the scheduler keeps it on a CPU for the whole ~0.5 s (≈100% of a core = ~0.5 s CPU), while the blocker is off the CPU (blocked state) the entire time, waking only at the end (~0.001 s CPU). (b) A short spin beats blocking when the expected wait is *shorter than the cost of a context switch pair* (sleep + wake); the crossover quantity is that context-switch cost — below it, spinning avoids two scheduler round-trips. (c) `cond_wait` costs a context switch to sleep and another to wake (plus scheduler bookkeeping); it is still right for a long wait because it frees the core for real work, and the two switches are negligible against a long wait. *Tempting wrong answer:* "blocking is always cheaper" — for sub-microsecond waits the two context switches can cost more than a brief spin. **Feed-forward:** §21.1, and Ch 16.

21.8 **Step 4:** Bug 1 is `if`-not-`while`: a worker checks the queue once, waits, and on a spurious wakeup (or after another worker took the task) pops from an empty queue and crashes. Bug 2 is a **lost wakeup**: the producer signals without holding the lock and enqueuing under it, so a worker between its check and its `cond_wait` misses the signal, and since the condition variable does not remember it, the task sits with no one woken. Both are intermittent and load-dependent because each needs a specific interleaving that is rare under light load and common under heavy concurrency. Corrected:

```
worker:                       producer(task):
  lock(m)                       lock(m)
  while (queue.empty())         queue.push(task)
    cond_wait(cv, m)            cond_signal(cv)      // under the lock, after enqueue
  t = queue.pop()               unlock(m)
  unlock(m); run(t)
```

The `while` fixes bug 1 (re-check on wake); enqueue-under-lock-then-signal fixes bug 2 (no window to miss the signal). *Tempting wrong answer:* "signal before locking so the worker wakes sooner" — that reintroduces the lost wakeup. **Feed-forward:** §21.2, §21.4.

21.9 (a) **Plain mutex** — a single shared counter needs exclusion, not waiting. (b) **Counting semaphore** initialized to 5 — "at most 5 at once" is exactly a permit pool. (c) **Condition variable + mutex** — "block until a predicate over shared state (an item exists) becomes true" is the condition-variable pattern. (d) **Condition variable + mutex, using `broadcast`** — a one-shot event to *many* waiters wants a broadcast (or a latch); a single `signal` would wake only one. *Method-selection cue:* just exclude → mutex; cap concurrency at N → counting semaphore; wait for a state predicate → condition variable (broadcast to wake all). **Feed-forward:** §21.2, §21.3.

21.10 (a) **False** — a `cond_signal` with no waiter is lost, not remembered (that is a semaphore's property). (b) **False** — `cond_wait` releases the mutex while it sleeps and re-acquires it on waking. (c) **True** — a binary semaphore (0/1) behaves much like a mutex (with caveats about ownership). (d) **False** — even without spurious wakeups, a stolen condition (another thread consumes it first) still requires the `while`; keep the loop. (e) **True** — a mutex, like a signal-handler flag, is a mechanism for coordinating two flows of control over shared memory (Chapter 18's single-threaded concurrency generalizes). *Tempting wrong answer:* calling (d) true because "spurious wakeups are the only reason for the loop" — the stolen condition is a second, independent reason. **Feed-forward:** §21.2, §21.3, and Ch 18.

21.11 (a) A monitor enforces that all access to the shared state goes through procedures that automatically hold the monitor's mutex and coordinate via its condition variables, so the "lock before touching state," "signal under the lock," and "wait in a loop" rules are built into the structure rather than re-implemented (and forgotten) at each call site. (b) Java `synchronized/wait/notify` hides the mutex and condition variable behind object methods; a thread-safe blocking queue hides the mutex, the two condition variables, and the predicate discipline behind `put/take`. (c) Because the primitives are where the subtle, load-dependent bugs live (`if`-not-`while`, lost wakeups, lock ordering), using a vetted higher-level abstraction removes whole bug classes; you would still drop to raw condition variables for a custom waiting policy the high-level type does not express (e.g. priority-ordered wakeups, or waiting on a compound predicate over several structures). *Tempting wrong answer:* "always hand-roll for performance" — the abstraction's overhead is usually negligible against the cost of a concurrency bug in production. **Feed-forward:** §21.4.

21.12 Machine-specific by design. A correct write-up reports: (a) the busy-poll waiter consumes ~100% of a core for the wait (on the book's machine ~0.499 s CPU for a ~0.5 s wait) while the condition-variable waiter consumes almost none (~0.001 s), because it is blocked, not runnable. (b) The bounded buffer's produced and consumed totals match (e.g. both 210 for items 1..20) and the buffer never exceeds its capacity, confirming correct coordination via the `while`-loops. (c) With a semaphore initialized to three, at most three of the ten threads are ever in the guarded section simultaneously. Label compiler/kernel; the CPU figures are illustrative. *Tempting wrong answer:* "the spinner is fine, it got the same result" — it wasted a core to do so; correctness plus efficiency both matter. **Feed-forward:** §21.1, §21.2, §21.3.

Chapter 22 — Solutions

22.1 Load `count` from memory into a register, add one, store the register back to memory. It is non-atomic across threads because another thread can read the same old value (or run in parallel) between the load and the store, so two increments produce one net change. *Tempting wrong answer:* "it's one C statement, so it's one operation" — one line of C is three machine instructions, and atomicity is a property of the machine operation, not the source. **Feed-forward:** §22.1.

22.2 The `LOCK` prefix makes the CPU hold the affected cache line exclusively for the entire duration of the read-modify-write, so no other core can read or write that line in between — the whole load-add-store is indivisible. `lock xadd` is **one** instruction, where a plain memory increment is three (`mov`, `lea`/`add`, `mov`). *Tempting wrong answer:* "it locks the whole bus so nothing else runs" — modern CPUs lock at the cache line via the coherence protocol, not the whole bus, except in rare unaligned cases. **Feed-forward:** §22.2.

22.3 On success it returns **true** (having written `desired` because `*obj` equalled `expected`). On failure it returns false and writes the *current* value of `*obj` into `expected`, leaving `*obj` unchanged. *Tempting wrong answer:* "on failure it leaves `expected` alone" — it overwrites `expected` with what it found, which is exactly what makes the retry loop reload for free. **Feed-forward:** §22.3.

22.4 (a) T1 load (0) → T2 load (0) → T1 add (1) → T1 store (1) → T2 add (1) → T2 store (1): final `count == 1`, one increment lost. (b) `atomic_fetch_add` compiles to a single `lock xadd` that holds the cache line exclusively for the whole read-modify-write, so T2's increment cannot begin until T1's has completed — the interleaving in (a) cannot occur. (c) At `-O2` the optimizer can collapse the million-iteration loop into a single `count += N`, shrinking each thread's race window to one load-add-store that rarely collides; a data race is undefined behavior, so "passes at `-O2`" proves nothing. *Tempting wrong answer:* "the `-O2` version is correct because it gives the right answer" — it is still a data race; it merely hides. **Feed-forward:** §22.1, §22.2.

22.5

```
void atomic_max(atomic_long *p, long v) {
    long cur = atomic_load(p);
    while (v > cur)
        if (atomic_compare_exchange_weak(p, &cur, v))
            return;                 /* installed v */
}
```

After a failed CAS, `cur` holds the value the CAS actually found in `*p`, so the `while` re-tests against fresh state and no explicit reload is needed. Use `_weak` in the loop because it is allowed to fail spuriously in exchange for cheaper code on load-linked/store-conditional machines, and a spurious failure merely costs one extra iteration. *Method-selection cue:* "update only if the old value satisfies a condition" → single CAS in a retry loop. **Feed-forward:** §22.3.

22.6 (a) T1 `atomic_load(&owner) == 0` (true) → T2 `atomic_load(&owner) == 0` (true) → T1 `atomic_store(&owner, 1)` → T2 `atomic_store(&owner, 2)`: both passed the test, both stored, and now T2 "owns" but T1 also believed it did. (b) `long expected = 0; if (atomic_compare_exchange_strong(&owner, &expected, my_id)) { /* I claimed it */ }` — exactly one thread finds `owner == 0` and swaps in its id; the other's CAS fails. (c) Any update that reads a value, decides based on it, and writes a value depending on that decision ("if it's still X, make it Y") must be a single CAS, because a separate atomic load and store leave a gap. *Tempting wrong answer:* "put the load and store close together so nothing runs between" — proximity in source is irrelevant; only a single indivisible operation closes the gap. **Feed-forward:** §22.3, §22.4.

22.7 (a) The atomic must own the cache line exclusively and acts as a reordering barrier, so it cannot overlap with surrounding work the way a plain store can; that serialization is the ~15×. (b) Under eight-core contention the same line ping-pongs between cores' caches — each atomic must first pull the line to its own core in exclusive state (coherence traffic, Chapter 3) — so per-op cost rises well beyond the uncontended 15 ns, sometimes by an order of magnitude. (c) When updates vastly outnumber reads, give each thread its own counter (no sharing, no coherence traffic) and sum them only when a total is needed; the cost moves to the occasional summation and to cache footprint, not to every increment. *Tempting wrong answer:* "atomics have a fixed cost, so contention doesn't matter" — contention can dominate, which is why sharding a counter helps. **Feed-forward:** §22.4, and Ch 3.

22.8 **Step 4:**

```
void record_max(atomic_long *hi, long v) {
    long cur = atomic_load(hi);
    while (v > cur)
        if (atomic_compare_exchange_weak(hi, &cur, v))
            return;
}
```

It terminates because each failed CAS means another thread installed a value at least as large, which is written back into `cur`; so `cur` is non-decreasing, and once `cur >= v` the loop exits without another attempt. The invariant it guarantees: at all times `hi` is greater than or equal to every `v` that has been submitted, and eventually equals the maximum submitted. *Tempting wrong answer:* "it could loop forever under contention" — every retry is caused by a real increase in `hi`, which is bounded by the largest submitted value, so retries are finite. **Feed-forward:** §22.3.

22.9 (a) **Atomic `fetch_add`** — a fixed arithmetic RMW with a dedicated instruction. (b) **Single CAS** (`expected = 0`, swap in 1) — the success/failure result tells you who won. (c) **CAS retry loop** — set the new node's `next` to the current head, then CAS the head; retry if it moved (the Treiber stack of Chapter 23). (d) **Plain access** — a value no thread writes concurrently needs no atomicity. *Method-selection cue:* fixed arithmetic → `fetch_*`; one-shot "claim" → single CAS; update depending on current shared state → CAS loop; no concurrent writer → plain. **Feed-forward:** §22.2, §22.3.

22.10 (a) **False** — a mutex is a program built *from* atomics (plus a blocking slow path); the atomic is the hardware primitive. (b) **True** — on x86 the atomic and plain forms differ by the one-byte `LOCK` prefix. (c) **False** — the two atomics leave a gap another thread can slip through; only a single CAS composes compare-and-set. (d) **True** — that is exactly the consensus hierarchy: CAS has consensus number infinity, plain reads/writes number 1. (e) **False** — an uncontended atomic increment is roughly an order of magnitude dearer than a plain one (~15 ns vs ~0.9 ns here), and worse under contention. *Tempting wrong answer:* calling (c) true because "each part is atomic" — atomicity of the parts is not atomicity of the whole. **Feed-forward:** §22.2, §22.3, §22.4.

22.11 (a) Consensus number 1 means reads and writes can solve consensus only for a single thread (trivially) — for two, no protocol of plain reads and writes lets both irrevocably agree on one value regardless of timing; whichever writes "last" is timing-dependent and a reader cannot atomically observe-and-decide, so the two can diverge. (b) CAS at infinity means any number of threads can reach consensus, and from CAS you can build a wait-free implementation of *any* object with a sequential specification (a universal construction) — CAS is a universal primitive. (c) Every lock's fast path is a CAS (or test-and-set) claiming the lock word, and every lock-free structure is a CAS loop committing an update only if nothing changed; the hardware provides CAS specifically because, unlike plain accesses or fixed-arithmetic atomics, it is strong enough to build all of them. *Tempting wrong answer:* "`fetch_add` is just as universal as CAS" — fetch-and-add has a finite consensus number; it cannot express arbitrary conditional commits. **Feed-forward:** §22.4.

22.12 Machine-specific by design. A correct write-up reports: (a) the plain `-O0` counter falls short of 4,000,000 (on the book's machine ~1.36 million, losing ~2.6 million) and varies run to run, while `atomic_fetch_add` is exactly 4,000,000 every run; the `-O2` plain version may "pass" because the optimizer collapses the loop to one add per thread, shrinking the race window (still undefined behavior). (b) The disassembly shows the atomic form carries a `lock` prefix (`lock xadd`) absent from the plain increment. (c) The CAS-loop atomic max agrees with the true maximum under many threads. Label compiler/CPU; timing figures are illustrative. *Tempting wrong answer:* "the `-O2` plain version works, so the bug was the optimizer's fault" — the program was always racy; `-O2` only hid it. **Feed-forward:** §22.1, §22.2, §22.3.

Chapter 23 — Solutions

23.1 That the system as a whole always makes progress: out of all contending threads, some thread completes its operation in a bounded number of steps, so the structure can never be globally stuck. *Tempting wrong answer:* "every thread is guaranteed to complete" — that is wait-free; lock-free permits an individual thread to starve while others succeed. **Feed-forward:** §23.1.

23.2 Push: the successful CAS that installs the new node as `top`. Pop: the successful CAS that swings `top` to `old->next`. *Tempting wrong answer:* "the `malloc` (for push) or the return (for pop)" — the item is neither published nor removed until the CAS commits; that is the instant the operation takes effect. **Feed-forward:** §23.2.

23.3 A CAS checks only that a location still holds the same *bits* you read; if the value went A → B → A, the bits match and the CAS succeeds, even though the object those bits denote may have been removed, freed, and replaced. *Tempting wrong answer:* "the same value means nothing changed" — it means the value is *equal now*, not that it never changed. **Feed-forward:** §23.4.

23.4

```
push(v): n = new node(v)
         old = load(top)
         do { n.next = old } while (!CAS(&top, &old, n))
pop():   old = load(top)
         while (old) { if (CAS(&top, &old, old.next)) return old }
         return NULL
```

(a) A failed CAS means another thread changed `top` first; the failure reloads `old` with the current top, so the loop re-points `n.next` (push) or re-reads `old.next` (pop) and retries against fresh state. (b) The successful CAS is the single instant the operation takes effect for all threads — the linearization point — so concurrent operations order by whose CAS won. (c) Lock-free, not wait-free: the system always advances because some CAS always succeeds, but a persistently unlucky thread can keep losing the race and never complete. *Method-selection cue:* "update a shared pointer only if it hasn't moved" → CAS retry loop. **Feed-forward:** §23.2.

23.5 (a) It cannot be one CAS because enqueue must both link the new node (`tail.next: NULL -> n`) and move `Tail`, two separate locations; no single-word CAS updates both. (b) After the first CAS but before the second, a second thread can see `Tail` lagging — pointing at a node whose `next` is no longer `NULL`; "helping" means that thread advances `Tail` to `tail.next` itself (via CAS) rather than waiting for the enqueuer, so no one stalls on the laggard. (c) The dummy node guarantees the list is never empty, so `Head` and `Tail` always reference a real node and there are no null-pointer boundary cases at either end. *Tempting wrong answer:* "the second thread must wait for the enqueuer to finish" — helping is precisely what avoids that wait. **Feed-forward:** §23.3.

23.6 Thread A reads `top = A`, reads `A.next = B`, and is preempted before its CAS. Now: another thread pops `A` (`top = B`), pops `B` (`top = C`), and frees both; then it allocates a new node that `malloc` places at `A`'s old address, sets some `next`, and pushes it, so `top = A` again — but this `A` is a different logical node and `A.next` is now `C` or garbage. A resumes and does `CAS(&top, A, B)`: it sees `top == A` (same bits), succeeds, and sets `top = B` — a node that was already popped and freed. The stack is now corrupt (dangling or lost nodes). A tagged pointer prevents it: the version counter incremented on each of those pops/pushes makes the pair `(A, version)` differ, so A's CAS fails. Hazard pointers prevent it too: `A` and `B` could not have been freed while A still referenced them. *Tempting wrong answer:* "the CAS is buggy" — the CAS is correct; the bug is freeing and reusing memory under it. **Feed-forward:** §23.4.

23.7 (a) Under low contention a CAS usually succeeds on the first try, so a push/pop is one atomic op with no blocking, no context switch, and no futex system call — cheaper than a mutex's lock/unlock (hence ~176 vs ~359 ns/op here); it spends its time on the single CAS rather than on scheduler round-trips. (b) Under very high contention on one hot pointer, most CASes fail and retry, and each attempt drags the cache line to the current core in exclusive state — a coherence storm — so throughput can fall below even the lock's, as work is wasted on failed attempts. (c) Elimination (pairing a push with a concurrent pop so both skip the stack) or per-thread sub-stacks cut contention; they trade strict LIFO/global ordering and simplicity for scalability. *Tempting wrong answer:* "lock-free is always faster" — under heavy contention a lock can win because it serializes instead of burning cycles on retries. **Feed-forward:** §23.2, and Ch 3.

23.8 **Step 4:** Hazard-pointer `pop`: (1) read `t = load(top)`; (2) publish `t` in this thread's hazard slot; (3) re-check `load(top) == t` — if it changed, restart, because a node could have been retired between (1) and (2); (4) now safe to read `next = t.next` and attempt `CAS(&top, t, next)`; (5) on success, `t` is unlinked — retire it to a per-thread list and free it later, only once no thread's hazard slot references it. This makes use-after-free impossible because a node any thread is dereferencing is published as hazardous and therefore never freed; and it makes ABA impossible because a node cannot be freed and its address reused while it is still referenced, so `top` cannot cycle back to a recycled `t` under a stalled reader. *Tempting wrong answer:* "just free in `pop` after the CAS" — another thread may still hold that node as its own `old`; the re-check-after-publish is what closes the window. **Feed-forward:** §23.4.

23.9 (a) **Lock-free** — an audio callback with a hard deadline must never block on a lock a slower thread holds. (b) **Mutex** — rarely contended and read once at startup; a lock is simpler and correctness is easier to reason about. (c) **Lock-free** (of a signal-safe kind) or a lock-free single-slot handoff — a signal handler must not take a lock the interrupted thread may already hold (Chapter 18's async-signal-safety). (d) **Mutex** — large, complex operations with modest contention favor the simplicity of a lock; making them lock-free would be intricate for little gain. *Method-selection cue:* must-never-block or signal context → lock-free; simplicity with tolerable contention → mutex. **Feed-forward:** §23.1, and Ch 18.

23.10 (a) **False** — lock-free code is built *from* atomic instructions (CAS); it means no locks and a system-progress guarantee, not no atomics. (b) **False** — a lock-free structure cannot deadlock; retries mean some other thread is succeeding (making progress), which is the opposite of deadlock, though one thread may starve. (c) **False** — the dummy node is a correctness device that removes null-boundary cases, not an optional optimization. (d) **True** — that is exactly the ABA problem: equal bits do not prove the object was untouched. (e) **True** — wait-free (every thread completes) implies lock-free (some thread completes) but not conversely. *Tempting wrong answer:* calling (b) true because "infinite retries look like being stuck" — the system is still making progress; only the unlucky thread is not. **Feed-forward:** §23.1, §23.3, §23.4.

23.11 (a) Helping solves the two-step enqueue: because linking the node and swinging `Tail` are separate CASes, an enqueuer can be preempted between them, leaving `Tail` lagging; a later thread advances `Tail` itself so it need not wait for the preempted enqueuer. (b) If every thread can finish any pending operation it encounters, then a thread preempted mid-update never blocks progress — whoever runs next completes (or helps complete) the operation, so the system always advances, which is the lock-free guarantee. (c) A lock has no helping: a preempted holder stalls everyone until it is rescheduled. Helping buys immunity to that at the cost of extra CASes (advancing another's tail) and more intricate invariants (every intermediate state must be well-defined and completable by anyone). *Tempting wrong answer:* "helping is just a nicety" — it is what makes the structure non-blocking despite the multi-step update. **Feed-forward:** §23.3.

23.12 Machine-specific by design. A correct write-up reports: (a) the eight-thread Treiber stack pushes and pops the full item count with none lost or duplicated (on the book's machine 800,000 in and 800,000 out, matching checksums). (b) Against a mutex-guarded stack under contention, ns/op for both, with the note that which wins depends on contention (here lock-free ~176 vs mutex ~359 ns/op). (c) The tagged-pointer version compiles with `-mcx16 -latomic`, passes the same correctness test, and the disassembly shows `lock cmpxchg16b`. Label compiler/CPU; throughput figures are illustrative. *Tempting wrong answer:* "the leaking version is fine since the test passes" — it passes only because it never frees; adding `free` without safe reclamation reintroduces ABA. **Feed-forward:** §23.2, §23.4.

Chapter 24 — Solutions

24.1 Sequential consistency: all cores observe memory operations as if interleaved into a single global order that respects each thread's program order. Real machines violate it because of the per-core **store buffer** (a store is buffered locally before becoming globally visible). *Tempting wrong answer:* "caches cause it, and cache coherence would fix it" — coherence keeps a single value per location consistent; the reordering here is the store buffer delaying visibility, which coherence does not prevent. **Feed-forward:** §24.1.

24.2 A data race is two accesses to the same location from different threads, at least one a write, not ordered by synchronization (happens-before), with at least one non-atomic. The standard says such a program has **undefined behavior** — no meaning at all, and the compiler may assume it cannot occur. *Tempting wrong answer:* "a data race just gives a torn or stale value" — it is undefined behavior, strictly worse than an unspecified value. **Feed-forward:** §24.2.

24.3 It guarantees atomicity (no torn or lost updates) and coherence on the same object; it does *not* impose any ordering on other memory operations and creates no synchronizes-with edge, so it cannot make surrounding writes visible to another thread. *Tempting wrong answer:* "relaxed still orders the writes right before it" — it orders nothing but the atomic's own value. **Feed-forward:** §24.3.

24.4 (a) Under SC the two stores take some global order; say `x = 1` is first. Then thread 1's later `r1 = x` must read 1 (the store is already globally visible), so `r1 == 0` is impossible — and symmetrically if `y = 1` is first, `r0 == 0` is impossible; hence both zero cannot happen. (b) On x86 each store enters a private store buffer and is not yet visible to the other core, so each thread's load reads the other variable's old value 0 before the store drains. (c) Make both stores (and ideally the loads) `seq_cst`; on x86 the seq_cst store compiles to a locked `xchg` that drains the store buffer, forbidding the store-load reorder. *Tempting wrong answer:* "making the loads acquire fixes it" — acquire/release does not forbid store-load reordering; only seq_cst (or an explicit full fence) does. **Feed-forward:** §24.1, §24.2, §24.4.

24.5 *Sequenced-before* is program order within a single thread. *Synchronizes-with* is a cross-thread edge, e.g. from a release-store to the acquire-load that reads its value. *Happens-before* is (roughly) the transitive closure of sequenced-before and synchronizes-with. Two conflicting accesses not ordered by happens-before are a data race because nothing establishes which is visible to the other — their relative order is undefined, so the standard declares it undefined behavior. *Tempting wrong answer:* "happens-before is just program order" — program order (sequenced-before) is only the within-thread part; the cross-thread edges come from synchronizes-with. **Feed-forward:** §24.2, §24.3.

24.6

```
producer:  payload = ...;                  consumer:  while (load_acquire(&flag) != R) {}
           store_release(&flag, R);                   use payload;
```

(a) The publishing store is `release`; the observing load is `acquire`. (b) When the acquire-load reads the value the release-store wrote, the store synchronizes-with the load, so everything sequenced-before the release (the payload writes) happens-before everything sequenced-after the acquire (the payload reads) — the payload is guaranteed visible. (c) With both `relaxed` there is no synchronizes-with and no happens-before, so the consumer may see the flag set but the payload stale (undefined in general); it may still pass on x86 because the hardware does not reorder store-store or load-load, so the writes happen to reach memory in order. *Method-selection cue:* "publish data behind a flag" → release on the publish, acquire on the observe. **Feed-forward:** §24.3, §24.4.

24.7 (a) x86-TSO never reorders store-store or load-load, so acquire (a load) and release (a store) need no barrier — they compile to plain `mov`, and only the compiler is restrained from reordering across them; seq_cst additionally needs store-load ordering, which the hardware *does* relax, so its store needs a real fence. (b) The `xchg` is a locked read-modify-write that drains the store buffer before proceeding, so the store is globally visible before the following load — forbidding the store-buffering outcome that a plain `mov` allows. (c) ARM/POWER reorder store-store and load-load, so acquire and release require explicit barrier instructions there; hence the weakest correct ordering matters for portable performance — an unnecessary seq_cst costs a fence on every architecture. *Tempting wrong answer:* "seq_cst is free like acquire/release because x86 is strong" — the seq_cst store still pays for the `xchg`. **Feed-forward:** §24.4.

24.8 **Step 4:** Make the pointer-publishing store `memory_order_release` and the reader's pointer load `memory_order_acquire`. Then, when the reader's acquire-load observes the published pointer, the release-store synchronizes-with it, so all the node's field writes (sequenced-before the release in the writer) happen-before the reader's field reads (sequenced-after the acquire) — the fields are guaranteed visible, no garbage. The bug hid on x86 because x86-TSO does not reorder store-store, so the field writes reached memory before the pointer even with relaxed ordering; ARM reorders store-store, exposing it. *Tempting wrong answer:* "add a sleep or a second read to let the writes catch up" — timing hacks do not create happens-before; only the release/acquire pairing does. **Feed-forward:** §24.3, §24.4.

24.9 (a) **relaxed** — a counter whose ordering relative to other data is irrelevant; only the final sum matters. (b) **release** on the publish and **acquire** on the consumer's read — it must make the buffer's writes visible. (c) **relaxed** — a stop flag whose late observation is harmless needs only atomicity, not ordering. (d) **seq_cst** — Dekker needs store-load ordering across the two flags, which only seq_cst (or a full fence) provides. *Method-selection cue:* no cross-variable ordering → relaxed; publish/observe data → release/acquire; need store-load / global order → seq_cst. **Feed-forward:** §24.3, §24.4.

24.10 (a) **False** — a data race is undefined behavior, not merely an unspecified value. (b) **True** — on x86 a release-store and a relaxed store both compile to a plain `mov`. (c) **False** — relaxed orders nothing but the atomic's own value; it does not make surrounding writes visible in order. (d) **True** — that is the DRF-SC guarantee. (e) **False** — seq_cst's store costs a locked `xchg` on x86; acquire/release do not. *Tempting wrong answer:* calling (e) true because "x86 is a strong model" — strong enough to make acquire/release free, but the seq_cst store still needs a fence. **Feed-forward:** §24.2, §24.4.

24.11 (a) By promising SC only to race-free programs, the standard lets the compiler and CPU reorder and cache freely wherever no synchronization observes it, while a program that synchronizes correctly still sees the simple sequentially consistent model — the best of both. (b) "Undefined" (not "unspecified") lets the compiler assume races never occur and optimize on that basis (e.g. keeping a shared variable in a register across a supposedly racy read); under a weaker "unspecified value" definition the compiler would have to reload shared memory conservatively and could not make those transformations. (c) A data race is undefined behavior exactly like a buffer overflow or signed overflow (Chapter 10): the optimizer assumes it cannot happen, so a racy program may work in testing and break under a new compiler, optimization level, or CPU — "it worked when I tested it" proves nothing about a program with UB. *Tempting wrong answer:* "undefined is just pedantry; in practice you get a stale value" — the compiler is entitled to transformations that make the failure far worse than a stale value. **Feed-forward:** §24.2, and Ch 10.

24.12 Machine-specific by design. A correct write-up reports: (a) with relaxed atomics the store-buffering outcome `r0 == r1 == 0` occurs a nonzero number of times (on the book's machine ~349,000 of two million, about 1 in 5), and switching to seq_cst drops it to zero. (b) The disassembly shows relaxed store, release store, and acquire load as plain `mov`, while the seq_cst store is a locked `xchg` — the one that differs. (c) The release/acquire message-passing handoff verifies a large number of payloads (e.g. one million) with zero stale reads. Label compiler/CPU; the reordering counts are hardware-dependent. *Tempting wrong answer:* "zero SC violations means my relaxed publish is fine" — on x86 you may see zero even for buggy relaxed publishing; the counts are hardware-dependent, and ARM would differ. **Feed-forward:** §24.1, §24.4.

Chapter 25 — Solutions

25.1 (1) **Creation/teardown cost**: creating a thread makes the kernel allocate a stack and task structure and schedule it, and joining tears that down — hundreds of microseconds, dwarfing a small task. (2) **Oversubscription**: more runnable threads than cores forces the scheduler to time-slice them, paying context switches and cache disruption without adding parallelism. *Tempting wrong answer:* “threads are bad because of locks” — the cost here is creation and oversubscription, independent of any locking. **Feed-forward:** §25.1.

25.2 It is the **producer-consumer** pattern (Chapter 21): producers enqueue tasks, worker-consumers dequeue and run them. The work queue uses a **mutex** (mutual exclusion over the shared queue's state) and a **condition variable** (workers block on it when the queue is empty and are signalled when a task is enqueued, so they wait without spinning). *Tempting wrong answer:* “it just needs a mutex” — without a condition variable an empty-queue worker either busy-waits or has no way to be woken. **Feed-forward:** §25.2.

25.3 A **future** is a handle to a value that may not be computed yet (returned immediately when a task is submitted). `.get()` on an unfinished task *blocks* until it completes and then returns the result; on an already-finished task it returns the result immediately. *Tempting wrong answer:* “a future is the result value” — it is a handle/placeholder for a result that may still be pending, not the value itself. **Feed-forward:** §25.3.

25.4 A pool creates its `W` workers once at startup and keeps them alive, so the ~220 us creation cost is paid `W` times total, not per task; every one of the `N` tasks reuses an already-running worker, so its cost is just enqueue + dequeue + run. In the measurement, thread-per-task cost 217 us/task while the pool cost 1.04 us/task — so on the order of 216 of the 217 us per task (well over 99%) was thread creation/teardown overhead, not useful work; the pool eliminates it by never creating a thread per task. *Tempting wrong answer:* “the pool is faster because it uses more threads” — it uses *fewer* (a fixed `W`); the win is amortized creation, not more threads. **Feed-forward:** §25.1, §25.2.

25.5 (a) The mutex protects only the queue; holding it while running `t.fn()` would let just one worker be inside the pool at a time, serializing all the work and destroying parallelism. (b) A `while` loop re-checks the predicate after waking because condition-variable waits can wake spuriously and because another worker may have taken the item first; an `if` would proceed on an empty queue (Chapter 21). (c) The worker *blocks* in `cond_wait` when the queue is empty, releasing the CPU until a signal arrives, so an idle pool consumes no CPU. *Tempting wrong answer:* “run under the lock so the task is safe” — the task's own data safety is its concern; the lock guards the queue, and holding it over the task serializes the pool. **Feed-forward:** §25.2.

25.6 Each worker owns a double-ended queue (**deque**). The owner pushes and pops from one end (the bottom) as a local, uncontended operation; a **thief**, when its own deque is empty, steals from the *other* end (the top) of a randomly chosen victim. Because owner and thieves touch opposite ends, the common local path needs no contention with thieves. Blumofe–Leiserson: a well-structured (fully-strict) computation with work `T1` and span `T∞` runs in expected O(`T1`/`P` + `T∞`) time on `P` processors; near-linear speedup needs enough parallelism, `T1`/`T∞` > `P`. *Tempting wrong answer:* “a thief steals from the same end the owner uses” — that would contend on every local push/pop; thieves take the opposite (top) end. **Feed-forward:** §25.4.

25.7 (a) Every worker must hold the one global lock for time `c` to dequeue; the lock is a serial resource, so the pool cannot dequeue faster than one task per `c`. Once `P` is large enough that `P·t` of work demands dequeues faster than the lock can grant them, throughput saturates at roughly 1/`c` tasks per unit time — the shared queue is the bottleneck (an Amdahl serial fraction, next chapter). (b) Per-worker deques let each worker enqueue/dequeue locally with no shared lock; stealing is rare, so the serial bottleneck largely disappears. (c) Coarser tasks mean more useful work `t` per dequeue `c`, so the lock is taken less often relative to work done, improving scaling; too coarse and you have too few tasks to balance load and fill all workers (poor parallelism, idle cores). *Tempting wrong answer:* “just add more workers” — past saturation, more workers only add lock contention. **Feed-forward:** §25.2, §25.4.

25.8 **Step 4:** This is **pool deadlock** (thread-pool starvation). It is a genuine deadlock: the 4 worker threads are the only resource that can run tasks, and all 4 are *held* by request tasks that are *waiting* on futures for subtasks; those subtasks need a free worker to run, but none exists — a circular wait between “workers held by waiters” and “subtasks needing a worker.” Fix 1 (structure): run blocking/dependent work in a *separate* pool from the CPU work, or use a work-stealing fork–join runtime whose join executes the pending subtask on the current thread instead of blocking. Fix 2 (expression): don't have a worker block on a future for same-pool work — compose dependencies with continuations/callbacks the runtime can schedule, so no worker sits idle holding a thread. *Tempting wrong answer:* “just make the pool a bit bigger” — any fixed size `W` deadlocks once `W` tasks block on un-started subtasks; size only shifts the load at which it hangs. **Feed-forward:** §25.4.

25.9 (a) **Right** — CPU-bound non-blocking tasks want about one worker per hardware thread; more just oversubscribes. (b) **Change it** — I/O-bound tasks spend most time blocked, so a larger pool (or async I/O) keeps cores busy while threads wait. (c) **Change it** — millions of tiny recursive subtasks want a work-stealing fork–join pool whose join doesn't block a worker, not a plain shared-queue pool. (d) **Change it** — separate the pools: a hardware-sized pool for the short CPU tasks and a distinct pool (or async) for the long blocking DB calls, so blocking work can't starve CPU work. *Method-selection cue:* CPU-bound → ~hardware-thread count; blocking → larger/separate pool or async; recursive fork–join → work-stealing runtime. **Feed-forward:** §25.2, §25.4.

25.10 (a) **False** — parallelism is capped at the number of cores; extra threads oversubscribe. (b) **True** — the work queue is a producer–consumer buffer under a mutex + condition variable. (c) **False** — an idle pool blocks in `cond_wait` and uses no CPU. (d) **True** — work-stealing replaces the one shared queue with per-worker deques (plus occasional stealing). (e) **False** — `get()` blocks only until *that task* finishes (or returns immediately if it already has), not until the whole pool is idle. *Tempting wrong answer:* marking (a) true because "each task has its own thread" — having a thread is not having a core to run it on. **Feed-forward:** §25.1–§25.4.

25.11 (a) The owner pushes/pops the bottom, so a thief taking from the top rarely touches the same slot — low contention — and the top holds the *oldest* task, which in a fork–join tree is the highest, coarsest-grained subtree; stealing a large task means one steal hands the thief a lot of independent work, so steals are infrequent and load balances well. (b) In $O($`T1/P` + `T∞`$)$, `T1/P` is the total work spread perfectly over `P` processors, and `T∞` is the span — the critical path that must run sequentially no matter how many processors you have. If `T∞` is close to `T1` (little parallelism), the `T∞` term dominates and adding processors barely helps — the same wall Amdahl's law describes with a serial fraction. *Tempting wrong answer:* "more processors always cuts the time by that factor" — only the `T1/P` term shrinks; the span `T∞` is a floor. **Feed-forward:** §25.4, and Ch 26.

25.12 Machine-specific by design. A correct write-up reports: (a) a per-thread `create`+`join` cost (on the book's WSL2 machine ~220 us; bare-metal Linux is typically far lower — label your environment). (b) The pool runs the same tiny tasks at a small fraction of the thread-per-task time (here 1.04 us/task vs 217 us/task, ~200×), because creation is amortized. (c) A fork–join sum showing near-linear speedup at small `P` (here ~1.98× at 2) that bends below linear as `P` grows (here ~4.38× at 8). Warm up and average; absolute numbers are hardware/OS dependent. *Tempting wrong answer:* "my pool is broken because 8 threads didn't give 8×" — sub-linear scaling is expected and is exactly Chapter 26's subject, not a bug. **Feed-forward:** §25.1–§25.4.

Chapter 26 — Solutions

26.1 speedup(**P**) = 1/(**s** + (1−**s**)/**P**). As **P** → ∞ the (1−**s**)/**P** term vanishes and speedup approaches 1/**s** — the ceiling set by the serial fraction. *Tempting wrong answer:* “speedup grows linearly with P” — only the parallel part divides by P; the serial part is a floor. **Feed-forward:** §26.1.

26.2 False sharing is when two threads write independent variables that happen to occupy the same cache line, so each write invalidates the other's cached copy and the line ping-pongs between cores — paying the cost of sharing while sharing no data. The granularity is the **cache line**, 64 bytes on a typical x86 machine. *Tempting wrong answer:* “it's caused by sharing a variable” — no variable is shared; the collision is at cache-line granularity, not variable granularity. **Feed-forward:** §26.4.

26.3 A lock's critical section runs one thread at a time, so if every thread must pass through the same lock on the hot path, that section executes serially — it is a serial fraction, and Amdahl caps speedup at one over it. *Tempting wrong answer:* “the lock only adds a small constant overhead” — it serializes an entire fraction of the work, and Amdahl's ceiling applies to that fraction. **Feed-forward:** §26.3.

26.4 **s** = 0.20. (a) Limit = 1/0.20 = **5×** with infinite cores. (b) At **P** = 4: 1/(0.2 + 0.8/4) = 1/0.4 = **2.5×**; at **P** = 16: 1/(0.2 + 0.8/16) = 1/0.25 = **4.0×**. (c) Quadrupling cores (4→16) improved speedup only from 2.5× to 4.0× — a factor of 1.6, not 4 — so “just add cores” gives sharply diminishing returns once the serial fraction dominates. *Tempting wrong answer:* “16 cores gives 4× the speedup of 4 cores” — it gives 1.6×; the serial 20% throttles it. **Feed-forward:** §26.1.

26.5 (a) The entire work is inside the one critical section, so it is effectively 100% serial — Amdahl ceiling 1×, meaning even in the best case adding threads cannot speed it up. (b) On top of that, the lock's own **cache line** (and the contended counter's line) must be transferred to whichever core acquires it next, so it bounces between cores; that coherence traffic and the wait/hand-off overhead grow with the number of threads, pushing the time *above* the single-thread time — negative scaling. (c) Fix 1: shard into per-thread counters combined once at the end — serial fraction drops toward 0. Fix 2: a lock-free atomic or shrinking the critical section — reduces the serialized portion and the contended line. *Tempting wrong answer:* “it's slow because the mutex syscall is expensive” — uncontended mutexes are cheap userspace operations; the cost here is serialization plus cache-line bouncing under contention. **Feed-forward:** §26.3.

26.6 (a) Version B (padded) is faster — about **10×** in the §26.4 measurement (2.56 s packed vs 0.25 s padded). In version A both counters share a cache line, so each thread's write invalidates the line in the other core, forcing a re-fetch every increment (the line ping-pongs); in B each counter owns its line, so writes stay core-local. (b) Neither has a data race: each thread writes only its own counter, and no location is accessed by two threads — the collision is at the cache-line level, not the variable level, so there is no shared variable at all. (c) Fix:

```
struct { long v; char pad[64 - sizeof(long)]; } c[2];   /* one counter per line */
/* or */  _Alignas(64) long c0;  _Alignas(64) long c1;   /* C11 */
/* or GCC: */  long c[2] __attribute__((aligned(64)));   /* + stride to a line each */
```

Method-selection cue: hot per-thread data written in a tight loop → give each thread's datum its own cache line. **Feed-forward:** §26.4.

26.7 (a) Efficiency = speedup/$\mathbf{P}$ = $1/(\mathbf{P}\cdot\mathbf{s} + (1-\mathbf{s}))$. As $\mathbf{P}$ grows the $\mathbf{P}\cdot\mathbf{s}$ term grows, so efficiency falls monotonically toward 0 for any $\mathbf{s} > 0$ — you keep more cores idle relative to the serial floor. (b) When efficiency has dropped so far that the marginal core does less useful work than it costs (money, power, coordination), more cores stop being cost-effective — well before the $1/\mathbf{s}$ ceiling. (c) Under Gustafson the problem grows with $\mathbf{P}$, so the parallel work per core stays roughly constant and the serial fraction of the *larger* job shrinks; efficiency can stay high because you are not dividing a fixed parallel part into ever-thinner slices. *Tempting wrong answer:* "efficiency is constant if the program is well written" — for a fixed problem with any serial part, efficiency always falls with P; only weak scaling avoids it. **Feed-forward:** §26.1, §26.2.

26.8 **Step 4:** This is **false sharing**. It gets worse with more threads because each additional core writing its slot adds another participant fighting over the same 64-byte line — more invalidations and cross-core transfers per unit time, so coherence traffic (not useful work) grows with the thread count. The fix is to give each thread's counter its own cache line: pad the element to 64 bytes (`struct { long v; char pad[56]; } counts[N];`) or align each counter with `_Alignas(64)` / `__attribute__((aligned(64)))` so no two threads' hot counters share a line. The §26.4 measurement showed this is worth roughly a **10×** speedup (2.56 s → 0.25 s) for the same arithmetic. *Tempting wrong answer:* "add a lock around each slot to make it thread-safe" — it is already correct; a lock would add real contention on top of the false sharing and make it worse. **Feed-forward:** §26.4.

26.9 (a) **Serial fraction (Amdahl)** — a plateau at ~10× with no lock implies ~10% inherently serial work; confirm by measuring the sequential portion's share of runtime (~1/10). (b) **Lock contention** — throughput dropping with a profiler pointing at `pthread_mutex_lock` is the signature; confirm with the lock's contention/wait counters. (c) **False sharing** — per-thread array, no locks, poor scaling; confirm by padding each element to its own cache line and seeing the slowdown vanish (or via a perf counter for cache-coherence/HITM events). (d) **Serial fraction (Amdahl)** — a fixed 2 s single-threaded merge is a literal serial section; confirm it stays constant as the parallel phase's core count changes. *Method-selection cue:* plateau & no lock → Amdahl; time in lock → contention; per-thread array & no lock → false sharing. **Feed-forward:** §26.1, §26.3, §26.4.

26.10 (a) **False** — a fixed problem is capped at 1/**s** by its serial fraction, no matter the core count. (b) **False** — the laws answer different questions (fixed problem vs problem grown with the machine); they are consistent. (c) **False** — false sharing is a performance bug, not a data race; each thread writes only its own variable, so behavior is well-defined. (d) **True** — a contended hot lock serializes work and bounces a cache line, so adding threads can slow the program (negative scaling). (e) **False** — padding changes only memory layout and timing, never the computed results. *Tempting wrong answer:* calling (c) true because “threads touch nearby memory” — touching nearby memory is not a race; a race needs conflicting access to the *same* location. **Feed-forward:** §26.1–§26.4.

26.11 (a) Fix the total work **W** = serial + parallel with serial share **s**; time on **P** is **s**·**W** + (1−**s**)**W**/**P**, so speedup = 1/(**s** + (1−**s**)/**P**) → 1/**s** — Amdahl. If instead the parallel work grows with **P** (fixed wall-clock), the work done is **s** + (1−**s**)**P** relative to one processor's serial-plus-one-share, giving scaled speedup **P** − **s**(**P**−1) — Gustafson, near-linear. (b) “Linear to 1000 cores” is plausible under Gustafson if they grew the simulation with the machine (weak scaling); it does not contradict Amdahl, which governs a *fixed-size* run. Ask: “Did the problem size stay fixed as you added cores, or grow with them?” (c) The span **T∞** is the critical path that must run sequentially no matter how many processors you have — exactly the role of Amdahl's serial fraction; in $O(T_1/P + T_\infty)$ the span is the irreducible floor. *Tempting wrong answer:* “the 1000× claim disproves Amdahl” — it measures weak scaling, a different regime, not a violation. **Feed-forward:** §26.1, §26.2, and Ch 25.

26.12 Machine-specific by design. A correct write-up reports: (a) measured speedups that track Amdahl's 1/(**s** + (1−**s**)/**P**) for the measured **s** and bend toward the 1/**s** ceiling (on the book's machine **s** ≈ 0.089, ceiling ~11×, 16 threads ≈ 6.4×). (b) The single-mutex counter showing flat or negative scaling (here 1.80 s at 1 thread rising to 7.06 s at 8). (c) A packed-vs-padded false-sharing slowdown of several times (here ~10×). Label CPU and cache-line size; numbers are hardware-dependent. *Tempting wrong answer:* “my parallel loop is buggy because 8 threads gave 4×” — sub-linear scaling is expected from the serial fraction and memory effects, not necessarily a bug. **Feed-forward:** §26.1, §26.3, §26.4.

Chapter 27 — Solutions

27.1 (1) Every value has an owner. (2) There is exactly one owner at a time. (3) When the owner goes out of scope, the value is dropped (its resources freed). *Tempting wrong answer:* "a value can have several owners as long as they cooperate" — rule 2 forbids it; shared access comes later via borrowing, not multiple ownership. **Feed-forward:** §27.2.

27.2 "Dropped" means Rust automatically runs the value's cleanup at the point its owner's scope ends; for a heap-owning value it corresponds to calling `free` on the buffer in C — but inserted by the compiler, deterministically, not by hand. *Tempting wrong answer:* "drop is like a garbage collector reclaiming it later" — drop happens at a fixed, known point (end of scope), not whenever a collector runs. **Feed-forward:** §27.2.

27.3 No — `s1` has been **moved from** and is invalid; the compiler rejects any use of it with a compile error (`error[E0382]`, borrow/use of moved value). *Tempting wrong answer:* "`s1` is still usable because assignment copies" — for an owning type assignment moves, not copies. **Feed-forward:** §27.3.

27.4 (a) After `char *t = s;` and `free(s);`: the `printf("%s", t)` is a **use-after-free** (reading a freed buffer through the alias), and `free(t)` is a **double-free** (the same block returned twice). (b) It compiles because C's type system does not track ownership or validity — a `char*` carries no information about whether its target is still allocated or who may free it. (c) In Rust the analogous `let t = s;` *moves* ownership from `s` to `t`, so `s` immediately becomes invalid; there is never a state with two live owners, so no second owner exists to free the buffer again — the double-free is unrepresentable. *Tempting wrong answer:* "Rust catches it by inserting a runtime check before the second free" — it is a compile-time rejection; there is no second free to check. **Feed-forward:** §27.1, §27.3.

27.5 Rule 2 (one owner) means only one binding is ever responsible for the buffer, so it is freed exactly once — preventing the double-free. Rule 3 (drop at end of scope) means that single owner's cleanup runs automatically when its scope ends, so the free is never forgotten — preventing the leak. Rule 1 ties every value to an owner so rules 2 and 3 always apply. No GC is needed because the drop point is known at compile time and inserted there. *Tempting wrong answer:* "you still need to remember to drop it" — drop is automatic at scope exit; forgetting is not possible for an owned value. **Feed-forward:** §27.2.

27.6 (a) **Move**: ownership transfers, source becomes invalid, no heap buffer duplicated (just the small stack record is bit-copied and the source invalidated). **`clone`**: allocates and deep-copies the heap buffer, source stays valid, two independent owners of two buffers. **`Copy`**: bit-copy of a stack-only value, source stays valid, no heap involved. (b) `String` owns a heap buffer, so a copy would create two owners of one buffer (unsafe) — hence it moves; `i32` is entirely on the stack with no resource to double-free, so it implements `Copy` and is duplicated harmlessly. (c) Reaching for `.clone()` to silence a move error is wrong when you only need to *read* the value — you should borrow a reference instead of paying for a deep copy. *Method-selection cue:* owns a resource → moves; plain stack data → `Copy`; need a real second copy → `clone`; only need to look → borrow. **Feed-forward:** §27.3.

27.7 (a) The move/drop machinery is resolved at **compile time** — the compiler decides where drops go and rejects use-after-move statically; the ownership *checking* adds no run-time cost. (b) `drop`-at-end-of-scope is **deterministic** — memory is freed at a known program point — whereas a tracing GC reclaims at unpredictable times; determinism means no GC pauses and tighter, more predictable peak memory. (c) A move only bit-copies a small stack record and invalidates the source (essentially free); `.clone()` allocates a new buffer and copies the contents (real time and memory proportional to the data). *Tempting wrong answer:* “ownership adds runtime overhead like reference counting” — plain ownership is entirely compile-time; ref-counting (`Rc`/`Arc`) is a separate, opt-in mechanism. **Feed-forward:** §27.2, §27.3.

27.8 **Step 4:** Fix A (no clone): have `process` take a *reference* — `process(&cfg)` — so it borrows rather than moves; `cfg` stays owned by the caller and `log(&cfg)` (or `log(cfg)` last) works. This is right whenever the functions only need to read/inspect the value. Fix B (clone): if `process` genuinely needs to *take ownership* (e.g. to store or consume `cfg`) and you still need it afterward, pass `process(cfg.clone())` and keep the original for `log`. Decide by asking whether `process` must own the value: if not, borrow (Fix A); if it must and you also need the original, clone (Fix B). Blindly cloning is the wrong default because it silently deep-copies data you may only be reading, discarding performance for no reason. *Tempting wrong answer:* “always clone — it's simplest” — it builds needless copies into hot paths; borrowing is the usual right answer. **Feed-forward:** §27.3, and Ch 28 (borrowing).

27.9 (a) **Move** — `String` owns a heap buffer and is not `Copy`; `a` becomes invalid. (b) **Copy** — `i32` implements `Copy`; `a` stays usable. (c) **Move** — `Vec<u8>` owns a heap buffer; passing by value moves it and the caller can no longer use it. (d) **Copy** — a tuple of `Copy` types (`i32`, `bool`) is itself `Copy`, so it copies and `a` stays usable. *Method-selection cue:* owns heap → move; all-stack `Copy` types → copy. **Feed-forward:** §27.3.

27.10 (a) **False** — Rust has no garbage collector; safety comes from compile-time ownership. (b) **False** — `s1` is moved from and invalid; only `s2` is valid. (c) **True** — move errors are compile-time and add no runtime cost. (d) **False** — `clone` deep-copies into a new buffer (both valid); a move transfers ownership and invalidates the source (no buffer copied). (e) **False** — ownership controls aliasing, which is also the basis for preventing data races. *Tempting wrong answer:* marking (a) true because “safe languages use GC” — Rust is the counterexample; it is safe without one. **Feed-forward:** §27.1–§27.4.

27.11 (a) Moving a value into a thread transfers ownership to that thread; by rule 2 the original thread is no longer an owner and cannot touch the value, so two threads cannot both own-and-mutate it — the accidental sharing that causes races is structurally excluded. (b) A use-after-free is one alias freeing/mutating memory another alias still uses; a data race is two threads accessing the same location with at least one write, unsynchronized — both are "two paths to the same data, at least one mutating." Ownership attacks that shared root by ensuring a single responsible owner, rather than patching use-after-free and races as separate problems. (c) Ownership-by-move alone won't let you *use* a value without giving it away, so you can't, e.g., let two functions read the same buffer without moving/cloning; you need **borrowing** (references), which must guarantee that references never outlive their value and that you never have a mutable reference aliased with any other reference — keeping aliasing safe without transferring ownership. *Tempting wrong answer:* "ownership by itself is enough to write real programs" — without borrowing you'd move or clone constantly; borrowing is what makes it practical. **Feed-forward:** §27.4, and Ch 24, Ch 28.

27.12 Toolchain-specific. A correct write-up reports: (a) the C snippet compiles with no warning and, at run time, aborts on the double-free (glibc "double free detected", SIGABRT) or, under `-fsanitize=address`, is flagged as a heap-use-after-free — demonstrating the C type system misses it. (b) With `rustc`, the `let s2 = s1; println!("{s1}")` program fails to compile with `error[E0382]` on the `println!` line (use of moved value); without `rustc`, name that exact line and explain the move at `let s2 = s1;` invalidated `s1`. (c) For three variables, correctly label each assignment move or copy from its type (owning type → move; `Copy` stack type → copy). *Tempting wrong answer:* "the C program is fine because it compiled" — compiling proves nothing about UB; it aborts (or corrupts) at run time. **Feed-forward:** §27.1, §27.3.

Chapter 28 — Solutions

28.1 A **shared reference** `&T` is a read-only borrow; any number may exist at once, and you may read but not mutate the value through one. A **mutable reference** `&mut T` is a read-write borrow; exactly one may exist, and while it lives no other reference (of either kind) to the value may. *Tempting wrong answer:* “`&mut` is just a pointer, like `&` but you can write” — it is that plus an *exclusivity* guarantee the compiler enforces. **Feed-forward:** §28.2, §28.3.

28.2 `&String` borrows: passing `&owner` lends a reference without moving the value, so the caller keeps ownership and `owner` stays valid. `fn length(s: String)` takes by value, which *moves* the `String` into the function and invalidates `owner`. *Tempting wrong answer:* “both work because the function only reads” — reading is irrelevant; taking by value moves regardless of what the body does. **Feed-forward:** §28.1.

28.3 The rule: at any time a value may be borrowed by *any number of shared references* (`&T`) *or* by *exactly one mutable reference* (`&mut T`), never both. It forbids two mutable borrows at once (`E0499`) and a mutable borrow while a shared borrow is live (`E0502`). *Tempting wrong answer:* “you can mix one `&mut` with a few `&` if they don't overlap in code” — if they are simultaneously live, that is exactly `E0502`. **Feed-forward:** §28.3.

28.4 (a) The bug is the `printf("via p", *p)` line: after `realloc` may have moved the block, `p` is a dangling pointer, so the dereference is a use-after-free. (b) Plain `gcc -O2 -Wall` gives no warning because C's type system does not track whether `p` still points at live memory after `realloc`; only AddressSanitizer, at run time, reports **heap-use-after-free**. (c) In Rust `p = &v[0]` is a shared borrow of `v`, and `v.push(4)` needs `&mut v`; the mutable borrow cannot coexist with the live shared borrow, so it is `E0502` at compile time. The checker does not need to know `push` reallocates — the rule “no mutable borrow while a shared borrow is live” is sufficient. *Tempting wrong answer:* “Rust catches it because it checks the reallocation at run time” — it is a static rejection with no runtime check. **Feed-forward:** §28.4.

28.5 (a) Two references to one value with at least one mutable would let one path write while another reads or writes — a torn or unexpectedly-changed read, or a pointer invalidated under the other, all in one thread. (b) That is the definition of a data race (Chapter 24) minus the threads: two accesses to one location, at least one a write, unsynchronized. (c) Exclusivity lets the compiler assume nothing else can see or change the value through the `&mut`, which makes mutation safe and enables optimizations (no aliasing to worry about). *Tempting wrong answer:* “exclusivity is just a style rule” — it is a soundness requirement; without it Rust could not rule out aliasing-plus-mutation. **Feed-forward:** §28.3, §28.4.

28.6 (a) Compiles — many shared references are allowed. (b) `E0499` — two mutable borrows at once. (c) `E0502` — mutable borrow while a shared borrow is live. (d) Compiles — because the shared borrow has ended (last use) before the mutable borrow starts, they do not overlap (non-lexical lifetimes). *Method-selection cue:* count simultaneously-live borrows; many `&` ok, one `&mut` alone ok, any overlap of a writer with anything else is rejected. **Feed-forward:** §28.2, §28.3.

28.7 (a) A `&T` is, at the machine level, just a pointer — the same pointer C would use — carrying no extra runtime data (for a sized `T`). (b) The borrow checking happens entirely at compile time; it adds zero runtime cost. (c) `RefCell` enforces the same one-writer-XOR-many-readers rule at run time, so it carries a borrow-flag it checks and updates on each borrow (a small time cost and a possible panic on violation); paying it is justified only when the borrow structure cannot be proven statically but you still need shared mutability. *Tempting wrong answer:* "references are cheap because they're reference-counted" — plain references are not counted at all; that is `Rc`/`Arc`, a separate mechanism. **Feed-forward:** §28.2, §28.3.

28.8 **Step 4:** Fix A (separate the passes): during a read-only pass, collect what you want to add into a new vector (`let additions: Vec<i32> = v.iter().map(|x| 2 * x).collect();`), then, after that shared borrow ends, `v.extend(additions)` under a single mutable borrow. Fix B (in-place, if that were the task): use `for x in v.iter_mut() { *x *= 2; }`, which is one exclusive borrow and is allowed because there is no aliasing. Cloning the entire vector to iterate a copy while mutating the original is the wrong default because it pays a full O(n) copy to hide a real hazard; separating read from write costs nothing extra and is what the rule is pointing at. *Tempting wrong answer:* "just `v.clone()` and iterate that" — needless copy, and it silently changes semantics (you iterate the old contents). **Feed-forward:** §28.4.

28.9 (a) Allowed — many shared references may coexist (no writer). (b) Forbidden — a mutable and a shared borrow simultaneously is `E0502`. (c) Forbidden — two mutable borrows is `E0499`. (d) Allowed — the shared borrows have ended, so the mutable borrow is the only live one. *Method-selection cue:* the only legal states are "N readers, 0 writers" or "0 readers, 1 writer." **Feed-forward:** §28.3.

28.10 (a) **False** — at most one `&mut` may exist at a time, regardless of timing (`E0499`). (b) **False** — a shared reference is read-only; mutation needs `&mut`. (c) **False** — borrow checking is compile-time; a dereference is just a pointer load. (d) **True** — that is the iterator-invalidation hazard; it is `E0502`. (e) **False** — a reference does not own the value and drops nothing; only the owner drops it. *Tempting wrong answer:* marking (c) true because "safety must cost something at run time" — the borrow rule's cost is paid entirely at compile time. **Feed-forward:** §28.2–§28.4.

28.11 (a) A use-after-free through an aliased pointer is one alias mutating/freeing memory another alias still reads; a data race is two threads accessing one location, one writing, unsynchronized — both are “two paths to one value, at least one writing.” The single rule “aliasing XOR mutation” forbids that combination, attacking the shared root rather than the two symptoms. (b) Unlimited shared references are safe because with no writer no reader can observe a changing value — reads never conflict with reads. Access becomes a problem only when a write is introduced, which the rule permits solely under exclusivity. (c) To extend “no data races” across threads you need the `Send`/`Sync` traits (and, for shared mutation, synchronization such as `Arc<Mutex<T>>`): the borrow rule governs one value in one thread, but sending references between threads requires the type system to also track which types are safe to share or move across threads. *Tempting wrong answer:* “the borrow rule alone already prevents all data races” — it prevents intra-thread aliasing; cross-thread safety needs `Send`/`Sync` on top. **Feed-forward:** §28.4, and Ch 24; the concurrency chapter ahead.

28.12 Toolchain-specific. A correct write-up reports: (a) the C snippet builds with no warning under `-O2 -Wall` and, rebuilt with `-fsanitize=address`, reports **heap-use-after-free** when the old pointer is dereferenced after a moving `realloc` — the C type system misses the dangling alias. (b) With `rustc`, the `let p = &v[0]; v.push(4); println!("{p}")` program fails with `error[E0502]` because `push`'s mutable borrow conflicts with the live shared borrow `p`; without `rustc`, name those two conflicting borrows. (c) The compiling version ends the shared borrow (use `p` before the `push`, or index `v[0]` directly and copy the value out) so no borrow overlaps the mutable one. *Tempting wrong answer:* “the C program is fine because it compiled without warning” — no warning proves nothing about UB; the sanitizer shows the use-after-free. **Feed-forward:** §28.4.

Chapter 29 — Solutions

29.1 A **lifetime** is a compile-time name for the region of the program over which a reference is valid (bounded by its referent's scope). The second borrow rule it enforces: a reference may never be used after the value it refers to has been dropped. *Tempting wrong answer:* "a lifetime is how long the value lives, which you can set" — it only *describes* existing scope facts; it cannot make anything live longer. **Feed-forward:** §29.1, §29.3.

29.2 `x` is a stack local dropped when the function returns, so the returned `&x` would point at reclaimed storage — a dangling reference / use-after-scope. *Tempting wrong answer:* "it's fine because `5` is a small constant" — the value's size is irrelevant; its storage is gone once the frame is popped. **Feed-forward:** §29.1.

29.3 The three `'a`s assert: there is some region `'a` such that both inputs `x` and `y` are valid for at least `'a`, and the returned reference is valid for `'a` — so the result may not outlive the shorter-lived of the two inputs. *Tempting wrong answer:* "`'a` makes both inputs live equally long" — it constrains the output's validity relative to the inputs; it does not change how long anything lives. **Feed-forward:** §29.2.

29.4 (a) Both return (directly or through a struct field) the address of a stack local whose storage is reclaimed when the function returns, so dereferencing it in the caller is a use-after-scope. (b) `gcc`'s `-Wreturn-local-addr` follows simple direct data flow and catches `return &local;`, but gives up when the address is stored in a struct and returned indirectly, so that version builds clean. (c) Rust's rule is not a heuristic — the borrow checker rejects *every* reference used past its referent's drop; the analogous Rust function is `error[E0597]`, "does not live long enough." *Tempting wrong answer:* "the struct version is safe because it compiled without warning" — ASan shows stack-use-after-scope; no warning proves nothing. **Feed-forward:** §29.1, §29.3.

29.5 (a) The containment condition: for every use of a reference, the region over which the reference is used must be contained within the region its referent is alive. (b) You can't fix it by annotating a longer lifetime because a lifetime only describes existing scope facts — no name can extend a value past its owner's drop. (c) The two structural fixes: return an *owned* value (move ownership out — right when the caller should own the result), or borrow from something that genuinely outlives the use (an input reference or an outer-scope value — right when the data lives elsewhere). *Tempting wrong answer:* "annotate it `'static`" — that only compiles if the data really is static, which a fresh local isn't. **Feed-forward:** §29.3, §29.4.

29.6 The rules: (1) each elided input reference gets its own lifetime; (2) one input lifetime is assigned to all elided outputs; (3) if one input is `&self`/`&mut self`, its lifetime is assigned to all elided outputs. (a) `first` has one input reference, so rule 2 assigns its lifetime to the output — no annotation needed. (b) `get` is a method with `&self`, so rule 3 gives the output `self`'s lifetime. (c) `longest` has two input references and neither is `self`, so rules 2 and 3 don't apply and the compiler can't tell which input the output borrows — you must write `'a`. *Method-selection cue:* one input ref → elided; method with `self` → elided; multiple non-self input refs feeding an output ref → annotate. **Feed-forward:** §29.2, §29.4.

29.7 (a) Lifetimes are checked during borrow-checking (compile time) and are *erased* before code generation. (b) A `&'a T` has the same runtime representation as the C pointer it replaces — a bare address, no extra data (for sized `T`). (c) A GC answers "still valid?" by tracing/reachability at run time (time cost, pauses); reference counting answers it by per-reference counter updates at run time (time and space); Rust answers it once at compile time, paying nothing at run time. *Tempting wrong answer:* "lifetimes are a runtime tag on each reference" — they are purely static and erased. **Feed-forward:** §29.3.

29.8 **Step 4:** Correct fix — change the return type to an owned `String` and return `s` by value: `fn greeting() -> String { String::from("hi") }`; ownership moves out to the caller, so there is no dangling reference. Alternative — if the caller already owns suitable data, take it as an input reference and return a slice of that: `fn greeting(src: &str) -> &str`, borrowing from something that outlives the call. Reaching for `'static` is wrong because a freshly built `String` is not static data; `'static` would either fail to compile or force an unintended constraint. *Tempting wrong answer:* "annotate the return `&'static str`" — the local isn't static, so this doesn't fix the dangle. **Feed-forward:** §29.3, and Ch 27 (moving ownership out).

29.9 (a) **Elided** — one input reference; rule 2 assigns its lifetime to the output. (b) **Required** — a struct with a reference field must name its lifetime (`&'a u8`), else `E0106`. (c) **Dangling-reference error** — returning a reference to a local is `E0597`. (d) **Required** — two non-self input references feeding an output reference; elision can't choose, so annotate `'a`. *Method-selection cue:* reference field → annotate; return-a-local → error; multiple input refs to one output ref → annotate; single input ref → elided. **Feed-forward:** §29.2, §29.4.

29.10 (a) **False** — `'static` asserts the referent already lives for the whole program; it does not make an arbitrary value live forever. (b) **False** — lifetimes are erased before code generation; a reference is a bare pointer with no extra data. (c) **False** — a struct with a `&str` field must name a lifetime, else `E0106`. (d) **True** — `E0597` fires precisely when a reference's use would extend past its referent's drop. (e) **False** — elision handles most signatures; explicit lifetimes are the exception. *Tempting wrong answer:* marking (b) true because "the compiler must store the lifetime somewhere" — it stores it only during checking, then erases it. **Feed-forward:** §29.3, §29.4.

29.11 (a) A struct storing a `String` owns its data — it costs an allocation but carries no lifetime parameter and may be used freely. A struct storing `&'a str` avoids the allocation but gains a lifetime parameter and may not outlive the `str` it borrows — every use is constrained by that borrow. (b) A self-referential struct is hard because the borrowing field's lifetime would have to name the struct's own storage, which moves (Chapter 27) — after a move the internal pointer would dangle, so plain lifetimes cannot express it safely (it requires pinning or indirection). (c) The own-vs-borrow choice is exactly C's "who owns this buffer, and who may free it" question (Chapters 9, 27); lifetimes make that question explicit in the type and let the compiler check the answer, instead of leaving it to convention. *Tempting wrong answer:* "always borrow to avoid allocations" — borrowing threads lifetimes through everything and forbids outliving the source; owning is often the simpler, correct choice. **Feed-forward:** §29.4, and Ch 27.

29.12 Toolchain-specific. A correct write-up reports: (a) under `-O2 -Wall` the direct `return &local;` warns with `-Wreturn-local-addr` while the struct-laundered version builds without warning, and `-fsanitize=address` flags the latter as **stack-use-after-scope** at run time. (b) With `rustc`, `fn dangle() -> &str { let s = String::from("hi"); &s }` fails with `error[E0597]`, "`s` does not live long enough"; without `rustc`, name the `&s` return line and explain `s` is dropped at the closing brace. (c) The `longest` function needs `<'a>` because of two input references; a struct with a `&str` field needs `<'a>` too, and omitting it is `E0106`, "missing lifetime specifier." *Tempting wrong answer:* "the struct C version is fine — no warning" — ASan proves the use-after-scope. **Feed-forward:** §29.1, §29.4.

Chapter 30 — Solutions

30.1 A **generic function** is written once over a type parameter `T` and reused at many concrete types. A **trait** names a set of behaviors a type can implement. A **trait bound** (`T: Trait`) requires a generic's type parameter to implement a trait, and in return lets the body use that trait's operations. *Tempting wrong answer:* “a trait is a base class you inherit from” — a trait is a set of required behaviors a type opts into, not an inheritance hierarchy. **Feed-forward:** §30.2, §30.3.

30.2 **Monomorphization** is the compiler emitting a separate specialized copy of a generic for each concrete type it is used with. After it, there is no type parameter at run time — each call goes to an ordinary type-specific function with direct, inlinable operations — so there is no runtime dispatch. *Tempting wrong answer:* “generics carry a type tag checked at run time” — the type is resolved and erased at compile time. **Feed-forward:** §30.2.

30.3 `error[E0277]` — “the trait bound `` `T: Trait` `` is not satisfied” — meaning you used a type that doesn't implement a trait in a place that required it. *Tempting wrong answer:* “it's a type-mismatch error, so cast the type” — the type isn't mismatched; it lacks the required behavior, so you implement the trait or relax the bound. **Feed-forward:** §30.3.

30.4 (a) `qsort` is generic in that it sorts arrays of any element type via `void*`; its two costs are lost type safety (element type erased to `void*`) and an indirect call through the comparator function pointer per comparison. (b) Passing, say, a `double` comparator while sorting `int`s compiles because both are `void*` to `qsort`; at run time the comparator reinterprets the bytes wrongly and produces a garbage ordering, and the compiler cannot catch it because the types were erased. (c) The `qsort` function-pointer mechanism corresponds to a Rust **trait object** (dynamic dispatch); **generics with monomorphization** give the duplication-free, fully-checked, fully-inlined alternative. *Tempting wrong answer:* “`qsort` is type-safe because you cast inside the comparator” — the cast is unchecked; a wrong-typed comparator still compiles. **Feed-forward:** §30.1, §30.4.

30.5 (a) The caller must supply a type `T` that implements `Summary`. (b) The body may call `summarize` (and any other `Summary` method) on `item` — impossible without the bound, since otherwise the compiler couldn't know `T` has it. (c) Rust checks the bound at the generic's *definition*, so the generic is verified once against its stated requirements; C++ checks at instantiation, so an unmet requirement surfaces as errors deep inside the template body, per call site. *Tempting wrong answer:* “the bound is just documentation” — it is enforced; violating it is `E0277`. **Feed-forward:** §30.3.

30.6 (a) Static dispatch (generics) resolves at compile time — a direct, inlinable call, zero dispatch cost; dynamic dispatch (trait objects) resolves at run time through a vtable — an indirect call per method, not inlinable. (b) A trait object is a pair of pointers: one to the value, one to the vtable of the trait's method implementations for that concrete type. (c) Dynamic dispatch is required for a heterogeneous collection (e.g. `Vec<Box<dyn Draw>>`) or run-time-chosen implementations; static dispatch is preferable for a hot loop over a single known type where inlining matters. *Method-selection cue:* known type(s) at compile time and speed matters → static; heterogeneity or run-time choice → dynamic. **Feed-forward:** §30.4.

30.7 (a) Monomorphization costs **binary size** (and compile time): a separate code copy per concrete type, which dynamic dispatch avoids by keeping one copy. (b) Dynamic dispatch costs an **indirect call per method invocation** (an indirect branch, not inlinable) that a monomorphized generic does not. (c) For a small method in a tight loop over a homogeneous collection, choose static dispatch — inlining and the absence of an indirect call dominate; for a large plugin interface with many implementations, choose dynamic dispatch — one code copy and run-time flexibility outweigh per-call cost, and monomorphizing over many types would bloat the binary. The deciding factor is per-call hot-path cost vs. code-size/heterogeneity. *Tempting wrong answer:* "always use dynamic dispatch to keep binaries small" — that pays indirect calls everywhere, including hot paths where static dispatch is free. **Feed-forward:** §30.2, §30.4.

30.8 **Step 4:** Corrected signature: `fn largest<T: PartialOrd + Copy>(list: &[T]) -> T` — the minimal bounds the body uses (`PartialOrd` for `>`, `Copy` for copying elements out). Bounding at the definition, not the call site, is what lets the compiler verify the generic *once*: the body is checked against the declared requirements, and every call is then checked to supply a satisfying type, so there is no per-call re-checking of the body (the C++ failure mode). Casting or wrapping to silence `E0277` is wrong because the error isn't a type mismatch — it's that the body needs a capability the type must actually provide; the fix is to state the bound (or implement the trait), not to cast. *Tempting wrong answer:* "add `T: Any` and downcast" — that throws away static checking to paper over a missing bound. **Feed-forward:** §30.3.

30.9 (a) **Static** — a hot numeric loop wants inlining and no indirect call; the type is known. (b) **Dynamic** — one `Vec` of mixed shape types needs a trait object (`Box<dyn Draw>`); a single monomorphized type can't hold different concrete types. (c) **Static** — one concrete type; monomorphization gives a direct call and no code bloat. (d) **Dynamic** — plugins chosen at run time can't be known at compile time, so a trait object is required. *Method-selection cue:* heterogeneity or run-time choice forces dynamic; otherwise static wins on cost. **Feed-forward:** §30.4.

30.10 (a) **False** — a generic is monomorphized; there is no type parameter or dispatch at run time. (b) **True** — a trait object dispatches through a vtable at run time. (c) **False** — `E0277` means the type doesn't implement a required trait; fix by implementing it or relaxing the bound. (d) **False** — monomorphization can make binaries *larger* (a copy per type); it trades size for speed. (e) **False** — a trait object is still fully type-checked (it can only hold a type that implements the trait), unlike C's `void*`. *Tempting wrong answer:* marking (e) true by analogy to `void*` — the analogy holds for the dispatch mechanism, not for type safety. **Feed-forward:** §30.2–§30.4.

30.11 (a) `Copy` is a trait; whether `let y = x;` copies or moves is decided by whether `T` implements it — exactly the same mechanism as `T: PartialOrd` deciding whether `>` is allowed: a capability named as a trait, required in the type, checked by the compiler. (b) `Option<T>` and `Result<T, E>` are generic enums; being generic, they carry the success value's type and force the caller to pattern-match/handle the absent or error case, making "did you check the return?" a compile-time question. (c) `T: Send` would mean "a value of `T` is safe to transfer to another thread"; requiring it at a thread-spawn boundary means the compiler refuses to move a non-thread-safe value across threads, so the aliasing-plus-mutation that causes a data race can't be set up — a compile error instead of undefined behavior. *Tempting wrong answer:* "traits are only for shared method behavior like `summarize`" — they also encode capabilities and guarantees (`Copy`, `Send`, `Sync`). **Feed-forward:** §30.5; the error-handling and concurrency chapters ahead, and Ch 24.

30.12 Toolchain-specific. A correct write-up reports: (a) the `qsort` demo prints the sorted array (e.g. `1 2 5 7 9`), and passing a comparator for a different element type still compiles because `qsort`'s parameters are `void*` — demonstrating type erasure. (b) With `rustc`, `largest<T: PartialOrd + Copy>` compiles at two types; removing the bounds yields `error[E0277]` on the comparison/copy lines (the `PartialOrd`/`Copy` bound is missing); without `rustc`, name those lines and bounds. (c) A `Vec<Box<dyn Draw>>` can hold two different implementors because each is a trait object dispatched via vtable; a single monomorphized generic `Vec<T>` is one concrete type and cannot hold two different types at once. *Tempting wrong answer:* "the wrong-typed `qsort` comparator is fine because it compiled" — it compiles but reinterprets bytes wrongly at run time. **Feed-forward:** §30.1, §30.4.

Chapter 31 — Solutions

31.1 `Option<T>` represents a value that may be absent — variants `Some(T)` and `None` — replacing the null pointer. `Result<T, E>` represents an operation that may fail with a reason — variants `Ok(T)` and `Err(E)`. *Tempting wrong answer:* "`None` is Rust's null pointer" — `None` is a variant the type system tracks, not a pointer value that inhabits every reference type; you cannot dereference it. **Feed-forward:** §31.2, §31.3.

31.2 On an expression `x` of type `Result<T, E>`, `x?` evaluates to the inner `T` if `x` is `Ok(T)`, and otherwise performs an early *return* of the `Err(E)` from the enclosing function (converting the error type through `From` if needed). *Tempting wrong answer:* "`?` panics on `Err`" — that is `unwrap()`; `?` returns the error to the caller instead. **Feed-forward:** §31.4.

31.3 `Result` is for *recoverable* failures the caller should handle; `panic!` is for *unrecoverable* bugs or violated invariants, and it unwinds the stack and aborts the thread. `unwrap()` is a shortcut for "if this is `Err`, treat it as a bug and `panic!`". *Tempting wrong answer:* "`panic!` is just Rust's exception you catch upstream" — it is meant for unrecoverable conditions, not routine control flow; recoverable errors use `Result`. **Feed-forward:** §31.4.

31.4 (a) The program did not check `fopen`'s return value; C signals that failure by returning a `NULL` pointer (and setting `errno`). (b) `-Wall` did not warn because checking a returned sentinel is a convention the compiler does not enforce; the warning it emitted was the unrelated `-Wunused-result` for ignoring `fgets`'s own `warn_unused_result` return. (c) The `NULL` `fp` is dereferenced inside `fgets`, causing a segmentation fault (exit 139); dereferencing an invalid pointer is undefined behavior (Chapter 10), so the compiler may assume it never happens and no diagnostic is required. *Tempting wrong answer:* "the warning means the compiler caught the NULL bug" — the warning was about `fgets`'s return, not the unchecked `fp`. **Feed-forward:** §31.1, §31.3.

31.5 (a) The success value lives inside the `Ok(T)` variant, so to reach a usable value you must write code that also names the `Err` case (a `match`, a combinator, or `?`). (b) `#[must_use]` makes the compiler *warn* if you discard the whole `Result` without using it — the enforcement C's convention lacks. (c) `unwrap()` is the closest analog to skipping C's check because it reaches the success value while turning any `Err` into a `panic!`; it is at least visible and greppable, unlike C's silent fall-through. *Tempting wrong answer:* "`unwrap()` is safe because it can't segfault" — it can't corrupt memory, but it still crashes the thread on a recoverable error. **Feed-forward:** §31.3.

31.6 (a) `?` propagates an `Err` by returning it from the enclosing function, but `load` returns `()`, which has nowhere to put a propagated error — so the operation is ill-typed and reported as `E0277`. (b) Corrected: `fn load(path: &str) -> Result<(), io::Error>` (with `Ok(())` at the end). (c) Adding `.unwrap()` compiles but changes the semantics: a missing file — a recoverable error — now `panic!`s and aborts the thread instead of being handed to the caller who could recover. *Tempting wrong answer:* "`E0277` means `File::open` returned the wrong type" — the operand type is fine; the enclosing function's return type is what disallows `?`. **Feed-forward:** §31.4.

31.7 (a) In C the worst case is undefined behavior at run time — the NULL dereference and segfault of §31.1, or worse, silent corruption. (b) In Rust, dropping the `Result` is caught at compile time as a `#[must_use]` *warning* (the value is unused, no crash), whereas reflexively `unwrap()`ing it downgrades a recoverable error to a run-time `panic!` that aborts the thread — bad, but a controlled unwind, not UB. (c) Ordered by severity: compile-time diagnostic (safest, before the program runs) < warning < controlled `panic!` < runtime UB (worst). This ordering matters because a system you cannot easily debug in production fails most cheaply when the compiler catches the omission, and most expensively when it is silent undefined behavior. *Tempting wrong answer:* "a Rust panic is as bad as a C segfault" — a panic is a defined, controlled unwind with a message; UB is undefined and may corrupt silently. **Feed-forward:** §31.1, §31.4.

31.8 Step 4: Corrected: `fn first(path: &str) -> Result<String, io::Error> { let s = std::fs::read_to_string(path)?; Ok(s) }` — the function now returns a `Result`, so the `io::Error` that `?` may propagate has somewhere to go, and the success value is wrapped in `Ok`. This is preferable to `.unwrap()` because a missing or unreadable file is a routine, recoverable condition the caller may want to handle (fall back to defaults, report to the user); `unwrap()` would `panic!` and abort instead. *Tempting wrong answer:* "return `String` and `unwrap()` so the signature stays simple" — that hides a recoverable failure behind a crash. **Feed-forward:** §31.4.

31.9 (a) **`Option`** — a key's presence is a plain absent/present question with no error reason (or `Result` only if the lookup itself can fail). (b) **`Result`** — malformed input is an expected, recoverable failure carrying a reason. (c) **`panic!`** is acceptable — an out-of-range index you just proved in-range would be a bug, so a panic (or plain indexing, which panics) documents the invariant; alternatively return `Result` if the proof is not airtight. (d) **`Result`** — a user-specified file may legitimately be missing or unreadable; the caller should recover. *Method-selection cue:* absence with no reason → `Option`; expected failure with a reason → `Result`; a violated invariant/bug → `panic!`. **Feed-forward:** §31.2–§31.4.

31.10 (a) **False** — `None` is an `Option` variant the type system tracks, not a null pointer that inhabits every reference type. (b) **False** — `?` is only valid in a function returning `Result` or `Option` (else `E0277`). (c) **False** — dropping a `Result` is a `#[must_use]` *warning*, not an error. (d) **False** — `unwrap()` on `Err` `panic!`s; `unwrap_or` is the one that returns a default. (e) **True** — `?` converts the error type through the `From` trait on the way out. *Tempting wrong answer:* marking (c) true by analogy to (b) — dropping a value is a warning, using `?` in the wrong context is the hard error. **Feed-forward:** §31.2–§31.4.

31.11 (a) Using `?` states a requirement on the enclosing function: “this function can absorb a propagated error” — i.e. its return type implements the residual/conversion machinery (built on `From`). That is a trait requirement, so its violation is the same `E0277` (“trait bound not satisfied”) as an unmet bound in Chapter 30. (b) Because `Option<T>` and `Result<T, E>` are *generic*, one `?` operator is defined over all their instantiations at once — it works uniformly on `Result<File, io::Error>`, `Result<String, ParseError>`, and every other fallible return without per-type code, exactly the reuse monomorphization gives. (c) A worker's result would travel as a `Result<Output, E>` (“the work, or why it failed”); a thread `panic!` is observed by the parent as an `Err` from `JoinHandle::join`, carrying the panic payload. *Tempting wrong answer:* “`?`'s error is a special language feature unrelated to traits” — it is built on the `From`/residual traits, which is why the diagnostic is a trait-bound error. **Feed-forward:** §31.5; the concurrency chapter ahead, and Chapter 30.

31.12 Toolchain-specific. A correct write-up reports: (a) `cerr.c` compiles (with only the `-Wunused-result` warning) and, on a missing file, terminates with SIGSEGV (exit status 139); adding `if (fp == NULL) { perror(...); return 1; }` converts the crash into a clean error message and non-crashing exit. (b) With `rustc`, `read_config` returning `Result<String, io::Error>` returns the error on a missing path (no crash); changing its return type to `()` produces `error[E0277]` on the `?` line. Without `rustc`, name the `?` line and the return type as the cause. (c) Replacing a `?` with `.unwrap()` changes a propagated, recoverable error into a thread-aborting `panic!`. *Tempting wrong answer:* “since `cerr.c` compiled, the NULL path is fine” — it compiles but is undefined behavior at run time. **Feed-forward:** §31.1, §31.4.

Chapter 32 — Solutions

32.1 `Send` means a value of the type is safe to *move* (transfer) to another thread; `Sync` means it is safe to *share* a reference to it across threads. The relationship: `T: Sync` exactly when `&T: Send`. *Tempting wrong answer:* "`Send` is for moving and `Sync` is for locking" — `Sync` is about shared references being sendable, not about acquiring locks. **Feed-forward:** §32.2.

32.2 A **data race** is two threads accessing the same memory concurrently, at least one writing, with no synchronization ordering the accesses. In C (and C++) it is *undefined behavior* (Chapter 24). In safe Rust it cannot occur: the type system (the `Send`/`Sync` bounds plus the borrow rules) rejects the unsynchronized sharing at compile time. *Tempting wrong answer:* "a data race just gives a wrong answer" — in C it is UB, so the program has no defined meaning at all, not merely a wrong number. **Feed-forward:** §32.1, §32.3.

32.3 `move` forces the closure to take *ownership* of the variables it uses rather than borrowing them, so the data lives as long as the thread. Without it, a closure that borrows a local and is handed to a thread that may outlive the borrow draws `error[E0373]` ("closure may outlive the current function"). *Tempting wrong answer:* "`move` copies the captured data" — it transfers ownership (a move), which for non-`Copy` types invalidates the original. **Feed-forward:** §32.3.

32.4 (a) It compiled because C has no rule against two threads touching shared mutable state; the data race is invisible to the compiler. (b) Each `counter++` is load–add–store; when two threads read the same value before either stores, one increment overwrites the other and is lost, so the total is below `4000000`, and because the interleaving is nondeterministic the exact loss differs each run. (c) The C fix is a mutex around every access to `counter`; it is fragile because it relies on you remembering the lock at *every* access site — one omission silently reintroduces the race, whereas Rust makes the unsynchronized access unrepresentable. *Tempting wrong answer:* "the total is low because the loop ran too few times" — the loop ran fully; updates were lost to interleaving. **Feed-forward:** §32.1, §32.4.

32.5 (a) `Send` rules out moving into the new thread any value that is not safe to transfer across threads — including any setup that would give two threads unsynchronized mutable access. (b) `'static` rules out the closure holding a borrow of data owned by the parent frame, which could dangle once the parent returns — the lifetime rule of Chapter 29. (c) An `Rc` is not `Send` (non-atomic refcount), so capturing it in a `spawn` closure leaves the closure non-`Send` and the bound fails. *Tempting wrong answer:* "`'static` means the thread runs forever" — `'static` bounds the *lifetime of borrows* the closure may hold, not the thread's duration. **Feed-forward:** §32.2.

32.6 (a) `E0373` reports that a closure would outlive a value it borrows — an instance of the Chapter 28–29 borrowing/lifetime rules applied to a closure escaping to a thread; `E0277` reports an unsatisfied `Send` (or `Sync`) bound — an instance of the Chapter 30 trait-bound check applied to `Send`. (b) Fix `E0373` with `move` (take ownership); fix `E0277` by using the thread-safe type (`Arc` for `Rc`, an atomic for a plain counter). (c) `unsafe impl Send` merely *asserts* thread-safety the type does not have, disabling the check and reintroducing the race — an unearned promise, not a fix. *Tempting wrong answer:* "`E0373` and `E0277` are the same borrow error" — one is a lifetime/borrow problem, the other a trait-bound problem, with different fixes. **Feed-forward:** §32.3.

32.7 (a) The `Mutex` costs a lock acquire/release per update; as thread count rises, **lock contention** (Chapter 26) grows and can serialize the threads, throttling throughput. (b) The channel costs a send/receive and the move of the value; moving ownership down the channel means the sender can no longer touch the value, so there is nothing to race on and no lock to contend. (c) For a high-contention counter, an **atomic** (Chapter 22, e.g. a lock-free `fetch_add`) avoids the lock entirely; you would still prefer the `Mutex` when each update touches *multiple* fields that must change together atomically, which a single atomic cannot express. *Tempting wrong answer:* "channels are always faster than a mutex" — for a single hot counter an atomic or even a mutex may beat channel overhead; it depends on the access pattern. **Feed-forward:** §32.4.

32.8 **Step 4:** Replace `Rc` with **`Arc`**: `let c = Arc::new(Mutex::new(0));` and, inside the loop, `let cc = Arc::clone(&c);` before the `move` closure. This is a real fix because `Arc`'s reference count is updated with atomic operations, so it is genuinely safe to send/share across threads and therefore `Send + Sync` — the bound is now satisfied by fact, not by fiat. `unsafe impl Send for ...` would instead assert `Send` for the non-atomic `Rc`, switching off the check while the underlying refcount race remains — silencing the diagnostic without removing the bug. *Tempting wrong answer:* "wrap the `Rc` in a `Mutex` too" — the problem is the non-atomic refcount of `Rc` itself, not the interior value; you need `Arc`. **Feed-forward:** §32.3.

32.9 (a) **Prevented** — sharing an `i32` mutably across threads requires either a non-`Sync` share or aliasing a `&mut`, both rejected; you must use a `Mutex`/atomic. (b) **Not prevented** — deadlock from opposite lock-ordering is a logical bug, not a data race; the type system does not analyze lock order. (c) **Prevented** — `Rc` is not `Send`, so sending it across threads is `E0277`. (d) **Not prevented** — holding a guard across a slow call is safe (no race) but a contention/granularity problem the compiler does not flag. *Method-selection cue:* data races (unsynchronized shared mutation) are prevented; higher-level bugs (deadlock, contention, granularity) are not. **Feed-forward:** §32.3, §32.4.

32.10 (a) **False** — `Send`/`Sync` are marker (auto) traits with no methods, derived structurally. (b) **False** — Rust's `Mutex` *owns* the data; the only way to the value is through `lock()`, so lock and data are not separable. (c) **True** — `Arc` is `Rc` with an atomic reference count, making it `Send + Sync`. (d) **False** — it prevents data races, not deadlock. (e) **False** — sending *moves* ownership; the sender can no longer use the value. *Tempting wrong answer:* marking (b) true by analogy to C's separate `pthread_mutex_t` — Rust deliberately bundles the data inside the `Mutex`. **Feed-forward:** §32.2–§32.4.

32.11 (a) `Send`/`Sync` extend aliasing-XOR-mutation across threads: sharing a `&mut` between two threads would create two mutable paths to one value at once (aliased mutation), which is exactly what the borrow rule forbids and what would be a data race — so it does not type-check. (b) All three are *trait-bound* failures: an unmet generic bound (Chapter 30), a `?` in a function that cannot absorb the error (Chapter 31, an unmet residual/`From` bound), and a non-`Send` type crossing threads (here) are all "a required trait is not implemented," which is `E0277`. (c) The C volumes made you prove by hand that memory is freed once and used while valid, references never dangle, and shared state is never raced on; Rust turns each such proof into a type-system rule the compiler checks (ownership, borrowing, lifetimes, `Send`/`Sync`) — same machine, same generated code, proof automated. *Tempting wrong answer:* "the reused error code is a coincidence" — it reflects that thread-safety, error propagation, and generic reuse are all expressed as trait requirements. **Feed-forward:** §32.5; Chapters 27–31.

32.12 Toolchain-specific. A correct write-up reports: (a) at `-O0` the totals are below `4000000` and differ run to run (e.g. ~2.1–2.8 million), since each `++` is a real load/add/store with a wide race window; at `-O2` the optimizer may collapse the loop into a single register update, shrinking the window so the race often does not manifest — a reminder that a data race is UB and its observable effect depends on optimization and scheduling. (b) A `pthread_mutex_t` around `counter++` makes the total exactly `4000000`, deterministically. (c) With `rustc`, the `Arc<Mutex<i32>>` version is deterministic; the `Rc` version fails with `E0277` on the `spawn` capturing the non-`Send` `Rc`. Without `rustc`, name that line. *Tempting wrong answer:* "`-O2` printing `4000000` proves the race is gone" — it proves only that the window closed this time; the program is still UB. **Feed-forward:** §32.1, §32.4.

Chapter 33 — Solutions

33.1 "Measure before you optimize": locate the real cost by measurement before spending effort, because bottlenecks are rarely where intuition says. A **profiler** tells you *which* code spends the runtime (find the target); a **benchmark** compares two versions of a known piece of code honestly (evaluate a fix). *Tempting wrong answer:* "profiling and benchmarking are the same thing" — profiling finds the hot spot across a whole program; benchmarking times a specific candidate. **Feed-forward:** §33.1, §33.4.

33.2 Because the loop's result is never used, so at `-O2` the optimizer proves the loop has no observable effect and deletes it entirely (Chapter 10) — nothing runs, so the timer reads `0.0000 s`. *Tempting wrong answer:* "the CPU is just that fast" — a billion iterations are not free; they did not happen. **Feed-forward:** §33.2.

33.3 A single run is disturbed by the scheduler, cache/TLB state, frequency scaling, and other tenants, so its time is noisy; running many trials and reporting the minimum (or median) gives the run least disturbed by outside interference — closest to the code's intrinsic cost — and you report the spread too. *Tempting wrong answer:* "average the runs" — the mean is dragged up by one-directional outliers (interference only slows a run), so the minimum is a more principled floor. **Feed-forward:** §33.3.

33.4 (a) At most 6.4% — perfectly optimizing a 6.4%-of-runtime part caps its whole-program payoff at 6.4% (Amdahl). (b) 0.30 × 93.6% = 28.1% of total runtime. (c) It violates §33.1 because effort was spent on the part that mathematically cannot matter (≤6.4%) while the 93.6% dominant cost went untouched — and the target was chosen by how the code "looks" rather than by a profile. *Tempting wrong answer:* "optimizing `parse` is fine because every bit helps" — Amdahl bounds "every bit" at 6.4%, a poor return versus `compute`. **Feed-forward:** §33.1, §33.4.

33.5 (a) When the loop's result is never observed, the compiler may delete it — the Chapter 10 principle that computations with no observable effect can be removed. (b) Make the result observable: print it as a checksum, or write it to a `volatile` (or a compiler-opaque sink). (c) Because the debug (`-O0`) build's costs are not the release build's — you must measure the code as it will actually be compiled and shipped, or the number describes a program you are not deploying. *Tempting wrong answer:* "add a `printf` inside the loop" — that dominates the timing with I/O; consume the result once, after the loop. **Feed-forward:** §33.2.

33.6 (a) An instrumenting profiler inserts counting/timing code at function entry and exit (exact call counts); a sampling profiler interrupts the program many times per second and records the program counter, building a statistical map. (b) Instrumentation perturbs timing and can badly distort tiny, hot functions whose own cost is dwarfed by the added instrumentation; sampling has negligible overhead but is statistical (needs enough samples for accuracy). (c) For a first look at a large program without recompiling, reach for the **sampling** profiler — low overhead, no rebuild, and it points at the dominant cost. *Method-selection cue:* exact counts on specific calls → instrumenting; low-overhead whole-program map → sampling. **Feed-forward:** §33.4.

33.7 (a) Three sources: the OS scheduler preempting the process (as in trial 4); cache/TLB and branch-predictor state differing between runs; and CPU frequency scaling (turbo/thermal) plus contention from other tenants. (b) Interference only ever makes a run *slower* — there is no mechanism that makes identical work finish faster than its intrinsic cost — so the minimum is the closest estimate of that intrinsic floor, whereas the mean is pulled upward by outliers. (c) The median is better when you care about *typical* behavior under realistic interference (e.g. tail-aware service work) rather than the best case; alongside either, always report the spread (range or a percentile) so the reader sees the variance. *Tempting wrong answer:* “discard the outlier and average the rest” — arbitrary trimming hides real variance; report the minimum/median and the spread instead. **Feed-forward:** §33.3.

33.8 **Step 4:** Accumulate the hash results and consume the accumulator once after the loop, and vary the input so the call is not loop-invariant, e.g. `unsigned long acc = 0; for (i=0;i<N;i++) acc ^= hash(key[i]); // vary input` then `printf("%lu\n", acc);` (or write `acc` to a `volatile`). This makes the work observable so the optimizer cannot delete it, and the varying `key[i]` defeats constant-folding/hoisting. Before trusting the number: time with `clock_gettime(CLOCK_MONOTONIC, ...)` at the shipping optimization level, warm up once, run several trials, and report the minimum and spread. *Tempting wrong answer:* “mark `hash volatile`” — consume the *result*; a volatile qualifier on the function is not the fix. **Feed-forward:** §33.2, §33.3.

33.9 (a) **Profile** — you do not yet know which part is slow; a profile finds it. (b) **Benchmark** — the hot function is confirmed; benchmark the two candidates head to head (honestly). (c) **Neither yet** — first profile to see if that 2% module even matters; per Amdahl a rewrite of 2% buys ≤2%, so likely do not optimize it at all. (d) **Benchmark** (or re-profile) — measure total runtime before and after to confirm the change actually moved the whole program, not just the microbenchmark. *Method-selection cue:* unknown target → profile; compare known candidates or confirm a whole-program win → benchmark; small share → do not optimize yet. **Feed-forward:** §33.1, §33.4.

33.10 (a) **False** — `0.0000 s` almost always means the loop was deleted (dead-code elimination), so nothing was measured. (b) **False** — `-O0` costs differ from the shipped `-O2` build; benchmark what you ship. (c) **False** — a sampling profiler needs no recompilation; it samples the running program. (d) **True** — Amdahl caps a 4%-of-runtime part's whole-program payoff at 4%. (e) **False** — even on an idle machine, cache state, frequency scaling, and cold-start effects vary a single run; run several. *Tempting wrong answer:* marking (e) true because "idle means no interference" — idle removes some noise, not cache/frequency/warm-up variation. **Feed-forward:** §33.2–§33.4.

33.11 (a) A loop can be slow at a low instruction count because of **cache misses** (Chapter 3) stalling on memory, **branch mispredictions** flushing the pipeline, and **memory-bandwidth** limits or poor data layout (Chapters 4–5) — the instruction is cheap; the data it touches is not. (b) To test a cache-friendlier layout, hold everything else constant (same inputs, same optimization level, same machine, same measurement harness), change only the layout, make the result observable so the loop runs, and run many trials reporting the minimum and spread for both layouts — then compare. (c) It is the scientific method: the profile is an observation, "the layout causes cache misses" is a hypothesis, changing one thing is a controlled experiment, and re-measuring tests the prediction — changing several things at once, or not re-measuring, forfeits the ability to attribute the result. *Tempting wrong answer:* "the profile already tells you why it's slow" — it tells you *where*, not *why*; the cause is a separate hypothesis to test. **Feed-forward:** §33.3, §33.4; the cache and data-layout chapters ahead.

33.12 Platform-specific. A correct write-up reports: (a) the unused-result loop times near `0.0000 s` (deleted), and after printing the variable it times to its real cost (a fraction of a second to seconds depending on `N` and hardware), with the difference attributed to dead-code elimination. (b) Five `CLOCK_MONOTONIC` trials of the honest version, with a minimum below the others and a spread (max − min) of some percent — e.g. a ~20% span is unremarkable. (c) With `perf`/`gprof`, the profiler agrees that the heavier function dominates; without them, hand timing shows the split (e.g. ~94% vs ~6%) and the write-up names the sampling profiler it would use and that it adds per-line/whole-program attribution without recompilation. *Tempting wrong answer:* "report the first run's time" — the first run pays cold-cache/warm-up costs; report the minimum of several. **Feed-forward:** §33.2–§33.4.

Chapter 34 — Solutions

34.1 **Spatial locality:** if you access one memory address, you are likely to access nearby addresses soon (so contiguous, small-stride access is cheap). **Temporal locality:** if you access an address, you are likely to access the *same* address again soon (so recently-used data should still be cached). Good spatial locality: `for (i) s += a[i];` (stride 1). Poor spatial locality: `for (i) s += a[i*1024];` (each access a new line). *Tempting wrong answer:* swapping the two definitions — "spatial" is about neighbors in space (address), "temporal" about revisiting over time. **Feed-forward:** §34.2, §34.4.

34.2 One fetch, in the best case: the first `double` pulls in its whole 64-byte line, and the next 7 `double`s (7×8 = 56 bytes) live in that same line, so they hit cache with no new fetch — 8 `double`s fill exactly one 64-byte line. *Tempting wrong answer:* "8 fetches, one per `double`" — the hardware moves a whole line, not a single element. **Feed-forward:** §34.2.

34.3 Because if a DRAM access costs ~100× an L1 hit, a loop that misses cache is stalled waiting on memory for most of its time, so its speed is set by *how it touches memory*, not by how many instructions it runs — two loops with identical instruction counts can differ by an order of magnitude in time. *Tempting wrong answer:* "instruction count still decides it" — the CPU idles through the miss regardless of how few instructions remain. **Feed-forward:** §34.1, §34.3.

34.4 (a) Only the *order* of memory access differs: row-major strides by one element (each 64-byte line serves 8 `double` accesses), column-major strides by a full row (each access lands in a fresh line and, since the matrix is 512 MiB, that line is evicted long before its 8 elements are revisited). (b) 1 of 8 — the column walk uses one `double` per fetched line and discards the other 7, and does not return to them before eviction. (c) Swap the loop nesting so the inner loop strides by one, or transpose the matrix so the column access becomes contiguous. *Tempting wrong answer:* "the column version does more arithmetic" — it does exactly the same 67M additions; only locality differs. **Feed-forward:** §34.3.

34.5 (a) At stride 1 each 64-byte line serves 16 `int` accesses, amortizing the fetch 16 ways; as the stride grows the accesses served per line fall (8, 4, 2), so each access carries more of the line-fetch cost and the per-access time climbs. (b) At a 64-byte stride each access lands in a distinct line and shares none with its neighbors — spatial locality is fully defeated, the first stride at which no two accesses reuse a line. (c) Larger strides also defeat the hardware prefetcher (it can no longer predict the next line) and stress the TLB (Chapter 4), each access touching a new page more often. *Tempting wrong answer:* "the array got bigger" — the array and the access count are fixed; only the stride changed. **Feed-forward:** §34.2.

34.6 (a) Separately `malloc`'d nodes are scattered across the heap, so `next` jumps to an unpredictable, distant address — each node likely a fresh cache line (and page), with no spatial locality between consecutive nodes. (b) Store the nodes in a contiguous array (or a pool/arena allocator) so traversal strides through adjacent memory, restoring spatial locality and letting the prefetcher work. (c) Both turn a logically simple traversal into one that lands in a new cache line on nearly every step — the pointer chase and the column walk are the same "stride across lines" pathology. *Tempting wrong answer:* "small nodes mean good locality" — small but scattered is still one line per node. **Feed-forward:** §34.2, §34.3; the layout chapter ahead.

34.7 (a) While S < C the array fits in cache, so after the first pass later passes mostly hit and the miss rate is low; as S grows past C, each pass evicts what the next pass needs, so the miss rate jumps toward one-miss-per-line. (b) Blocking processes a chunk of size ≤C completely — doing all the work that touches it — before moving on, so each loaded line is reused many times while still cached, keeping the working set within a cache level. (c) The arithmetic operation count is unchanged, but blocking removes the memory stalls that dominated the time, so the loop stops waiting on DRAM and becomes limited by the arithmetic it was always doing — memory-bound to compute-bound with no fewer operations. *Tempting wrong answer:* "blocking does fewer operations" — it does the same operations in a cache-friendlier order. **Feed-forward:** §34.4.

34.8 **Step 4:** Ship `for (i) for (j) sum += A[i][j]` — the inner loop strides by one element along the contiguous (row-major) dimension, so each 64-byte line serves 16 `float` accesses and spatial locality is perfect, while the other nesting strides a full 16 KiB row per access and misses almost every time. To measure: compile both at the shipping optimization level (`-O2`), time each as the minimum of several `clock_gettime(CLOCK_MONOTONIC)` trials, hold everything else constant (same matrix, same machine, same harness), make the sum observable so neither loop is deleted (Chapter 33), warm up once, and report the minimum and spread for each; the row-order loop should be several times faster. *Tempting wrong answer:* "time each once and compare" — a single run is noisy; run several and report the minimum. **Feed-forward:** §34.3; Chapter 33.

34.9 (a) **Spatial** — column order strides across lines, wasting most of each fetched line. (b) **Temporal** — the data is reused every scan but a 100 MiB set overflows the 24 MiB L3, so it is evicted before reuse. (c) **Neither (both good)** — a 1 KiB array fits in L1 and is reused, so spatial and temporal locality are both excellent. (d) **Spatial** — scattered nodes give one line per node with no neighbor reuse. *Tempting wrong answer:* calling (b) "spatial" — the scan is contiguous (good spatial); the problem is the set does not stay cached (temporal). **Feed-forward:** §34.2, §34.4.

34.10 (a) **False** — reading one byte fetches its whole 64-byte cache line. (b) **False** — identical instruction counts can differ by an order of magnitude if locality differs (the §34.3 19.5×). (c) **False** — the compiler often may *not* reorder loops (it can change results or be illegal), so the traversal order is your responsibility. (d) **True** — a working set that fits in L2 is hit by repeated passes. (e) **False** — blocking reorders access to improve reuse; it does not reduce the arithmetic operation count. *Tempting wrong answer:* marking (c) true because “the optimizer handles everything” — loop interchange is limited and not guaranteed. **Feed-forward:** §34.2–§34.4.

34.11 (a) Hypothesis: the loop is memory-bound because of poor locality (striding across lines or a working set larger than cache); test it by improving locality (transpose/reorder, or shrink the working set) and re-measuring, or with a cache-miss counter where available. (b) Wall-clock experiment: run the loop over inputs of growing size and watch for a sharp per-element slowdown as the working set crosses a cache size (a memory-bound signature), or compare the loop against a locality-fixed version; if the fixed version is much faster, it was memory-bound, if not, compute-bound. (c) Transposing/reordering is a one-line, low-risk change that attacks the actual cause (locality) and is easy to re-measure, whereas rewriting in assembly is high-effort, high-risk, and does nothing about a memory stall the arithmetic was never causing. *Tempting wrong answer:* “assembly is always faster” — assembly cannot beat the memory ceiling that poor locality created; fix locality first. **Feed-forward:** §34.4; Chapter 33.

34.12 Platform-specific. A correct write-up reports: (a) the machine's real L1/L2/L3 sizes and line size from `/sys` (e.g. 64-byte lines). (b) The row-major sum timed several times and the column-major sum timed several times, both as minima, with a ratio of a few× to >10× depending on hardware, explained by the column walk using one element per fetched line. (c) A stride sweep whose per-access cost has a knee at the line size (contiguous cheap, one-line-stride expensive). *Tempting wrong answer:* “report the first, cold run” — the first run pays cold-cache costs; report the minimum of several (Chapter 33). **Feed-forward:** §34.2–§34.3.

Chapter 35 — Solutions

35.1 **AoS (array-of-structs):** one array whose elements each hold all a record's fields together — `struct P{float x,y,vx,vy;} p[N];`. **SoA (struct-of-arrays):** a separate contiguous array per field — `float x[N],y[N],vx[N],vy[N];`. *Tempting wrong answer:* “SoA is just an array of structs each holding arrays” — no; each *field* gets its own flat array, so one field's values are contiguous. **Feed-forward:** §35.2.

35.2 Useful bytes per fetched line = of every 64-byte line the hardware pulls in, how many bytes the loop actually reads. Summing one field of a 64-byte AoS struct reads 8 bytes (one `double`) of each fetched line and ignores the other 56, so 7 of every 8 bytes of memory traffic are wasted. *Tempting wrong answer:* “it uses the whole line because it fetched it” — fetching is not using; the other fields are dragged in and discarded. **Feed-forward:** §35.2, §35.3.

35.3 False sharing: two threads write two *different* variables that happen to sit in the *same* cache line, so the coherence protocol bounces that line between cores on every write even though no data is truly shared. Fix: **padding** (or alignment) so each thread's hot data sits on its own cache line. *Tempting wrong answer:* “add a lock” — a lock addresses real contention; false sharing has none, and a lock would make it worse. **Feed-forward:** §35.4.

35.4 (a) AoS uses 8 of 64 bytes per line; SoA uses 64 of 64 — an 8× better useful-bytes ratio, and since the loop is limited by memory traffic, ~8× less wasted traffic predicts a large speedup. (b) The measured 4.08× is below 8× because the hardware prefetcher partly hides AoS's waste (it streams ahead), and other fixed costs (the add, loop overhead) do not shrink — so the memory saving does not translate one-for-one. (c) A loop that reads *all* fields of one element before moving on: then AoS puts the whole record in one fetched line and uses all of it, while SoA would touch several arrays (several lines) per element. *Tempting wrong answer:* “SoA must give exactly 8×” — byte ratio is an upper bound, not a guarantee. **Feed-forward:** §35.2.

35.5 (a) Split into hot `struct{float x,y;} hot[N];` and cold `struct{char name[..]; Inv inventory; Ach achievements;} cold[N];`, indexed in parallel. (b) The per-frame loop now streams the dense `hot[]` array — every fetched line is full of `x,y` — instead of striding a fat struct and wasting the line on cold fields, even though no field was deleted. (c) Cost: touching a player's full state now means two arrays (worse locality for the rare all-fields access), and the code is more complex and easy to desynchronize; not worth it when the hot loop reads most fields anyway, or when the struct is already small. *Tempting wrong answer:* “always hot/cold split” — it hurts loops that need all fields of one element. **Feed-forward:** §35.3.

35.6 (a) Alignment: `double b` must sit on an 8-byte boundary, so the compiler pads after `char a` (7 bytes) and after `char c` (7 bytes) to keep the struct's size a multiple of its alignment — about 24 bytes, not 10. (b) Reorder largest-to-smallest: `{ double b; char a; char c; }` — `b` takes 8, `a` and `c` pack into the next 2 bytes, then 6 bytes of tail padding to a multiple of 8: 16 bytes. (c) At 16 vs 24 bytes, 50% more elements fit per 64-byte line, so a sweep over a million of them fetches fewer lines and runs faster. *Tempting wrong answer:* “the struct is 10 bytes, the size of its fields” — alignment padding makes it larger. **Feed-forward:** §35.3.

35.7 (a) From the cache-coherence protocol: the four counters share one line, so each thread's write invalidates the line in the other three cores' caches, forcing the line to bounce (ping-pong) between cores — pure coherence traffic, not data sharing. (b) Padding puts each counter on its own line, so a thread's writes never invalidate another's line; it trades ~56 wasted bytes per counter (harmless here) for eliminating the bounce, an 8× win. (c) It would shrink to nothing: with one thread there is no other core to bounce the line to, so packed and padded run the same — false sharing is a purely concurrent cost. *Tempting wrong answer:* “the writes themselves are slow” — the increment is trivial; the cost is the line moving between cores. **Feed-forward:** §35.4.

35.8 **Step 4:** Choose the **hot/cold split** (a packed `{cx,cy,cz,r}` array plus a parallel cold array): the intersection loop reads exactly those four fields together per sphere, so a 32-byte hot record gives it two spheres per 64-byte line with everything it needs, while full SoA would make it touch four separate arrays (four lines) per sphere — worse for a loop that uses all four hot fields at once. (Full SoA would win only if the loop swept a single field.) Measure it the Chapter 33 way: compile original and split at the shipping level (`-O2`), same scene, same machine, make the result observable, warm up, and report the minimum and spread of several `clock_gettime(CLOCK_MONOTONIC)` trials for each; the split should be several times faster. *Tempting wrong answer:* “full SoA is always best” — here the loop reads four fields together, so a packed hot record beats four separate arrays. **Feed-forward:** §35.2, §35.3; Chapter 33.

35.9 (a) **SoA** — one field swept across many records is pure spatial locality when that field is its own array. (b) **AoS** — processing one record's full state at a time wants the whole record in one line. (c) **Hot/cold split** — 3 hot fields streamed every frame go in a dense array, the 30 cold ones elsewhere. (d) **Pad to a cache line** (a layout change, not AoS/SoA) — per-thread counters must not share a line, or false sharing bites. *Tempting wrong answer:* calling (d) “SoA” — the issue is cross-thread line sharing, fixed by padding. **Feed-forward:** §35.2–§35.4.

35.10 (a) **False** — SoA wins for one-field sweeps but AoS wins when a loop uses all fields of one element. (b) **True** — reordering fields to reduce alignment padding shrinks the struct without removing a field. (c) **False** — false sharing is two threads writing *different* variables that share a cache line, not the same variable. (d) **False** — padding wastes bytes single-threaded but prevents false sharing across threads, an 8× win in §35.4. (e) **False** — same operation count, different layout, can change runtime by several× (the AoS/SoA 4×). *Tempting wrong answer:* marking (e) true from RAM-model intuition — locality, not operation count, sets the time. **Feed-forward:** §35.2–§35.4.

35.11 (a) Chapter 34 (reorder accesses over a fixed layout) fits when you cannot change the data structure — e.g. iterating a given matrix in the right order; this chapter (change the layout) fits when you own the types — e.g. AoS→SoA for a hot field sweep; use both when you both pick the layout and choose the loop order over it (SoA arrays, swept contiguously). (b) A vectorizer loads 8 contiguous elements into one register; SoA makes a field's values contiguous, so one wide load fills a vector, while AoS scatters them across struct-sized gaps and cannot. (c) Converting everything before profiling optimizes code that may be nowhere near the dominant cost (Chapter 33/Amdahl), risks pessimizing loops that wanted AoS, and spends effort with no measured payoff — profile first, convert the hot path, re-measure. *Tempting wrong answer:* “SoA everywhere is strictly better” — it hurts all-fields-per-element loops and ignores Amdahl. **Feed-forward:** §35.2; Chapters 33, 36.

35.12 Platform-specific. A correct write-up reports: (a) an AoS field sum and an SoA field sum timed as minima of several trials, with a ratio of a few× explained by the useful-bytes-per-line difference (struct size vs field size). (b) the ratio growing as the struct grows (16→128 bytes), because a bigger struct wastes more of each line in AoS while SoA is unchanged. (c) if threads are available, a packed-counters vs padded-counters run with a several× slowdown from false sharing, vanishing at one thread. *Tempting wrong answer:* “time each layout once” — single runs are noisy; report minima of several (Chapter 33). **Feed-forward:** §35.2, §35.4.

Chapter 36 — Solutions

36.1 SIMD (Single Instruction, Multiple Data): one instruction operates on a whole vector of elements at once, on a single core. A 256-bit AVX register holds 8 `float`s (32 bits each) or 4 `double`s (64 bits each), and one AVX arithmetic instruction processes all of them — 8 or 4 — in one step. *Tempting wrong answer:* “SIMD means multiple cores” — it is width on one core, not more cores. **Feed-forward:** §36.1.

36.2 **Contiguity** (the elements adjacent in memory so one load fills a vector register) and **independence** (no lane depends on another's result within the vector, so no loop-carried dependency). Contiguity is supplied by Chapter 35's struct-of-arrays layout. *Tempting wrong answer:* “the loop just has to be short” — length is irrelevant; layout and independence are what matter. **Feed-forward:** §36.1, §36.3.

36.3 (a) `-O3` (gcc does not vectorize at `-O2` in this version); more generally `-ftree-vectorize`. (b) `-march=native`, letting it use the widest vectors the build CPU supports (AVX2 here). (c) `-fopt-info-vec-optimized` (or `-fopt-info-vec-missed` to see why a loop did *not* vectorize). *Tempting wrong answer:* “-O2 is enough” — it is not, in this gcc; you need `-O3`. **Feed-forward:** §36.2.

36.4 (a) Because vectorization depends on the build: `-O2` emitted scalar code, `-O3` used 128-bit SSE2 (4-wide), `-O3 -march=native` used 256-bit AVX2 (8-wide) — same source, three code generations. (b) Vectorizing also let the compiler use fused multiply-add (one instruction for `a*b+c`) and unroll the loop, cutting per-iteration overhead, so the gain is lane count times these extra wins. (c) Confirming with `-fopt-info-vec` gives evidence the loop vectorized (and how wide), whereas assuming it did risks shipping a “vectorized” loop the compiler actually left scalar. *Tempting wrong answer:* “13× proves a bug in the model” — no; FMA and unrolling legitimately push it past raw lane count. **Feed-forward:** §36.2.

36.5 (a) Vectorizing a sum groups the additions differently (e.g. 8 partial sums combined at the end), and floating-point addition is not associative, so the rounded result can differ from the strict left-to-right order the source specifies; the standard forbids the compiler from changing your result, so it declines. (b) `-ffast-math` (or `-funsafe-math-optimizations`) permits reassociation; it also allows other unsafe assumptions (no NaNs/infinities, reordering that changes rounding), which can break code that relies on strict IEEE behavior. (c) The dot product is **memory-bandwidth-bound** — it streams arrays far larger than cache, so it waits on DRAM, not the adder; that is a bandwidth ceiling, unrelated to the reduction/associativity issue. *Tempting wrong answer:* “the small gain is the reduction still being scalar” — once `-ffast-math` vectorizes it, bandwidth still caps it. **Feed-forward:** §36.3, §36.4.

36.6 (a) **Yes** — elementwise, contiguous, independent lanes. (b) **No** — loop-carried dependency: `a[i]` needs `a[i-1]`, so lanes cannot be computed in parallel. (c) **No (or poorly)** — a heavy data-dependent branch per element breaks the uniform lane operation (though light branches can sometimes be masked/predicated). (d) **No** at plain `-O3` — a floating-point reduction needs `-ffast-math` to reassociate. *Tempting wrong answer:* marking (b) yes because “it's a simple loop” — simplicity is irrelevant; the dependency is fatal. **Feed-forward:** §36.3.

36.7 (a) Arithmetic intensity = floating-point operations performed per byte moved to/ from memory. (b) Dot product: ~2 flops (a multiply and an add) per two 4-byte loads = 8 bytes, so ~0.25 flop/byte — very low, so once the arrays exceed cache it is limited by how fast DRAM delivers bytes: bandwidth-bound. (c) The §36.2 kernel: ~11 flops per 4-byte element already resident in L1, so ~2.75 flops/byte and almost no DRAM traffic — the core's arithmetic throughput binds: compute-bound. (d) SIMD helps loops that are compute-bound (high intensity); for bandwidth-bound (low intensity) loops the fix is locality/layout, not more lanes. *Tempting wrong answer:* “more flops per element always means SIMD helps” — it also depends on whether the bytes are in cache (intensity counts bytes *moved*). **Feed-forward:** §36.4.

36.8 **Step 4:** 1.2× is expected, not a bug: the loop is bandwidth-bound (tiny arithmetic intensity, 100 MiB arrays streamed from DRAM), so the memory bus caps it and widening the add from 1 lane to 8 cannot help — the adder was never the bottleneck. Confirm the diagnosis by timing the plain `-O3 -march=native` auto-vectorized version (checking `-fopt-info-vec` that it vectorized): it should match ~1.2×, showing the intrinsics are fine and memory is the ceiling. The real speedup must come from moving fewer bytes — better locality and layout (Chapters 34–35), e.g. fusing this with adjacent passes so the data is touched once, or shrinking the element type — not from more lanes. *Tempting wrong answer:* “keep debugging the intrinsics” — they are correct; the model, not the code, was wrong. **Feed-forward:** §36.4.

36.9 (a) **Large** — compute-bound, cache-resident, independent lanes: SIMD's ideal case. (b) **None (to speak of)** — a 1 GiB copy is pure bandwidth, so wide loads/stores are already bandwidth-limited; lanes do not add throughput. (c) **Large** — cache-resident and arithmetic-heavy, so the vector unit binds, not memory (a reduction needs `-ffast-math` to vectorize). (d) **None** — a loop-carried dependency is inherently serial; it cannot vectorize as written. *Tempting wrong answer:* calling (b) large “because wider loads move more” — the bus, not the load width, is already the cap. **Feed-forward:** §36.3, §36.4.

36.10 (a) **False** — 8 lanes bound arithmetic throughput at up to 8×, but bandwidth-bound loops gain far less and compute-bound ones can exceed 8× via FMA/unrolling. (b) **False** — gcc will not vectorize a floating-point reduction at `-O3` without `-ffast-math` (associativity). (c) **True** — `-march=native` targets the build machine's widest supported vectors. (d) **False** — SIMD helps compute-bound loops, not bandwidth-bound ones. (e) **False** — an AoS field is strided, so it cannot be loaded into a vector in one go; packed SoA can. *Tempting wrong answer:* marking (a) true from the lane-count intuition — the ceiling, not the lane count, sets the gain. **Feed-forward:** §36.1–§36.4.

36.11 (a) SIMD needs contiguous data (one wide load fills a register) and data that reaches the core fast (or the vector unit starves), which are exactly the layout of Chapter 35 and the locality of Chapter 34 — without them the vectorizer either cannot fire or gains nothing. (b) Ask in order: what is this kernel bound by — arithmetic, cache misses, or bandwidth (measure scalar vs vectorized, vary input size to find cache cliffs)? If cache/bandwidth-bound, fix locality (Chapter 34) and layout (Chapter 35) first; if compute-bound, vectorize (Chapter 36) and confirm with `-fopt-info-vec`; re-measure after each change (Chapter 33). (c) Because the true limit is usually a specific resource (cache, bandwidth, vector width), and raising a ceiling you are not under does nothing — "fewer instructions" ignores that a memory-bound loop is idle waiting on bytes regardless of instruction count. *Tempting wrong answer:* "always vectorize the hot loop" — pointless if it is bandwidth-bound; find the binding ceiling first. **Feed-forward:** Chapters 33–36.

36.12 Platform-specific. A correct write-up reports: (a) an L1-resident compute kernel at scalar / `-O3` / `-O3 -march=native`, with throughputs rising and a speedup at or above the lane count, and the `-fopt-info-vec-optimized` line naming the vector width. (b) a cache-overflowing dot product that barely vectorizes without `-ffast-math` and gains little even with it, explained by low arithmetic intensity (bandwidth-bound). (c) the toolchain version, CPU, and widest vector width, noting these are wall-clock throughputs (no hardware counters) if `perf` is unavailable. *Tempting wrong answer:* "report the first run" — cold-start noise; report the minimum of several trials (Chapter 33). **Feed-forward:** §36.2–§36.4.

Chapter 37 — Solutions

37.1 A deeply pipelined core has many instructions in flight at once (Chapter 2), so stalling at every branch until its condition is computed would drain the pipeline and throw away the instruction-level parallelism that makes the core fast. Instead it **predicts** the outcome with a hardware branch predictor and speculatively executes down the guessed path, committing the results if the guess was right and squashing them if it was wrong. *Tempting wrong answer:* "it just evaluates the condition first" — it cannot; the condition is often not ready for many cycles, and waiting for it is exactly the stall speculation exists to avoid. **Feed-forward:** §37.1.

37.2 The misprediction penalty is the cost of a wrong guess: the speculatively executed instructions after the branch are squashed and the front end refills the pipeline from the correct target. On modern x86 it is roughly **15–20 cycles** — a published figure (Agner Fog; Hennessy & Patterson), not measured here, since `perf` is unavailable. It is about the depth of the pipeline because that is how many stages must be re-primed before useful work resumes. *Tempting wrong answer:* "it is the cost of the comparison" — the `cmp` is one cheap instruction; the penalty is the refill. **Feed-forward:** §37.1.

37.3 Because the branch's cost is not the comparison but the *misprediction* the comparison's outcome can trigger. If the branch is a data-dependent coin flip, the predictor is wrong on about half of the iterations and each miss costs a pipeline-depth refill (~15–20 cycles), which dwarfs the handful of arithmetic instructions the branch guards. So the same branch is nearly free when predictable and dominant when not — the cost lives in the data, not the opcode. *Tempting wrong answer:* "a branch is always cheap because `cmp` is cheap" — that conflates the one-cycle comparison with the pipeline-refill penalty a wrong guess pays. **Feed-forward:** §37.1–§37.2.

37.4 (a) On unsorted data the branch is a ~50/50 coin flip with no learnable pattern, so the predictor misses roughly half of the ~655 million branches and each miss costs a ~15–20-cycle refill; the accumulated penalty is essentially the whole 15× gap. On sorted data the branch is false for the entire run of values below 128 and then true for the rest, so it flips once and is predicted correctly almost always. (b) It is predictable on the sorted array because the outcomes arrive in two long runs the predictor learns immediately; unpredictable on the unsorted array because consecutive outcomes are independent. (c) Nothing about the arithmetic, the instruction count, or the number of elements summed differs — only the *predictability* of the branch, which is a property of the data order. *Tempting wrong answer:* "sorting makes the arithmetic cheaper" — the arithmetic is bit-for-bit identical; only prediction changed. **Feed-forward:** §37.2.

37.5 (a) `int m = (x - 128) >> 31; sum += x & ~m;` — the arithmetic right shift turns the sign of `x-128` into an all-ones or all-zeros mask, which is ANDed against `x`, so the loop always executes the same instructions regardless of the data: there is no conditional jump, hence nothing to predict or mispredict. (b) On sorted data the real branch is predicted correctly almost always, so it does the work of only the taken path and skips the rest for free (0.50 s); the branchless version always computes both paths' work — the subtract, the shift, the AND, the add — on every element (~0.9 s), so it loses to a well-predicted branch. (c) Branchless wins only when the branch is genuinely unpredictable, i.e. when the misprediction it avoids costs more than the extra both-paths arithmetic it always pays. *Tempting wrong answer:* “branchless is always faster because it has no branch” — it pays for both paths every iteration and loses to a predictable branch. **Feed-forward:** §37.3.

37.6 (a) At plain `-O2` gcc applied **if-conversion**, turning the `if` into a conditional move (`cmovg`) — a branchless select — so no branch survives; confirm it by reading the emitted assembly (`gcc -S`) and finding `cmovg` where a conditional jump would be. (b) At `-O3 -march=native` it vectorized the loop with a masked AVX2 add (8 elements per instruction, no branch), confirmable with `-fopt-info-vec`. (c) You cannot read branch cost off the C source: whether a branch even exists in the binary depends on if-conversion and vectorization decisions the compiler makes at your optimization level and target, so counting `if`s and multiplying by a cycle penalty often prices a branch that is not there. *Tempting wrong answer:* “the source has an `if`, so there is a branch” — the build, not the source, decides. **Feed-forward:** §37.4.

37.7 (a) The extra cycles spent on mispredictions are approximately $K \cdot f \cdot p$. (b) At $f = 0.5$ this is $K \cdot 0.5 \cdot p$ — a large per-iteration tax that, with $p \approx 15$–20, dwarfs a few-cycle loop body; at $f \approx 0$ it is ~0 and the branch is essentially free. (c) Branchless does w extra both-paths cycles per iteration and never mispredicts, costing $K \cdot w$; it is faster than the branch when $K \cdot w < K \cdot f \cdot p$, i.e. when $w < f \cdot p$. So the more unpredictable the branch (larger f) and the deeper the pipeline (larger p), the more room branchless has to win. *Tempting wrong answer:* “branchless always wins because $f{\cdot}p$ is large” — only when $w < f{\cdot}p$; a predictable branch (small f) flips the inequality. **Feed-forward:** §37.3.

37.8 **Step 4:** They saw no sorted-vs-unsorted difference because at `-O2` the compiler if-converted the `if` into a `cmovg`, so there was no branch in the binary to mispredict — with nothing to predict, data order is irrelevant. The single change that would let them observe the effect is to forbid if-conversion (`-fno-if-conversion -fno-if-conversion2 -fno-tree-loop-if-convert`) so a real conditional jump survives; then the sorted run is dramatically faster. The correct conclusion is about the *build*, not the CPU: branch prediction is present and working, but this build removed the branch, so “prediction is a myth on this CPU” mistakes a compiler transform for a hardware fact. *Tempting wrong answer:* “the CPU's predictor is broken or absent” — the predictor is fine; the branch was compiled away. **Feed-forward:** §37.4.

37.9 (a) **Well** — a loop back-edge taken 10 million times is mispredicted essentially once, on the final exit; it is the most predictable branch there is. (b) **Badly** — `rand() & 1` is a 50/50 coin flip with no pattern, the worst case (~50% misprediction). (c) **Well** — a branch that is almost always the same way is learned in a few iterations and predicted right almost always. (d) **Badly** — a uniform distribution over 8 cases is near-random with no learnable pattern, so a `switch` the compiler keeps as a data-dependent jump mispredicts most of the time. *Tempting wrong answer:* "loops are unpredictable because they branch every iteration" — the back-edge is precisely the branch the predictor gets right. **Feed-forward:** §37.2.

37.10 (a) **False** — only if the branch survives compilation and is actually mispredicted; the compiler often if-converts or vectorizes it away, and a predictable branch costs nearly nothing. (b) **False** — branchless loses to a well-predicted branch because it always does both paths' work. (c) **True** — a correctly predicted branch commits already-completed speculative work and costs essentially nothing. (d) **True** — `-O3 -march=native` can turn a scalar `if`-loop into a masked vector loop with no branch (§37.4). (e) **False** — whether there is even a branch to mispredict depends on the build, so you must read the emitted assembly and time the binary. *Tempting wrong answer:* marking (a) true from "it is a branch in the source" — the build, not the source, decides whether the branch exists. **Feed-forward:** §37.1–§37.4.

37.11 (a) In order: first confirm the branch exists in the shipped build — read the emitted assembly for a conditional jump versus a `cmov`, or check `-fopt-info-vec` for vectorization; if no branch survives, there is nothing to rewrite. If a real branch does survive, measure branch versus branchless on representative data (Chapter 33), because the right choice flips with predictability; only then change code. (b) §37.4's masked vectorization is branch elimination carried further: instead of a `cmov` selecting one scalar, the vectorizer computes all lanes and blends them with a mask, so a data-dependent `if` becomes uniform SIMD arithmetic — the same "remove the branch, do both paths" idea as Chapter 36, applied across lanes. (c) "Branches are slow, avoid them" leads you to rewrite branches the compiler already deleted and to make predictable branches slower with always-both-paths code; "the cost is a property of the build and the data" tells you to check the binary and measure predictability first, which is where the real decision lives. *Tempting wrong answer:* "always rewrite the hot branch branchless" — pointless if the branch is not in the binary, and a regression if it is predictable. **Feed-forward:** §37.4–§37.5.

37.12 Platform-specific. A correct write-up reports: (a) the summing loop at plain `-O2` showing little or no sorted-vs-unsorted difference (the compiler if-converted to a `cmov`) and the if-conversion-disabled build showing a large swing (sorted much faster), with the assembly confirming `cmov` versus a conditional jump. (b) a branchless mask version running the same time on both orders, order-independent. (c) the toolchain version and CPU, noting these are wall-clock minimum-of-trials times with no hardware counters if `perf` is unavailable. *Tempting wrong answer:* "report the `-O2` build as proof that prediction does not matter" — that build has no branch; you must disable if-conversion to observe the effect at all. **Feed-forward:** §37.4.

Chapter 38 — Solutions

38.1 On a single call a general-purpose `malloc` may walk a size-segregated free-list to find a block, split a larger block (or on `free` coalesce adjacent free ones), update the block's size/status header, take a lock or use a per-thread cache for thread-safety, and occasionally make a `brk`/`mmap` system call for more memory. A stack allocation does none of this because it is a single decrement of the stack pointer with a statically known lifetime — the frame — so there is no shared pool to search or manage. *Tempting wrong answer:* "`malloc` just returns the next free address" — there is no "next" in a shared heap with arbitrary free order; it must search and manage. **Feed-forward:** §38.1.

38.2 The two costs are the **per-call cost** of `malloc`/`free` — the free-list search, splitting/coalescing, locking, and occasional syscall — paid at allocation and deallocation time; and the **locality cost** of the scattered memory the allocator returns, paid *later*, on every traversal of the allocated structure, as a cache miss per node. The first is bounded (a few nanoseconds a call); the second is usually larger. *Tempting wrong answer:* "the only cost is the time in `malloc`" — that ignores the larger, deferred locality cost charged to your traversal loops. **Feed-forward:** §38.2.

38.3 An arena is freed all at once: you release the single underlying block with one `free()` (or reset the bump pointer), discarding every object it held in one operation, regardless of how many there were. So the deallocation cost is O(1) for the whole region rather than O(1) *per object* — amortized over a million objects, the per-object free cost is essentially zero, versus a separate `free` call (with its header read and coalescing) for each object under a general allocator. *Tempting wrong answer:* "the arena frees each object very quickly" — it does not free them individually at all; it drops the whole region. **Feed-forward:** §38.3.

38.4 (a) On the contiguous layout the accesses march sequentially through memory, so the hardware prefetcher fetches the next cache line before the loop asks for it and nearly every access hits cache (~8 ns); on the scattered layout each `p = p->next` jumps to an unpredictable address that is neither cached nor prefetchable, so the core stalls on DRAM — often plus a TLB miss — for essentially every node (~177 ns), giving ~22×. (b) If the ~244 MiB working set fit in the 24 MiB L3, even scattered accesses would mostly hit cache and the gap would shrink or vanish; exceeding L3 is what forces the scattered walk to DRAM and exposes the latency. (c) Only the *memory layout* of the nodes differs — contiguous versus scattered — and the `malloc`-per-node allocation pattern, not the loop, decided it. *Tempting wrong answer:* "the scattered loop does more work" — identical instructions and node count; it waits on memory, it does not compute more. **Feed-forward:** §38.2.

38.5 (a) The arena does not search a free-list for a suitably sized block (it just increments a pointer) and does not maintain per-object headers or coalesce on free (it drops the whole region), and it need not support freeing objects individually or interleaving with the rest of the program. (b) The free side is a bigger win because a general allocator does per-object work on every `free` — read the header, mark free, coalesce — whereas the arena frees a million objects with one `free()`, so the arena's per-object free cost is ~0 against ~8.6 ns, a larger ratio than the allocation side. (c) In return the arena cannot free individual objects, only the whole region; it therefore suits phase- or request-scoped work where everything allocated in a phase dies together (parse a document, serve a request, render a frame). *Tempting wrong answer:* "the arena is just a faster `malloc`" — it is faster precisely because it drops generality (individual free, arbitrary reuse) the general allocator must keep. **Feed-forward:** §38.3.

38.6 (a) **Arena** — everything allocated while handling the request dies together when the response is sent, exactly the phase-scoped lifetime an arena exploits (pointer-bump allocation, O(1) bulk free, contiguity). (b) **Pool** (slab) — a stream of same-sized objects individually created and destroyed is the fixed-size, individually-reused pattern a pool serves with O(1) push/pop and packed layout. (c) **None — keep general `malloc`** — a handful of long-lived, wildly-varying-size objects at startup has no lifetime or size structure to exploit and negligible allocation volume, so a custom allocator buys nothing. *Tempting wrong answer:* "always use an arena, it's fastest" — an arena cannot free individual objects, so it is wrong for (b)'s long-run individual reuse. **Feed-forward:** §38.3.

38.7 (a) Total time ≈ $N \cdot c$ (allocation) + $t \cdot N \cdot m$ (traversals). (b) With t=10: allocation is N·18 = 18N ns whichever allocator (or 3.6N with the arena); traversal is 10·N·8 = 80N ns contiguous versus 10·N·177 = 1770N ns scattered. The layout choice changes the total by ~1690N ns, more than a hundred times the ~14.4N ns saved by switching allocator — layout dominates. (c) Setting the per-call cost equal to the extra scattered-traversal cost: $N{\cdot}c = t{\cdot}N{\cdot}(177-8)$, so $t = 18/169 \approx 0.11$. Interpretation: even a fraction of a single traversal's extra scattering cost already exceeds the entire per-call cost, so any program that walks the data even once is dominated by locality, not by allocation speed. *Tempting wrong answer:* "the per-call cost $N{\cdot}c$ is the big term" — it is dwarfed as soon as $t \geq 1$ with scattered layout. **Feed-forward:** §38.2.

38.8 **Step 4:** The microbenchmark understates the real cost because its allocate-then-immediately-free pattern stays on the `tcache` fast path — no lock, no search, and (crucially) the objects never accumulate, so it never exposes the scattering and fragmentation that long-lived, out-of-order allocations cause in the real program. The colleague should measure the *representative* workload — objects kept live and traversed as the program actually uses them — and watch for time in the traversal loops with a high cache-miss rate, not just time inside `malloc`. The arena would fix the *locality* cost here, not the per-call cost: it packs the objects contiguously so the later traversals hit cache, which is where the real time is going — the ~18 ns per call was never the problem. *Tempting wrong answer:* "the benchmark proves allocation is cheap" — it proves the fast path is cheap, not that the real program's scattered layout is. **Feed-forward:** §38.4.

38.9 (a) **Per-call overhead** — millions of tiny allocate/free per second with no bulk traversal means the cost is the calls themselves (pool or batch them). (b) **Locality** — ten million separately-`malloc`'d nodes walked repeatedly is the classic scattered pointer-chase; the traversals' cache misses dominate (arena or flatten to arrays). (c) **Locality** — allocating thousands of AST nodes but walking the tree many times means the repeated traversals, not the one-time allocation, cost the most (arena the nodes for contiguity). (d) **Neither, really** — one large allocation has trivial per-call cost and a single contiguous buffer streams with good locality; the cost, if any, is bandwidth, not allocation. *Tempting wrong answer:* calling (a) a locality problem — it never traverses in bulk, so scattering is irrelevant; the calls are the cost. **Feed-forward:** §38.2, §38.4.

38.10 (a) **False** — the traversal does not call `malloc`; a faster allocator cannot fix cache misses over already-scattered nodes. (b) **False** — an arena frees only the whole region, never individual objects. (c) **False** — the largest cost is usually the locality of the scattered result, paid in traversals, not the time inside `malloc`. (d) **True** — an arena hands out objects consecutively, so they are contiguous and later traversals prefetch and hit cache. (e) **False** — allocate-then-immediately-free hits the `tcache` fast path and hides the scattering a long-running program suffers. *Tempting wrong answer:* marking (a) true from “allocation is slow, so a faster allocator must help” — the slow part is the traversal's memory latency, not allocation. **Feed-forward:** §38.1–§38.4.

38.11 (a) For most programs the dominant allocation cost is not the call but the cache misses of traversing the scattered objects the call returned — which is exactly Chapter 34's locality ceiling — so the fix is a contiguous layout (an arena, or a flat array with indices instead of a graph of pointers), not a faster allocator. (b) Check, in order: is the time actually in `malloc`/`free` (a per-call problem) or in the traversal loop (a locality problem)? If the traversal, is its cache-miss rate high and the working set larger than last-level cache? Only after confirming it is scattered-traversal latency, not allocation, would you change the layout. (c) “Allocate fewer, more contiguous objects” targets the real cost — memory latency on traversal — and is achievable with an arena or array; “find a faster allocator” optimizes the few-percent per-call cost and leaves the scattered layout, and its cache misses, untouched. *Tempting wrong answer:* “profile, see time near allocation, swap the allocator” — the time is near the *allocated data's* traversal, not the allocator. **Feed-forward:** §38.4–§38.5.

38.12 Platform-specific. A correct write-up reports: (a) per-object costs for `malloc`/`free` versus an arena, with the arena a few times cheaper on allocation and dramatically cheaper on free (one region `free()` for all N). (b) a contiguous versus shuffled linked traversal with the working set chosen above last-level cache, showing the contiguous walk a large factor faster (order 10–20×) and attributing it to prefetchable sequential access versus a cache miss per node. (c) the toolchain version, CPU, last-level cache size (from `/sys`), and allocator, noting these are wall-clock minimum-of-trials times with no hardware counters if `perf` is unavailable. *Tempting wrong answer:* “report the allocate-then-free microbenchmark as the allocation cost” — it hits the fast path; keep the objects live and traverse them to see the real cost. **Feed-forward:** §38.2, §38.4.

Chapter 39 — Solutions

39.1 (1) **Measure** the real workload and set a baseline (minimum of several trials). (2) Find the **dominant cost** — the largest share of runtime. (3) **Identify the ceiling** that cost is under (compute, cache, bandwidth, control, allocation). (4) **Raise that ceiling** with its matching tool, and only that tool. (5) **Re-measure** against the baseline and repeat on the next ceiling. *Tempting wrong answer:* “pick the fastest technique and apply it” — that skips steps 1–3, the diagnosis that decides which technique is even relevant. **Feed-forward:** §39.1.

39.2 Because the tools are not interchangeable — each raises one specific ceiling, so applying one before naming the ceiling risks raising a roof the program is not under, which does nothing or loses. Example: SIMD on the §36.3 bandwidth-bound dot product gave only 2.4×, not the 13× it gave a compute-bound kernel, because the loop waited on DRAM, not the adder (other valid examples: branchless code losing to a predictable branch, Chapter 37; a faster allocator doing nothing for a scattered traversal, Chapter 38). *Tempting wrong answer:* “identify the ceiling afterward, once the tool didn't help” — that wastes the effort and the added complexity the workflow exists to avoid. **Feed-forward:** §39.1–§39.2.

39.3 (a) **Compute**-bound — widening arithmetic (SIMD) moves the number. (b) **Bandwidth**-bound — streaming an array larger than cache waits on DRAM. (c) **Control**-bound — a data-dependent branch swinging with input order is misprediction. (d) **Latency**-bound (a memory ceiling) — a scattered pointer chase misses cache per node. *Tempting wrong answer:* calling (b) compute-bound because “it does arithmetic” — it does, but it is limited by byte delivery, not by the adder. **Feed-forward:** §39.2.

39.4 (a) Overall speedup = $1 / ((1-p) + p/s)$, where p is the fraction of runtime optimized and s the local speedup. (b) With p=0.20, s=4: 1 / (0.80 + 0.20/4) = 1 / 0.85 ≈ **1.18×**. (c) The maximum, at $s \to \infty$, is 1 / (1−0.20) = 1.25× — so even infinite speedup on this function caps the program at 1.25×, which says the other 80% is where the larger remaining gains live: look there next. *Tempting wrong answer:* “4× on 20% gives about 4×” — no; the 80% you did not touch dominates, giving only ~1.18×. **Feed-forward:** §39.3.

39.5 (a) It is **latency**-bound (a memory ceiling): a pointer chase over scattered nodes with a working set beyond L3 misses cache on essentially every node; confirm by checking that time is in the traversal (not in `malloc`), that the miss rate is high (or, without `perf`, that shrinking the working set below cache collapses the time), and that access is non-sequential. (b) The fix is **contiguity** — an arena or a flat array with indices instead of pointers (Chapter 38, echoing Chapters 34–35) so the walk prefetches; SIMD, branchless code, and a faster allocator would *not* help, because none of them changes the scattered layout. (c) With the loop now 30% and the profile flat, Amdahl's law says the next single change is bounded low — stop optimizing this, or move to whatever the new dominant cost is if one emerges. *Tempting wrong answer:* “swap in a faster allocator” — the time is in the traversal's cache misses, not in allocation. **Feed-forward:** §39.2–§39.3.

39.6 (a) **Compute** ceiling → SIMD; echoes the 13× kernel of §36.2. (b) **Bandwidth** ceiling → locality/layout (SIMD barely helps); echoes the 2.4× dot product of §36.3. (c) **Control** ceiling → make the branch predictable or branchless; echoes the 15× swing of §37.2. (d) **Allocation/locality** ceiling → arena or flatten for contiguity; echoes the 22× scattered-vs-contiguous traversal of §38.2. *Tempting wrong answer:* giving (b) SIMD as the fix "because it's arithmetic" — it is bandwidth-bound, so lanes don't help; layout does. **Feed-forward:** §39.2.

39.7 (a) Optimizing A (p=0.60, s=5): 1 / (0.40 + 0.60/5) = 1 / 0.52 ≈ **1.92×**. Optimizing B (p=0.30, s=5): 1 / (0.70 + 0.30/5) = 1 / 0.76 ≈ **1.32×**. (b) Do **A** first — for the same effort and same local speedup, the larger fraction yields the larger overall gain; Amdahl's law makes the bigger share always the better target. (c) After speeding A 5×, A's time falls to 0.12 of the original while the rest (the 0.40 of non-A work) is unchanged, so the new runtime is 0.52 of the original; B is now 0.30 / 0.52 ≈ 58% of the new runtime, so B has become the dominant cost and is now clearly worth doing. *Tempting wrong answer:* "do B first because it's easier to reason about" — effort is equal by assumption, so the larger fraction wins. **Feed-forward:** §39.3.

39.8 **Step 4:** The ~1× is expected, not a bug: the loop is bandwidth-bound, so it is limited by how fast DRAM delivers bytes, and SIMD widens the arithmetic — a ceiling the loop was never under — so the memory bus caps it at roughly the scalar throughput. A profile beforehand would have shown the loop stalled on memory (a high miss rate or, without `perf`, throughput flat as you widen the arithmetic and rising only if you shrink the working set below cache), identifying the bandwidth ceiling before any code was written. The PR should be dropped despite the "more advanced" code because it adds CPU-specific complexity for no measured gain, and the real lever — if the loop must be faster — is moving fewer bytes (locality and layout, Chapters 34–35), not more lanes. *Tempting wrong answer:* "keep it, advanced code is better" — complexity with no measured speedup is a net loss. **Feed-forward:** §39.2, §39.4.

39.9 The shared trap is **acting on a model of the hardware instead of a measurement of it** — reasoning from "fewer instructions," "more lanes," "no branch," or "faster allocator" without evidence. What would have caught each: (a) a profile showing the function is 2% of runtime (Amdahl caps the win at ~1.02×); (b) timing scalar vs vectorized, or noting the working set exceeds cache — the number does not move; (c) reading the emitted assembly and finding a `cmov`, not a conditional jump, so there is no branch to remove; (d) a profile showing the time is in the traversal's cache misses, not in `malloc`. *Tempting wrong answer:* treating them as four unrelated mistakes — they are one mistake (no measurement) in four costumes. **Feed-forward:** §39.4.

39.10 (a) **False** — the fastest program is the one that raises the ceiling it is actually under; a cache-hostile loop with fewer instructions can be slower than a cache-friendly one with more (Chapter 34). (b) **True** — 1 / 0.95 ≈ 1.05× is the Amdahl ceiling for a 5% part. (c) **False** — each raises a different ceiling (compute, control, allocation) and does little or nothing against the others. (d) **True** — re-measuring is mandatory; the layers routinely make the “obvious” change do nothing or lose. (e) **False** — a program is under one dominant ceiling at a time; raise that one, re-measure, and the next surfaces. *Tempting wrong answer:* marking (a) true from the instruction-count intuition — the machine counts stalls, not just instructions. **Feed-forward:** §39.1–§39.4.

39.11 (a) Every trap is reasoning from a model of the hardware — operation counts, lane counts, source branches, allocator speed — when the machine's interacting layers (compiler, cache, predictor, allocator, bus) routinely make the intuitive answer wrong; the one common defense is evidence: profile and time before, re-measure after. (b) What decides the outcome is whether the tool matches the ceiling the loop is actually under: SIMD gave 13× on a compute-bound kernel but 2.4× on a bandwidth-bound one; a branch swung 15× when it survived and was unpredictable but nothing when the compiler removed it; contiguity gave 22× on a scattered traversal — same techniques, different ceilings, so the match, not the technique, sets the number. (c) Techniques change with hardware and language, but “find the binding ceiling and raise that one” is the invariant method that tells you *which* technique to reach for and when to stop — it survives every new tool and every new machine. *Tempting wrong answer:* “master all the techniques and apply them everywhere” — without diagnosis that is how the regressions in (b) get shipped. **Feed-forward:** §39.4–§39.5.

39.12 Open-ended and platform-specific. A correct write-up: (a) names a real hot loop found by profiling or timing and gives a baseline as the minimum of several trials; (b) identifies the ceiling with an actual test — a cache cliff as input size crosses last-level cache, a scalar-vs-vectorized timing, the emitted assembly for a surviving branch, or whether time is in the allocator versus the traversal — then applies only the matching tool and re-measures; (c) reports before/after honestly, including the outcome (common and valuable) that the change did *not* move the number and the ceiling was elsewhere, plus toolchain, CPU, and the note that these are wall-clock times if `perf` is unavailable. *Tempting wrong answer:* “apply SIMD and report the speedup” — that skips the diagnosis and may raise a ceiling the loop is not under. **Feed-forward:** §39.1–§39.2.

Chapter 40 — Solutions

40.1 File descriptor (indexing the open-file description that holds the offset) → the VFS → the **page cache** → the filesystem (offset-to-block mapping) → the block layer → the device driver and hardware. A cache **hit at the page cache** returns the bytes from DRAM without any device I/O; the layers below it are reached only on a miss. *Tempting wrong answer:* "`read` goes straight to the disk" — for a regular file it almost never does on a warm system; it goes to the page cache first. **Feed-forward:** §40.1.

40.2 Because per-byte I/O pays two system calls per byte, and the fixed per-call cost (the trap into the kernel and back, VFS dispatch, the copy) dwarfs the work of moving a single byte; a 64 KiB buffer pays that same per-call cost once per 65,536 bytes, amortizing it to under a nanosecond each — so the gap (~1604 vs ~0.53 ns/byte, ~3000×) is the number of syscalls, not the number of bytes. *Tempting wrong answer:* "the disk is the bottleneck" — the bytes are identical and (warm) never leave DRAM; the cost is the call frequency. **Feed-forward:** §40.2.

40.3 (1) The **user-space buffer** — owned by the application (stdio's internal buffer, or one you allocate) — turns many small logical reads/writes into few large system calls, solving the per-call cost. (2) The **page cache** — owned by the kernel, in DRAM — holds recently used file pages so a read that hits it skips the device and a write can return before reaching it, solving the device latency for resident data. *Tempting wrong answer:* "there is only the disk cache" — there are two independent buffers, and defeating the user-space one (per-byte calls) is catastrophic even with a warm page cache. **Feed-forward:** §40.2–§40.3.

40.4 (a) It is a region of otherwise-free DRAM in which the kernel keeps recently used file pages. (b) The warm read is ~13× faster because its pages are resident, so the read is a memory copy; the cold read must fetch from the storage device, which is far slower. (c) *Readahead* is the kernel fetching pages ahead of the current position on a sequential read, overlapping device latency with the program's work — so streaming access hides the latency that random access, which cannot be predicted ahead, pays in full. *Tempting wrong answer:* "warm is faster because the CPU cache holds the file" — 256 MiB far exceeds any CPU cache; it is the page cache in main memory. **Feed-forward:** §40.3.

40.5 (a) `read` returns short at end-of-file or when interrupted by a signal (and always may on a pipe/socket with less data ready); `write` can accept only part of the buffer when a pipe or socket buffer is nearly full. (b) `p = buf; left = n; while (left > 0) { w = write(fd, p, left); if (w <= 0) handle_error; p += w; left -= w; }`. (c) Because on exactly the inputs where the count is short — large files, slow pipes, signals — the naive code silently processes only part of the data and reports success, a correctness bug, not just a slow path. *Tempting wrong answer:* "one `read` gives the whole file" — it may, for small files on a warm cache, which is why the bug hides until it matters. **Feed-forward:** §40.4.

40.6 (a) The fixed per-call cost s is divided across W bytes, so per-byte cost falls roughly as $s/W + b$ — steeply while W is small. (b) Once W is large enough that s/W is negligible next to the per-byte term b (and the copy runs at memory bandwidth), further growth cannot remove a cost that is already gone, so the curve flattens. (c) "Large enough" — tens to hundreds of KiB — captures nearly all the win; "as large as possible" wastes memory and cache for no measurable gain. *Tempting wrong answer:* "keep doubling the buffer for more speed" — past the knee it buys nothing and costs memory. **Feed-forward:** §40.2, §40.4.

40.7 (a) Total = $k \cdot s + B \cdot b$ (the per-byte term depends on total bytes, not on how they are split). (b) Per-byte: $k = B$, so total = $B(s+b)$ — dominated by s when $s >> b$. Buffered: $k = B/W$, so total = $B \cdot s/W + B \cdot b$; the syscall term shrinks as $1/W$ and becomes negligible once $s/W << b$, i.e. $W >> s/b$ — beyond which only the fixed $B \cdot b$ remains and larger W barely helps. *Tempting wrong answer:* writing total as $k \cdot s$ only — the byte-moving term matters once the syscall is amortized. **Feed-forward:** §40.2.

40.8 **Step 4:** 5 GiB/s is a memory number because the read was served entirely from the page cache — the pages the write left resident — so it measured a DRAM copy, not the storage device, which never saw the read. To measure cold storage the colleague should evict the file's pages first (`posix_fadvise` with `POSIX_FADV_DONTNEED`, or a cache drop on an administered machine) and then time the read, and should report the buffer size, filesystem, and device. The difference can be an order of magnitude — this machine showed ~13× cold-to-warm and a spinning disk far more — so a warm number can promise throughput that production, which reads cold, will never deliver. *Tempting wrong answer:* "run it a few more times and average" — every warm run repeats the same DRAM measurement; averaging cache hits does not reveal the disk. **Feed-forward:** §40.4.

40.9 (a) **System-call frequency** — a cache-resident `getc` loop never touches the device; confirm by buffering and watching the time collapse. (b) **Device latency** — ten thousand cold, scattered small files each pay a device access (and often a seek); confirm that the same files run far faster warm. (c) **Neither dominates as a problem** — a 1 MiB buffer amortizes the syscall and the warm data is in DRAM, so it runs near memory speed; confirm the buffer is large and the pages resident. (d) **System-call frequency** — one `write` per event is per-byte I/O in disguise; confirm by batching writes. *Tempting wrong answer:* calling (a) device-bound "because it reads a file" — the file is already in DRAM. **Feed-forward:** §40.2–§40.4.

40.10 (a) **False** — `read` may return a short count when interrupted or on a pipe/socket; correct code loops. (b) **False** — a successful `write` means the bytes are in the page cache, not on the device; durability needs `fsync` (Chapter 41). (c) **False** — stdio buffers in user space (~4 KiB) and issues a real syscall only when the buffer fills/drains. (d) **True** — a page-cache hit is a DRAM copy with no device access. (e) **False** — the per-byte cost flattens past a knee (tens to hundreds of KiB); bigger buffers then waste memory for no gain. *Tempting wrong answer:* marking (b) true — the write-back cache is exactly why a returned `write` is not yet durable. **Feed-forward:** §40.2–§40.4.

40.11 (a) Both speedups remove work the machine was doing needlessly, not add hardware: buffering removes redundant *system calls* (the fixed per-call cost, paid per byte in the slow version), and the warm cache removes redundant *device accesses* (the bytes were already in DRAM). (b) As the server runs, the kernel fills the page cache with the file pages it reads, so an increasing fraction of reads hit DRAM instead of the device — throughput rises until the working set is resident, then stabilizes. (c) Because I/O time is dominated by call frequency and residency, both of which you control in software; disk speed is a fixed lower layer that only matters once the data is genuinely cold and the calls are already amortized — so those two questions find the fixable cost first. *Tempting wrong answer:* “faster disks are the general fix” — they do nothing for a syscall-bound or warm-cache workload. **Feed-forward:** §40.2–§40.5.

40.12 Open-ended and platform-specific. A correct write-up: (a) reports per-byte and 64 KiB-buffered copy times as the minimum of several trials and their ratio (expect a very large factor); (b) reports cold throughput after an explicit eviction and warm throughput after a pre-warm, clearly labelled, with the cold number lower; (c) states toolchain, CPU, filesystem, and buffer size, and notes these are wall-clock times without hardware counters, identifying which reads were cold and which warm. *Tempting wrong answer:* “time the read twice and report the faster one” — that reports the warm cache and hides the storage cost the exercise is about. **Feed-forward:** §40.3–§40.4.

Chapter 41 — Solutions

41.1 After `write` returns, the data is a **dirty page in the kernel's page cache**, in volatile DRAM, scheduled for write-back to the device later. A power loss, kernel panic, or hard reset before that write-back **loses it**, even though the call returned success. *Tempting wrong answer:* "it is on the disk" — the return only means the kernel accepted the bytes into the cache. **Feed-forward:** §41.1.

41.2 `fsync` forces the file's dirty pages out of the page cache to the storage device and waits until the device confirms them durably stored — a synchronous durability barrier `write` never provides. `fdatasync` skips flushing non-essential metadata (such as the modification timestamp), so when only the data and size need to be durable it can avoid an extra metadata write and be slightly cheaper. *Tempting wrong answer:* "`fdatasync` skips the data" — it skips only inessential metadata, never the data. **Feed-forward:** §41.2.

41.3 `fflush` empties the C library's **user-space** buffer into the **kernel's page cache** — one buffer down, but the data is still in volatile DRAM and not on the device, so it is not durable. Durability requires `fsync/fdatasync` after the flush to cross the device barrier. *Tempting wrong answer:* "`fflush` writes to disk" — it writes to the kernel, which writes to disk later. **Feed-forward:** §41.1–§41.2.

41.4 (a) A buffered `write` returns after a memory copy (~0.9 µs); `fdatasync` must push the data through the block layer to the device and wait for the device to confirm it durable (~8.6 ms here), so the durable version pays a device round trip per record. (b) The ~8.6 ms is the time for the physical storage to accept and confirm a flush — a device property (media, controller, flush path) — not CPU work, which is why it dwarfs the memory-bound write. (c) **Group commit**: append many records, then `fsync` once for the whole batch, so the fixed flush cost is amortized across, say, 1000 records; it trades a small crash-window (the records in the current un-synced batch may be lost) for throughput, while every *acknowledged* record is still on the device before its acknowledgement. *Tempting wrong answer:* "sync less often and hope" — group commit is precise: acknowledge only after the batch's flush returns. **Feed-forward:** §41.2.

41.5 (a) It bypasses the page cache; data moves directly between the device and the user buffer by DMA, with no kernel copy and no caching. (b) The buffer address, file offset, and length must be aligned to the device block size (512 or 4096 bytes), so an arbitrary `malloc`'d buffer can fail with `EINVAL` — you need `posix_memalign`. (c) A **database** with its own buffer pool wants it (the page cache would just be a redundant, uncoordinated second cache); a **program that rereads a small file often** is harmed (it forfeits the huge cache-hit win of §40.3). *Tempting wrong answer:* "`O_DIRECT` is always faster because it is direct" — it forfeits readahead, write-back, and cache hits, losing badly for cache-friendly access. **Feed-forward:** §41.3.

41.6 (a) `mmap` avoids the copy from the page cache into a user buffer; a mapped access reads the page-cache page in place. (b) They came out close because `mmap` trades the saved copy for a **minor page fault per 4 KiB page** on first touch (and the page-table setup), so for a single pass the two costs roughly cancel. (c) `mmap` clearly wins for **random or repeated access** to a large file (the fault cost is amortized and you get zero-copy in-memory access); buffered `read`/`write` wins for a **single sequential streaming pass**, where readahead is already optimal and you want explicit control of when I/O happens. *Tempting wrong answer:* "`mmap` is always faster because it is zero-copy" — the eliminated copy is offset by page-fault cost, so it is workload-dependent. **Feed-forward:** §41.4.

41.7 (a) Per record with sync-each: $w + f$. Per record with one `fsync` per k: $w + f/k$. (b) At k=1: 0.9 µs + 8600 µs ≈ **8.6 ms/record** (~116/s). At k=1000: 0.9 µs + 8.6 µs ≈ **9.5 µs/record** (~105,000/s). (c) The batch risks losing up to the last k un-acknowledged records if a crash happens before their shared flush, so the tolerable data-loss window (in records or time) sets the upper bound on k — you make the batch as large as that window allows, no larger. *Tempting wrong answer:* "bigger k is strictly better" — it is, for throughput, only up to the crash-loss tolerance. **Feed-forward:** §41.2.

41.8 **Step 4:** Faster disks help only marginally because each acknowledgement is one device-flush round trip, and even a fast SSD flush is tens to hundreds of microseconds — a modest constant, not the 100×–1000× the app needs; the real lever is the *number* of flushes, which batching cuts. Amortizing one ~8.6 ms flush across 1000 records gives ~9.5 µs/record, on the order of 100,000 records/s — a ~1000× gain with no hardware change. The batch still guarantees that every *acknowledged* record is durably on the device (the batch is acknowledged only after its flush returns); what it does not guarantee is the survival of records written into the current batch but not yet flushed when a crash hits — that bounded tail is the deliberate trade. *Tempting wrong answer:* "buy faster disks and keep syncing each" — the per-flush floor still caps throughput far below the requirement. **Feed-forward:** §41.2, §41.4.

41.9 (a) **Not durable** — the data was only in the page cache; add `fdatasync(fd)` before acknowledging. (b) **Durable** — `fdatasync` returned, so the device has the data. (c) **Data durable but possibly unreachable** — the file's contents are synced but the directory entry from the rename may not have reached the disk; add an `fsync` on the *directory* after the rename. (d) **Not durable** — dirtied mapped pages need `msync` (and the underlying `fsync` physics) to be forced; a clean process exit does not guarantee write-back before the power loss. *Tempting wrong answer:* calling (c) fully durable "because the file was fsync'd" — the directory entry is separate metadata that also needs syncing. **Feed-forward:** §41.1–§41.4.

41.10 (a) **False** — it is in the page cache, not on the disk, until write-back or `fsync`. (b) **False** — `fsync` is slow because it waits for a physical device flush, not because of the byte copy. (c) **False** — `O_DIRECT` forfeits readahead, write-back, and cache hits, so it is slower for cache-friendly workloads. (d) **True** — multiple processes can map the same file and share one set of physical pages. (e) **False** — durability is defined by surviving a crash, so a claim untested against a real crash is unproven. *Tempting wrong answer:* marking (a) true from the "write means written" intuition — the write-back cache makes it false. **Feed-forward:** §41.1–§41.4.

41.11 (a) Because a single `fsync` is ~9000× a buffered write, syncing every scattered data-file update would be ruinous; a write-ahead log turns all the changes of a transaction into one sequential append and `fsync`s that *once*, so durability costs one flush per transaction (or per group) instead of one per modified page. (b) A database's buffer pool is already a cache of pages with its own replacement policy; letting the page cache hold a second copy doubles memory use and lets two uncoordinated caches fight, so `O_DIRECT` removes the redundant kernel cache and gives the database sole control. (c) Because the durability barrier is the one step bounded by physical storage (~9000× a memory write), the count of barrier crossings dominates a storage system's throughput, so "how often must I `fsync`" sets the architecture — logging, batching, group commit — before any other choice. *Tempting wrong answer:* "optimize the data-file writes first" — those are cheap buffered writes; the flush is the cost. **Feed-forward:** §41.2–§41.5.

41.12 Open-ended and platform-specific. A correct write-up: (a) reports per-record cost with and without `fdatasync` and a large ratio; (b) shows per-record cost falling roughly as $w + f/k$ as the batch grows, flattening once the per-record flush share is small; (c) reports `read` and `mmap` throughput on a warm file with their run-to-run variance, noting the two are close and workload-dependent; and states toolchain, CPU, filesystem, and device, flags wall-clock timing without hardware counters, and identifies a virtual disk if present since it sets the absolute flush cost. *Tempting wrong answer:* "report the single fastest `fsync` time as the durable throughput" — durable throughput is bounded by the flush you must do per acknowledgement, not the best case. **Feed-forward:** §41.2, §41.4.

Chapter 42 — Solutions

42.1 A blocking `read` uses no CPU because the kernel takes the thread off the run queue entirely and wakes it only when data arrives — waiting is not spinning. Two costs a thread carries regardless of activity: its **stack** (often a megabyte or more of reserved address space) and its presence as a **scheduler-visible entity** (run-queue bookkeeping and context-switch cost when it wakes). *Tempting wrong answer:* "idle threads are free" — they hold memory and scheduler state whether or not they run. **Feed-forward:** §42.1.

42.2 It makes the `read` return immediately instead of parking the thread, reporting the error `EAGAIN` (equivalently `EWOULDBLOCK`) to mean "nothing ready now, try later." *Tempting wrong answer:* "it returns 0 bytes / EOF" — 0 means end-of-stream; not-ready is the distinct `EAGAIN`. **Feed-forward:** §42.2.

42.3 `select` passes the whole descriptor set on every call and the kernel scans all of them to find the ready ones; `epoll` registers interest once and each `epoll_wait` returns only the descriptors that have become ready. **`select`'s** cost grows with the total number watched. *Tempting wrong answer:* "`epoll` is just a faster `select`" — it is a different model (register-once, return-active), not merely a tuned scan. **Feed-forward:** §42.3.

42.4 (a) Each connection needs its own thread, and each thread costs a **stack** (gigabytes of address space across ten thousand) and **scheduler churn** (context switches to wake the few that become ready). (b) The costs are properties of having the thread, not of running it — an idle thread still reserves its stack and still sits in the scheduler's structures. (c) The event loop ties only a **descriptor and a little state** to each connection, not a thread and a stack, so a few threads can hold hundreds of thousands of connections. *Tempting wrong answer:* "shrink the stacks and it scales" — smaller stacks help a little, but the scheduler cost and the fundamental one-thread-per-wait mismatch remain. **Feed-forward:** §42.1, §42.4.

42.5 (a) Because the thread never sleeps: it spins issuing `read` after `read`, mostly collecting `EAGAIN`, so it uses a full core asking "ready yet?" instead of waiting for free. (b) A tight scan of 10,000 descriptors at ~199 ns/syscall is ~1.99 ms per full pass, so a single thread does on the order of **~500 passes/s = ~5 million syscalls/s**, nearly all returning `EAGAIN` — a whole core burned to discover that almost nothing is ready. (c) Readiness notification lets the thread **sleep in one call** until at least one descriptor is ready and wake with the list of ready ones, restoring the CPU-free wait of blocking while still serving many descriptors. *Tempting wrong answer:* "add a small sleep between passes" — that trades CPU for latency and still scans everything; readiness notification removes the scan. **Feed-forward:** §42.2–§42.3.

42.6 (a) `select` climbs roughly linearly — ~0.7 µs at 10, ~2.6 at 100, ~14.6 at 500 — while `epoll_wait` stays flat at ~0.4 µs. (b) On every call `select` copies and scans the entire descriptor set to find the ready ones (O(N)); `epoll` already holds the registered interest and returns only the descriptors that became ready (O(active)), doing no full scan. (c) `FD_SETSIZE` (1024 here) is the fixed size of the `fd_set` bitmap, so `select` cannot watch a descriptor numbered at or above it — fatal for a server needing tens of thousands of descriptors. *Tempting wrong answer:* "raise `FD_SETSIZE` and `select` scales" — even ignoring the cap, the O(N) rescan still dominates at large N. **Feed-forward:** §42.3.

42.7 (a) `epoll` is dramatically better when M is large but a is small — many watched, few ready per wakeup (the typical server), so $p \cdot M$ is huge while $q \cdot a + r$ is tiny; they are comparable when $a \approx M$ (nearly all ready every wakeup), since both then do work proportional to M. (b) At M=500, a=1: ~14.6 µs vs ~0.4 µs, about **35–40×**. (c) When almost every watched descriptor is ready each wakeup, `epoll` must return them all, so its cost rises toward `select`'s — the advantage comes from the active set being much smaller than the watched set. *Tempting wrong answer:* "`epoll` is always O(1)" — it is O(active), which equals O(N) when everything is active. **Feed-forward:** §42.3.

42.8 **Step 4:** The one thread runs all handlers in sequence, so while that handler is blocked in the cold `read` for milliseconds it cannot call `epoll_wait` or service any other ready descriptor — every connection that thread owns waits the same milliseconds, which is why the spikes are correlated across thousands of unrelated requests. The load test missed it because the config file was warm in the page cache, so the read returned instantly and never actually blocked — the bug only appears when the file is cold, a condition the test did not create. The fix is to remove all blocking from handlers: read the config once at startup (or via a thread pool) rather than in a request handler, and generally dispatch any potentially-blocking operation to a thread pool whose completion returns to the loop as an event. *Tempting wrong answer:* "add more event-loop threads" — that reduces but does not remove the blast radius; the blocking call must leave the loop entirely. **Feed-forward:** §42.4.

42.9 (a) **Safe** — a non-blocking `read` returning `EAGAIN` is exactly the model; it returns at once. (b) **Move to a pool** — `fsync` blocks on a device flush (~8.6 ms, §41.2) and would stall the loop. (c) **Safe** — parsing 200 bytes already in memory is a trivial, bounded CPU cost. (d) **Move to a pool** — a synchronous DNS lookup blocks on the network (use an async resolver or a pool). (e) **Move to a pool** — a CPU-bound image resize runs long and would monopolize the loop; offload it. *Tempting wrong answer:* calling (e) safe "because it does not do I/O" — long CPU work stalls the loop just as a blocking syscall does. **Feed-forward:** §42.4.

42.10 (a) **False** — a blocking `read` sleeps off the run queue and uses no CPU while waiting. (b) **False** — `epoll_wait`'s cost grows with the number of *active* descriptors, not the total registered. (c) **False** — non-blocking I/O alone forces a busy-poll; you also need readiness notification to sleep until something is ready. (d) **True** — one thread drives all its connections, so a blocking call stalls them all. (e) **False** — `select` cannot handle a descriptor at or above `FD_SETSIZE` (1024 here). *Tempting wrong answer:* marking (b) true by analogy to `select` — the whole point of `epoll` is that it does not scale with the total watched. **Feed-forward:** §42.1–§42.4.

42.11 (a) Blocking gives a CPU-free wait but only on one descriptor; non-blocking lets one thread touch many descriptors but, alone, forces a wasteful busy-poll; readiness notification supplies the one missing call that sleeps until some descriptor is ready and returns which — together they are the event loop, each supplying what the others lack. (b) Thread-per-connection fails when connections vastly outnumber active work (a heavy thread per idle wait); the event loop fails when one handler blocks (a whole thread's connections stalled by one operation) — the single principle is to match the resource to the work: a cheap descriptor for a waiting connection, a thread for genuinely blocking or CPU-heavy work. (c) Because the I/O model's cost is set by how many descriptors you watch and how many are active per wakeup — the exact variables that separate `select`'s O(N) rescan from `epoll`'s O(active) delivery — so those two numbers pick the model before any code is written. *Tempting wrong answer:* “always use an event loop” — at small scale thread-per-connection is simpler and fine; the numbers decide. **Feed-forward:** §42.4–§42.5.

42.12 Open-ended and platform-specific. A correct write-up: (a) reports a minimal kernel-entering syscall as ns/call (a few hundred ns is typical, higher under virtualization) and contrasts it with a vDSO call that is several times cheaper because it avoids a real kernel entry; (b) shows `select`'s per-call cost rising with the descriptor count while `epoll_wait` stays flat, and notes the `FD_SETSIZE` point past which `select` cannot go; (c) states toolchain and CPU, flags wall-clock timing without hardware counters, and notes that absolute syscall cost varies with the environment while the scaling shape is portable. *Tempting wrong answer:* “report `epoll` is faster by a fixed factor” — the factor grows with the descriptor count; the scaling, not a single ratio, is the result. **Feed-forward:** §42.1–§42.3.

Chapter 43 — Solutions

43.1 Server: `socket` → `bind` (attach to a local address/port) → `listen` (mark passive, set the backlog) → `accept` (block until a client connects). Client: `socket` → `connect`. `accept` returns a **new descriptor** for the one accepted connection, leaving the listening socket open for the next client. *Tempting wrong answer:* “`accept` returns the listening socket” — it returns a distinct per-connection socket; the listener keeps listening. **Feed-forward:** §43.1.

43.2 `SOCK_STREAM` selects TCP; `SOCK_DGRAM` selects UDP. TCP guarantees reliable, ordered delivery (and flow control) that UDP does not; UDP preserves **message boundaries** (one `sendto` = one `recvfrom`) that TCP's byte stream does not. *Tempting wrong answer:* “UDP is just TCP without the checksum” — UDP drops delivery, ordering, flow, and congestion control, not merely error checking. **Feed-forward:** §43.2–§43.3.

43.3 `TCP_NODELAY` (via `setsockopt` at level `IPPROTO_TCP`) disables Nagle's algorithm; set it on latency-sensitive small-message request/response protocols (RPC, databases, interactive services). *Tempting wrong answer:* “always set it, on every socket” — on bulk-transfer sockets Nagle does no harm and coalescing helps, so it is not a universal win. **Feed-forward:** §43.4.

43.4 (a) Any split is possible and TCP permits it: one `read` returning all 300; 60 then 240; one byte per `read`; etc. — because TCP is a byte stream and coalesces or splits writes into whatever segments it chooses. (b) TCP guarantees only that all 300 bytes arrive **in order and without loss or duplication** — nothing about grouping. (c) Length-prefix framing: prepend each message with its byte length; the receiver reads into a buffer in a loop, first reading the fixed-size length, then reading until that many body bytes have accumulated, then delivering one message and repeating. *Tempting wrong answer:* “one `read` per message because I sent one `write` per message” — writes and reads do not correspond on a stream. **Feed-forward:** §43.2.

43.5 (a) Client → `SYN` (proposes its initial sequence number); server → `SYN+ACK` (acknowledges the client's and proposes its own); client → `ACK` (acknowledges the server's). After it, both sides know each other's starting sequence numbers and the channel is open. (b) Setup costs a round trip because the handshake requires a message from each side before data can flow; a client that opens a fresh connection per request therefore pays one RTT of pure setup latency (plus TIME_WAIT churn) on every request — the argument for connection pooling and keep-alive. (c) TIME_WAIT is entered by the side that closes *first* (the active closer), which holds the socket for twice the maximum segment lifetime so that stray delayed segments from the closed connection cannot be mistaken for data on a new connection reusing the same port. *Tempting wrong answer:* “TIME_WAIT is a bug/leak to eliminate” — it is a correctness mechanism; `SO_REUSEADDR` lets a server rebind despite it, but it is doing a job. **Feed-forward:** §43.1.

43.6 (a) Bulk file transfer: **TCP** — you need every byte, in order, with flow/congestion control. (b) DNS query: **UDP** — one tiny request and reply; a TCP handshake would cost more than the query, and a lost query is cheaply retried. (c) Live VoIP: **UDP** — a late packet is useless, so TCP's retransmit-and-reorder would *add* latency and stall the stream; better to skip the lost audio. (d) Bank transaction: **TCP** — reliability and ordering are non-negotiable. (e) New protocol wanting its own congestion control and 0-RTT: **UDP** — TCP's fixed handshake and in-order delivery are exactly the machinery you want to replace (this is QUIC). *Tempting wrong answer:* “UDP for (a) because it is faster” — without reliability you would corrupt the file. **Feed-forward:** §43.3.

43.7 (a) The client's second small write is held by Nagle until the first segment is acknowledged; the server's delayed-ACK holds that acknowledgement (up to ~40 ms) hoping to piggyback it on a reply, but the server cannot reply until it has the whole request, whose tail is stuck in the client's Nagle buffer — so nothing moves until the delayed-ACK timer fires. (b) 25 round trips: with Nagle, 25×43.8 ms ≈ **1,095 ms** (over a second per user request); with `TCP_NODELAY`, 25×90 µs ≈ **2.3 ms**. (c) A throughput benchmark streams large buffers that fill segments, so Nagle never holds anything and the timer never fires — the pathology needs the small-write request/response shape, which throughput tests do not have. *Tempting wrong answer:* “the 40 ms is network latency in the rack” — intra-rack RTT is tens of microseconds; the 40 ms is a timer. **Feed-forward:** §43.4.

43.8 **Step 4:** Writing the length prefix and the body as two separate `write` calls with Nagle on means the second write is held until the first is acknowledged; the server's delayed-ACK will not acknowledge the prefix on its own, waiting to piggyback on a reply it cannot send until it has the full request — so every call waits out the ~40 ms delayed-ACK timer. The load test missed it because it measured aggregate throughput with the pipeline full, where the timer overlaps other in-flight work and never shows as per-call latency; only isolated sequential round trips expose it. Fixes, best first: (1) assemble the prefix and body into one buffer and issue a single `write`, so Nagle never engages; (2) set `TCP_NODELAY` on the socket. *Tempting wrong answer:* “switch to UDP” — that abandons reliability to dodge a one-line fix. **Feed-forward:** §43.4.

43.9 (a) **Buggy** — one `send` does not imply one `recv`; the stream may deliver your message in pieces or joined to the next, so you must frame and loop. (b) **No effect (harmless but pointless)** — a bulk-upload socket sends full segments, so Nagle never holds anything; `TCP_NODELAY` changes nothing there. (c) **Buggy** — UDP does not redeliver lost datagrams; if you need reliability you must implement it. (d) **Correct** — reading in a loop until a full length-prefixed message is assembled is exactly the right way to consume a stream. (e) **Buggy** — you must talk to the client on the descriptor `accept` returned, not the listening socket. *Tempting wrong answer:* calling (b) a real speedup — it is a no-op on that traffic pattern. **Feed-forward:** §43.2–§43.4.

43.10 (a) **False** — a connected socket is a descriptor and supports `read`/`write` like any other. (b) **False** — UDP does not guarantee ordering; datagrams may arrive in any order. (c) **True** — the three-way handshake spends one RTT before data. (d) **False** — a socket that only sends large buffers already fills segments, so disabling Nagle changes nothing. (e) **False** — on a byte stream, writes and reads do not correspond one-to-one. *Tempting wrong answer:* marking (b) true by analogy to TCP — ordering is precisely one of the guarantees UDP drops. **Feed-forward:** §43.1–§43.4.

43.11 (a) "Just a descriptor" is true in that once connected, the entire I/O API (`read`/`write`/`close`/`epoll`/`O_NONBLOCK`) applies unchanged — that uniformity is what lets the event loop of Chapter 42 treat sockets and files alike; it is incomplete because setup (bind/listen/accept or connect, the handshake) and transport semantics (byte stream, framing, boundaries) have no file analogue. (b) The accurate framing is *which guarantees you get for free versus build yourself*: TCP gives reliability, ordering, flow, and congestion control as a fixed package; UDP gives boundaries and nothing else, so you supply exactly the guarantees you need — QUIC proves it by building reliability, ordering, and its own congestion control atop UDP precisely to escape TCP's fixed choices, which no "reliable vs fast" story explains. (c) Nagle and delayed-ACK are each a correct local optimisation (fewer small packets; fewer bare ACKs) that compose into a 40 ms global stall — a recurring systems shape where independently sensible policies interact pathologically, which is why integration and end-to-end latency testing (not just component or throughput tests) is essential. *Tempting wrong answer:* "one of the two optimisations is simply wrong" — each is reasonable alone; the defect is their interaction with a chatty write pattern. **Feed-forward:** §43.2–§43.5.

43.12 Open-ended and platform-specific. A correct write-up: (a) shows the write-write-read round trip stalling near the delayed-ACK timer (~40 ms on Linux) with Nagle on and dropping to tens of microseconds with `TCP_NODELAY`; (b) demonstrates that a single `read` can return more than one back-to-back message, then shows a length-prefix loop recovering them; (c) states toolchain and CPU, flags wall-clock timing without hardware counters, and explains that the ~40 ms is a protocol timer (Nagle waiting on a delayed ACK) rather than network latency, since loopback RTT is negligible. *Tempting wrong answer:* "report the stall as the socket being slow" — the stall is a specific timer, reproducible and removable, not a bandwidth or CPU limit. **Feed-forward:** §43.2–§43.4.

Chapter 44 — Solutions

44.1 Readiness asks “which descriptors can I operate on without blocking?” (and then you do the operation yourself); completion asks “perform this operation and tell me when it is finished.” *Tempting wrong answer:* “they ask the same thing, one is just faster” — they ask fundamentally different questions, which is why completion works for disk and readiness does not. **Feed-forward:** §44.1.

44.2 The **submission queue** (SQEs) — the application writes it, the kernel reads it — and the **completion queue** (CQEs) — the kernel writes it, the application reads it. *Tempting wrong answer:* “both are written by the kernel” — the application fills submissions; that is the point of the shared submission ring. **Feed-forward:** §44.2.

44.3 **One** `io_uring_enter` submits all 200 and waits for their completions, versus **200** separate `pread` system calls — a 200× reduction in kernel crossings. *Tempting wrong answer:* “one `io_uring_enter` per operation” — the whole design batches the boundary; one call handles the batch. **Feed-forward:** §44.2–§44.3.

44.4 (a) `epoll_wait` only tells you a descriptor is ready; you must still cross the boundary to do the `read`/`write` — so each handled operation is at least one more syscall. (b) A regular file is always “ready” (you can always attempt the read), but the attempt still blocks on the device on a cache miss, and there is no event meaning “this read will not block,” so `epoll` has nothing to watch for a file. (c) “Tell me when it is done” is meaningful for any operation — the kernel performs it and reports completion whether the wait was for socket data or a disk seek — whereas “which are ready” needs an observable ready state that disk files do not have. *Tempting wrong answer:* “use non-blocking mode on the file” — `O_NONBLOCK` does not make ordinary file reads non-blocking on a cache miss. **Feed-forward:** §44.1.

44.5 (a) An SQE holds an opcode (e.g. `IORING_OP_READ`), a descriptor, a buffer address, a length, an offset, and a 64-bit `user_data`; a CQE holds the result `res` (byte count or negative error) and the echoed `user_data`. `user_data` lets you match a completion to the request that produced it. (b) Fill three SQEs, advance the submission ring tail, call `io_uring_enter` once to submit and wait; then read three CQEs from the completion ring, using each `user_data` to find its request, and advance the completion head. (c) `SQPOLL` has a kernel thread poll the submission ring, so the application submits by writing shared memory with *no* system call at all — removing even the per-batch `io_uring_enter`. *Tempting wrong answer:* “`user_data` is required by the kernel to route the I/O” — it is opaque to the kernel and purely for the application's own matching. **Feed-forward:** §44.2.

44.6 (a) Because each `io_uring_enter` submitted and reaped a batch of 256 operations, so 65,536/256 = 256 calls covered all of them. (b) The reads hit the page cache, so each one is a 4 KiB in-kernel copy; that copy done 65,536 times dominates the total and dwarfs the few-hundred-nanosecond boundary cost that batching removed — so cutting the crossings barely moved the clock. (c) Real wins: (i) huge numbers of tiny operations (high-IOPS storage, packet processing) where the syscall boundary *is* the dominant cost; (ii) genuine device I/O, where the dominant cost is milliseconds of device latency the kernel absorbs on its own threads while yours submits more work and never blocks. *Tempting wrong answer:* “the measurement was flawed; io_uring must be faster” — the measurement is correct and instructive: batching helps only in proportion to the boundary's share of the cost. **Feed-forward:** §44.3.

44.7 (a) Ratio = $N(b+w)$ / $(Nw + (N/k)b)$ = $(b+w)$ / $(w + b/k)$. (b) Large when b is much greater than w (boundary dominates): the ratio approaches $b/(b/k) = k$; near 1 when w is much greater than b (work dominates), since both numerator and denominator are ≈ w. (c) Cache-hot reads have w = a 4 KiB copy (hundreds of ns to microseconds), well above b (a few hundred ns), so the ratio is near 1 — exactly the measured break-even; a real device makes w include milliseconds of latency, but that latency is absorbed by the kernel asynchronously rather than blocking your thread, so the relevant comparison becomes throughput/concurrency, not this per-op ratio, and completion wins decisively. *Tempting wrong answer:* “bigger k always helps a lot” — k only helps to the extent b matters; when w dominates, k is nearly irrelevant. **Feed-forward:** §44.3.

44.8 **Step 4:** The SQE points at a stack buffer whose lifetime ends when the function returns, but the kernel writes the read data into that buffer only when the operation completes — after the function has returned and the stack frame (and its buffer) has been reused by other calls, so the read lands in memory now holding something else, corrupting it. It is intermittent and passes tests because under light load the completion comes back before the frame is reused (the data lands correctly by luck); only when load or a slow device delays the completion past the frame's reuse does it corrupt. Fix: give the buffer a lifetime tied to the completion, not the call — allocate it in per-request state kept alive until the CQE arrives (found via `user_data`), or use a registered buffer, and free/reuse it only after reaping the completion. *Tempting wrong answer:* “add a small delay before returning” — that masks the race under some timings and still corrupts under others; the buffer must genuinely outlive the operation. **Feed-forward:** §44.4.

44.9 (a) **Wrong** — `io_uring_enter` returning means the batch was submitted (and possibly some completed if you waited), not that a given read is done; you know a read is done only when its CQE appears. (b) **Wrong** — completions arrive in finish order, not submission order; you must use `user_data` to match. (c) **Correct** — completion notification makes disk reads asynchronous, which readiness cannot. (d) **Correct** — one `io_uring_enter` can submit a whole batch. (e) **Wrong** — batching wins only when the boundary is a large share of the cost; on cache-hot data it was break-even. *Tempting wrong answer:* marking (e) correct from the “fewer syscalls” intuition — fewer syscalls is not fewer wall-clock ms unless the syscall was the bottleneck. **Feed-forward:** §44.3–§44.4.

44.10 (a) **False** — a regular file has no such readiness event; a cache-miss read blocks regardless. (b) **True** — the SQE's `user_data` is echoed in its CQE. (c) **True** — the buffer must stay valid until completion. (d) **False** — the rings are shared into the application's address space via `mmap`; both sides see them, which is what makes the boundary cheap. (e) **True** — batching 256 at a time turned 65,536 reads into 256 crossings. *Tempting wrong answer:* marking (d) true from “the kernel owns I/O” — the shared-memory rings are precisely how io_uring avoids per-op copying and syscalls. **Feed-forward:** §44.2–§44.4.

44.11 (a) Chapter 40's blocking `read` pays one boundary crossing and blocks the thread; Chapter 42's `epoll` removes the thread-per-wait but still pays a crossing per operation and cannot help disk; `io_uring` removes the per-operation crossing (batching) and makes disk truly asynchronous (completion), leaving the per-operation *work* (the copy, the device latency) as what remains — and the next chapter removes the copy. (b) The break-even result teaches the part's core discipline directly: a technique only helps in proportion to the cost it removes as a fraction of the total, so measuring reveals that batching the boundary is worthless when the copy dominates — a speed-up would have hidden that lesson behind a happy number. (c) Power-for-explicitness recurs throughout: manual memory management (Chapter 9) buys control at the cost of tracking lifetimes yourself; lock-free code (Chapter 23) buys scalability at the cost of reasoning about the memory model by hand; here completion buys async unification at the cost of managing buffer lifetime across the submit-complete gap. *Tempting wrong answer:* “io_uring makes the earlier models obsolete” — readiness is still right for sockets and simpler code; io_uring is the tool when the boundary or device concurrency is the bottleneck. **Feed-forward:** §44.1–§44.5.

44.12 Open-ended and platform-specific. A correct write-up: (a) submits an `IORING_OP_NOP` and reaps its CQE, confirming the ring setup and enter/reap loop work; (b) reports the synchronous loop crossing the kernel once per read and `io_uring` crossing once per batch (far fewer), with wall-clock times close on cache-hit data because the per-read copy dominates; (c) states kernel version, toolchain, and CPU, flags wall-clock timing without hardware counters, and notes that for uncached device reads the kernel absorbs the latency asynchronously so a single thread keeps many operations in flight — the regime where completion wins. *Tempting wrong answer:* “conclude io_uring is slower because wall-clock was break-even” — the measured win is in kernel crossings and in device-I/O concurrency, not in cache-hot throughput. **Feed-forward:** §44.3–§44.4.

Chapter 45 — Solutions

45.1 Twice: once from the page cache into the user-space buffer (by `read`), and once from that buffer into the socket send buffer (by `write`). *Tempting wrong answer:* “once” — people forget the buffer is copied *out* to the socket as well as *in* from the cache. **Feed-forward:** §45.1.

45.2 It removes the two CPU copies through the user-space buffer (and the buffer itself), sending the file to the socket inside the kernel; measured here it cut the system calls from 16,384 to **1**. *Tempting wrong answer:* “it makes the copy faster” — it removes the user-space copies, it does not speed them up. **Feed-forward:** §45.2.

45.3 Any transformation of the bytes — for example encrypting them for TLS (also compressing, transcoding, inspecting) — because the data never enters your address space, so there is no point at which your code can run over it. *Tempting wrong answer:* “none — sendfile can do anything read/write can” — it cannot touch the bytes, by design. **Feed-forward:** §45.4.

45.4 (a) DMA disk → page cache; CPU page cache → user buffer (`read`); CPU user buffer → socket buffer (`write`); DMA socket buffer → NIC/wire. (b) `sendfile` removes the two CPU copies through the user buffer; they are waste because the buffer is a pure relay that never reads or changes the data. (c) On ordinary hardware `sendfile` still copies once inside the kernel, from the page cache into the socket buffer; a NIC with scatter-gather DMA and checksum offload removes even that by gathering the page-cache pages straight onto the wire. *Tempting wrong answer:* “sendfile removes all copies on any hardware” — the last in-kernel copy needs scatter-gather DMA to disappear. **Feed-forward:** §45.1–§45.3.

45.5 (a) It removed the two user-space CPU copies (memory bandwidth) and the per-block `read/write` boundary crossings (the syscall tax) — 16,384 syscalls down to 1. (b) Because `sendfile` still does real work: on this hardware one in-kernel copy from the page cache to the socket buffer plus the DMA, so it removed a portion of the cost, not all of it — a ~2.3× win, not an unbounded one. (c) A real network card can use scatter-gather DMA to send straight from the page cache, removing the last in-kernel copy that the loopback path still pays, so the CPU leaves the data path entirely — a larger relative win. *Tempting wrong answer:* “the 2.3× is the theoretical max” — with hardware offload the CPU copy count reaches zero, beyond what loopback shows. **Feed-forward:** §45.2–§45.3.

45.6 (a) `splice` moves data between *any* two descriptors without a user-space copy, where `sendfile` is fixed to file-in, socket-out. (b) One end must be a pipe because the pipe is the kernel's internal conduit — a queue of references to kernel buffers (page-cache pages) — and `splice` moves those references rather than copying bytes. (c) For example: proxying one socket to another (socket → pipe → socket), and logging a socket to a file (socket → pipe → file). *Tempting wrong answer:* “the pipe is just a temporary buffer that holds a copy” — it holds references, not a byte copy; that is the point. **Feed-forward:** §45.3.

45.7 (a) Two CPU copies per byte at rate R: $\boldsymbol{2R/B}$ CPU-seconds per second. (b) One in-kernel copy: $\boldsymbol{R/B}$. (c) With scatter-gather DMA, **0** — the CPU makes no copy, so those cycles are freed for real work (request handling, TLS if offloaded, other connections). A file server becomes copy-bound because at high R, $2R/B$ can approach or exceed one core's worth of time spent purely on `memcpy`, so the CPU saturates moving bytes it never inspects — exactly the waste zero-copy targets. *Tempting wrong answer:* “copying is negligible, servers are network-bound” — at multi-GB/s a memory-bandwidth-bound double copy is a real, often dominant, CPU cost. **Feed-forward:** §45.1–§45.3.

45.8 **Step 4:** TLS requires transforming (encrypting) every byte before it goes on the socket, which needs the bytes in a buffer the code can run over; `sendfile` deliberately never brings the bytes to user space, so there is no point in its path to encrypt them — the fast path structurally cannot serve TLS. The benchmark did not reveal it because it measured plaintext static serving, where no transformation is needed and `sendfile` shines; the requirement to transform was simply not exercised. Two ways forward: (1) keep zero-copy by enabling **kernel TLS** (kTLS), which moves encryption into the kernel so it encrypts page-cache data on the way to the socket and `sendfile` works over TLS; (2) give up zero-copy on the encrypted path and use the ordinary `read`/encrypt/`write` loop, optimising it differently. *Tempting wrong answer:* “encrypt the file at rest once and sendfile the ciphertext” — TLS session keys are per-connection, so you cannot pre-encrypt a shared file for all clients. **Feed-forward:** §45.4.

45.9 (a) **Applies** — plain HTTP relays the file unchanged; `sendfile` is ideal. (b) **Does not apply** — TLS must encrypt every byte and there is no kTLS, so the bytes must go through user space. (c) **Applies** — unchanged socket-to-socket relay is exactly what `splice` does via a pipe. (d) **Does not apply** — gzip transforms the bytes, so they must be in a buffer. (e) **Applies** — with kTLS the kernel encrypts on the way out, so `sendfile` keeps the zero-copy path over TLS. *Tempting wrong answer:* calling (b) applicable “because it is still just sending a file” — the encryption requirement forbids zero-copy without kTLS. **Feed-forward:** §45.2–§45.4.

45.10 (a) **False** — `sendfile` copies inside the kernel and never uses a user buffer. (b) **True** — a scatter-gather NIC gathers page-cache pages straight to the wire, so no CPU copy is needed. (c) **False** — encrypting in the app needs the bytes in user space, which `sendfile` refuses; the two cannot coexist in one step without kTLS. (d) **True** — `splice` requires one descriptor to be a pipe. (e) **False** — the two *DMA* copies are unavoidable and cheap; the waste is the two *CPU* copies through user space. *Tempting wrong answer:* marking (e) true by miscounting which copies are the waste — DMA copies are hardware and necessary. **Feed-forward:** §45.1–§45.4.

45.11 (a) Chapter 42 removed *blocking* — one thread waits on many descriptors via readiness instead of a thread per wait — leaving one syscall per operation and no help for disk; Chapter 44 removed the *per-operation kernel boundary* via completion-based batching and made disk async, leaving the per-byte copy; this chapter removes the *user-space copy*, leaving only the unavoidable DMA. (b) Zero-copy is fast *because* the bytes never enter your address space, and it cannot transform them *because* the bytes never enter your address space — one fact — so the fastest path is unusable whenever the job requires touching the bytes (TLS, compression), and the correct choice depends on whether transformation is needed, not on raw speed. (c) The four-copy diagram makes every cost visible as a physical move: two DMA copies (necessary), two CPU copies (waste), two boundary crossings — so "where are the bytes and who copies them" directly names what each technique in the part removes, from the event loop's avoided waits to `sendfile`'s avoided copies. *Tempting wrong answer:* "always pick the fastest primitive" — the usable primitive depends on whether you must transform the data. **Feed-forward:** §45.1–§45.5.

45.12 Open-ended and platform-specific. A correct write-up: (a) reports the `read`/`write` loop's throughput and its two syscalls per block against `sendfile`'s higher throughput and one (or few) syscalls; (b) attributes the gain to both removed user-space copies and removed boundary crossings, using the syscall counts to separate them; (c) states toolchain and CPU, flags wall-clock timing without hardware counters, and explains that loopback understates the win because there is no NIC to use scatter-gather DMA — the feature that removes the final in-kernel copy on a real network path. *Tempting wrong answer:* "conclude the loopback number is the real-world ceiling" — a scatter-gather NIC removes a copy loopback still pays. **Feed-forward:** §45.2–§45.3.

Chapter 46 — Solutions

46.1 Fastest to slowest: L1 hit ~1 ns, DRAM ~100 ns, flash-SSD read ~tens–100 µs, hard-disk seek ~10 ms (all published/typical). Each step down is roughly three orders of magnitude. *Tempting wrong answer:* putting SSD and DRAM in the same ballpark — an SSD read is about a thousand times slower than DRAM, not comparable. **Feed-forward:** §46.1.

46.2 A **seek** (moving the head to the right track) and **rotational latency** (waiting for the target sector to rotate under the head). Random reads pay both delays every time, while sequential reads pay them once and then stream adjacent sectors. *Tempting wrong answer:* naming only the seek — rotational latency (~4 ms at 7200 RPM) is a comparable second cost. **Feed-forward:** §46.2.

46.3 Because flash cannot be reprogrammed in place: to rewrite a page you must first *erase* its surrounding block, and erase operates on a whole **erase block** (hundreds of pages), not one page. *Tempting wrong answer:* "you erase a single page" — erase is per block; the read/program unit (the page) is much smaller than the erase unit. **Feed-forward:** §46.3.

46.4 (a) time = seek + rotational latency + transfer, naming the head move, the wait for the sector, and the data read. (b) Random 4 KiB: ~9 ms seek + ~4.2 ms half-rotation + ~0.03 ms transfer ≈ ~13 ms, dominated by seek and rotation; the transfer is negligible. (c) 1 MiB sequential: one seek + rotation up front, then ~7 ms of transfer at 150 MB/s — so per byte the positioning is amortized to near zero and the transfer rate governs. *Tempting wrong answer:* assuming the transfer term dominates a random read — for small reads it is the smallest term. **Feed-forward:** §46.2.

46.5 (a) A random read is electronic (no head or platter to move), so reaching any page costs the same; a small random write can't overwrite in place, so the controller writes elsewhere, remaps, and later erases/garbage-collects. (b) Write amplification: one logical small overwrite triggers read-erase-reprogram of a block plus garbage collection, so physical writes exceed logical ones. (c) Both prefer sequential writes — the hard disk because it avoids seeks, the SSD because large sequential writes align with erase blocks and cause less amplification. *Tempting wrong answer:* "an SSD's cheap random reads mean cheap random writes" — writes are the asymmetric, expensive direction. **Feed-forward:** §46.3.

46.6 (a) **I/O size** (1 MiB requests amortize per-request overhead over 256× more data than 4 KiB) and **access pattern** (random reads forfeit readahead and locality). (b) The single-4 KiB comparison being only ~1.24× means the pattern penalty is small on this flash-backed device (no seek), so most of the 49× is the I/O-size effect. (c) On a spinning disk each random read would pay a mechanical seek, so the access-pattern term would explode and come to dominate. *Tempting wrong answer:* attributing the whole 49× to "random is slow" — on this SSD the bigger factor is fetching in bulk. **Feed-forward:** §46.4.

46.7 (a) Design A: $N \times t_r$. Design B: $(N/100) \times t_r$. (b) With $t_r \approx 10$ ms and $N = 10^6$: A ≈ 10,000 s (~2.8 hours); B ≈ 100 s. (c) The 100× is the number of records that share one page read: packing related records into a page means one disk trip serves 100 of them, turning 100 random accesses into one — the entire argument for the page as the unit of I/O. *Tempting wrong answer:* ignoring that B still pays for locality — if the 100 records were scattered across 100 pages, B collapses back to A. **Feed-forward:** §46.2, §46.4.

46.8 **Step 4:** The 6 GB/s measured the page cache in DRAM, not the disk — a plain buffered `read` loop over a warm file never reaches the device, so the number is a memory-bandwidth figure. The conclusion is backwards: because they measured DRAM, they saw no difference between patterns and inferred layout doesn't matter, when the device (which they never touched) shows a ~49× gap that makes layout decisive. They should bypass the cache (`O_DIRECT` or drop caches) and vary both access pattern and I/O size, as in §46.4. *Tempting wrong answer:* "6 GB/s is the disk, their disk is just fast" — no disk here does 6 GB/s at 4 KiB random; that is DRAM. **Feed-forward:** §46.4.

46.9 (a) **Seek/positioning** — random reads on a spinning disk pay a mechanical seek each. (b) **I/O size** (and raw transfer rate) — large sequential SSD reads are bandwidth-bound, not seek-bound. (c) **DRAM ceiling** — a warm cache serves from memory, so "random" costs a memory access, not a disk one. (d) **Flash write amplification** — tiny random overwrites trigger erase/garbage-collection work. (e) **I/O size** — same device and pattern, only the request size differs, so 1 MiB amortizes overhead that 4 KiB cannot. *Tempting wrong answer:* calling (c) seek-bound "because it's random" — a cache hit never reaches the disk. **Feed-forward:** §46.1–§46.4.

46.10 (a) **False** — an SSD read (~tens–100 µs) is ~1000× slower than DRAM (~100 ns). (b) **False** (as stated) — the transfer rate is the same; sequential is faster because it pays seek and rotation once instead of per access. (c) **False** — random writes suffer the erase-in-place limitation and amplification; only random reads are cheap. (d) **True** — the block layer's smallest unit is a block, so a database reads/writes in block-multiple pages. (e) **False** — on this machine the `rotational` flag is a WSL2 virtio artifact; the backing store is an NVMe SSD. *Tempting wrong answer:* marking (b) true by crediting a higher transfer rate rather than amortized positioning. **Feed-forward:** §46.1–§46.4.

46.11 (a) "An access to disk is enormously more expensive than computation, and random access far more than sequential." The B-tree minimizes the *number* of disk accesses per lookup (high fanout, a few levels); the log turns writes into *sequential* I/O; the buffer pool keeps hot pages in DRAM so most accesses never reach the disk. (b) Both are true because they measure different things: ~49× is large sequential streaming versus small random I/O (dominated by I/O size on this flash device), while ~1.24× is sequential versus random at the *same* small 4 KiB size (isolating the pure pattern penalty, tiny without a seek). (c) The unifying discipline is knowing where bytes physically are and what reaching them costs — Volume 1's I/O part asked it of the page cache and the wire; here it is asked of the device, and the same answer (touch it rarely, in bulk, in the pattern it prefers) falls out. *Tempting wrong answer:* claiming the two ratios contradict — they measure different comparisons. **Feed-forward:** §46.1–§46.5.

46.12 Open-ended and platform-specific. A correct write-up: (a) reports the three `/sys/block` values for the right device; (b) gives sequential-large-block and random-small-page throughput, per-read latency, and IOPS under `O_DIRECT`; (c) states toolchain and CPU, flags wall-clock timing without hardware counters, notes whether the machine is virtualised (so absolute numbers reflect the stack, not a bare device), and separates the I/O-size effect from the access-pattern effect — for instance by also measuring single 4 KiB reads sequential vs random, which isolates the pattern penalty. *Tempting wrong answer:* reporting a buffered-`read` number as the device speed — that measures the page cache. **Feed-forward:** §46.4.

Chapter 47 — Solutions

47.1 Page 10 begins at offset 10 × 4096 = 40,960, and one page read transfers the whole page — 4,096 bytes. *Tempting wrong answer:* reading "just the record" — the storage manager always transfers a full page, not a byte range. **Feed-forward:** §47.1.

47.2 The **slot directory** (with a header) grows downward from the top; the **record data** grows upward from the bottom; free space sits between. One slot holds the (offset, length) of a record within the page. *Tempting wrong answer:* "a slot holds the record itself" — it holds a pointer to the record, not the bytes. **Feed-forward:** §47.2.

47.3 `write` only puts data in the OS page cache and returns, so a crash before flush loses it; `fsync` blocks until the device reports the data persisted, making it durable. *Tempting wrong answer:* "`write` writes to disk" — it writes to the cache; durability needs the barrier. **Feed-forward:** §47.4.

47.4 (a) A record identifier is (page number, slot number): the page number costs one page fetch (a buffer-pool hit, ~tens of ns, or a disk miss, ~hundreds of µs), the slot number an array index within the page. (b) Because outside references name the *slot*, not a byte offset, compaction can slide records to reclaim deletion holes and only update each slot's stored offset — every (page, slot) reference still resolves. (c) A forwarding pointer is left in a record's original slot when the record grows beyond its page's free space and must move to another page; it costs one extra page access to follow on read. *Tempting wrong answer:* "the record ID is the byte offset" — that would freeze records in place and break under compaction. **Feed-forward:** §47.2.

47.5 (a) Fixed part: null bitmap, then the `int` and `double` inline, then offsets/lengths for the two `varchar`s; variable-length area at the end holds the two strings' bytes. (b) The `int` and `double` have known sizes, so their positions are constant offsets; the `varchar`s have data-dependent lengths, so a stored offset must locate them. (c) A null bitmap uses one bit per nullable column — dense and cheap to test — and lets the column's storage be omitted, rather than reserving a whole sentinel value in every row. *Tempting wrong answer:* putting `varchar` bytes inline at a fixed position — their length isn't known at schema time. **Feed-forward:** §47.3.

47.6 (a) An `fsync` forces an actual, acknowledged trip to the device and cannot be pipelined, so per page it serializes the workload behind a full device round trip — ~3–4 ms each here versus ~30 µs amortized when one sync covers 2,000 pages. (b) The batched alternative writes changes to a sequential write-ahead log and issues one `fsync` per group of commits (group commit), keeping the same crash guarantee because the log is durable before the commit is acknowledged. (c) It passes correctness tests because each record really is durable after its sync, but it caps throughput at a few hundred durable writes per second, which only fails under load. *Tempting wrong answer:* "batching weakens durability" — group commit is equally durable, just amortized. **Feed-forward:** §47.4.

47.7 (a) Per record: $N \times s$. Per batch of k: $(N/k) \times s$. (b) With $s = 3$ ms, $N = 100{,}000$: $k = 1$ gives 300 s; $k = 1000$ gives 0.3 s. (c) Sustainable durable-commit rate is ~333/s for $k = 1$ and ~333,000/s for $k = 1000$ — a 1000× gain from amortizing one barrier over 1000 commits, which is exactly why group commit exists. *Tempting wrong answer:* assuming throughput scales with CPU, not with the sync barrier — the barrier is the bottleneck. **Feed-forward:** §47.4.

47.8 **Step 4:** Losing the last write leaves a consistent older page; a torn write leaves a page that is *neither* the old nor the new version — part of each — so it is corrupt and cannot be read as either, which is why the database won't open. A logging scheme fixes it by recording the page's intended contents durably *before* overwriting it in place: e.g. full-page logging or a double-write buffer writes the whole new page to a separate durable location first, so after a crash recovery detects the torn page and reconstructs it from the logged copy. *Tempting wrong answer:* “just retry the write” — the old contents are already gone, so without a logged copy there is nothing to recover to. **Feed-forward:** §47.4.

47.9 (a) **Fixed part** — a known-size 4-byte value read at a constant offset. (b) **Variable-length area** — a variable-size string stored at the record's end. (c) **Null bitmap** — one bit recording the column's absence. (d) **Fixed part** — a known-size 8-byte value. (e) **Fixed part** — the offset that locates (b) is itself fixed-size and lives inline. *Tempting wrong answer:* putting the description's *offset* in the variable area — the offset is fixed-size and must be at a known position to be found. **Feed-forward:** §47.3.

47.10 (a) **False** — a slotted page need not store records in sorted order; the slot directory decouples reference order from physical order. (b) **False** — a record identifier is (page number, slot number), not a raw byte offset. (c) **False** — after `pwrite` the data is in the page cache, not on the device; `fsync` is required. (d) **False** — one `fsync` per batch is equally durable, just amortized. (e) **False** — a torn write is corruption (part old, part new), not merely a lost update. *Tempting wrong answer:* marking (d) true, conflating “less frequent” with “less safe.” **Feed-forward:** §47.2–§47.4.

47.11 (a) Record identifier (page, slot) → fetch the *page* (buffer-pool hit or disk read — the cliff is here) → index the *slot* to get an (offset, length) → read the *record* bytes at that offset. The page level buys the unit of I/O and caching; the slot level buys stable addressing under compaction. (b) The ~100× per-page `fsync` penalty means durability cannot be paid per record and survive real load, so it must be batched behind a sequential log with one grouped barrier — the write-ahead log is the shape durability is *forced* into, not an optional tuning. (c) A record on disk carries its own variability (stored offsets for variable fields) and optionality (null bitmap) because it must be self-describing to survive serialization and be read back later, possibly under a newer code version — things an in-memory struct, tied to one program's live layout, never needs. *Tempting wrong answer:* treating the log as a mere speed optimization — the measurement shows it is a structural necessity. **Feed-forward:** §47.1–§47.5.

47.12 Open-ended and platform-specific. A correct write-up: (a) reports total time, per-page time, and durable writes/s for one-`fsync`-at-end versus per-page `fsync`; (b) repeats with `fdatasync` and notes it syncs data without necessarily all metadata, often similar here; (c) states toolchain and CPU, flags wall-clock timing and whether the disk is virtualised, and argues that the *ratio* (per-page vs batched) is the portable lesson while the absolute per-sync time depends on the device and stack. *Tempting wrong answer:* reporting the absolute per-sync time as a universal constant — it varies by device and virtualization. **Feed-forward:** §47.4.

Chapter 48 — Solutions

48.1 It maps a page identifier (page number) to a frame index. A lookup returns either “resident in frame N” (a hit) or “not resident” (a miss). *Tempting wrong answer:* “it maps offsets to bytes” — it maps page numbers to frames, the unit is a whole page. **Feed-forward:** §48.2.

48.2 A nonzero pin count means the page is in use by at least one caller and must not be evicted (evicting it would pull memory out from under an in-flight read). A dirty victim must be written back first because it has been modified since load, so the on-disk copy is stale and dropping it would lose the update. *Tempting wrong answer:* “write back every victim” — a clean victim's on-disk copy is already current, so it can just be dropped. **Feed-forward:** §48.2.

48.3 A hit (memcpy from a resident 4 KiB frame) was ~36 ns; a miss (`O_DIRECT pread`) was ~350 µs — a ratio of about 10,000×. *Tempting wrong answer:* guessing ~100× by analogy to a cache/DRAM gap — the disk cliff is four orders of magnitude, not two. **Feed-forward:** §48.3.

48.4 (a) EAT(h) = $h \times 36$ ns + $(1-h) \times 350$ µs. (b) $h = 0.90$ → ~35 µs; 0.99 → ~3.5 µs; 0.999 → ~0.4 µs. (c) Because misses cost ~10,000× a hit, they dominate the average until the hit rate is very near one; going 0.99→0.999 removes nine-tenths of the *remaining* misses (a ~10× cut) while 0.90→0.99 also removes nine-tenths but from a larger base — the marginal value keeps rising as you approach one, so sizing the pool to hold the working set is the high-leverage move. *Tempting wrong answer:* treating each 9% hit-rate gain as equally valuable — the value is in eliminating the last misses. **Feed-forward:** §48.3.

48.5 (a) Each frame has a reference bit set on access; a hand sweeps the circular frames — on a frame whose bit is *set*, it clears the bit and moves on (a second chance); on a frame whose bit is *clear*, it evicts that page. (b) True LRU would reorder a list on every access — costly on the hot path; clock approximates LRU with a single bit and no per-access reordering. (c) Clock's 18.1% sitting between FIFO's 17.2% and LRU's 18.6% confirms it captures most of LRU's benefit at far lower cost. *Tempting wrong answer:* “clock equals LRU” — it approximates LRU; the small gap (18.1 vs 18.6) is the approximation. **Feed-forward:** §48.4.

48.6 (a) With 110 pages cycling through 100 frames, by the time the scan wraps back to a page LRU has evicted it as least-recently-used to fit the 10 pages seen since — so every access is a miss, 0%. (b) It is worse than being uncached because the scan fills the pool with single-use pages, evicting other queries' hot pages and collapsing the whole system's hit rate while it runs. (c) Defenses: LRU-K (a once-touched scan page isn't counted as recently used, so it isn't kept), and a dedicated ring buffer for scans (so scan pages never displace the shared pool); DBMIN's per-query locality sets do the same by budgeting buffers per query. *Tempting wrong answer:* “make the pool bigger” — a scan larger than any pool still floods. **Feed-forward:** §48.4.

48.7 (a) When $W \le F$ the whole working set stays resident, so after warm-up the hit rate approaches 100%; in the looping-scan case with $W > F$ under LRU each page is evicted just before reuse, so the hit rate collapses toward 0% — a cliff at $W = F$. (b) Below the cliff EAT ≈ hit ≈ ~36 ns; above it EAT ≈ miss ≈ ~350 µs — about four orders of magnitude apart. (c) The trap is quantitatively this cliff: crossing $W = F$ flips nearly every access from a ~36 ns hit to a ~350 µs miss, so a scan that nudges the working set past the pool size is catastrophic, not gradual. *Tempting wrong answer:* assuming the hit rate degrades smoothly with W — under looping LRU it falls off a cliff at F. **Feed-forward:** §48.3–§48.4.

48.8 **Step 4:** Each scan page, used once, evicts a hot point-query page under LRU, so the point queries' resident working set shrinks and their hit rate falls (99.5% → 85%). Effect on their average access time (hit ~36 ns, miss ~350 µs): at 99.5% EAT ≈ 0.005 × 350 µs ≈ 1.75 µs; at 85% EAT ≈ 0.15 × 350 µs ≈ 52 µs — roughly a 30× jump, matching the observed latency spike. Fix without enlarging the pool: make the scan scan-resistant — read it through a small ring buffer or use LRU-K/DBMIN so its single-use pages don't evict the point queries' working set. *Tempting wrong answer:* “add RAM” — the scan exceeds any pool, so it will still flood without a scan-resistant policy. **Feed-forward:** §48.4.

48.9 (a) **Pin count** — a pinned (in-use) page is un-evictable. (b) **Dirty bit** — a modified victim is written back before reuse. (c) **Page table** — resolves a page number to a frame index. (d) **Replacement policy** — a scan-resistant policy keeps a scan from evicting the working set. (e) **Writeback ordering** — the log-before-page rule holds a dirty page until its log is durable. *Tempting wrong answer:* attributing (e) to the dirty bit alone — the dirty bit marks the page, but ordering the flush against the log is a separate mechanism. **Feed-forward:** §48.1–§48.2.

48.10 (a) **False** — a hit (~36 ns) and a miss (~350 µs) differ by ~10,000×. (b) **False** — clock only approximates LRU (18.1% vs 18.6% here). (c) **False** — a clean victim's on-disk copy is current, so it is dropped without a write. (d) **False** — a scan larger than any pool still floods; the fix is a scan-resistant policy. (e) **False** — at 90% the misses (each ~10,000× a hit) make up essentially the entire average. *Tempting wrong answer:* marking (e) true by intuition that “90% of accesses are hits, so hits dominate” — cost, not count, sets the average. **Feed-forward:** §48.2–§48.4.

48.11 (a) Query wants row R → resolve its record identifier (page, slot) → buffer-pool lookup on the page number: a hit reads the frame from DRAM (~36 ns — cheap), a miss reads the page from disk (~350 µs — the cliff) into a frame; then index the slot and read the record. The cliff is the buffer-pool miss. (b) The database knows its own access patterns and data structure — e.g. it can pin a page in use, order dirty writes against the log, or refuse to cache a scan — none of which the OS page cache exposes, and each changes the design toward correctness and scan-resistance. (c) Because a miss is ~10,000× a hit and the effective-access-time curve stays miss-dominated until the hit rate is very near one, the number that governs storage performance is the hit rate (how often you avoid the disk), not the disk's raw speed — you cannot make the disk fast, only visit it rarely. *Tempting wrong answer:* "faster disks are the main lever" — hit rate dominates until it is extremely high. **Feed-forward:** §48.1–§48.5.

48.12 Open-ended and platform-specific. A correct write-up: (a) reports a hit (memcpy from a resident buffer) and a miss (`O_DIRECT` `pread`) with their ratio; (b) reports FIFO, clock, and LRU hit rates from a fixed, seeded reference string including a looping scan larger than the pool (expect ~0% there); (c) states toolchain and CPU, flags the hit/miss times as wall-clock and whether the disk is virtualised, and distinguishes the *measured* latencies (device- and machine-dependent) from the *simulated* hit rates (exactly reproducible, a comparison of policies) — the distinction matters because only the simulation is a claim independent of hardware. *Tempting wrong answer:* presenting simulated hit rates as timed measurements — they are a deterministic policy comparison, not a clock reading. **Feed-forward:** §48.3–§48.4.

Chapter 49 — Solutions

49.1 It promises only that every record inserted and not yet deleted is returned exactly once by a scan. It deliberately does *not* promise any order — rows come back in whatever physical order they happen to occupy, because they were placed wherever there was free space. *Tempting wrong answer:* “rows come back sorted by key” — that ordering is an index's job, not the heap's. **Feed-forward:** §49.1.

49.2 The page number is a direct index into the file's linear page space and the slot number is a direct index into that page's slot array, so no search is needed: index the file to compute the byte offset of the page, fetch that one page through the buffer pool, and index the slot array to read the record's in-page offset and length. Two array indexings, one page fetch. *Tempting wrong answer:* “you scan the page for the record” — the slot array gives the offset directly. **Feed-forward:** §49.1.

49.3 The linked-list heap cost about 3,542 data-page fetches per insert; the page directory cost zero data-page fetches to *locate* free space (the search is an in-memory scan of the directory array), plus one fetch to write. The linked-list number is not a constant because it walks the chain of pages until one has room, and as the file fills with mostly-full pages that walk gets longer — the cost grows with file size. *Tempting wrong answer:* “both are roughly constant” — only the directory is; the list is *O(pages)*. **Feed-forward:** §49.2.

49.4 (a) A linked-list heap finds room by fetching each page and reading its free-space header, walking from the front; more pages (and fuller ones) mean a longer walk, so cost rises with the file. (b) A page directory keeps a compact per-page free-space entry in a separate in-memory array; the search scans that array (a memory op) and touches no data page until it fetches the single chosen page to write. (c) Linked list: *O(pages)* page fetches to find room; page directory: *O(1)* data-page fetches (an in-memory scan plus one write). For a growing table the first is unbounded and the second is flat — decisive. *Tempting wrong answer:* “the directory is a constant-factor win” — it changes the asymptotics, not just a constant. **Feed-forward:** §49.2.

49.5 (a) Same-size-or-smaller fits in the record's existing space, so the heap overwrites in place and only adjusts the slot length; a larger record may exceed the page's remaining free bytes, so it cannot stay. (b) Moving it would change its physical location — its (page, slot) RID — and every index entry storing that RID would dangle. (c) A forwarding pointer leaves a tombstone at the old RID redirecting to the record's new page, keeping the RID valid; the cost is two page fetches on every later read — the original page for the pointer, then the page where the row now lives. *Tempting wrong answer:* “just move it and update the indexes” — that requires touching every index and is what the forwarding pointer avoids. **Feed-forward:** §49.3.

49.6 (a) A workload that repeatedly grows rows after insertion — a status field that lengthens, a JSON/array column that gains elements, a null filled with a long string — so updates keep outgrowing their pages. (b) Each migration adds one forwarding pointer, turning one row's reads from one fetch to two; the aggregate is a slow one-to-two I/O drift that appears with no schema or index change and is easy to misattribute. (c) Compaction/reorganization (rewrite rows into fresh dense pages, collapsing redirects) and fill-factor headroom (leave free space per page so a grown update stays in place) — the first removes existing forwards, the second prevents new ones. *Tempting wrong answer:* “add an index” — an index does not remove forwarding pointers; the RID it stores still lands on the tombstone. **Feed-forward:** §49.3.

49.7 (a) A full scan reads all P pages, cost $O(P)$ page reads, and they are sequential — front-to-back over the file's page space — which Chapter 46 measured as far cheaper than random because the device streams and the system reads ahead. (b) A point look-up by RID is one page fetch if the row has not migrated, two if it has (tombstone then real page). (c) Average cost ≈ $(1-m) \times 1 + m \times 2 = 1 + m$ fetches; compaction lowers m (fewer forwards, cheaper look-ups) and raises tuple density (more live rows per sequential page, cheaper scans). *Tempting wrong answer:* “the scan is random because rows are unordered” — the pages are read in file order, which is sequential regardless of record order within them. **Feed-forward:** §49.4.

49.8 **Step 4:** The scan is slow because it reads every page whether dense or not, and a year of updates has left pages half-empty and dotted with forwarding pointers, so the scan streams many pages that carry few live rows — the device is fast but it is moving mostly dead space and overhead. The fix without an index is to **reorganize/compact** the table (e.g. `VACUUM FULL` / a table rewrite), which packs live rows densely into fresh pages and collapses the forwards, so the same sequential bandwidth now returns far more live rows per page and the scan touches fewer pages. *Tempting wrong answer:* “buy faster storage” — the device is already streaming; the waste is low density, which compaction, not bandwidth, fixes. **Feed-forward:** §49.4.

49.9 (a) **RID** — the index stores the (page, slot) address, so following it is one direct fetch. (b) **Page directory** — the free-space search is an in-memory array scan, no data-page fetches. (c) **Forwarding pointer** — the tombstone at the old RID keeps the address valid after the row moves. (d) **Slot array** — indirection lets a deleted record leave a hole reclaimed at compaction. (e) **Sequential scan** — reading all pages in file order is cheap sequential I/O. *Tempting wrong answer:* attributing (a) to the page directory — the directory is for free space, the RID is the row address. **Feed-forward:** §49.1–§49.4.

49.10 (a) **False** — a heap file returns rows in physical order, which need not be insertion order (deletes and reuse scramble it). (b) **True** — the directory search is in memory, so locating free space costs zero data-page fetches. (c) **False** — a shrink fits in place; only a grow past the page needs relocation. (d) **False** — a forwarding pointer makes reads *more* expensive (two fetches), not cheaper; it only preserves the RID. (e) **False** — a full table scan reads pages in file order, which is sequential I/O. *Tempting wrong answer:* marking (a) true because inserts append — reuse of freed slots breaks insertion order. **Feed-forward:** §49.1–§49.4.

49.11 (a) Insert: scan the free-space *directory* in memory to pick a page with room (§49.2); fetch that page through the buffer pool (§48); write the record into free space and add a slot pointing at it (§47); update the page and directory free-space counts; return the RID (page, slot). (b) Both a free-space directory and an index are a small, searchable structure kept beside the data so you need not scan the data to answer a question: the directory answers "which page has room for a record of size *b*?" and an index answers "which page (RID) holds key *k*?" (c) A single always-sorted file would make inserts expensive (shifting to maintain order) while a heap keeps inserts cheap (§49.2) and cheap sequential scans (§49.4), and lets indexes supply order/selectivity separately — so the heap-plus-index split gives cheap writes, cheap full reads, and cheap selective reads at once. *Tempting wrong answer:* "a sorted file is strictly better because lookups are fast" — it pays for that with costly ordered inserts the heap avoids. **Feed-forward:** §49.1–§49.5.

49.12 Open-ended and platform-specific. A correct write-up: (a) reports data-page-fetch counts to locate room for a linked-list walk versus a page-directory scan over the same seeded reference string, their ratio, and how the linked-list count rises as the file grows (the directory stays flat); (b) reports the two-fetch read rate after injecting growing updates that leave forwarding pointers; (c) states toolchain and CPU and distinguishes these *exact counts* from a deterministic simulation (reproducible and hardware-independent — the claim that travels) from any wall-clock timing (machine- and disk-dependent). *Tempting wrong answer:* presenting the fetch counts as timings — they are structural counts, not clock readings, which is exactly why they are portable. **Feed-forward:** §49.2–§49.4.

Chapter 50 — Solutions

50.1 The storage manager owns physical storage and exposes it as a narrow interface of pages and records, hiding the buffer pool, slotted pages, free-space directory, and device. Two of its calls: `fetchPage`/`unpinPage` (read a page), `allocatePage`/`freePage` (grow/shrink), `readRecord`/`insertRecord` (records). *Tempting wrong answer:* "it manages the disk hardware" — it manages the abstraction of pages and records *over* storage, hiding the hardware. **Feed-forward:** §50.1.

50.2 A PageId is (file identifier, page number within that file). The RID is properly (PageId, slot) because a real database has many files — tables, indexes, log — so a bare page number is ambiguous; the file id disambiguates, and page number × page size gives the offset. *Tempting wrong answer:* "a global page number is enough" — it ties every page to one file and one growth path. **Feed-forward:** §50.2.

50.3 About 258 overflow pages (⌈1 MiB / ~4072 B⌉). In place of the value the row keeps a small pointer to the head of the overflow chain. *Tempting wrong answer:* "256" from 1 MiB / 4 KiB exactly — per-page overhead reduces the usable bytes, so it is 258, not 256. **Feed-forward:** §50.3.

50.4 (a) The buffer pool (page table, pin counts, dirty bits, eviction), the device beneath it, and whether the access was a hit or a disk read. (b) The B-tree calls only `fetchPage`/`unpinPage` and treats a node as a page, so it never issues a `pread` or picks a victim — it can be written purely in terms of pages. (c) Example: the buffer pool switching from clock to LRU-K eviction, or the heap switching from a linked list to a page directory — the layers above call the same interface and are unaffected, because the interface names pages/records, not mechanisms. *Tempting wrong answer:* "the B-tree still needs to know the eviction policy for correctness" — it does not; pinning is the only guarantee it needs, and the interface provides it. **Feed-forward:** §50.1.

50.5 (a) $\lceil S / (P-h) \rceil$ overflow pages. (b) With $P-h$ = 4072: 64 KiB → ⌈65536/4072⌉ = 17; 1 MiB → ⌈1048576/4072⌉ = 258. (c) Benefits: the row stays small (many rows per heap page, dense scans) and the value is pay-per-use (fetched only when its column is read). Cost: a linked overflow chain must be walked to reach a deep offset, so random access into the middle of a large value is *O(chain)*. *Tempting wrong answer:* using P not $P-h$ — the per-page overhead reduces usable bytes. **Feed-forward:** §50.3.

50.6 (a) $\lfloor U/R \rfloor$ rows fit a page; inlining a 1 MiB value makes R > a page, so the count drops to one row per page (or fewer, with its own overflow). (b) A full scan reads whole pages; if each page holds one bloated row, the scan reads a megabyte per row regardless of which columns the query wants — it cannot skip the inline bytes. (c) Store values beyond a threshold (roughly a quarter of a page) out-of-line; this keeps rows small so density — and the cheap scan — is preserved. *Tempting wrong answer:* "the scan reads only the requested columns" — I/O is by page, so inlined bytes are read whether wanted or not. **Feed-forward:** §50.3.

50.7 (a) Small rows: about $\lceil N/d \rceil$ pages; inlined 1 MiB value: about N pages (one row each), each ~256× larger. (b) The inlined scan moves on the order of d × (1 MiB / small-row-size) times more bytes — many hundreds of times — all at the sequential per-page cost of Chapters 46/48, so the scan is I/O-bound on data it never uses. (c) Out-of-line storage puts only a pointer in the row, so the scanned page size depends on the inline columns; the large value's bytes are elsewhere and are read only if that column is selected. *Tempting wrong answer:* "sequential reads are cheap, so size doesn't matter" — cheap per byte, but hundreds of times more bytes is still hundreds of times more time. **Feed-forward:** §50.3–§50.4.

50.8 **Step 4:** One page fetch. `readRecord` does a single `fetchPage` for the heap page, indexes the slot, and reads the inline `id` and `title`; `body` is only a pointer in the row, so its size is irrelevant — the overflow chain is never touched. If the query *did* select `body`, the storage manager would follow the pointer and fetch the overflow chain — about ⌈|body| / usable-bytes⌉ further page fetches — so the cost would then scale with the value's size. *Tempting wrong answer:* "the big value is always read with the row" — out-of-line means it is fetched only when its column is selected. **Feed-forward:** §50.3–§50.4.

50.9 (a) **The interface (encapsulation)** — the B-tree uses `fetchPage`, never raw I/O. (b) **Out-of-line storage** — the row holds a pointer to the document's overflow chain. (c) **PageId** — a small comparable (file, page) name across files. (d) **RID** — the (PageId, slot) row address stored in the index leaf. (e) **fetchPage** — it returns a pinned frame pointer whether the page was resident or read from disk. *Tempting wrong answer:* attributing (a) to `fetchPage` alone — it is the interface abstraction, of which `fetchPage` is one call, that provides the independence. **Feed-forward:** §50.1–§50.3.

50.10 (a) **False** — it exposes pages and records; disk offsets and frame indices are exactly what it hides. (b) **True** — a page keeps its PageId for its lifetime, which is why stored PageIds stay valid. (c) **False** — compression may shrink it but a 10 MiB value generally still exceeds a page and needs out-of-line storage. (d) **False** — inlining a large value *slows* scans that never read it, by reading its bytes on every page. (e) **True** — the value lives elsewhere and is fetched only when its column is read. *Tempting wrong answer:* marking (c) true because "compression fixes size" — it reduces but does not remove the need for overflow. **Feed-forward:** §50.1–§50.3.

50.11 (a) RID = (file, page, slot) → `fetchPage(PageId)` → buffer-pool page-table lookup: hit = frame in DRAM (~36 ns), miss = evict a victim (write back if dirty), compute offset, read the 4 KiB page from the device (~350 µs, Chapters 46/48) → pin, index the slot array (Chapters 47/49) → read bytes, following a forwarding pointer (§49.3) or overflow chain (§50.3) if present → unpin, return. (b) A B-tree node *is* a page reached by `fetchPage` and a leaf stores a *RID*; both are exactly what this part defined, so the B-tree is buildable purely on the interface, with no I/O code of its own. (c) Both put a small, stable name in front of variable, growing data: a PageId names a page across growing files; an out-of-line pointer names a value that outgrew its page — the same instinct at two scales. *Tempting wrong answer:* "the B-tree needs its own disk layer" — the storage manager already is that layer. **Feed-forward:** §50.1–§50.5.

50.12 Open-ended and platform-specific. A correct write-up: (a) sketches the six-call interface and implements `readRecord` as one `fetchPage` plus a slot index (no raw I/O above the interface); (b) computes overflow-page counts as an exact ceiling division $\lceil S/(P-h) \rceil$ for several sizes and confirms the pattern; (c) if it measures the density effect, states toolchain and CPU and separates the *exact arithmetic* (page counts, reproducible and hardware-independent) from any *timed* scan (machine- and disk-dependent). *Tempting wrong answer:* reporting the overflow-page arithmetic as a measurement — it is exact arithmetic, not a clock reading. **Feed-forward:** §50.1–§50.4.

Chapter 51 — Solutions

51.1 The page fetch. A wide, shallow tree is favored because each page fetch is enormously expensive (the disk cliff), so the structure that minimizes fetches wins; a binary tree makes the fewest *comparisons* but one page fetch per level, ~30 levels over a billion keys, which is 30 fetches. *Tempting wrong answer:* “minimize comparisons” — that is the RAM-model cost; on disk comparisons are free and fetches are the cost. **Feed-forward:** §51.1.

51.2 Internal nodes hold sorted keys and child pointers and only route (no data); leaves hold the index entries — (key, RID) pairs pointing at heap rows — and are linked in key order. The linking lets a range scan, once it reaches the first matching leaf, walk the leaf chain in sorted order without returning to the root. *Tempting wrong answer:* “internal nodes store the data too” — that is the plain B-tree; the B+-tree keeps all data in the leaves, which is what makes range scans efficient. **Feed-forward:** §51.2.

51.3 Fanout ≈ (4096−24)/(8+8) ≈ 254; a billion keys need 4 levels, so a look-up is 4 page fetches. *Tempting wrong answer:* “a few dozen fetches” — that is the binary-tree figure; the B-tree's high fanout makes it single digits. **Feed-forward:** §51.3.

51.4 (a) fanout ≈ (page size − header) / (key size + pointer size). (b) (4096−24)/(16+8) ≈ 169. (c) With fanout 169, a billion keys need height ≈ ceil(log-base-169 of (1e9/169)) which is about 5 levels; doubling the key size only lowers the fanout (here 254→169), and since height is a *logarithm* of N in that base, a modest base change adds at most a level — it does not double the height. *Tempting wrong answer:* “bigger keys double the height” — height depends on log-base-fanout, so it changes slowly. **Feed-forward:** §51.3.

51.5 (a) The full leaf's entries are divided into two half-full leaves; a new leaf node is created and linked into the leaf chain; the separator key (the first key of the new right leaf) is pushed up into the parent with a pointer to the new leaf. (b) If the push fills the parent, the parent splits by the same rule, and so on up; if it reaches and splits the root, a new root is created above the old one. (c) Because growth happens only by adding a new root on top, every existing leaf keeps its depth and a whole new level is added uniformly — so all leaves stay at the same depth. *Tempting wrong answer:* “the tree grows by making a leaf deeper” — leaves never deepen individually; the tree grows at the root. **Feed-forward:** §51.4.

51.6 (a) About 30 levels ($\log_2$ of a billion), so ~30 page fetches if the tree is not resident. (b) 4 fetches (height 4). (c) The ~30 comparisons on the binary descent cost tens of nanoseconds total and are irrelevant; the on-disk cost is entirely the ~30 page fetches versus 4, each fetch being potentially a ~350 µs disk read. *Tempting wrong answer:* “both do ~30 comparisons so they're similar” — comparisons are free; the fetch count differs 30 vs 4. **Feed-forward:** §51.1–§51.3.

51.7 (a) The root is 1 page; the second level is at most ~254 pages (one per root child) — together a few hundred pages, a fraction of a megabyte, trivially held resident in any real buffer pool. (b) Given that, typically only the leaf fetch (the 4th level) reaches the disk. (c) Effective cost ≈ 3 hits + 1 miss ≈ 3 × 36 ns + 350 µs ≈ ~350 µs — dominated by the single leaf miss, essentially one disk read. *Tempting wrong answer:* “4 disk reads” — the upper levels are cached, so it is ~1. **Feed-forward:** §51.3.

51.8 **Step 4:** Because increasing keys always split the rightmost leaf 50/50 and the left half is never revisited, pages settle near 50% full, so the index needs roughly twice as many pages as a densely packed one — about twice the expected size. Lower fill also means lower effective fanout, which can add a level to the height and thus a page fetch to some look-ups, and it wastes buffer-pool space on half-empty pages. The fix that keeps sequential loads dense is **append-aware (unbalanced) splitting** — keep the full left page and start a fresh right page — or a bottom-up sorted **bulk load**, both of which reach near-100% fill. *Tempting wrong answer:* “switch to random keys” — that spreads inserts but thrashes the cache and destroys locality; the fix is the split policy, not the key. **Feed-forward:** §51.4.

51.9 (a) **Leaf linking** — a range scan walks the linked leaves in order. (b) **The balance invariant** — all leaves at the same depth, so every look-up is the same height. (c) **Fanout** — hundreds of (key, pointer) pairs per page. (d) **Node split** — splitting and growing at the root keeps the tree balanced as it grows. (e) **The RID in the leaf** — the leaf entry points at the heap row. *Tempting wrong answer:* attributing (b) to fanout — fanout sets width, the balance invariant sets uniform depth. **Feed-forward:** §51.2–§51.4.

51.10 (a) **False** — a B+-tree stores data only in the leaves; internal nodes hold routing keys and pointers. (b) **False** — a leaf split pushes a key up but does not itself add a level; height grows only when the root splits. (c) **False** — height is log-base-fanout of N, so doubling N adds far less than a level. (d) **True** — naive 50/50 splits on sorted inserts leave left pages half-full forever. (e) **False** — on disk the binary tree is a poor choice; the cost is page fetches, not comparisons. *Tempting wrong answer:* marking (b) true because a split “made the tree bigger” — only a root split adds height. **Feed-forward:** §51.1–§51.4.

51.11 (a) Descend root → internal → … → leaf, each node a `fetchPage` (§50) through the buffer pool (§48): for a height-4 tree that is 4 fetches, of which the upper 3 usually hit (cached) and the leaf usually misses; read the RID at the leaf, then one more `fetchPage` for the heap row (§49) — so ~5 fetches, ~1–2 disk misses. (b) A B-tree node *is* a page from `fetchPage` and a leaf stores a RID, both defined by the storage manager, so the B-tree needs no I/O code of its own — that interface is exactly what let it be written as “a tree of pages.” (c) A B-tree updates in place, so each small write is a random page update paying Chapter 46's random-write cost; a write-heavy workload of many such updates is expensive, which is precisely what motivates a log-structured engine that turns those random writes into sequential ones. *Tempting wrong answer:* “the look-up is one fetch” — it is the height plus the heap fetch; caching reduces the *misses*, not the fetch count. **Feed-forward:** §51.1–§51.5.

51.12 Open-ended and platform-specific. A correct write-up: (a) computes fanout ≈ (page−header)/(key+pointer) and tabulates height across N, showing single-digit heights even at a trillion keys; (b) implements a working B-tree of some order *m*, inserts in a seeded order, reports measured height and split count, and checks height against the bracket ceil(log-base-*m* of N) ≤ height ≤ ceil(log-base-⌈*m*/2⌉ of N); (c) states toolchain and CPU and notes the fanout table is *exact arithmetic* and the tree height is a *structural outcome* of a deterministic insert order — neither is a wall-clock timing, so both are reproducible and hardware-independent. *Tempting wrong answer:* reporting the tree height as a performance measurement — it is a structural property, not a clock reading. **Feed-forward:** §51.3–§51.4.